University of Warsaw
Faculty of Mathematics, Informatics and Mechanics

Piotr Styczyński
Student no. 386038

An integrated and practical database solution for storage and efficient processing of large genomic datasets

Master's thesis
in COMPUTER SCIENCE

Warsaw, 2026-07-28

Abstract

Genomic workflows commonly connect specialized programs through files. This arrangement separates sequences from relational metadata, repeats parsing and indexing work, and prevents the database planner from representing the cost of alignment. This thesis presents BioSeqDB, a PostgreSQL extension that stores compact biological sequences and runs a minimap2-derived seeding, chaining, and extension pipeline during database scans.

BioSeqDB provides alphabet-generic packed sequence views, scalar and single instruction, multiple data (SIMD) minimizer extraction, and Vulkan implementations for selected sketching and chaining operations. Bounded CPU dynamic-programming kernels and GPU alignment routes perform final extension. Optional SneakySnake and GateKeeper filters reject unsuitable candidate regions before that stage. The database layer adds persistent minimizer indexes, bounded ingestion, external-provider relation definitions, coordinated caching, and generated PostgreSQL boundary declarations.

Alignment is not evaluated as an independent scalar function for every row. Planner support introduces custom paths whose execution nodes batch sequences, execute the explicitly selected implementation when the input and hardware pass admission, and place alignment results into planned expressions. The same operation can therefore participate in joins, filtering, metadata projection, and persistent storage. The thesis describes how the mapper's data and control flow become a database execution primitive while retaining explicit resource bounds and support for heterogeneous processors.

Keywords

sequence alignment, bioinformatics, minimap2, PostgreSQL, databases, genomic data management, SIMD, GPU computing

Thesis domain (Socrates-Erasmus subject area codes)

11.3 Informatics and Computer Science

Subject classification

Applied computing, life and medical sciences, bioinformatics

Tytuł pracy w języku polskim

Zintegrowane i praktyczne rozwiązanie bazodanowe do przechowywania i wydajnego przetwarzania dużych zbiorów danych genomicznych

Contents

Introduction . . . 9
Timeline . . . 12

1. Biological foundations of sequence analysis . . . 13
1.1. Biological context . . . 13
1.1.1. DNA, ribonucleic acid (RNA), and protein alphabets . . . 14
1.2. Alignment operation . . . 14
1.3. Technical considerations . . . 15
1.3.1. FASTA . . . 15
1.3.2. FASTQ . . . 16
1.3.3. CIGAR . . . 17

2. Sequence alignment and search strategies . . . 19
2.1. Alignment foundation and specification of a problem . . . 19
2.2. Dynamic programming definition . . . 20
2.3. Seed and extend . . . 22
2.4. Limitations of existing tools . . . 24

3. The BioSeqDB alignment pipeline . . . 27
3.1. Alignment as a database execution pipeline . . . 27

4. Minimizer extraction . . . 29
4.1. From every base to a sparse, comparable sketch . . . 29
4.1.1. Canonical k-mers and minimizer windows . . . 29
4.2. Methods of minimizer generation . . . 32
4.2.1. Scalar rescan . . . 32
4.3. A1: Naive SIMD rescan . . . 33
4.4. A2: Two-stack algorithm . . . 35
4.4.1. One window minimum from two saved values . . . 35
4.5. A3: Chronological SIMD . . . 36
4.6. Bioseq prototype . . . 39
4.7. Seeding prototype V1–V6: Naive SIMD rescan . . . 40
4.8. Seeding prototype V8: Implementing A2 algorithm . . . 41
4.8.1. Seeding prototype V8.5: Deferring hash calculation . . . 42
4.9. Seeding prototype V9–V11: Stream interleaving and forced inline . . . 42
4.9.1. Seeding prototype V12: Forced inline . . . 43
4.9.2. Seeding prototype V13: Existmator fix . . . 44
4.9.3. Seeding prototype V14: directly use nthash . . . 45
4.9.4. Seeding prototype V15: Smaller minimizer space . . . 46

4.9.5. Seeding prototypes summary . . . 46
4.10. GPU acceleration for seeding . . . 47
4.10.1. Sketch pseudocode and work . . . 48
4.11. BioseqDB GPU seeding . . . 49
4.12. Packed minimizer representation . . . 52
4.13. Configured CPU algorithms . . . 54
4.13.1. Vector width and indexed queries . . . 55

5. Chaining seed anchors . . . 57
5.1. From local seed hits to candidate mappings . . . 57
5.2. Anchor geometry and ordering . . . 57
5.3. Intuitive interpretation . . . 59
5.4. Chaining components . . . 59
5.5. Chaining algorithms . . . 61
5.5.1. Shared pair score . . . 61
5.6. B1: minimap2 dynamic programming . . . 65
5.6.1. Skipping with counters and marked anchors . . . 66
5.7. BioseqDB SIMD DP: exact and bounded batch . . . 69
5.7.1. Candidates in SIMD vectors . . . 69
5.7.2. Summary of forward chaining algorithms for CPU . . . 73
5.8. Reverse chaining (backtracking) . . . 73
5.8.1. Formal definition for backtracking . . . 74
5.8.2. Minimap2 backtracking . . . 74
5.8.3. BioseqDB SIMD backtracking . . . 78
5.9. Statistical modelling of CPU forward and backward passes . . . 82
5.9.1. Problematic chains . . . 82
5.9.2. Score distribution for random cases . . . 90
5.9.3. Does the bounded SIMD U limit help? . . . 91
5.9.4. BioseqDB `max_drop` analysis . . . 92
5.9.5. Conlusions . . . 94
5.10. BioseqDB SIMD performance comparison . . . 95
5.11. Range minimum query chaining . . . 95
5.11.1. Minimap2 RMQ . . . 96
5.11.2. RMQ re-chaining in the pipeline . . . 99
5.11.3. RMQ in BioseqDB . . . 100
5.12. G3SA GPU chaining . . . 101
5.13. BioseqDB implementation . . . 101
5.13.1. How BioseqDB RMQ differs from minimap2 . . . 102
5.13.2. BioseqDB Vulkan work split . . . 102

6. Final alignment; prefiltering and postprocessing . . . 105
6.1. From a seed chain to a base-level alignment . . . 105
6.1.1. The two CPU alignment contracts . . . 105
6.2. Why filter after chaining but before alignment? . . . 106
6.3. Postprocessing . . . 106
6.3.1. Seed divergence and position order . . . 106

7. Sequence representations and lazy evaluation . . . 107
7.1. Designing sequence abstraction . . . 107
7.2. Packed symbols and a common C++ interface . . . 111
7.3. Slices keep logical coordinates . . . 113
7.4. Composing sequences without joining their payloads . . . 115
7.5. Reading only what the algorithm requires . . . 116
7.5.1. Copying ranks from a PostgreSQL sequence . . . 118

8. PostgreSQL API and custom scans . . . 121
8.1. Designing SQL API for an aligner . . . 121
8.2. Integrating Postgres . . . 124
8.2.1. Designing SQL API . . . 124
8.2.2. Plan lifecycle . . . 125
8.2.3. Different types of scans . . . 127
8.3. Dispatch buffer as glue between Postgres scan and alignment pipeline . . . 128
8.3.1. Returning results back to Postgres . . . 128
8.3.2. Supporting joins . . . 130
8.3.3. Dispatch buffer internals . . . 130
8.3.4. MVCC support . . . 134
8.3.5. Filter placement . . . 134
8.4. Joins support . . . 135
8.5. Caching results . . . 138
8.5.1. Reusing fragments of queries . . . 139

9. Physical sequence storage in PostgreSQL . . . 141
9.1. Store sequences together with their metadata . . . 141
9.2. Columnar storage . . . 142
9.2.1. Columnar layout . . . 143
9.2.2. Smart reads . . . 146
9.2.3. Compression . . . 147
9.3. Raw byte storage . . . 148
9.4. Locator implementation . . . 150
9.4.1. Two levels of sharding . . . 152
9.5. MVCC and deletion for sequence fragments . . . 154
9.5.1. BioseqDB all cache layers . . . 156

10. Persistent sequence indexes . . . 159
10.1. Keep the lookup work between queries . . . 159
10.2. What existing sequence indexes store . . . 159
10.2.1. Indexing: minimap2 . . . 159
10.2.2. Indexing: mmseqs2 . . . 160
10.3. BioseqDB indexing abstractions . . . 160
10.4. BioseqDB supported indexing . . . 165
10.4.1. Index versioning . . . 167
10.4.2. Symbol count index . . . 170
10.5. Building MPS . . . 170
10.6. Selective reads . . . 172
10.6.1. The reader also determines query execution . . . 172
10.6.2. Repeated control scans in MPF . . . 173

10.6.3. Why grouped MPF has a reader file . . . 173
10.7. When indexes are built and when queries use them . . . 178
10.7.1. Maintenance policies . . . 178
10.7.2. Reading policies . . . 180
10.7.3. Construction, deferred work and other settings . . . 181
10.7.4. Reusing bytes already in memory . . . 182
10.8. What the database integration adds . . . 183

11. BioSeqDB architecture . . . 185
11.1. Connecting sequence processing to the database . . . 185
11.1.1. Reuse at the alphabet boundary . . . 186
11.2. Contracts describe the operations a stage needs . . . 186
11.3. Ownership completes the interface . . . 187
11.4. Runtime choices and compiled implementations . . . 188
11.4.1. Turn SQL settings into a request . . . 189
11.4.2. Separate algorithm selection from hardware selection . . . 189
11.4.3. Require compatible indexes and execution routes . . . 190
11.5. Schedule independent work within a backend . . . 191
11.5.1. PostgreSQL processes and oneTBB threads . . . 191
11.5.2. Choose the unit of parallel work . . . 192
11.5.3. Admit memory before increasing overlap . . . 194
11.5.4. Completion, cancellation, and progress . . . 195
11.6. Load sequence files through a reusable parser . . . 196
11.6.1. Separate byte sources from record framing . . . 196
11.6.2. Deliver chunks while a record remains open . . . 196
11.6.3. Route header fields into SQL columns . . . 197
11.7. Remote sequences and local persistence . . . 198
11.8. Generation and validation . . . 200
11.8.1. Deriving the PostgreSQL boundary . . . 200
11.8.2. Controlling specialization cost . . . 201
11.8.3. Benchmark specifications and validation . . . 202
11.9. Authorship, development and AI assistance . . . 202
11.9.1. Where AI tools were used . . . 203
11.10. Validation limits of the complete system . . . 203

12. Benchmarking and performance evaluation . . . 205
12.1. Benchmarking and performance evaluation . . . 205
12.1.1. What the results show . . . 206
12.1.2. Measurement procedure and reading the full tables . . . 219
12.1.3. Recorded machines and completed coverage . . . 224
12.1.4. Comparison with other aligners . . . 266
12.1.5. Vulkan on a Vast.ai GPU instance . . . 272

13. Conclusions . . . 275

References . . . 277

1. A runnable SQL walkthrough . . . 285

2. Configuration reference 297
2.1. Physical storage 297
2.2. Planning and execution 299
2.3. Task scheduling 300
2.4. Index construction and reading 301
2.5. Minimizers and seed filtering 303
2.6. Chaining 304
2.7. Primary and secondary alignments 305
2.8. Base alignment 306
2.9. Vulkan execution 307
2.10. Diagnostics 308
2.11. Optional query-result cache 309
2.12. Controls present only in test builds 309
2.13. Per-search options 310

Introduction

Sequencing instruments turn biological material into collections of strings. The useful interpretation of those strings rarely exists in isolation. A read has an instrument and sample provenance. A reference interval belongs to an assembly and annotation release. An alignment has a score, a strand, a position, an edit path, and a relationship to experimental metadata. Conventional workflows keep the strings in FASTA and FASTQ sequence files, Sequence Alignment/Map (SAM) and Binary Alignment/Map (BAM) alignment files, or mapper-specific index files while keeping much of the surrounding information in tables or ad-hoc scripts. Each tool sees only the portion of the workflow represented by its command-line arguments.

Relational database systems solve a different part of the problem. PostgreSQL provides durable storage, transactions, typed values, indexing, joins, extensible operators, and a cost-based planner. It does not, by itself, provide the representations or execution kernels required for billions of packed symbols and computationally expensive sequence searches. A naive user-defined function is not sufficient. Evaluating one alignment function independently for every row prevents batching, hides accelerator opportunities, and gives the planner little information about a predicate that can dominate the query.

BioSeqDB investigates the boundary between these two systems. It is implemented as a PostgreSQL extension whose non-database core is written in C++26. Biological symbols are represented by generic compact ranges rather than by a deoxyribonucleic acid (DNA)-only byte string. The alignment path is derived from minimap2: representative k-mers are extracted as minimizers, matching minimizers become anchors, compatible anchors are chained, and CPU or GPU alignment routes extend the selected regions. The port is not a line-for-line translation. The implementation introduces template-selected alphabets and layouts, Highway SIMD kernels, task-based parallel execution, Vulkan compute paths, persistent PostgreSQL-backed indexes, and prefilters for candidate regions. These changes must preserve the semantics that make minimap2 useful while operating on data that may be lazy, fragmented, cached, or obtained from an external provider.

The database boundary is equally important. An SQL expression such as `mmsearch(a, b)` has a conventional function definition so that PostgreSQL can type and plan it. The ordinary function body is not an alignment fallback: it fails closed if no BioSeqDB custom executor owns the expression. Its planner-support function assigns the unowned expression a high startup cost, while structural planner hooks can construct an owning custom path with explicit nonnegative costs. The resulting custom scan batches input, invokes the alignment pipeline, and makes its results available to planned expressions through injected PostgreSQL nodes. This is what permits alignment to participate in filters, projections, and joins while retaining the execution structure required by central processing unit (CPU) vector units and graphics processing units (GPUs).

The central research question is therefore not whether an existing aligner can be called from SQL. It is whether the aligner's data and control flow can become a database execution

primitive without losing algorithmic fidelity, resource bounds, or the ability to exploit modern processors. The investigation is organized around four supporting questions:

RQ1. Which parts of seeding, chaining, and extension can be expressed over a common compact sequence abstraction without eager conversion?

RQ2. Which stages admit SIMD or GPU implementations, and which input and resource constraints govern their selection?

RQ3. How can PostgreSQL plan and execute a set-valued, batch-oriented search while presenting an ordinary typed SQL interface?

RQ4. How should sequences, indexes, ingestion, external data, caches, and generated boundary code fit around the kernels so that the resulting system remains inspectable and reliable?

This thesis answers these questions through implementation and analysis of algorithmic and execution contracts. Historical designs appear only when they explain a constraint or a rejected optimization. They are not presented as simultaneous production architectures. Comparisons with minimap2, SIMD-Minimizers, the G3SA GPU seed-chain-extend system, Block Aligner, SneakySnake, GateKeeper, and Citus Columnar distinguish inherited mechanisms from BioSeqDB-specific changes.

The main contributions developed in the following chapters are:

C1. An alphabet-generic sequence representation that supports packed, fragmented, lazily composed, and PostgreSQL-backed data through a common range contract.

C2. A staged minimap2-derived mapper with scalar, SIMD, and Vulkan components and explicit runtime dispatch.

C3. Optional GPU-capable SneakySnake and GateKeeper candidate filters integrated without changing the meaning of an accepted alignment.

C4. Persistent sequence and minimizer-index storage derived from a columnar PostgreSQL access method, with bounded loading, caching, and selective cold-index reads.

C5. A planner and custom-scan interface that connects SQL expressions and joins to batch-oriented alignment.

C6. Structural generation and validation of SQL installation definitions and PostgreSQL function-manager boundary declarations from annotated C++ sources.

The document first establishes the biological and algorithmic context, then states the shortcomings that motivate an integrated system. The algorithm chapters follow data through minimizer extraction and chaining. They then describe the expensive downstream alignment stage before stepping back to the prefilter that guards that stage. This expository inversion makes the filter's safety and cost contract concrete without changing the runtime order, which remains chaining, prefiltering, and alignment. It then moves outward through sequence abstractions, PostgreSQL planning, physical storage, persistent indexes, ingestion, external providers, and generated SQL boundary code.

- Chapter 1 introduces biological sequences and sequencing data. It explains variation and the computational questions that motivate database-resident sequence analysis.

- Chapter 2 develops exact dynamic programming and seed-and-extend mapping. It also presents graph-based approaches and the limits of existing tools.
- Chapter 3 presents the complete processing path with its stage contracts and CPU and accelerator routes.
- Chapter 4 explains minimizer sketching and rolling hashing. It compares the SIMD and Vulkan implementations with minimap2 and SIMD-Minimizers.
- Chapter 5 derives the chaining objective and compares its scalar RMQ SIMD and GPU forms. It then states the selection policy.
- Chapter 6 describes affine-gap alignment with KSW2 and block-based SIMD extension. It also covers traceback and accelerator integration.
- Chapter 7 develops compact sequence ranges and bit-level views. It then presents fragmented and lazy sequences with PostgreSQL-backed adapters and alphabet dispatch.
- Chapter 8 follows an mmsearch expression through planning and batched execution. It explains joins and per-row result substitution.
- Chapter 9 explains the columnar-derived storage layout and fragment metadata. It also covers caching and request coordination.
- Chapter 10 describes index construction and storage with compatibility checks and selective cold loading. It also explains updates and differences from minimap2 indexes.
- Chapter 11 connects algorithm selection and oneTBB scheduling to a reusable processing core. It also covers sequence ingestion; remote access; code generation; validation and authorship.
- Chapter 12 measuring overall performance of BioseqDB
- Chapter 13 answers the research questions and summarizes the final system. It identifies the principal limitations and possible extensions.

Timeline

This thesis required several years of exploration and experiments. I began without a formal background in computational biology, so learning the field was part of the research. I proposed the database idea and prepared the initial design for our internal team. I directed the 2022 prototype, whose implementation was written by colleagues.[1] The MMseqs2 discussions did not lead to a collaboration. The Python prototypes that led to the implementation studied here began in December 2023.



Selected milestones from my project history. Spacing is not proportional to elapsed time. Linked entries point to the related discussion and prototype repositories.

[1] The current BioSeqDB is a separate project, designed and written by me. It contains no code or intellectual property from the early BWA-MEM integration experiments.

[2] I was a co-founder (shareholder) until 15 July 2022 (KRS update on 3 Jun 2022) in Covid Genomics Sp. z o.o. (registered in Poland in Jun 2020, KRS 0000846289) which then changed its legal name to CGen Sp. z o.o. (on 3 Jun 2022) and then changed its branding name to Deepflare (on Jul 2022). I was also a Chief Engineering Officer from 1 Apr 2020 till 31 Aug 2022.

Chapter 1

Biological foundations of sequence analysis

1.1. Biological context

"Significant scrutiny and attention have been given to genome-sequencing costs and how they are calculated since the beginning of the field of genomics in the late 1980s". Institute, The Cost of Sequencing a Human Genome NHGRI (National Human Genome Research Institute) has carefully tracked costs per genome and expose this data publically.

In the last 15 years, there has been seen a massive development of sequencing technologies. Companies like Applied Biosystems, Illumina, Roche, and Pacific Biosciences have developed a plethora of sequencing platforms, e.g. SOLiD, GS FLX, and PacBio. These tools are able to generate an enormous amount of data in a matter of days. As an example, Illumina NextSeq generates up to 360 Gbp, split into reads of length up to 150 bp in merely 2 days (data taken from Illumina website).

Although those platforms vary in detail, the general idea behind them is as follows:

1. DNA is fragmented through physical and chemical methods. Adapters are appended to the ends of the fragments to form what is called a sequencing library.
2. Fragments of unwanted size or self-ligated adapters are removed from the library.
3. The library is immobilized on a chemical bead so that it can be steadily observed.
4. Amplification is performed, e.g. through PCR emulsion
5. A combination of optical devices and fluorophores is used to make reads.

With new advancements in sequencing technologies, the cost of sequencing a genome has been decreasing significantly to the point where most popular open databases store more than 10s of petabytes of data. To make this data usesable researchers often need to perform search operations:

- For bioinvestigative resarch: does this genomic sequence already contains known genes? Does it contain a known mutation? Is it a known pathogen?
- For evolutionary biology: is this genomic sequence a known evolutionary lineage? What is the genetic distance between this sequence and a known sequence?
- For legal research: is this DNA sample from known individual? Does it represent known kinship to someone else?

This search operation is using an operation called "sequence alignment" which is the core concept of this thesis.

1.1.1. DNA, ribonucleic acid (RNA), and protein alphabets

DNA is a polymer containing one of four bases: adenine, cytosine, guanine and thymine. Their conventional one-letter symbols form the alphabet

$$\Sigma_{\text{DNA4}} = \{\texttt{A}, \texttt{C}, \texttt{G}, \texttt{T}\}.$$

Four symbols are sufficient for an unambiguous DNA string, but experimental data introduces ambiguity. The symbol `N` conventionally means that the nucleotide at a position is unknown or was not resolved. Other International Union of Pure and Applied Chemistry (IUPAC) codes describe sets of possible bases, such as a purine at a position.

RNA uses adenine, cytosine, guanine and uracil, so a basic RNA alphabet replaces `T` with `U`. Many RNA molecules are transcribed from DNA. Crick, "Central Dogma of Molecular Biology"

A protein sequence is written over the twenty common amino-acid symbols. It's alphabet is notably larger, but in most cases we can translate it to a DNA sequence and use the DNA alphabet directly.

Sequence class	Core symbols
DNA4	`A,C,G,T`
RNA4	`A,C,G,U`
DNA5	`A,C,G,T,N`
RNA5	`A,C,G,U,N`
PROT20	Twenty common amino-acid symbols

Table 1.1: Biological alphabets and the distinctions that a sequence-processing system must not erase. The table describes biological data in general. It does not assert that every alphabet or ambiguity convention is exposed by every BioSeqDB SQL path.

1.2. Alignment operation

Due to nature of biological sequences, search must be able to match fragments that are non-exact, thus alignment operation is similar to popular fuzzy-search operations on normal text. The alignment can be viewed as a processes that optimises gaps (insertions and deletions) in a way to optimise similarity metric. High similarity score between parts of sequences can indicate that they share same evolutionary origin or have a common purpose. By comparing newly found sequences to already researched ones, we can deduce lots of relevant biological data. Juliusz K and W, "Master thesis: An integrated and practical database solution for storage and efficient processing of large genomic datasets"

We will consider finite nonemptystrings over the DNA4 alphabet. A gap is a symbol indicated by `-`. An alignment is a process of adding gaps to the sequences so that they are of the same length and at the same time, there is no index where a gap occurs in both sequences.

As an example, let us present an alignment of $GATCAGATA$ and $CATTCAATCA$. One of the possible alignments is:

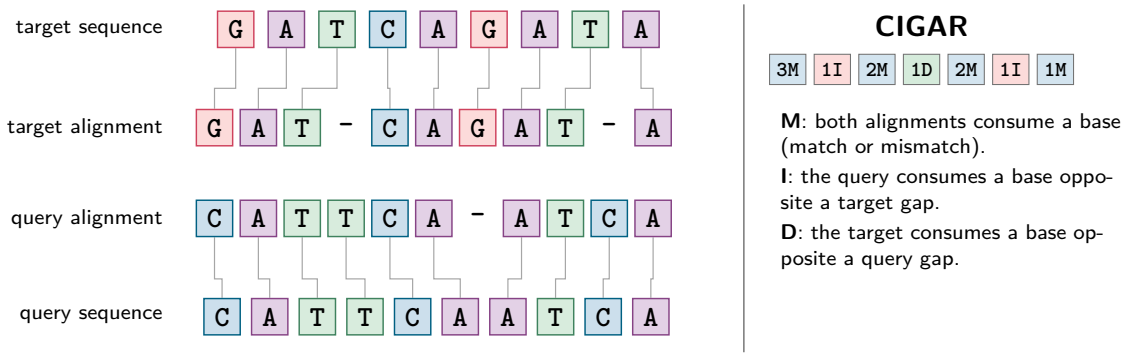


Figure 1.1: Alignment of the target sequence `GATCAGATA` and query sequence `CATTCAATCA`. Orthogonal connectors map each ungapped sequence position to its aligned column; gaps are intentionally unboxed.

There are 4 cases that can occur during alignments:

1. Insertion - a gap has been added to the first sequence
2. Deletion - a gap has been added to the second sequence
3. Mismatch - letters at a given index are different
4. Match - letters at a given index are the same

1.3. Technical considerations

A typical workflow for sequence analysis includes 3 data formats - FASTA, FASTQ, and SAM/BAM. Reference genome, stored in FASTA, is aligned with millions of reads in the FASTQ file and the result of alignment is saved as SAM/BAM file. The reason for the reference genome being in FASTA and reads in FASTQ is that we assume that the reference genome is correct, but reads produced by sequencers can contain various errors. Moreover, we're more likely to score alignments with high-quality bases higher than the ones with low-quality.

1.3.1. FASTA

FASTA is a text-based file format for storing nucleotide sequences. It is one of the most widely used formats in bioinformatics. The current standard specification is given by NCBI. Below is a description of the usually encountered version.

It consists of a series of entries separated by a new line. Each entry starts with a single-line header. The header starts with `>` and contains various metadata about the following sequence, like the description of the sequence, library accession number, and submission date. After the header comes the sequence itself represented with standard IUB/IUPAC acid codes, split into lines of length usually not exceeding 80 characters. Aside from A, C, T, G commonly encountered is code N, which means any of A/C/T/G. Lowercase letters are treated the same as uppercase. The usual extension is .fasta and .fa. An example FASTA file:

```
>PartOfHumanGenome|HG_9357/2022|2022-03-01
ACTTTGAGCTAGCATCGATTGGATCAGATGCAT
ACTCATCGGCAGTCGCATCAGCTTGGACTACGT
>PartOfMouseGenome|MG_5417/2019|2019-08-14
ACTATAGGAGCAGANNNNNNNNCTAGAGACATC
TCTGCGTNTNTTCCGGACGTACGTAGCATAGTC
```

The advantages of this format are simplicity and human readability. The downside is that it is not optimal in terms of size. To adhere to this issue, compression software is used, like gzip.

1.3.2. FASTQ

FASTQ is an extension of the FASTA format. It enhances FASTA by adding quality scores of sequences. It is a standard output format of sequencers.

Each entry consists of 4 lines. The first line starts with `@` and contains metadata about the sequence. The second line contains the sequence itself. The third line starts with `+` and may contain some additional metadata. The fourth line contains quality scores, represented by ASCII characters with codes from 33 to 126, one character per nucleotide. The file extension used is .fq and .fastq. Again, gzip can be used to reduce file size. An example FASTQ file:

```
@HumanGenomeSequencerRead
ACTACTAGACGG
+Genome20200301
*!]^##[$*@*!(
```

The quality score is usually a phread score, meaning Q satisfies $Q = -10 \log_{10} p$, where p is the probability of a corresponding nucleotide being incorrect. ASCII codes from 33 to 126 are mapped from 0 to 93.

1.3.3. CIGAR

CIGAR is an acronym for Concise Idiosyncratic Gapped Alignment Report and has been originally defined by the Exonerate alignment program. The CIGAR format is designed to contain the minimal information necessary for the reconstruction of an alignment. This format does not include nucleotides. Sweden, The Cost of Sequencing a Human Genome

- M - match or mismatch
- D - deletion
- I - insertion
- S - soft clipping. Can occur only at the beginning or at the end of the alignment. Indicates that nucleotides are clipped from the alignment.
- H - hard clipping. The same as soft clipping, except that the clipped part is not present in the SEQ field of the SAM file.
- N - skipped region. Used for representing introns (part of the gene that is non-coded) in the alignment.
- = - match
- X - mismatch

Chapter 2

Sequence alignment and search strategies

2.1. Alignment foundation and specification of a problem

Alignment can be done in different ways, where the core algorithms stays the same, but some of the conbditions change. It's important to understand what are the capabilities of standard dynamic programming algorithms and how they can be used to solve different problems.

Formally: Let $X = x_1 \dots x_n$ and $Y = y_1 \dots y_m$ be the sequences we align. An alignment is a sequence of columns

$$A = ((a_1, b_1), \dots, (a_L, b_L)), \qquad (a_r, b_r) \in (\Sigma \cup \{-\})^2 \setminus \{(-,-)\},$$

where first and second elements reciver contents of X and Y after gaps are removed:

1. An element of shape $a_r, -$ signifies an insertion in X
2. An element of shape $-, b_r$ signifies a deletion in Y
3. An element of shape m_r, m_r signifies a match
4. An element of shape a_r, b_r where $a_r \neq b_r$ signifies a substitution

This alignment is an optimized sequence correspondence under a scoring model. For substitution score $s(a, b)$ and gap cost $g(\ell)$, the score is

$$S(A) = \sum_{\text{symbol pairs } r} s(a_r, b_r) - \sum_{\text{gap runs } u} g(|u|).$$

The scoring model may be formulated in different ways, but one of the simplest and most-common conventions is to use an affine gap cost model:

$$g(\ell) = q + \ell e, \qquad \ell \geq 1, \tag{2.1}$$

where q is the opening cost and e is the extension cost.

A gap of size one costs then $q + e$. Some scientific papers mentioning $q + (\ell - 1)e$ use a different parameter convention. Affine scoring charges one opening cost per gap run and can be evaluated with a fixed number of states per matrix cell Gotoh, "An improved algorithm for matching biological sequences."

This scoring model isn't sufficient to define entire problem:

1. Global alignment consumes both complete sequences Needleman and Wunsch, "A general method applicable to the search for similarities in the amino acid sequence of two proteins"

2. Local alignment may restart at zero and selects the best pair of substrings Smith and Waterman, "Identification of common molecular subsequences"
3. Semi-global and overlap goals consume selected ends while allowing specified prefix/postfix to remain unaligned
4. Extension begins from a fixed anchor and chooses an endpoint under a stopping rule
5. Clipping will just finish on the rightmost gap and keep postfix unaligned
6. Local-extension fixes the start but permits an interior endpoint

Figure 2.1 shows schematic examples of these modes.

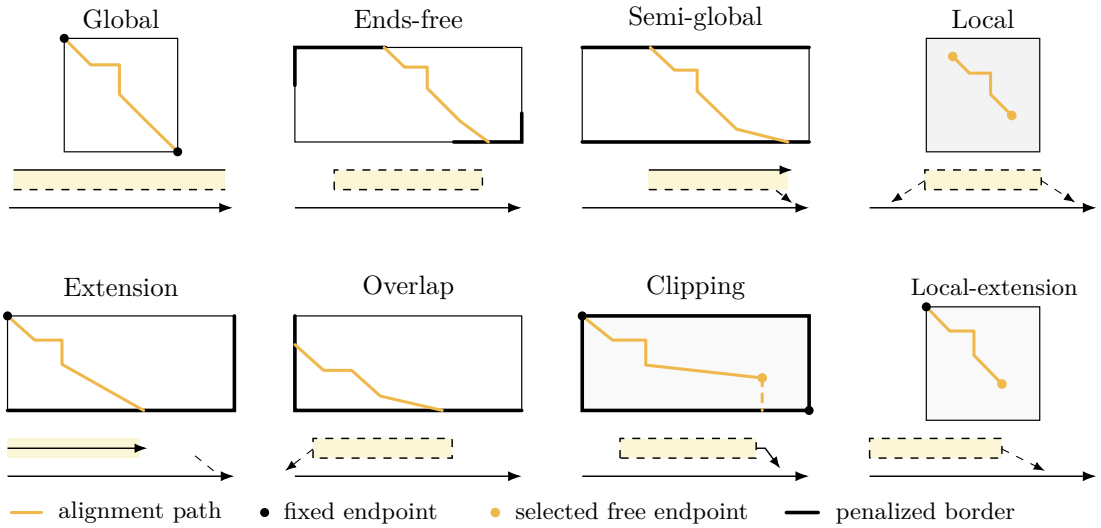


Figure 2.1: Eight alignment boundary modes. Yellow lines depcict possible tracebacks. Black dots mark fixed endpoints (for example for global alignment we always align entire sequence so endpoints are 0 and len(Y)-1), yellow dots mark selected free endpoints, and bold border segments mark penalized or anchored boundaries (usually the border is the length of sequence, but under certain conditions we might have limits on the alignment length and coordinates). In each lower schematic, the solid arrow is the reference and the yellow dashed track is the query. These panels showcase how different libraries/algorithms handle different goals across number of scientific resarch

Various practical implementations also use special stopping criteria, which usualy can be upper bound on the $S(A)$ score.

2.2. Dynamic programming definition

Let $H_{i,j}$ be the best score after consuming i symbols of X and j symbols of Y. For a simpler linear gap cost model d, we can define the algorithm's state matrix as:

$$H_{i,j} = \max\{H_{i-1,j-1} + s(x_i, y_j), H_{i-1,j} - d, H_{i,j-1} - d\}. \tag{2.2}$$

This is the intuitive definition based on 3 possible options (replacement or two variants of gap). Free-end alignment (like local) change the score values for the cells from which traceback may begin.

For affine scoring model we need to hold main state and one state for each gap orientation. Let $I_{i,j}$ and $D_{i,j}$ end in gaps in X and Y, respectively, and let $M_{i,j}$ end in a symbol pair. Under Equation 2.1, one formulation is

$$\begin{aligned} M_{i,j} &= H_{i-1,j-1} + s(x_i, y_j), \\ I_{i,j} &= \max\{H_{i,j-1} - q - e,\ I_{i,j-1} - e\}, \\ D_{i,j} &= \max\{H_{i-1,j} - q - e,\ D_{i-1,j} - e\}, \\ H_{i,j} &= \max\{M_{i,j}, I_{i,j}, D_{i,j}\}. \end{aligned} \tag{2.3}$$

Using H to open a gap permits a transition from any previous state. Local alignment also includes zero in the final maximum.

Algorithm 1 Affine-gap alignment with an explicit boundary contract

Require: sequences $X[1 \dots n]$, $Y[1 \dots m]$, scores s, q, e, and a boundary contract B
Ensure: score, endpoints, and an optional CIGAR string

```
initialize H, I, and D according to B
for i ← 1 to n do
    for j ← 1 to m do
        update M_{i,j}, I_{i,j}, D_{i,j}, and H_{i,j} with Equation 2.3
        store the predecessor of each winning state when traceback is requested
select the endpoint permitted by B
follow predecessors to an allowed start and run-length encode the operations
```

The algorithm state matrix can be used to retrieve the edit path as CIGAR string. This is the standard compact representation of gapped alignments described in chapter 1SAM/BAM Format Specification Working Group, Sequence Alignment/Map Format Specification

Affine model is some scenarios might not be enough as it doesn't distinguish between short and long gaps. The two-piece affine model can support such feature without complicating the scoring function too much:

$$g_2(\ell) = \min(q_1 + \ell e_1,\ q_2 + \ell e_2)$$

This model permits different cost slopes for short and long gaps. In our software we use single affine model for the fastest alignment modes (adaptive block), while KSW2 variants support dual affine model.

For large datasets evaluating entire large $O(nm)$ matrix can make significant practical difference:

- We can evaluate alignments only close to diagonal. Being close to diagonal means that there aren't many large gaps between matching fragments. Designing algorithms around diagonals can make us skip evaluation of many far cells that do not contribute to the result.
- In case of features that do not require traceback (CIGAR construction requires traceback, but score or coordinates of best match do not) we can just store relevant slice in memory and avoid allocating large memory region.
- We can introduce X-drop stop condition: stop when score falls below best score seen so far by more than X

- We can introduce Z-drop stop condition: the same as X-drop, but if the distance from diagonal is too high then stop as well (this means that we stop when there are too many gaps cummulatively)

Smart evaluation startegy can combine all these approaches. KSW2 and Block aligner for example use all of them. Block aligner Liu and Steinegger, "Block Aligner: an adaptive SIMD-accelerated aligner for sequences and position-specific scoring matrices" evaluates only the most promising rectangular region in the matrix while KSW2 Suzuki and Kasahara, "Introducing difference recurrence relations for faster semi-global alignment of long sequences" focuses on diagonal-centered evaluation.

2.3. Seed and extend

Applying local dynamic programming between a query of length m and a reference of length N costs $O(Nm)$. Most reference positions don't contribute to the result. Long-read mappers therefore organize search into three stages Li, "Minimap2: pairwise alignment for nucleotide sequences."

First step is to use divide and conquer approach and split sequence into representatives known as seeds:

$$\begin{aligned}\mathsf{Seed}(x_1,\ldots,x_n) = \{(s_1,i_1,j_1),\ldots,(s_k,i_k,j_k)\},\\ \text{where } 1 \le i_a, j_a \le n \quad \text{for all } a \in \{1,\ldots,k\}.\end{aligned} \tag{2.4}$$

Seed (s_a, i_a, j_a) is some represenative of the sequence substring $x_a = x_{i_a} x_{i_a+1} \ldots x_{j_a}$. This can be implemented in different ways, this definition is the most abtract one. If a seed occurs in both query and reference, then we can consulde that two represented substrings are very similar. Using some concrete seeding function we can obtain intersection for query and reference:

$$S_{X\cap Y} = \left\{ (s, i_x, j_x, i_y, j_y) \;\middle|\; \begin{matrix}(s,i_x,j_x) \in \mathsf{Seed}(X),\\ (s,i_y,j_y) \in \mathsf{Seed}(Y)\end{matrix} \right\}. \tag{2.5}$$

1. Seeding Generate $Seed(X)$ and $Seed(Y)$ and obtain $S_{X\cap Y}$
2. Chaining As matching seeds will cover only parts of both sequences that match without errors, we still need to fill-in the gaps between them. We connect seeds from $S_{X\cap Y}$ into chains using similar dynamic programming approach as in usual aligner. This gives us some approximate alignments.
3. Extension Perform usual alignment algorithm on the approximate fragments to obtain final results.

Minimizers are one of many possible seeding model definitions. Given parameters w and k, a window W of length $l = w + k - 1$ contains w consecutive k-mers. The minimizer of the window is the smallest k-mer (subsequence of length k) in the window. For random minimizers, k-mers are ordered by a pseudo-random order, usually given by comparing hashes of the k-mers. In case of ties, the leftmost smallest k-mer is chosen. Roberts et al., "Reducing storage requirements for biological sequence comparison" Overlapping windows often retain the same representative, which is emitted once.

Canonical minimizers: DNA is double-stranded and majority of sequencing technologies do not distinguish these two strands. Genomic sequences thus need to satisy additional constraint: sequence and its reverse complement ($A \leftrightarrow T, C \leftrightarrow G$) should be handled in the same

way by the software (they should be indistinguisable) In our case this means that for minimizer of a window W at position p we need to have minimizer $\overline{W^c}$ at position $w-1-p=|W|-k-p$. Groot Koerkamp and Martayan, "SimdMinimizers: Computing Random Minimizers, fast" In most of the software due to simplicity truly cannonical minimizers aren't used. Instead we take minimum of hashes of k-mer and its reverse complement: $m^{\text{cannonical}} = \min(m, \overline{m^c})$

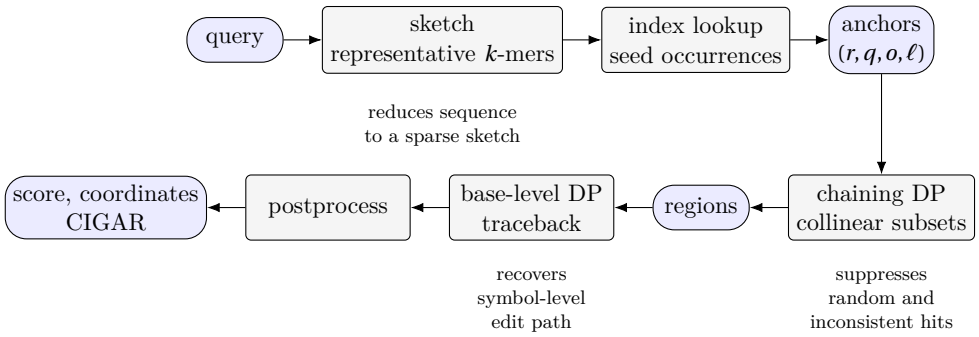


Figure 2.2: The seed–chain–extend decomposition

In case of Minimap2 and BioseqDB $S_{X\cap Y}$ is represented as tuple (r,q,o,ℓ) which holds reference coordinate r, query coordinate q, orientation o and span ℓ (length).

After sorting anchors by ending reference coordinate, with implementation-specific tie keys, a chaining score can be written

$$f(i) = \max\Big(w_i,\ \max_{j<i}\{f(j)+\alpha(j,i)-\beta(j,i)\}\Big), \tag{2.6}$$

where α rewards newly covered matching bases and β penalizes large gaps and diagonal changes. A representative minimap2 reward is

$$\alpha(j,i) = \min\big(\min(y_i-y_j,\ x_i-x_j),\ w_i\big).$$

Testing every $j<i$ is $O(A^2)$ for A anchors. Minimap2 scans a bounded recent window and stops after a configured number of unproductive indices, making the practical cost much smaller Li, "Minimap2: pairwise alignment for nucleotide sequences."

Algorithm 2 Conceptual bounded predecessor chaining

1. Input. Anchors $a_1, \ldots, a_A$ in canonical coordinate order.
2. Endpoint loop. For each $i = 1, \ldots, A$, perform the following steps:
   (a) Initialize $f[i] \leftarrow \mathsf{span}(a_i)$ and $p[i] \leftarrow \perp$.
   (b) For each candidate $j < i$ in the active distance and iteration window, perform the following steps:
      i. If a_j and a_i have valid order, strand, gap, and bandwidth, compute
      $$v \leftarrow f[j] + \mathsf{reward}(a_j, a_i) - \mathsf{gapPenalty}(a_j, a_i).$$
      ii. If the candidate is valid and $v > f[i]$, set $f[i] \leftarrow v$ and $p[i] \leftarrow j$.
      iii. Optionally stop the predecessor loop after the configured skip heuristic fires.
3. Backtracking. Backtrack from high-scoring endpoints through p.
4. Result. Return the chains that remain after filtering by score, seed count, and overlap.

2.4. Limitations of existing tools

Most of the current most popular tools implement variations of the seed and extend approach described in the previous section. However the algorithm outlined here is heavily underspecified to be directly implemented as a computer program:

- $S_{X \cap Y}$ can be implemented as interection of two hash maps where $Seed(X)$ is realized as map from seed to its start/end: $S \to \natural^2$ where seed itself is represented by it's hash $S = \mathbb{Z}$. This opens few important questions: what hashing strategy to choose? Which map implementation to choose?
- As $Seed(X)$ can be computed before alignment starts we can store it in memory and persist it to disk to speed up processing in the future. Which binary format we want to choose?
- Anchors as tuples will be implemented as some concrete binary structure and authors need to specify their representation (do we store them as AOS or SOA? What are widths of elements and their types?). Each decision will imapct performance as there can be millions of anchors per long sequence and additional bytes sum up to MBs of additional memory to be processed (and slowdowns).
- Finally, what is the most effective way to parallelize this computation (the most straightforward approach is to split input sequences X and Y into chunks)

Most of the current state of the art tools choose to:

1. File-based operation
2. Suboptimal or non-extensible internal data structures
3. Optimise only selected parts of the algorithm like hash function properties or one specific stage.

This leads to different consequences for the user.

File-based operation: Operating directly on the disk with simplistic wire format means that seed index is usually one giant binary blob not prepared for highly-concurrent access and interface doesn't offer modern tools like buffer pooling or lazy loading. The format of indexes are largely undocumented by authors. Missing of lazy loading feature is the most hurting to the user. If index is large it will be loaded entirely into memory and can overwhelm SWAP space. This degrades performance by several orders of magnitude and can be avoided by splitting sequences into smaller chunks. User doesn't know what is the exact relationship between sequence length and required memory space, so this process takes time and involves guessing. Additionally index implementation sometimes offer complex parameters to adjust its partitioning (like in case of MMseqs2), but entire responsibility is still shifted to the user and that leads to user frustration andrecurring OOM issues or severly degraded performance. Youngblut, MMseqs2 Github issue 338 Zhan, MMseqs2 Github issue 517 Software not only treats indexes as files, but it operates on FASTA files directly. This means the user cannot easily sort/split/extract or filter records without using external tools.

Suboptimal or non-extensible internal data structures: Minimap2 uses 31 bits to encode anchor's position meaning that practically entire human genome (>3Gbps) cannot be aligned at once and require splitting via other tools before any operation is performed. LexiMap (not using seed and extend approach) limits maximum genome size to 268Mbp Shen, LexicMap FAQs. Additionally software is implemented without proper abstractions - minimap2 code is non-easily extensible C code, while G3SA doesn't offer extension points for new types of indexes or sequences interfaces.

Selective optimisations: Optimisations are usually fixed for one single platform (AVX512 or SSE) and either get severe performance penalties (scalar fallbacks) or user cannot use them at all. SIMD support is quite popular even though it supports one or two selected platforms, but GPU support exists usually only for NVIDIA graphics cards. This leads to a difficult situation for the user as they need to choose between several versions of the same tool (Saurabh Kalikar, mm2-fast: Accelerating minimap2 for long-read sequencing applications on modern CPUs, Sadasivan H, mm2-gpu: Accelerating Minimap2 for Accurate Long Read Alignment on GPUs), while better, more modern software like G3SA exists purely on NVIDIA hardware Han et al., "G^{3}SA: A GPU-Accelerated Gold Standard Genomics Library for End-to-End Sequence Alignment" Modern hardware like MacBook Pro M3 with arm64 architecture won't be able to utilise any integrated GPU acceleration. While minimap2 offers compatibility layer to port SSE code to NEON, it does not vectorize all operations and code ported from SSE doesn't account for lane widths and instruction level parallelism available in M2-M4 chips. Feintzeig, SIMD + Parallelism: Ray Tracer Optimization Part 3

I wanted to address these limitations by implementing BioseqDB, Postgres extension that will provide the following features:

- Tight integration with Postgres meaning that user can operate on sequences (loaded FASTA files) directly using SQL rather than resorting to external toolkits.
- Support for indexes and storage structures designed with parallelism and lazy-loading from the start, meaning that software can be run on very RAM-constrained device and still perform well. At the same time design will incorporate assumptiuon about arbitrary large sequences so the user will be able to operate on >3Gbps records without any additional steps.

- Provide layer of abstractions written using latest C++26 standard, so the algorithms can be reused in separate software or extended to different types of new sequences and indexes.
- Provide optimised kernels for SIMD and GPU architectures written using architecture-agnostic, but performant code.

Tight integration with Postgres also means that as a side effect, we can utilise some benefits of persistent state: deferring expesive operations, keeping large indexing structures in memory in case use reruns query and more effective memory-disk caching schemes.

Chapter 3

The BioSeqDB alignment pipeline

3.1. Alignment as a database execution pipeline

This is empty for now.

Chapter 4

Minimizer extraction

4.1. From every base to a sparse, comparable sketch

The first computational stage of the BioSeqDB aligner needs to compute $\mathsf{Seed}(X)$. A target seed becomes a key in an index. A query seed is looked up in that index, and each compatible occurrence becomes an anchor for the chaining stage described in Section 5.1. The seed set must be sparse enough to reduce index and lookup costs, but dense enough that related sequences retain common seeds.

BioSeqDB uses minimizers. A minimizer is the smallest by some score function k-mer in a window of consecutive k-mers Roberts et al., "Reducing storage requirements for biological sequence comparison."

4.1.1. Canonical k-mers and minimizer windows

Let $S = s_0 s_1 \dots s_{n-1}$ be a nucleotide sequence. The k-mer beginning at zero-based position i is

$$K_i = S[i, i+k).$$

For DNA, the forward and reverse-complement encodings are

$$F_i = \mathrm{enc}(K_i), \qquad R_i = \mathrm{enc}(\overline{K_i}).$$

A canonical k-mer uses the same key on both strands. A common choice is

$$C_i = \min(F_i, R_i), \qquad z_i = \mathbf{1}[R_i < F_i],$$

where z_i records the selected strand. Applying a hash function H gives a candidate

$$x_i = (h_i, p_i, z_i), \qquad h_i = H(C_i).$$

The position p_i may denote the first or final base of K_i. BioSeqDB's minimap-compatible schemes use the final base because that is the convention used by minimap2 Li, "Minimap2: pairwise alignment for nucleotide sequences."

For a window of w consecutive candidates,

$$W_j = (x_j, x_{j+1}, \dots, x_{j+w-1}), \qquad m_j = \min_{x \in W_j} h(x).$$

Every identity attaining m_j is a possible minimizer. We must define which equal minimum wins and whether different positions with the same minimum hash are emitted. We should

lso avoid emitting the same hash again when that candidate remains minimal as the window moves. For independent random hashes, the expected density is approximately $2/(w+1)$ minimizers per candidate position.

Figure 4.1 follows one k-mer through hashing, window selection, and adjacent deduplication. The per-window minimum positions are calculated from the displayed hashes. In particular, the window beginning at hash position five has minimum position eight, not position five.

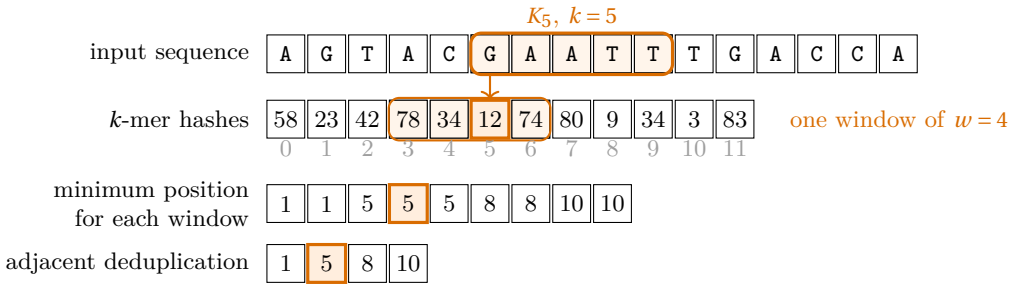


Figure 4.1: Minimizer extraction for $k=5$ and $w=4$ with illustrative hashes. Positions in this walkthrough are zero-based k-mer starts. BioSeqDB's packed minimap-compatible record converts them to final-base positions. The highlighted candidate has hash 12 and remains minimal for three consecutive windows, so adjacent deduplication emits its position once.

Minimizer stochastic properties

Minimizer density counts selected positions in one sequence. Let $N=L-k+1$ be the number of candidate k-mer starts in a fragment and let $M_L^{(1)}$ be the number of distinct positions selected when every window chooses its leftmost minimum. Every full window of w candidates selects one position. Therefore, for $N \geq w$,

$$\left\lfloor \frac{N}{w} \right\rfloor \leq M_L^{(1)} \leq N-w+1. \tag{4.1}$$

The lower bound follows by partitioning the candidates into disjoint windows. The upper bound equals the number of full windows. A sequence of strictly decreasing ranks attains it. An implementation that emits every tied minimum uses a different count $M_L^{(\mathrm{all})}$, whose upper bound is N.

Fix an ordering rule and break ties by choosing the leftmost minimum. For an infinite sequence whose bases are sampled independently and uniformly from alphabet Σ, the stochastic density is

$$\rho = \lim_{L\to\infty} \frac{\mathbb{E}[M_L^{(1)}]}{L-k+1}.$$

The general bounds are

$$\max\left\{\frac{1}{w}, \frac{1}{|\Sigma|^k}\right\} \leq \rho \leq 1. \tag{4.2}$$

Under independent continuous scores at candidate positions, the density is $2/(w+1)$. For a random fixed order on k-mer values, the density also depends on k, w, and overlap between adjacent k-mers Shur, On Minimizers of Minimum Density. The value $2/(w+1)$ is not a universal bound for a fixed order or sequence.

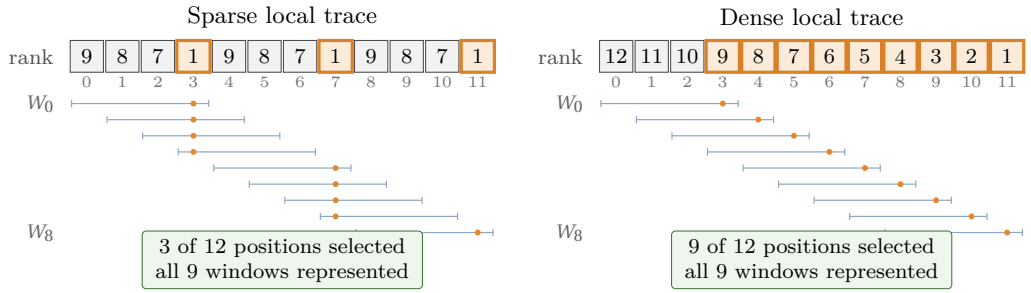


Figure 4.2: Two minimizer traces for $w = 4$. Thin brackets mark candidate windows, and orange dots mark their selected minima. The left trace reuses positions 3 and 7. The right trace selects a new position after each slide.

Density alone does not determine the number of anchors. Let $C_X(v)$ count the selected occurrences of canonical k-mer value v in fragment X. The number of raw anchor pairs between fragments U and V is

$$A(U,V) = \sum_{v} C_U(v) C_V(v). \tag{4.3}$$

Two fragments can each contain $\Theta(L)$ selected positions while their sets of selected values are disjoint. In that case they produce no anchors. Occurrence filters can only remove pairs from Equation 4.3.

Minimap2 minimizer sketch
mm_sketch 77

Anchor lookup and frequency filtering

Figure 4.3 uses fixed four-base seeds on both sides. The symbols h_1, h_2, and h_3 are some computed hashes.

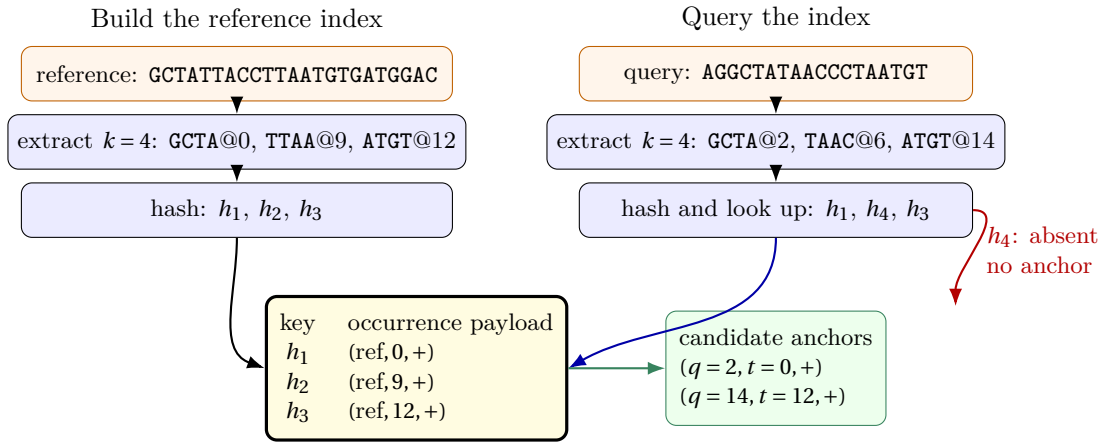


Figure 4.3: Reference-index construction and query lookup. Positions are seed starts in this example. A usable seed index stores some seed metadata like position and strand

Frequent seeds can produce very large occurrence lists. BioSeqDB can exclude seeds whose target frequency exceeds a configured threshold before anchors are constructed. This is common practice in other software as well: minimap2 can remove most common minimizers for specific sequence by using percentage threshold (`--q-occ-frac` $\in [0, 1]$)

4.2. Methods of minimizer generation

4.2.1. Scalar rescan

The most straightforward algorithm to select smallest hash in a given sliding window is to implement scalar rescan. We keep track of current minimum until it leaves the window. Then we need to do ”rescan” part which calculates new minimum from scratch. Additionally we perform deduplication of adjacent hashes, so if the minimum is unchanged, we don’t emit new minimizer. However if there’s new equal hash on different position, then we treat that as a new minimizer.

This approach is implemented by minimap2.

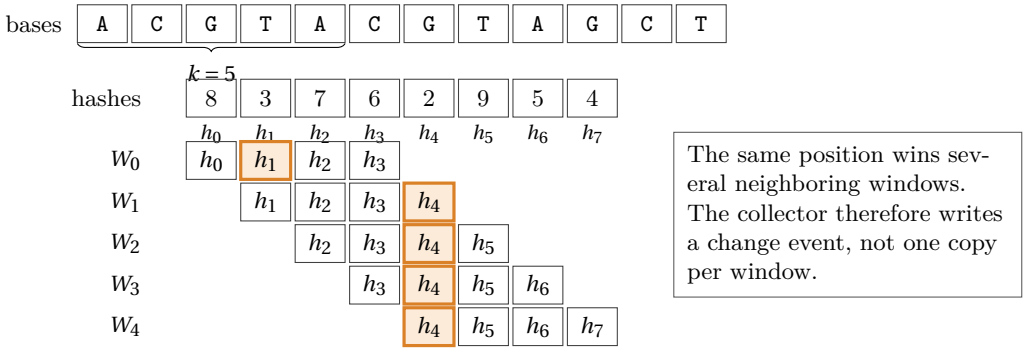


Figure 4.4: Minimizer extraction with illustrative $k = 5$, $w = 4$ hashes. Each row is one window of four adjacent k-mers. Orange marks the selected position. The hash value 2 remains selected while the window moves, so those windows do not require repeated output.

Equal hashes at different positions are not duplicate records. We wanmt to keep both to guarantee higher minimizer density.

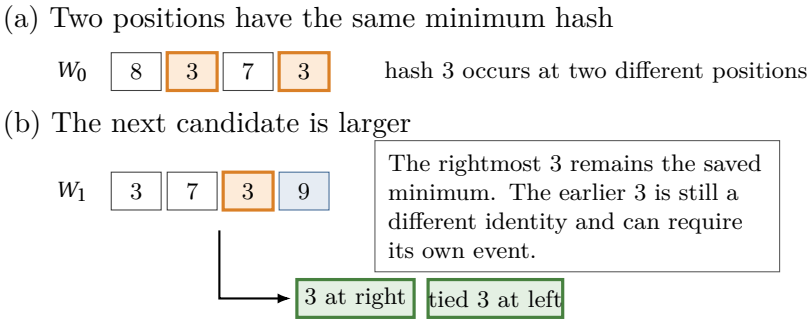


Figure 4.5: Why a hash value alone does not define the output. Both orange cells have hash 3, but they refer to different k-mer positions. Minimap2 keeps the rightmost equal position as its saved minimum and has explicit rules for writing other tied identities.

Minimap2 stores the most recent w candidates in a circular buffer It also stores the current minimum and the array slot containing it.

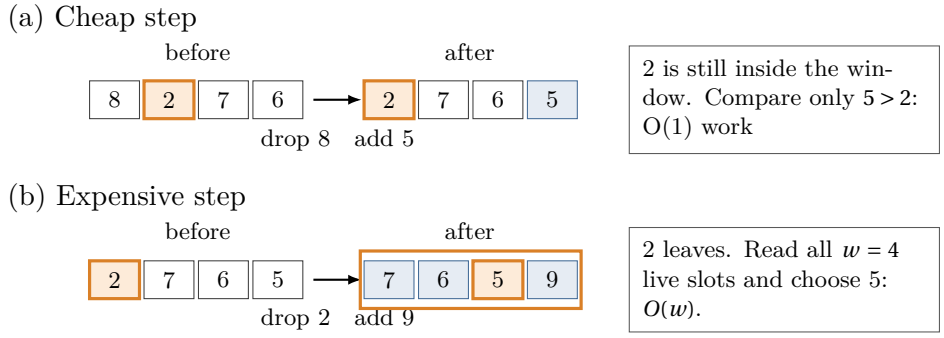


Figure 4.6: The two costs of one minimap2 window advance. Gray cells are already in the ring, blue is the entering candidate, and orange is the saved or newly found minimum. A normal step performs one comparison. When the saved minimum leaves, the algorithm reads the complete window again (rescan)

Minimap2 stores kmers and apply hash function each time from scratch.

Algorithm 3 Scalar minimap2-style rescan

1. Store k-mer binary representation and its reverse complement and update them each step. Calculate hash for both of them and take the minimum.
2. Write this hash into the next slot of the w-slot circular buffer.
3. If new hash is less than or equal to the saved minimum, emit new minimizer.
4. Else, if the overwritten slot held the current minimum, emit new minimizer and scan all w slots from oldest to newest, and save the rightmost record having the smallest hash as new minimum.
5. At the end of the sequence, write the remaining saved minimum from buffer as final minimizers.

4.3. A1: Naive SIMD rescan

Let's assume we have scalar linear iteration over data. The first though is to unroll the loop so each element of a SIMD vector corresponds to one iteration. This approach however works only if each iteration step is independent. If we have any condition that requires knowledege of results of previous step the SIMD vector's elements will be dependent on each other and won't give us any advantage. The other approach would be to threat each element not as consecutive input items, but as separate entities handling different parts of the input. Each lane will handle separate parts of iteration, so we will avoid data dependency of vector's elements on each other and replace it with the dependency between current state vector and previous state vector. In this document, I will define a SIMD lane as one numbered element of a vector register.

Each SIMD lane will load fragments of input data:

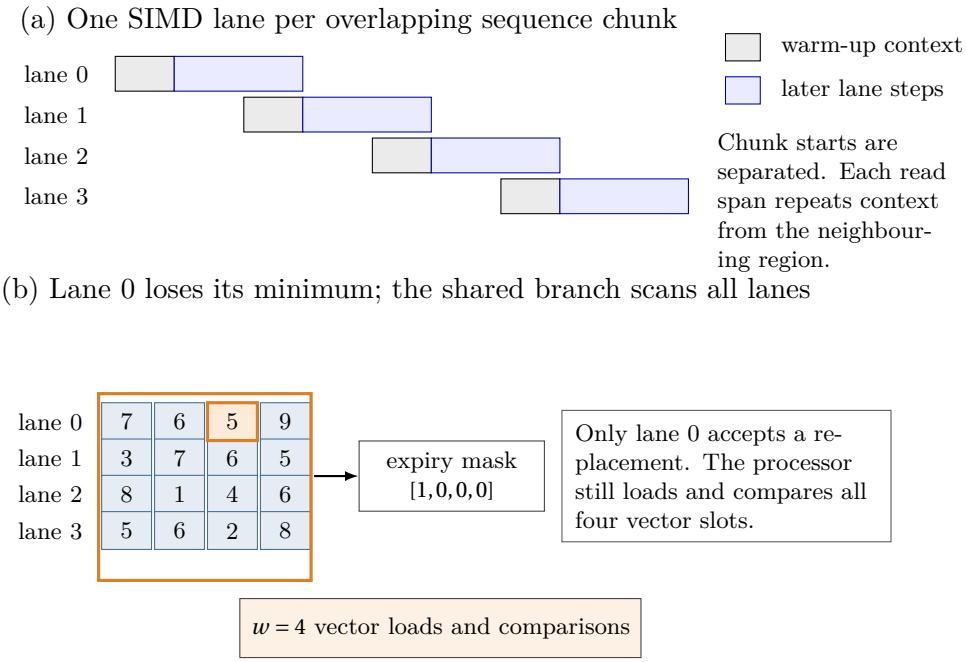


Figure 4.7: Why direct SIMD rescan is costly. Each lane follows a different sequence piece. A nonzero expiry mask enters one shared loop over all w vector slots. Masking protects lanes 1–3 from changing their answers, but it does not remove their loads and comparisons.

Note that algorithm fundamentally stays the same, we only now divided input sequence into chunks and translated each operation (hashing, comparisons) into SIMD instructions:

Algorithm 4 SIMD minimap2-style rescan, with the output cases stated plainly

1. Split input sequence into chunks each corresponding to the lane of SIMD vector. Each lane handles different chunk (chunks overlap slightly to make sure we don't omit any window)
2. Store k-mer binary representations and its reverse complements and update them each step for each lane. Calculate hashes for both of them and take the minimum for each lane.
3. Write all hashes for each laneinto the next slot of the w-slot circular buffer (this buffer now stores SIMD vectors instead of single hashes).
4. If the new hash vector is less than equal to the minimum vector element-wise, extract corresponding elements and emit new minimizers.
5. Else (new hash vector is greater then minimum vector on each position), if the overwritten slot held the current minimum for any lane, emit new minimizers and scan all w slots from oldest to newest, and save the new minimums vector.
6. At the end of the sequence, emit all remaining minimums as final minimizers.

As you can see this algorithm works well only if new hash values are greater than current minimum on each position. If not, we still have branching. This is problematic as vectorized algorithms works well if the branching is minimal and CPU pipeline isn't penalized for wrong branch prediction. Even if we assume the predictor is perfect, we still do expensive rescan for each lane if at least one needs to be updated. Assume one lane needs a rescan with probability

$1/w$, then a group of L independent lanes enters the shared slow path with probability

$$1-\left(1-\frac{1}{w}\right)^{L}.$$

For $w=10$ and $L=4$, this is about 34.4%. This means that we will need to duplicate work much more often than in scalar case.

4.4. A2: Two-stack algorithm

In different approach we can preserve prefix and suffix minima to calculate the window minima wrom both of them. SIMD-minimizers paper introduces this approach as well as usage of rolling hashes (nthash) instead of calculating hash each time from scratch. Groot Koerkamp and Martayan, "SimdMinimizers: Computing Random Minimizers, fast"

4.4.1. One window minimum from two saved values

Split the hash stream into blocks of w values. Within each block, let P_i be the smallest value from the block start through position i, and let Q_i be the smallest value from position i through the block end. A window starting at j and ending at $e=j+w-1$ crosses one block boundary and has minimum

$$\min(Q_j, P_e).$$

The suffix values are recalculated in a fixed w-step loop each w steps. This means that we still have "rescan" part, but it becames unconditional.

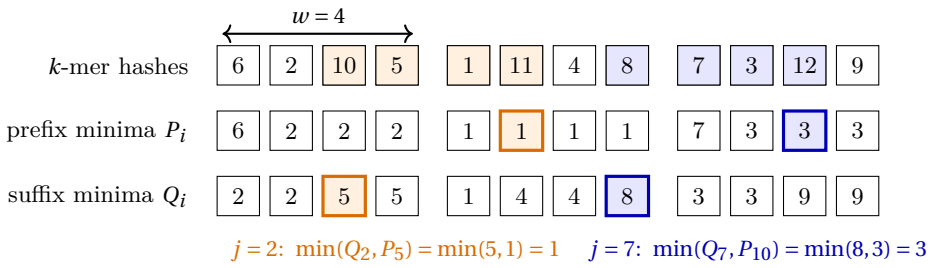


Figure 4.8: Sliding minima from blockwise prefixes and suffixes. Blocks have the same width as the minimizer window. The two shaded crossing windows each use one suffix value from the starting block and one prefix value from the ending block.

Fixed-width SIMD vectors

The SIMD-Minimizers use fixed-vector size architecture, where the SIMD vector is set to be uint32 times 8. The split per lane is the same as discussed previously. On Neon architecture only 128-bit-wide register is available, so the 256 fixed vector generates separate store intructions.

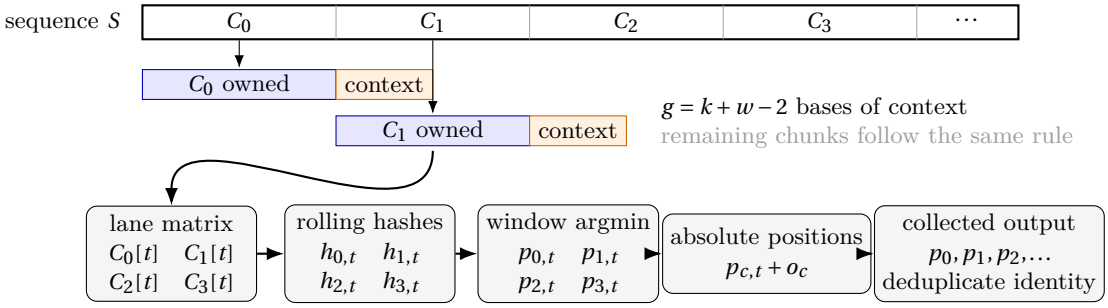


Figure 4.9: Vectorized minimizer extraction over logical chunks. Each chunk owns a disjoint range and reads enough context to complete its boundary windows. Lane-wise rolling hashes and minima produce local positions. Adding each chunk origin and deduplicating full identities yields a global sketch. The symbolic chunk size is an execution parameter rather than part of minimizer semantics.

Note that in this setup we can store full minimum vectors and later use matrix transposition to deduplicate them (in case that minimum stays the same over window): At first, one SIMD vector contains the selected positions for several far-apart streams at the same step. After several steps, the SIMD-Minimizers paper algorithm rearranges that output matrix so one row contains consecutive windows from one stream. Consecutive windows often select the same position. Once those positions are adjacent in a row, the paper compares neighbors and keeps only positions that changed. Finally, when the ordered stream results are joined, it removes the repeated boundary position caused by the overlapping input pieces.

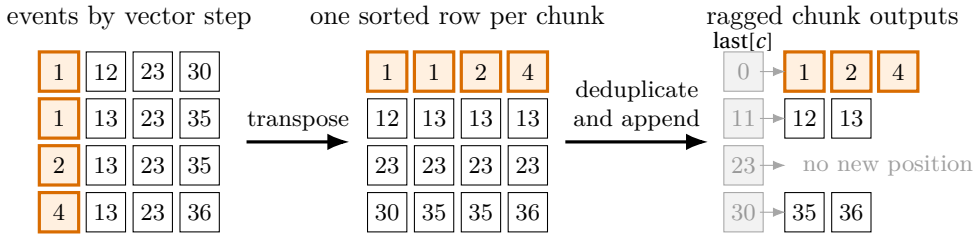


Figure 4.10: Transposition and boundary deduplication for four logical chunks. The input rows are vector steps and the columns are chunks. Transposition makes each chunk a sorted row. Each row has its own previous-position carry, shown in gray. Only new positions are appended, so padding cells are not part of the output.

4.5. A3: Chronological SIMD

Looking at A1 we observed that the main problem was that far-apart sequence chunks can have different characterisistics which cause us to do lots of additional work. Instead of splitting sequence that way, we can work on just one part of neighbouring positions.

We will try to calculate minimums for all local windows i.e calculate result for $X[p, p+1, ..., p+k+w-1]$ (this is window of size w for kmers of size k). Physically we only have L SIMD lanes so we will call that a group and advance only by L positions at the time (i.e from $X[p, p+1, ..., p+k+w-1]$ to $X[p+L, p+L+1, ..., p+k+w-1+L]$).

Potentially we can have more than one group per call. It offers some advantages discussed later. If the group number is higher, then we just execute their operations serially and perform result emission only after all groups execute.

The idea for one group is as follows:

1. We load L new sequence positions into L SIMD lanes each one is shifted by 1 using load instructions

2. For each lane we compute minimum along with strand direction and information which position it is. We call that new candidates: $x_0, \ldots, x_{L-1}$

3. We iterate through all candidates i.e $D \leftarrow T \parallel (x_0, \ldots, x_{L-1})$ where T are previous $w-1$ candidates and calculate new minimums. This gives us M which are L minimums for entire parts of sequence:

$$\begin{aligned}\{&\mathsf{Seed}(X[p, p+1, \ldots, p+k+w-1]),\\ &\mathsf{Seed}(X[p+1, p+2, \ldots, p+k+w]), \ldots,\\ &\mathsf{Seed}(X[p+L, p+L+1, \ldots, p+k+w+L])\}.\end{aligned}$$

   Now this along last previous result c^* forms P: $\mathbf{P} \leftarrow (c^\star, M_0, \ldots, M_{L-2})$

4. For each lane ℓ we compare P_ℓ with x_ℓ. If $h(x_\ell) \le P_\ell$ then P_ℓ is a new minimizer and we emit it. Otherwise if $\mathrm{pos}(P_\ell) = \mathrm{pos}(x_\ell)$ then we see P_ℓ expired so we emit it.

5. If the expiry happens then we go through all w windows and check if their minimum has the same value but different position. This only requires checking $m_0, m_1, ..., m_{w-1}$. If yes then we do slow path: extract the ties in order (hence the name of the algorithm: chronological)

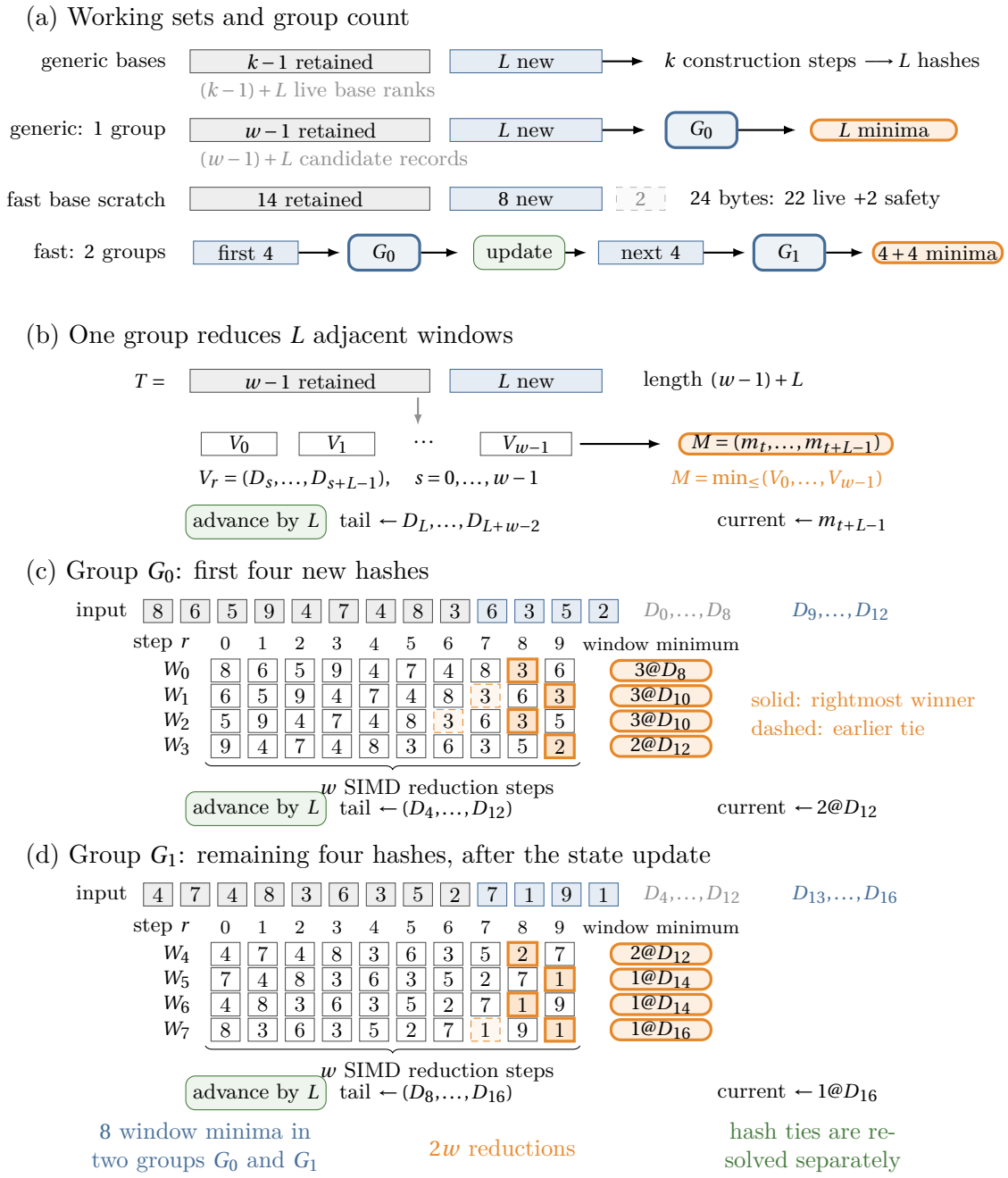


Figure 4.11: Working sets and execution of the chronological adjacent-window kernel. One group combines previous $w-1$ candidates with L new candidates, then loads w shifted vectors, and reduces them componentwise to the rightmost minima of L adjacent windows. Orange cells represent selected minimums (candidates for potential minimizer emit). A solid orange cell is the rightmost equal minimum selected by the source's $\leq$ comparison and a dashed cell is an earlier equal minimum.

The figure 4.11 illustrates calculation of window minima for $L = 4$ lanes: $x_0, x_1, ...x_{L-1}$. The entire algorithm looks as follows:

Algorithm 5 One iteration: build L k-mers and rescan L adjacent windows

Carry: C = last $k-1$ bases, T = last $w-1$ candidates, $c^\star$ = preceding window minimum
while the next L candidate bases are available do
  $U \leftarrow$ next L bases
  $Y \leftarrow C \parallel U$
  $\mathbf{F}, \mathbf{R} \leftarrow \mathbf{0}$
    Build forward/reverse kmers symbol by symbol.
  for $t = 0, \ldots, k-1$ do
    $\mathbf{z} \leftarrow \mathrm{load}_L(Y[t \,..\, t+L))$
    $\mathbf{F} \leftarrow F \parallel z$
    $\mathbf{R} \leftarrow \overline{z} \parallel R$
  Calculate hashes from kmers.
  for all lanes ℓ do
    $a_\ell \leftarrow H(\min(F_\ell, R_\ell))$
    if $F_\ell = R_\ell$ or $a_\ell = h_{\max}$ then
      $x_\ell \leftarrow \infty$
    else
      $x_\ell \leftarrow \mathrm{pack}(a_\ell, \mathrm{position}_\ell, [R_\ell < F_\ell])$
  $D \leftarrow T \parallel (x_0, \ldots, x_{L-1})$
  $\mathbf{M} \leftarrow \infty$
    Calculate M, which holds the rightmost minima of all w windows.
  for $s = 0, \ldots, w-1$ do
    $\mathbf{V} \leftarrow D[s \,..\, s+L)$
    $\mathbf{M} \leftarrow \mathrm{select}(h(\mathbf{V}) \le h(\mathbf{M}), \mathbf{V}, \mathbf{M})$
  One step = three field loads + one compare + three selects.
  Hence M_ℓ is the rightmost minimum of the complete window ending at x_ℓ
  $\mathbf{P} \leftarrow (c^\star, M_0, \ldots, M_{L-2})$
  for all lanes ℓ do
    $e \leftarrow \emptyset$
    if $x_\ell = P_\ell$ then
      $e \leftarrow \mathrm{event}^{\mathrm{NEW}}(P_\ell)$
    if $\mathrm{pos}(P_\ell) = \mathrm{pos}(x_\ell)$ then
      $e \leftarrow \mathrm{event}^{\mathrm{EXPIRED}}(P_\ell)$ expiration
    if e is Expired then
      Check all w windows for an equal hash on a different position
      $e \leftarrow \mathrm{event}^{\mathrm{TIE}}(P_\ell)$
    if e is Tie then
      Reread the w records only to construct ordered output slow path
  $C \leftarrow Y[L \,..\, L+k-1)$
  $T \leftarrow D[L \,..\, L+w-1)$
  $c^\star \leftarrow M_{L-1}$
if $0 < q < L$ bases remain then
  Run the same body once with q active lanes and sentinels elsewhere
  Both loops still execute k and w steps
  $C \leftarrow Y[q \,..\, q+k-1)$, $T \leftarrow D[q \,..\, q+w-1)$, $c^\star \leftarrow M_{q-1}$

4.6. Bioseq prototype

We established different approaches to calculate minimizers in previous sections.Now we want to discuss practical considerations when it comes to implementing those algorithms. The work started by exploring minimap2 source code and understanding its behaviour. The first step was to port minimap2 code into modern C++. This was required to have a stable code where

we could test and benchmark different algorithms. I encountered several key problems that lagged implementation by considerable number of months:

1. Minimap2 code uses direct C arrays access for lots of different data structures. Minimizers are encoded as uint128 type represented by a struct with two uint64 fields with abstract filed names. Circular buffers are represented by single raw pointer and require macros for any operations. The encoding and semantics of fields can sometimes change even though underlying uint128 representation stays the same (it's the same field name). This complicates any manual static analysis of the code I conducted.

2. Minimap2 uses raw instrinsics for SIMD operations which makes translation to generic SIMD very tedious.

3. Some parts of minimap2 hash logic are undocumented either in official paper or in-code.

The porting involved several iterations over the minimap2 source to understand how index is split, how parallelization works and how exactly minimizers are represented. After achieving runnable pipeline (with seeding, chaining, alignment and estimator) I started to work specifically on the minimizer extraction part as it's first in the pipeline.

Label	Main change What it means for the minimizer algorithm
V1–V6	naive SIMD rescan approach equivalent to A1
V7–V8	hybrid algorithm A1 where we use two-stack for minimum calculation
V8.5	defer minimap2 hashing to later stages
V9-V10	hand-written input and matrix loop
V11	two independent Highway lane groups
V12	force the per-base body inline
V13	fixes of estimator

Table 4.1: History of the first prototypes

4.7. Seeding prototype V1–V6: Naive SIMD rescan

The first approach (apart from direct minimap2 scalar rescan already existing in the ported code) was implementation of A1 SIMD rescan.

As correctly predicted our benchmark (Table 4.2) showed that the scalar `mm_sketch` took 12.5 ms on a 903 kbp pair, while the "vectorised" `mm_sketch_simd` took 24 ms – a 2× slowdown. This was caused by the fact that any minimum going outside the window causes rescan of all lanes.

Phase	scalar (ms)	V6 SIMD (ms)	V6 over scalar
`mm_sketch`	12.50	14.94	+19 %
`mm_sketch_simd`	0	23.88	—
`target_index_build`	16.0	14.3	−11 %
`collect_seed_hits`	6.5	11.0	+69 %
`mg_lchain_dp`	16.0	0.0	— (replaced)
`mg_lchain_simd`	0.0	8.9	—
`mm_est_err`	0.7	7.2	+929 %
`mm_align_skel.`	87.5	86.4	−1 %
Pipeline total per pair	158.4	170.7	+8 %

Table 4.2: Phase breakdown at 903 kbp, V3 baseline – pre-SIMD-Minimizers work. The naive SIMD path (V6) is +8 % slower than scalar despite four lanes of parallelism. The slowdown is split between three regressions: (a) the SIMD minimizer's `mm_sketch_simd` is itself slower than scalar `mm_sketch`; (b) the SIMD chainer feeds anchors into `collect_seed_hits` that hit a worse code path; and (c) the SIMD chain anchors are not cleanly indexable into the sorted query-minimizer array, so `mm_est_err` explodes. The V13 work mostly fixes (a) and (c).

The downstream regressions on `collect_seed_hits` and `mm_est_err` were caused by SIMD chainer producing a slightly different anchor distribution than scalar. This problem is orthogonal to the minimizer extraction and we mention it later.

4.8. Seeding prototype V8: Implementing A2 algorithm

The single most important external input to this work was the SIMD-Minimizers paper of Groot Koerkamp & Martayan Groot Koerkamp and Martayan, "SimdMinimizers: Computing Random Minimizers, fast." Its central insight is that the rescan method is fundamentally data-dependent, and that any data-dependent branch costs more in SIMD than in scalar because lanes' branch targets are tied to each other.

Algorithm 6 sketches the V8 inner loop.

Algorithm 6 V8 `collect_minimizers_two_stack_simd` (per-step body)

```
state: prefix_min[L], ring[w][L], step, ring_idx
hash ← rolling_kmer_hash.Next(d, c_in, c_out)
strand, palindrome ← StrandMask(hash)
valid ← ¬palindrome
hs ← IfThenElse(strand, hash | 1, hash & ~1)        strand bit packed in LSB
elem ← IfThenElse(valid, ((hash & 0xffff_0000) | pos_local), 0xffff_ffff)
ring[ring_idx] ← elem                               single ring buffer slot per lane
better ← elem < prefix_min
prefix_min ← IfThenElse(better, elem, prefix_min)
ring_idx += 1, step += 1, pos_local += 1
if ring_idx = w then                                rebuild suffix
    ring[i] ← min(ring[i..w − 1]) for all i         O(w) uniform branch
    prefix_min ← elem
    ring_idx ← 0
window_min ← min(prefix_min, ring[ring_idx])
if step ≥ k + w − 1 and window_min.pos ≠ last_emit_pos then
    emit window minimum to per-lane scalar collector
    last_emit_pos ← window_min.pos
```

The V8 numbers were already encouraging: `mm_sketch_simd` dropped from V6's 24 ms to 17 ms at 903 kbp – a 30 % reduction on the minimizer-only phase, which translated into the SIMD pipeline being only 1.4 % behind scalar overall (down from V6's 8 % behind).

Note that in this prototype we weren't using nthash rolling hash, but the original minimap2 hash function.

4.8.1. Seeding prototype V8.5: Deferring hash calculation

Till this time we were using minimap2 hash function for minimizers2, but the observation was that we can use cheaper hash function for ordering values inside the inner loop and then hash the emitted values. If the hash function used will be significantly cheaper we can have proper speedup. The cheap inner hash needs only one property: be a bijection on the $2k$-bit kmer space, so that $\text{lex}(\text{kmer}_1) = \text{lex}(\text{kmer}_2) \iff \text{kmer}_1 = \text{kmer}_2$. We use the "lexicographic-canonical hash" – the bit-packed $\min(\text{forward}, \text{rev_compl})$ but multiplied by Knuth's golden-ratio constant $0x9E3779B9$ Knuth, The Art of Computer Programming, Volume 3: Sorting and Searching.

The arithmetic of the saving:

$$\underbrace{2.25 \cdot 10^5}_{\text{iters / Mbp}} \times \underbrace{(7\,\text{stages} - 2\,\text{stages})}_{\text{ops saved per iter}} \approx 1.1 \cdot 10^6\,\text{SIMD ops saved}$$

The measured saving was ~ 5 ms per 1 Mbp pair on `mm_sketch_simd`, taking the cost from V8's 17 ms down to 12 ms.

4.9. Seeding prototype V9–V11: Stream interleaving and forced inline

A close reading of the assembly created by Apple Clang LLVM revealed inner loop has unfriendly data dependency chain:

`shift_fwd` → `or` → `and` → `Min(fwd, rc)` → `Mul(lex, golden)`.

The Apple M1 has four 128-bit NEON pipes that issue independently, so a single 4-lane stream wastes three out of four pipes for any given cycle of the dependency chain. This feature is called ILP (Instruction Level Parallelism).

This is limitation that wasn't observed in the original SIMD minimizers paper as Rust code used fixed-size SIMD vectors, where we use adaptive vector size equal to possible physical register size. The possible optimisation here is to keep two independent 4-lane streams in flight simultaneously and interleave their operations in source order:

```
shift_fwd_lo, shift_fwd_hi = ShiftLeft(state_fwd_lo / hi)
state_fwd_lo, state_fwd_hi = And(Or(shift_fwd_, bp_), kmer_mask)
state_rc_lo,  state_rc_hi  = Or(ShiftRight(state_rc_), new_high_)
lex_lo,       lex_hi       = Min(state_fwd_, state_rc_)
hash_lo,      hash_hi      = And(Mul(lex_, golden), hash_mask)
elem_lo,      elem_hi      = IfThenElse(valid_, hash_|pos_, max)
```

We chose 2 streams rather than 4 execution stream because the per-stream state plus the shared ring buffer for $w = 10$ already consumes ~ 26 NEON registers per stream. Doubling it could make the ring buffer spill to stack losing the ILP benefit.

The wall-clock effect at 903 kbp was a ~ 4 ms reduction on `mm_sketch_simd` (from 12 ms to 8 ms); Figure 4.12 plots the full evolution.

4.9.1. Seeding prototype V12: Forced inline

V11 was good but disappointing: the wall-clock saving was less than the theoretical 2× ILP win we expected. `xcrun llvm-objdump -d build/bioseq/bioseq_bench` on the V11 wrapper revealed the underlying problem.

```
ldp q0, q1, [sp, #0x150]       // load bp_row from stack
ushr.4s v2, v1, #0x2           // extract per-BP value
movi.4s v1, #0x3
stp q0, q2, [sp, #0x150]       // store back
and.16b v0, v0, v1
and.16b v1, v2, v1
add x0, sp, #0x5, lsl #12      // prepare lambda's captured-this
add x0, x0, #0x770
movi.2d v2, #0xffffffffffffffff
add x1, x22, #0x8
bl 0x1002acca4 <step_one_bp_pair lambda>   // the offending call!
```

Despite the per-step body being decorated with `[[gnu::always_inline]]`, Clang lowered the lambda as a function call at every BP step. Each call paid:

- Function-call prologue/epilogue (`stp x28-x30 [sp,...]`).
- Captured-state spill/reload via stack – every per-stream state register was loaded from the captured-this pointer at the start of each lambda invocation, and stored back at the end.
- No ILP across iterations – each lambda call is its own function frame, so dependent state spills to memory between consecutive iterations.

Figure 4.12 shows full evolution of the prototypes. The scalar part involves unavoidable SIMD overhead of collecting results into map, scalar scramble hashing (minimap2 hash) and encoding output into the bit structure.

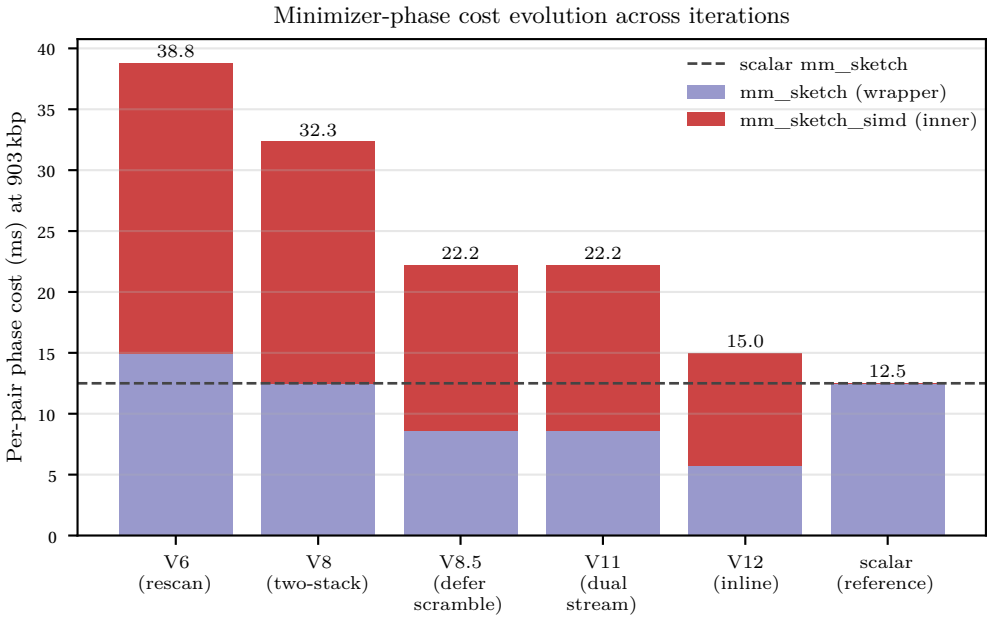


Figure 4.12: Evolution of the minimizer-phase cost (`mm_sketch` wrapper plus `mm_sketch_simd` inner) at 903 kbp across the iteration rounds discussed inuujj §4.7–§4.9. Numbers are means of three runs at `-O3 -DNDEBUG`. V8 is the two-stack port, V8.5 defers the minimap2 hash to emit time, V11 adds dual-stream interleaving, and V12 forces inlining of the per-step body via a free-function template. The dashed horizontal line is the scalar `mm_sketch` baseline. Even with some optimisations still SIMD routine is slower than scalar execution.

4.9.2. Seeding prototype V13: Existmator fix

For each chain, minimap2 estimates the per-base sequence divergence from a Poisson model on the minimizer-set intersection Li, "Minimap2: pairwise alignment for nucleotide sequences":

$$\hat{\epsilon} = \frac{1}{k} \log \frac{n}{m}$$

where n is the number of query minimizers in the chain's reference span and m is the number of those that are actually seeded (i.e. that appear as anchors in the chain). Computing n requires knowing which positions are query minimizers, which is a set membership test against the query's minimizer-position set.

The divergence estimator is used to measure the quality of the seeding. If the divergence is too high, the seeding is not good and the chain is not valid.

When porting minimap2 code we had hidden one important detail visible on table 4.2. The `mm_est_err` section had significant 9-times slowdown. This is because we are using separate chunks for each lane, but the output minimizers were stored in the buffer. Minimap2 normally receives its `mini_pos` array already sorted (because of how scalar loop works). This means that in our case we did the most naive approach:

1. Perform `std::sort` on the array to sort minimizers by positions
2. Perform `std::lower_bound` on the array to test membership

This caused severe slowdown, which we mistakenly attributed to seeding algorithm itself. More on divergence is described in the postprocessing sections.

Figure 4.13 stacks the per-phase cost at 903 kbp for V12-pre versus V13. The two large gains are `mm_est_err` (from 7.88 ms to 0.85 ms) and the V12 `mm_sketch` stack (from 14.99 ms to 9.23 ms, already covered in previous section.

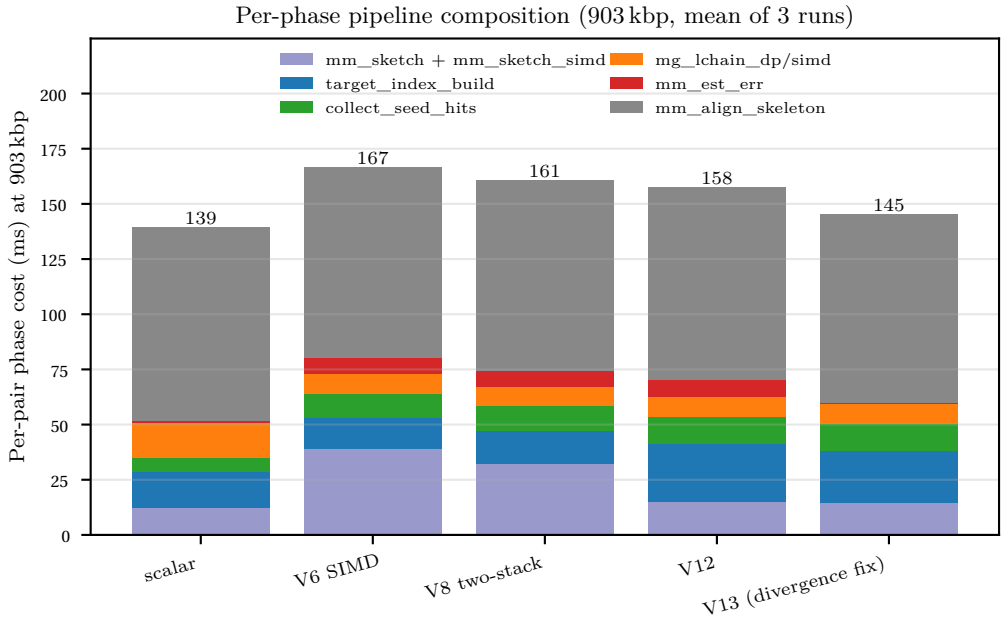


Figure 4.13: Per-phase composition of the per-pair pipeline at 903 kbp we can see that V13 reduced exploded divergence phase

4.9.3. Seeding prototype V14: directly use nthash

So far we utilised minimap2 hash function directly. One other approach was replacing this hash function directly. This should have given us measurable speedup. For nthash we can construct LUT (lookup table) accessed via SIMD instruction.

- `mm_sketch_simd` cost increased by 2 ms (the table lookup used NEON `tbl.16b` instruction and hashing required two rotates, and some numbers XORs per stream per sequence symbol).
- `collect_seed_hits` dropped by 0.4 ms (this isn't significant enough).

My investigation pointed me to the fact that I was trying to optimise incorrect region of computations. The benchmark during all experiments evolved and improved with each iteration. This time the measurement didn't report that most of the time was spent on inserting minimizers into the index.

The gap in performance didn't come from the minimizer hash distribution (or hashing speed itself) but from the usage of `std::unordered_map`. The prototype's target index used STL's default hash table, while the C minimap2 path uses an open-addressing `khash_t<uint64_t>`. This shows another problem with directly porting minimap2 into C++. `std::unordered_map` is known problem with standard library that we overlooked during implementation similarily to how `std::sort` was used for divergence estimation. The cleanest solution here was implementing indexing structures as flat hash maps directly.

4.9.4. Seeding prototype V15: Smaller minimizer space

In all of the experiments we used adjustable minimizer represetations with hash width of 16 bits (minimizer representation is explained in separate section). However smaller represetations mean more efficient algorithms. Existing iterations of seeding algorithm all used adjustable SIMD vector widths provided by C++ Highway library. This means that narrower input type will automtically increase number of available lanes L thus increasing potential performance in all algorithms. However having smaller type means that we need to either get rid of strand information (1 bit) or use narrower position representation (this is not always possible) or use narrower hash representation. We conducted early experiment testing V12 prototype with narrower hash of 14 bits.

	V12 (pre)	V12 (post-Goal-2)	delta
mm_sketch_simd	9.0 ms	9.5 ms	+0.5 ms (insignificant change)
target_index_build	24.7 ms	4.9 ms	−19.8 ms (~85% fewer keys)
collect_seed_hits	12.3 ms	842.0 ms	+830 ms (70x regression)
mg_lchain_simd	8.9 ms	299.9 ms	+291 ms (34x regression)
mm_align_skeleton	87.2 ms	4811.0 ms	+4724 ms (56x regression)

Hash-space collision amplification

The kmer space stored in `target_index` hash buckets collapsed to ~16000 distinct keys for 168000 query minimizers. Each query had ~10 collisions per bucket, meaning the chain-anchor count exploded, and the alignment had to walk through all of them. This means that we sped up index lookups significantly (less memory accesses required), but at the same time the more costly parts of the pipeline get more false positives that need to be verified.

4.9.5. Seeding prototypes summary

TODO

Cite this stack overflow answer: https://stackoverflow.com/a/25027750

The combination of the V12 inlinening fixes, the V13 estimator fix and replacing `boost::unordered_map` (marked as $V13^*$) yielded observable improvements. We didn't want to implement new hashing scheme at this time, only see the impact of using different hash storage.

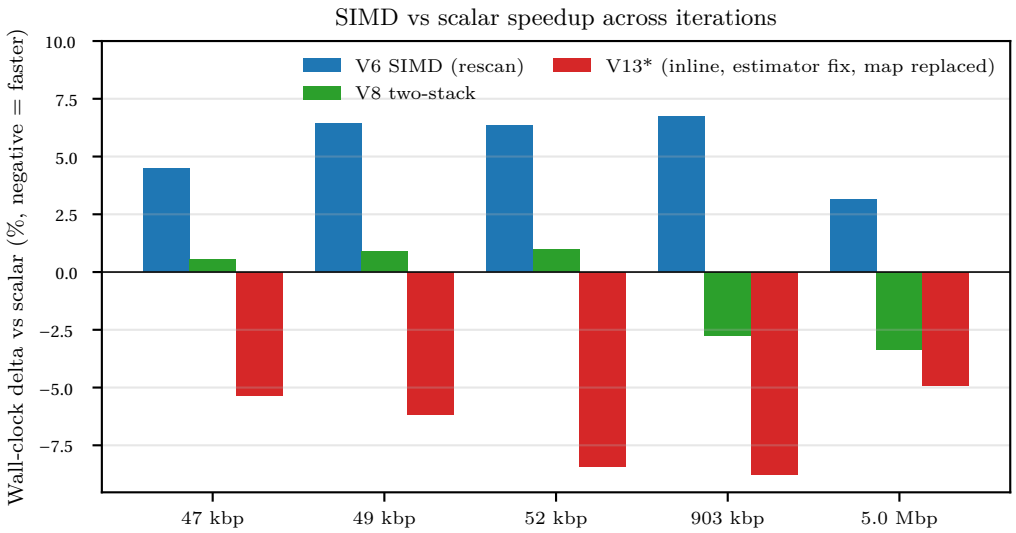


Figure 4.14: Wall-clock delta versus scalar for the three SIMD configurations, expressed as percent (negative = faster). $V13^*$ now performs better than scalar, however this is nowhere near results reported in SIMD Minimizers paper.

The summary table:

Length	scalar (ms)	V13 (ms)	V13 vs scalar	V13 vs V6 SIMD
47 kbp	14.56	13.78	−5.4%	−9.4%
49 kbp	14.94	14.02	−6.2%	−11.8%
52 kbp	15.09	13.82	−8.4%	−13.9%
903 kbp	188.93	172.38	−8.8%	−14.5%
5.0 Mbp	1171.0	1113.3	−4.9%	−7.8%

Table 4.3: Total measured time for alignment (mean of 3 runs from `-O3 -DNDEBUG`

4.10. GPU acceleration for seeding

G3SA Han et al., "G^{3}SA: A GPU-Accelerated Gold Standard Genomics Library for End-to-End Sequence Alignment" provides implementation of multiple useful kernels including one specific to minimizer generation phase.[1]

G3SA seeding kernel
mm_sketch_device 83:1

G3SA implements the A3 adjacent-window computation graph, in CUDA SIMT. Let $L = 32$ be the number of CUDA threads. For group $r \in \{0,1,2,3\}$ and thread $t \in [0, L)$, define

$$j^\star_{r,t} = \max \{ \arg\min \}_{0 \le j < w} h(C_{rL+t+j}), \qquad W_{rL+t} = C_{rL+t+j^\star_{r,t}}.$$

The outer maximum keeps the rightmost record among equal minimum hashes. The vector $(W_{rL}, \ldots, W_{rL+L-1})$ is the componentwise reduction of w shifted candidate vectors. This is the same exact computation as Algorithm 5, but CUDA threads replace SIMD lanes. One difference is that there's no some global state preserved so it's need to be recomputed at initialisation.

[1] Inspected code from official G3SA repository that supplies CUDA kernels along with CPU host-side code, https://github.com/sunwookim028/G3SA

The computation is divided according to the following geometry:

$$L = 32, \qquad G = 4, \qquad n_{\text{owned}} = LG = 128, \qquad n_{\text{shared}} = L(G+1) = 160.$$

We have 128 blocks of 32 threads. Block b handles reads $b, b+128, b+256, \ldots$, one read at a time. For each read it advances by $n_{\text{owned}} = 128$ window starts. In one advance, thread t attempts candidate slots $t, 32+t, \ldots, 128+t$ and then reduces the four windows beginning at $t, 32+t, 64+t$, and $96+t$. Figure 4.15 shows this transpose between candidate construction and window reduction.

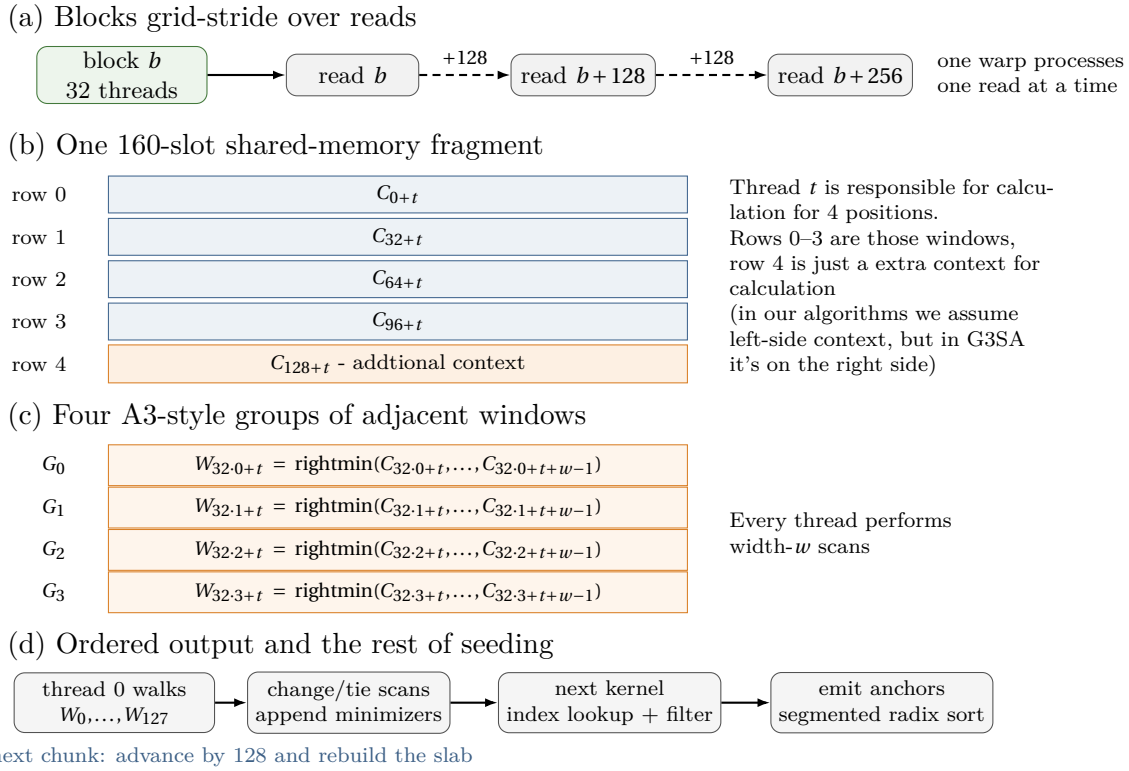


Figure 4.15: G3SA algorithm reconstructed from the CUDA code. This is similar to A3, but has specific work split per thread/warp (standrd CUDA SIMT model)

4.10.1. Sketch pseudocode and work

Here, PublishBlockWrites returns a block-scoped phase token. The token becomes ready only after every thread in the block has reached that phase and the named shared-memory writes are visible to every thread; awaiting it is therefore a collective rendezvous.

Algorithm 7 G3SA sketching CUDA algorithm

Require: concatenated reads S, offsets O, lengths N, index parameters k, w

```
shared Base[160], Cand[160], Winner[128]
procedure SketchBlock(b, t)   b is a block; t ∈ [0,32) is its thread
    for q ← b; q < |N|; q ← q + gridDim_x do
        p ← O[q], e ← O[q] + N[q], m ← 0
        while p < e do
            for r ← 0 to 4 in every thread do
                u ← 32r + t
                Base[u] ← (p + u < e) ? S[p + u] : NUL
                Cand[u] ← ∞
            baseReady ← PublishBlockWrites(Base, Cand)
            await(baseReady)
            for r ← 0 to 4 in every thread do
                u ← 32r + t
                Cand[u] ← GetHashedKmer(Base, u, k, p − O[q])
            hashReady ← PublishBlockWrites(Cand)
            await(hashReady)
            for r ← 0 to 3 in every thread do
                s ← 32r + t, best ← ∞
                for j ← 0 to w − 1 do
                    if Cand[s + j].hash ≤ best.hash and p + s + j + k − 1 ≤ e then
                        best ← Cand[s + j]
                if p + s + w + k − 1 ≤ e then
                    Winner[s] ← best
                else if N[q] < w + k − 1 and s = 0 and best < ∞ then
                    Winner[0] ← best   source short-read case
                else
                    Winner[s] ← 0
            winnerReady ← PublishBlockWrites(Winner)
            await(winnerReady)
            if t = 0 then
                m ← AppendChangesAndHashTies(Cand, Winner, w, O[q], m)
            p ← p + 128
        if t = 0 then
            minimizerCount[q] ← m
```

4.11. BioseqDB GPU seeding

The BioseqDB minimizer generation uses minimap2's scalar rescan approach ported to GPU. Specifically input is divided into tiles, where each invocation handles one tile using rescan algorithm.

When talking about GPU shaders, Vulkan's workgroup correspond to CUDa thread block, while subgroup is a direct equivalent of CUDA warp.

Let Q be the uploaded query batch and let $\mathcal{T} = (t_0, \ldots, t_{T-1})$ be its flat tile list. The ordered dispatch uses $G = \min(\lceil T/64 \rceil, G_{\max})$ workgroups, where $G_{\max}$ is the device's workgroup limit.

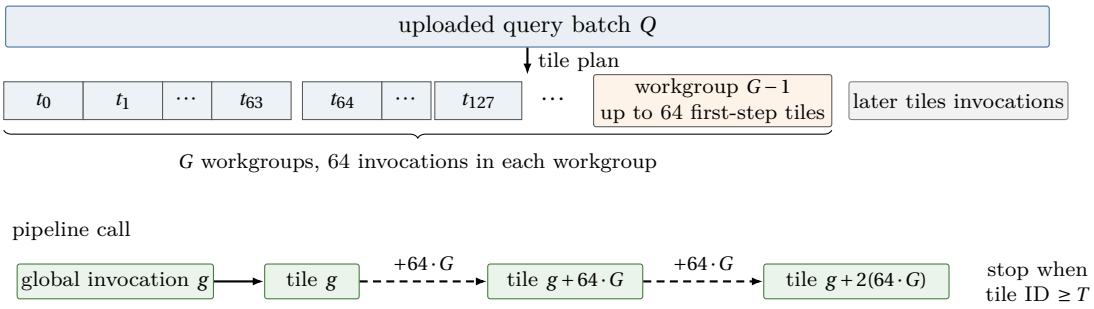


Figure 4.16: Tile assignment in an ordered-sketch dispatch. Each of the G workgroups has 64 invocations. Invocation g processes one tile at a time and advances by $64 \cdot G$ while another tile remains.

When we collect results we need to either: have fixed memory offsets per tile and perform another host pass to extract results or we can do lazy approach where we execute pipeline twice. First we use count mode to caculate where to place results and then do a second pass to actually write them. This makes pipeline do twice as much work, but avoids any additional host pass and we can directly use output results. The limtiations of our work is that we didn't measure the difference between these approaches or between G3SA approach.

The core part is OrderedSketchTile which runs the rescan algorithm in count mode and writes Count. The Count[j] is the number of output records for tile j. PrefixScanBlock reads Count. Each of its B single-invocation workgroups scans at most $S = 256$ entries. It writes local offsets into Offset and one total per block into Block. Two stages (calls) of PrefixScanAdd are used to calculate global offsets.

The positive value A is the pessimistic output capacity planned by the caller using heuristic. HostAndDevice creates an A-record device output before count. If the actual pipeline overflows, then we return error.

Algorithm 8 Vulkan scheduling in BioseqDB

Require: packed query batch Q, k, w, device group limit $G_{\max}$

Require: positive planned output capacity A

Require: policy $P \in \{\text{HostAndDevice}, \text{DeviceOnly}\}$

$\mathcal{T} \leftarrow \text{PlanTiles}(Q, k, w, G_{\max})$
$T \leftarrow |\mathcal{T}|$
return without a GPU dispatch if $T = 0$
$G \leftarrow \min(\lceil T/64 \rceil, G_{\max}), \quad S \leftarrow 256, \quad B \leftarrow \lceil T/S \rceil$
PrepareBoundDeviceInputs$(Q, \mathcal{T})$
allocate device buffers $\underline{\text{Count}}[T]$, $\underline{\text{Offset}}[T]$, and $\underline{\text{Block}}[B]$
if $P = \text{HostAndDevice}$ then $\mathit{Output} \leftarrow \text{HostCreateDeviceOutput}(A)$
BeginCommandBuffer
RecordHostToGPUBarrier
$\mathit{countTask} \leftarrow \text{RecordOrderedSketchTile}(\text{Count}, Q, \mathcal{T}, k, w, G, \mathit{Count})$
AwaitGPU($\mathit{countTask}$)
$\mathit{localTask} \leftarrow \text{RecordPrefixScanBlock}(\mathit{Count}, \mathit{Offset}, \mathit{Block}, T, S, B)$
AwaitGPU($\mathit{localTask}$)
$\mathit{blockTask} \leftarrow \text{RecordPrefixScanAdd}(0, \mathit{Block}, B)$
AwaitGPU($\mathit{blockTask}$)
$\mathit{offsetTask} \leftarrow \text{RecordPrefixScanAdd}(1, \mathit{Offset}, \mathit{Block}, T, S)$
if $P = \text{DeviceOnly}$ then
    SubmitAndHostAwait($\mathit{offsetTask}$)
    $N_{\text{out}} \leftarrow \mathit{Offset}[T-1] + \mathit{Count}[T-1]$
    return overflow if $N_{\text{out}} > A$
    $\mathit{Output} \leftarrow \text{HostCreateDeviceOutput}(N_{\text{out}})$
    BeginCommandBuffer
    RecordHostToGPUBarrier
AwaitGPU($\mathit{offsetTask}$)
$\mathit{emitTask} \leftarrow \text{RecordOrderedSketchTile}(\text{Emit}, Q, \mathcal{T}, k, w, G, \mathit{Offset}, \mathit{Output})$
SubmitAndHostAwait($\mathit{emitTask}$)
if $P = \text{HostAndDevice}$ then
    $N_{\text{out}} \leftarrow \mathit{Offset}[T-1] + \mathit{Count}[T-1]$
    return overflow if $N_{\text{out}} > A$
    $\mathit{HostOutput} \leftarrow \text{CopyMapped}(\mathit{Output}[0 \ldots N_{\text{out}}))$
else
    $\mathit{HostOutput} \leftarrow \varnothing$
return $(\mathit{Output}, \mathit{HostOutput}, N_{\text{out}})$

Algorithm 9 `RecordOrderedSketchTile` call

1: procedure RecordOrderedSketchTile($g, mode$)

Require: global invocation ID $0 \le g < 64 \cdot G$

Require: dispatch-wide $mode \in \{\text{Count}, \text{Emit}\}$

Require: bound $Q, \mathcal{T}, T, k, w, G, Count$
from Algorithm 8

Require: bound Offset and Output in Emit mode

```
2:  j ← g
3:  while j < T do
4:      localCount ← 0
5:      for all record e streamed by RunScalarRescanTile(Q, T[j], k, w) do
6:          if mode = Emit then
7:              Output[Offset[j] + localCount] ← e
8:          localCount ← localCount + 1
9:      if mode = Count then
10:         Count[j] ← localCount
11:     j ← j + 64 · G
```

4.12. Packed minimizer representation

When running BioseqDB minimizer generation we need to have a way to store the results in a format that can be resused in other stages of the pipeline.

Minimap2's `mm_sketch` emits each minimizer in an `mm128_t`, two `uint64_t` words named `x` and `y`. The C++26 declaration below shows only interpretation of the fields, while minimap2's itself uses binary shifts and masks to set and extract specific values (which made its manual code analysis very challenging).

Listing 4.1: Logical fields packed into a minimap2 minimizer.

```
struct MinimapSketchFields {
  std::uint64_t kmer_hash : 56;   logical k-mer hash
  std::uint64_t span      :  8;   input-base span
  std::uint64_t strand    :  1;   canonical orientation bit
  std::uint64_t last_pos  : 31;   zero-based final-base position
  std::uint64_t rid       : 32;   reference/query-segment ID
};
```

Minimap2's C code construction is done via:

$$x = (hash \ll 8) \mid span, \tag{4.4}$$

$$y = (rid \ll 32) \mid (last_pos \ll 1) \mid strand. \tag{4.5}$$

Homopolymer compression

Normally span equals k (or is $< k$ for first $k-1$ bases). Minimap2 suppors homopolymer compression (HPC) togglable by command line option `-H` which collapses strings of one repeating character into single character occurence. If HPC is enabled, span can have values $0 \leq span \leq 255$. If span is increased beyond 255 (there are 255 same characters in the input sequence next to each other), then minimizer is completely abandoned until new symbol is found.

TODO

Find out if BioseqDB uses HPC.

The `rid` value identifies a reference during indexing and a query segment during query sketching. Note that the ID itself isn't useful during seeding, chaining or alignment, it's only useful at the end when reporting results.

When porting the core of minimap2 algorithms we observed the following facts:

1. The span contains no useful information as HPC is disabled by default and we don't have to support it at this time. For HPC disabled it can only carry information if the k-mer was at the start of the sequence (this can be checked via last_pos field)
2. rid is completely useless during most of the pipeline
3. last_pos which is the end position of the k-mer is only 31 bits which introduces requirement to split long sequences before using minimap2.
4. kmer_hash is wide, but using more bits for hash than binary representation of k-mer is not giving any advantage (for example for DNA4 alpohabet if we encode each symbol in 2 bits then k=15 requires 30 bits for k-mer representation. Hashing 30-bits into 56-bit wide hash doesn't give any advantage as huge part of hashing space won't be used)

BioSeqDB's minimizer producers and collectors use `minimizer_representation<L,K>`. Every form stores the hash, zero-based final position, and strand.

The longest form still uses span set to k, but it's not used for anything other than just padding of fields. Here L is the number of position bits and K is the k-mer length. This L is unrelated to the SIMD lane count used earlier in the chapter.

The type first reserves

$$H = \min(2K, 56)$$

hash bits. Let $R = H + L + 1$, where the last term is the strand bit. The type uses the first width $W \in \{16, 32, 64\}$ for which $R \leq W$. Every bit above the position and strand fields then belongs to the hash, so the stored hash field has $W - L - 1$ bits. If $R > 64$, the record uses two 64-bit words. One word contains a 56-bit hash and an 8-bit span. The other word contains the position, the strand bit, and any unused high bits.

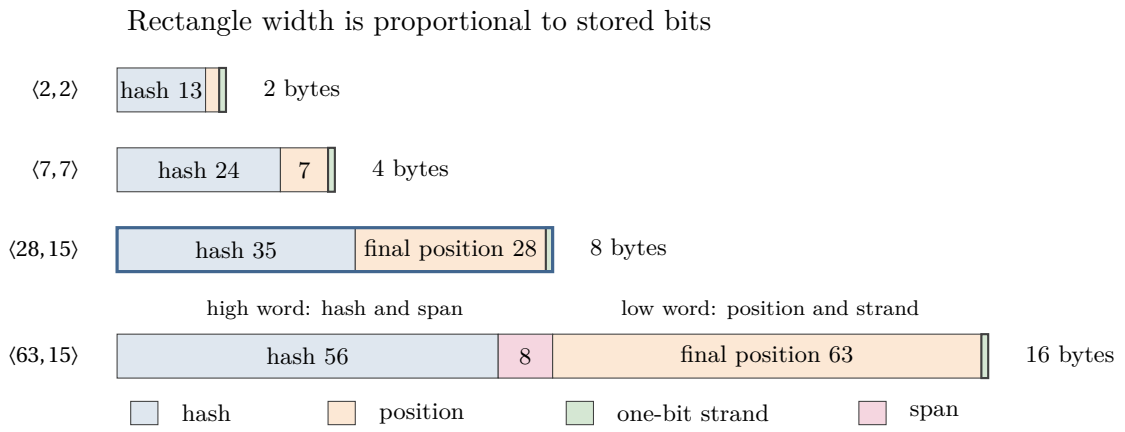


Figure 4.17: Bit fields in four `minimizer_representation<L,K>` instantiations, shown from high bits on the left to low bits on the right. Blue is the hash, orange is the zero-based final position, green is the one-bit strand, and purple is the span in the two-word form. The highlighted $\langle 28,15\rangle$ record is the default 64-bit sketch-data form.

The L-bit field represents final positions from zero through 2^L-1, so it can describe a local sequence span of at most 2^L bases. For $K=15$, values through $L=33$ still fit in 64 bits, while $L=34$ requires the two-word form. The $\langle 28,15\rangle$ form has a 35-bit hash field, but its minimap2 integer hash occupies only 30 bits.

Currently the main parameter that drives used representation width is the number of position bits L (this is compared against kmer_size_buckets array). Theoretically the template can produce a two-byte record, but this bucket isn't suppored at this moment (such small width make's it problematic to adjust as we need to have very short hash, which isn't ideal as shorter k-mers are more prone to collisions or limit the size of sequence to less than 256, which would be very specific optimisation) The $\langle 2,2\rangle$ on the figure is just the demonstrative example to show that template is supported even in very degenerated edge cases (we don't actually use special form for sequences with less than 4 bases as it would make very little sense).

4.13. Configured CPU algorithms

The PostgreSQL setting `bioseq.aligner.minimizer.algorithm` chooses the minimizer implementation. Its current default is `bounded_nthash_simd`. Table 4.4 maps the four CPU values to the algorithm families described earlier. Additionally we allow dispatching Vulkan kernel.

Setting value	Current CPU route
`scalar`	Scalar rescan with the minimap2 integer hash.
`simd_minimap2_reference`	A1 with the minimap2 integer hash for DNA4 and a registered rank-based SIMD route for DNA5.
`bounded_nthash_simd`	A2 with 32-bit ntHash for DNA4 and DNA5. A specialized $k=15, w=10$ kernel accompanies the generic runtime kernel.
`two_stack_nthash`	Normally A3 chronological SIMD with the minimap2 integer hash.

Table 4.4: CPU minimizer implementations exposed by the PostgreSQL setting.

Kernel selection

- Bounded ntHash accepts $1 \le k \le 31$ and $1 \le w \le 64$ within a compatible registered representation. Changing k, w does not automatically select scalar code.
- Older measurements can contain an explicitly recorded SIMD-reference-to-scalar fallback. Such a row describes that historical effective path, not the current DNA5 SIMD implementation.
- The Vulkan implementation uses the rescan approach described in Section 4.11; it is not G3SA's kernel.

In the bounded DNA5 kernel, `N` has its own registered ntHash seed and participates in rolling hashes like the other alphabet symbols. This differs from minimap2, which skips k-mers spanning an ambiguous base; DNA5 therefore changes seed semantics as well as packed storage.

4.13.1. Vector width and indexed queries

A wider vector does not by itself predict the speed of a complete query. The bounded kernel maintains eight logical streams: it uses two four-lane vectors on NEON and one eight-lane vector where that shape is available. This applies the instruction-level parallelism idea from Section 4.9, while the cost of each vector operation still depends on the processor. The two-stack resident route also has a dual-stream implementation, but indexed queries require its chronological continuation route. These are separate implementations, not a universal doubling of every SIMD stage.

Reusing a target index removes target sketch construction from a query. It does not remove sketching of the incoming query sequence. The selected minimizer scheme must agree on both sides, as explained in Section 11.4.3. Consequently, nearly equal indexed times can mean that query sketching contributes little to the total; they do not establish that the selected kernels are identical.

Position locality

In case of generating minimizers we can operate on local coordinates meaning that our minimizer representation can just track position relative to the sequence fragment. Sequence fragmentation in storage is described in later chapters. However for next steps of processing (chaining) we use global coordinates. It would be possible to take advantage of that locality for example by building index that stores only minimizers for some shorter sequence fragments (this is also described in later chapters), but for simplicity even in that case we just map local coordinates afterwards to global ones at all times. This have the cost of anchors being wider in memory (as we need to use int64 for global coordinates).

Chapter 5

Chaining seed anchors

5.1. From local seed hits to candidate mappings

When minimizers are generated and we take the intersection (by hash value) of the minimizer sets between the query and the reference, we get a set of seed hits which is encoded as anchors.

5.2. Anchor geometry and ordering

A seed hit is represented by an anchor

$$a_i = (r_i, q_i, \sigma_i, \ell_i, g_i, \phi_i),$$

where r_i and q_i are inclusive seed-end coordinates on the reference and query, σ_i is the strand, ℓ_i is the query span, g_i identifies a query segment, and ϕ_i stores additional ordering flags.

Below you can see the physical representation of the anchor in memory in minimap2 and in our implementation. We could potentially use adaptive representation similar to minimizers, but anchors are fewer than minimizers, so the benefit would be limited for additional complexity of C++ templates.

```
typedef struct {
  uint32_t n;                    Size of cr array (number of matches in the reference sequence)
  uint32_t q_pos;                Query position
  uint32_t q_span       : 31;    Query minimizer span in symbols
  uint32_t flt          : 1;     If the minimizer is filtered out (very frequently occuring in the refe
  uint32_t seg_id       : 31;    Query segment identifier
  uint32_t is_tandem    : 1;     Tandem repeat flag
  const uint64_t *cr;            Multiple reference coordinates per anchor
} mm_seed_t;
```

```
struct anchor {
  std::uint64_t ref_position;    Absolute reference coordinate
  std::uint64_t query_position;  Absolute query coordinate
  std::uint32_t is_reverse;      Reverse-strand flag encoded as zero or one
  std::uint8_t query_span;       Query minimizer span in symbols
  std::uint8_t flags;            Seed flags propagated by collection
  std::uint16_t segment_id;      Query segment identifier
};
```

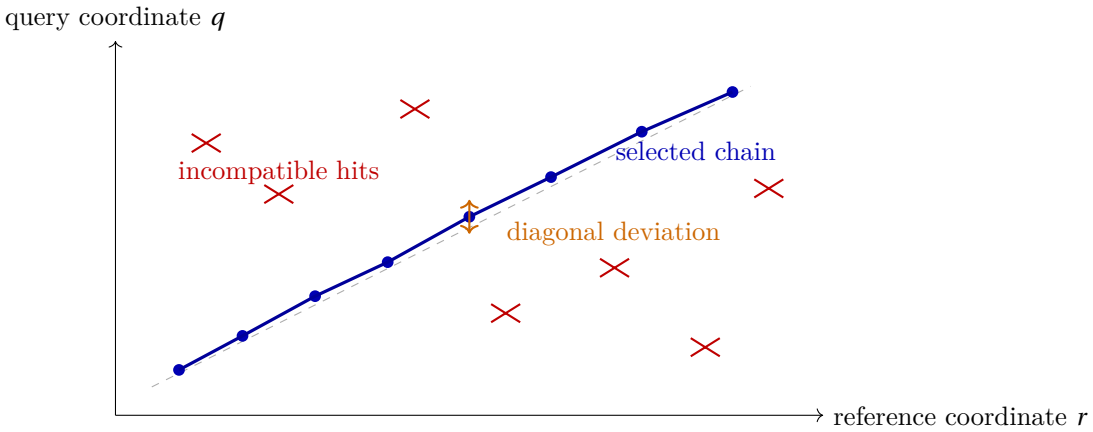


Figure 5.1: Chaining selects growing, approximately collinear path so we can get closer to the final result. Base-level alignment (later stage of the pipeline) will later resolve gaps between anchors.

TODO

Improve anchor API

Figure 5.1 shows viasual interpretation. Valid anchors that can be used for chaining lie below and to the left of the current anchor. A chain can move between nearby diagonals to represent an insertion or deletion, but large diagonal changes receive a penalty or are not considered at all.

Why we look for anchors that are colinear?

In the perfect scenario without gaps $r_i - q_i$ is constant. An increasing diagonal coordinate indicates that in reference we matched more symbols than in the query (meaning insertion/deletion has occured). Chaining needs to specify costs for abandoning the main diagonal too much (intuitevely we don't want the case in which we introduce very large gap in the reference sequence just to match one or two symbols in the query, because it's improbable that some very large fragment was replaced during evolutionary processes by just a few bases)

As we would like to have some well defined order of anchors, we sort them by strand and reference coordinate first, then by the additional metadata:

$$\begin{aligned} K_r(a_i) &= (\sigma_i \ll 63) \mid \big(r_i \,\&\, (2^{63}-1)\big), \\ K_q(a_i) &= (g_i \ll 48) \mid (\phi_i \ll 40) \mid (\ell_i \ll 32) \mid (q_i \bmod 2^{32}). \end{aligned} \tag{5.1}$$

Comparison is lexicographic in K_r, K_q, and the full q_i. The hashing doesn't need full 64 bits for query position i.e if query coordinates q_i exceed 2^{32} we still have correct order as the leftmostpart is still the fragment number g_i (fragments won't be larger than 2^{32}). Fragmentation of sequences is discussed later in the storage chapter.

The ordering is required, because the chaining wants to process anchors in sequential order (increasing query positions), while we build chains of anchors using reference positions.

5.3. Intuitive interpretation

We need to first understand how intuitively human would align sequences. First we choose a random fragment of query and use text search in our editor to find that exact fragment. The longer fragment is better, because it's less likely that we found it just because of random subsitutions ocurring. At first we won't find it, but we repeatadly delete chacters from the fragment start or end until match is found. Each match corresponds to possible location in the target sequence. If we repat the process enough for other fragments of query we will establih the most probable locations of query in the target sequence (by seeing groups of matches). Throughout this chapter we will never gave any reason to treat query and target asymetrically as scoring functions don't distinguish between both inputs. The distinction was rather coming from their role in the alignment flow (the larger set of sequences can be called the set of references or targets, while the smaller one could be called queries).

Longest increasing sequence

If the reference coordinates are increasing, then naive approach would be finding LIS (longest increasing subsequence) of query coordinates. You can think of the chaining as "improving" this simplistic approach by changing query coordinates to more complex scoring function which can handle gaps and other complex cases.

The sorting mentioned in section 5.2 is required, because what we would like to intuitively do is check where the first reference fragment can be placed in query sequence and when we move to the next anchor (increasing reference positon), the position of query should increase too (logically if the middle of query matches somewhere, then we should see the end of it, not the beggining). This introduces some emergent assymetry: it will be very likely to see one anchor in query matched with multiple anchors in referece (query can occur multiple times in reference), however what happens if one reference fragment is matched to multiple query fragments? This situation means that is hard to distinguish which part of the query is best to match:

- Reference $\leftarrow XYZVW$, Query $\leftarrow YZV$ - it's immediately obvious that for V in the reference we found the end of query
- Reference $\leftarrow XYZVW$, Query $\leftarrow VXVXVXV$ - we don't know what part (is this start or the end of query, or the middle part?) we should consider without looking at neighbours of V.

Emergent assymetry

In the following chapters I will present new optimisations that are not discussed in the field of computational biology. The emergent assymetry will play important role when talking about the experimental results in separate chapter 5.9.

5.4. Chaining components

We defined the anchors that are input to chaining pipeline stage, however chaining itself isn't a monolith. It can be viewed as composition of several steps.

Chaining pipeline

Although chaining can be decomposed into several steps that are dispatched to differrent functions, in BioseqDB this is an internal implementation detail and we don't allow users to alter for example which backtracking backend is used (although we could expose parts of chaining as composable components in the future)

Forward chaining (which is the core component) produces the score array f and predecessor array p. Second most important step is backtracking (as typically in case of DP algorithms which fill some state matrices) reads the sorted anchors, f, and p. The backtracking writes theflat output anchor array supplied by the caller, denoted by A (A can be viewed as type `anchor A`

).

The chaining can be fully decomposed into the following steps:

1. Forward chaining reads the sorted anchors and writes the temporary f and p arrays.
2. Backtracking reads these inputs and writes the accepted chain records plus full anchor copies in A.
3. Region generation converts each chain record into one alignment region while retaining A.
4. Region selection filters the regions while retaining A.
5. Base-level alignment reads the selected regions, their slices of A, the query and target sequences, and the alignment parameters. It fills the base-level fields in the regions supplying exact start/end coordinates, match qualities etc.

BioseqDB represents every chain record with the following structure:

```
struct chain {
  int32_t score        = 0;   Accepted-chain score
  int32_t anchor_count = 0;   Anchors in the chain
  int32_t anchor_start = 0;   Start of the chain slice in A
};
```

Where `anchor_start` is the index of the first anchor in the chain in the array A.

For a non-empty record c, its anchors occupy consecutive positions in forward order:

$$A[c.\texttt{anchor_start}], \ldots, A[c.\texttt{anchor_start} + c.\texttt{anchor_count} - 1].$$

The array A is constructed using f and p arrays. For every anchor i, the two stored values mean:

f_i the score of the best chain that the selected forward algorithm found with anchor i at its end.

p_i the index of the anchor immediately before i in that chain. The value is -1 when the best chain found for i starts at i

For example,

$$p_7 = 5, \qquad p_5 = 2, \qquad p_2 = -1 \quad \Longrightarrow \quad 7 \to 5 \to 2 \to -1.$$

5.5. Chaining algorithms

Similar to the minimizer chapter here we will discuss different approaches to chaining. The algorithm itself is based on dynamic programming scoring, but the details differ.

Table 5.1: Comparison of different chaining algorithms

ID	Algorithm	Defining decision	Implementations
B1	Ordered predecessor scan	Visit recent candidates first. Increase the skip counter when a marked candidate does not improve the score. Decrease a positive counter after an improvement. Stop when the counter exceeds its limit. Also score cached predecessor m when it exists and the main scan does not reach it	minimap2 non-RMQ chaining, BioSeqDB scalar and exact SIMD
B2	Source-to-target DP	Apply each completed source score in parallel to later targets admitted by the kernel's index and distance checks	G3SA first CUDA pass
B3	Ranking-value second pass	Rank predecessors by a value calculated from their saved scores and coordinates. Keep at most one predecessor with the largest accepted ranking value. If one was saved, calculate its full pair score	G3SA second CUDA pass
B4	Range-minimum-query tree DP	Use a balanced tree whose nodes cache the smallest stored priority in each subtree, then calculate the full score for the selected predecessor	minimap2 RMQ and BioSeqDB's experimental RMQ route, which includes both query-coordinate boundaries and omits the bandwidth check
B5	Per-element batch-limited DP	Keep one running score for each vector element. Stop after enough consecutive batches leave all of those scores unchanged	BioSeqDB bounded CPU SIMD
B6	Per-target batch-limited DP	Keep one best score for the target. Stop after enough consecutive batches fail to increase that score	BioSeqDB Vulkan

Scalar and exact SIMD both implement B1 because they make the same ordered decisions. G3SA selects a short kernel or one of two tiled launch configurations according to range size within B2 and B3. BioSeqDB CPU SIMD and Vulkan use different batch stopping rules, so they are B5 and B6.

All implementations pass their saved scores and predecessors to later code that builds and classifies mappings.

The BioSeqDB RMQ implementation includes both query-coordinate boundaries. It can select a predecessor with the same query coordinate as the target and accept it without a bandwidth test. We classify the tree design as B4 and describe these defects in Section 5.13.

5.5.1. Shared pair score

Let's say we have two anchors with coordinates q_j, q_i and r_j, r_i on the query and reference. Let's assume the minimizer span is always k. We now can say that after incorporating (r_j, r_i) anchor we will cover $q_i - q_j$ symbols on the query and $r_i - r_j$ symbols on the reference, however those two jumps can be interpreted as taking some symbols from query and assuming they form replacement and the rest forms a gap. For example let's say $q_i - q_j > r_i - r_j$. This means

that we match more symbols in query than in reference, so indicates some deletion in the reference. So we can say $r_i - r_j$ symbols are replaced (or stay the same) while the leftover $(q_i - q_j) - (r_i - r_j)$ symbols form this deletion. This thinking is symmetrical so we just only need to use minimum and absolute value to make it more generic.

Now we introduced $g(j,i)$ (the advance shared by the reference and query) and $\delta(j,i)$ (the remaining advance in whichever sequence moves farther). As k-mers represent hashed fragment of length k we can assume that in our match we have exactly $\min\{q_i - q_j, r_i - r_j, k\}$ new symbols that matched. Hence we now define our first primitive score as $\min\{q_i - q_j, r_i - r_j, k\} = \min\{g(j,i), k\}$.

This scoring has some problems with the most important one being that it doesn't penalize the case when we have a large gap in the reference sequence just to match one or two symbols in the query. This is improbable that some very large fragment was replaced during evolutionary processes by just a few bases. For that reason we will use function $\lfloor \text{Penalty}(\delta, g) \rfloor$. The penalty function should satisfy the following properties:

- There's no penalty for doing nothing at all: $\text{Penalty}(0,0) = 0$
- For $g \le k \wedge \delta = 0$ there's no gap so there's shouldn't be any penalty: $\text{Penalty}(0,g) = 0$
- Penalty should be monotonically increasing with respect to δ: $\text{Penalty}(\delta, g) \le \text{Penalty}(\delta + 1, g)$
- Penalty should be increasing slower than linearly with respect to g: $\text{Penalty}(\delta, g+1) < \text{Penalty}(\delta, g) + 1$. This guarantees that increasing g without touching δ benefits overall score (we prefer replacement region over gap region)
- For small g and δ our penalty function should prefer increase in g rather than δ. This is equivalent of saying that we want to stay close to the diagonal and reduce the gap size introduced: $\text{Penalty}(\delta + 1, g) \ge \text{Penalty}(\delta, g + 1)$
- For large g and δ our penalty function should be asymptotically the same for both δ and g. This is equivalent of saying that if we encounter very large gaps, then it doesn't matter if we increase the matching region between two fragments or increase the gap as both fragments are very dissimilar: $O_\delta(\text{Penalty}(\delta, g = \text{const})) = O_g(\text{Penalty}(\delta = \text{const}, g))$

Symmetry

Using g and δ as only parameters for penalty function guarantees that scoring is symmetric. Symmetry makes sure that gap costs are the same for both query and reference.

Our final score is then something of the following structure:

$$\begin{aligned} \text{Score}(j,i) &= \min\{g(j,i), k\} - \lfloor \text{Penalty}(\delta(j,i), g(j,i)) \rfloor \\ &= \min\{g(j,i), k\} - \lfloor \alpha * g(j,i) + \beta * \delta(j,i) + \gamma * f(\delta(j,i)) \rfloor \\ &\qquad 0 \le \alpha, \beta, \gamma \le 1 \end{aligned} \tag{5.2}$$

The minimap2 scoring definition E looks exactly as what we have just specified:

$$\begin{aligned} d_q(j,i) &= q_i - q_j, & d_r(j,i) &= r_i - r_j, \\ \delta(j,i) &= |d_r(j,i) - d_q(j,i)|, & g(j,i) &= \min\{d_r(j,i), d_q(j,i)\}. \end{aligned} \tag{5.3}$$

$$E(j,i) = \min\{\ell_j, g(j,i)\} - P(j,i), \tag{5.4}$$

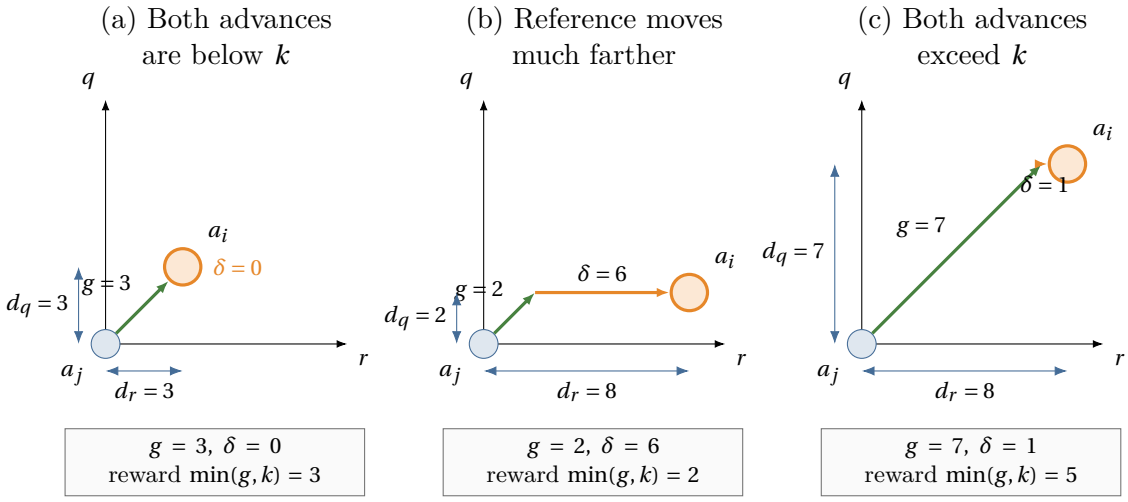


Figure 5.2: Three anchor displacements for $k = 5$. The blue brackets are the actual reference and query advances. The green diagonal step advances both coordinates by $g = \min(d_r, d_q)$. The orange horizontal step is the remaining $\delta = |d_r - d_q|$. It would be vertical when $d_q > d_r$. Panel (c) also shows that the reward stops at k, even though $g > k$.

where

$$P(j,i) = \begin{cases} 0, & \delta(j,i) = 0 \ \wedge \ g(j,i) \le \ell_j, \\ \left\lfloor \lambda_g \delta(j,i) + \lambda_s g(j,i) + \frac{1}{2}\,\mathrm{mglog}_2\big(\delta(j,i)+1\big) \right\rfloor, & \text{otherwise.} \end{cases} \tag{5.5}$$

The function mglog_2 is minimap2's polynomial approximation of $\log_2$. So far in the abstract definition of scoring we didn't mention diagonal, but nonnegative coefficient λ_g can be viewed as diagonal-gap penalty (further away from diagonal we penalize more), where the nonnegative coefficient λ_s is the penalty for distance. The value ℓ_j is the query span of predecessor j. (In our example above we used k for all k-mers sizes, but as mentioned before minimap2 with specific settings can compress regions of same repeating symbols so we just replace k with kmer span ℓ_j. All the logic stays the same)

Minimap2 chaining link score
comput_sc 113

Score units

In this definition we view scoring function as expressed in number of matched symbols (not some arbitrary large numbers), so the coefficients must be set accordingly so the values are scaled correctly. In the dynamic programming formulation we then can take maximum of E value and ℓ_i to make sure that in the worst case scenario we assume that we at least matched the length of the minimizer. There are no formal limitations on this normalization, but it makes reasoning about the scores easier.

It's worth noting that defined scoring is also symmetrical in terms of coordinates i.e the score doesn't take r_i, q_i positions as parameters so reversing coordinate systems doesn't change the outcome. However, in dynamic programming we often want to use the locally close predecessor as the best one.

We see that scoring is not easy to work with as it depends on both anchors. If we used a linear gap scoring similar to what we discussed in alignment foundations chapter 2.1 we

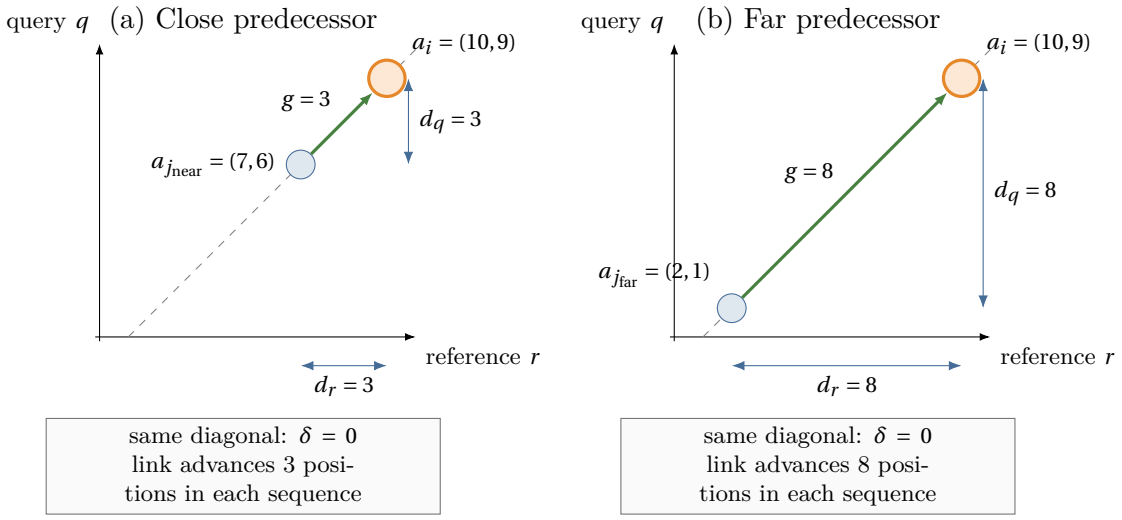


Figure 5.3: A close and a far predecessor for the same current anchor $a_i = (10,9)$. Both candidates lie on the diagonal $r - q = 1$, so both have $\delta = 0$. Their link lengths are nevertheless different: the close link has $g = 3$, whereas the far link has $g = 8$. Thus zero diagonal deviation does not mean zero or equal distance.

would get a score that is not dependent on both anchors. Note that the following is true due to trivial relationship between arithmetic average, distance and minimum:

$$\begin{aligned} \phi(j,i) &= \frac{d_q(j,i) + d_r(j,i)}{2} \\ \delta(j,i) + 2 * g(j,i) &= \phi(j,i) \end{aligned} \tag{5.6}$$

So we can simplify our scoring function to be only dependent on $\phi(j,i)$ (I replaced $min(\ell_j, g(j,i))$ with just ℓ_j and replaced penalty function with a simple linear function dependent on $\phi(j,i)$):

$$E(j,i) = \ell_j - \lambda_\phi * \phi(j,i) \tag{5.7}$$

Now if we assume that every anchor has constant span (as most of them will have length k), then we can define:

$$\begin{aligned} f_j &= \max_{i<j}\{f_i + E(j,i)\}, \\ \text{GlobalScore}(i) &= f_i + \lambda_\phi * (q_i + r_i) \end{aligned} \tag{5.8}$$

We can observe that:

$$\begin{aligned} f_i + E(j,i) > f_k + E(k,i) &\iff \\ f_i + \ell_j - \lambda_\phi * \phi(j,i) > f_k + \ell_k - \lambda_\phi * \phi(k,i) &\iff \\ \text{GlobalScore}(i) + \ell_j > \text{GlobalScore}(k) + \ell_k &\iff \\ \text{GlobalScore}(i) > \text{GlobalScore}(k) & \end{aligned} \tag{5.9}$$

This means that we can compute GlobalScore(i) independently for each anchor and use it to calculate global score. This is what was done in minimap2 as described in minigraph paper Li, Feng, and Chu, "The design and construction of reference pangenome graphs with minigraph." The problem is that the new linear gap scoring is very simplistic and does not exhibit some of the properties I discussed earlier.

5.6. B1: minimap2 dynamic programming

As our scoring model $E(i,j)$ depends only on two anchors we can define recursive equations for dynamic programming:

$$f_i = \max\left(\ell_i, \max_{j\in\mathscr{C}_i}\{f_j + E(j,i)\}\right), \qquad p_i = \arg\max_{j\in\mathscr{C}_i}\{f_j + E(j,i)\}. \tag{5.10}$$

Where $\mathscr{C}_i$ is the set of valid predecessors (selected by being close to diagonal - this is form of specifying cutoff condition so we don't need to check all possible predecessors all the time). Maximum function is used here to make sure that in worst case if our definition of $E(i,j)$ yield very low score we can say that we matched at least length of the minimizer span.

If no predecessor improves the base value of score ℓ_i, then $p_i = -1$.

We will call iteration over $\mathscr{C}_i$ a scan. Minimap2 does this scan by visiting anchors in order from newest to oldest.

After sorting, minimap2 numbers the anchors $0,1,\ldots,n-1$. When the DP algorithm reaches anchor i, the scores $f_0,\ldots,f_{i-1}$ and predecessor indices $p_0,\ldots,p_{i-1}$ have already been calculated. It starts a new chain at i with

$$f_i = \ell_i \qquad \text{and} \qquad p_i = -1,$$

Minimap2 tests earlier anchors in the order $j = i-1, i-2,\ldots$ (larger index is tested first). For every pair that passes the distance checks, it calculates

$$c_j = f_j + E(j,i).$$

If $c_j > f_i$, anchor j becomes the new predecessor: $f_i \leftarrow c_j$ and $p_i \leftarrow j$ (strict equality is important here so we avoid using older anchors with the same score)

Trying every earlier anchor would make this dynamic program expensive. Minimap2 applies the following strategies to limit the work performed:

- `max_iter` limits how many previous anchors we scan: this is the number of highest-numbered array positions before i.
- `max_skip` is the limit on skip counter s. It starts at zero for every new i and is increased by one on marked anchor that fails to improve the score and decreased by one on marked anchor that improves the score. If skip counter exceeds limit we finish scan loop early and do not scan other previous anchors.
- `m` is an extra candidate that is tested after the scan loop. It is the anchor with the highest score among all previously tested anchors (this is a measure to prevent skipping good chains due to too aggressive skip counter limit)
- `CanScore(j,i)` checks the anchor distances to make sure the anchor lies within the band (close to diagonal). It uses D_x and D_y for limits on reference and query distances respectively and B for distance from diagonal.

Minimap2 contains separate rules for multi-segment and cDNA input. The key algorithm stays the same and differences are very subtle:

- There are differences in handling scoring case of $g_i = g_j$
- Multi-segment non-cDNA input adds special requirement $d_r \le D_y$

- cDNA input does not force $D_y \geq B$
- There are differences in penatly factors for scoring function $E(j, i)$

In description below I assume we talk about default case of one-segment, non-cDNA input.

Different kinds of input

Multi-segment input is in case of Minimap2 described as a situation in which single query maps to different fragments of the reference section. I do not support this use case in BioseqDB out of the box at this time. This situation is also known as split-read or supplementary alignment and normally occurs due to due to structural variations, translocations, or chimeric artifacts in the genome. cDNA in this case describes the same situation, but has slightly different scoring parameters. Even though that can be interesting, we don't want to describe these modes here since those are very specialized heuristics

5.6.1. Skipping with counters and marked anchors

As discussed above minimap2 scores only valid pairs of anchors:

$$0 < d_r(j, i) \leq D_x, \qquad 0 < d_q(j, i) \leq \min(D_x, D_y), \qquad \delta(j, i) \leq B, \tag{5.11}$$

where $B = \texttt{bw}$ where all parameters like `bw` are minimap2's CLI arguments.

$$D_x = \max(\texttt{max_dist_x}, B), \qquad D_y = \max(\texttt{max_dist_y}, B).$$

After a candidate j passes this checks, we calculate $E(i, j)$ score and then update p_j if we see better score. We want to that p_j was updated in scan for i. This can be understood as building chains in parallel where we want to save information about predecessor for each anchor:

$$t_{p_j} \leftarrow i \qquad \text{when } p_j \geq 0.$$

If some time later in other scan we see that $t_{p_j} = i$ then we know that p_j was updated in scan for i (we already tested candidate with a larger anchor). If marked anchor x fails to improve f_i, then s increases by one. A failed unmarked anchor leaves s unchanged. An improvement subtracts one from a positive s. It does not necessarily reset the counter to zero. The loop stops only when $s > \texttt{max_skip}$

Interpretation

This marking with skip counter can be intuitevely understood as a situation in which we try to improve the best chain by adding new anchor. If repeatedly this chain fails to improve the score, we don't want to waste more time on it.

We don't perform this marking when this is candidate thatraises s above `max_skip` (in that case we stop the scan loop early).

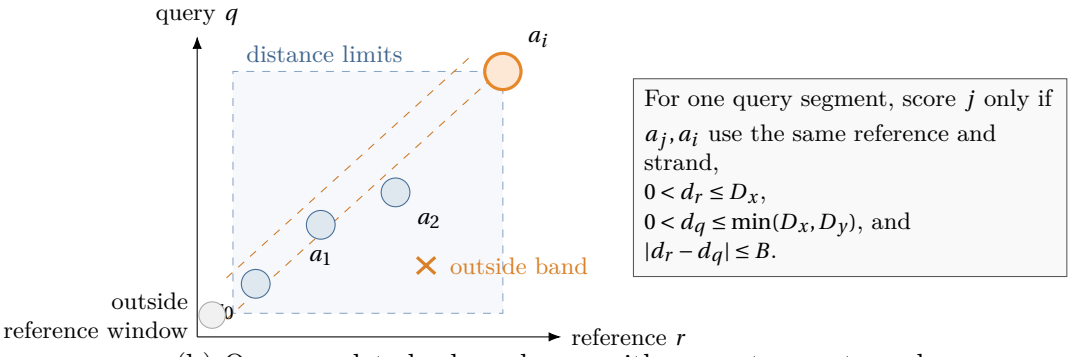
Minimap2 keeps the loop index in e. If the skip test stopped the loop while it was processing j, then $e = j$. e is used to check if we can score anchor m (if $m < e$ then we can).

m is additional index saved by minimap2 on the side, which we call the saved high-score anchor. The saved value is an anchor index whose previously calculated f_m is large. If m

is absent or has left the reference-distance window (is too far), minimap2 first rebuilds it by finding the largest f_j in the current `max_iter` set.

This saved anchor can then remain saved after $m < i - \texttt{max_iter}$. Each time we check $e < m$ (see if we already scored this anchor) and if not we calculate $E(m,i)$ and do update checks similar to other anchors. m is introduced to make sure to take into consideration anchor that could be potentially a little bit further away. Minimap2 doesn't use priority queue or list here, because chaining accuracy is already good enough and this approach is much simpler.

(a) Which earlier anchors can be scored



(b) One complete backward scan with concrete counter values

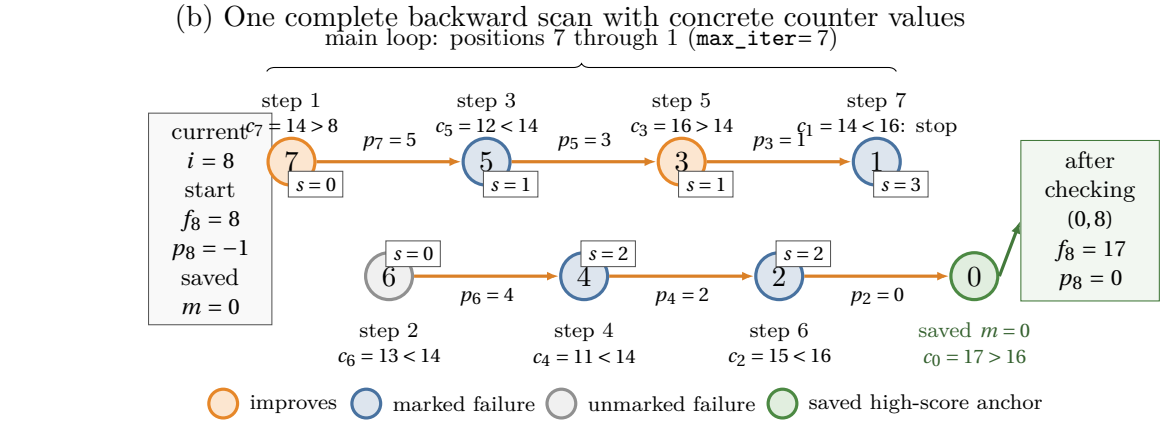


Figure 5.4: (a) The reference and query limits and diagonal band decide whether a pair can be scored or not. (b) If distance checks pass, then we perform scans in a loop. The picture shows one pass of the scan for the current anchor $i = 8$. The numbers inside the circles are anchor indices. The orange arrow from j to p_j shows the predecessor previously saved for anchor j. After j is scored without stopping, the arrow's destination receives the current mark $i = 8$. The numbered steps visit anchors $7, 6, \ldots, 1$. The small s boxes show the single skip counter for current anchor 8. Step 2 doesn't affect s as this anchor doesn't have a predecessor in this scan pass (it's unmarked). Step 5 improves the score and changes $s = 2$ to $s = 1$. Anchor 0 is already outside the main loop because $\texttt{max_iter} = 7$, so we don't scan it, but we check it later with $m = 0$. Step 7 stops with $e = 1$, so saved $m = 0$ satisfies $m < e$ and is checked once. The score of $m = 0$ equal to 17 wins, but in some other scan other anchor could have a better score.

Algorithm 10 Minimap2's ordered predecessor scan for one query segment

Require: sorted anchors $a_0, \ldots, a_{n-1}$; ordinary non-cDNA mode; configured distance limits, $B = \texttt{bw}$, `max_iter`, and `max_skip`
Ensure: chain scores f_i and predecessor indices p_i

$D_x \leftarrow \max(\texttt{max_dist_x}, B)$
$D_y \leftarrow \max(\texttt{max_dist_y}, B)$
function CanScore(j, i)
    $d_q \leftarrow q_i - q_j, \quad d_r \leftarrow r_i - r_j$
    if a_j, a_i use different references or strands then
        return false
    if $d_r \le 0$ or $d_r > D_x$ then
        return false
    if $d_q \le 0$ or $d_q > D_x$ or $d_q > D_y$ then
        return false
    if $|d_r - d_q| > B$ then
        return false
    return true
$t_j \leftarrow 0$ for every j; $m \leftarrow -1$; $st \leftarrow 0$ m is one saved anchor index
for $i \leftarrow 0$ to $n-1$ do
    while $st < i$ and (a_{st} has a different reference or strand from a_i, or $r_i - r_{st} > D_x$) do
        $st \leftarrow st + 1$
    $st \leftarrow \max(st, i - \texttt{max_iter})$ hard lower bound for the main scan
    $f_i \leftarrow \ell_i, \quad p_i \leftarrow -1, \quad s \leftarrow 0, \quad e \leftarrow st - 1$
    for $j \leftarrow i-1$ down to st do
        if CanScore(j, i) then
            $c \leftarrow f_j + E(j, i)$
            if $c > f_i$ then
                $f_i \leftarrow c, \quad p_i \leftarrow j$
                if $s > 0$ then
                    $s \leftarrow s - 1$
            else if $t_j = i$ then
                $s \leftarrow s + 1$
                if $s > \texttt{max_skip}$ then
                    $e \leftarrow j$
                    break
            if $p_j \ge 0$ then
                $t_{p_j} \leftarrow i$ mark one older anchor for this scan
    if $m < 0$, or a_m has a different reference or strand from a_i, or $r_i - r_m > D_x$ then
        $m \leftarrow -1$
        for $j \leftarrow i-1$ down to st do
            if $m < 0$ or $f_j > f_m$ then
                $m \leftarrow j$
    if $m \ge 0$ and $m < e$ then
        if CanScore(m, i) then
            $c \leftarrow f_m + E(m, i)$
            if $c > f_i$ then
                $f_i \leftarrow c, \quad p_i \leftarrow m$
    if $m < 0$, or (a_m, a_i use the same reference and strand, $r_i - r_m \le D_x$, and $f_i > f_m$) then
        $m \leftarrow i$

Algorithm details

We check if a_{st} has a different reference or strand from a_i, because anchors in minimap2 can originate from different refences. In BioseqDB this won't happen, but we still need to check for strand direction. Also please note that formally we use different values for marking $t_{p_j} \leftarrow i$, but it's main goal is just to detect visited anchors for the same scan pass (like in binary map, but assigning i just means that we don't have to clear it.)

5.7. BioseqDB SIMD DP: exact and bounded batch

For one target anchor a_i, scalar B1 tests possible predecessors one at a time. During the sacan it starts at a_{i-1} and walks towards the lower end of the search window. SIMD implementations both implement the same outer loop, but the scan itself is rewritten: We place each predecessor candidate in batches of size L (each batch is one SIMD vector) - one vector call can calculate as many as L values of $E(j,i)$. We have two kinds of SIMD algorithm:

1. Exact SIMD B1 called `exact_visited` It it the most naive implementation: it calculates up to L pair scores together in a SIMD vector, but F, P, and s are still single scalars as in original B1.

2. Bounded batch SIMD B1 called `bounded_batch_approximate`. Algorithm removes the mark array completely, while scalar skip counter in now counting batches. Lane λ keeps its own temporary best score H_λ and predecessor P_λ. A shared counter u records how many batches in a row failed to improve any lane (the algorithm stops when u reaches its batch limit U). We take the largest score across the lane winners and test the saved anchor m once.

We will discuss bounded batch SIMD B1 first, as it is more interesting one.

5.7.1. Candidates in SIMD vectors

The candidates are placed in batches. So now in loops we use $b = 0$ instead of j where 0 is the first batch, $b = 1$ is the next older batch, and so on. Within one batch, $\lambda \in \{0, \ldots, L-1\}$ is the SIMD lane number. The candidate placed in that lane is $j_{b,\lambda}$.

First we introduce the most boring thing - coordinates mapping:

$$\begin{aligned} \mathrm{end}_b &= i - bL, & \mathrm{begin}_b &= \max(st_i, \mathrm{end}_b - L), \\ a_b &= \mathrm{end}_b - \mathrm{begin}_b, & j_{b,\lambda} &= \mathrm{begin}_b + \lambda. \end{aligned} \tag{5.12}$$

Here end_b is the position immediately after the batch, begin_b is its first candidate, and a_b is its number of active lanes. As candidates are split into batches, the last one can be partial (so a_b will be used to mask out the inactive lanes).

For lane λ, the pair scorer first calculates $\epsilon_{b,\lambda} = E(j_{b,\lambda}, i)$. The code then loads the earlier chain score $f_{j_{b,\lambda}}$ and adds it to $\epsilon_{b,\lambda}$. The complete proposed chain score is $C_{b,\lambda}$, and $\mathbf{C}_b$ is the vector containing one such total per lane. A rejected candidate or an inactive lane receives $-\infty$.

Figure 5.5 depicts that now instead of single candidate being checked, we check batches:

- $C_{b,\lambda}$ holds total score for the candidates which corresponds to c in original B1 algorithm

- $\mathbf{j}_b$ holds position of the candidate which corresponds to j in original B1 algorithm
- $\mathbf{H}^{(b)}$ holds the best score for the batch which corresponds to f_i in original B1 algorithm (note that now we hold vector instead of scalar, so at the end we need to select minimum value)
- $\mathbf{P}^{(b)}$ is shown later on the next figure and holds the index for $\mathbf{H}^{(b)}$

Note that only $\mathbf{H}^{(b)}$ and $\mathbf{P}^{(b)}$ are presisted between scans. The superscript (-1) over H means that we got this value from previous scan.

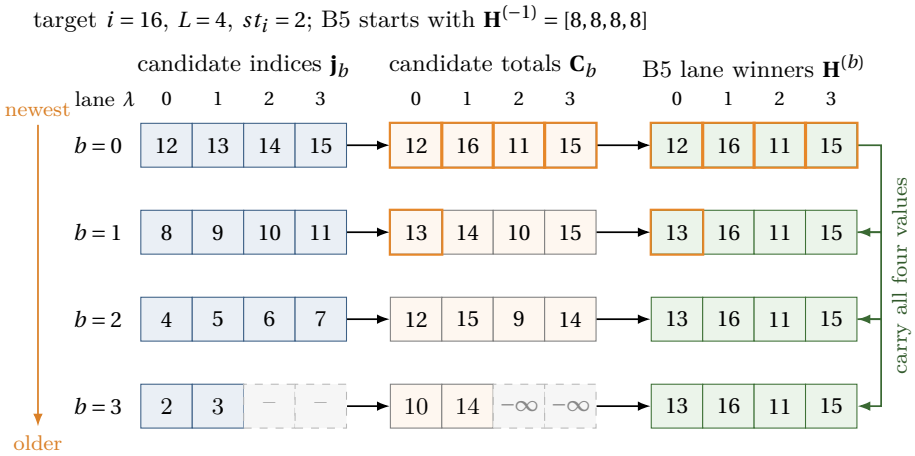


Figure 5.5: How predecessor candidates are placed in four SIMD lanes. The lane number λ selects the corresponding column in all three vectors. Blue boxes are candidate indices, orange boxes are calculated scores, and green boxes are best scores (the scores are improved vertically). When a lane improves its new maximum has an orange outline. Batch 3 is partial (we exhausted all candidates), so its two dashed lanes are inactive and their totals are $-\infty$.

We operate on batches so we need to adjust our definition of skip counter. Let $S =$ `max_chain_skip`:

$$U = \max\left\{1, \left\lfloor \frac{S}{L} \right\rfloor + 1\right\} \tag{5.13}$$

As a helper we introduce $I_{b,\lambda}$ which is just binary mask checking lane-wise which C value is better than H, so we update skip counter according to the following rules:

$$u_b = \begin{cases} 0, & \text{if any } I_{b,\lambda} \text{ is true,} \\ u_{b-1} + 1, & \text{otherwise.} \end{cases} \tag{5.14}$$

When $u_b \geq U$ the algorithm stops. In original B1 we had guarantee that we will stop after S skipped candidates, but here we stop after U such that: $S < LU \leq S + L$. This doesn't however bother us at all, as this is only very minor implementation detail user doesn't need to see.

The figure below depcits two kinds of SIMD B1 algorithms. The first one (a) wasn't discussed in detail as it only puts the candidates in a single SIMD vector to speed up only few operations like finding the maximum, where the rest of the code stays unchanged. The second parts of figure (b) is more interesting as it shows the approach we discussed with vector batching.

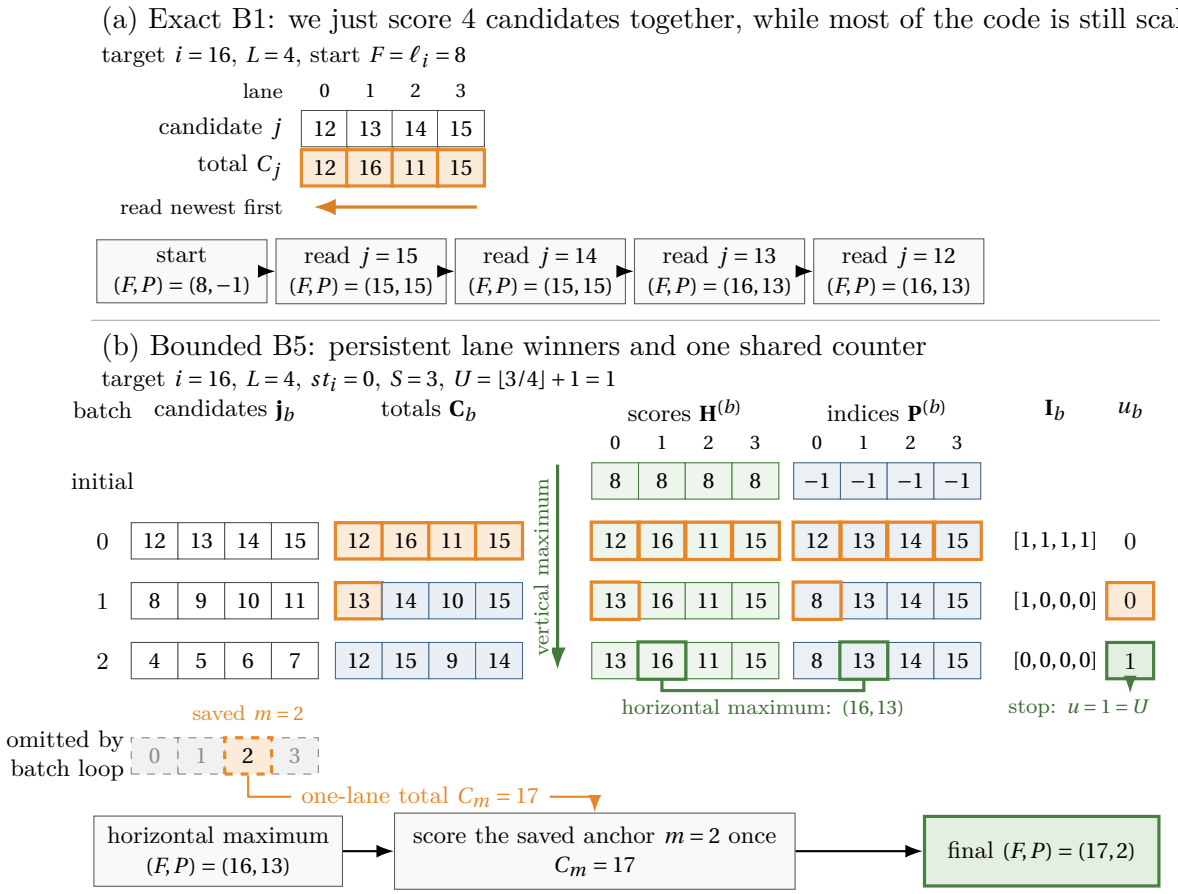


Figure 5.6: Two strategies for calculating SIMD B1 (a) Exact B1 calculates four totals together, then applies state changes in the scalar order whenever some lane can improve. (b) Bounded SIMD is more clever - it starts with $\mathbf{H}^{(-1)} = [8,8,8,8]$ and $\mathbf{P}^{(-1)} = [-1,-1,-1,-1]$. Each row shows both vectors after its batch. Green boxes hold scores and blue boxes hold their predecessor indices. An orange border around C marks a new improvement (maximum). Orange borders around H and P show the stored pair that changed. All four lanes change in batch 0. In batch 1, only lane zero changes, to $(H_0, P_0) = (13, 8)$. Batch 2 changes neither vector, so $u = 1 = U$ and candidates 0 through 3 (bottom gray area) are omitted. The downward arrow shows that each lane keeps its best (H, P) pair while the scan moves through older batches. The horizontal maximum then compares the four lane scores. Lane 1 supplies the pair selected by that horizontal maximum, $(H_1^{(2)}, P_1^{(2)}) = (16, 13)$, marked by the green bracket and outlines. The orange box marks saved candidate $m = 2$, whose separate one-lane total $C_m = 17$ replaces the result with $(17, 2)$.

After the last scored batch, the algorithm takes horizontal maximum of a vector once more:

$$f_i = \max_\lambda H_\lambda, \qquad p_i = \max\{P_\lambda : H_\lambda = f_i\}. \tag{5.15}$$

The algorithm is described in detail in Algorithm 11.

Algorithm 11 Bounded SIMD B1

Require: target i, window $[st_i, i)$, SIMD width L, $S = \texttt{max_chain_skip} > 0$, $\texttt{max_chain_iter} > 0$, saved index m

Ensure: f_i, p_i

$U \leftarrow \max\{1, \lfloor S/L \rfloor + 1\}$ whole-batch stopping limit

$\mathbf{H} \leftarrow [\ell_i]^L, \quad \mathbf{P} \leftarrow [-1]^L, \quad u \leftarrow 0$ one temporary winner per lane

$o \leftarrow st_i$ no omitted interval yet: $[st_i, o)$ is empty

for $b \leftarrow 0, 1, \ldots$ while $\mathbf{end}_b > st_i$ do

  calculate $\mathbf{begin}_b, \mathbf{end}_b, a_b, \mathbf{j}_b$ using Equation 5.12

  $\mathbf{A} \leftarrow [\lambda < a_b]_{\lambda=0}^{L-1}$ mask unused lanes in the oldest partial batch

  $\boldsymbol{\epsilon} \leftarrow \text{PairScoreBatch}(\mathbf{j}_b, i, \mathbf{A})$ $E(j, i)$, or $-\infty$ for a rejected pair

  $\mathbf{C} \leftarrow \text{Select}(\boldsymbol{\epsilon} \neq -\infty, \mathbf{f}[\mathbf{j}_b] + \boldsymbol{\epsilon}, -\infty)$ total candidate scores

  $\mathbf{I} \leftarrow (\mathbf{C} > \mathbf{H})$ strict lane-wise improvements

  $\mathbf{H} \leftarrow \text{Select}(\mathbf{I}, \mathbf{C}, \mathbf{H})$

  $\mathbf{P} \leftarrow \text{Select}(\mathbf{I}, \mathbf{j}_b, \mathbf{P})$ equal older candidates do not replace

  if $\text{Any}(\mathbf{I})$ then

    $u \leftarrow 0$

  else

    $u \leftarrow u + 1$

  if $u \geq U$ and $\mathbf{begin}_b > st_i$ then

    $o \leftarrow \mathbf{begin}_b$ batch loop omits $[st_i, o)$

    break

$f_i \leftarrow \text{HorizontalMax}(\mathbf{H})$

$\mathbf{T} \leftarrow \text{Select}(\mathbf{H} = f_i, \mathbf{P}, -1)$

$p_i \leftarrow \text{HorizontalMax}(\mathbf{T})$ largest predecessor index wins a score tie

if $f_i = \ell_i$ then

  $p_i \leftarrow -1$

if $m < st_i$ or m is outside the current reference and strand window then

  $m \leftarrow \arg\max_{st_i \leq j < i}(f_j, j)$, or -1 if this interval is empty compare score first, then index

if $st_i \leq m < o$ then

  $\epsilon_m \leftarrow \text{PairScoreOneSIMDLane}(m, i)$ $-\infty$ when the pair is rejected

  if $\epsilon_m \neq -\infty$ then

    $c \leftarrow f_m + \epsilon_m$

    if $c > f_i$ then

      $f_i \leftarrow c, \quad p_i \leftarrow m$

if $m < 0$ or $f_i \geq f_m$ then

  $m \leftarrow i$ B5 moves the cache to the newer anchor on a tie

5.7.2. Summary of forward chaining algorithms for CPU

mm2-fast is another version of minimap2 where specific parts were accelerated. The chaining algorithm in BioseqDB was inspired by mm2-fast (but note that as in case of other scientific work, mm2-fast provided only AVX2/AVX512 kernel, not universal code like in case of BioseqDB). There are some slight differences in high-level algorithm design:

- mm2-fast doesn't seem to have counter of anchors that does not improve the best score in the current batch (In BioseqDB bounded SIMD B1 we introduced such counter and in the pseudocode it's called U)
- mm2-fast uses scalar code to check for the lane winner, while BioseqDB bounded SIMD B1 uses SIMD instructions
- mm2-fast fallbacks to scalar code for fewer anchors than lane count

Table 5.2: Summary of all forward chaining implementations for CPU

Method	How candidates are processed	When the scan stops	How the result is selected
minimap2 B1	Score one predecessor j at a time. After each candidate, update the shared best pair and scalar skip/mark state	At the end of the chain, or when the scalar `max_skip` rule fires while j decreases	The shared (f_i, p_i) is already final
BioseqDB exact SIMD B1	Score one predecessor per active lane. Inspect a batch in decreasing j order when it may improve the best score or locate the exact `max_skip` stop	At the same candidate as scalar B1	The shared (f_i, p_i) is already final
mm2-fast	Score one predecessor per active lane. Each lane keeps its best pair across blocks	At the end of chain as there is no `max_skip`	Scalar code scans the lane winners
BioseqDB bounded batch SIMD B1	Score one predecessor per active lane. Each lane keeps its best pair, and one counter tracks batches with no lane-local improvement	At the end of the chain, or after U such consecutive batches	Choose the best lane result

Experimental justification

Note that although I was inspired by mm2-fast I still decided to apply the U limit with a counter (for counting anchors that do not improve score). This is not justified for now. Experimental data supporting this decision is presented in a separate section

5.8. Reverse chaining (backtracking)

The forward pass has already produced one score f_i and one predecessor p_i for every anchor. Backtracking follows the predecessor links and turns them into the chain records and flat anchor array A defined at the start of this chapter.

5.8.1. Formal definition for backtracking

Anchors have their score assigned: f_i. We would like to start by considering the highest scores as they are the best chains according to our score function. We sort the anchors in descending order of their scores: $f_{a_0} \geq f_{a_1} \geq \ldots \geq f_{a_{n-1}}$.

Now each anchor is the chain end so we need to find its start. We will traverse anchors according to p_i i.e $i \to p_i \to p_{p_i} \to \ldots \to -1$. This traversal doesn't have to go till the end of the chain. Usually there will be some part of the chain that is too low quality (not contributing to the score significantly).

We define function $\Delta_e(b)$ as the score assigned to the anchors followed from e up to, but not including, b:

$$\Delta_e(b) = \begin{cases} f_e, & b = -1, \\ f_e - f_b, & b \geq 0. \end{cases} \tag{5.16}$$

The value of $\Delta_e(b)$ will be used to determine the best cut point for each anchor:

$$\text{Cut}([\Delta_e(e), \Delta_e(p_e), \Delta_e(p_{p_e}), \ldots, \Delta_e(-1)]) \in [e, p_e, p_{p_e}, \ldots, -1] \tag{5.17}$$

The backtracking is then defined as:

$$\text{Backtrack}(e) = \{c = (a_i, p_{a_i}, \ldots \text{Cut}(a_i)) | a_i \in [a_0, a_1, \ldots, a_{n-1}] \ \wedge f_{a_i} \geq F_{\min} \wedge |c| \geq C_{\min}\} \tag{5.18}$$

The backtracking function finds best cutoff point and filters anchors for minimal score $F_{\min}$ and minimal chain length $C_{\min}$.

5.8.2. Minimap2 backtracking

Minimap2 uses two types of markings. The array t (similar to forward chaining), records which anchors have already been marked:

1. 0 means unmarked: we didn't visited this anchor yet
2. 1 means permanently marked: we won't consider this anchor as end of the chain, because it's already part of some found best chain. However anchors marked as 1 still can be part of the chain. This marking prevents a situation in which we would find chain, then it's exact subchain etc.
3. 2 means temporarily marked while one path is tested: this marking is used for Cut function and cleared after the path is tested. It purpose was to detect cycles (see cycle note below)

> **$t = 2$**
>
> Minimap2's implementation could remove usage of $t = 2$ markings, because under usual conditions there will be never a cycle of chains (anchor will always point to other anchor will lower position due to how forward chaining works). In listings below I provide the pseudocode corresponding to the actual current implementation, not some optimized one.

Minimap2 cut function marks visited anchors with $t = 2$ and stops on any anchor with non-zero t mark. We remember the best cut with the largest value of Δ_e. If a later cut is worse than the best cut by more than `max_drop`, the search stops and returns the best cut seen so far. After everything we clean the $t = 2$ temporary markings.

Figure 5.7 shows three interations of the main loop: we detect cuts for three anchors a_{11}, a_{10}, a_9. The example uses `min_sc` = 40 (this is minimap's minimum score $F_{\min}$), `min_cnt` = 2 (this is minimap's minimum chain length $C_{\min}$), and `max_drop` = 5 (maimum allowed score drop between best cut and current anchor in the chain). A Δ value inside anchor a_b is the score that would result if a_b was the selected cut. Orange boxes are anchors where either cut happens or we stop the search for the cut. Additionally I added "marked" label to a_5 to depict the fact that a_5 was marked by first $e = 11$ chain, so it cannot be end of the chain (it's still selected for $e = 10$, because it's start of the chain) Green boxes are anchors that are copied into the resulting array (everything excluding the cut anchor itself).

The minimap2 backtracking uses the same technique as BioseqDB Vulkan minimizer seeding: in the first pass we only count the number of copied anchors, while in the second pass we store them in the resulting array. This phase $\in \langle$count, write$\rangle$ iteration is depicted in (c) panel. The last step of minimap2's backtracking depicted in panel (d) is reversing the chains (so the start is on the left rather than on the right side) and sorting by reference coordinates of the chain's starts, so the chains are in natural left-to-right order.

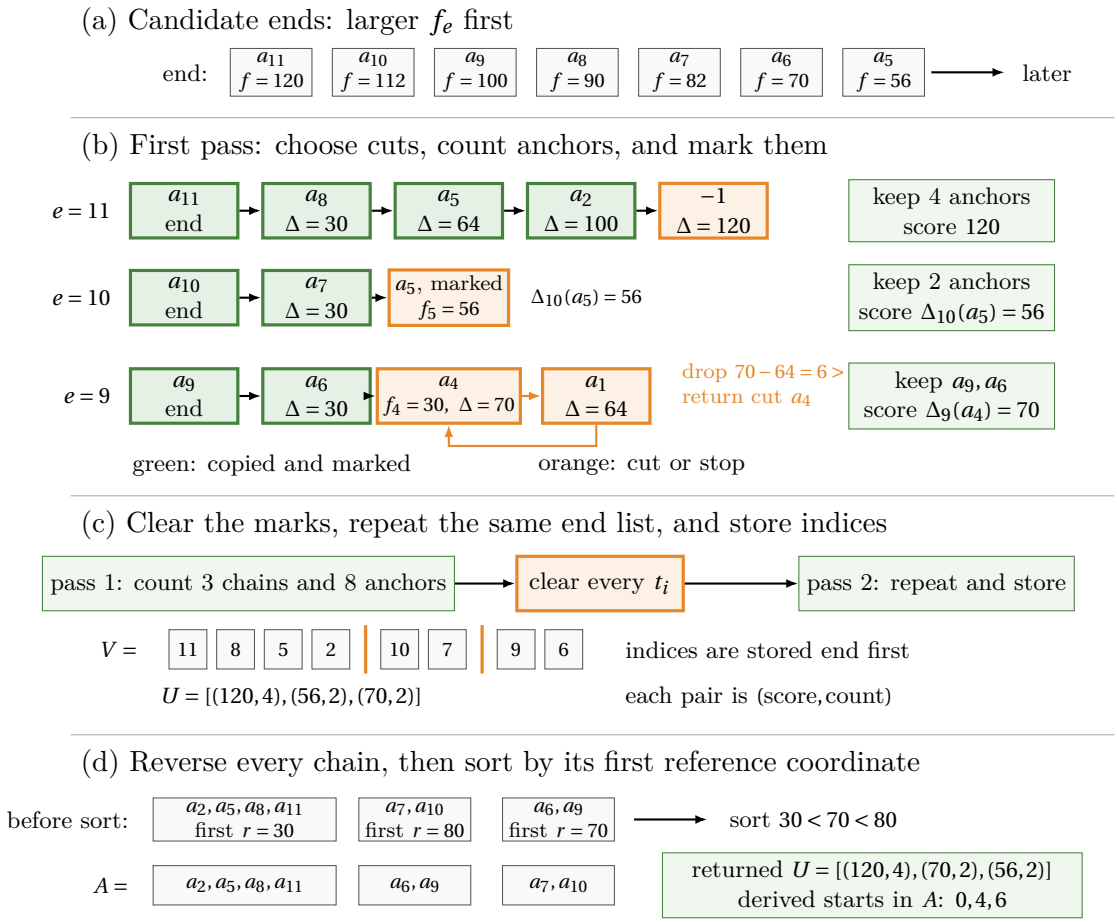


Figure 5.7: Minimap2 scalar backtracking on three anchors. Panel (b) shows that the anchor that triggers `max_drop` doesn't have to be the cut that is returned. Panel (c) shows the separate count and write passes. Panel (d) shows the final forward anchor order.

Algorithm 12 shows the code for the cut search loop.

Algorithm 12 Minimap2 cut selection function for anchor e

Require: unmarked end e, scores f, predecessors p, marks t

Require: drop limit $D = \texttt{max_drop}$
Ensure: cut b, which is excluded from the new chain

```
function ChooseCut(e, f, p, t, D)
    i ← e,   b ← e,   δ_best ← 0
    repeat
        t_i ← 2   temporary mark
        i ← p_i
        if i < 0 then
            δ ← f_e
        else
            δ ← f_e − f_i
        if δ > δ_best then
            δ_best ← δ,   b ← i
        else if δ_best − δ > D then
            break
    until i < 0 ∨ (i ≥ 0 ∧ t_i ≠ 0)
    h ← i,   j ← e   h is where this search stopped
    while j ≥ 0 and j ≠ h do
        t_j ← 0,   j ← p_j   clear only temporary marks
    return b
```

Algorithm 13 shows also how two passes for count/write are integrated. Minimap2 uses first pass to count all anchors i.e N_C (this value is used to allocate array U). Array U holds anchor count for a chain in lower 32 bits and score in higher 32 bits (this packing is defined by Pack function). U is then chain-reversed into B_γ array, which undergoes sorting by reference coordinates ReferencePos($B_\gamma[0]$) The final array A contains all anchors, while output array U contains packed `uint_64t` metadata for chain (count and score).

Algorithm 13 Minimap2 scalar backtracking and final chain order

Require: input anchors $a_0, \ldots, a_{n-1}$, scores f, predecessors p

Require: limits $S = \texttt{min_sc}$, $K = \texttt{min_cnt}$, and $D = \texttt{max_drop}$
Ensure: packed minimap2 records U and forward-order anchor array A

define $\mathrm{Pack}(s, c) = (\mathrm{uint64}(s) \ll 32) \mid c$
$Z \leftarrow [(f_i, i) \mid 0 \le i < n \text{ and } f_i \ge S]$
sort Z by increasing score only — read it backward below
use temporary array $V[0 \ldots n)$ — reuse an existing n-entry array
for phase $\in \langle \mathrm{count}, \mathrm{write} \rangle$ do
    $t[0 \ldots n) \leftarrow 0, \quad N_C \leftarrow 0, \quad N_V \leftarrow 0$ — accepted records and retained indices
    for $k \leftarrow |Z| - 1$ down to 0 do
        $e \leftarrow Z_k.\mathrm{index}$
        if $t_e = 0$ then
            $b \leftarrow \mathrm{ChooseCut}(e, f, p, t, D)$
            $N_V^{\mathrm{start}} \leftarrow N_V, \quad i \leftarrow e$ — cursor before tracing this end
            while $i \ne b$ do
                if phase = write then
                    $V[N_V] \leftarrow i$
                $N_V \leftarrow N_V + 1, \quad t_i \leftarrow 1, \quad i \leftarrow p_i$
            $c \leftarrow N_V - N_V^{\mathrm{start}}$
            if $b < 0$ then
                $s \leftarrow f_e$
            else
                $s \leftarrow f_e - f_b$
            if $s \ge S$ and $c > 0$ and $c \ge K$ then
                if phase = write then
                    $U[N_C] \leftarrow \mathrm{Pack}(s, c)$
                $N_C \leftarrow N_C + 1$
            else
                $N_V \leftarrow N_V^{\mathrm{start}}$ — discard indices but keep the marks
    if phase = count then
        allocate $U[0 \ldots N_C)$
$o \leftarrow 0$
for $\gamma \leftarrow 0$ to $N_C - 1$ do
    $c_\gamma \leftarrow \mathrm{uint32}(U[\gamma])$ — low 32 bits store the anchor count
    $B_\gamma \leftarrow [a_{V[o+c_\gamma-1]}, \ldots, a_{V[o]}]$ — reverse one chain
    $o \leftarrow o + c_\gamma$
$\pi \leftarrow \mathrm{argsort}_{0 \le \gamma < N_C} \mathrm{ReferencePos}(B_\gamma[0])$ — increasing ReferencePos
$U_{\mathrm{old}} \leftarrow U, \quad U \leftarrow [\,], \quad A \leftarrow [\,]$
for $m \leftarrow 0$ to $N_C - 1$ do
    $\gamma \leftarrow \pi_m$
    append $U_{\mathrm{old}}[\gamma]$ to U
    append every anchor in B_γ to A
return (U, A)

Start positions

Minimap2 stores only the score and anchor count, but it does not store an anchor start. The start of a chain is the sum of the counts of all earlier entries in U and later code uses that sum. I were initially very suprised and confused by summing coordinates, but this is the explanation.

5.8.3. BioseqDB SIMD backtracking

Exact SIMD B1 and bounded SIMD B1 call the same SIMD backtracking function. Its main differences from minimap2 are the following.

1. Minimap2 follows and marks one end at a time. BioseqDB handles the end with largest (f_e, e) alone, then collects up to L later paths together.
2. Minimap2 can cut a path with `max_drop`. BioseqDB follows a path until -1 or an anchor that is already marked.
3. Minimap2 repeats the complete backtracking pass so it can count chains before allocating U. BioseqDB runs one pass and appends accepted chains to output arrays that grow as needed.
4. Minimap2 does not specify an anchor order for equal scores. BioseqDB tries the higher end position first on a score tie.
5. Minimap2 finally sorts complete chains by ReferencePos of their first anchor. BioseqDB leaves accepted chains in the order in which it processes their ends.

In Bioseq chaining we consider 3 different operations:

1. Single-path: This is a special case for consequitive anchor indices. If anchor has a chain built from consecutive anchor indices, then we can follow only one path at the time (SIMD vector contains only fragment of one chain) and mark anchors in bulk.
2. Multi-path: This is expected case of SIMD-ified algorithm. In SIMD vector we store multiple anchors and follow each of the chains in parallel.
3. Multi-path marking: This is the step that we do after multi-path - we need to mark all anchors and we do that by transposing the matrix created in multi-path step, where now each of L lanes contains consecutive indices for one chain. (In reality we don't transpose any matrix, it's here only as illustration as in the code we put chains directly into final path rows)

In BioseqDB we only have binary marks:

$$t_i = 0 \quad \text{means unclaimed,} \qquad t_i = 1 \quad \text{means claimed.}$$

There is no temporary value 2, which was obsolete as mentioned earlier. A mark is never cleared, including when the marked prefix later fails the score or count test.

If L consecutive links satisfy $p_j = j - 1$, then we run single-path algorithm version, where one vector comparison checks the L links. Formally this condition is specified as:

$$\mathrm{Run}_L(j, c) \iff j \geq L-1 \ \wedge \ c \leq n - L \ \wedge \ \bigwedge_{u=0}^{L-1} p_{j-u} = j - u - 1.$$

Chain order heuristic

At first it might not be intuitive why we are interested in chains of consecutive anchors. We can think of inverse of that condition. If the anchors are not consecutive, then it means that for some anchor there exists alternative very good match i.e for one target position there are multiple competing chainings for different query positions that fight over few sibling anchors. This means that fragment in query is not easily recognisable as query is repetitive and each repetition is equally good (very low divergence rates). For now we will just assume that this condition $\mathsf{Run}_L(j,c)$ is non-trivial algorithm specialisation that is discussed in detail in separate chapter.

If the $\mathsf{Run}_L(j,c)$ condition is met then we use only two store operations to mark L anchors of that single chain (we pay the cost of checking the condition, but the we benefit by using less instructions). If not "multi-path" version follows one chain per lane. This concept is illustrated in Figure 5.8 section (b). At first each lane contains consecutive indices which are highest-scoring anchors used as ends. Then we iterate trhough all of them until we reach the stop condition. Each lane λ follows one chain and can stop earlier than others. We don't want to explicitly hold this structure as matrix and transpose it. Instead for iteration (depth) d we directly store (scatter)lanes:

$$V_{\lambda,d} \quad \overrightarrow{\text{memory address}} \quad \text{Address}(V) + (\lambda n + d) * \text{sizeof}(V_{0,0}).$$

where n constant is the arbitrary number of SIMD vectors we want to store ($d < n$)

Figure 5.8 illustrates the algorithm with number of lanes: $L = 4$, minimum score $F_{\min}$: $S = 30$, and minimum chain length $C_{\min}$: $K = 2$. The relevant scores for the example are:

$$f_{19} = 100, \quad f_{18} = 90, \quad f_{17} = 85, \quad f_{16} = 80, \quad f_{15} = 75, \quad f_9 = 40, \quad f_{11} = 55, \quad f_{10} = 50.$$

- First we start with f_{19} as this is the highest-scoring anchor. We look for $p[19]$ and load $[p[19], p[18], p[17], p[16]] = [9,8,7,6]$ into SIMD vector. This is compared to vector $[p[19], p[19], p[19], p[19]] - [0,1,2,3]$, which in this case are equal. This means (toegether with bound checks) that $\mathsf{Run}_L(19,0)$ is true. Hence we run the single-path algorithm. I specifically split the second vector in the comparison to illustrate how SIMD vectors are constructed and compared in the real code. Then the single-path (a) will use store operation to mark L anchors as visited and reverse the indices and store them in the result vector.

- For first path (a) when we encounter 1 (the end of the array of anchors) we fallback to multi-path behaviour (this is the simplest way to avoid out-of-bounds access. Otherwise we would need to implement complicated masking). This is observed in (a) as 1 and 0 being in separate fragments rather than single one.

- The multi-path is illustrated in (b). The next highest scoring anchor after f_{19} is f_{18} with score 90. We look for $p[18]$ and load $[p[18], p[17], p[16], p[15]] = [14,13,12,10]$ into SIMD vector. The comparison in this case will yield false as $p[15] \neq 11$. This means that we follow each index one by one. Each time we load markings for the index and check what lanes were already marked (this forms a SIMD binary mask). If all lanes are masked-out then we can stop iteration completely. At each iteration we store lanes values in correct indices in our V array.

- After multi-path (b) runs we still needs to filter the chains by score and length.

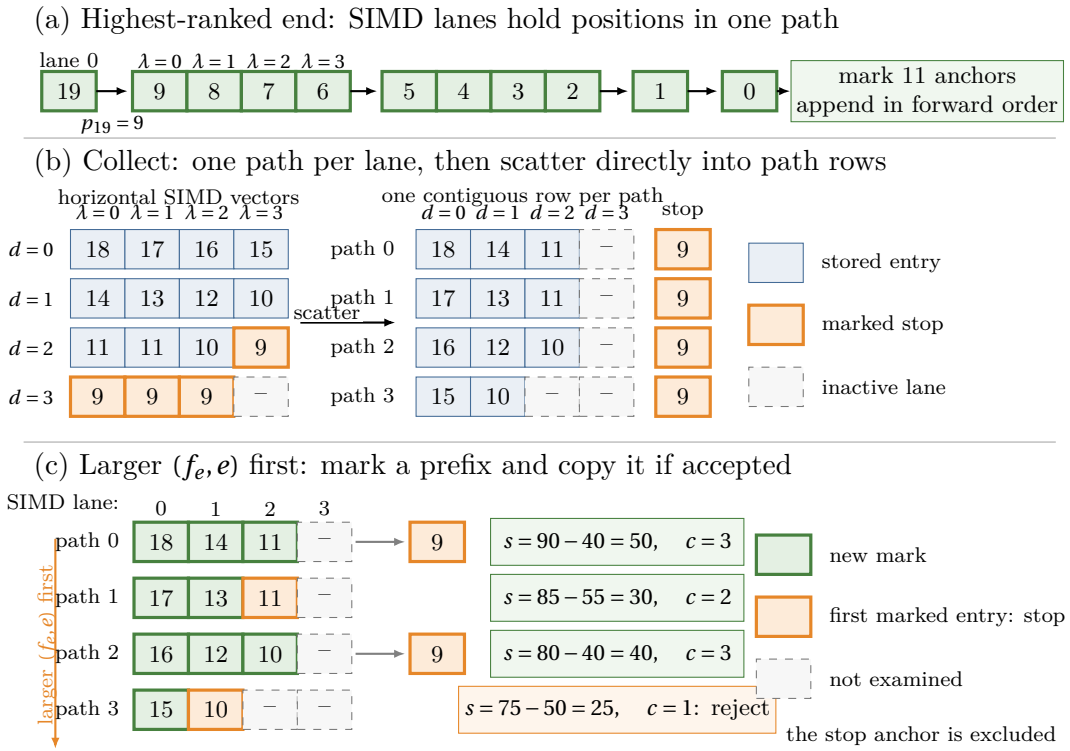


Figure 5.8: BioseqDB SIMD backtracking with four L lanes. Panel (a) shows the special handling of the first path. If we detect Run_L is true, then we use entire SIMD vector just for this one chain (SIMD vector contains anchors for $L = 4$ different chains). Panel (b) shows the other case when Run_L if false. The SIMD vector changes with each iteration $d = 0, 1, 2, 3$ and it's used to store (scatter) anchors indices into the buffer (on the right size). Panel (c) reads one row at a time from the buffer starting at highest-scoring anchors first (now the SIMD vector as in case of Panel (a) contains one chain), we check for marked nodes via FirstMarked function. If there are no marked anchors detected then we iterate this processing multiple times going through the chain. After reaching either end or marked node, we save the chain if it's score is greater than required threshold and it's length is sufficient.

The accepted output in this example is

$$C = [(100, 11, 0), (50, 3, 11), (30, 2, 14), (40, 3, 16)],$$
$$A = [a_0, \ldots, a_9, a_{19} \mid a_{11}, a_{14}, a_{18} \mid a_{13}, a_{17} \mid a_{10}, a_{12}, a_{16}].$$

Each record in C is (score, count, start). For a_{15} (in the Panel (c)) we mark it before testing its thresholds, but then the next anchor has too low score and is rejected. Because the chain length is just 1 which is lower than minimum required length, we don't copy that chain into A.

In the listing below, I present the code for this backtracking algorithm.

In the pseudocode we use Run_L to set g which is either L or 1 depending if we choose multi-path or single-path iteration. Additionally, we didn't define $FirstMarked$ function in the pseudocode as it's just helper to determine left-most marked anchor in the row (limited to only g entries):

$$\text{FirstMarked}(t, V_\lambda, o, g) = \min\big(\{u \mid 0 \le u < g \wedge t_{V_{\lambda, o+u}} = 1\} \cup \{g\}\big).$$

It returns g when none of the g entries is marked. On the pseudocode listing for row λ of V denoted as V_λ:

- ℓ_λ is the number of stored entries in that row
- b_λ is the row's stop anchor
- α_λ says whether its collection lane is active (this is SIMD mask to determine if we need to terminate the chain if we either reach -1 or already marked anchor)

Algorithm 14 BioseqDB SIMD path collection and backtracking

Require: anchors $a[0\ldots n)$, scores f, predecessors p, thresholds S, K, width L, and empty outputs C, A

function ChooseCutsSIMD($\mathbf{e}, p, t, V, n$)
  $q \leftarrow |\mathbf{e}|, \quad \mathbf{j} \leftarrow \mathbf{e}, \quad \boldsymbol{\ell} \leftarrow \mathbf{0}, \quad \mathbf{b} \leftarrow -\mathbf{1}, \quad \boldsymbol{\alpha} \leftarrow \mathbf{1}$
  for $d \leftarrow 0, \ldots, n-1$ while Note that all things detnoted by λ are evaluated for $0 \le \lambda < q$ in one SIMD vector $\sum_{\lambda=0}^{q-1} \alpha_\lambda \neq 0$ do
    $h_\lambda \leftarrow [\alpha_\lambda = 1 \wedge t_{j_\lambda} = 1], \quad \eta_\lambda \leftarrow \alpha_\lambda \wedge \neg h_\lambda$ check if we reached marked anchor in the lane
    $b_\lambda \leftarrow j_\lambda$ if $h_\lambda = 1$, and $(V_{\lambda,d}, \ell_\lambda) \leftarrow (j_\lambda, \ell_\lambda + 1)$ if $\eta_\lambda = 1$ scatter
    $j_\lambda \leftarrow \begin{cases} p_{j_\lambda}, & \eta_\lambda = 1 \\ -1, & \text{otherwise} \end{cases}, \quad \alpha_\lambda \leftarrow [j_\lambda \ge 0]$
  assert $\sum_{\lambda=0}^{q-1} \alpha_\lambda = 0$ All of the lanes should terminate chains
  return $(\boldsymbol{\ell}, \mathbf{b})$
function BacktrackSIMD(a, f, p, S, K, L, C, A)
  allocate $V \in \text{int32}^{L \times n}$, and set $t[0\ldots n) \leftarrow 0$ 0: unclaimed, 1: claimed
  $Z \leftarrow \text{sort}_\uparrow[(f_i, i) \mid 0 \le i < n \wedge f_i \ge S], \quad k \leftarrow |Z| - 1$
  if $k \ge 0$ then
    $e \leftarrow Z_k.\text{index}, \quad j \leftarrow e, \quad c \leftarrow 0$
    while $j \ge 0 \wedge c < n$ do
      $g \leftarrow \begin{cases} L, & \text{Run}_L(j, c) \\ 1, & \text{otherwise} \end{cases}$
      $(V_{0,c+u}, t_{j-u}) \leftarrow (j-u, 1)$ for $0 \le u < g$, then $(j, c) \leftarrow (p_{j-g+1}, c+g)$
    assert $j < 0$
    if $f_e \ge S \wedge c \ge K$ then append $(f_e, c, |A|)$ to C and $(a_{V_{0,c-1}}, \ldots, a_{V_{0,0}})$ to A
    $k \leftarrow k - 1$
  while $k \ge 0$ do
    $q \leftarrow \min(L, k+1), \quad \mathbf{e} \leftarrow (e_0, \ldots, e_{q-1}), \quad e_\lambda \leftarrow Z_{k-\lambda}.\text{index}$
    if $(S > 0 \vee K > 0) \wedge \bigwedge_{\lambda=0}^{q-1} t_{e_\lambda} = 1$ then $k \leftarrow k - q$ and continue
    $(\boldsymbol{\ell}, \mathbf{b}) \leftarrow$ ChooseCutsSIMD($\mathbf{e}, p, t, V, n$)
    for $\lambda \leftarrow 0$ to $q-1$ do
      $c \leftarrow 0, \quad o \leftarrow 0$ c holds length of chain and o is the offset in the A where we store it
      while $o < \ell_\lambda$ do
        $g \leftarrow \min(L, \ell_\lambda - o), \quad r \leftarrow \text{FirstMarked}(t, V_\lambda, o, g)$
        $t_{V_{\lambda,o+u}} \leftarrow 1$ for $0 \le u < r$, and $c \leftarrow c + r$
        if $r < g$ then $b_\lambda \leftarrow V_{\lambda,o+r}$ and break, otherwise $o \leftarrow o + g$
      $s \leftarrow \begin{cases} f_{e_\lambda}, & b_\lambda < 0 \\ f_{e_\lambda} - f_{b_\lambda}, & b_\lambda \ge 0 \end{cases}$
      if $s \ge S \wedge c \ge K$ then append $(s, c, |A|)$ to C and $(a_{V_{\lambda,c-1}}, \ldots, a_{V_{\lambda,0}})$ to A
    $k \leftarrow k - q$
  return (C, A)

Minor improvements

Note that we check $\bigwedge_{\lambda=0}^{q-1} t_{e_\lambda} = 1$ for the first iteration to check if we have chains that all go through other chains that were already all marked. This could be implemented via SIMD masks operations, but in my code I count the true bits and compare their count. This means there are still some very minor optimisation opportunities that could be explored

5.9. Statistical modelling of CPU forward and backward passes

The first loop in Algorithm 14 can process L anchors at once only when $\mathsf{Run}_L(j, c)$ is true. We also use update counter U in Algorithm 11 to terminate loop if the consecutive updates don't improve any SIMD lane. At the same time minimap2 uses max_drop in its algorithm 5.7 to terminate backtracking if the score drop is too large.

We never explained any proper justification for using or not using these heuristics. This section talks about random synthetic data and intuitive explanation behind the choices. The subsections talk about:

- Preparing synthetic data (randomly generated sequences with insertions/deletions) and specially constructed examples aimed at absuing the scoring function and thresholds
- Answering if counting number of updates that do not improve score in forward chaining algorithm 11 improve performance
- Answering if the anchors during backtracking 14 are just continous sequences of indices incremented by 1 ($\mathsf{Run}_L(j, c)$ condition)
- Answering if max drop would help us increase performance of SIMD backtracking algorithm or not 5.7

Synthetic data

Primary modelling here uses synthetic data built from randomly generated sequences assuming default substitution rate of 0.001 and insertion/deleteion probability of 0.0005. I prepared special cases of sequence construction using gaps or equal regions to see what happens in cases where extremally large gaps occur as secondary method of analysis. Overall the analysis is intended to motivate the approach and formulate conditions under which an optimisation's cost exceeds its benefit.

5.9.1. Problematic chains

As mentioned in section 5.3 there is a case of emergent assymetry: if one anchor matches with multiple anchors in the query, then we cannot easily tell which part of the query is best to match. The following sections demonstrates what happens in this scenario. The special query/target constructions are prepared in that asymmetry in mind.

Reference and query structures

In our experiments we use δ as input parameter for divergence (controlling substituiton rates). The exact substituion count (number of replaced bases in entire sequence) can be expressed

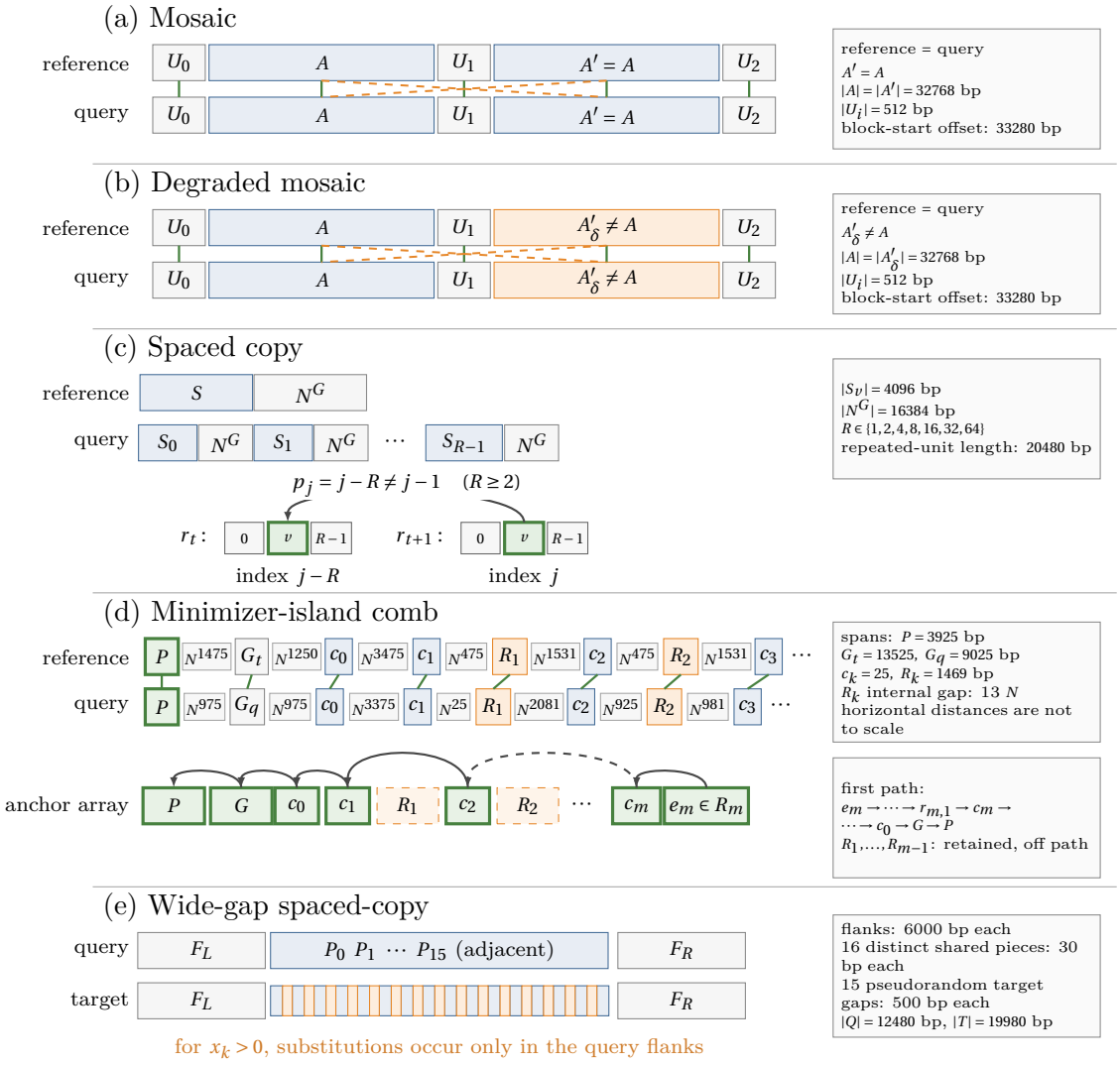


Figure 5.9: Panel (a) presents mosaic, with $A' = A$ so $T = Q = U_0 A U_1 A'_\delta U_2$ where $A' = \text{Randomize}_\delta(A)$. Panel (b) is degraded mosaic, so the A and A' are independently generated, so the divergence between them should be very high, used as control example. Panel (c) is shows spaced copy, which is contructed in the following way $T = S N^G$, $Q = (S N^G)^R$, $|S| = 4096$, $G = 16384$ Panel (d) is the Minimizer-island comb. I construct it by assuming that every c_k and $r_{k,\ell}$ is an isolated 25-base minimizer island. Then $r_{k,\ell}$ are grouped together with 13-N intervals to create R_k groups. The construction is calculated in such a way that $|d_r - d_q|$ is lower for c_k anchors than for R_k anchors. Panel (e) is wide-gap spaced-copy, where we insert gaps between 16 30-base pieces of query with absurdly large 500-base gaps.

as:

$$s_\delta = \sum_{\text{fragment} \in \text{query}} \left\lfloor \delta|A| + \frac{1}{2} \right\rfloor \tag{5.19}$$

The $+\frac{1}{2}$ is just used to round the result to the nearest integer. The sum goes over all fragments we can change (this depends on exact construction as some of them need to contain specific exact regions)

Figure 5.9 depicts all of my constructions:

- Random reference and query - parameters to experiments are $|Q|$ and $|T|$, number of occurences of query $1 \leq N$. I first generate random target, then place N query occurences in it and use divergence to create substitution in the occurences.
- (A) Mosaic - reference and query with identical bytes. A is randomly generated and I perform substitutions on it. This is example of exact duplication of long fragment. The specific A length was choosen to be very long to generate long, very similar chains.
- (B) Degraded mosaic - this is version of mosaic, where we use two regions that aren't equal and do not perform any substitutions on them. This is control case, where the generation of both A regions is independent. Meaning the expected divergence should be very high (as sequences are random).
- (C) Spaced copy - we insert very long blocks N^G between parts of reference, where N denotes symbol N (in dna4 alphabet it denotes unknown symbol). Normal processing just ignores these blocks (minimap2 doesn't generate minimizers for them), but they distrupt anchor positions. You can think of this construction as very special case of gapped repeatitions of same sequence, where instead of random gap-filler we use symbol N. That kind of contruction shouldn't occur in real-world example, it's here to illustrate that we can utilize N symbol to sabotage chaining.
- (D) Minimizer-island comb - we also use N symbol as filler between inslands. As we use default minimap2 parameters including $w = 10, k = 15$, we adjusted $|c_k|$ to be $w + k = 25$ bases, so these fragments generate maximum of two non-N minimizers. R_k groups are internally built from the same 25-symbol "islands" separated by 13 N bases (13 was arbitrary choice). This creates very interesting scenario where we can control which minimizers occur where. Then we adjust wider gaps between "islands" with tricky criteria. As DP algorithm selects one "next" best anchor to "connect" chain to, we adjust the gaps so the difference between target and query distances are large for the anchors just at the edge of R_k but smaller for ck_kj. You can look at the example and see that if we try to chain c_1 to c_0 the $\delta(c_0, c_1) = |d_r - d_q| = 3475 - 3375 = 100$ but δ for R_1 will be $\delta(c_0, R_1) = (3475 + 475) - (3375 + 25) = 550$. We exploit the feature of scoring function to convince the minimap2 that c_k is "better" match being closer to diagonal than R_k. This will create long chain starting at R_k and going through all c_k anchors (ommiting all other R)
- (E) This example is more practical one, where we don't overuse N symbols and divide the query to add random gaps between its pieces. However the gaps are very large compared to the "islands" so this example is "exaggerated" as well.

Overuse of N symbol

Usually N is used only to distinbguish processing errors (from the sequencing hardware) and do not occur with high quantities. The construction of the examples use them as gap-fillers to have easier control over occurence of minimizers and their behavior. Cases (A) and (E) are closer to real-world examples, where repetitions of genomes happen frequently.

Constructions (a), (b) and (c) are primary using the same idea. After collecting and sorting anchors the anchor table looks like this for spaced copy (sorted by target coordinate):

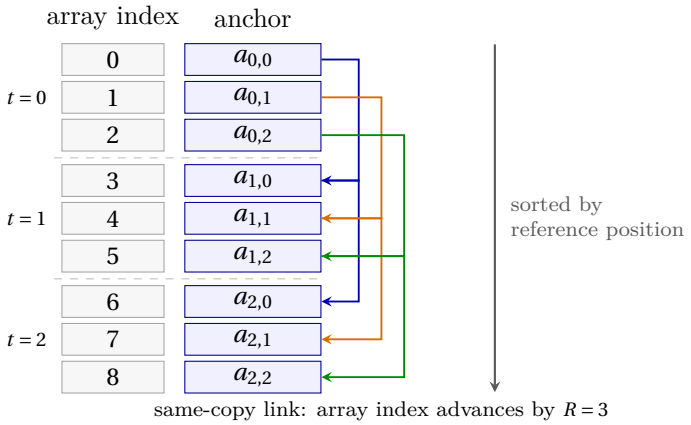


Figure 5.10: Coordinate-sorted spaced-copy anchors for the concrete case $R=3$. As the query contains exact R copies we will have "layers" ($t=0,1,2...$) of the same anchors with the same reference position. Anchor $a_{t,v}$ is the anchor for v-th copy in the query. Colored arrows follow mark the next anchor in the chain. Naturally each anchor points to the next anchor in the same v-th copy. This means that each anchor will just point R anchors forward.

The construction (d) - minimizer-island comb was created by adjusting gap sizes between query and target islands:

transition	target N-run	query N-run
$P \to G$	1475	975
$G \to c_0$	1250	975
$c_0 \to c_1$	3475	3375
$c_k \to R_k$	475	25 (k odd), 925 (k even)
$R_k \to c_{k+1}$	1531	2081 (k odd), 981 (k even).

Default minimap2 parameters

As we discussed due to larger shifts between target and query coordinates, the R_k anchors will have larger pentalty due to higher distance from diagonal i.e δ. Minimap2 by default uses max bandwith of $B = \text{bw} = 500$. Every anchor in R_k and c_{k+1} differ by 550 between the target and query distances, so those anchors fail the bandwidth check. This means that the best chain will be going through: $R_m, c_m, c_{m-1}, \ldots c_0, G, P$ as depicted in the figure 5.9 The original dimentions of constrution can be calculated from the following formula ($r_{k,\ell}$ is the ℓ-th anchor in R_k), L is the count of "islands" in R_k:

$$B = 500, L = 39, s(k) = \begin{cases} 1 & \text{if } k \text{ is odd} \\ -1 & \text{if } k \text{ is even} \end{cases}$$

$$c_k \to c_{k-1} : (3500,\ 3500 - 100 * s(k)),$$
$$r_{k,\ell} \to c_k : (B + (L-1)(\ell - 1),\ B - 450 * s(k) + (L-1)(\ell - 1)), \quad 1 \le \ell \le L.$$

Experiment design

Minimap2 2.31-r1302 was run with the `map-ont` preset (default one). I added instrumentation to record the score and predecessor arrays.

- Let $L \in \{2, 4, 8, 16, 32, 64\}$ be the number of anchors viewed at the same time similar to our SIMD algorithms.
- Let $\text{Run}_L(j, c) \iff j \ge L - 1 \wedge c \le n - L \wedge \bigwedge_{u=0}^{L-1} p_{j-u} = j - u - 1$ be the condition used in BioseqDB SIMD algorithms.
- Let B_L be the number of evaluations for which Run_L is true
- Let E_L the number of times the first-path loop evaluates Run_L
- Let P_L the number that satisfy both bounds $j \ge L - 1$ and $c \le n - L$.
- Let q be the number of anchors on the first predecessor path.
- Let Fire_L be the observed rate at which my heuristic fires
- Let Contiguous_L be the the ratio Fire_L but with bounds checks excluded (we measure firing rate but only after bounds checks)
- Let Coverage_L be the fraction of path anchors consumed by vector batches (this is a metric that answers the question "How many anchors we can cover by SIMD batches")

The exact definitions are as follows:

$$\text{Fire}_L = \frac{B_L}{E_L}, \qquad \text{Contiguous}_L = \frac{B_L}{P_L}, \qquad \text{Coverage}_L = \frac{LB_L}{q}.$$

Table 5.3: Observed first-path branch-firing rate $100B_L/E_L$, in percent. The Mosaic, Degraded mosaic, and $R = 1$ Spaced copy rows report medians over three inputs generated from fixed random-number-generator seeds. The $R \geq 2$ Spaced copy row covers the tested powers of two from 2 through 64, all of which gave zero. Minimizer-island-comb values are deterministic and agreed with the recorded counters.

Input	$L = 2$	$L = 4$	$L = 8$	$L = 16$	$L = 32$	$L = 64$
(a) Mosaic	1.17	0.59	0.29	0.14	0.07	0.02
(b) Degraded mosaic (independent A')	100.00	99.94	99.74	99.24	97.01	91.12
(c) Spaced copy, $R = 1$	100.00	98.97	94.12	88.89	80.00	66.67
(c) Spaced copy, $R \geq 2$	0.00	0.00	0.00	0.00	0.00	0.00
(d) Minimizer-island comb, $m = 2$	95.74	84.62	45.45	29.41	6.67	0.00
(d) Minimizer-island comb, $m = 1024$	4.21	2.10	0.96	0.48	0.19	0.00

We can make the following observations:

- For minimizer-island the prefix will contain some $C > 0$ anchors, meaning that we will get only continuous parts in R_m at the end and in the sequence prefix: $B_L \simeq \lfloor \frac{39}{L} \rfloor + \lfloor \frac{C}{L} \rfloor, q = m + C$ This means that for $L = const$: $\mathsf{Fire}_L, \mathsf{Coverage}_L < \theta(1/m)$ and for large enough $L > 39 \wedge m \to \infty$ they will approach 0.
- For spaced copy the anchors are always separated by R so Fire_L should be zero for all $R > 0$
- Degraded mosaic is just simulating two random fragments, so Fire_L should be almost 100% with high measured divergence.
- Mosaic in this case was simulated with zero divergence, so the Fire_L should measure anchors only in the fillers between equal parts i.e $0\% < \mathsf{Fire}_L < \frac{|U_i|*3}{|U_i|*3+|A|} < 4\%$

The table 5.3 confirms our assumptions.

Now we would like to see how divergence δ affects the mosaic rates. Table 5.4 shows selected data points from Mosaic runs at δ. s_δ is the number of substitutions made to the target sequence. We can see that the higher number of same-target anchor alternatives (multiple query positions per anchor), the worse results we get (as predicted).

Table 5.4: Few most important data points for mosaic construction for different values of δ and degraded mosaic (should have effective high divergence). Each entry in the last three columns id the median followed by the observed minimum and maximum over three inputs generated from fixed random-number-generator seeds, in percent.

Case	s_δ	$100\hat{\delta}$ (%)	Path anchors with same-target alternatives (%)	$100\,\mathsf{Fire}_4$ (%)	$100\,\mathsf{Coverage}_4$ (%)
$\delta = 0$	0	0.000	97.69 [97.64, 97.69]	0.59 [0.58, 0.59]	2.31 [2.29, 2.33]
$\delta = 0.0025$	82	0.250	93.51 [93.40, 93.52]	1.06 [1.01, 1.18]	4.12 [3.91, 4.57]
$\delta = 0.0050$	164	0.500	89.55 [89.47, 89.68]	1.56 [1.47, 1.79]	5.96 [5.64, 6.79]
$\delta = 0.0100$	328	1.001	82.24 [82.19, 82.29]	2.61 [2.57, 2.80]	9.69 [9.55, 10.34]
$\delta = 0.0200$	655	1.999	69.33 [68.93, 69.42]	5.14 [5.08, 5.30]	17.82 [17.64, 18.28]
$\delta = 0.0500$	1638	4.999	41.41 [41.33, 41.63]	14.97 [14.97, 15.38]	41.32 [41.32, 42.09]
$\delta = 0.1000$	3277	10.001	17.23 [16.81, 17.64]	37.99 [37.64, 38.72]	71.02 [70.71, 71.65]
$\delta = 0.2000$	6554	20.001	3.04 [2.66, 3.36]	79.41 [77.58, 81.93]	93.91 [93.26, 94.77]
Independent A'	n/a	75.043	0.00 [0.00, 0.00]	99.94 [99.90, 100.00]	99.98 [99.98, 100.00]

Let's consider that we are looking at mosaic with divergence rate $\delta = 0$, so $A = A'$. Then we have h pairwise-disjoint "blocking" minimizer windows $W_1, \ldots, W_h$. Each W_i has some

minimizer that now forms an anchor that makes $Run_L(j,c)$ evaluation false. W_i covers $k+w-1$ bases. Each of the windows needs to receive at least one subsitution so they're changed and do not generate one anchor with two different positions. So with probability $\frac{(s_\delta)_h}{(M)_h}$ all h windows will receive at least one substitution (for $(x)_h = x(x-1)\cdots(x-h+1)$). We can choose one base from every window with b^h ways, so this gives us very loose upper bound on the probability:

$$\begin{aligned}\Pr_\pi\big[\mathrm{Run}_L(j,c) \text{ changes from false to true}\big] &\le \Pr_\pi\big[S_\delta \cap W_i \neq \varnothing \text{ for every } i\big] \\ &\le b^h \frac{(s_\delta)_h}{(M)_h} \le (b\widehat{\delta})^h.\end{aligned}$$

Obiously disjoint-window model is a bad assumption as substitution can affect several overlapping windows and conversely with high divergence we can spontaneusly get random matches with other fragments like U_i (although this is unlikely). However we can loosely bound the probability by $\theta((k\widehat{\delta})^h)$. This gives us idea that the worst sequences are the one with perfect repeats (low divergence).

Figure 5.11 shows the results for mosaic for different levels of divergence (the degraded mosaic - as control sample - is show separately as a gray rectangle on the right side of chart)

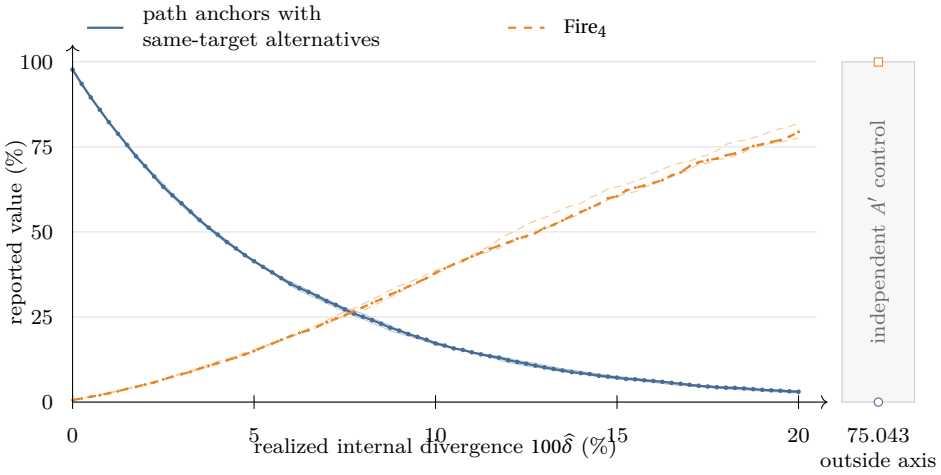


Figure 5.11: The lighter colours represent minimum/maximum while the darker ones represent median. As we can see the chart confirms that the Run_L firing rates are tied to number of anchors with same-target positions which are competing for the same anchor. Additionally we observe that divergence levels of at least 12% mitigate the negative effect of highly-repetive query.

As we can see from Figure 5.11, table 5.4 and table 5.3 the most inconvenient case is the high number of repetitions in the query. These repetitions can be mitigated by increased divergence levels. This was intuitive consulsion that we now sucessfully confirmed with the experiments.

Random-sequence subsitutions

I also wanted to measure effects of random sequences (not specially constructed) on the Run_L firing rates:

- We generate random sequences query/target parameterized by their lengths: $|T|$ and $|Q|$. We choose N positions in target to be the query and replace its fragments ac-

cordingly. By default we test each such occurence base with probability 0.001 to be substituted, 0.0005 to be inserted/deleted.

- Here I also use the default minimap2 parameters: $k = 15$, $w = 10$, occurrence filtering, and chaining scores.

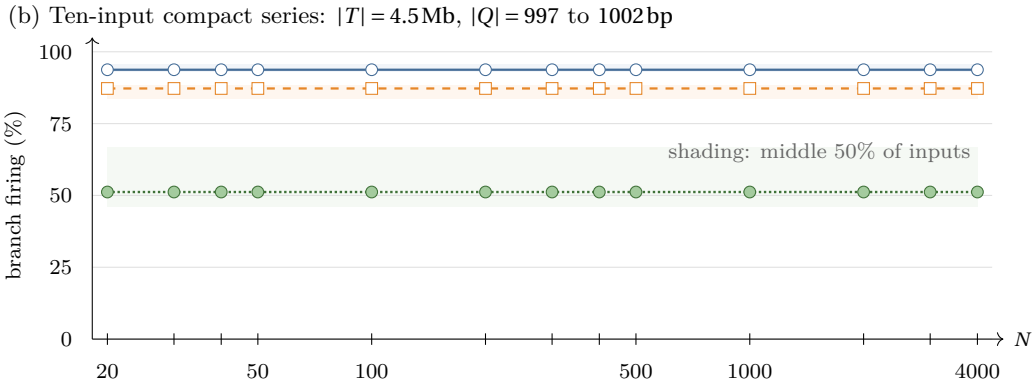


Figure 5.12: Branch firing rate $100\,\mathrm{Fire}_L$ for different values of L and N for random-sequence occurrences. As we can observe the N doesn't affect the firing rate as expected. The light colours represent middle 50% of inputs. As we observe as L gets larger it's more likely to disrupt the condition randomly so the standard deviation should be higher.

Figure 5.12 was used to just confirm that random generation is correct and N doesn't affect the probability (it can affect probability if the target size is too small and query occurences are too crowded and start to overlap). Figure 5.13 shows more interesting data. I tested $Fire_L$ (panel a) and $Reduction_L$ (panel b)for different valujes of δ and L.



Figure 5.13: Each data point represents sum of samples (averaged rates) over N=2,3,4...,4000 randomized experiments. The samples for $N = 1$ were marked with circles as we expect that they will be significantly higher (as there's one ocurrence)

Firing rate

Naturally the firing rate for $L = 4$ is higher, but $Reduction_L$ is initially higher for $L = 16$ meaning that on the hardware supporting more vectors (longer SIMD vectors), we exchange more condition checks (it's harder to guarantee $Run_L(i, j)$), but for more saved instructions (we can skip more L anchors). In practice we should test this on different hardware as the ideal loop reduction on panel (b) doesn't involve real costs of the instructions in cycles or cache latency etc.

5.9.2. Score distribution for random cases

If we plot the anchor count versus the cummulative chain score, we should get a curve that is roughly linear (this follows from formulation of the scoring function). The divergence should break long chains and make the curve score axis more narrow. This can be observed on Figure 5.14. I supply the charts to show how random inputs can behave as supplementary resources to the theoretical chapters. I also provide figure 5.15 to better illustrate "narrowing" of the chart

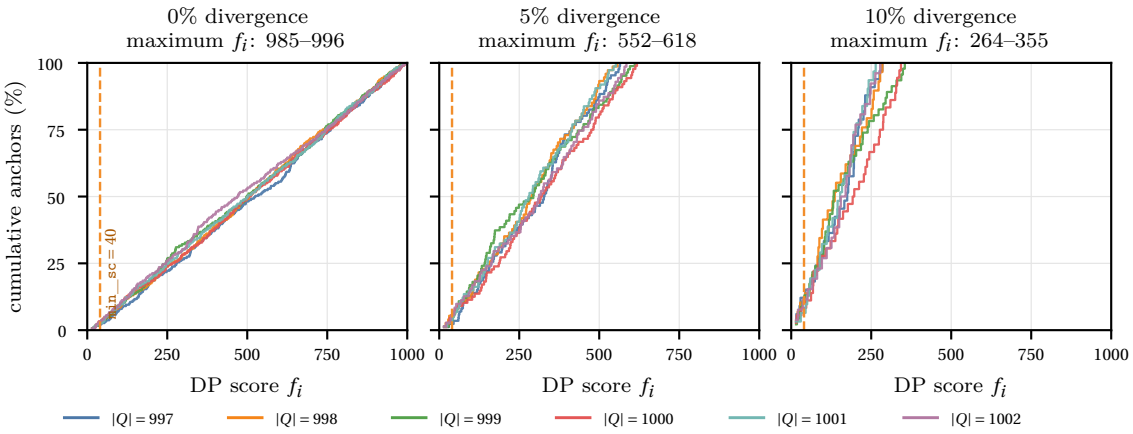


Figure 5.14: Results from randomized sequences for slightly different query lengths (997-1002 bases). The orange line is the minimum score threshold for chain (minimap default of `min_sc` = 40)

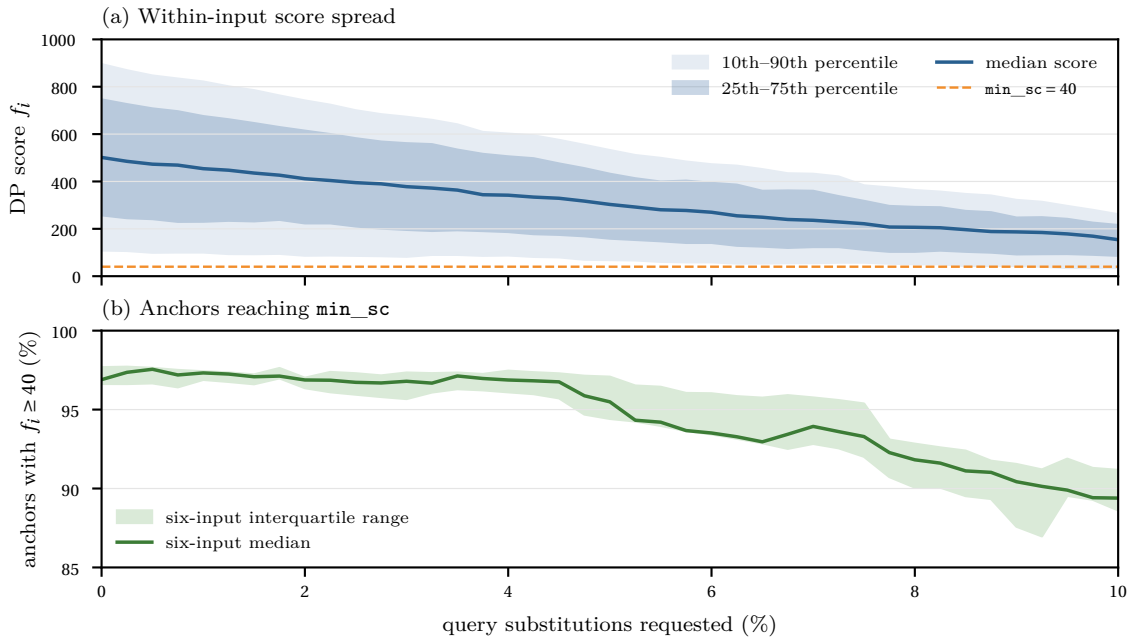


Figure 5.15: Cummulative results for all the experiments previously described (query lenghts 997-1002 bases). In the chart I take 6-input median (median over all 6 experiments with slightly different query lengths)

5.9.3. Does the bounded SIMD U limit help?

We analyzed the Run_L heuristis for chaining backtracking. Now I would like to apply similar measurements to determine how much limit on batch iteration U help us.

As reminder U is the maximum number of batches that consecutively do not improve the score for any chain during forward chaining 11 Minimap2 uses skip heuristic while we treat batch as a whole.

For SIMD width L, let E_L be the number of executed batches (same as in previous sections), Z_L the executed batches for which no lane satisfies $C_\lambda > H_\lambda$ (no score improvement), and Q_L the batches

in the complete window without the U stop. Similarily we define the firing rate and reduction rate:

$$\mathsf{NoUpdate}_L = Z_L / E_L, \qquad \mathsf{Reduction}_L^U = (Q_L - E_L)/Q_L.$$

The measurements on Figure 5.16 are pooled (summed over random-seed experiments), while Table 5.5 contains the separate runs for 0% and 10% divergence rates.

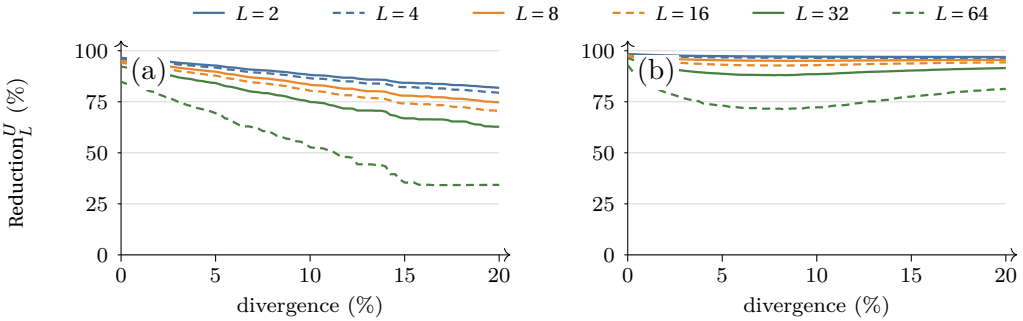


Figure 5.16: Pooled U-only reduction at each divergence for (a) random inputs and (b) sum over mosaic and degraded mosaic

Table 5.5: $\mathsf{NoUpdate}_L$ / $\mathsf{Reduction}_L^U$ (percent). Random uses $N = 4000$, a 4.5-Mbase target, 997–1002-base queries, and the same divergence levels as presented previously. Spaced copy uses $R \in \{1,2,4,8,16,32,64\}$. The Minimizer-island comb uses $m \in \{1,2,4,\ldots,1024\}$. Defaults are $k = 15$, $w = 10$, $S = 25$, and `max_chain_iter`=5000.

Subset	Input	$L=2$	$L=4$	$L=8$	$L=16$	$L=32$	$L=64$
0% divergence	Random occurrences	92.9/96.5	87.6/96.0	80.1/95.0	66.8/94.1	50.1/92.2	50.1/84.8
	Mosaic	92.9/98.4	87.5/98.2	80.0/97.8	66.6/97.3	49.9/96.4	49.7/92.9
10% divergence	Random occurrences	93.0/88.1	87.7/86.5	80.3/83.3	67.1/80.4	50.5/74.9	50.5/52.7
	Degraded mosaic	91.6/97.1	82.8/96.5	68.9/95.1	47.3/92.9	26.1/88.5	21.2/72.3
Pooled	Random occurrences	93.0/92.9	87.7/92.0	80.2/90.0	67.0/88.2	50.4/84.6	50.4/70.2
	Mosaic and Degraded mosaic	91.7/97.4	83.1/96.9	69.7/95.7	48.8/94.0	27.7/90.5	23.0/77.4
	Spaced copy	97.3/99.3	92.9/99.1	82.2/98.7	62.6/98.5	35.4/97.9	15.4/90.6
	Minimizer-island comb	93.2/53.1	88.0/47.4	80.7/36.8	67.6/28.8	51.0/17.8	27.0/0.0

As we can observe the firing rate is high for most of the cases. This is different to Run_L predicate, because in this case we don't care if we follow different chains as the same time (this is purpose of U limit). We only care about the fact that at some point the chain will end and we won't be able to improve it. The boundary limits like D checking the distances to diagonal will kick in, but we can resonably expect that we will always have some anchor nearby. This means that U heuristic is very useful for forward chaining. My constructions became problematic only for very high L where it's very likely that some chain will improve anyway.

5.9.4. BioseqDB `max_drop` analysis

The last part to analyse is if the max_drop heuristic can be useful for backtracking 5.7. As reminder, the BioseqDB backtracking procedure follows predecessor links until it reaches the sentinel $p_j = -1$ or an anchor that was already assigned to an earlier chain. Minimap2 can stop a predecessor scan earlier when its running score falls sufficiently far below the best score seen on that scan.

Formally let f_i be minimap2's dynamic-programming score for the best chain ending at anchor i. The minimap2 finds an eligible chain end e, let B be the number of steps made

by the BioseqDB-style baseline scan, which stops only at $p_j = -1$ or at a previously marked anchor and let b_m be the boundary reached after predecessor step m, and define

$$s_m = \begin{cases} f_e - f_{b_m}, & b_m \geq 0, \\ f_e, & b_m = -1, \end{cases} \qquad M_m = \max(0, s_1, \ldots, s_m).$$

Define the score drop at step m as

$$d_m = M_m - s_m, \qquad \Delta(e) = \max_{1 \leq m \leq B} d_m.$$

Thus d_m is the distance from the best score already seen in this backtrack to the score at the current step. The value $\Delta(e)$ is the largest such drop in the scan from end e. Minimap2 predicate will want to terminate the execution when $d_m > D$.

I tried measuring how frequently minimap2 fires the predicate. Table 5.6 contains very dissapointing results. By default max drop treshold is $D = 500$, but we didn't even observed that. The max delta is always zero because the score always improves along the chain.

T (Mbp)	query length (bp)	B	$\Delta_{\max}$
1.125	247–252	25 705	0
1.125	997–1002	109 715	0
1.125	3997–4002	440 525	0
1.125	15997–16002	1 757 774	0
4.500	247–252	25 705	0
4.500	997–1002	109 715	0
4.500	3997–4002	440 525	0
4.500	15997–16002	1 757 741	0
18.000	247–252	25 670	0
18.000	997–1002	109 715	0
18.000	3997–4002	440 525	0
18.000	15997–16002	1 757 741	0

Table 5.6: Default-$D = 500$ target- and query-length screen

Thankfully the wide-gap construction can help in this case. The 19,980-base target contains the same parts in the same order, but we insert a random 500-base. Figure 5.9(e) shows this construction. This means that we can get lots of anchors that doesn't match to improve the score at all. The "requested substituiton" rate refers to substitutions in the separated target fragments.

For panel (a) 5.17 I calculate how many steps we perform compared to the place where we could terminate execution (lower percentage means that the heuristic won't save anything). I don't mark the anchors so we always follow chains right to the end. Panel (b) 5.17 demonstrates that when marking chains the panel (a) don't show full truth. Max drop will save us scanned anchors, but because we won't mark the chains the actual work is larger (later we need to scan the anchors).

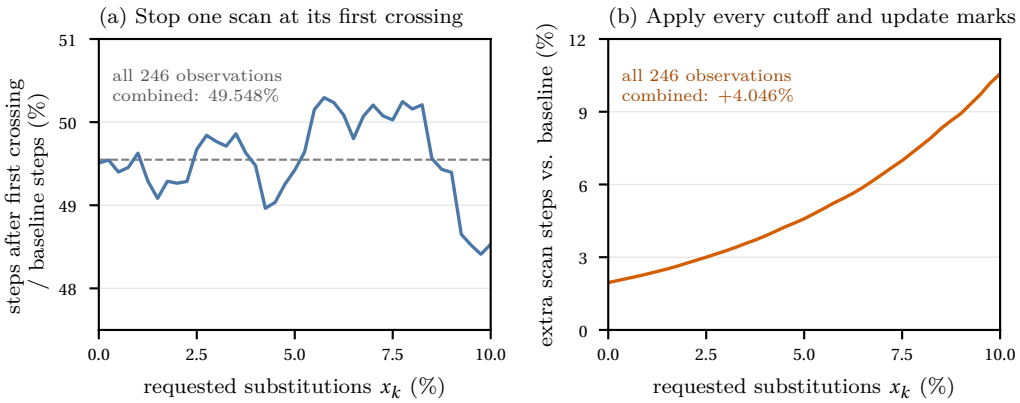


Figure 5.17: Scan-step accounting for Wide-gap spaced-copy at default $D = 500$. Its construction and exact dimensions are shown in Figure 5.9(e). Panel (a) counts the part of each fixed-start baseline scan after we reach max drop for first time. The dashed value is $\sum S / \sum B$ over all 246 observations. Panel (b) applies every cutoff and updates the marks, which increases the complete scan work. These are logical scan steps, not instructions, elapsed time, or a measured slowdown.

After consulting relevant work we see that max drop heuristic isn't used to reduce number of scanned anchors, but rather to prevent chaining unrelated fragments of sequences. The minimap2 will try to chain large gaps as unrelated fragments, which increases amount of work performed. This however is reported to increase matching accuracy Li, "Minimap2: pairwise alignment for nucleotide sequences"

In BioseqDB I didn't implement this heuristic in the SIMD algorithm at this point.

5.9.5. Conlusions

Final conlusions

From the experimental samples we can observe that in the random scenario:

- Run_L is beneficial to performance increase
- U limit is very beneficial to performance increase
- max_drop increases work done scalarily by minimap2, but according to literature increases quality of the alignment

Thus I decided to leave these heuristics in, while the max_drop can be implemented in the future (as observed on the random cases, it only even triggers in case of very large gaps, so it requires careful testing on real-world datasets as well as estimating how SIMD implementation will penalize the execution times)

5.10. BioseqDB SIMD performance comparison

I perfomed intermediate testing to verify if predictions coming from synthetic results can be correct.

The test used $|T| = 500Mbps$ and $|Q| = 5Mbps$ sequences with just one $N = 1$ occurence (this means that this is very optimistic case of random scenario)

TODO

Replace this section with final results or remove it entirely (the b panel is misleading and there's no interpretation of the results.). For anyone reading: this is just old test I found to illustrate performance so it's probably outdated and misleading because for backtracking there is single SIMD algorithm. Benchmarking framwork resuses labels but it's the same algorithm in panel B for B1/B5 and it makes no sense at all

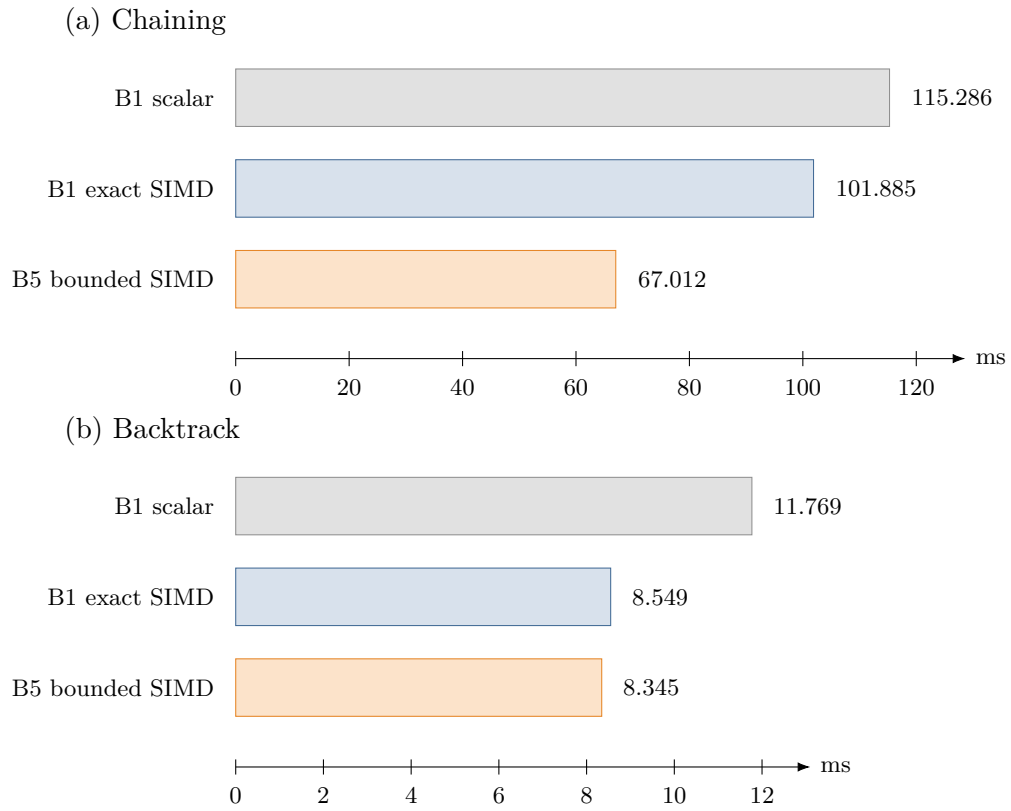


Figure 5.18: Recorded times for the three CPU modes

The figure above depicts only two runs and presents only quick prototyping test measurements to judge if we are going in right direction.

5.11. Range minimum query chaining

The chaining score model is described in Section 5.5.1 gives an ability to better express relationships between anchors we want to optimize. However it's more complex to calculate than simple linear scoring model. As noted in equation 5.8, we can compute GlobalScore(i)

independently for each anchor author of minimap2 did in minigraph paper Li, Feng, and Chu, "The design and construction of reference pangenome graphs with minigraph." The problem then is reduced to finding the maximum among all values in continous segment of anchors.

Range minimum query cp-algorithms contributors, Range Minimum Query (RMQ) algorithm is used to calculate the linear score $h(j)$ for anchor, while minimap2 later calculates actual score $f_j + E(j,i)$ (and if the RMQ result is not sufficiently "good", then it tries again with different parameters)

$$h(j) = f_j + \frac{\lambda_g}{2}(r_j + q_j). \tag{5.20}$$

5.11.1. Minimap2 RMQ

I didn't initially found papers going into details of minimap2 RMQ's implementation, so here I present detailed description of the algorithm. The minigraph paper Li, Feng, and Chu, "The design and construction of reference pangenome graphs with minigraph" describes only the high-level approach, without going into details of the implementation. Minimap2's RMQ is implemented using two AVL trees cp-algorithms contributors, Segment Tree. First of them is used to query anchors in the interval $0 < r_i - r_j \le D$. Each time minimap2 removes nodes from the tree if either they go outside of this interval or node count reaches C (the total limit of nodes per tree). The second tree has the same structure, but is used to query anchors in the interval $0 < r_i - r_j \le D_{\text{in}}$ with the intent to have $D_{\text{in}} \le D$ so if we get an anchor candidate that is not sufficiently "good", we can then perform another query on second tree, where we will get scores for smaller neighbourhood of an anchor. I will call this second lookup the "local scan". Minimap2 D_{in} can be set to 0 to disable second AVL tree entirely.

While the original anchor array is sorted by increasing reference coordinates r_i, the AVL trees order the node primarily by increasing query coordinate, then by increasing anchor index in the array (pairs (q_j, j)). As we want to prevent chaining anchors with the same reference coordinate, we will not add single anchor to the trees (a "reference group" is the continous sequence in the anchor array that has the same reference position, but different query positions). We will add entire reference group to the tree at once. This is implemented via InsertCompletedReferenceGroup(T, a, f, i) pseudocode function which checks if current anchor reference coordinate is different to the saved one.

The trees also need to remove anchors that go outside reference coordinates i.e $0 < r_i - r_j \le D_{\text{in}}, D$. This is implemented via RemoveExpiredEntries(T, a, i, D, C) pseudocode function. Such operation could be implemented by visiting each node in the entire tree, however internally minimap2 saves pointer (cursor) to the first anchor in the window and moves it when going to next anchor. This results in better reemoval theoretical complexity.

The last complex operation is local scan. After best candidate j^* is selected from the main tree T, we calculate it's score $c_{j^\star} = f_{j^\star} + E(j^\star, i)$ and accept it only when the anchor is "exact":

$$\delta(j^\star, i) = 0 \quad \wedge \quad g(j^\star, i) \le \ell_{j^\star}. \tag{5.21}$$

meaning that two anchors advcance equally and no more than k-mer span (as discussed earlier $k = \ell_{j^\star}$ for HPC disabled 4.12). If that's not the case, then minimap2 runs the local scan. The scan goes through all anchors in the query coordinate interval $q_i - D_{\text{in}} \le q_j \le q_i - 1$. This is implemented by first finding the first anchor in the tree with largest q_i value that is strictly less than q_j and then iterating candidates in decreasing order of q_j value (this is ordinary tree traversal). The scan itself is implemented via ScanLocalInterval$(T_{\text{in}}, i, D_{\text{in}}, B, S, f, p, t)$ pseudocode function and is very similar to forward chaining loop in minimap2 scalar algorithm 10. The only difference is that in the forward chaining loop without RMQ minimap2 iteration

was driven by *max_iter* limit for anchors not sorted by query coordinate, while in the RMQ scan we don't have *max_iter* limit (the minimap2 skip counter is used in the same way - it stops iteration on anchors that don't improve score several times in a row).

Theoretical complexity

In the previous forward chaining loop 10 we iterated over all anchors and bounded the number of iterations by *max_iter* limit. In the simplest calcualtion we can say that number of candidates is bounded by $\Theta(n)$ for one anchor, so entire pass does $\Theta(n^2)$ work. In case of RMQ each insertion and deletion (inserting reference groups and removing expired anchors) is amortized to $O(\log n)$ work for each anchor (each anchor can be added and removed at most once), so if the number of the candidates is the same, then overall work is bounded by $\Theta(n \log n + n^2)$. The RMQ is also inherently less friendlier to efficient pipelining and cache locality as we will need to jump across the tree frequently and it uses linear scoring model which can be less accurate. The benefit comes from the fact that RMQ can be more efficient with larger boundary limits, so it can be used for cases of long sequences with large insertions/deletions with lots of noise. Li, Feng, and Chu, "The design and construction of reference pangenome graphs with minigraph"

Figure 5.19 illustrates how RMQ chaining works.

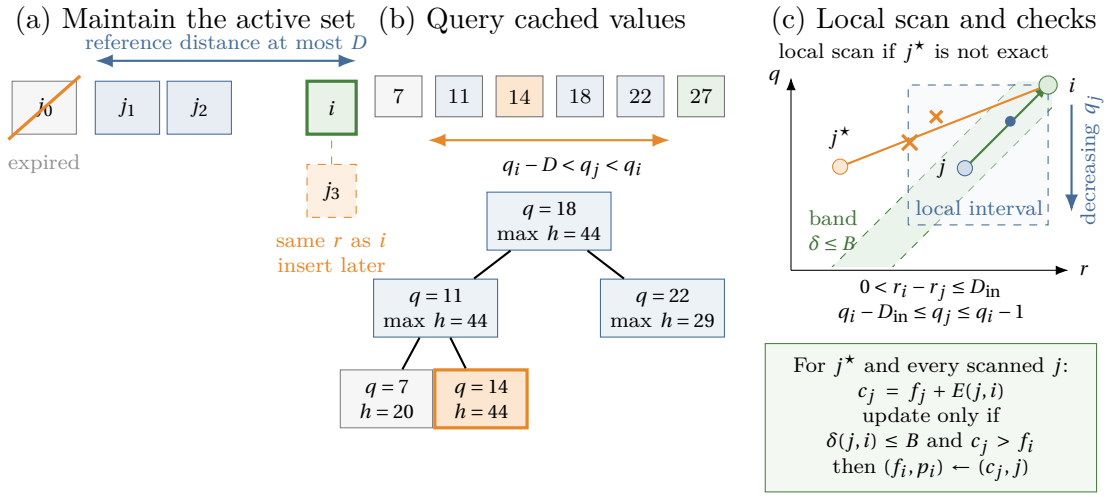


Figure 5.19: Minimap2 RMQ for one target. (a) We maintain the range of reference coordinates in the array to know where the window starts (what is the anchor with lowest reference coordinate that still fits in the $r_i - D$ window). (b) Select the best anchor according to linear scoring model$j^\star$; "max h" is the largest value in a node's subtree. (c) If the selected anchor $j^\star$ is not exact (Equation 5.21), then we run the local scan. The blue rectangle is the local interval the green band satisfies $\delta \le B$, orange crosses mark rejected candidates, including one inside the local interval. A scanned link (blue dot) replaces the current result only if both checks below pass. The local scan can stop early and do not iterate over all anchors in the local interval if for example skip counter S is reached.

Algorithm 15 shows how RMQ forward chaining is implemented inside minimap2. RangeBest function returns the anchor with the largest $h(j)$ in the interval, or $\perp$ if the interval contains

no retained anchor, while CompleteLink function calculates $E(j,i)$ from Equations 5.4–5.5, together with δ and the exact (local scan) condition in Equation 5.21.

Algorithm 15 Minimap2 RMQ forward chaining

Require: sorted anchors $a_0,\ldots,a_{n-1}$, distance D, inner distance D_{in}, bandwidth B, tree cap C, skip limit S
Ensure: scores f and predecessor indices p

```
function Minimap2RMQForward(a, D, D_in, B, C, S)
    D ← max(D, B),   D_in ← min(D, max(0, D_in))
    T ← ∅,   T_in ← ∅   augmented AVL trees
    t ← (0, ..., 0)   B1 marks for the optional local scan
    for i ← 0 to n − 1 do
        InsertCompletedReferenceGroup(T, a, f, i)
        if D_in > 0 then
            InsertCompletedReferenceGroup(T_in, a, f, i)
        RemoveExpiredEntries(T, a, i, D, C)
        if D_in > 0 then
            RemoveExpiredEntries(T_in, a, i, D_in, C)
        f_i ← ℓ_i,   p_i ← −1
        j* ← RangeBest(T, (q_i − D, INT32_MAX), (q_i, 0))
        if j* ≠ ⊥ then
            (e, δ, exact) ← CompleteLink(j*, i)   e = E(j*, i)
            if δ ≤ B ∧ f_j* + e > f_i then
                f_i ← f_j* + e,   p_i ← j*
            if ¬exact ∧ T_in ≠ ∅ ∧ q_i > 0 then
                (f_i, p_i) ← ScanLocalInterval(T_in, i, D_in, B, S, f, p, t)   Algorithm 16
    return f, p
```

Algorithm 16 shows how local scan is implemented. I didn't want to show other tree maintenance operations as there aren't many valuable details to discuss (this is usual AVL tree maintenance).

Similarily to minimap2 forward chaining we use t_j array to mark visited candidates i.e mark $t_j = i$ means that a candidate already tested for target i points to j through its saved predecessor. It must be noted that the for loop in the local scan algorithm 16 utilises the fact that anchors are sorted by query coordinate so we can just go to the leftmost node in the interval and iterate each leaf from left to right (I'm reiterating this to showcase that the for loop hides this complexity in hte pseudocode).

Algorithm 16 Minimap2 local scan after the primary RMQ query

Require: maintained inner tree T_{in}, target i, local limit D_{in}, bandwidth B, skip limit S

Require: completed earlier scores and predecessors f, p, marks t, and current result (f_i, p_i)
Ensure: updated (f_i, p_i) and marks t

```
function ScanLocalInterval(T_in, i, D_in, B, S, f, p, t)
    s ← 0
    for retained j ∈ T_in with q_i − D_in ≤ q_j ≤ q_i − 1, by decreasing (q_j, j) do
        c ← f_j + E(j, i),   δ ← |d_r(j, i) − d_q(j, i)|
        if δ ≤ B then
            if c > f_i then
                f_i ← c,   p_i ← j,   s ← max(0, s − 1)
            else if t_j = i then
                s ← s + 1
            if s > S then
                break
            if p_j ≥ 0 then
                t_{p_j} ← i
    return (f_i, p_i)
```

5.11.2. RMQ re-chaining in the pipeline

Normally Minimaps2 disables RMQ for chaining by default, the manual recommends`--rmq` for alignment through long insertions and deletions Li, Minimap2 manual, but at the same time RMQ is cheaper so minimap2 can use it to perform "re-chaining".

RMQ re-chaining

RMQ re-chaining means running chaining again with a larger bandwidth to try to connect chains across long gaps. This is optional step that runs in minimap2 after the forward chaining and chaining backtracking - after step 3 5. Minimap2 uses array A which is sorted by radix sort by reference coordinates again and provided to RMQ chaining algorithm. Please note that this means that minimap2 does some duplicate work - first it groups anchors by the chain they belong to (placing them in A array), but rechaining sorts anchors again. Minimap2 also discards f and p arrays from first chaining step. RMQ re-chaining will then work on the subset of original input anchors from seeding (not every anchor will form a chain that will be accepted by its length and score limits 13). As RMQ re-chaining is special case not enabled by default in minimap2, it wasn't in scope of this thesis - I didn't measure how many anchors are retained and how RMQ re-chaining affects quality of alignments and performance.

BioseqDB does not support re-chaining

I don't provide any specialized stage in pipeline for re-chaining. Some of the software in literature support increasingly precise queries to improve alignments iterativelyKallenborn et al., "GPU-accelerated homology search with MMseqs2." This can be implemented in the future.

Normally in minimap2 RMQ re-chaining will be only enabled iff all conditions are satisfied:

- RMQ re-chaining is enabled only for specific presets (assembly presets), so user have to select specific preset or use `--rmq` flag explicitly

- There's more than one chain (easy to satisfy)
- There's only 1 query in the mapping call
- Either query length minus span of first chain is greater than 1000 bases or span of first chain is greater than 10% of query length (very long insertions and deletions)

The conditions above were created by me after inspection of source code, so the list might not be complete, but it illustrates why re-chaining wasn't first priority for me.

5.11.3. RMQ in BioseqDB

In BioseqDB I implemented the same algorithm as minimap2 with some differences:

Table 5.7: Differences between minimap2 and BioseqDB RMQ

	Minimap2	BioseqDB
Local scan	Optional scan with complete scores and the B1 skip rule	Local scan is implemented by disabled by default by setting $D_{\text{in}} = 0$
Query interval	Composite bounds in Algorithm 15	Includes both $q_i - D$ and q_i
Bandwidth check	Additionally rejects anchor when $\delta > B$	Does not check that
RMQ re-chaining	At most one extra forward run and backtracking step with a larger bandwidth	No any re-chaining stage is implemented
Backtracking	Uses the minimap2 backtracking algorithm 13	Performs scalar BioseqDB backtracking which is similar algorithm to minimap2, however we could potentially use SIMD backtracking (backtracking isn't separate pipeline stage, so SIMD chaining selects both forward/backtrack in SIMD, while RMQ has it's own scalar backtracking)

⚠ Other algorithmic limitations

There are some other limitations:

- Including $q_j = q_i$ allows a link with no query advance. BioseqDB searches from $(q_i - D, \text{INT64_MIN})$ to $(q_i, \text{INT64_MAX})$, so every anchor at either query boundary is eligible for selection (it should have minor impact on performance)
- Omitting the bandwidth reject i.e $\delta > B$ allows a link farther from the diagonal than the configured B (this was ommited due to simpler code and RMQ having lesser priority than oder algorithms)

I didn't measure effects of these limtations neither on performance nor on quality of alignments.

Future work

The backtracking for RMQ can be removed and I could use SIMD backtracking. This would require separating forward/backward chaining into separate pipeline stages. Moreover in the future I can use specialized GPU acceleartion for RMQ (as noted before G3SA contains kernels for RMQ which weren't used). There exists some liteature Kreis et al., GPU-RMQ: Accelerating Range Minimum Queries on Modern GPUs discussing GPU acceleration for RMQ problems using ray tracing and CUDA cores (one limitation is that in the paper the RMQ base array is static, so it would require additional design work to design GPU-friendly algorithm).

5.12. G3SA GPU chaining

TODO

This is not done at all!

5.13. BioseqDB implementation

BioseqDB exposes four chaining choices through C++ and PostgreSQL. The PostgreSQL setting is

`bioseq.aligner.chaining.algorithm`.

The `chaining_algorithm` field inside per-call `aligner_options` uses the same values 0,...,3. Choice 0 is the default in both interfaces. The chosen value determines how f_i, p_i are calculated. In the generic C++ chain-stage implementation, backtracking runs on the CPU except for Vulkan queries that request coordinates only. PostgreSQL choice 3 uses the complete Vulkan pipeline described below.

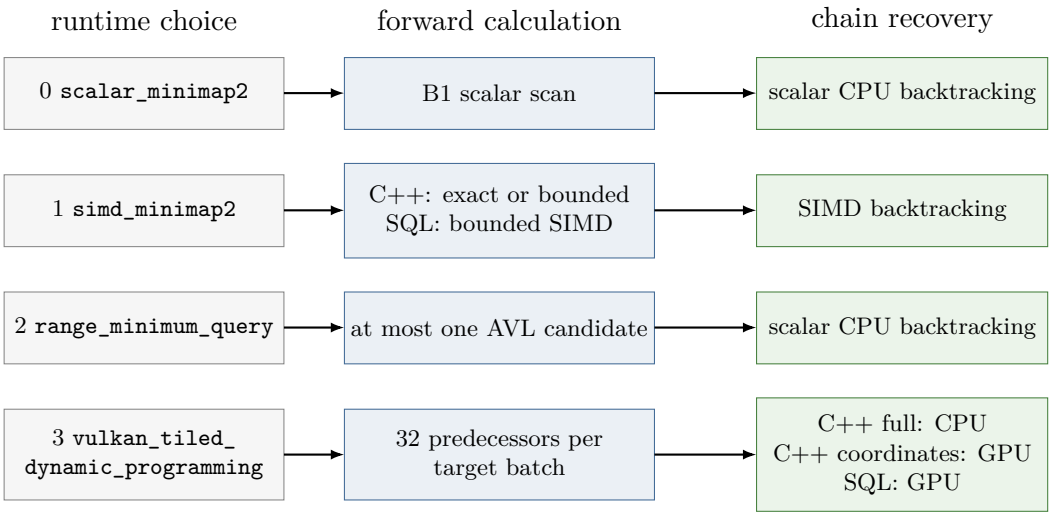


Figure 5.20: The four BioseqDB chaining choices. Choice 1 has exact and bounded internal modes. PostgreSQL always uses its bounded mode. Choice 3 shows the generic C++ recovery route. PostgreSQL performs its chain recovery on the GPU.

scalar_minimap2, value 0. This is the PostgreSQL and high-level C++ default. It runs the scalar B1 predecessor scan and BioseqDB scalar backtracking.

simd_minimap2, value 1. The C++ API lets a caller choose one of two internal modes:

- `exact_visited` reproduces BioseqDB scalar B1 predecessor choices.
- `bounded_batch_approximate` is bounded SIMD B1. For SIMD width L, it may stop after $\lfloor \texttt{max_chain_skip}/L \rfloor + 1$ consecutive batches in which no lane improves its saved lane score.

PostgreSQL exposes choice 1 but not these two mode names. It always uses the bounded mode. Both modes use the SIMD backtracker described above. Bounded mode requires positive `max_chain_skip` and `max_chain_iter`. Otherwise choice 1 returns an error.

range_minimum_query, value 2. This selects BioseqDB's CPU AVL implementation. The current route uses only the primary tree. For each target, that tree returns at most one candidate. If one exists, BioseqDB calculates $f_j + E(j,i)$. Scalar CPU backtracking follows.

vulkan_tiled_dynamic_programming, value 3. This selects the Vulkan implementation. Its work split and chain recovery are described below.

A selected SIMD or Vulkan algorithm does not silently change to scalar chaining when it cannot run. It returns an error. Dispatch ignores the internal field `chaining_mode`. The four-value choice shown in Figure 5.20 does. None of these choices runs G3SA's two-pass source sweep.

5.13.1. How BioseqDB RMQ differs from minimap2

BioseqDB stores anchors in an AVL tree keyed by query coordinate and records the best $h(j)$ in each subtree. The current choice-2 route leaves the optional local-repair distance at zero, so no repair scan runs. Neither `max_chain_skip` nor `max_chain_iter` changes this forward calculation.

For configured main distance D, the BioseqDB query uses inclusive keys

$$K^{\mathrm{lo}}_{i,\mathrm{Bio}} = (q_i - D, \mathrm{INT64_MIN}), \qquad K^{\mathrm{hi}}_{i,\mathrm{Bio}} = (q_i, \mathrm{INT64_MAX}).$$

These keys permit anchors on both query-coordinate boundaries. BioseqDB calculates the selected link's diagonal width but does not compare it with the configured bandwidth. It runs RMQ once and omits minimap2's optional wider second chaining pass.

Choice 2 therefore does not reproduce minimap2 RMQ.

5.13.2. BioseqDB Vulkan work split

Like minimap2, the Vulkan algorithm fixes one target i and tests earlier predecessors j. It does not fix one source and update many targets as G3SA does. One contiguous anchor range assigned to the shader is handled by one 32-lane workgroup. After a workgroup finishes one range, it may advance by the total number of workgroups and process another range. Within one range, it finishes targets one after another. Its 32 lanes calculate up to 32 complete candidate scores. A reduction across the lanes selects the batch winner, then updates scalar f_i, p_i. When `max_chain_skip` is positive, it stops after

$$\left\lfloor \frac{\texttt{max_chain_skip}}{32} \right\rfloor + 1$$

consecutive batches do not improve the target. A zero value disables this early stop. It also limits the predecessor scan to at most 3,000 anchors, even when `max_chain_iter` is larger. These are not the scalar B1 stopping rules.

For a range with $n > 131{,}072$ anchors, the implementation sets

$$s_{\text{thin}} = \left\lceil \frac{n}{65{,}536} \right\rceil,$$

keeps anchors $0, s_{\text{thin}}, 2s_{\text{thin}}, \ldots$, and always keeps the last anchor. Chaining uses only this retained set. In the generic C++ entry point, full-result queries calculate f, p on Vulkan and backtrack on the CPU. Coordinate-only queries examine at most 16 unvisited chain ends in descending score-and-index order and return at most one accepted path.

In the complete PostgreSQL Vulkan route, choice 3 performs chaining and trace selection on the GPU. This route can also keep a bounded CIGAR calculation on the GPU when Vulkan extension is enabled. It does not enter the generic CPU backtracker. If its requirements are not met, PostgreSQL returns an error rather than retrying with a CPU or SIMD algorithm.

The GPU route measured in the campaign. The published GPU campaigns also contain hybrid routes. The local result annotations identify GPU sketching and chaining alongside host SIMD Block alignment. For the indexed local MPF case with $k = 20, w = 15$, they also explicitly identify CPU chain recovery. The published Vast.ai campaign likewise calculated chaining scores on the GPU, then recovered chains on the CPU and ran the selected CPU base aligner. These elapsed times therefore include host stages as well as GPU execution. The complete Vulkan route described above has a different execution boundary, so its design cannot be used to attribute all of the campaign time to GPU kernels.

Chapter 6

Final alignment; prefiltering and postprocessing

6.1. From a seed chain to a base-level alignment

Chaining identifies a plausible correspondence between query and target positions. Final alignment resolves the bases between the anchors: it assigns a score and, when requested, reconstructs the substitutions, insertions and deletions as a CIGAR. This is the dynamic-programming problem introduced in Section 2.2. BioseqDB divides a chained region into an extension on each flank and fills between selected anchors. Consequently, the number and geometry of these subproblems matter as well as the total read length.

6.1.1. The two CPU alignment contracts

KSW2 and Block Aligner both use SIMD, but they do not search the same set of matrix cells or implement the same scoring contract. KSW2 supports two affine gap penalties, a fixed band and Z-drop stopping. Z-drop allows extra score loss when the path moves away from its previous diagonal, so that a long gap need not stop an extension immediately. The ordinary gap-fill path also uses an approximate maximum; requests for an exact edit count disable its band and Z-drop restrictions.

Block Aligner uses the first affine gap penalty and a block that moves, grows and shrinks through the matrix Liu and Steinegger, "Block Aligner: an adaptive SIMD-accelerated aligner for sequences and position-specific scoring matrices." Fills between anchors use its global mode; flanks use its own X-drop rule, which requires two consecutive low-scoring steps. The current pipeline supplies minimum and maximum block sizes of 16 and 32. Increasing these limits can explore more cells around gaps, at the cost of more work and trace storage. Neither the read length nor an end-to-end timing alone establishes which limits should be changed.

The implementation also tries to prove that an equal-length gap has a unique straight-diagonal optimum before running Block dynamic programming. A successful proof produces the score and trace directly; other gaps still use the adaptive recurrence. This shortcut has its own geometry, sequence-storage and scoring restrictions. Both aligners otherwise combine dynamic programming, trace storage, traceback and CIGAR construction in one invocation. A timer surrounding that invocation therefore measures more than output formatting. Equal match counts do not establish equal CIGARs, scores or search effort between the two contracts.

6.2. Why filter after chaining but before alignment?

TODO

Write prefilter!

6.3. Postprocessing

The pipeline derives several different summaries from an alignment candidate. Seed divergence estimates how much sequence differs from the pattern of matching minimizers. An exact edit count instead comes from the selected base-level path. Mapping quality describes confidence in the chosen mapping relative to competing candidates. These values answer different questions; none can replace the other merely because they are all reported with a match.

6.3.1. Seed divergence and position order

The current divergence estimator counts query minimizers at each position and forms cumulative counts. It can then test whether a position contains a minimizer and count minimizers in an interval without sorting the emitted positions. This matters because a SIMD sketch can emit positions in an order different from their order along the query, as the historical estimator problem in Section 4.9.2 illustrates. Building the count array takes work proportional to the query length plus the number of minimizers. The estimator runs before final alignment; it is separate from the fused alignment and CIGAR work described in Section 6.1.1.

Chapter 7

Sequence representations and lazy evaluation

7.1. Designing sequence abstraction

Suppose user stores large genome sequence in one row, but only wants to align specific small part of it (let's say we know location of one protein-coding human gene and are interested only in finding occurences of that gene). It would be very wasteful if I didn't provide built-in functionality to load only that small region. This region could be stored on disk, so we only have it's locator poiting to a specific chunk in the table.

Physical storage

"Locator" refers to a small structure holding sequence metadata and pointer to its actal location in the storage system. More on is explained in physcial storage chapter 9.1. For now, I assume it's only an opaque pointer to sequence and use term "hydration" describing the process of converting it into a sequence itself or its fragment (as continous byte array in RAM).

On the other hand, we can load sequences from fasta file, where parser can be writing to RAM, so our sequence could refer to location in memory, where all sequences are pooled into one memory region to avoid constant allocation/reallocation. In yet another case, in tests we would like to use standard library containers to prepare and examine sequences (potentially even somebody could directly call some internal BioseqDB functions as standalone alignment library). This created a design challenge: same sequence can arrive as relationship locator, region in memory pool in RAM, can be provided as a whole as standrad container or as only a fragment (if user wants to operate only on specific slice or if it does not fit in RAM or RAM is limited, so we cannot fit entire sequence directly there), while these fragments should also be shareable, so parallel alignment to the same sequence doesn't have to load them multiple times.

There are several existing libraries for interfacing with genomic sequences. SeqAn[1] is a C++ library that provides modern interface for composing efficient sequence-analysis software from generic C++ components. Its library includes sequence representations, indexes and configurable alignment algorithms, with SIMD alignment, but it has it's own limitations.

[1] Reinert et al., "The SeqAn C++ template library for efficient sequence analysis: A resource for programmers"; SeqAn Project, SeqAn Manual.

However SeqAn SIMD algorithms are mostly limited to auto-vectoriztion provided by the compiler (which doesn't have to be portable in most cases) and offers very narrow set of features for custom pipelines. I would be hard to directly reuse it for implementation of mentioned feautres. Using SeqAn would require deep integration, that's why I decided to address the design challenges by designing own C++ system of templates with different specializations driven by C++ concepts (inspired by nice-looking HPC MPI code Uhl, Schimek, and Brommer, Concepts in Practice: C++ MPI Bindings for the HPC Ecosystem. From a Standardizable Core to a Composable Interface).

SeqAn compatibility

Because the architecture of BioseqDB pipeline is very custom, I decided to only use SeqAn's alphabet layer[a] and utilities exposed by its public API. This means that libraries using SeqAn code as base, could be slightly adjusted to be able to work with BioseqDB pipelines.

[a] SeqAn Project, Alphabets in SeqAn.

Table 7.1 lists every specialization of all BioseqDB sequence representations, to showcase how they are used in different scenarios. SeqAn offers abstractions over lazy sequences as well SeqAn Project, SeqAn Tutorial: Sequences In-Depth, but they're not as sophisticated as in my case. The lazy interface is really important for BioseqDB to improve it's performance utilising late-materialization strategies similar to those described in literature[3]

Table 7.1: Iterfaces for sequence representations in BioSeqDB.

Template	Specialization, and when it is selected	Description and usage context
`compact_sequence_range<R>` This is the core type representing sequence that resides completely in memory. Template allows only four available specializations depending on what `R` is and each of them have entirely separate handling logic.	`R` is a container of alphabet values, such as `std::vector<dna4>`	This is only adapter for existing sequence containers to provide compatibility with SeqAn and standard library containers. There is no alphabet packing and the type borrows `const R&` Adapter just provides common sequence interface. Adapting one costs $O(1)$ as we proxy all operations to the underlying container
	`R` is a packing configuration `sequence_custom_alphabet<A, U,b>`	Bitpacked vector that stores sequences according to alphabet rules passed as type paramter. Owns storage. Can convert to and from other bitpacked or not-bitpacked sequences, but if constructed from non-bitpacked sequence, then constructor costs $O(N)$
	`R` is an SeqAn alphabet type, as in `compact_sequence_range<dna4>`	The same as `sequence_custom_alphabet<A,U,b>`, but has sensible defaults for used underlying alphabet's storage unit width etc. Commonly used in tests. Code interfacing directly with BioseqDB C++ API should use this specialization.

[3] Chernishev et al., "A Comprehensive Study of Late Materialization Strategies for a Disk-Based Column-Store."

Template	Specialization, and when it is selected	Description and usage context
	`R` is `compact_sequence_stdc_instance<...>`	Holds only reference to the underlying object. Adapter to convert `compact_sequence_stdc_instance<...>` to `compact_sequence_range<R>`. `compact_sequence_stdc_instance<...>` is a type that comes from `compact_sequence_stdc<...>` - described below Used internally
`compact_sequence_stdc<...>` Represents standard layout C type cppreference.com contributors, C++ named requirements: StandardLayoutType that is used to type-erase `compact_sequence_range<R>` types i.e allow user to have one type for storing possibly multiple different sequences types. Metadata like alphabet type and storage unit width are stored inside the struct and specialized template dispatch lambda is called to convert it back to the `compact_sequence_range<R>` type. This is heavily used by Postgres code to store sequences as C structures.		Owns everything and stores fixed header that records struct (self) size in bytes (compatible with Postgres varlena), alphabet code, unit width, padding, and symbol count Used heavy by Postgres code as we cannot store template instances directly and need to hold them in one object that converts type parameters of templates to runtime values (with correct runtime dispatch). If you want to store sequences of different alphabets in one single container, you should use this.
	`compact_sequence_stdc_instance<Stdc,A,W>`, internal type obtained when dispatching the type at runtime	Carries nothing, this is primarily runtime tag Shouldn't be used directly, one should receive `compact_sequence_range<compact_sequence_stdc_instance<...>>` when using dispatch methods of the stdc template
`lazy_sequence<R, Scanner>` Represents sequence that can be loaded in fragments by some storage provider represented by type parameter `Scanner`	`Scanner` has static constexpr method `is_fragmented()` that returns false (this means that Scanner always holds entire sequence in memory and there's only one fragment). This is used as compatibility layer so you can directly create lazy sequence from compact sequence range instance. I have `resident_sequence_scanner<U>` for trivial implementation of non-fragmented scanners so you can write `lazy_sequence<R,resident_sequence_scanner<R>>` and don't have to implement scanner type on your own.	Holds single non-owning pointer to `compact_sequence_range<R>` Used as adapter to convert compact sequence range to lazy sequence. All primary pipeline operations should accept it as parameter

Template	Specialization, and when it is selected	Description and usage context
	`Scanner` is fragmented (`is_fragmented()` returns true)	Ows scanner, internal small cache for sequence fragments that are neither copyable or movable Allows iteration over data source of sequences that doesn't fit into memory at once. This is primary specialisation for loading sequences from PostgreSQL relations. The Postgres side implements large scanner type that tells the lazy sequence template how to fetch blocks of sequences
`bit_casted_view<Base, Out>` Only legal for few types of alphabets and storage widths.	Special non-owning type that is used to provide zero-cost changes to pitpacking. It allows to cast bitpacked sequences to different storage widths i.e cast sequence encoded on 32-bit int to be read as sequence encoded on 64-bit or 16-bit or 8-bit units. There are special requirements for alphabet to be able to do that (discussed below).	Holds only reference to underlying compact sequence range This is adapter type that provides zero-cost conversion between different specialisations of compact sequence range. Used for example by SIMD algorithms to have specific vector lane widths
`lazy_sequence_view<A>` Type-erased, non-owning facade over every representation listed above. It is templated on the alphabet only, so exactly one type name crosses module and Postgres boundaries. The object itself is two pointers: opaque backend state and a static `backend_ops` table that holds `bp_size`, four orientation-specific `read_at` slots (read forward or backward, write forward or backward), `try_resident_packed`, `copy_packed_sequence` and `worker_shareable`. Specialization here means which callback table is installed.	`owned_sequence_adapter<R, Scanner>`, created by `make_lazy_sequence_view` for a concrete `lazy_sequence<R,Scanner>`	Complete table over an owning lazy sequence. This is the table behind every lazy sequence that pipeline code receives, where Postgres provides pointers to actual functions.
	`compact_range_adapter<R>`, created by `make_lazy_sequence_view` for `compact_sequence_range<R>`	Used as convenient adapter to convert compact sequence range to lazy sequence view without worrying about scanner type and ownership. Useful for testing.
	`facade_adapter<A>`, created by `make_pg_sequence_call_owner` on the Postgres side	This struct holds Postgres Scanner state - it is either a detoasted flat value or a piece tree root together with its read session, because sequences need to be represented as trees of slice operations

Template	Specialization, and when it is selected	Description and usage context
	`resident_adapter<A>`, created by `admitted_packed_sequence::view()`	Postgres Scanner for sequence only holds descriptor for already materialized sequence
`retained_lazy_sequence_view<A>` Has only one form. This type pairs view mentioned above with opaque lifetime pointer. It's used in Postgres code for management of sequence lifecycle during DB operations	Constructed from a view, an opaque lifetime pointer and a `noexcept` release callback, so there is nothing to specialize	Used wherever a view has to outlive the expression that produced it. The Postgres read session allocates the owner in its own memory context and registers it there, so the retain is dropped even when Postgres unwinds with `longjmp` and skips C++ destructors. This means that this type is specific version of `std::unique_ptr` that allows us to provide custom lifecycle management during scans.
`admitted_packed_sequence<A>` Supporting type to propagate sequence along with memory permit (memory permits are used in BioseqDB to manage large memory regions for sequences)	Created during materialization by `materialize_packed_sequence`. In BioseqDB there's a little sequence access planner that is responsible for deciding if we can materialize it all at once and directly or it needs to be streamed. `plan_sequence_access` picks this type when a contiguous resident region and we cannot use existing materialization (for example we access specific nucleotide that doesn't align well with storage unit boundary, but the GPU algorithm requires continous region of sequence, so we need to copy it)	Owns the permit, the aligned buffer and its descriptor together, so accounted bytes are released with the object. Its `view()` returns an ordinary view over `resident_adapter<A>`, which is how SIMD and GPU kernels obtain contiguous packed input from a sequence that is tree-backed or fragmented

7.2. Packed symbols and a common C++ interface

The template-driven design allows usage of any arbitrary alphabet for sequences, though for now I only support DNA4 and DNA5.

Sequence is encoded using bitpacking - the symbols are converted into integers values with specified fixed number of bits per symbol, then several symbols are packed into one single storage unit. Storage unit can be integer of any physical size. This is generalisation of bitpacking strategies commonly used in other software like packed-seq Groot Koerkamp and Martayan, packed-seq: Bitpacked sequence trait and implementation Rust library developed by author of SIMD minimizers paper. packed-seq Rust library offers non-alphabet-generic packing Groot Koerkamp and Martayan, packed_seq crate documentation. My design was more inspired by SeqAn approach where packing container is available, but very limited in features SeqAn Project, SeqAn Manual.

SeqAn alphabet assigns each symbol an integer called its rank. For DNA4, the four ranks are $A = 0$, $C = 1$, $G = 2$, and $T = 3$. A rank is an encoding of a symbol. Each `compact_sequence_range<R>` type needs to know certain properties of the encoding that are held

(using CRTP) in `compact_sequence_range<R>::traits` type. In my storage symbols never spill to other storage units i.e to retrieve some symbol you always need to access at most one storage unit. The symbols in the storage unit are always packed in "little-endian" order i.e the first symbol is in the lowest bits of the storage unit and the last symbol is in the highest bits. The properties in traits type are either types or constexpr static expressions/functions:

- `alphabet_type` - type of the alphabet
- `storage_unit` - type of the underlying storage unit (usually `uint8_t`, `uint16_t`, `uint32_t`, `uint64_t`)
- `alphabet_size` - number of symbols in the alphabet
- `unit_width` - width in bits of the storage unit: sizeof(storage_unit) $\times 8$
- `bits_per_char` - stored bits per symbol. The default packed representation uses $\lceil \log_2(\text{alphabet_size}) \rceil$, giving two bits for DNA4 and three for DNA5.
- `letters_per_unit` - number of symbols that can be packed into one storage unit: $\left\lfloor \frac{\text{unit_width}}{\text{bits_per_char}} \right\rfloor$
- `unit_used_bits` - number of bits used in the storage unit (for alphabet sizes that are not a power of two, some bits must stay unassigned): letters_per_unit $\times$ bits_per_char
- `unit_leftover_bits` - number of bits leftover in the storage unit: unit_width$-$unit_used_bits
- `char_mask` - mask for the last symbol in the storage unit
- `rev_char_mask` - mask for the first symbol in the storage unit
- `used_storage_mask` - mask for the used bits in the storage unit
- Conversion functions for converting between storage unit and symbol: `letter_to_symbol`, `letter_to_char`, `letter_complement`
- Utilities for generating k-mer masks: `kmer_mask`

The figure below 7.1 shows an example of how I bitpack DNA4 sequence in storage units.

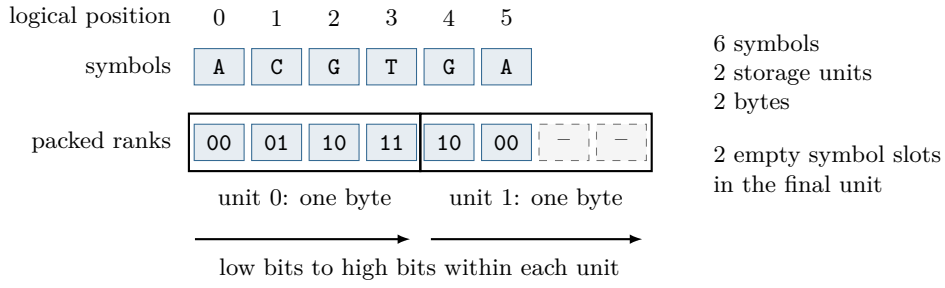


Figure 7.1: Packing `ACGTGA` with four DNA4 symbols per byte. The dashed slots are padding, which occurs at the end of the sequence as rarely sequence size is multiple of `compact_sequence_range<R>::traits::letters_per_unit` so they are not part of the sequence.

Template instance explosion

In BioseqDB I tried to allow user to specify any storage width that is power of two (8, 16, 32, 64) and alphabet (currently supported is DNA4 and DNA5). Because both of parameters are constexpr template parameters, this leads in some parts to creation of hundreds separate versions of pipeline. This slows downs compilation times and led to creation of absurdly large BMIs resulting in multi-gigabyte Clang .pcm files. Resulting binary still is optimised down to more convenient size, however I put lots of burden onto the compiler and linker. The goal of user-supplied storage widths was to observe how they affect performance and if they affect it at all. Currently in some parts of the pipeline I still cast the stored types to uint32_t, this happens because some algorithms like SIMD chaining require larger widths to store indices and scores. Extracting symbols directly from storage unit is a complex process involving lots of bitwise operations, so at this time most algorithms (SIMD/GPU) cannot operate directly on packed sequences as this would require lots of additional engineering effort. I added conversion helpers like one presented in the listing below 7.1 that convert requested ranges of sequence into non-bitpacked sequence that is easier to work with. This operation is done on demand and stored in separate memory pool region, so pipeline doesn't have to convert sequence from bitpacked to non-bitpacked format multiple times. Directly unpacking sequence inside pipeline at the latest point in time is potential opportunity for future improvements.

The following example uses actual code snippet from BioseqDB - in this case the caller owns `output`, which has at least $last - first$ bytes, and passes bounds satisfying $0 \le first \le last \le N$.

Listing 7.1: A consumer requests only the ranks it needs.

```
template<supports_compact_sequence_range R>
void read_fragment(const compact_sequence_range<R>& sequence,
                   std::size_t first, std::size_t last,
                   std::span<std::uint8_t> output) {
    // One rank per output byte, in forward order.
    sequence.template to_encoded_into<false>(output, first, last);
}
```

7.3. Slices keep logical coordinates

Postgres support array subscription operations natively, so I decided to support slicing as well in first version of BioseqDB.

A slice selects positions $[a, b)$: it includes a and excludes b. BioSeqDB uses zero-based positions, so $S[0]$ is the first symbol of the slice.

0-based indexing

This contradicts the fact that Postgres slices are 1-based, but it wasn't intentional decision. I had limited experience with Postgres array functionality and this realisation came only after most of the support was already implemented. What is more interesting that SeqKit - tool for operating on FASTA/Q files use 1-based indexing Shen, Sipos, and Zhao, SeqKit usage documentation: the subseq subcommand. This incompatibility might be something to reconsider in the future.

I provided utility functions for common operations like reversing and reverse-complementing slices as well, example available in listing 7.2.

Listing 7.2: Zero-based slices and sequence transforms in SQL.

```
WITH v(s) AS (
  VALUES ('ACGTAACCGTTA'::bioseq.seq('dna4', 32))      -- ACGTAACCGTTA
)
SELECT s[2:10]::text AS middle,                        -- GTAACCGT
       ((s[2:10])[1:5])::text AS nested,               -- TAAC
       s[-4:]::text AS suffix,                         -- GTTA
       bioseq.reverse(s[2:10])::text AS rev,           -- TGCCAATG
       bioseq.reverse_complement(s[2:10])::text AS rc -- ACGGTTAC
FROM v;
```

The bounds follow a small set of intuitive rules simiar to design of slices in Python Python Software Foundation, Python 3 Documentation: Common Sequence Operations:

- A missing lower bound means zero. A missing upper bound means the sequence length.
- A negative bound counts from the end (example: for length 12, -4 means 8) and bounds outside the sequence are clamped to its ends.
- An upper bound below the lower bound gives an empty slice, so `s[4:2]` has length zero.
- User can extract any element (symbol) from slice, which has separate SQL type `LETTER`. An out-of-range element, such as `s[12]` in this example, gives SQL `NULL`. An explicit null slice bound also gives `NULL` i.e $s[a:\text{NULL}]$ is equivalent to $s[\text{NULL}:a]$ and is also equivalent to $s[\text{NULL}:\text{NULL}]$ (all of them result in `NULL`)

The slicing can be implemented by old holding slice coordinates, without actually loading any sequences, by very simple coordinates arithmetic visible at figure below 7.2 For a slice view $V = S[a,b)$, view position j refers to $S[a+j]$. A second slice therefore combines its offsets directly:

$$V[c,d) = S[a+c, a+d).$$

The example's $S[2,10)[1,5)$ is $S[3,7)$, which reads `TAAC`. Reversal is represented by reversing start and end coordinates: For $W = \text{reverse}(S[a,b))$ we have:

$$W[j] = S[b-1-j], \qquad W[c,d) = \text{reverse}(S[b-d, b-c)).$$

Complementing operation leaves the coordinates unchanged, but alters binary flag representing if the slice is complemented or not.

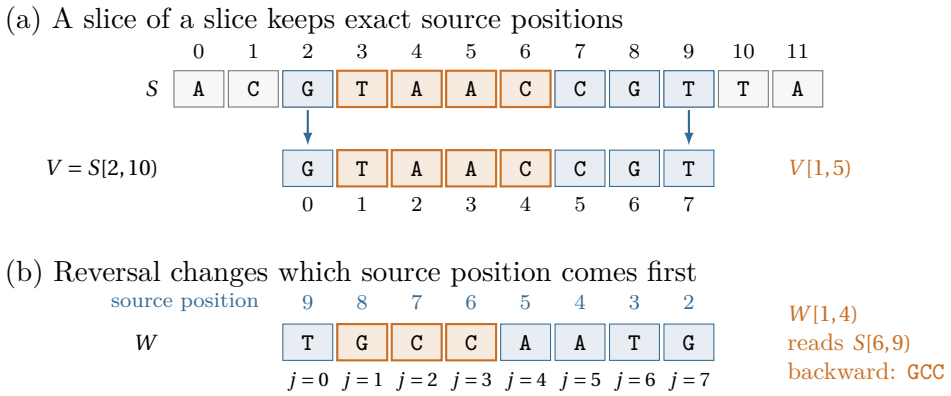


Figure 7.2: Blue cells form the first slice and orange cells show a second slice

7.4. Composing sequences without joining their payloads

The slice model I showed is very primitive. If we add ability to concatenate sequences we can represent any slice as a tree of pieces that represent operations on original sequence. In BioseqDB this tree refers to Postgres operators and datums.

The slice tree is constructed from the following elements:

- A source leaf describes interval in the source sequence.
- A literal leaf holds literal value referencing Postgres datum with additional metadata.
- A branch node that represents concatenation of two children. It holds total length, piece count (total number of leafs in subtree) and tree height. We collect total length to estimate how long the memory region in the pool should be to accomodate the materialized sequence.
- An edge that connects a node to its child. It holds reverse and complement flags, which affect the order of traversal and the interpretation of the leafs.

Figure 7.3 illustrates concatenation of three pieces including two source leaves and one literal leaf:

$$Z = S[2,6) \parallel \mathsf{reverse}(S[8,12)) \parallel \texttt{AC},$$

The result is `GTAAATTGAC`, with length $4+4+2=10$.

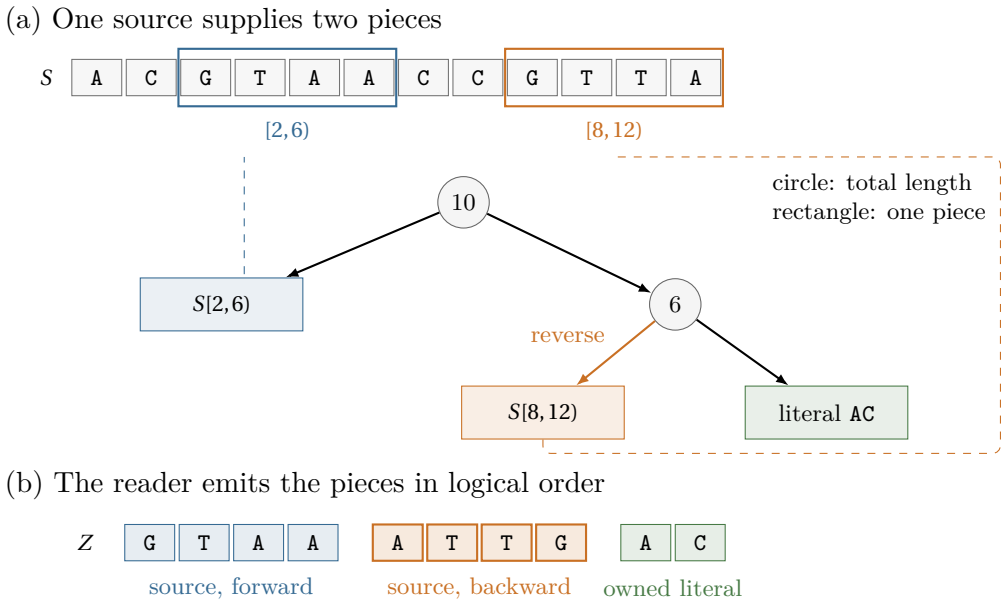


Figure 7.3: A piece tree describes how BioseqDB process slices of a sequence.

Each of the operations can easily be implemented by traversing the tree: concatenation joins roots and restores tree balance, slicing splits at the two bounds and leaves the middle part, reversing just as complementation flipps flags on the edges.

Reading actual slice content in BioseqDB is implemented using `lazy_sequence_view<A>` of table 7.1. It's built for the duration of a single call and the tree operations read only metadata from sequence locators (such as lengths). This means that we don't need to fetch large relations with sequence content and slicing is very performant. In the pipeline specific stages (like SIMD or GPU kernels) can request contiguous region of sequence and BioseqDB will materialize it into its own packed buffer first.

I implemented concatenation support for SQL text literals as well. This means that we can concatenate sequences with text directly without casting them, but this has it's own limitations:

- Slicing and reversal can operate on text without no known alphabet because they need only positions and order.
- Joining a known-alphabet sequence with a text literal validates the literal against that alphabet, so we require to go through entire literal (which shouldn't be a problem as we don't expect very long literals as this should be done via loading FASTA first into the table)
- Reverse-complement requires known alphabet with a complement rule for every symbol in that alphabet.
- Two different alphabets are not converted automatically to one another and this will result in an error.

7.5. Reading only what the algorithm requires

The storage can be provided in different forms as explained previously in the first section 7.1 In BioseqDB main implementation of Postgres-based storage is done via `lazy_sequence_`

`view<A>` and `retained_lazy_sequence_view<A>` mentioned in table 7.1. These types holds pointers to functions that implement the storage and represent the following operations:

- Report sizes of sequences
- Copy selected intervals into caller-owned memory, which in this case can be GPU-owned memory in GPU-accelerated pipeline stages
- Reading successive chunks
- Checking if the compatible interval was already materialized and can be borrowed (caching)

Internally I distinguish several access policies which are specified by the pipeline stage, they are the following choices:

1. Stream. If the algorithm accepts a stream, read the selected interval in bounded chunks and save it to reusable scratch memory (large memory pool)
2. Borrow. If the algorithm needs packed data, first check if's already in cache and safe to borrow.
3. Pack. In case if borrowing is not possible, algorithm needs to request new memory permit (this is routed through custom pool allocator). Caller is then responsible for this memory chunk.

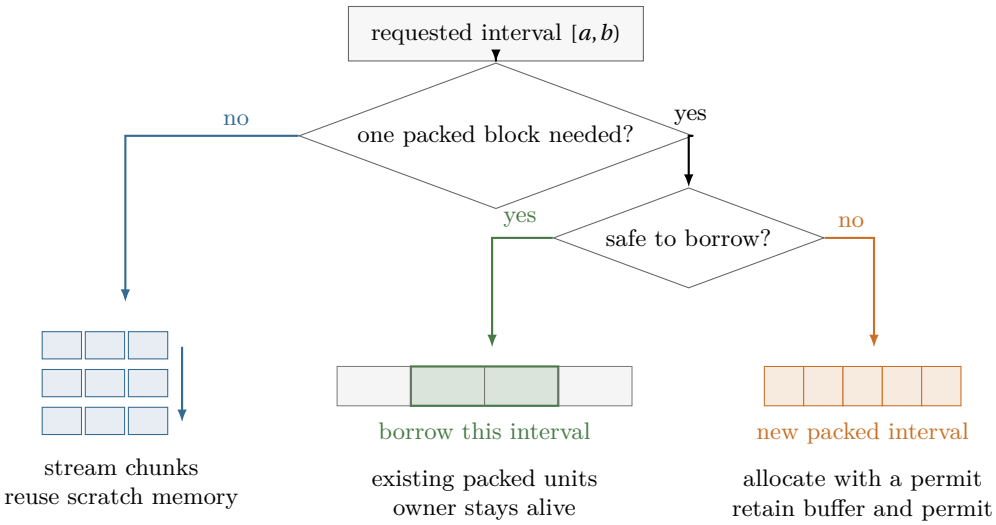


Figure 7.4: Simplified view of different access policies

Borrowing

Borrowing can fail if the region for example starts in the middle of storage unit in packed sequence, then we cannot reuse that memory chunk as it's misaligned

7.5.1. Copying ranks from a PostgreSQL sequence

The PostgreSQL source-provider view keeps its scanner and packed fragment slots in the read session. A rank request visits the requested symbols, looks up each symbol's fragment in that scanner and unpacks its rank. Repeated small requests can reuse packed fragments, but still repeat the callback and symbol-extraction work; they need not read the fragment again from disk or decode the whole fragment into ranks. Retaining a packed target for the alignment stage avoids these source-provider calls, although an algorithm requesting byte ranks must still unpack the selected interval.

At this point I need to describe lifecycles of various entities to illustrate how reading is performed. Figure 7.5 shows how what is happening under the hood during the scan:

- (a) Each row of a batch gets its own scanner with 16 fragment slots for caching. A miss looks in the decoded chunk cache (this is lower-layer cache for sequences chunks which may store multiple unrelated sequences and is discussed in later chapters), then in the columnar relation, and copies one fragment into a slot.

- (b) The incoming queries are grouped in batches. Each batch has it's own session, which is destroyed when batch processing is finished. The session is different entity to the chunk cache so relation isn't reopen when batch is finished and we keep large sequence chunks still in memory. At the start of the batch the backend first builds one packed copy in memory and distributes that copy to workers from the pool. It's important that in this architecture we only have one backend thread which is responsible for reading relation. This means that the actual pipeline is run by single thread that submits various work to worker pool. I needed to read the relation on a single thread to avoid race conditions on PostgreSQL's process-local state which is not thread-safe - PostgreSQL starts (forks) a new server process for each client connection, and even parallel SQL starts extra background worker processes rather than extra threads inside the same backend.[4] The same read could still be done in parallel by giving each reader its own backend: one way would be to fork a background worker with a database connection (which in this context is problematic as forking with pool running will result in segfaults) or open a second client session through libpq (libpq is thread-safe when each thread uses its own connection).[5] I did not take those paths for fragment reads during a query as it would complicate the implementation and this is something to be done in the future.

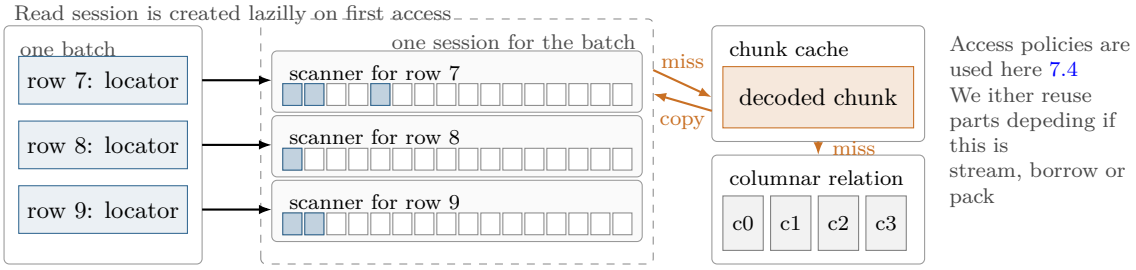
- (c) A view is built from two pointers - `state` is a piece tree together with the session that holds one scanner per source row and `ops` contains length, two flags and two function pointers. Providing function pointer serves as abstraction layer to divide non-Postgres code with actual Postgres implementation. Using templated types could be possible here, but they still would need type erasure. A retained view additionally holds a release callback (C-style destructor)

- (d) At every batch end, in this order: (1) release retained views, (2) finish each scanner so its slot copies go, (3) drop the session's memory. The worker pool and the snapshot naturally outlive the batch as discussed here. It might happen that we encounter an

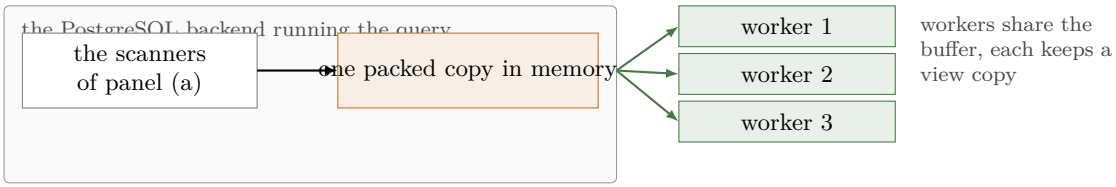
[4] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Architectural Fundamentals; PostgreSQL Global Development Group, PostgreSQL 17 Documentation: How Parallel Query Works.

[5] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Background Worker Processes; PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Behavior in Threaded Programs.

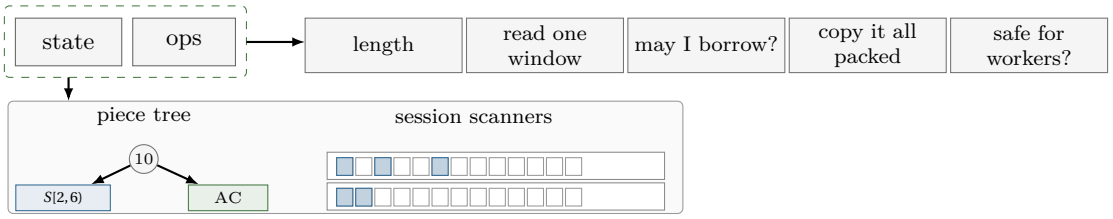
(a) One batch of rows gets reads session with multiple scanner. Each scanner has it's own slots for caching.



(b) Reading is single-threaded with distribution to multiple thread workers



(c) View copy sent to workers holds two base pointers (and release callback)



(d) Timeline of the scan

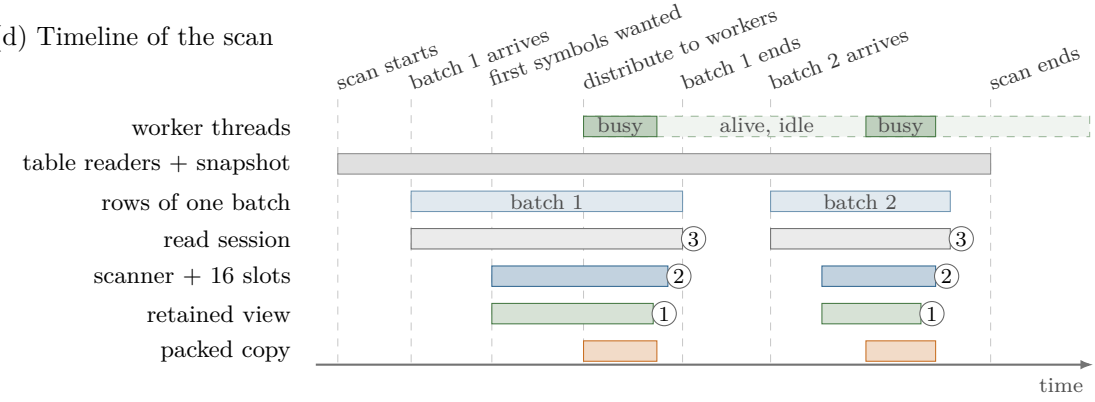


Figure 7.5: Illustration of structures used and operations performed on sequences during scan lifecycle

error and failing query skips C++ destructors, so steps 1–3 also run from a memory context callback (retained view release callback)

Panel (d) of Figure 7.5 has a gap between the end of batch 1 and the start of batch 2. In that gap the backend collects the next tuples from the relation (metadata and sequence locators, not entire sequences), applies the query's `WHERE` clauses, and places surviving tuples in the dispatch buffer. When enough tuples are collected, it submits the next batch. That collection still uses one relation-reading thread per PostgreSQL process. There's however support for `bioseq.scheduler.postgres_parallel.enabled` for any query that do not use alignment. The reasoning behind that is that only the thread pool wasn't designed around being multi-process safe and current statistics collection doesn't support it. The scans that do not use alignment are marked parallel-safe. This means that example shown on lsiting below 7.3 can be executed by postgres parallel scan.

Scans in details are covered in the next chapter 8.1 in detail.

Listing 7.3: Simple table read

```
SELECT id, bioseq.to_text(s[0:8]) FROM t;
```

Parallel scan

This is potential optimisation in the future. Statistics collection can be easily made multi-process safe by implementing reduce function, while thread pool design needs to be revisited and reimplemented.

Chapter 8

PostgreSQL API and custom scans

8.1. Designing SQL API for an aligner

In the previous chapter I presented how the sequence abstrations work and how they interact with the workers, we also had opportunity to look on how batching is done on figure 7.5. This chapter focuses on the main problem - how I incorporated this infrastructure into Postgres.

First, I want to discsuss how we can design SQL API for the aligner. Currently BioseqDB can be used in the following way:

Listing 8.1: Search once and return the fields of each hit.

```
CREATE TABLE targets (
  id bigint PRIMARY KEY,
  seq bioseq.seq('dna5', 64)
) USING bioseq;

SELECT t.id,
       bioseq.dp_score(t.seq) AS score,
       bioseq.target_start(t.seq) AS start,
       bioseq.cigar(t.seq) AS cigar
FROM targets AS t
WHERE t.id > 20
  AND mmsearch(t.seq, $1::bioseq.seq('dna5', 64));
```

The first `mmsearch` argument is the target sequence and the second is the query, - here query is supplied by `$1` (text parameter in prepared statement). The `score` alias means the dynamic-programming score of the current hit, CIGAR string describes the alignment and we also request position of the match in the sequence coming from target table.

One insteresting observation is that naturally each alignment can produce multiple hits, so this operation can return multiple rows for a single input row. We intuitively expect that there will be two kinds of selected expressions: Some of them will not depend on alignment (in this case it's primary key), so they will be the same for all these hits, while some of them like score and cigar will be different. Figure 8.1 illustrates that alignment should be set-returning function.

The first idea for implementation of such search was the simplest one - implementing alignment as callable Postgres function, which is very easy to do in C++. The function accepts query as raw SQL string:

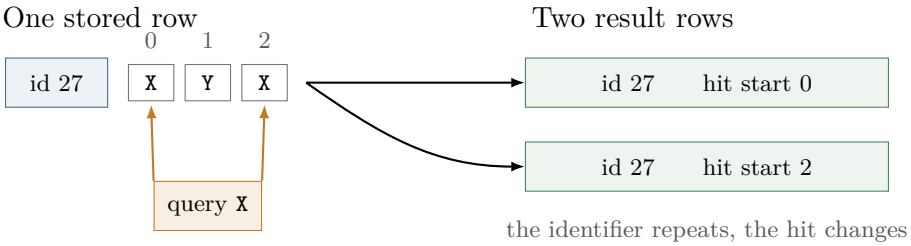


Figure 8.1: Searching `XYX` for `X` gives two occurrences, so the function needs to ”clone” metadata of tuple (id key in this case) and for each generated tuple fill-in the alignment hit position.

Listing 8.2: An early interface sketch passes SQL as text.

```
SELECT * FROM search_sql(
  'targets',
  'SELECT id, seq FROM reads WHERE id > 20'
);
```

This however introduces serveral problems:

- Which is the column we want to align? We can make a choice to hardcode column name to be `seq` at all times, but this limits flexibility heavily. Other choice is to allow passing the aligned column name as a parameter, which resolves this problem.
- How do optimiser know about inner query? As the inner query can be complex the optimiser doesn't know about it. We can call libpq inside the function, which can optimise the query separately, but then we should change it depending on the context: you can imagine very trivial case that the outer SELECT statement only selects primary key for some reason, so we don't have to do any alignment. In that case we would expect the function to do this optimisation, but this means that its implementation needs to do the same work as Postgres optimizer.

Listing 8.3: An outer condition cannot automatically filter a hidden scan.

```
-- Hypothetical SQL-string interface
SELECT h.*
FROM search_sql('targets', 'SELECT id, seq FROM reads') AS h
WHERE h.target_id = 27;
-- The function must itself restrict the hidden target scan.
```

The listing illustrates point stated by me: the function sees the target table through the string `'targets'`, but it's used in a context, where we will do only one alignment. If the function doesn't interpret its context, then it will perform alignment on entire table. The solution could be to pass this reponsibility to the user, but this generates huge inconvenience - for more complex queries, user would need to transfer SQL statements to and from the raw text pased to search function and manually be responsible for optimisation. In that case this should mean that when we forgot to do that the different is enormous - the statement will align two tables instead of one (could be orders of magnitude slower).

SQL already has a direct way to express one-to-many output. A set-returning function in a `LATERAL` join could return a record for every hit.[1]. The `LATERAL` clause lets that call refer to the current target row.[2]

Listing 8.4: A different API could return explicit hit records.

```
-- Alternative API sketch, not an existing BioSeqDB function
SELECT t.id, h.score, h.start, h.cigar
FROM targets AS t
CROSS JOIN LATERAL align_hits(t.seq, $1) AS h
WHERE t.id > 20;
```

This means that optimiser can work with queries, but user needs to do `CROSS JOIN LATERAL` for each alignment, which adds lots of boilerplate code. Additionally we loose ability to conditional evaluation in this case. Sometimes alignment may not require CIGAR which is more expensive operation as it requires running base alignment and the `SELECT` clause may only specify score. This prevents us from evaluating alignments differently based on what fields are needed.

To more easily indentify what parts of the alignment are needed (because otherwise we would need to track where the record type is used and how), I decided to separate returned record type into calls to functions. The function takes sequence as argument and returns some information about alignment. This concept is presented below.

Listing 8.5: Hidden state does not establish an evaluation order.

```
-- Hypothetical functions with shared hidden state
SELECT id, last_score(seq)
FROM targets
WHERE save_search(seq, $1)
  AND last_score(seq) > 100;
```

Now the fields are easy to track as we can directly look for a call to a function in the AST expression tree. However if the extension implemented only Postgres function this example couldn't work. We would need to pass some internal state between function calls - `save_search` would need to save some pointer to lazy execution object invisible to the user so it can be later called by `last_score`. This way we compute only what we need. Declaring these functions `VOLATILE` limits some planner transformations, but does not guarantee execution order between them[3] This means that it's not possible to reliably pass this hidden lazy object between functions.

We can see that example above is much cleaner than `LATERAL` join and easier to understand, but to implement it we need to tell Postgres how we want to execute this query - in which order expressions are evaluated and create our own tuples inside. Historically before PostgreSQL 9.5 only way this could be done was via foreign-data wrapper interface, which treated remote table as opaque object. Later Postgres version introduced custom scans and table access methods. There's overlap in all these interfaces, but the main advantage of TAMs and custom scans was that the logic wasn't only tied to one specific table and there was deeper and more cleaner integration with the executor.

[1] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Set Returning Functions.
[2] PostgreSQL 17, `LATERAL` Subqueries.
[3] PostgreSQL 17, Function Volatility Categories.

1. PostgreSQL 9.1 added foreign tables and the foreign-data wrapper (FDW) API, a partial implementation of SQL/MED.[4] PostgreSQL 9.3 added `postgres_fdw`, the in-tree wrapper that queries another PostgreSQL server.[5]

2. PostgreSQL 9.5 added custom path and scan methods, so an extension can insert its own executor node into a plan.[6]

3. PostgreSQL 12 added table access methods (TAM) through `CREATE ACCESS METHOD …TYPE TABLE`. Heap remains the default TAM.[7]

Table 8.1 summarises the split.

Table 8.1: PostgreSQL interfaces that can supply rows to a query.

	FDW	Custom scan	TAM
First in core	9.1 API, 9.3 `postgres_fdw`	9.5	12
Attaches to	a foreign table	a plan node	a local table
Stores bytes here	no	no	yes
Typical work	fetch remote rows	replace a scan or join	layout, insert, vacuum

The BioseqDB implements the following components:

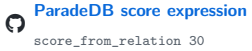
- Custom scans - allow to implement syntax suggested here with selective evaluation of the pipeline, efficient batching integrated with optimizer, so I rely on optimisations from Postgres as well. Theoretically custom scan can replace scans to any table, but I don not offer support for tables other than BioseqDB tables (created using `CREATE TABLE USING bioseq` TAM)

- Table access method - stores data in columnar format, so I can load large chunks of metadata and sequence in single row fetch

8.2. Integrating Postgres

8.2.1. Designing SQL API

A full text search interface (`pg_search`) from ParadeDB was an important motivation for the SQL API design of BioseqDB. ParadeDB is an extension written in Rust that uses custom text storage engine under the hood. I deeply inspected its source code to understand how we can implement our own syntax in BioseqDB. ParadeDB resolves similar problem as FTS can also return multiple hits for a single query.



To return custom results instead of relying on Postgres evaluating the function (We don't want to let Postgres call actual `bioseq.score()` function, but rether provide it ourselves. ParadeDB has similar syntax and the same problem) ParadeDB replaces score expressions

[4] PostgreSQL Global Development Group, PostgreSQL 9.1 Release Notes; PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Foreign Data.

[5] PostgreSQL Global Development Group, PostgreSQL 9.3 Release Notes; PostgreSQL Global Development Group, PostgreSQL 17 Documentation: `postgres_fdw`.

[6] PostgreSQL Global Development Group, PostgreSQL 9.5 Release Notes; PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Writing a Custom Scan Provider.

[7] PostgreSQL Global Development Group, PostgreSQL 12 Release Notes; PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Table Access Method Interface Definition.

with typed placeholders. These placefolders are filled from the current search result before projection. ParadeDB placeholder replacement inject_placeholders 493 Similar design was used in BioseqDB - described in Section 8.3.

Listing 8.6: ParadeDB pg_search examples

```
-- Some examples for pg_search
SELECT id, title, pdb.score(id) AS score
FROM documents
WHERE body ||| 'postgresql search'
ORDER BY score DESC, id;

SELECT id, pdb.snippet(body) AS excerpt
FROM documents
WHERE body &&& 'postgresql indexing';
```

Listing 8.6 shows how similar ParadeDB's problem is to my initial API designs.[8][9]

8.2.2. Plan lifecycle

To implement custom scan, we must understand different objects in Postgres:

1. Path: Describes possible inputs, estimated work, output row count, and ordering. The planner compares several paths and keeps the cheapest one[10].[11] The Genetic Query Optimizer is used only when the number of joins exceeds `geqo_threshold`. A single-table `BioseqScan` is ordinary cost comparison, not GEQO. The path is later converted into a plan. I register my own paths so PostgreSQL knows how to access the table.

2. Plan: In our case `CustomScan` contains child plans, expression trees, and descriptions of the search.

3. State: When plan is executed, postgres creates its state via my code. `CustomScanState` holds the input cursor, alignment buffers for sequences and metadata, and slots for generated hit. Postgres will call scan repeatedly to get new rows, so I use the state to hold any information that needs to be preserved between calls.

All types of Postgres API described in table earlier 8.1 mainly rely on callbacks (in Postgres called hooks). Extension fills the structure with function pointers and let postgres know which call is handled by which function. In this case PostgreSQL calls `PlanCustomPath` when it converts a selected custom path into a plan and later calls `CreateCustomScanState` to create the executor state[12] Figure 8.2 shows when PostgreSQL calls those hooks, when `BioseqScan` runs, and when the columnar table access method returns a tuple.

`ProcessUtility_hook` is used for utility statements such as `CREATE`, `DROP`, `COPY`, and `CREATE INDEX`. This is useful to forbid user from doing certain operations (Currently I only use it to log statements).

The TAM stores the user table in columnar format: A long sequence is not kept in that table row, only a locator is stored there. The sequence itself is stored in companion fragment

[8] Pigsty, pg_search: full text search for PostgreSQL using BM25.
[9] Neon, The pg_search extension.
[10] PostgreSQL Global Development Group, PostgreSQL Documentation: Planner/Optimizer.
[11] Conway, Inside the PostgreSQL Query Optimizer.
[12] PostgreSQL 17, Creating Custom Scan Paths and Creating Custom Scan Plans.

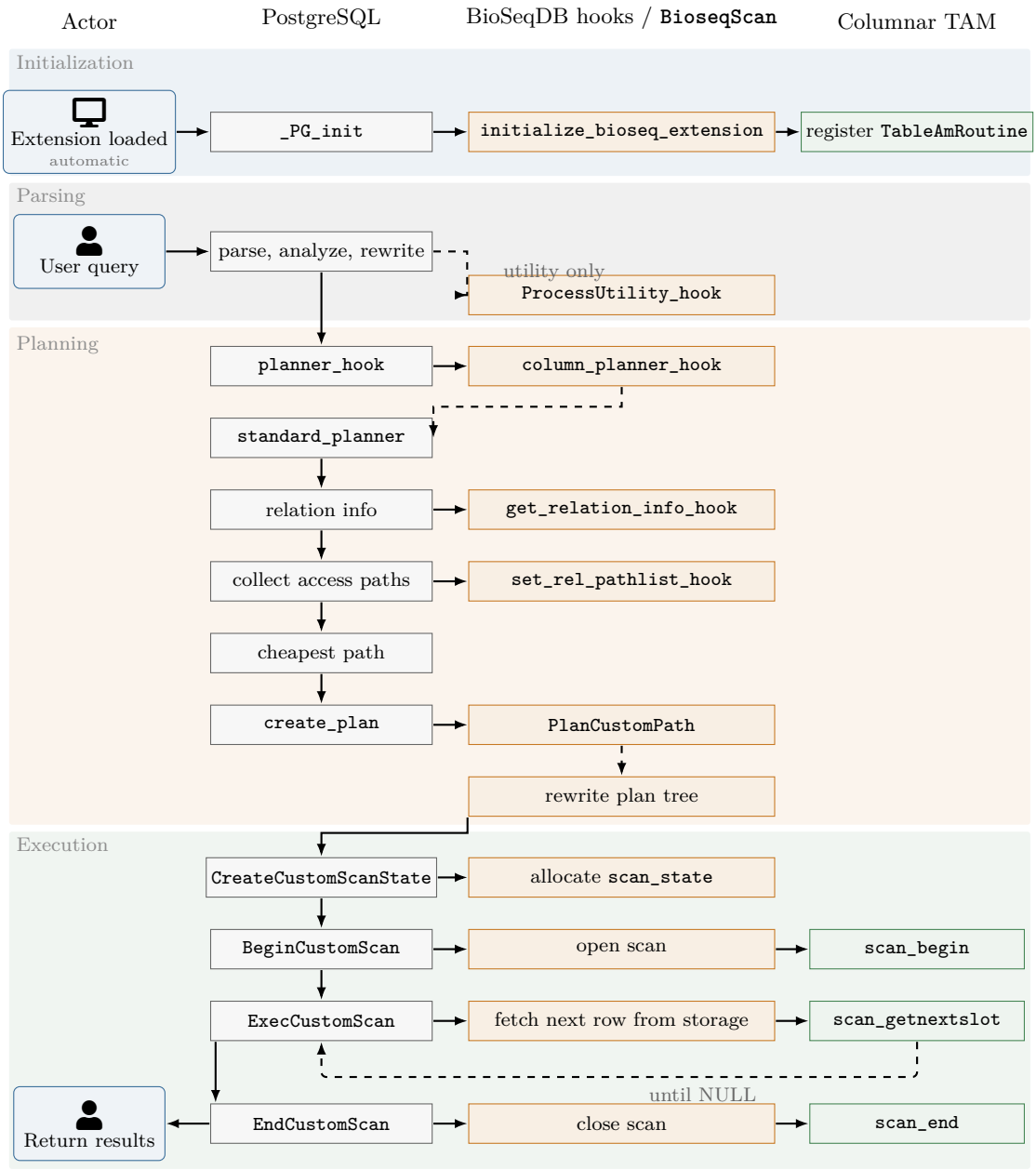


Figure 8.2: Lifecycle of `BioseqScan` and the columnar table access method. Bands mark exeuction states: initialization, parsing, planning, and execution. Boxes have various colours: Grey boxes are PostgreSQL, orange boxes are BioSeqDB hooks or the custom scan and green boxes are TAM callbacks. Arrows follow the order in which Postgres calls specific hooks. `column_planner_hook` calls `standard_planner`, then rewrites the finished plan, so we don't explicitly control GQO, but can affect it using costs functions, registering paths and even after plan is choosen, we have complete constrol over it (we are free to rewrite its parts). The scan is calling TAM to fetch rows from storage and returns them to Postgres until it returns NULL (no more tuples to fetch)

sub-tables, and all of them use the bioseq TAM as well. More on storage format is discussed in Chapter 9.1. I called the process of fetching fragment from locator "hydration" and TAM is responsible for it.

BioseqDB have control over the plan using `column_planner_hook`. This hook is called when PostgreSQL needs to perform planning and SQL query is already parsed. First BioseqDB hook validates `mmsearch` options, then it calls `standard_planner`. `get_relation_info_hook` and `set_rel_pathlist_hook` hooks, where we set options like disabling parallel scans or index-only scans and register custom access paths for cust scan, run inside that call. Then GQO is run and finds the best path that needs to be converted into a plan, so the `PlanCustomPath` is run. When `standard_planner` returns we are still in the `column_planner_hook`, so I'm free to rewrite plan tree freely. The rewrite mutor performs the following changes:

- Enable vectorized execution for simple aggregates Using modernized Citus/Hydra columnar vectorized optimisations I allow simple aggregates on metadta columns to be vectorized (processed in batches) for example: `COUNT(*)` or `SUM(foo_bar)`. If current node is Postgres `Agg` it will be replaced by custom aggregator `VectorAggNode`. If this plan has `VectorAggNode` as parent, then I return batches of rows instead of single tuples. Rewrite adds falgs to mark that the plan needs to change its output mode. Additionally Postgres `Agg`

- Wrap an index scans on columnar tables PostgreSQL can still choose a metadata index on a `USING bioseq` table and use external `IndexScan`. If that happens this index can read some tuples that shouldn't be red (columnar TAM allows removing rows. If that happens row is marked in bitmask, but index scan can go around it and still read the row, because it's physically there). To prevent that the `IndexScan` is replaced by custom `BioseqIndexScan` that hounours bitmask.

As entities like `bioseq.score(id)` are still ordinary functions, I discourage Postgres by using prohibitive execution costs[13]. The function implementation itself mostly just emits an error, because it shouldn't be ever called. Instead `ExecCustomScan` will using special Postgres expression nodes i.e `Const*` node to tell Postgres that the scan itself calculated the function's value and it shouldn't evaluate it by itself.

8.2.3. Different types of scans

When I talk about a custom scan I focus on `BioseqScan`, which is the primary way to read a `USING bioseq` table, but there are other types of custom scans used for different purposes in BioseqDB, some of them already mentioned earlier. All scans types are the following ones:

- Dedicated join support: `BioseqBatchJoin` is a custom scan with two children: one child is the target plan, the other is the query plan. Normally Postgres would use nested loop join that would read one target and one query and perform alignment for each pair, however we want to avoid this behaviour as target index can be loaded once (running `BioseqScan` for each pair would cause index for one table be loaded, then unloaded and loaded again for each query). This scan, dedicated for joins, prepares that target once (open the stored minimizer reader, or build index manually if not available) and runs the same logic as `BioseqScan` for each query in the batch. Dispatch buffer has separate API for joins, but they mostly effectively do same processing as for `BioseqScan` (collect queries in batches etc.)

[13] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Function Optimization Information.

- Slices across operations: `BioseqSliceProject` wraps a child plan when the query asks for a slice or other piece-tree operation, for example `SELECT seq[100:200]` or `mmsearch(t.seq[a:b], q.seq)`. It evaluates those expressions into expanded piece-tree Datums (Section 7.4) and sends them to Postgres as read-only expanded objects in a virtual slot. This means that the piece-tree is stored in the tuple itself and can be passed around other scans, joins etc. without translating it into a string yet: later `BioseqScan` or `BioseqBatchJoin` can align that slice lazily. When user requests a slice as in `SELECT seq[100:200]` and it doesn't go through other operation nodes, then it will be "hydrated" from piece-tree into actual byte sequence.

- Support for indexing: `BioseqIndexScan` is the wrapper around `IndexScan` described above, that allows to correctly read deleted columnar rows (support for bitmasks).

- Sorting by sequence: `BioseqBatchSortScan` is only for sorting by the sequence itself, for example `ORDER BY seq` or `ORDER BY seq DESC`. The planner will choose it only when there is exactly one `ORDER BY` key and that key has type `bioseq.seq`, and only when `bioseq.planner.sequence_sort_scan.enabled` is on (`ORDER BY seq, pk` and `ORDER BY seq::text` are still ordinary PostgreSQL sorts) This is experimental feature to allow faster sorting by sequence without the need to flatten entire sequence first. The custom scan buffers whole child rows, compares the sequence column as lazy views (not as printed text), and emits those same rows in the resulting permutation.

8.3. Dispatch buffer as glue between Postgres scan and alignment pipeline

I still need to explain how exactly `CustomBioseqScan` works with worker-architecture shown in Figure 7.5. The main abstraction here is the dispatch buffer which is reposible for transferring data between Postgres and workers.

Figure 8.3 shows how Panel (a) of Figure 7.5 looks in the context of Postgres scan. `BioseqScan` asks for the next row from TAM. This row is placed into dispatch buffer (locator only) and dispatch buffer holds all the structures presented before 7.5 - scanners. The dispatch buffer code then handles all view copies to the workers.

The collect loop is the gap between batch 1 and batch 2 in panel (d) of Figure 7.5. `BioseqScan` may call the user-table TAM many times before it returns to fill in the dispatch buffer. The backend thread that owns the scanners performs fragment reads, while workers only see the view copy described in panel (b).

8.3.1. Returning results back to Postgres

Now I not only need to build the tuple but somehow explain to Postgres that certain values correspond to requested values of function calls like `bioseq.dp_score(t.seq)`. PostgreSQL evaluates output expressions through an expression tree. Before returning its first alignment hit, the executor walks this tree and replaces the function calls with `Const` nodes. It writes the hit's values and null flags into these nodes and rebuilds the output-expression evaluator so that it reads the changed fields. This makes Postgres know that we already computed these values. The idea was created by analyzing how ParadeDB provides its search results

ParadeDB current-result score
exec_custom_scan 1784

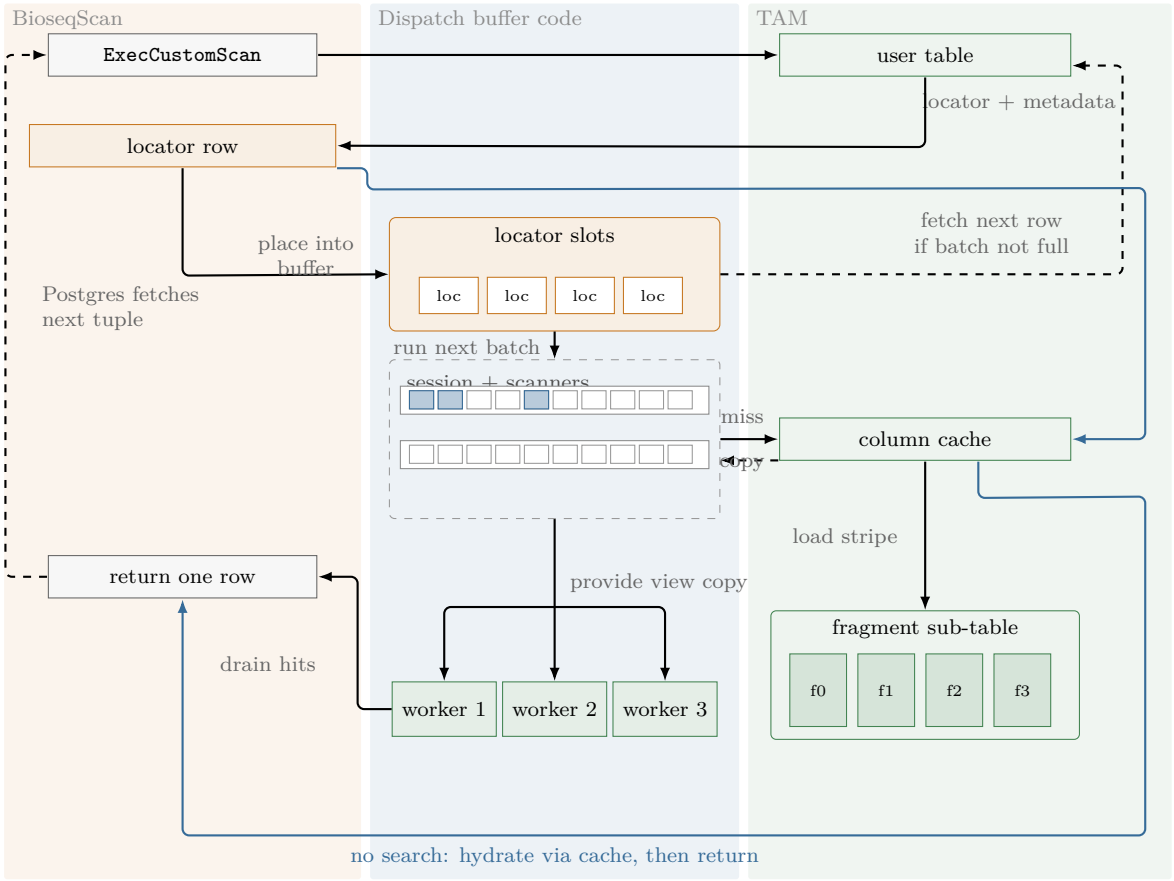


Figure 8.3: Inside `ExecCustomScan`, the following things happen: `BioseqScan` holds dispatch buffer and fetches a locator handles it to the buffer logic and retrieves results from it. The dispatch buffer stores locators, then builds a session that holds scanners. A scanner miss reads the fragment sub-table through the TAM column cache and copies the fragment into a slot. Workers receive a view copy from the scanners. Black arrows show the normal alignmen execution path, however it might happen that we don't perform alignment and just want to return the sequences itself. In that case the flow uses blue arrows (there's no `mmsearch` in the query) and `BioseqScan` hydrates the sequence row through cache and just returns it without interacting with dispatch buffer. Dashed arrows repesent loops here.

Algorithm 17 How to return tuple

1. Restore the original columns.
2. For every requested BioseqDB function like `score(seq)`, `target_start(seq)`, `cigar(seq)` and so on, write its field from this hit into the corresponding typed value node.
3. Rebuild the evaluator, then evaluate the output expressions.
4. Return the resulting SQL row

8.3.2. Supporting joins

A join can lift a helper such as `bioseq.score(t.pk)` out of the scan that produced the hit and an outer join can also emit a null row when one input has no match.

The `Const` rewrite from the previous subsection is nice execution trick inside `BioseqScan`, but the join planner does not see those constants. It still sees an ordinary function call, so PostgreSQL may copy that call onto the other input of a `LEFT JOIN`, or evaluate it again in a `Join` or `Aggregate` node.

PostgreSQL's `PlaceHolderVar` exactly to resolve this issue, however it's suprisingly low volume of documentation about it. I only found old mailing list archive, where it's described as an anti-COALESCE: the expression keeps its value on a matching row and is forced to `NULL` on a missing row.[14] The `PlaceHolderVar` records the relation at which the expression must be computed and the value then rises through the plan like a column. When an outer join nulls that relation, the placeholder becomes `NULL` (a `Const` cannot do this as it's always the same value).

PostgreSQL expression placement
PlaceHolderVar 2750

Without that wrapper the following could happen:

- The inner scan of a `LEFT JOIN` receives `bioseq.cigar(t1.pk)` even though the helper belongs to `t1` and that scan rewrites a function it does not own.
- A `Join` or `Aggregate` node calls the C++ stub. As I mentioned the function does not compute an alignment (the actual value is only scan-provided), so we get an error.
- Mishandling of `NULL` values:An unmatched outer-join row keeps a filled score instead of SQL `NULL` or `ORDER BY bioseq.score(t.pk)` after a join can sort on all-null keys.

ParadeDB uses the same mechanism for `pdb.score` and its planner function wraps the call in a `PlaceHolderVar` when the query has a join or an aggregate, so the custom scan keeps emitting the score and upper nodes do not re-run the function.

ParadeDB score placement
placeholder_support 314

In BioSeqDB I do this using `SupportRequestSimplify` hook as `bioseq_placeholder_support`, where we kip placeholders when there's no joins or aggregates.

8.3.3. Dispatch buffer internals

The main component I want to discuss here is the dispatch buffer: a container abstraction reponsible for retrieving, storing, distributing and collecting of the tuples with results. This requires managing memory and lifetime of Postgres tuples (both incoming from table and outgoing to the user).

Handling tuples

Postgres offsers `TupleTableSlot*` type, which represents reusable holder for a tuple. The Postgres executor keeps one scan slot on the expression node and asks the table access method to fill that same holder on every fetch.

This holder consist of a tuple descriptor, current column values, null flags, and a small operations (functions pointers) table that knows how to read or copy the physical layout currently stored inside it (PostgreSQL Global Development Group, PostgreSQL 17 source: tuple table slots). The tuple descriptor is the struct holding information about the columns schema (how many attributes it has, names of the attributes and their types with additonal metadata).

[14] Lane, Re: what exactly is a PlaceHolderVar?

PostgreSQL offsers the following types of slots Builtin tuple table slot types TTSOpsMinimalTuple 38:

- Buffer heap tuple (`TTSOpsBufferHeapTuple`). The slot points at a row that still lives in a shared buffer page.
- Heap tuple (`TTSOpsHeapTuple`). The slot owns a `palloc` copy of a full on-disk row with system columns (added by Postgres) (PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Database Page Layout).
- Minimal tuple (`TTSOpsMinimalTuple`). The slot owns a compact copy of the user columns only without system columns added by Postgres. Sort, hash, and materialize nodes use this type because it is smaller than a heap tuple.
- Virtual tuple (`TTSOpsVirtual`). Instead of storing entire row it stores arbitrary data in array (usually pointers to the actual values, so we can reference existing tuple).

The TAM has one slot that is used to return tuples and it's overwriten with each tuple fetch (`getnextslot` call). This means that dispatch buffer has to store the tuple somewhere. In a dispatch buffer method called `submit_scan` I allocate a fresh slot with `TTSOpsMinimalTuple`, copy the current user columns with `ExecCopySlotMinimalTuple`, and store it with `ExecStoreMinimalTuple`. The third argument is `shouldFree=true`, so the copied slot owns the bytes and will `pfree` them when the buffer later drops the slot.

Sequence tuples in the dispatch buffer needs to represent lazy-loadable seqences, so during their lifetime they can become of of the 3 types (I need to distinguish them, because in code they are all just minimal tuples, but semantically express different concepts):

- Locator: This object is what is usually stored in the user table and represents entire sequence stored in separate internal table. This is "unhydrated" sequence.
- Expanded: Postgres has a flag `VARATT_IS_EXTERNAL_EXPANDED` (I call this in a code "expanded recipe") to mark the value is not a flat byte array yet, but complex structure in memory. This happens when sequence holds piece-tree (slice graph) 7.4, so we have multiple slice operations to perform if we want to convert this value to flat sequence. This is also different form of "unhydrated" sequence.
- Varlena: Representing "hydrated" sequence fragment, so the slices or locator were converted to some flat byte array.

The alignment pipeline uses first two representations directly and as mentioned before when discussing sequence abstractions 7.1 the sequence will be kept as a locator or piece-tree until the actual data is requested 8.3. Then I load requested bytes and return them directly. However in some cases we need to convert the tuple into 3rd form. If user requests slice projection directly it goes through separate branch of BioseqDB handling where that conversion is performed:

Listing 8.7: Direct projection of a sequence slice.

```
SELECT seq[2:3] FROM table;
```

Buffer API

Table 8.2 lists the most important methods of the dispatch buffer that are used during `ExecCustomScan` call. `Snapshot` here means the set of row versions this statement is allowed to see (this is mechanism for MVCC support in PostgreSQL) (PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Introduction to Concurrency Control) and is discussed later.

Table 8.2: Dispatch-buffer methods used by the Bioseq custom scans.

Method group	When the custom scan calls it	What it does
`configure_params`, `replace_executor_snapshot`, `bind_sequence_source_provider`	`BeginCustomScan` after the scan has a relation and a snapshot. PostgreSQL parallel workers (only for queries that do not use alignment 7.3) restore a copy of that snapshot and call `replace_executor_snapshot`. The worker must use its own registered `Snapshot` pointer for later fragment reads.	Stores aligner settings, the user table identity, the borrowed snapshot, and the statement-owned sequence source that scanners will read.
`configure_mmsearch`, `configure_mmsearch_per_row`, `configure_mmsearch_per_row_evaluated`	`BeginCustomScan` once the planner-private list has named the search, or on `ReScan` when a join parameter changes the query.	Records which columns are the target and (if needed) the per-row query, copies a constant query, and turns the buffer into the writing stage.
`submit_scan`, `submit_scan_evaluated`	Writing stage of `ExecCustomScan`, after a TAM row passed the filter conditions. The ”evaluated” method is called when parameter in `mmsearch` is an expression that needs to be evaluated.	Copies that row into a buffer-owned MinimalTuple 8.3.3 slot and records its row number, primary key, and sequence locator (or evaluated Datums). Does not run alignment yet.
`ready_to_dispatch`, `has_finished_writing`, `mmsearch_active`	The `ExecCustomScan` loop, after each submit and after each drained hit.	Returns flags indicating whether the physical buffer has reached 8192 rows, whether the current batch has already been aligned, and whether this scan is an `mmsearch` scan at all.
`run_batch_and_build_cursors`	When the buffer is full, or when the TAM source is exhausted.	Streams the buffered locators into the alignment pipeline, then builds an output cursor for retrieving hits.
`read_next_output`	Reading stage of `ExecCustomScan`, once writing has finished.	Returns the next hit together with a pointer to the copied scan slot. The scan then fills the match-field `Const` nodes from Section 8.3 and projects.
`clear_slots`	After a full 8192-row batch has been drained while the TAM source still has rows, and on `ReScan` or `EndCustomScan`.	Frees every copied MinimalTuple slot and the payload arena so the next physical batch can start empty.
Support for joins: `configure_mmsearch_join`, `begin_join_target`, `prepare_join_target_queries`, `set_join_query_batch`, `run_join_query_batch_and_build_cursors`, `finish_join_target`, `reset_join_runtime_for_rescan`, `abandon_join_runtime_after_pg_error_noexcept`	Methods called only by `BioseqBatchJoin` custom scan for custom join (discussed in next section 8.4). PostgreSQL `ReScan` restarts a plan node from its first row, often with new parameters (for example the inner side of a nested loop for the next outer row, or a cursor rewind). `ReScanCustomScan` therefore calls the reset helper so leftover join targets and hits are dropped.	Holds one join target in the buffer, streams inner queries in batches of at most 8192, runs alignment against that stored target, then drops the target. The reset and abandon helpers clear join state without asking the caller to redo `submit_scan`.

The code in `ExecCustomScan` pulls one TAM row, runs filtering conditions (these can reject the row based on metdata) and on success calls `submit_scan` to let buffer copy the tuple. When the copies in buffer reach the set capacity `dispatch_batch_capacity` $8192 = 2^{13}$, `ready_to_dispatch` flag changes to true so the scan calls `run_batch_and_build_cursors`. This method is also invoked when there are no more tuples in TAM source.

`run_batch_and_build_cursors` is where the batch starts and scanners and workers from Figure 8.3 run. It groups the locators already stored in the MinimalTuple copies, lets scanners fetch fragment bytes through the TAM under the Postgres table snapshot object and then distributes the views to workers.

Then the scan needs to read results. Reading stage calls `read_next_output` on every later `ExecCustomScan` to retrieve the next result (this means that in this semantic first call to `ExecCustomScan` takes very long as it needs to align first batch, while later calls return tuples one-after-another from the output cursor until they are exhausted and process repeats). This is the place where we need to set values of nodes and project the tuple to let Postgres construct final results 8.3. After Postgres consumes that projected tuple, the next call drops the previous result slot and asks for the next hit. When the cursor is empty and the TAM source is not yet exhausted, the node calls `clear_slots` and returns to the writing stage (this corresponds to finishing batch and starting collecting rows again)

Execution statistics. Each completed call to `run_batch_and_build_cursors` writes a `pipeline_executior` aggregate onto the buffer, where I collect debugging statistics. `EXPLAIN ANALYZE` reads then through `execution_stats()`. Table 8.3 presents some of them as the list is long and not every one is relevant to the common user.

Table 8.3: Grouped execution statistics collected by the dispatch buffer.

Group	Fields (batched)	What they tell you
Backend choice	requested versus actual backend, backend declines, backend fallbacks	Whether the scan asked for GPU or CPU, and whether the runtime had to use fallback (hardware unavailable or alogirhtm is not supported).
Alignment jobs	attempted, completed, failed	How many target and query pairs the pipeline tried, how many finished.
Target index	target rows, persisted index rows, missing persisted rows, recomputed rows	Whether minimizer indexes were read from storage or rebuilt for this batch.
GPU dispatch	seed, chain, backtrack and other stages	How much of the aligner actually ran on the device
Stage timings	per-stage nanosecond counters (query sketch, index build, seed, chain, extend, …)	Which pipeline stage dominated wall time on this statement.
Integrity flags	mixed environment, mixed persisted identity, saturated counters	Additional flags indicating overflow or aggregation of counters and ”mixing” which means that different stages of pipeline run in different modes (incompatible k/w or GPU/CPU mixing) - this is used for debugging

8.3.4. MVCC support

PostgreSQL holds its current input row in temporary storage called a tuple slot, but another transaction may also update that row. `Snapshot` in Postgres is set of row versions some statement is allowed to see as it needs to perform visibility checks before doing anything (PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Introduction to Concurrency Control). `configure_params` borrows this snapshot object so later fragment reads and bulk fetches observe the same MVCC view as the original TAM fetch. Let's see if this is compatible with the BisoeqDB model:

- Copying values: Dispatch buffer always copies the original tuple into an owned MinimalTuple slot, so Postgres won't overwirte this data.
- Saving Postgres snapshot object: PostgreSQL uses a snapshot to decide which row versions a command can see. The sequence saves scan's Snapshot and checks a stored reference against the row (visibility check). Stored source fragments are immutable. An update publishes a new version, while an older snapshot can still read its earlier version. Delaying a read does not switch it to the latest committed sequence.[15]
- Saving row numbers: While the primary key is important for user, the row internally holds a row number that is used to indentify results.
- Opening relation: In scan I use `ACCESS SHARE` relation lock to prevent conflicting operations such as dropping or rewriting a relation while it is in use (concurrent updates and deletes are still possible, but avoided due to snapshot mechanism)[16]

I use existing Postgres mechanisms to support MVCC so it didn't require any additional custom code.

8.3.5. Filter placement

As this mentioned here, before saving tuple into the buffer, scan checks if the filter predicate on non-alignment dependent values evaluates to true. This is done to avoid aligning rows that will be discarded anyway. In BioseqDB there are two types of filter conditions:

- A row condition such as `t.id > 20` depends only on stored columns, so is evaluated before alignment. Sometimes columns that are comparable (like Postgres primitive types) can allow min/max chunk prunning to be used. This is special strategy to skip entire columnar chunks with data 2
- A hit condition This is special case of alignment prefilter. I have a separate stage in the pipeline that can be invoked to run cheaper estimation of edit distance for each sequence. If I detect special predicate of form `bioseq.edit(t.seq) < 10` I run pipeline with prefilter enabled. This is different to the case above, because it still requires batching tuples and running pipeline, but it allows for faster termination. If the distance estimation yields false, then the rest of stages are skipped (there's no seeding, chaining, backtracking etc.) so we avoid even the cost of fetching the minimizers from the saved index. This results in great speedups, but at this time is yet an experimental feature using code from libraries that weren't developed by me.

[15] PostgreSQL 17, Transaction Isolation.
[16] PostgreSQL 17, Explicit Locking.

Listing 8.8: Filter input rows and then filter alignment hits.

```
SELECT t.id, bioseq.dp_score(t.seq) AS score
FROM targets AS t
WHERE t.id > 20
  AND mmsearch(t.seq, $1::bioseq.seq('dna5', 64))
  AND bioseq.edit(t.seq) < 10
LIMIT 5;
```

Potential features

Currently I do not optimise limits so `LIMIT 5` will still evaluate all tuples in batch even after more than 5 results are found. I also do not provide any optimisations for score ordering, so `ORDER BY score DESC LIMIT 5` doesn't result in any specialised algorithm. However there are some very speicific cases that would be interesting to optimise. We can use faster chaining if we know that there's only one good hit per sequence. This isn't expressable in SQL with this API, but you can imagine providing custom limit per one row: `WHERE bioseq.count(t.seq) = 1`.

8.4. Joins support

Asssime we have a search between two tables `reads` and `targets`:

Listing 8.9: Search sequence pairs from two tables.

```
SELECT t.id AS target_id, q.id AS query_id,
       bioseq.dp_score(t.seq) AS score,
       bioseq.cigar(t.seq) AS cigar
FROM targets AS t
JOIN reads AS q ON mmsearch(t.seq, q.seq)
WHERE t.id > 20 AND q.id <= 100;
```

There are several possible plans for searches:

- Search before a join: A base `BioseqScan` can search a target against a constant query. This is the most straightforward operation, but isn't possible for the described join.
- Search with an outer parameter: Postgres can create a nested-loop plan that used `BioseqScan` inside. This was discussed in scan list 8.2.3 and causes the target to be loaded multiple times, which reduces performance heavily.
- Search at the join: `BioseqBatchJoin` replaces the nested-loop plan and allows to keep the target index loaded only once, while iterating over queries. This is the most efficient plan for the described join.

Interesting thing happens if we want to make two searches for the same row. Consider the example below:

Listing 8.10: Two searches with one ambiguous target binding.

```
-- Unsupported: this will cause an error
SELECT t.id, bioseq.dp_score(t.seq)
FROM targets AS t
WHERE mmsearch(t.seq, $1::bioseq.seq('dna5', 64))
  AND mmsearch(t.seq, $2::bioseq.seq('dna5', 64));
```

In this case due to my choise of SQL API, we don't know which score `bioseq.dp_score(t.seq)` refers to as we have two potential results with different scores. To resolve this issue we can do a self-join on the same table:

Listing 8.11: Give each independent search its own target binding.

```
-- Supported: this is valid use-case within BioseqDB
SELECT a.id,
       bioseq.dp_score(a.seq) AS score_a,
       bioseq.dp_score(b.seq) AS score_b
FROM targets AS a
JOIN targets AS b ON a.id = b.id
WHERE mmsearch(a.seq, $1::bioseq.seq('dna5', 64))
  AND mmsearch(b.seq, $2::bioseq.seq('dna5', 64));
```

Now, let a_i be hit i of the search bound to alias `a`, and let b_j be hit j of the search bound to alias `b`. Figure 8.4 shows their combinations for one target row. We can see that natural extension of the used syntax would be to generate cross-product of hits. It's worth noting that currently we are not able to express search-specific limits 8.3.5 meaning that there's no optimisation for example if we wanted to have only best-hit per each sequence (only tuple with best score on both searches). In that case all tuples needs to be evaluated and user can construct a query manually to select the highest-scoring hits:

Listing 8.12: Keep one highest score from each of the two searches, per stored row.

```
SELECT a.id,
       MAX(bioseq.dp_score(a.seq)) AS score_a,
       MAX(bioseq.dp_score(b.seq)) AS score_b
FROM targets AS a
JOIN targets AS b ON a.id = b.id
WHERE mmsearch(a.seq, $1::bioseq.seq('dna5', 64))
  AND mmsearch(b.seq, $2::bioseq.seq('dna5', 64))
GROUP BY a.id;
```

The intereesting part is that self-joins in relational databases are commonly used for hierarchical structures (trees), detect duplicates, extract consecutive elements (and other comparison operations). In this case it's used to create new alias for the column without any additional operations.

There is however one interesting exception that is a bug in the BioseqDB code (initially I supported conflicting calls under one alias and there is leftover branch of the code that still supports it). Users should be aware that there still can be some undiscovered edge cases that are not correctly detected. I initially planned to support same-alias searches if no feature is used (we don't specify CIGAR, score etc.), but this results in usability issue that adding these properties suddently requires user to rewrite the query completely. For this reason I later decided to disable any conflicting alias searches even if we don't specify any properties in the `SELECT` list.

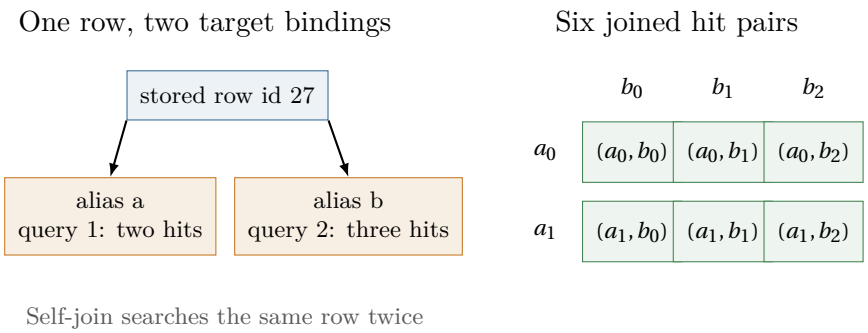


Figure 8.4: Two hits from the first search and three from the second produce six combinations. Additional hit conditions may select subset of this cross-product matrix, but this is responsibility of the user as I don't provide any specialised optimisation for example to extract only one best-hit for each search.

Listing 8.13: Low priority bug that still exists in the code

```
-- This shouldn't be possible, but currently works (bug)
SELECT t.id, bioseq.dp_score(t.seq) AS score
FROM queries AS q, targets AS t
-- index_read_requirement is a special debug parameter to disable BioseqDB from
    loading index
-- for some reason enabling it allows you to trigger specific BioseqJoin path
    that doesn't correctly detect the conficting calls
-- The queries table needs to be normal table, not defined using "USING bioseq"
WHERE mmsearch(t.seq, q.seq, aligner_options(index_read_requirement => 1))
  AND mmsearch(t.seq, 'ACGTACGTACGTACGT'::bioseq.seq('dna5', 64));
```

8.5. Caching results

As mentioned before I provide layers of caching:

- Chunk (strip) cache that stores rows of columnar stored (compressed blobs that can potentially store multiple sequences) 8.3. Described more in detail in next chapters 9.1
- Lazy interface cache implemented by scanner slots 8.3
- I also provide long-term experimental cache layer for alignments results described here.

An alignment query may run for hours while returning a small result. If two sessions submit the same request, running the alignment twice wastes work. I created separate small `bioseq_cache` extension that is provided along with BioseqDB.

Experimental support

This chapter doesn't go into details of chache implementation, because it's still experimental. I didn't perform enough testing to be confident enough about all edge cases correctness as I needed to be very careful how to lock queries and split work (it needs comparing expression trees to detect query identity or overlap, serializing them to store them, parallel-safe coordination of locking). The subextension isn't loaded by default and user need to execute `CREATE EXTENSION bioseq_cache` manually and restard Postgres server. There are some problems with this approach as the MVCC Postgres visiblity can change during execution (there could be insert when alignment is running). This problem wasn't addressed so the user would see old version of the data. That's why I decided to disable caching now. Moreover the "excact results" setting is fully supported now and can be enabled if user loads extension, while "reusable fragments" have most of the code ready except that it was only tested on scalar types (INTs, FLOATs etc.) and sequence support will require some alterations (planner will fallback to "exact results" anyway)

The cache subextension coordinates requests and stores completed results. The extension provides two forms of reuse:

- Exact results: If user executes two identical queries at the same time, we provide special custom scan in plan that waits for currently running job and loads results from it. It needs to happen during planning stage - planner needs to detect that alignment pipeline is running and then during execution stage block execution until results are available.
- Reusable fragments: This is more complicated part of the code that doesn't completely block execution of the query. If similar query is currently running, planner will try to alter current query to still execute it, but try to cover region that is not covered by running jobs.

producer is the query runner that currently executes alignment job. In that timeframe another Postgres-spawned process with other user query waits for results. Submission of partial results would be possible, although very complicated to implement correctly. sealing here means that we store last tuple, collect some metadata like tuple counts and create new UUID for the result. Figure 8.5 shows what happens during the execution of two queries. The cache has expiration TTL time after which the results cannot be reused anymore.

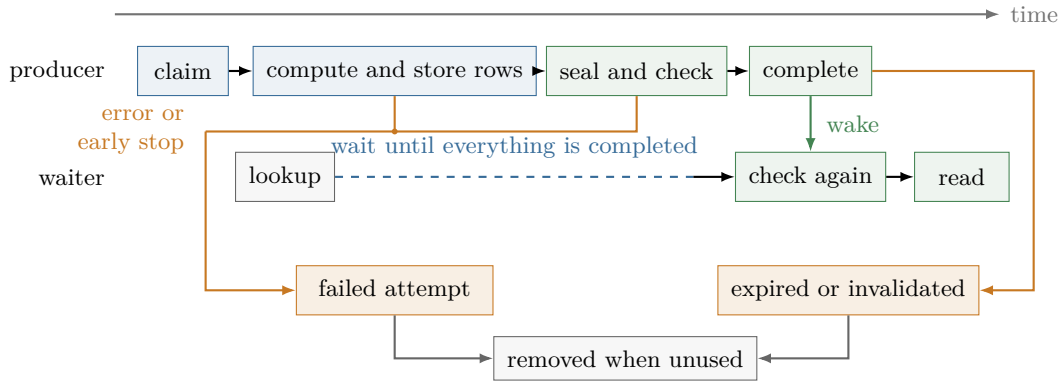


Figure 8.5: Consumer just waits for all results from producer and then reads them.

There's one lock that is taken by any consumer during execution just to access shared state. The consumer sleeps and preiodically acquires and releases the lock to check if it can continue execution (polling). If the producer fails or is cancelled I wake all consumers that registered for this job and then each of them will try manually run the query (so the consumer won't fail due to producer error or cancellation).

The interesting problem is that matching SQL text alone is insufficient. I needed to include analyzed query, parameter values, result types, relevant execution settings, and the database context objects to make sure we reliably detect identical queries. Additionally I keep track of query dependencies (all relations) and if the relation changes (row insert) during processing, then cache will be invalidated. This means that insert during consumer's wait time will cause the consumer to execute query by themselves (although this require more testing to confirm correctness)

8.5.1. Reusing fragments of queries

Suppose that table is kept unchanged, but user slightly changes some metadata condition (for example instead of aligning all sequences with dates between some range, we extend the range to include few more dates). For indentiy comparison I store the analyzed query plan as well so we can say what filter conditions were used for that cache entry.

The general idea of describing what is cached, reading useful part and computing remainer was very loosely inspired by semantic caching paper[17]

For example, select input rows with $40 \le id < 100$. Completed sources cover $0 \le id < 60$ and $50 \le id < 80$. We read $[40, 60)$ from the first source, $[60, 80)$ from the second, and compute $[80, 100)$. Let's define it formally via predicates:

- $T(t)$: t is an input tuple
- $A(t, h)$: h is tuple with alignment of tuple t
- $p(t)$: the new SQL predicate P evaluates to `TRUE` on t
- $c_i(t)$: cached source i has tuple with alginment for which the original predicateP_i evaluates to `TRUE` on t.

[17] Dar, Franklin, Jónsson, Srivastava, and Tan, Semantic Data Caching and Replacement, VLDB 1996, pp. 330–341.

Order the selected cached sources from 1 to m, source i supplies part B_i, while the reminder we have to compute is D:

$$B_i = \{(t,h) \mid T(t) \wedge A(t,h) \wedge p(t) \wedge c_i(t) \wedge \forall j < i : \neg c_j(t)\}, \tag{8.1}$$

$$D = \{t \mid T(t) \wedge p(t) \wedge \forall i \in \{1,\ldots,m\} : \neg c_i(t)\}. \tag{8.2}$$

Each requested hit is returned from the first valid source covering its input, or computed from D:

$$\{(t,h) \mid T(t) \wedge A(t,h) \wedge p(t)\} = B_1 \uplus \cdots \uplus B_m \uplus \{(t,h) \mid t \in D \wedge A(t,h)\}. \tag{8.3}$$

Here $\uplus$ joins disjoint sets of occurrences. This simple formal description can be implemented as large `UNION ALL` operation of the entries in cache. Figure 8.6 shows the same example, including the overlap between the two sources.

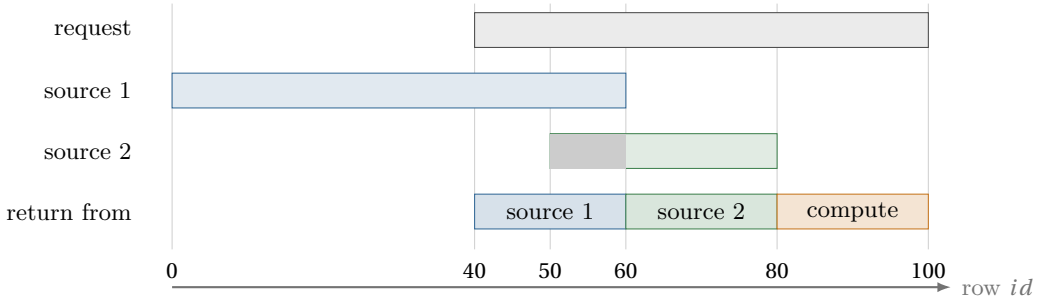


Figure 8.6: Example of resuable query with one predicate on the primary ID column.The new request uses source 1 for $[40,60)$, source 2 for $[60,80)$, and computes $[80,100)$. The gray overlap in source 2 is omitted because source 1 already contains those occurrences. Cached rows outside the new request are also omitted.

With both sources valid, the missing inputs have this equivalent SQL description:

Listing 8.14: Input rows on which the expensive operation is still needed.

```
SELECT id, seq
FROM sequences
WHERE id >= 40 AND id < 100 -- New query we want to compute
  AND (id >=  0 AND id < 60) IS NOT TRUE -- Minus source 1 results
  AND (id >= 50 AND id < 80) IS NOT TRUE; -- Minus source 2 results
```

There are few important details about the implementation:

- I need to correctly handle NULLs For a nullable predicate, `NOT P` also becomes unknown when P is unknown. `P IS NOT TRUE` includes both false and unknown, so it keeps every input excluded from the earlier cache.[18]
- Handle source failures If source 1 fails, c_1 is falsethen D should become $[40,50) \cup [80,100)$ so on error we don't abort execution and just adjust computed region (currently this is realised by adding barrier for each query and computing D at the end)
- Column differences There could be a case in which source supplies only subset of columns. I don't optimise this scenario as computing new columns still requires running entire pipeline again, so we assume here that the column list is always the same.

[18] PostgreSQL 17, Comparison Functions and Operators, Table 9.2.

Chapter 9

Physical sequence storage in PostgreSQL

9.1. Store sequences together with their metadata

I already mentioned before for example when discussing sequence abstractions 7.1 that I usually operate on locators, that just point to location where actual sequence is stored.

PostgreSQL already has mechanism for that: it stores large values out of line (outside actual row) through TOAST.[1] This mechanism however, has it's limitations - it's storage is limited to 1GB per row, which is an effect of maximum width of integer storing row's OID (object indentifier is internal numbering scheme in Postgres). If we assume that DNA4 sequence is bit-packed and it's entire human genome (lengths of around 3.1 - 3.2 Gbps) this gives us conservatively $3 * 10^9 * \frac{2}{8}B = 750MB$ of storage per row. If we want to store both pairs of chromosomes we can pesimisitcally require $1.6GB$ of storage per row. In most cases researchers split the sequence into smaller chunks, but this is manual process requiring additional tools.

Moreover, most of the workflows are OLAP-type.: user is expected to do bulk inserts, do not perform frequent updates and deletes when running alignment. Usual TOAST wouldn't take advantage of that specific workflow assumption as we cannot read TOAST in bulk.

That's why I decided to implement custom TOAST-like mechanism in BioseqDB and use columnar storage appropiate for OLAP workflows:

- User table holds all the data in columnar format, where the sequence column itself is replaced with small locator object that points to sequence fragment.
- Sequence fragments are stored in multiple sharded tables that are normally invisible to the user. All of these tables also use columnar storage, so short sequences are treated the same way as metadata in columnar context - they will be merged into one batch. I call these batches of rows "strips" in the code.
- Persistent indexes are separate entities stored as tables that are normally not visible to the user as well. They are responsible for providing pre-computed objects for sequences. There's small abstraction layer in BioseqDB which allows to extend it by adding different types of indexes that provide different information.

[1] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: TOAST.

Adapted from Citus and Hydra Columnar

BioSeqDB's columnar layer adapts code from both Citus Columnar and Hydra Columnar.[a] Hydra extended the Citus code with features including very primitive vectorized SQL execution and a cache of stripes. In BioSeqDB I modernized the adapted C code by rewriting it completely into C++26 modules with automatic code generation and added support for newer PostgreSQL 19.0 (which in case of Hydra original codebase was not supported). Only small of the original code was left untouched. This code was used as a low-level storagebase, where concepts like sequence locators, fragment readers, and sequence indexes are purely my innovations.

[a] Citus Data contributors, Citus Columnar storage implementation; Hydra contributors, Hydra Columnar: PostgreSQL columnar storage extension.

The linked Hydra columnar documentation was unavailable as the theam building it moved to create a different database system. Although I was basing my implementation in the Hydra codebase, understanding the chunk operations required multiple manual investigations to understand it completely.[2] Current BioseqDB storage design is an effect of this process.

It's worth noting that I mention both Citus[3] and Hydra sources here as Hydra was fork of Citus that diverged at some point to later be abandoned. Citus columnar is still actively maintained but it lacks features like row deletion, vectorization and it's very simple columnar interface. Meanwhile Hydra offered these features, but due to being abandoned didn't correctly build for newer Postgres version and lacked bugfixes done by Citus team since the forking moment. Modernizing the Hydra codebase thus involved not only manually inspecting the code, but also tracking commits between Hydra and Citus. This work thus was complicated and very time-consuming involving also detection of new bugs in Hydra code that were undected after retirement of the project.

9.2. Columnar storage

ClickBench is a benchmark for analytical DBMS and it also reported results for both Hydra and Citus. Figure 9.1 uses historical benchmark submissions on AWS machines.[4] The benchmark consisted of around 43 queries over a dataset of around 100 million rows (3 runs with minum of second two taken as into the sum).

Both columnar submissions store about 19 GB, compared with PostgreSQL's 77.8 GB. This was due to support of compression on stored columns. Hydra[5] had much better performance due to introduction of vectorized SQL execution for aggregates and column cache that wasn't present in Citus.[6] These results do not translate directly to BioseqDB as the benchmarks tests some arbitrarily-built queries, however it motivated me to put big effort into modernizing Hydra codebase and incorporating it into BioseqDB as mentioned in previous section 9.1.

[2] Hydra contributors, Hydra Columnar: PostgreSQL columnar storage extension.
[3] Cubukcu et al., "Citus: Distributed PostgreSQL for Data-Intensive Applications."
[4] ClickHouse contributors, ClickBench: analytical database benchmark.
[5] Hydra contributors, Hydra Columnar: PostgreSQL columnar storage extension.
[6] Cubukcu et al., "Citus: Distributed PostgreSQL for Data-Intensive Applications."

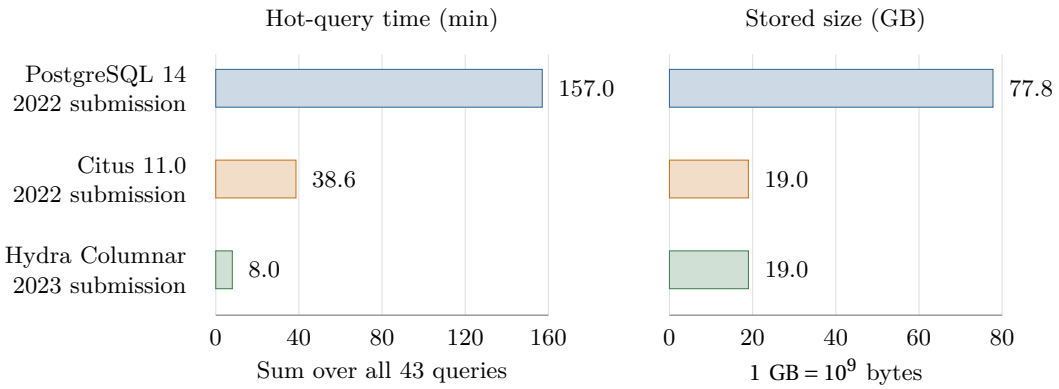


Figure 9.1: Historical ClickBench, calculated from the repository snapshot of 20 September 2023. Each submission uses `c6a.4xlarge` machine with 16 vCPUs, 32 GiB RAM, and 500 GB of GP2 storage. PostgreSQL and Citus results are dated 1 July 2022, and Hydra results 19 September 2023. The times use the sum of shorter of runs two and three for every query. Note that these are "best-fit" results so the configuration was different. It's good however for quick comparison of performance.

9.2.1. Columnar layout

The columnar layout has three levels:

- A stripe is a batch of chunks coresponding to set of rows. Normally one stripe holds maximum is 150,000 rows.
- A chunk group is a group of chunks for the same rows inside a stripe. By default one chunk group holds maximum of 10,000 rows.
- A column chunk is the most basic entity that just holds values for one group as a one column. It also stores aggregated values (count, min, max) to enable faster aggregate operations (vectorization) and the bitmask (marking which values are NULLs).

Stripe and chunk-group sizes are options per table, where larger ones allows more efficient compression and smaller ones allows more fine-grained read access.[7]

Figure 9.2 shows this model for the six rows. By default Postgres row-based storage (heap tuple) stores `id`, `sample_id`, and the sequence locator together. The stripe is columnar representation of these rows. Each column occupies its own sequence of chunks. A chunk group is only an abstract concept for chunks that are related to the same rows. The last group of a stripe can be shorter if total row count isn't a multiple of chunk group size.

[7] Davis, Citus 10 brings columnar compression to Postgres.

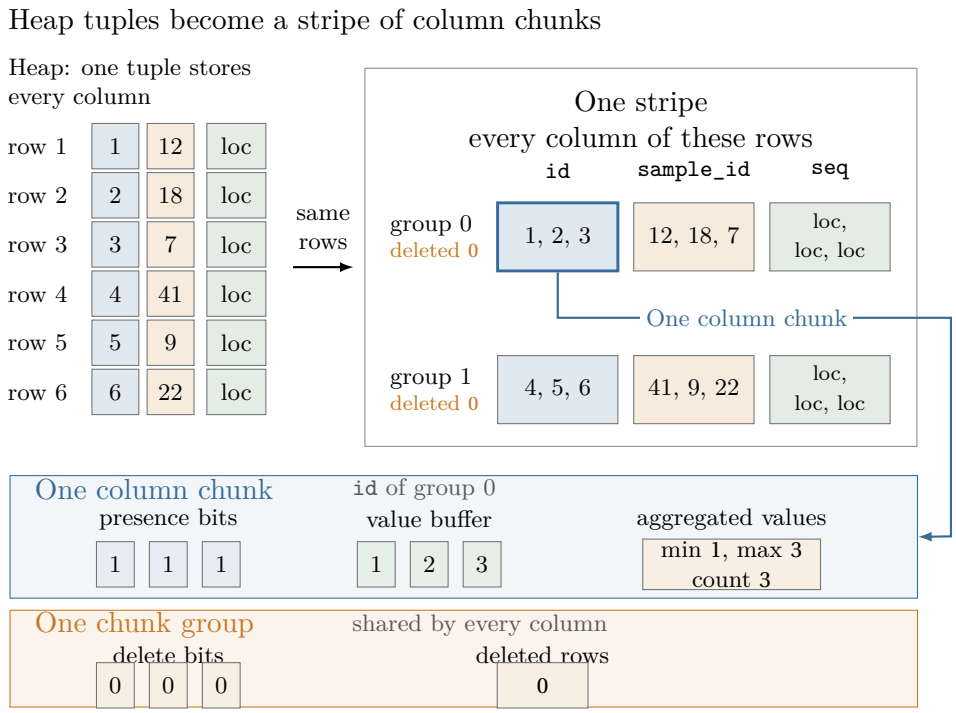


Figure 9.2: The same six rows as heap tuples and as one stripe. Matching colours mark the same column. The blue callout is one column chunk: presence bits, present values, min/max, and slot count. The orange callout is the chunk group: a deletion mask and `deleted_rows`

When I want to store row, each time the writer append this row to the current chunk group (in memory). After sufficient number of row is reached (max chunk group size) the code serializes each column chunk to a stripe buffer that still is in memory If the number of rows per stripe is reached (max stripe size) then stripe is "flushed", which means writing the stripe. Commonly the maximums of stripe row counts will be hit, but there's additional check if the current chunk group for specific column buffer exceeds $1GB$ limit, then it forces flush of the entire stripe. This means that if we store large BLOB along very small column, the BLOB will force the stripe to end i.e stripes always need to account for the same number of rows in each column (this also implies that the decision to store sequences via locators increases efficiency of that storage model as more tuples with non-sequence fields can be packed into single stripe). Additionally there's hard limit on maximum size of $256MiB$ for a single value in row.

> **Storing large BLOBs**
>
> Users should avoid storing large BLOBs along small columns - either store BLOBs of similar sizes together or only store smaller payloads in the same table. This is potential optimisation to be implemented in the future, although it's very low priority.

The figure 9.3 shows 3 main stages: we save tuples first to accumulate enough of them, then serialize them to a buffer and when stripe's buffer eaches $1GB$ or the strip max row count, only then it's flushed. Algorithm 18 presents pseudocode for that process as well.

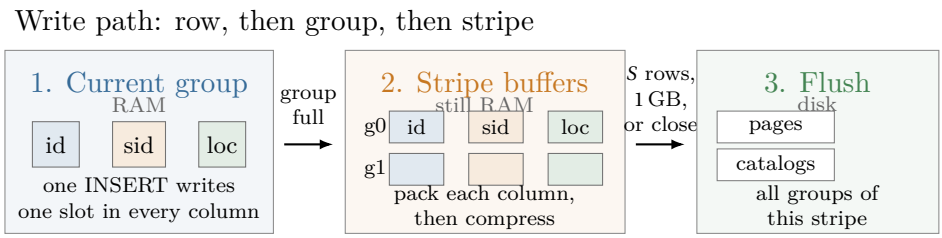


Figure 9.3: How we pack chunks groups into stripe buffers, then to flush them if necessary.

Algorithm 18: Appending rows

Require: columns C, group limit G (default 10,000), stripe limit S (default 150,000), comparators Compare_c (possibly NULL)

$G_{\max} \leftarrow \lceil S/G \rceil$
$s \leftarrow$ NULL — open stripe, or none

procedure AllocateNewStripe
  $s.n \leftarrow 0$ — rows already in this stripe
  $s.P \leftarrow 0^{|C| \times G}$ — presence bits of the current group
  $s.B \leftarrow \varepsilon^{|C|}$ — serialized present values of the current group
  $s.K \leftarrow \{\min : \bot, \max : \bot, cnt : 0\}^{|C| \times G_{\max}}$ — skip min, max, and slot count of every column chunk
  $s.\text{chunk} \leftarrow \{P : \varepsilon, V : \varepsilon\}^{|C| \times G_{\max}}$ — packed streams of every serialized group
  $s.N \leftarrow 0^{G_{\max}}$ — row count of each serialized group
  return s

procedure WriteRow(v)
  if $\text{IsOpen}(s) \wedge \exists c \in C : \text{Size}(s.B_c \| v_c) > 1\,\text{GB}$ then
    FlushStripe(s)
  if $\neg\text{IsOpen}(s)$ then
    $s \leftarrow$ AllocateNewStripe
  $g \leftarrow \lfloor s.n/G \rfloor$, $r \leftarrow s.n \bmod G$
  for all $c \in C$ do
    if $\text{IsNull}(v_c)$ then
      $s.P_{c,r} \leftarrow 0$
    else
      if $\text{Align}(v_c) \geq 256\,\text{MiB}$ then
        error
      $s.P_{c,r} \leftarrow 1$
      $s.B_c \leftarrow s.B_c \| \text{Serialize}(v_c)$
      if $\text{Compare}_c \neq$ NULL then
        $s.K_{c,g}.\min \leftarrow \min_{\text{Compare}_c}(s.K_{c,g}.\min, v_c)$
        $s.K_{c,g}.\max \leftarrow \max_{\text{Compare}_c}(s.K_{c,g}.\max, v_c)$ — $\bot$ is the identity
    $s.K_{c,g}.cnt \leftarrow s.K_{c,g}.cnt + 1$
  if $r = G - 1$ then
    SerializeGroup(s, g, G)
  $s.n \leftarrow s.n + 1$
  if $s.n \geq S$ then
    FlushStripe(s)

procedure SerializeGroup(s, g, k)

```
35:     s.N_g ← k
36:     for all c ∈ C do
37:         s.chunk_{c,g}.P ← PackBits(⟨s.P_{c,r}⟩_{r=0}^{k-1})
38:     for all c ∈ C do
39:         s.chunk_{c,g}.V ← Compress(s.B_c)   identity if incompressible
40:         s.B_c ← ε

41: procedure FlushStripe(s)
42:     k ← s.n mod G
43:     if k > 0 then
44:         SerializeGroup(s, ⌊s.n/G⌋, k)
45:     m ← ⌈s.n/G⌉
46:     for all c ∈ C do
47:         Write(⟨s.chunk_{c,g}.P⟩_{g=0}^{m-1})
48:         Write(⟨s.chunk_{c,g}.V⟩_{g=0}^{m-1})
49:     WriteCatalog(s)
50:     s ← NULL

51: procedure WriteCatalog(s)
52:     Write(⟨s.N, s.K, 0^{s.n}⟩)   Saves row count for group, aggegate stats and deletion mask
```

In's important that closing the writer also flushes any leftover stripe, so all in-memory buffers are saved to disk when transaction is committed.

As presented on figure 9.2 column chunk stores a presence bitmap, a value buffer of only the present values, and aggregated values used later for delettion and vectorized aggregation: the smallest and largest non-NULL values (when the type has a comparison function), and a row count (`value_count`) counting all values in the chunk including NULLs.

To support deletion, the chunk group stores `deleted_rows`, along with separate row mask representing which rows were deleted. `DELETE` only sets the matching mask bit in the chunk group's mask and increments `deleted_rows`. This allows BioseqDB to offer fast deletion without the need to rewrite the columns themselves. The wasted storage can be however reclaimed:

- `VACUUM (TRUNCATE)` may rewrite last stripes in table whose undeleted rows still fit in one stripe and whose decompressed size is below 1 GB. A single last stripe is rewritten only when more than 20% of its rows are deleted. The rewrite copies undeleted rows into a new stripe and drops the old ones.
- The internal tables storing sequence fragments cannot be rewritten as each locator needs to store the row number. Supporting VACUUM for sequence fragments would need to be dispatched to all tables including user table. Currently VACUUM is only supported on the user table.
- `VACUUM FULL` can rewrite the whole table to reclaim wasted space.

9.2.2. Smart reads

In BioseqDB it's unnecessary to read all chunks all the time. The two optimisations are:

1. Projection means that only selected columns from `SELECT` query are red. This is trivial optimisation.

2. Min/max pruning is more interesting one. For an ordered column, the writer records the smallest and largest non-NULL values in each chunk (in aggregates fields seen in figure 9.2). The BioseqDB TAM presented in figure 8.2 can determine if none of the values in the chunk group satisfy the filter. If that happens, entire chunk group is skipped. For example, a group with sample identifiers from 1 to 100 cannot satisfy `BETWEEN 500 AND 600`.

Figure 9.4 shows both optimisations. Group that satisfies both of these conditions still needs to apply filters to each row individually, but I save time but not touching groups that are not revelant to the search. The figure assumes query `SELECT id FROM table WHERE sample_id BETWEEN 500 AND 600` is executed.

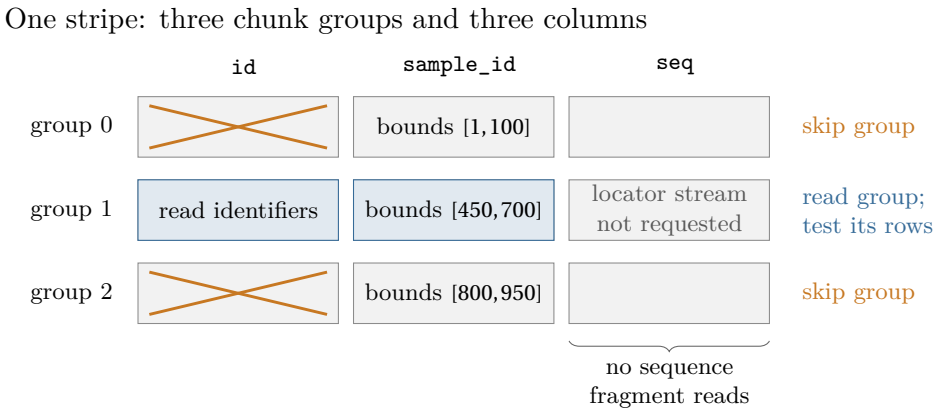


Figure 9.4: Column selection and group pruning for simple query `SELECT id FROM table WHERE sample_id BETWEEN 500 AND 600`. Blue cells are read, orange crosses mark groups whose aggregates exclude all the rows. Unrequested sequence locators remain unread, because they were not specified in the `SELECT` clause. The drawing shows only logical representation of chunks, not actual disk layout.

Prunning for sequence columns

Sequence columns with locators are excluded for min/max pruning, but it's possible to implement that feature theoretically. I decided not to do it, because storing min/max values would be simple as locator isn't big object, however computing these values for very large sequences will be computionally expensive and would negatively impact table inserts.

9.2.3. Compression

Figure 9.5 shows in detail how presence bitmap is computed. If the NULLs are very frequent, I will save the space by not storing them.

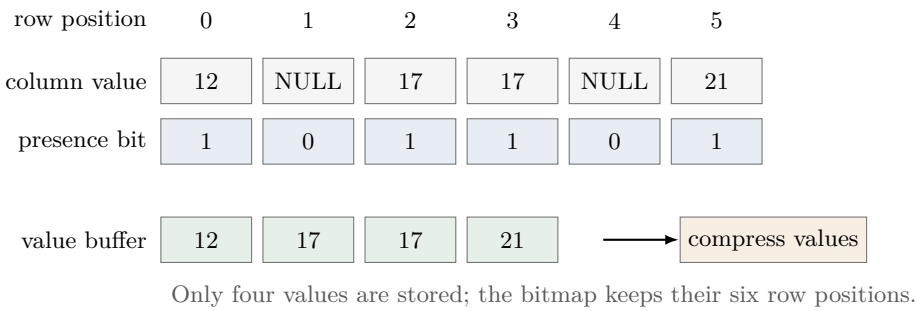


Figure 9.5: Presence bitmap is useful to enable better NULL compression

The Hydra code that BioseqDB columnar TAM was based on supported PGLZ, LZ4, or Zstandard, depending on the build and table options. In BioseqDB all compression is disabled by default as I would like to benchmark in the future best (and more modern) compression algorithms decicated for example to sequence storage. Abadi et al. show that the choice of compression method depends on both the data and the query workload,[8] so I leave it as future extension.

9.3. Raw byte storage

After stripe is serialized and flushed it needs to be written to disk - function *Write* and *WriteCatalog* in the pseudocode 18

- Physically first written is the presence bitmap, then the value buffer for a column, then each column follow. These are placed in the user relation database pages.
- Catalog (row counts, aggregates and deletion mask) is written separately from the stripe data and it's placed in usual heap rows i.e BioseqDB introduces separate relations for storing chunks information.

Catalog is represented by four relations inside the BioseqDB:

[8] Abadi, Madden, and Ferreira, "Integrating compression and execution in column-oriented database systems."

Listing 9.1: Metadata catalogs for flushed stripes.

```
CREATE TABLE bioseq_internal.stripe (
  storage_id bigint,
  stripe_num bigint,
  file_offset bigint,
  data_length bigint,
  column_count integer,
  chunk_row_count integer,
  row_count bigint,
  chunk_group_count integer,
  first_row_number bigint
);

CREATE TABLE bioseq_internal.chunk_group (
  storage_id bigint,
  stripe_num bigint,
  chunk_group_num integer,
  row_count bigint,
  deleted_rows bigint
);

CREATE TABLE bioseq_internal.chunk (
  storage_id bigint,
  stripe_num bigint,
  attr_num integer,
  chunk_group_num integer,
  minimum_value bytea,
  maximum_value bytea,
  value_stream_offset bigint,
  value_stream_length bigint,
  exists_stream_offset bigint,
  exists_stream_length bigint,
  value_compression_type integer,
  value_compression_level integer,
  value_decompressed_length bigint,
  value_count bigint
);

CREATE TABLE bioseq_internal.row_mask (
  id bigint,
  storage_id bigint,
  stripe_id bigint,
  chunk_id integer,
  start_row_number bigint,
  end_row_number bigint,
  deleted_rows integer,
  mask bytea
);
```

When writing stripe data I need to take care about page boundaries. Let BLCKSZ be the PostgreSQL page size in bytes (this is compile-time constant provided by Postgres), H the page-header size (technically sizeof(PageHeaderData)),[9] and $P = \text{BLCKSZ} - H$ the remaining

[9] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Database Page Layout.

page capacity. If $x \geq 0$ is the total offset of the x-th byte, then page coordinates are defined as:

$$\mathbf{block}(x) = 2 + \lfloor x/P \rfloor, \qquad \mathbf{position}(x) = H + (x \bmod P).$$

BioseqDB will write multiple pages for each stripe, so I need to take additional care about buffers going through multiple page boundaries as seen on figure 9.6.

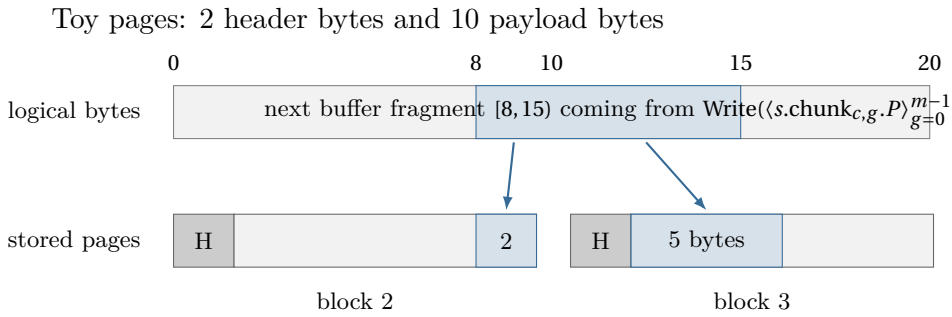


Figure 9.6: I use logical offsets $\mathbf{block}(x)$ and $\mathbf{position}(x)$ to hide page boundaries. Diagram shows that we want to write some fragment of in-memory buffer $[8, 15)$ to the disk, but it needs to be split across multiple pages. In reality the dimensions are larger than just 20 bytes, but it illustrates the idea.

The original idea comes from Citus page writer that it's avaialble in their public repository

Citus columnar page mapping
columnar_storage.c 118

To allow concurrent access (queries during inserts) I need to note that the writer does the following things:

- Reservation: Identifiers for chunks and groups is done under the relation extension lock.
- Write: Writes to pages are done using PostgreSQL buffer locks and generic write-ahead logging (WAL).[10]
- Record completed data: The logic uses Postgres' Snapshots so data will be visible only after the transaction is committed.

9.4. Locator implementation

Sequence locator are implemented as 40-byte struct in C++. The struct must also include the Postgres header so it's effectively 56 bytes in total. The Postgres header stores alphabet code and base storage unit as `TYPMOD` field (custom field for storing type modifier). I don't provide small-sequence optimisation in BioseqDB, because that would complicate the implementation (instead of storing locator for small sequences we can potentially store sequence directly in the user table. This would require storing it's padding so potential maximum sequence length that could be stored inline would be $(40B - 1B) * 8/2 = 156$ symbols for DNA4).

[10] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Generic WAL Records.

Listing 9.2: The 40-byte locator stored after the 16-byte sequence header.

```
struct sequence_locator {
  std::uint8_t  payload_version;   // format version used by me to change the locator format
    (currently it's just set to 3)
  std::uint8_t  bucket_index;      // length bucket 0..34
  std::uint16_t reserved_16;       // pad so shard_index is 4-byte aligned, must be 0
  std::uint32_t shard_index;       // fragment relation inside the bucket
  std::uint32_t fragment_count;    // consecutive fragment rows
  std::uint32_t reserved_32;       // pad so logical_length is 8-byte aligned, must be 0
  std::uint64_t logical_length;    // sequence length in symbols
  std::uint64_t first_fragment_row;// row of fragment 0
  std::uint64_t source_epoch;      // immutable source version, must be > 0
};
```

- the internal tables are divided into groups by sequence length and these groups are divided into shards (so I record both of these value to identify sequence)
- the number of row for sequence first fragment and number of fragments (i.e number of rows occupied by sequence)
- sequence length in symbols (logical length)
- version of the sequence in case it's changed (source epoch)
- version of locator format (not relevant for users)

Locator format optimisation

Currently locators size could be reduces as I don't need to store version (it was used for developent) and I waste too much space for padding. It's potential oppornunity to optimsie in the future (it wasn't a priority as locators occupy overall small space in comparison to sequences themselves).

Each sequence as mentioned in section desribing sequence interfaces 7.2 has it's alphabet type and base storage unit. Base store unit is configurable from SQL. User can declare any power of 2 that is width of supported machine integer:

Listing 9.3: Examples of different base storage units

```
SELECT 'ACTGTGCAACCCCTG'::bioseq.seq('dna4', 64);
SELECT 'ACTGTGCAACCCCTG'::bioseq.seq('dna4', 32);
SELECT 'ACTGTGCAACCCCTG'::bioseq.seq('dna4', 16);
SELECT 'ACTGTGCAACCCCTG'::bioseq.seq('dna4', 8);
-- Example for DNA5 (floor(64/3) = 21 symbols per unit)
SELECT 'ACTGTGCAACCCCTG'::bioseq.seq('dna5', 64);
-- This is illegal as there's no uint128_t. 128-bit unit is just equivalent to
    two 64-bit units.
SELECT 'ACTGTGCAACCCCTG'::bioseq.seq('dna4', 128);
```

Sequence fragments

Sequences are divided into fragments to enable lazy loading. A fragment is 1024 packed storage units, so actual bytes occupied depend by the width of storage supplied by the user. The maximum possible fragment size is for 64-bit storage unit and it's $1024 * 8B = 8192B$ (note that this goes beyond default postgres page size of $8192B$, because page needs to include the page header as well)

Fragment abstraction is a separate layer above columnar storage. Shards for fragments are internally held as `CREATE TABLE ... (seq bioseq.seq(...)) USING bioseq`, so each fragment is one TAM row in that single sequence column.

Storage considerations

Fragments always storing 1024 storage units which works well for long sequences, however if the sequence is short then only small number of storage units will be occupied, while the rest is filled with zeros. Compression (default is pglz, but zstd is avialable as well and others for manual building) will take care of this problem and shrink the fragment. However it still has measureable effects on processing in the pipeline as BioseqDB will waste space in memory for very short sequences. This along with locator format optimisation is a task for future work. If we assume occupancy rate of 50% then minimal sequence size for 32-bit storageshould be $1024 * \frac{32}{2} * 0.5 = 8192$ symbols and for smallest possible storage: $1024 * \frac{8}{2} * 0.5 = 2048$ symbols.

9.4.1. Two levels of sharding

Postgres tables have limits on the number of stored rows, so in the most extreme cases the amount of sequence data can be more than one table can hold, especially if we store many full human genomes. A (single copy i.e haploid) human genome is about 3.1 to 3.2 billion bases (3.2×10^9 symbols and the 32-bit DNA4 unit from the figure above). PostgreSQL documents a maximum table size of 32 TB at $8192B$ default page size.[11] That bound is a result of internal Postgres 32-bit page index.

$$
\begin{aligned}
F &= 1024 \times \frac{32}{2} = 16384 && \text{symbols per fragment} \\
n_{\text{frag}} &= \left\lceil \frac{3.2 \times 10^9}{F} \right\rceil = 195313 && \text{fragment rows per genome} \\
B &= 16 + 1024 \times 4 = 4112 && \text{bytes per fragment row} \\
P &= 8192 - 24 = 8168 && \text{usable bytes per page} \\
n_{\text{page}} &= \left\lceil \frac{n_{\text{frag}} B}{P} \right\rceil = 98327 && \text{pages per genome} \\
n_{\text{seq}} &= \left\lfloor \frac{4294{,}967{,}294}{n_{\text{page}}} \right\rfloor = 43680 && \text{genomes in one table}
\end{aligned}
\tag{9.1}
$$

Without compression, one fragment table can therefore hold about 43,680 human-genome sequences before it runs out of pages. Practically compression rates for human genomes were already measured Kryukov et al., "Nucleotide Archival Format (NAF) enables efficient lossless reference-free

[11] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Appendix K. PostgreSQL Limits.

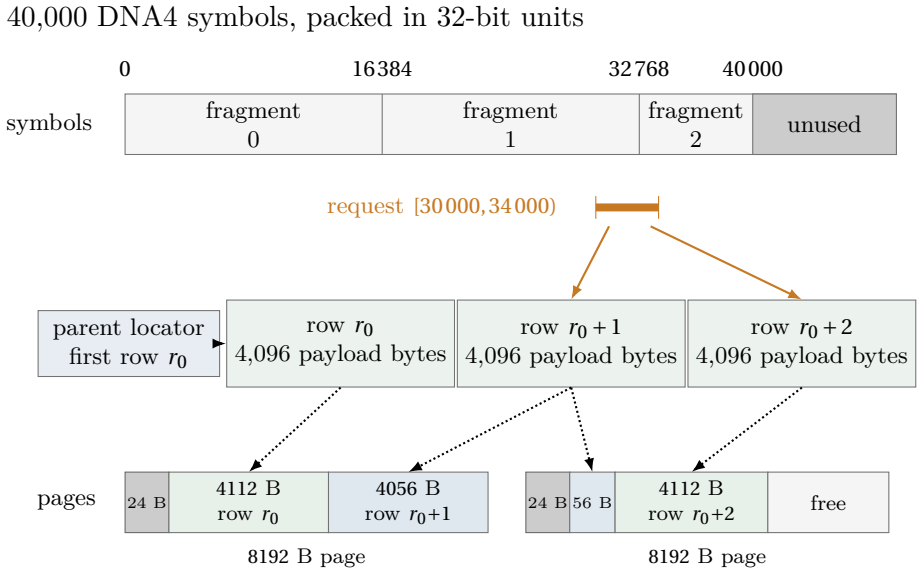


Figure 9.7: A 40000-symbol DNA4 value with 32-bit units uses $1024 \times 32/2 = 16384$ symbols per fragment, so two full fragments and one partial fragment. Request $[30000, 34000)$ reads fragments 1 and 2. Each stored row is 4112 bytes (4096 payload plus the 16-byte sequence header). The TAM concatenates those in column buffers via writer code 18 and during flush, writes them on 8192-byte pages, skipping the 24-byte `PageHeaderData` (8168 usable bytes). One fragment fits in a page, but two concatenated fragments do not ($8224 > 8168$), so fragment 1 continues after the next header. The compression will shrink the last fragment as it's mostly zeroed, but the illustration assumes no compression happened.

compression of DNA sequences" and zstd is capable of compressing a human genome to effectively 790 MB (7.9×10^8 bytes). With that size, each genome occupies 96,719 pages, so the same limit allows 44,406 genomes in one table.

Table limits

I present this calculation to present that hitting the table limit isn't very improbable. There exist datasets bigger than this limit. To prvent users from hitting limit and requiring them to manually spli tables, I decided to provide internal sharding.

The internal sharding allows fragments to spill over into multiple tables. If the underlying table achieves $1TB$ limit (which is lower than theoretical 32 Postgres limit), fragements are stored in newly created columnar relations. The sharding is two-level:

- length bucket: groups sequences by their logical number of symbols - precisely $max(0, min(\lfloor log_2(|s|) \rfloor - 8, 34))$ is the bucket index.
- shard: inside one length bucket BioseqDB can create multiple tables when existing tables reach $1TB$ limit (potentially to store extremally large datasets)

The current layout has 35 buckets. The first covers lengths $[0, 2^8)$ and each next bucket covers $[2^i, 2^{i+1})$ for $i = 8..40$. The largest bucket is designated to store sequences longer than 2^{41} symbols.

Sharding limitations

It's very important to note that current during stream access policy 7.5 the sequence size is not known until its end is reached, thus my code in BioseqDB decides to store the sequence in the largest bucket (34) even when the sequence is short. This code path is utilised by IO submodule that loads sequences from FASTA file. This means that each sequence loaded from FASTA currently will be stored in the largest bucket. I plan to resolve this problem in the future - solution could be adding ability to IO submodule to search for first sequence end and assume all sequences are similar sizes. Alternatively I could write one sequence and then adjust the bucket after size is known. Both of these solutions however add additional overhead and complexity to the FASTA loader module. Additionally length-based sharding can allow better utilisation of fragments memory (so fragments are smaller even without compression 9.4 and they require fewer bytes when allocated in RAM)

9.5. MVCC and deletion for sequence fragments

I discussed MVCC support when talking about Postgres Snapshots in chapter about BioseqDB custom scans 8.3.4. In custom scan case support is rather simple and provided by the Postgres engine itself. It only required me to correctly use it. Storing fragments is entirely separate problem to visibility in columnar storage, because multiple queries can operate on locators, while some other process can modify sequence data in the same time.

Deleting sequence require doing two operations:

- Deleting row for fragment, which under the hood will mark the row as deleted in the row-mask metadata.

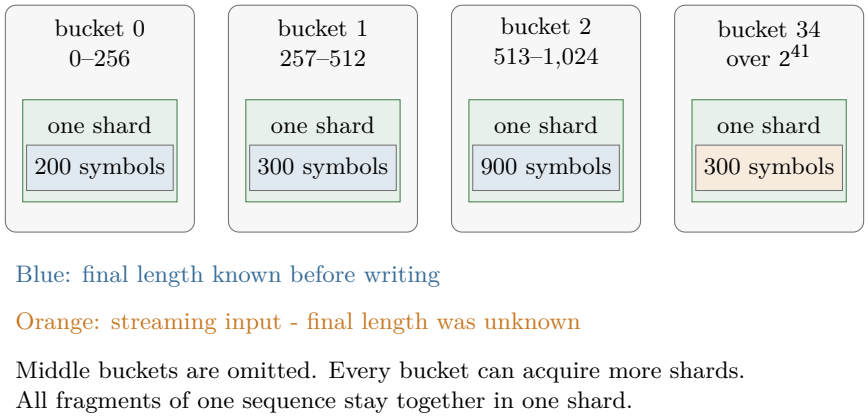


Figure 9.8: Length-based buckets are choosen based on the sequence length log 2, however sequences coming from FASTA/FASTQ have no known length and are streamed, thus internal code always store them in the largest bucket (34). There are multiple tables (called shards) per one bucket depending if we reach $1TB$ limit.

- As fragment is separate abstraction layer than columnar storage, I also store versions of rows in locators, so deletion just needs to invalidate that version (epoch)

Each epoch has stored separate row in the catalog table (so it's one entry per sequence version):

Listing 9.4: Heap catalog for one immutable sequence version. Invalidation updates `state`.

```
CREATE TABLE bioseq_internal.sequence_source_version (
  source_epoch bigint PRIMARY KEY, -- immutable version id, must be > 0
  user_relid oid,                  -- parent user table
  database_oid oid,                -- parent database
  tablespace_oid oid,              -- parent tablespace
  main_relfilenode oid,            -- parent file node
  main_storage_id bigint,          -- parent TAM storage generation
  column_attno smallint,           -- 1-based sequence column
  locator_version smallint,        -- locator format, currently 3
  bucket_idx smallint,             -- length bucket 0..34
  shard_idx integer,               -- shard inside the bucket
  fragment_relid oid,              -- fragment table, NULL if empty
  fragment_storage_id bigint,      -- fragment TAM storage generation
  first_sub_row_no bigint,         -- row of fragment 0
  num_fragments bigint,            -- consecutive fragment rows
  total_bp_size bigint,            -- sequence length in symbols
  alphabet_code smallint,          -- packed alphabet
  storage_unit_byte_width smallint,-- unit width in bytes
  padding_letters integer,         -- unused letters in last packed unit
  source_content_checksum bigint,  -- CRC-64 of decoded symbols
  state smallint                   -- 0 unpublished, 1 ready, 2 invalid
);
```

The specific epoch's `state` follows very simple linear transitions:

- Unpublished (0). An insert has allocated an epoch (sequence version) and written fragments. Locator is then filled with state 0 which is default one. It means that the

locator in the user table is not yet written (so the fragment is there, but there's no saved locator that points to it)

- Ready (1). Locator was written to a table, so I update the epoch state to mark that this sequence version is used somewhere (not doing that might lead to dangling fragments that occupy storage but have no existing locator). This state is crucial to prevent a situation in which locator write fails or is rolled back, but fragment was written.
- Invalid (2). Locator cannot read this row version anymore, the underlying fragment shouldn't be accessed. This happens when user either calls `DELETE` or `UPDATE`.

Note that the catalog contains far more information that is neccessary. I decided to save CRC hashes to make sure the fragment is not malformed during operations or shard coordinates shared by the locator itself. As each row version requires separate entry this limits the maximum number of sequences stored in the BioseqDB, however entries are cheap so I didn't incorporate it into estimate calculations 9.1.

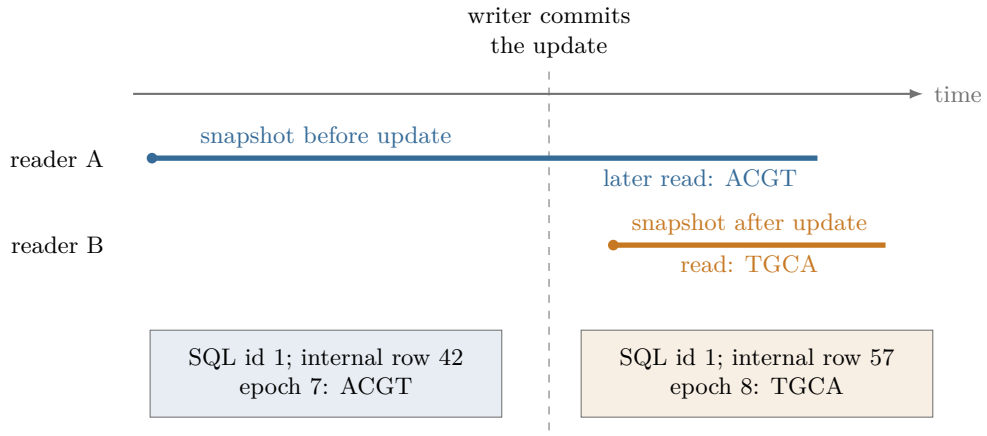


Figure 9.9: Postgres snaphot mechanism allows to check visivility of rows in MVCC model, but it's not sufficient on its own. Let's assume that reader A and reader B are reading the same sequence before and after other process commited update. Using fragments epochs versioning system means that read A will see older locator via snapshot, while reader B will get newer locator row with changed epoch. Both of them can complete reads as both epochs are still valid.

Listing 9.5 gives SQL code for this example:

9.5.1. BioseqDB all cache layers

The optional column cache stores decompressed stripe chunks. It is separate from the scanner-owned packed fragment slots in Figure 7.5.

The setting `bioseq.column_cache.enabled` controls the decompressed cache. The original benchmark campaign disabled it, whereas binary releases enable it by default. It stores decoded chunks inside one PostgreSQL backend process. Its key contains relation, stripe, chunk group, and column identifiers (the configured capacity when enabled defaults to 200 MiB)

The cache can return borrowed pointer to a decompression buffer for a chunk, which allows to prevent additional chunk loads wasting time on decompression multiple times. As

Listing 9.5: A repeatable-read transaction keeps the old sequence version. Initially, row id 1 begins with ACGTACGT.

```
-- Session A
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT bioseq.to_text(seq[0:8]) FROM reads WHERE id = 1;
-- ACGTACGT

-- Session B, while A remains open
BEGIN;
UPDATE reads
SET seq = repeat('TGCA', 4096)::bioseq.seq('dna4', 32)
WHERE id = 1;
COMMIT;

-- Session A
SELECT bioseq.to_text(seq[0:8]) FROM reads WHERE id = 1;
-- ACGTACGT
COMMIT;
SELECT bioseq.to_text(seq[0:8]) FROM reads WHERE id = 1;
-- TGCATGCA
```

I discussed all methods of caching within BisoeqDB already, I present table with comparison of all cache layers:

Table 9.1: Stored bytes and reusable memory have different owners.

Object	Retained data	Scope
PostgreSQL buffer	Relation page bytes	Internal buffer manager in Postgres pins each page and loads/unloads pages on demand so multiple accesses to the same page are efficient
Column cache 9.5.1	Decoded column value buffer	Can cache decompressed chunks, so loading and decompression of the same chunks can be efficient
Fragment slots 7.5	Copy of one packed fragment	Scanner-owned packed copies reused within a read session, independently of the column-cache setting.
Query-result cache 8.5.1	Completed SQL result rows	Separate `bioseq_cache` subextension can store and share results after pipeline execution is finished so same or similar queries can be efficient (experimental and disabled by default)

Chapter 10

Persistent sequence indexes

10.1. Keep the lookup work between queries

In previous chapter 9.1 I describe how columnar and sequence storage layers are implemented in BioseqDB. There's however also aspect for storing indexes for sequences. We would like to be able to store computed minimizers (seeds/sketches) 4.1 in a way that enables performant computations of alignments. They should be computed once when sequence is loaded from file or updated and then reused for subsequent select queries.

I want to discuss existing aligners' indexing first.

10.2. What existing sequence indexes store

10.2.1. Indexing: minimap2

After computing minimizers they need to be grouped by their hash, minimap2 choose to use the lowest b hash bits (by default $b = 14$) to choose one of 2^b buckets.[1] Minimap2 builds a hash table per bucket whose keys are the remaining hash bits. This buckets with hasmaps are serialized into `.mmi` file. I performed in-depth analysis of how the `.mmi` format is specified based on available Github old issues[2] and original code from minimap2's `mm_idx_dump` and `mm_idx_load` functions.

Writing and loading an index part
mm_idx_dump, mm_idx_load 475

The index file is concatenation of blocks presented in the Figure 10.1:

- Magic bytes The bytes are `MMI` followed by `0x02` (`MM_IDX_MAGIC`).
- Metadata Minimap2 saves w, the k-mer length k, used bitcount b, the number of sequences n_{seq} and a flag word (for various minimap2-specific bitflags)
- Sequence table For each of the n_{seq} sequences the file stores 1-byte name length ℓ followed by ℓ ASCII characters representing the sequence name (header taken from original FASTA file), and a 32-bit sequence length
- Buckets The buckets store the hash-map, where key is remaining hash and bit indicating if there's single value for that key or not. The hash-map is stored as two components however. First is the value list p which is used when there are more than 1 value per

[1] Li, "Minimap2: pairwise alignment for nucleotide sequences"; Li, minimap2.

[2] minimap2 contributors, How to interpret the mmi index file? Issue 820; Lanius, parse_mmi.py: Python parser and lookup functions for minimap2 indexes.

key. Second is the hash array itself, which stores key/value pairs. If there are more than one value per key (last bit of key set to 1), then the value represents offset and count into the value list p The values, either as values in the key/value pairs or entries in p store sequence id, position and strand bit.

- Bit-acked bases (optional) If the `MM_I_NO_SEQ` flag is unset manually by user, bit-packed reference bases are saved at the end.

The depiction of the index file layout is presented in the Figure 10.1 below.

The idea of storing hash-map as two lists inside bucket is also presented in the Figure 10.2 below to allow easier understanding of the index concept.

10.2.2. Indexing: mmseqs2

There are alternative approaches in literature to building an index. MMseqs2 stores one index file that contains counts and offsets, and then writes larger index files that contain all occurence lists instead of bucketing them like minimap2 does. It's worth noting that 2017 MMseqs2[3] paper uses spaced k-mers which wasn't the focus of this thesis, so I didn't investigate more how the alignment is precisely done.

MMseqs2 offers great performance in clsutering, which is different problem to alignment, however connected as it involves estimation of edit distances for large datasets. The software also allows sets of taxonomy operations and alignment-based searches as well.[4][5][6][7]

Count and fill the MMseqs2 index
IndexBuilder::fillDatabase 56

Read a word's entry range
getDBSeqList 178

Keep distinct word codes within one target
addSequence 350

I illustrate the idea of MMseqs2 index on a small example in the Figure 10.3 below, where I just assume that we use 2-letter k-mers of natural language for simplicity.

10.3. BioseqDB indexing abstractions

Indexes are stored separately in BioseqDB 9.1 to all internal sequence fragmented tables 9.8 and they are separate entities to the usual indexes in Postgres which provide generic access paths to the tables.

There is thin layer of abstraction over the indexes, so in the future implementation of other types of indexes is easy. The storage abstraction requires implementer only to specify the type and depending on it we only need to provide logic to write the index chunks. One problem with current state of BioseqDB is that the index reading is not abstracted, so we encapsulate details of implementation, but they leak into pipeline (as the pipeline currently needs to understand how we store minimizers).

[3] Steinegger and Söding, "MMseqs2: sensitive protein sequence searching for the analysis of massive data sets"; Steinegger and Söding, "MMseqs2 enables sensitive protein sequence searching for the analysis of massive data sets."

[4] Steinegger and Söding, "Clustering huge protein sequence sets in linear time."

[5] M. Mirdita et al., "Fast and sensitive taxonomic assignment to metagenomic contigs."

[6] Milot Mirdita, Steinegger, and Söding, "MMseqs2 desktop and local web server app for fast, interactive sequence searches."

[7] Kallenborn et al., "GPU-accelerated homology search with MMseqs2."

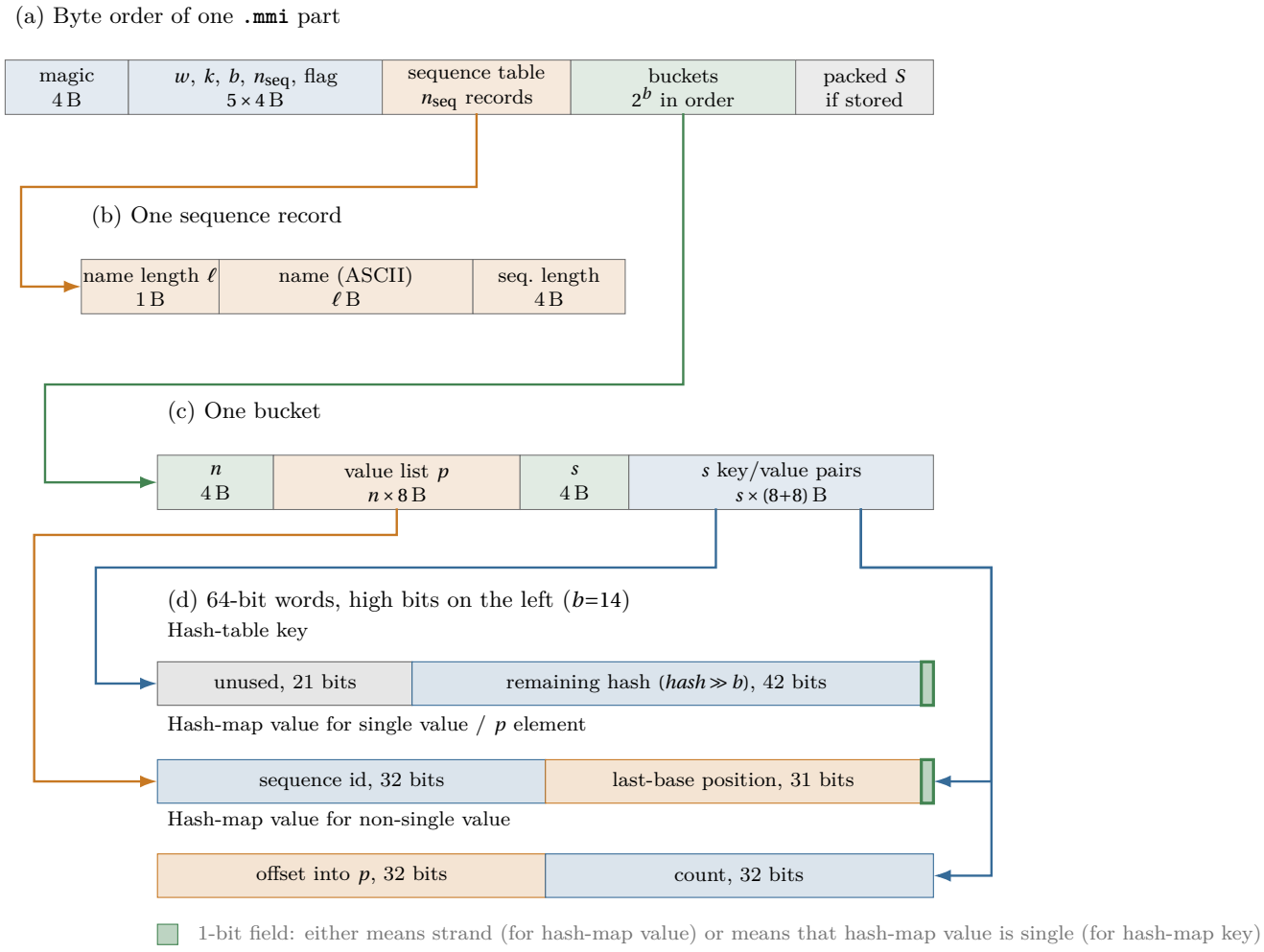


Figure 10.1: Interpretation of one `.mmi` index. For very large datasets index files are split into multiple parts where each of them follows the same format. Minimap2 needs to store sequence ID (RID 4.1), because the index parts are generated for a set of sequences, not just one, so each part needs to identify which reference sequence the minimizer comes from. Arrows represent how specific elements of arrays look like in close inspection. Panel (a) depicts high-level structure of the index 10.2.1. Panel (b) presents records storing sequence information. Panel (c) shows each bucket of minimizers, which stores its own hash-map via two lists. Panel (d) shows how elements of p looks like and key/values in the hash-map list. Hash map key is always remaining 42 bits with bit indicating that there's single value for that key. If the bit is set, then value in the key/value pair is (sequence id, position, strand bit), if not, then value points via offset/count pair into the list p. The list p stores the same (sequence id, position, strand bit) values.

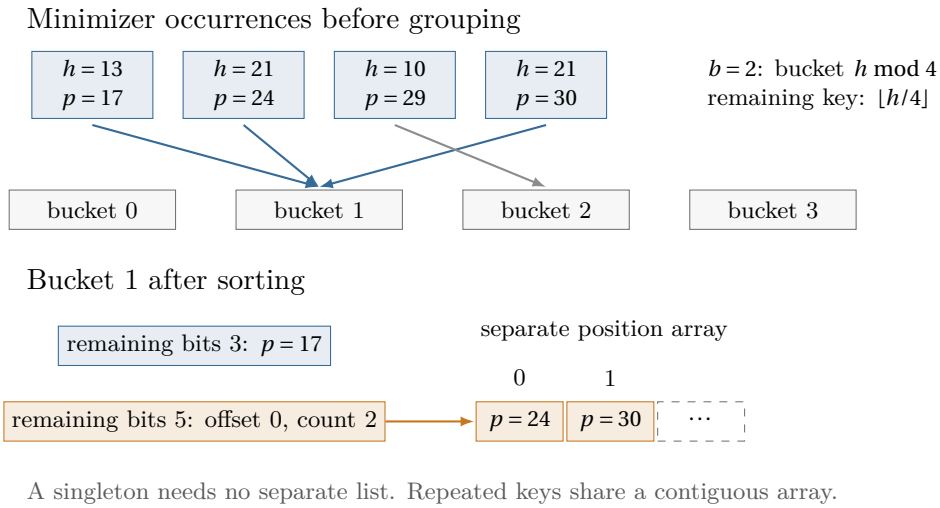


Figure 10.2: A simplified minimap2 bucket: hashes 13 and 21 have the same lowest two bits, but different remaining keys, so they land in bucket 1 together. The two occurrences of hash 21 become adjacent in the bucket's position array.

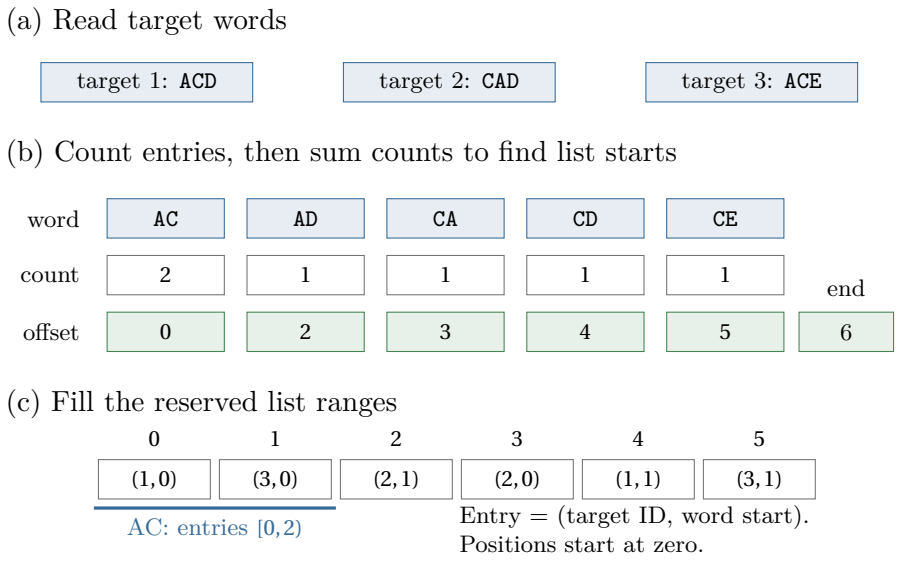


Figure 10.3: A small example of MMseqs2's lists construction, where word/count/offsets triplets are stored in single file, file the actual (very large) lists of entries are stored separately

Seed abstraction

In the future I want to improve curent implementation to use abstract notion of ”seed”. I already extensively use templates, so introducing new concept to represents types of seeds isn't a problem. This abstraction will allow to fully decouple indexing from the pipeline. Pipeline stages then could automatically detect how to compute certain seeds (like minimizers) and how to read and store them. Current BioseqDB implementation only uses minimizers, so I decided not to decicate my time to this idea.

I assume here that index store chunks which are separate abstraction to the chunks in columnar tables. One chunk consists of bytes the index implementation needs to write or store and I assume the chunk cannot be larger than $960MB$ (due to Postgre's TOASTing limits). The chunks are stored in internal tables that does not use columnar storage, they're ordinary heap tables - I decided that columnar storage in this case won't give any advantage (although I didn't A/B test this approach). Additionally, index tables are create `UNLOGGED` to avoid unnecessary logging and to reduce the amount of data written to the disk.

There are several supported methods of storing index:

- **`CHUNKED_PER_FRAGMENT`**. One fragment 9.4 of a sequence can have multiple chunks in the index.
- **`INLINE_PER_SEQUENCE`**. One sequence has a single chunk that is stored as Postgres varlena. This means that entire data stored for one sequence cannot exceed $960MB$ if that happens BioseqDB will error. `symbol_counts` is BiosqeDB index that uses this layout. It stores how many times each symbol occurs in that sequence and the payload is so small that it nicely fits into one small row.
- **`CHUNKED_PER_TABLE`**. This is special layout that in its implementation exposes only simple methods to access random chunks of the table. The idea is that some indexing structures may want to implement custom strategies, for example trees of sequences or graphs of fragments that do not fit into other representations. I don't use this layout at the moment, but it's supported to provide extension point in the future.

Figure 10.4 shows what three of these layouts are representing.

In reality the fragment's number can be viewed as a generic sub-key. This distorts the original meaning of **`CHUNKED_PER_FRAGMENT`**, because in relity it's just ”per-sequence” but with extra key: ⟨query_key,
sub_key,
chunk_no⟩, while **`CHUNKED_PER_TABLE`** has ⟨chunk_no⟩. This idea is depicted below by real SQL schema of internal storage:

Listing 10.1: One heap row of a per-fragment index.

```
CREATE TABLE bioseq_internal.seqidx_mpf_... (
  query_key   BIGINT NOT NULL, -- which sequence in this shard
  sub_key     BIGINT NOT NULL, -- which item of that sequence (can be fragment
    number but doesn't practically have to)
  -- Then we store byte array defined by index implementation but split across
    chunks:
  chunk_no    INT    NOT NULL, -- 0, 1, 2, ... of that item's byte string
  chunk_bytes INT    NOT NULL, -- how many bytes this chunk holds
  blob        BYTEA  NOT NULL, -- the bytes of this chunk
```

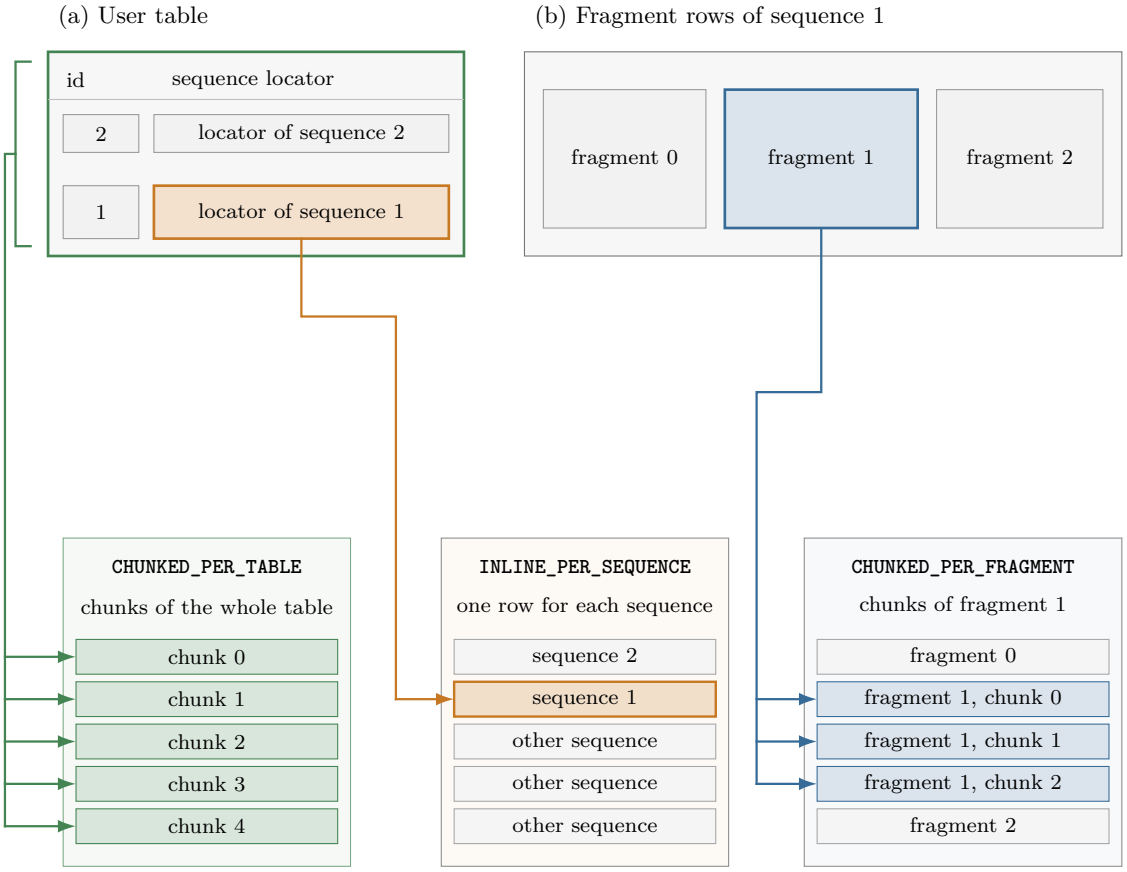


Figure 10.4: Each index is drawn as five rows. The green brace represents the fact that the index structure represents all sequences stored in the table. The orange arrow shows that one locator corresponds to exactly one chunk (one row in the index table). The blue arrow shows that one fragment can have multiple chunks. In case of `CHUNKED_PER_TABLE` we only store chunk number and for `CHUNKED_PER_FRAGMENT` I store fragment's number and chunk number.

```
  PRIMARY KEY (query_key, sub_key, chunk_no)
);
```

The payload of an index is one byte array. `CHUNKED_PER_FRAGMENT` stores that array as rows of Listing 10.1. When the array is longer than one chunk, it is divided into chunk 0, chunk 1, and the following chunks. Those rows keep the same `query_key` and the same `sub_key`. A reader orders them by `chunk_no` and concatenates `blob`.

Storage details

It's important to note that for minimizers per fragment MPF the sub-key number is the fragment number. I use the same column to store another number when the index is not split by fragment. Technically this means that **`CHUNKED_PER_FRAGMENT`** should have its name changed to something more appropriate.

In code `pg_supported_indexes` is a special templated that connects those implementation classes to the registered indexes. This is frequent practice used by me - encoding implementation vs. abstraction pairs as tuples in type lists created using template metaprogramming, so if extension is required we only need to write new type and add it into the `pg_supported_indexes` template list. Checks if implementation is valid are done via C++26 concepts 10.2:

Listing 10.2: Minimizers per fragment require fragment-chunk storage.

```
template <class p_storage>
struct minimizers_per_fragment_index {
  static_assert(is_index_chunked_storage_for<
      p_storage, sequence_index_layout::fragment_chunks>);
  static constexpr sequence_index_layout index_type =
      sequence_index_layout::fragment_chunks;
  using storage_type = p_storage;
};
```

10.4. BioseqDB supported indexing

There are 3 implementations of indexes inside BioseqDB:

- **`minimizers_per_fragment`** (shortened to `mpf`) stores minimizers calculated per one fragment and uses **`CHUNKED_PER_FRAGMENT`** layout.
- **`minimizers_per_sequence`** (shortened to `mps`) stores minimizers calculated per sequence, but as the number of minimizers grows they would not fit in a single chunk, thus one may consider using **`CHUNKED_PER_TABLE`**. However, as mentioned, the sub-key 10.3 is a number, so in MPS I use this number as a hash-group id, not as a hash. The rows stay in the **`CHUNKED_PER_FRAGMENT`** layout even though these numbers are not fragment numbers, as seen on the figure below 10.5.
- **`symbol_counts`**: stores counter of occurences of each symbol in a sequence, uses **`INLINE_PER_SEQUENCE`** layout, because the counter struct is very small, so Postgres inlines the varlena into the row.

The main idea behind MPF and MPS was:

- MPF: I can keep small hashmaps separated so random queries during seeding stage, where we need to find common minimizers between query and target, can hit CPU caches more often as the accesses are localized.
- MPS: The most frequent minimizers are trimmed by minimap2 and MPF trimming is done per fragment, so it's less reliable and can remove minimizers that are locally frequent, but globally rare reducing total number of minimizers and speeding up processing at the cost of potentially missing some matches. Additionally, MPF needs to perform multiple hash lookups if we want to determine all locations of minimizer's occurences so there's also performance penalty. This penalty can be also mitigated by the fact that MPF is easier to parallelize. Thus we can see there are several important trade-offs and it's not clear if the implementation of MPS should be more or less performant in theory. In case of BioseqDB as it would be very inneficient to store all minimizers individually, I decided to still generate minimizers per fragment, but store them and merge later during process that can potentially spill to disk (if the entire index fits into RAM then no spillage occurs, but if it doesn't there are heavy performance penalties). Additionally I must note that MPS uses the trick described here 10.3, so the sub-key is a hash-group number, not fragment number. This implies that if the query is short there will be not many groups to check, and as each group is held as heap rows, then MPS can be very fast, because it reads only the list of one group, not the entire index.

The figure 10.5 below shows 6 minimizer occurrences in a sequence, and panel (c) shows the byte strings that store them. Panels (a) and (b) show the same six minimizers in two groupings. Panel (c)and later illustrate the encoding of the strctures in binary strings (every rectangle onward is the binary string `blob` of one heap row from Listing 10.1), where the label on the left is that row's `query_key` and `sub_key`. If a string is longer than one chunk, later chunks continue it under the same two numbers.

Binary coding of non-grouped MPF

There are two types of MPF index depedning on BioseqDB setting flag: `bioseq.aligner.grouped_soa_generation.enabled`. The non-grouped version is depicted in panel (c). The sub-key is just the fragment number. The entry intself contains two arrays similar to what minimap2 10.1 does. The first array with `controls` contains hash value, start position in `positions` array and count of how many positiosn have that same hash. `positions` just represent minimizer positions. Usually in headers there's additional metadata and some of it is redundant as the exact shapes evolved during my implementation. There are versions and special magic values to potentially error if index was corrupted.

Binary coding of grouped MPF

I tried to improve MPF by doing something more sophisticated. The MPF can changed format based on grouped generation setting flag. Panel (d) shows how grouped MPF looks like:

- This is similar to MPF buit instead of storing array of structures (AoS) we store structure of arrays (SoA) so hash, start, and count are now three arrays in a row.
- I use special bit i.e 2^{61} to store positions array separately in separate row.

- There also is one special directory row using unique `sub\_key` value. The directory is list of $128B$ entries mapping fragment number to specific `sub\_key` value.
- Each directory row $128B$ entry constains useful metadata like min(hash), max(hash) so some rows doesn't have to be read at all if they don't contain hash values we are interested in.
- Using this `sub\_key↔fragment` mapping allows to potentially split fragment hash/start/count entry into multiple ones. However this is not used in the code currently and effectively `sub_key` = `fragment` for all entries.
- The directory item also contains information that is redundant for example `controls\_first` is number of first chunk storing hash/start/count binary string. This is always `0` as the binary strings are split into chunks, which are zero-indexed. The same logic applies to other fields and the entry size could be massively reduced.
- Grouped-MPF is very generic format and it also supports variable-width entries. The width of hash/start/count/positions can be adjusted independently for example for very short sequences we could use $2B$ `position` array and for very low minimizer density we could use $2B$ for `start\_array` or for very unrepetitive sequences where unique minimizers are very common, we could use $2B$ for `count\_array`. Practically, current code uses currently same widths for most of the fields and hash is adjusted depending on k and sequence length from minimizer type 4.17.

On the figure Panel (d) I didn't depict all fields in the MPF grouped directory as not all fields are relevant. Table 10.1 below lists all fields that are stored:

Binary coding for MPS

The MPS index stores the same rows as non-grouped MPF, but this time we also have additional one directory row. Also the fragment keys are just not reffering to fragments anymore, but they represent groups of hashes similarily to minimap2 10.1.

The directory contains metadata information like how many distinct hashes are there in total and for every group it stores min(hash), max(hash) to allow us to quickly skip groups if certain hash values do not exist there.

MPS index building needs to spend additional work on sorting and merging so later queries can skip unrelated hash ranges, while MPF just keeps smaller hashmaps based on fragments. The current default BioseqDB active minimizer index is MPF.

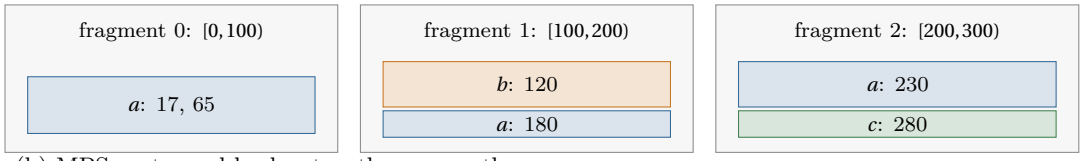
10.4.1. Index versioning

Internal sequence storage tables use `epoch` to provide sequence versioning 9.2, indexes don't have to use separate versioning field. I decided not to store epoch in the index table 10.1, but instead `query_key` is number of the row inside the table shard. They are stable with respect to `INSERT`s, `DELETE`s and `UPDATE`s as update will create new sequence version. Special handling is only required when tables are rewritten using `VACCUMFULL`: vacuum full would cause the rows to change numbering, however the indexes tables contain a hash of database, table space and Postgres `relfilenode`, so if the table is rewritten (`relfilenode` will change) BisoeqDB won't detect the old index. This is the most naive approach, but provides basic protection against table rewrites.

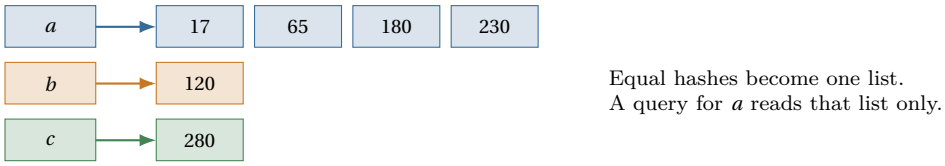
Table 10.1: Fields of one 128-byte grouped MPF directory entry.

Bytes	Size	Field
0–3	4	partition number (this is an index of entry in the directory row's array)
4–7	4	fragment
8–15	8	min(hash)
16–23	8	max(hash)
24–31	8	sub-key (in theory was meant to use to allow dividing entry for fragment into multiple rows, but currently it's always equal to frament number)
32–35	4	first chunk of the control byte string (chunks are zero-indexed so it's always 0 and should be removed in the future)
36–39	4	how many control chunks (counts in how many chunks hash/start/count binary array will use)
40–43	4	first chunk of the position array (chunks are zero-indexed so it's always 0 and should be removed in the future)
44–47	4	how many position chunks (counts in how many chunks position binary array will use)
48–55	8	how many controls (number of items in each of hash/start/count arrays)
56–63	8	how many positions (number of items in the position array)
64–71	8	byte size of the control byte string
72–79	8	byte size of the position array
80–87	8	checksum of the control byte string
88–95	8	checksum of the position array
96	1	hash width: 2, 4, or 8
97	1	start-array width: 2, 4, or 8
98	1	count width: 2, 4, or 8
99	1	position width: 2, 4, or 8
100	1	Used to represent differnt orders of hash/start/count arrays. Now meaningless and fixed to 1
101	1	Used to reresent how many positions there are per one hash entry. It's always 1, so this is meaningless.
102	1	Meaningless now, fixed to 2
103	1	Number of extra entries in start aray at the end. Now it's meaningless and fixed to 1 (and we don't need to store this one extra item at all)
104–127	24	zeros

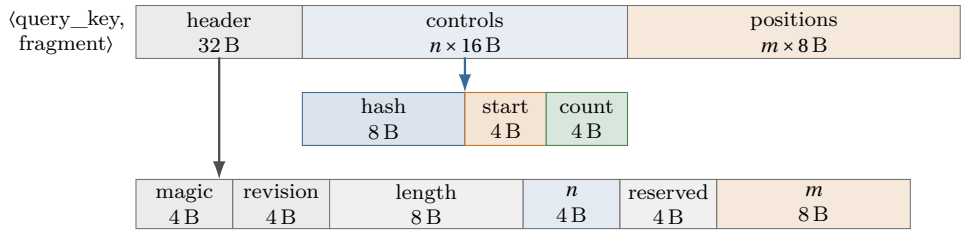
(a) MPF: keep each source interval together



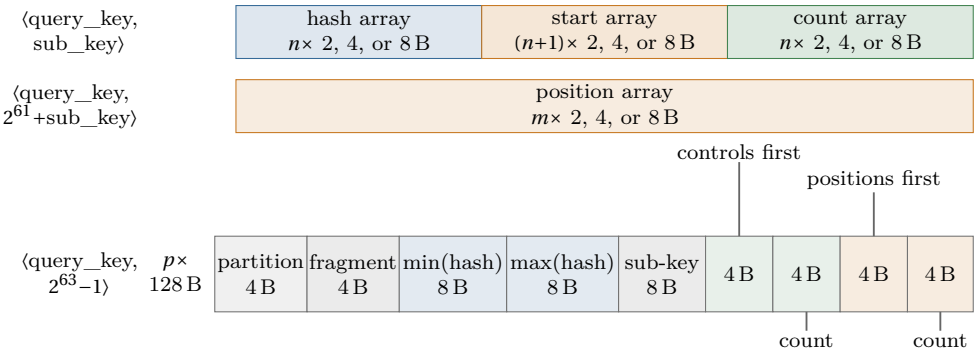
(b) MPS: put equal hashes together across the sequence



(c) Ordinary MPF: one fragment is one byte string



(d) Grouped MPF



(e) MPS group: the same byte string as (c)

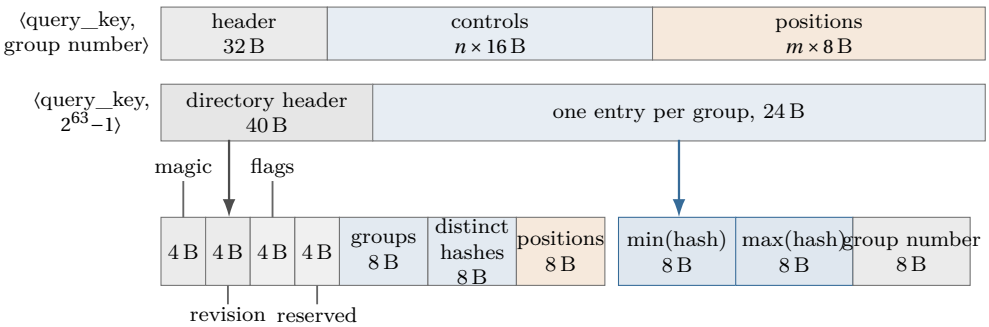


Figure 10.5: Each row with two key in brackets ⟨query_key, sub_key⟩ represent binary string that will be split into multiple chunks that could be find by complex key ⟨query_key, sub_key, chunk_number⟩. There's always one special sub-key for directory for grouped MPF and MPS, these formats also store two rows per one sub-key.

10.4.2. Symbol count index

I implemented one index type that does not store minimizers at all: `symbol_counts` index keeps a very small symbol histogram for one sequence. Each symbol has 64-bit counter (for DNA4, the counters are A, C, G, T so sequence `AACGTA` gives $(3, 1, 1, 1)$). The counts are stored separately because as we use heap rows the value will be stored inline in Postgres table. This allows very fast access and usually users don't need to count symbols.

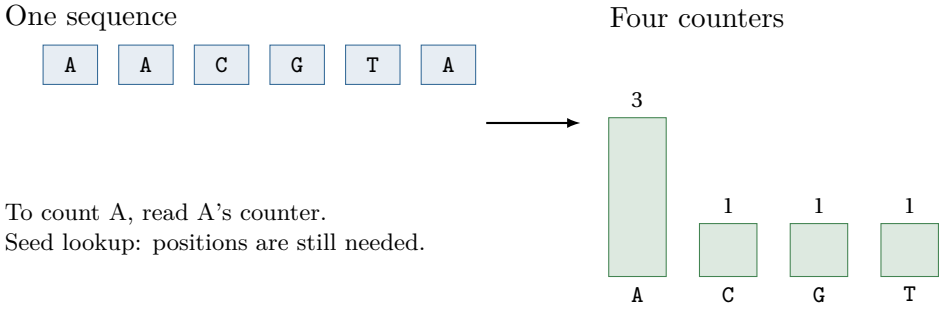


Figure 10.6: Hitsogram counter for symbols is stored separately so BioseqDB access it only when specified in the query

Listing 10.3: Request a symbol count for each stored sequence.

```
SELECT id, bioseq.symbol_count(seq, 'A') AS a_count
FROM reads;
```

Using this small metadata index allows me to never access the sequence content at all, meaning the read is very fast. If the index entry is not found, then BioseqDB will traverse entire sequence. The primary benefit is that this can be computed when loading FASTA or inserting rows autmatically so the user pays the cost only once (or later when infrequent `INSERT` or `UPDATE` happen)

10.5. Building MPS

Building MPF index is rather simple as we traverse each fragment and can separately construct hashmaps with minimizers (and hashmaps for fragmenst are relatively small compared to all RAM capacity). Constructing MPS is more complex as I need to somehow store all minimizer globally and this can potentially be bigger than entire RAM capacity. Index builder could collect every $(hash, position, strand)$ record and sort the complete vector, but for maps bigger than RAM this couldn't be done efficiently, instead current code does similar thing to MPF with a slight differences:

- To reduce memory access costs as much as possible MPF treats each fragment as entirely separate sequence, thus minimizers on edges won't be generated (the first and last $k + w - 2$ symbols). This reduces minimizer density at edges but means that we access only one fragment to generate index entry.
- For MPS I use subtly different approach and I collect all minimizers. This means that during parallelized fragment read, the reader need to read not only own fragment, but also parts of fragment to the right and left. This is depicted as "context" on the figure below 10.7.

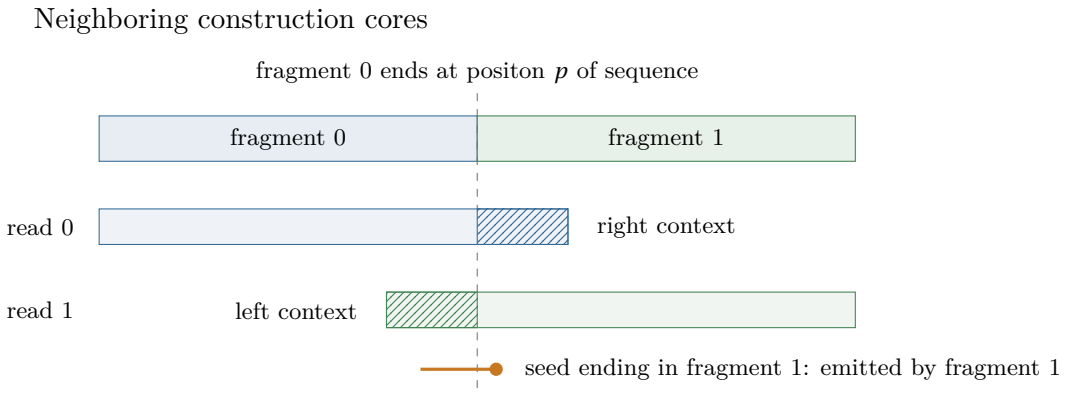


Figure 10.7: Each reader to generate minimziers for one fragment in MPS read also neighbouring fragments so first and last minimizers in fragment can be generated. Size of "context" is $w+k-2$ symbols.

MPS does the following steps to build entire index:

1. Read fragments and for one fragment compute minimizers, sort them by hash and remove duplicates similarily to MPF.
2. Write this computed MPF-like part into temporary file storage.
3. After all fragments have temporary files, then select fixed number of them and merge them into one sorted temporary file.
4. Continue this merging procedure until one temporary file remains.
5. Read the final temporary file and build directory row inside storage.

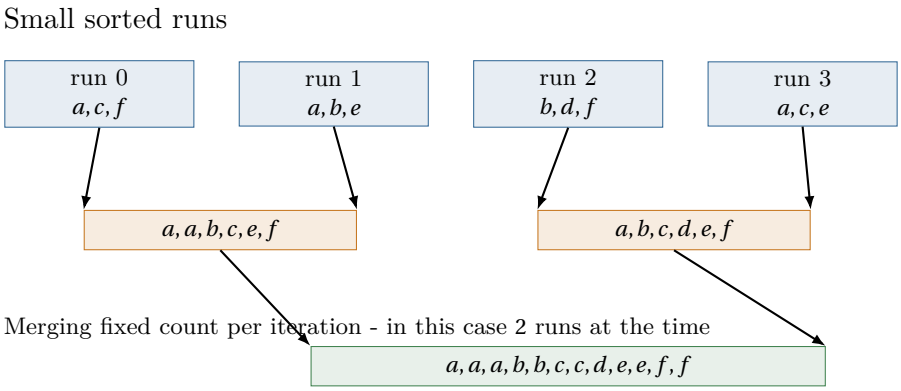


Figure 10.8: Merging of hash entries (hash value and minimizer position)until one final entry is left and can be stored permamently in index storage. Letters represent different hash values.

The spill path uses PostgreSQL's `BufFile` temporary-file interface. The sorter retains sorted runs in RAM when memory admission succeeds and spills them when it cannot reserve that memory. It can also move a retained run to a temporary file to make room for later

work. Thus the merge structure in Figure 10.8 does not require every run to be written to disk.

10.6. Selective reads

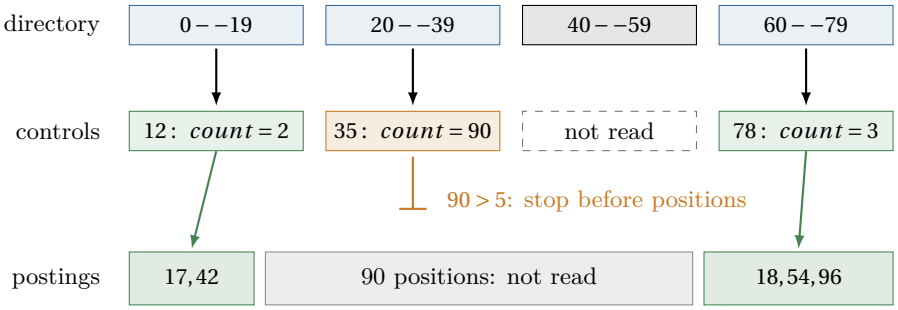
Main benefit of MPS is that each group can be a single row within Postgres (if n, m are not causing the array to spill over multiple chunks 10.4). This means that minimap2 has to read and check every entry in index, while BioseqDB can only read entiore directory row and using $min(hash)$ and $max(hash)$ values to decide if we should read certain groups of minimizers.

The similar logic applies to grouped MPF index as well, but in case of grouped MPF we can also check count array and if the count is very high, then we can abandon readind reparate position array at all (the most frequent minimizers should be filtered out as they are not good candiates for anchors).

The hash-major MPS reader performs the following steps:

1. Read and check the complete directory, then find hash ranges that intersect with query hashes.
2. Read the selected controls to find each key's exact global occurrence count
3. Apply the occurence count threshold: if the count is high, we don't process this hash
4. Read the chunks selected by this filtering and read positions form them

Query hashes {12,35,78}; occurrence limit 5



Query filters out the most frequent minimizers according to configured threshold and skips min-max ranges of hashe

Figure 10.9: Two decisions save different reads. All directory entries are read; blue ranges select the controls. Counts reject a frequent hash before its long list is loaded. Positions and thresholds are illustrative.

10.6.1. The reader also determines query execution

The current CPU MPS builder uses the hash-major format for supported DNA4 profiles. Other alphabets retain the grouped-SoA format. Its older reader reads and authenticates the complete stored payload, while retaining only the query's matching positions. The selective-read steps above therefore do not describe every MPS route. The native batch path can materialize a shared target for MPF and hash-major MPS, then run independent queries

in parallel. Grouped MPS instead uses the PostgreSQL scanner path, whose query loop is serial. This follows the worker boundary in Section 11.5.1: workers cannot fetch PostgreSQL tuples themselves. The choice therefore affects later sequence access and alignment, as well as index loading.

10.6.2. Repeated control scans in MPF

The mapped MPF reader has two lookup routes. Hashes in its supported compact domain can use direct lookup, described in Section 10.6.3. Wider hashes use the stored control arrays instead. This route counts matching occurrences, prepares anchor ranges, and then emits anchors through separate passes over the controls. The current scheduler groups these scans into query waves limited by worker concurrency, so the controls are visited again for each wave. The direct route instead attempts to prepare the whole batch at once, reducing its wave size if memory admission fails (Section 11.5.3). Keeping the file mapped avoids rebuilding the index, but does not remove repeated traversal or guarantee that its pages remain in physical memory (Section 10.7.4).

10.6.3. Why grouped MPF has a reader file

The reader file lets BioseqDB's CPU workers read a stored minimizer index without asking the PostgreSQL backend thread to fetch each index chunk. It is an experimental part of the batch-join implementation. There is a public SQL path to it, but an ordinary single-table `mmsearch` query does not use it. To explain the problem it addresses, consider this join:

Listing 10.4: Search every query sequence against stored reference sequences.

```
SELECT r.id AS reference_id, q.id AS query_id
FROM refs AS r JOIN queries AS q
  ON mmsearch(r.seq, q.seq);
```

Here `r.seq` is the target and `q.seq` is the query sequence. An eligible `BioseqBatchJoin` retains the query rows, prepares one target, searches the queries against it, and then moves to the next target. A target may have many fragments, each with index chunks in heap rows. Saving those chunks avoids computing the target minimizers again. However, reading them through PostgreSQL still requires tuple access and extraction of the `bytea` values. BioseqDB's CPU worker threads cannot perform that PostgreSQL work themselves. The backend must supply the bytes before those workers can use them.

The reader file moves this work to index construction. The builder copies the index payloads from the heap rows into consecutive file ranges. A later query opens that file with `mmap`, which gives the process an address range backed by the file. Workers can then read the immutable arrays through ordinary memory accesses. They do not fetch heap tuples for every index chunk or reconstruct a complete private copy of the saved index. This concerns access to the minimizer index. The join still reads its input tables and fetches sequence symbols when alignment needs them.

Listing 10.4 explains the workload, but its default read policy does not select the mapped CPU reader. That branch currently requires an explicit request for a compatible stored index. Section 10.7.2 explains this policy, and Listing 10.5 adds the request to the same join. The target must also be a complete stored sequence with a compatible index generation. Sliced or transformed targets do not meet that requirement. Requiring an index on an unsupported query shape produces an error.

Figure 10.10 shows what changes in target preparation. Ordinary MPF readers load stored fragment indexes and assemble a lookup for the hashes needed by the queries. The grouped reader uses the arrays in its file. Both reuse the prepared target across query batches. If the executor does not use a stored index, it instead computes a temporary lookup from the target sequence.

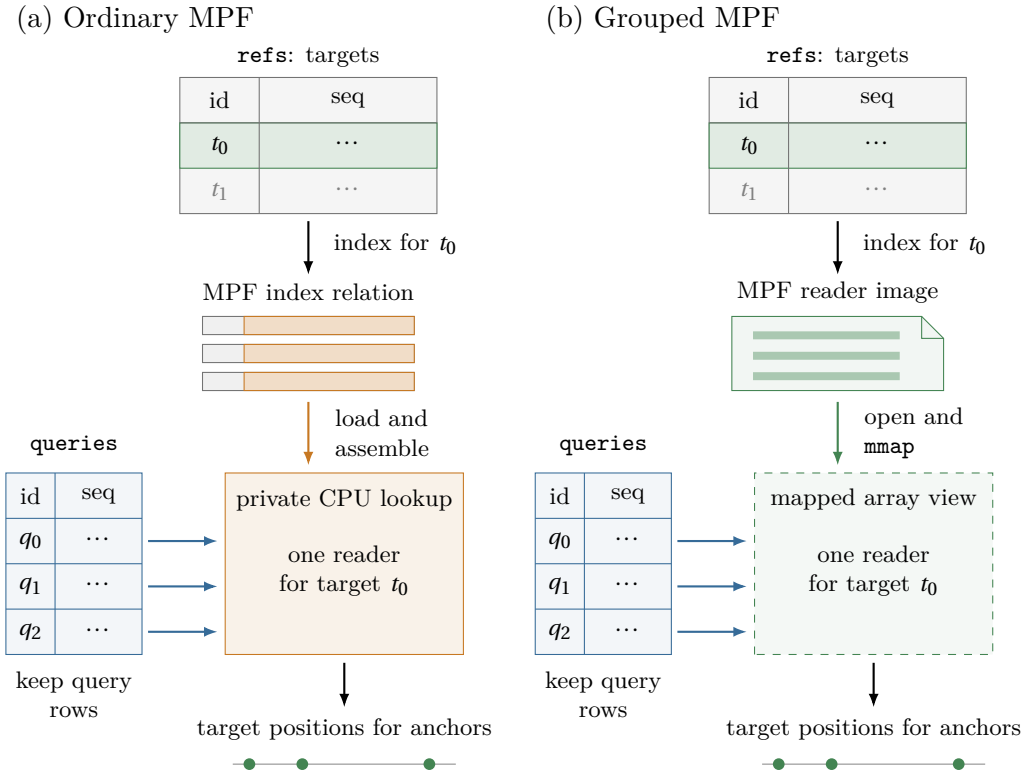


Figure 10.10: Two ways to prepare the target index for a batch join. Both read the query and target tables. For target t_0, ordinary MPF loads fragment indexes and assembles a private lookup. The grouped MPF reader maps an existing file and probes its arrays. The blue arrows represent searches for query hashes. Target t_1 is processed afterwards. The figure shows the reader alternatives, not a switch controlled by the construction setting alone. The mapped CPU route requires the read policy from Table 10.4.

Build the relation and file together

The extra file introduces a consistency problem. BioseqDB now stores index bytes in both a relation and a file. I handle this by publishing complete generations, rather than modifying a file while queries read it. One generation describes one stored sequence version and one minimizer profile. The profile identifies the algorithm, its parameters and the representation of the saved minimizers.

For grouped MPF, an explicit `bioseq.update_index` call performs these steps for each sequence:

1. Compute the minimizers and write the new generation's index rows.

2. Read those rows back in index order and validate their contents. Copy their payload bytes into a private construction file. Copying the payload does not copy heap-page headers or recompute the minimizers.

3. Add the directory and generation metadata. Where the hash domain supports it, also build the direct lookup described below.

4. Finish and validate the file. Make it read-only, synchronize it and install a unique final name without replacing another generation's file.

5. Record that file's identity and replace the active-generation record in the same database transaction. Other transactions can select the new generation after commit.

The relation remains in the database. Thus this approach pays for a second copy of the index bytes and a full reread during construction. In exchange, later readers can open the prepared file. A query does not repeat the export.

Algorithm 19 shows the construction and read operations separately. Here s is the complete stored target sequence, R_g is its index relation data for generation g, and σ is the query snapshot. The procedure names are pseudocode.

Algorithm 19 Publish a complete MPF generation, then open its reader file.

```
1:  procedure BuildMappedMPF(s)
2:      g ← a new generation number
3:      R_g ← BuildAndStoreMPF(s, g)
4:      F ← CreatePrivateFile
5:      for all payload chunks c of R_g, in stored index order do
6:          CheckChunk(c)
7:          AppendPayload(F, c)
8:      AddDirectoryAndMetadata(F, R_g)
9:      if the hash domain supports direct lookup then
10:         AddDirectLookup(F, R_g)
11:     F_g ← CheckAndPublishReadOnlyFile(F, g)
12:     UpdateDescriptorAndActiveGeneration(s, g, F_g)
13:     return g  the caller commits the transaction

14: procedure OpenMappedMPF(s, σ)
15:     D ← ReadAndCheckVisibleDescriptor(s, σ)
16:     F_g ← OpenAndCheckFileReadOnly(D)
17:     M ← MapFileReadOnly(F_g)
18:     CheckRequiredHeadersAndRanges(M, D)
19:     return a reader whose session keeps M and F_g open
```

What the reader finds in the file

The file contains the grouped fragment arrays already described in Figure 10.5, together with the metadata needed to locate them. A worker can search the fragment controls and then read the selected positions. This is the `control_block_scan` route.

For supported 32-bit hash domains, the writer also adds a sequence-wide hash lookup. It combines equal hashes from the target's fragments, so the reader does not have to search every fragment for the same hash. This is the `direct_compact_hash_lookup` route. It contains:

- Membership bits. Bit $B[h]$ is one when hash h has a saved position list. The hash itself is the bit address.

- List records. Records follow increasing hash order. The number of set bits before h, called its rank, gives the record number. A saved count at each 256-hash boundary

avoids counting from the start of the bitmap. Each record gives a list start, its total count and the number of strand-zero positions.

- Positions. The list start is an entry number in a shared position array. Strand-zero positions precede strand-one positions, and each encoded position includes its strand bit. A single 32-bit position can be stored directly in its list record.

In Figure 10.11, one set bit precedes hash 5. Its record number is therefore 1. Record 1 gives start 2 and count 2, which selects entries [2,4) containing positions 40 and 230.

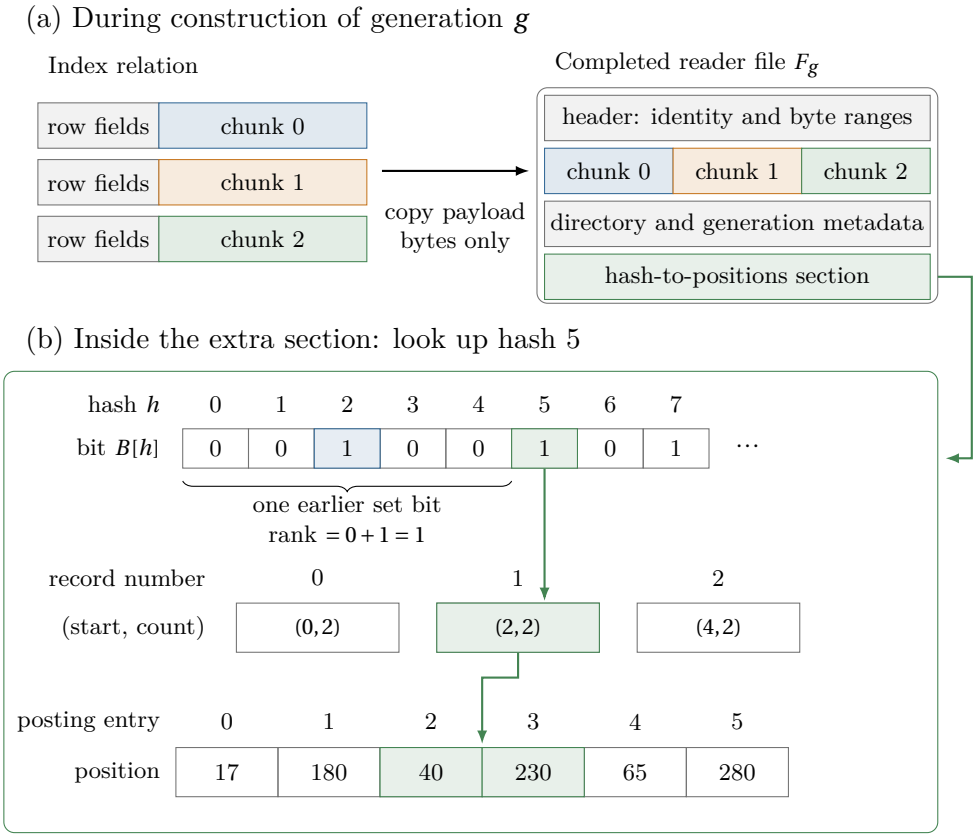


Figure 10.11: The reader file for an MPF generation with direct hash lookup. Panel (a) shows the index bytes copied from heap rows into consecutive file ranges. The direct-lookup section is added for supported 32-bit hash domains. Panel (b) looks up hash 5 in a small illustrative bitmap. One set bit precedes it, so its record number is 1. That record selects position entries 2 and 3. All illustrated positions have strand zero. The stored strand count is omitted.

An unset bit can mean that the hash is absent or that its sequence-wide occurrence count exceeds the saved limit, currently 65,534. The original fragment payload remains in the file. A direct-lookup query can use a lower positive occurrence limit, but cannot recover a list omitted at construction by asking for a higher limit.

Direct addressing has a substantial space cost. With the minimap2 integer hash and $k = 15$, the bitmap covers 2^{30} hashes and occupies 128 MiB. The saved rank counts occupy another 16 MiB. The current default bounded ntHash profile uses a 32-bit hash domain, giving 512 MiB and 64 MiB instead. Both figures exclude list records and positions. The writer

builds this section for supported domains even if a later query chooses fragment controls. Wider hashes use the fragment controls without this section.

Requesting the mapped reader from SQL

Listing 10.5 assumes committed `refs(id,seq)` and `queries(id,seq)` tables using the `bioseq` table access method. Their sequence columns have type `bioseq.seq('dna4',32)`. The target table starts without an older ordinary MPF relation. The example uses the current defaults listed in Section 10.7, including grouped construction and disabled insertion maintenance.

The default minimizer algorithm, $k = 15$ and $w = 10$ are sufficient and need no extra `SET` statements. Construction uses the configured profile when it is present in the compiled profile registry. A later query must match it.

Listing 10.5: Build grouped MPF and request its persisted CPU reader.

```
SELECT bioseq.update_index('refs'::regclass,
                           'minimizers_per_fragment');

SELECT r.id AS reference_id, q.id AS query_id
FROM refs AS r JOIN queries AS q
  ON mmsearch(r.seq, q.seq,
              aligner_options(index_read_requirement => 2));
```

MPF is the default active read index. If the table previously selected MPS, `bioseq.set_active_read_index` must select MPF again. This catalog choice is separate from building the index. Table 10.4 explains the value 2 in the listing.

The code path is concrete: the batch executor prepares a target through `prepare_cpu_persisted_native_target`, which calls `open_exact_minimizer_fragment_image_pg` for grouped MPF. The resulting reader remains available across query batches for that target. `EXPLAIN` can show an executable `require_compatible_persisted` plan. To establish that a particular query actually read the index, execution must also succeed and report persisted targets in `EXPLAIN ANALYZE`. The custom node's name alone is insufficient, since transformed-input variants share that name.

Grouped MPS uses its relation-backed streams. The grouped MPF GPU reader also opens the mapped file, then prepares device data from it. The CPU worker access explained here is therefore one use of the file.

What happens when the source changes?

There is no continuously updated file mirroring a mutable index relation. For grouped MPF, the relation rows and file are published as one immutable generation. The following cases explain what that means for SQL writes:

- Insert a new row with the defaults. The source row is saved, but insertion maintenance is disabled, so no minimizer generation or reader file is built for it. A later explicit `update_index` builds the missing index along with the table's other visible sequences. An eligible query requiring persistence fails if it reaches an unindexed target. A permissive query can compute its lookup during execution.
- Enable insertion maintenance. Ordinary SQL insertion rejects grouped minimizer maintenance when an enabled minimizer index has `UPDATE_ON_INSERT` policy. It does not update the relation and quietly leave the file behind. Section 10.7.1 explains ordinary maintenance and the separate batched `bioseq.load` path.

- Update a sequence. BioseqDB writes a replacement source row. Its index must describe that replacement. The old generation cannot serve it because readers validate the source identity and visibility before opening the file. With insertion maintenance disabled, rebuild after committing the replacement.
- Rebuild the index. A new generation and file replace the active descriptor transactionally. An existing file is not overwritten.
- Abort the build. The relation changes roll back. Registered abort actions attempt to remove the build's new file after checking its identity. A committed file remains available for later queries.

Costs and unfinished work

A mapping still uses RAM. The operating system loads file pages when they are touched and can share those pages between read-only mappings. This avoids retaining a complete private payload copy in every backend. It does not remove disk reads for a cold file, the temporary buffers used by the query, or the cost of writing the extra representation.

Closing a reader unmaps its address range and closes its file descriptor. It does not delete the file. Rebuilding retains retired generations, and deleting a source row does not remove its reader file. A required read of a missing or invalid file fails. It does not reconstruct that file during the search. An explicit rebuild is needed to publish another generation.

These limits are why I treat the path as experimental even though grouped construction is now enabled by default. Automatic reclamation of retired files is unfinished. The file is outside PostgreSQL relation WAL, and its index payload relations are unlogged. File synchronization alone does not provide crash recovery or a backup protocol that keeps descriptors and files together.

10.7. When indexes are built and when queries use them

Index maintenance and index reading are separate decisions. Saving an index does not force every query to use it, and selecting an index for reading does not build it. The policies below concern BioseqDB's sequence indexes, not PostgreSQL's native indexes on table columns.

10.7.1. Maintenance policies

Each registered index has a maintenance policy for each sequence column. `bioseq.set_index_policy` changes that policy for all current sequence columns in the named table. Table 10.2 describes the two choices, and Table 10.3 gives their defaults.

Table 10.2: Maintenance policies selected with `bioseq.set_index_policy`.

Policy	Meaning
`UPDATE_ON_INSERT`	Request automatic maintenance for newly stored sequences when the global insertion-maintenance setting permits it. Ordinary indexes are built for the new sequence, without rebuilding older rows.
`UPDATE_MANUALLY`	Skip automatic insertion maintenance for this index. Build or rebuild it with `bioseq.update_index`. Existing index contents remain available to eligible readers.

Table 10.3: Default policies before a table has explicit overrides.

Index	Maintenance policy	Default read role
`minimizers_per_fragment`	`UPDATE_ON_INSERT`	Active minimizer index
`minimizers_per_sequence`	`UPDATE_MANUALLY`	Not selected
`symbol_counts`	`UPDATE_ON_INSERT`	Used by supported symbol-count operations

There is also a global setting, `bioseq.index.insert_maintenance.enabled`, whose current default is `off`. Thus a default `INSERT` saves the sequence but builds none of these indexes. In particular, the MPF policy's name does not imply that it is being maintained while this setting is off.

The following example deliberately selects ordinary MPF maintenance. Disabling grouped construction is necessary because the ordinary insertion hook cannot publish a grouped minimizer generation. I also select the scalar algorithm because the default bounded ntHash requires the grouped format to record its hash identity. The table uses `bioseq.seq('dna4',32)` and starts without a grouped MPF index.

Listing 10.6: Enable ordinary MPF maintenance for an inserted sequence.

```
SET bioseq.aligner.grouped_soa_generation.enabled = off;
SET bioseq.aligner.minimizer.algorithm = 'scalar';
SET bioseq.index.insert_maintenance.enabled = on;
SELECT bioseq.set_index_policy(
  'reads'::regclass, 'minimizers_per_fragment', 'UPDATE_ON_INSERT');

INSERT INTO reads (id, sample_id, seq)
VALUES (101, 7, repeat('ACGT', 2000)::bioseq.seq('dna4', 32));
```

The switches in Listing 10.6 are alternatives to the grouped workflow in Section 10.6.3. They are not additional steps to run after that example. They also permit maintenance of other indexes with an on-insert policy, including symbol counts. An `UPDATE` writes a replacement sequence row and applies insertion maintenance to that replacement. Changing the policy does not backfill older rows, erase existing indexes, or choose the active read index.

When grouped construction and insertion maintenance are both enabled, ordinary `INSERT` rejects an enabled minimizer index with on-insert policy. For this workflow, the user must leave insertion maintenance off, commit the source rows and explicitly build their index. This prevents an ordinary relation-only write from leaving a grouped reader file inconsistent, as explained in Section 10.6.3.

`bioseq.load` has a separate batched path. When insertion maintenance is enabled, it first finishes the streamed source writes, then calls the explicit builder for indexes with `UPDATE_ON_INSERT`. This permits grouped construction after loading within that command. It is not the per-row `INSERT` hook, and the grouped builder's usual restrictions still apply.

For example, this policy keeps MPS outside automatic load maintenance:

Listing 10.7: Keep MPS construction separate from loading.

```
SELECT bioseq.set_index_policy('reference_sequences'::regclass,
  'minimizers_per_sequence', 'UPDATE_MANUALLY');
```

MPS already has this policy by default. The explicit call is useful when a table previously selected automatic MPS maintenance. It changes neither MPF nor symbol-count policy.

After loading and committing the source rows, the next listing builds MPS. It assumes the current defaults, with generation construction enabled and both asynchronous-persistence switches off. The target starts without an older ordinary MPS relation. Changing a construction setting does not convert an existing ordinary relation into a grouped one.

Listing 10.8: Build MPS explicitly after loading and committing the source.

```
BEGIN;
SELECT bioseq.update_index('reference_sequences'::regclass,
                           'minimizers_per_sequence');
COMMIT;
```

The builder uses the configured algorithm, k and w for a supported compiled profile. The explicit rebuild runs in the caller's transaction and does not depend on the insertion-maintenance gate. Supported CPU MPS profiles use generation storage grouped by hash across the sequence. Other supported MPS routes use the grouped arrays described earlier. Neither creates the MPF reader file.

10.7.2. Reading policies

A search has a per-call `index_read_requirement`, supplied through `aligner_options`. Its default is `allow`, encoded as 0. It is not a GUC. A GUC is a PostgreSQL configuration setting changed with `SET`, such as `bioseq.index.sequence_read.enabled`.

Table 10.4: The search's `index_read_requirement` option.

Value	What the search requires
0: `allow` (default)	Permit a supported compatible stored read when `bioseq.index.sequence_read.enabled` is on. The executor can instead compute a temporary lookup. This value does not select the grouped MPF CPU reader described in Section 10.6.3.
1: `require_absent`	Do not read a persisted index or reuse its assembled lookup. Compute the lookup for this execution. The name refers to execution without persisted reads, not to an assertion that no index exists on disk.
2: `require_compatible_persisted`	Require a compatible saved generation through a supported reader. A missing or incompatible generation is an error. Recomputing it during the query cannot satisfy this request. This requirement overrides the optional-read GUC.

The optional-read GUC defaults to `on`. Setting it to `off` prevents optional stored reads, but does not prevent index construction. Conversely, setting it to `on` does not create missing index data. Requiring a temporary lookup does not prohibit separately permitted background persistence after the query.

Compatibility means more than finding a file. The saved generation must belong to the visible source sequence and use the minimizer profile expected by the query. An index built with one hash algorithm cannot supply the positions for another algorithm's hash values. Changing k or w can also change which minimizers exist. The reader checks these identities before using the saved data.

The executor must support the requested read as well. A complete stored target in an eligible two-input `BioseqBatchJoin` can use the grouped MPF file with requirement 2. An

ordinary single-table `BioseqScan` currently rejects that requirement. It does not become a batch join because a saved index exists.

`bioseq.set_active_read_index(table, index_name)` chooses which logical minimizer index the table's sequence columns should use. It does not build that index. Existing maintenance policies are retained. If the call creates a new policy row, that row starts with manual maintenance, so an explicit selection can differ from the implicit defaults in Table 10.3.

⚠ Alpha software and restrictive memory checks

Please note that BioseqDB is still alpha software and has numerous issues. One problem is that its memory checks are too restrictive for small queries. The current policy keeps at least 2 GiB of system memory available, so even an allocation of a few hundred bytes can be refused when other applications use too much RAM. You should leave more than 2 GiB available, plus space for the query itself. On macOS, the estimate counts free and inactive pages. There is also a process-memory limit of 80% of physical RAM.
This can produce the misleading error `transient query execution stats disagree with its generated backend`. The query may simply have failed to allocate its minimizer buffer. It does not mean that the target index is missing or that query minimizers rejected that index. The saved index describes the target sequence. BioseqDB must still compute minimizers for the query sequence to find matching hashes.
If this happens under memory pressure, close other applications and retry. Rebuilding the target index does not fix the memory shortage. The error handling needs to report allocation failures separately from actual minimizer-statistics errors. If the message remains when enough memory is available, that requires further investigation.

10.7.3. Construction, deferred work and other settings

Table 10.5 collects the switches that decide whether index work is performed. They supplement the per-index and per-query policies. The defaults here are those of the current implementation.

Table 10.5: BioseqDB index construction and reading switches.

Setting	Default	Effect
`bioseq.index.insert_maintenance.enabled`	off	Global gate for automatic maintenance, subject to each index's policy and the grouped-insertion restriction described above.
`bioseq.index.sequence_read.enabled`	on	Allow optional stored reads. Per-call requirements 1 and 2 override this choice.
`bioseq.aligner.grouped_soa_generation.enabled`	on	Select generation-based minimizer builds. MPF publishes grouped arrays and a reader file. Supported CPU MPS profiles publish data grouped by hash across the sequence. Changing this setting alone builds or converts nothing.
`bioseq.index.async_persistence.enabled`	off	Permit eligible deferred persistence work. Explicit `update_index` remains synchronous.
`bioseq.index.recompute_async_persistence.enabled`	off	Additionally permit a recomputing query to request deferred persistence. It needs the preceding switch and does not guarantee a saved result.

The deferred implementation is limited. Its surviving query-side submission path handles ordinary MPS, with $k = 15, w = 10$ and an accepted backend. It excludes the default bounded ntHash algorithm. It also requires an eligible enabled index with on-insert policy and an available worker queue. Its policy selection prefers MPF before MPS, independently of the active read index, but the submission path rejects MPF. Thus enabling both switches alone is not enough to arrange an MPS rebuild.

Accepted requests retain source identifiers and parameters. They are submitted after transaction commit and discarded on abort. A worker opens its own transaction and checks the source and maintenance policy again. It does not inherit the query's snapshot or relation pointers. This machinery does not maintain grouped MPF reader files. The explicit grouped builder rejects either asynchronous-persistence switch being on.

There are additional controls for representation and resource use, listed in Table 10.6. None introduces another read-policy value or a separate switch for creating the MPF reader file.

Table 10.6: Other controls that affect BioseqDB sequence indexes.

Setting and default	Role
`bioseq.aligner.minimizer.algorithm` `bounded_nthash_simd`	Selects minimizer generation and its hash identity. Construction and reading must agree on a compatible profile.
`bioseq.aligner.minimizer.kmer_size`: 15 `bioseq.aligner.minimizer.window_size`: 10	Select k and w. The grouped builder validates them against compiled profiles.
`bioseq.aligner.fragment_minimizer.lookup_route` `direct_compact_hash_lookup`	Use the direct section for supported 32-bit MPF hashes. The alternative `control_block_scan` searches fragment controls. Wider hashes use controls even under the direct default.
`bioseq.index.memory_cap_mb`: 0	Additional resident-index memory cap. Zero uses the common runtime budget. It does not mean unlimited memory.
`bioseq.index_build.run_buffer_bytes`: 0	Internal superuser setting for external-sort run size. Zero chooses a size from the memory budget.
`bioseq.index.sequence_minimizer_sort.algorithm` `std_sort`	Select the ordinary MPS sorting implementation. The other choices are `simd_u128_vqsort` and `simd_u128_vec_stream_1`, `_2`, or `_4`.
`bioseq.aligner.vulkan.index_build.enabled`: off	Permit the GPU index-build route when Vulkan execution and the selected algorithm support it.
`bioseq.aligner.vulkan.index_cache.enabled`: on	Retain compatible GPU index data for reuse. A search may override it through `aligner_options(vulkan_index_cache_enabled => ...)`.

Algorithm and occurrence-filter settings still affect the search itself. For example, the query's occurrence limit must fit the direct file's saved limit from Section 10.6.3. CPU or GPU selection and scheduler resources also determine which implementation can run. Those are execution choices, not additional policies for maintaining stored indexes.

10.7.4. Reusing bytes already in memory

Persistent storage avoids rebuilding an index after every query. Memory caches avoid some of the reads and preparation within a backend's lifetime. They hold different objects:

- Serialized chunks. Ordinary relation readers have a bounded cache of index bytes. A reader may still copy a cached chunk into its own buffer.
- Prepared CPU lookups. Ordinary MPF and MPS readers can retain assembled objects identified by source, index, algorithm and interval.
- Mapped file pages. The operating system caches pages touched through the reader-file mapping. The grouped route does not fill the ordinary serialized-chunk cache.
- GPU arrays. Device data can remain available while a batch or a compatible cache entry owns it.

Evicting a cache entry removes the cache's reference. An active reader can still own the object, so a cache limit is not a bound on the backend's total memory. Query buffers and other readers can overlap with it. Similarly, a file's mapped address range is not the amount of RAM currently occupied by its pages.

Abort cleanup discards ordinary chunk-cache entries written by the failed transaction. Source and index invalidation discard affected cached objects. Reusing a prepared lookup still requires matching identities. Releasing these memory resources does not remove the stored index or reader file.

10.8. What the database integration adds

BioseqDB keeps minimizer indexes alongside versioned sequence storage. Ordinary maintenance can index newly inserted sequences, while grouped construction publishes complete generations for selective readers. The MPF reader file gives workers direct access to those saved arrays. These choices avoid repeated work, but require explicit rules for construction, compatible reads and source changes. Recovery of unlogged data and removal of retired files remain limitations of the current implementation.

Chapter 11

BioSeqDB architecture

11.1. Connecting sequence processing to the database

Consider a search between stored reference sequences and query sequences. Both tables below have an identifier `id` and a sequence column `seq`. One statement supplies comparisons to the alignment system:

Listing 11.1: A sequence join connects SQL input to the processing core.

```
SELECT r.id AS target_id, q.id AS query_id
FROM refs AS r JOIN queries AS q
  ON mmsearch(r.seq, q.seq);
```

PostgreSQL supplies the rows and owns their visibility. BioSeqDB selects compatible implementations, obtains sequence and index views, schedules the comparisons, and returns matches for SQL projection. This chapter explains how those responsibilities fit together.

A batch groups input comparisons for processing. A wave is a subset admitted together under the memory policy. A task is a unit of work submitted to the task scheduler and can process one or more comparisons. These terms describe how work is grouped, not different alignment algorithms.

The preceding chapters describe the work performed by a sequence search. Minimizers produce sparse seed keys (Section 4.1), chaining groups matching positions (Section 5.13), and final alignment compares the selected intervals at base level (Section 6.1). These stages must also fit into a database execution. The sequence store determines how symbols can be read, persistent indexes retain lookup work, and the custom scan turns matches into SQL rows.

BioSeqDB connects these components through a reusable C++ core and PostgreSQL-specific adapters. The core expresses biological operations over logical sequences and index views. The adapters connect those operations to database resources. This division gives three kinds of code different duties:

- Algorithms consume symbols, positions and scores. Their interfaces do not require a PostgreSQL tuple or relation.
- Adapters construct suitable views and keep their backing resources valid. They also convert returned results into database values.
- PostgreSQL execution controls planning, snapshots, locks and the lifetime of a statement. These mechanisms remain native to PostgreSQL.

The build reflects this division. A shared object-library target owns the PostgreSQL-free modules, and the extension consumes their compiled objects and module interfaces. PostgreSQL-dependent modules are compiled separately. The common code is therefore shared with non-server consumers without recompiling it under PostgreSQL-specific definitions. This is a source and compilation boundary. It does not by itself define a separately distributed library or remove PostgreSQL discovery from the project's build configuration.

11.1.1. Reuse at the alphabet boundary

SeqAn provides an established example of composing efficient sequence-analysis software from generic C++ components. Its library includes sequence representations, indexes and configurable alignment algorithms, with SIMD alignment already described in the 2017 overview.[1] BioSeqDB uses SeqAn3's alphabet layer within its own representation. A rank is a small integer code for a symbol, and the alphabet's cardinality gives the number of possible ranks. Rank and character conversions give those ranks their biological meaning, and complement operations apply where the alphabet supports them.[2] The implementation also uses related type traits and a few mathematical utilities.

BioSeqDB defines the packed storage, lazy views and search composition around these alphabet semantics. This lets the same logical symbol model participate in storage reads and in the local alignment kernels. It also lets index preparation produce the lookup form needed by a particular execution. Keeping this composition local gives BioSeqDB control over when storage is read, which stages run, and how long their data survives. SeqAn's own generic interfaces and optimized algorithms remain a precedent for that design. The choice of a local pipeline does not establish a performance advantage over a pipeline assembled from SeqAn components.

Two applications illustrate why composition matters. Masai describes seed search over conceptual trees but chooses concrete index structures for each role and combines the search with banded bit-vector verification.[3] DREAM-Stellar groups query segments by reference region and uses bounded batches with shared sequence ownership before local verification.[4] These are examples of separating algorithms, representations and scheduling. BioSeqDB makes those choices around its database inputs and result contracts.

11.2. Contracts describe the operations a stage needs

The implementation is built in C++26 mode. It uses templates, concepts and named modules to express these boundaries, although concepts and modules entered the language in C++20. A concept states which operations a type must provide. A template constrained by that concept can accept different representations without requiring inheritance from one common base class.

The following reduced API excerpt shows the contiguous minimizer-lookup contract. It keeps the current operation and return type, while shortening the template parameter names. A lookup receives a hash and returns a borrowed span of encoded positions.

[1] Reinert et al., "The SeqAn C++ template library for efficient sequence analysis: A resource for programmers"; SeqAn Project, SeqAn Manual.

[2] SeqAn Project, Alphabets in SeqAn.

[3] Siragusa, Weese, and Reinert, Fast and sensitive read mapping with approximate seeds and multiple backtracking.

[4] Aasna et al., "DREAM-Stellar: parallel and space efficient exact local alignment."

Listing 11.2: The contiguous lookup API constrains the operation visible to a consumer.

```
template<class I>
concept contiguous_minimizer_lookup_index =
    requires(const I& index, std::uint64_t hash) {
        { index.lookup(hash) }
            -> std::same_as<std::span<const std::uint64_t>>;
    };
```

This is one supported lookup shape. Other consumers can receive segmented position spans or canonical anchors prepared for a query. The consumer needs the meaning and lifetime of those positions, rather than the PostgreSQL relation from which they came. In particular, the total occurrence count remains authoritative when a selective reader omits the postings of a high-frequency seed. The storage choice must preserve the pruning decision.

Index construction uses a separate contract. It describes typed chunk reads and writes, enumeration and temporary sorted-run storage. Concrete PostgreSQL types satisfy the contracts for per-fragment chunks, a single per-sequence value, and table-wide chunks. Compile-time assertions check the required interfaces. Separating these storage operations from the query's lookup operation lets each side use the representation suited to its work. The persistence mechanisms are developed in Section 10.1.

Sequence access follows the same principle. A scanner supplies size and fragment-access operations. Its PostgreSQL implementation reads the fragments described in Section 9.1, while an in-memory implementation exposes caller-owned packed storage. Algorithms normally use the logical views introduced in Section 7.1, so they do not need to repeat either scanner's storage logic. A smaller example appears in column comparison: a result-sink concept asks for result allocation and writable value and null masks. The PostgreSQL sink supplies those buffers using database allocation rules. Character comparison that requires PostgreSQL collation remains in the PostgreSQL adapter.

These are selective abstractions. A minimizer lookup needs occurrence positions, but it does not need an abstract join operator. A sequence reader needs valid source access, but it does not implement an alternative MVCC system. Keeping the database mechanisms in PostgreSQL preserves their existing ownership and visibility rules. The common contracts instead expose the operations needed by the biological stages and their test callers.

11.3. Ownership completes the interface

A successful concept check proves that an expression has a usable type. It does not prove that a returned span remains valid or that its source may be read from another thread. Figure 11.1 separates the operation interface from the resources that make an execution valid.

The custom scan from Section 8.1 owns its locked relation and registered snapshot. Read adapters borrow them within the statement's lifetime. A lazy sequence view can therefore be PostgreSQL-free as a C++ interface while still requesting data through a PostgreSQL-backed source. The source rejects calls from the wrong thread before entering PostgreSQL error-handling machinery. The sharing rules in Section 7.5 apply here: worker eligibility and source lifetime are separate requirements. Depending on the route, preparation may copy data into admitted resident storage or retain an immutable mapped index until its readers finish.

Memory also has several owners. The core context separates temporary alignment storage from longer-lived pipeline storage. Numerical worker tasks install a context for their execut-

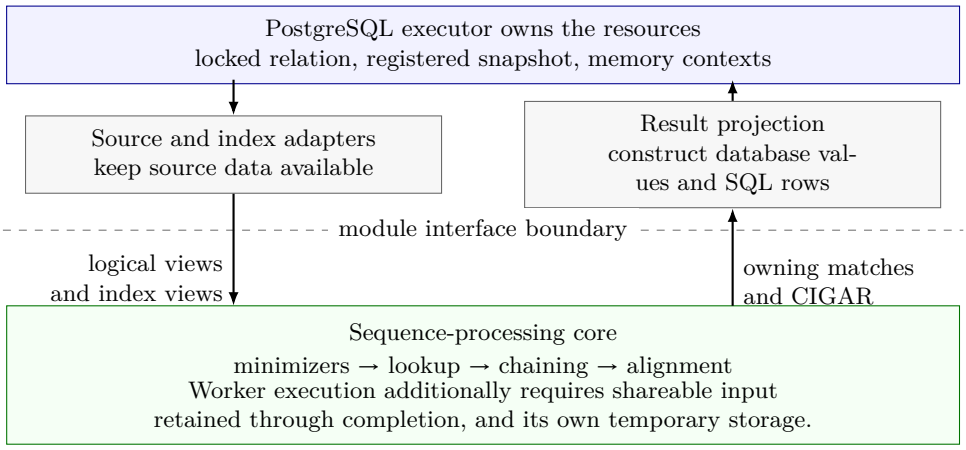


Figure 11.1: Data crosses a narrow interface while the PostgreSQL executor retains database ownership. Arrows show runtime data flow. Static module dependencies point from the adapters to the core contracts. A logical view may still request a PostgreSQL read on the owning backend thread. Worker execution requires an additional check of input sharing and lifetime.

ing thread, while database buffers remain under PostgreSQL memory contexts. A memory permit accounts for a reservation but does not own the allocation itself. The bytes must be released before that reservation becomes available to another task. Returned matches and their CIGAR operations use owning C++ vectors, which the PostgreSQL adapter can consume without retaining a temporary alignment arena.

Error handling follows the same division. PostgreSQL failures are translated at checked calls so that surrounding C++ owners can unwind normally. Database resource cleanup also has to remain valid on PostgreSQL's own error path. PostgreSQL interrupt checks occur at supported adapter boundaries, including between rows, rather than inside every native numerical loop. Thus a generic function signature alone makes no promise about cancellation latency.

The remaining shared assumptions are concrete. Both sides must agree on alphabet ranks, coordinate conventions and index encodings. A prepared index must describe the source version and minimizer rules expected by the query. The build must contain the selected implementation. These agreements make the stages composable, while the adapters keep each execution within the database's resource and visibility rules.

11.4. Runtime choices and compiled implementations

An alignment request names biological work, but it does not identify one machine-code function. BioSeqDB must also account for the sequence alphabet, packed representation, available indexes, and execution hardware. These decisions select an already compiled implementation and connect the algorithms described earlier to the storage and execution layers.

The logical flow remains sketching, seed lookup, chaining, region selection, and optional base alignment. Sections 4.1, 5.13, and 6.1 explain those algorithms. Here the concern is how the system selects their implementations and supplies compatible inputs.

11.4.1. Turn SQL settings into a request

PostgreSQL configuration parameters, usually called GUCs, provide session defaults. The three settings in Listing 11.3 select minimizer extraction, chaining and base alignment, respectively. Their defaults are `scalar`, `scalar_minimap2` and `block_aligner`. The available CPU minimizer choices are described in Section 4.13 and Table 4.4.

A constant `aligner_options` record overrides individual settings for one search. Unspecified or NULL fields inherit the session values. The following example instead selects all three algorithm families through named GUC values: scalar sketching, scalar chaining, and KSW2 base alignment. Both `refs` and `queries` are stored sequence tables with an `id` column and a `seq` column of type `bioseq.seq('dna4',32)`.

Listing 11.3: Select algorithm families before planning a sequence join.

```
BEGIN;
SET LOCAL bioseq.aligner.minimizer.algorithm = 'scalar';
SET LOCAL bioseq.aligner.chaining.algorithm = 'scalar_minimap2';
SET LOCAL bioseq.aligner.base_alignment.algorithm = 'ksw2';
EXPLAIN
SELECT r.id, q.id
FROM refs AS r JOIN queries AS q ON mmsearch(r.seq, q.seq);
COMMIT;
```

An ordinary `BioseqScan` stores the defaults and overrides in its plan. A `BioseqBatchJoin` reads the defaults again when execution starts and reapplies the constant overrides. Changes to the corresponding alignment GUCs invalidate cached plans. The request also fixes the selected chaining policy: PostgreSQL SIMD chaining uses `bounded_batch_approximate`, whereas the C++ interface additionally permits `exact_visited`. They are distinct policies, as described in Section 5.13. Likewise, the internal batched KSW2 implementation is not an additional PostgreSQL base-aligner setting.

Required results also determine which stages run. A coordinate-only request can avoid base alignment. A CIGAR or exact edit-distance request requires additional alignment work. Divergence and mapping quality have their own requirements. Thus selecting an aligner does not imply that every search executes it. A prebuilt query sketch also lets execution enter after sketching.

11.4.2. Separate algorithm selection from hardware selection

Figure 11.2 separates the decisions that constrain execution. A requested combination must have an implementation and satisfy its input contract. The system does not treat every combination of algorithm names as interchangeable or supported.

For CPU kernels, Highway selects an instruction-set target that is both compiled into the extension and supported by the running processor. BioSeqDB binds the resulting kernel table once at the alignment-batch entry and passes it to the selected implementation. It does not repeat CPU feature detection for each seed or choose a different biological algorithm because another instruction set is available. PostgreSQL vectorized tuple processing is a separate execution choice from the SIMD kernels inside an aligner.

Here a specialization is a compiled variant of a C++ template. Runtime selectors choose among these variants. The dispatch chain fixes a representation bucket and algorithm traits before calling a typed pipeline. Representation-width choices depend on the selected k-mer bucket, avoiding a full independent product of both axes. When an outer caller has already

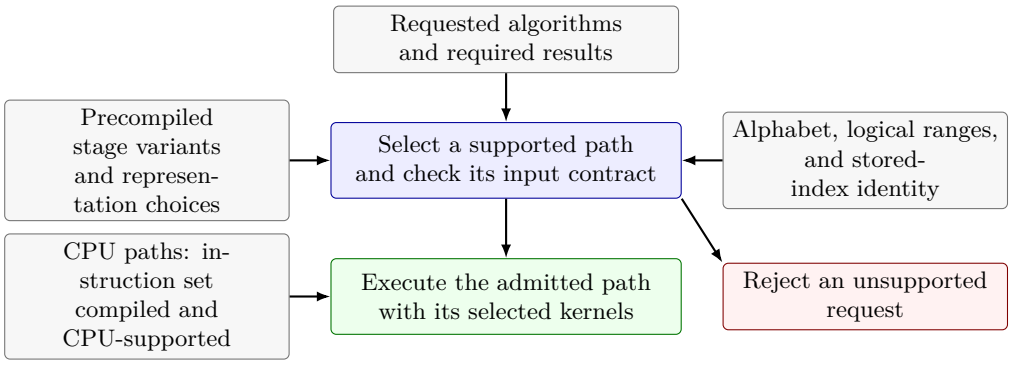


Figure 11.2: Algorithm selection, input compatibility, and CPU target selection have different responsibilities. The diagram shows their dependencies, not a universal order of planner callbacks. GPU execution has additional admission requirements. A runtime failure can still occur after admission.

selected the representation, the inner pipeline retains that choice. When sketches are supplied, it also removes the unnecessary sketch-algorithm dispatch dimension.

Not every numerical parameter needs its own specialization. Compile profiles control which specialized forms are instantiated. Other values use a compiled generic implementation that reads the requested k and w at runtime. This specialization fallback preserves the biological parameters. It is different from the explicit short-pair adjustment of untouched defaults, which can choose smaller k and w before execution. An indexed path prepares query sketches with the parameters required by the stored lookup.

11.4.3. Require compatible indexes and execution routes

An index is usable only when its identity matches the selected operation. The relevant information includes the source generation, alphabet, minimizer scheme, producer identity, and k, w values. Matching the latter two numbers alone is insufficient. In particular, the resident and indexed continuation routes within the two-stack family cannot be mixed merely because they share an algorithm name. The indexed query uses the selection route required by its index. Section 10.7 describes the separate policies for allowing or requiring persisted reuse.

Admission also limits acceleration:

- CPU SIMD. Selected kernels check their supported alphabet, representation, parameter range, and vector shape. Bounded ntHash has registered DNA4 and DNA5 kernels with both specialized and generic parameter handling, as summarized in Section 4.13. SIMD chaining requires a vector target and valid limits for its selected policy. A rejected request is not retried with another chaining policy or a scalar implementation.

- Biological Vulkan stages. PostgreSQL requires the complete supported resident combination, including Vulkan sketching, seed lookup, chaining, fused execution, and permission to build GPU indexes when needed. It requires usable hardware and a batch containing at least two jobs. Unsupported mixtures and device failures produce errors instead of entering the general CPU pipeline.

- GPU output. The persisted grouped-SoA route currently supports coordinates only. Transient resident execution also has a bounded CIGAR route, which retains host

packing and result construction. Admission to a GPU route therefore does not mean that every substage runs on the device.

These rules have specific exceptions and should not be reduced to a blanket ban on fallback. Historical reference-SIMD measurements can explicitly record scalar execution, whereas current registered DNA5 profiles have a SIMD route. A Vulkan rank prefilter can accompany CPU biological stages. A permissive index policy can rebuild an absent or inapplicable lookup. Finally, a rejected parallel work split can run serially with the same algorithm. Each alternative belongs to a particular layer and preserves that layer's stated contract.

11.5. Schedule independent work within a backend

The ownership boundary in Section 11.3 also determines where parallel execution is possible. Two queries can read the same immutable target, but they must not overwrite the same alignment scratch space. A database-backed reader may expose the right sequence interface while still requiring access from its creating PostgreSQL thread. The scheduler therefore needs more than a collection of independent mathematical problems. It needs inputs that remain readable, separate output locations, and enough memory for the work that overlaps.

BioSeqDB expresses this work through oneAPI Threading Building Blocks (oneTBB). A task is a unit of executable work, such as processing a range of independent queries. A worker thread is an operating-system thread that executes such tasks. An arena is oneTBB's scheduling domain with a limit on how many threads may participate simultaneously. Tasks describe available parallelism without assigning a permanent thread to each operation. This is the distinction emphasized by oneTBB's task-oriented and data-parallel interface.[5]

11.5.1. PostgreSQL processes and oneTBB threads

PostgreSQL executes a client session in a backend process. Its original server process, traditionally called the postmaster, starts backends and remains responsible for accepting connections. It does not execute each client's alignment tasks.[6] Inside an executing backend, BioSeqDB has one process-local shared scheduling interface. It owns a oneTBB arena used by alignment and other supported work. The name "global pool" refers to this process-local scope.

The extension also registers a separate PostgreSQL background supervisor when it is preloaded. The supervisor maintains shared scheduling configuration and service state. It can perform background work such as servicing index-persistence requests. It does not own one central CPU arena to which every client sends its sequence pairs. Figure 11.3 separates these roles.

Initialization follows the same boundary. During preload, BioSeqDB registers PostgreSQL hooks and background-worker metadata. It delays arena initialization until execution in a backend or the background process. At a scheduling boundary, the backend adopts the current configuration generation and configures its own arena. A policy restart can advance that generation and recreate the caller's arena. Other backends adopt it at a later synchronization point. This is not an immediate stop and restart of every query in the database.

oneTBB's arena limit includes the application thread when it participates. An arena limit of four therefore does not promise four extra workers in addition to the backend main thread. The scheduling interface's range operations enter the arena through `task_arena::execute`,

[5] UXL Foundation Contributors, oneTBB Benefits.
[6] PostgreSQL 17, Architectural Fundamentals.

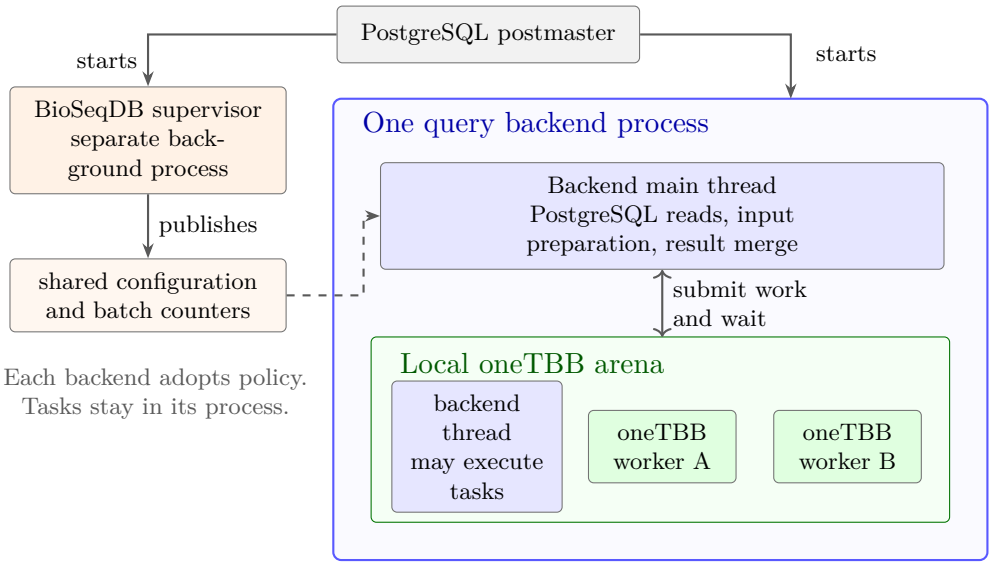


Figure 11.3: The supervisor and the query backend are different PostgreSQL processes. The arena belongs to the query backend. Its worker threads share that process's address space, and the backend main thread may participate in arena computation. Dashed communication carries shared policy, not a cross-process queue of alignment tasks. The three arena participants are illustrative.

which permits the caller to execute work. The configured concurrency is also distinct from a live census of threads that happen to be busy.[7]

PostgreSQL's parallel-query workers are different resources. A `Gather` plan obtains additional PostgreSQL processes, subject to PostgreSQL's worker-process limits.[8] A oneTBB thread inside a backend does not become one of those processes and does not consume a PostgreSQL parallel-worker slot. Nevertheless, both forms of concurrency use the same machine's processors and memory. BioSeqDB clamps its arena width to one when running inside a PostgreSQL parallel worker unless composition is explicitly enabled. This is a defensive policy for such callers. It does not make the alignment custom scan parallel-safe, nor does it allocate a common CPU quota across all client sessions. Several ordinary backends can still compete for the same cores.

11.5.2. Choose the unit of parallel work

The scheduling interface uses oneTBB range operations, task groups, and explicit task submission. The biological pipeline's stages remain the operations described in the earlier algorithm chapters. They are not assigned to a row of permanent threads, with one thread sketching and another chaining. Instead, the implementation exposes independent work where its data and algorithm dependencies permit it.

At dispatch level, eligible in-memory sequence pairs can run independently. Each item writes to its own result collection. The backend combines those collections after the range has completed. A join waits for submitted work to finish before its results are combined. A second useful shape is several queries against one shared target. The backend first validates the query and target identities and prepares the supported target reader. When the required

[7] UXL Foundation Contributors, oneTBB Specification: task_arena.

[8] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Resource Consumption.

resident target can be admitted, queries borrow the same immutable target state and retained query sketches. They still own separate alignment scratch and results.

Figure 11.4 shows why the number of tasks need not equal the number of threads. Query costs vary with the number of anchors and the extension work they induce. oneTBB can subdivide ranges and schedule ready tasks as participants become available. Its scheduler favors recently created local tasks and can steal older ready work from another participant's deque. This balances available work while seeking locality, without guaranteeing a particular task order.[9]

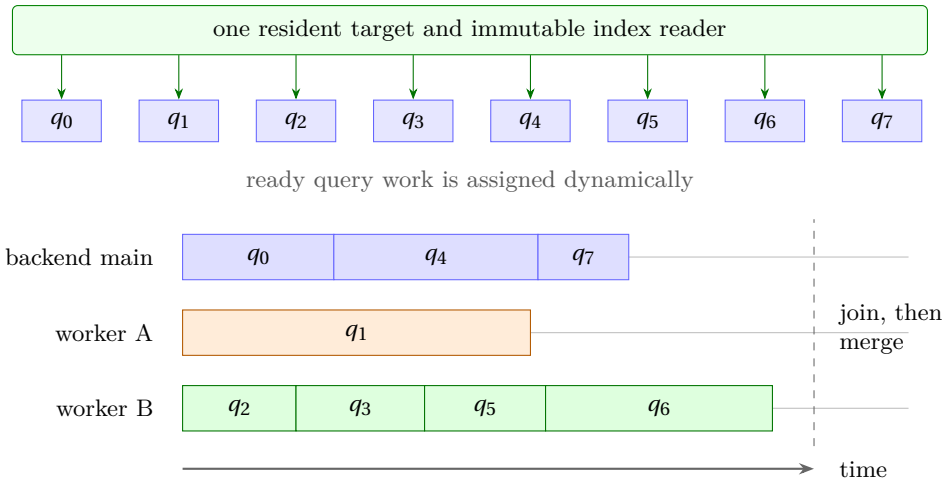


Figure 11.4: Eight independent queries can share three arena participants. Each query has its own result slot, regardless of which thread executes it. Colors distinguish participants. Block lengths and assignments illustrate uneven costs, not measurements or a guaranteed oneTBB schedule. A range task may also contain several queries. Shared target bytes remain owned until all their consumers finish.

Within one query, sufficiently large groups of independent alignment regions can also run in parallel. Large-target recomputation can expose several admitted segments. These choices depend on the selected algorithm and on whether the relevant sequence views can safely cross threads. A PostgreSQL reader does not become shareable because its C++ interface matches a core concept. Such work must retain a supported resident view or use the backend's serial reading path, as described in Section 11.3.

The scheduler provides three modes:

- Single-thread: run dispatch and coarse pipeline regions on the caller.
- Dispatch-parallel: admit eligible pairs from a batch.
- Task-parallel: also allow supported query, region, segment and index-fragment work.

These are runtime scheduling policies, not another dimension of biological algorithm selection. Running a supported region serially does not by itself choose a different minimizer, chainer, or aligner.

Coarse parallel regions share a process-wide guard. Once one such region has acquired it, another coarse region executes serially instead of opening another level of parallel work. For example, a parallel query batch does not also multiply its active workers by independently

[9] UXL Foundation Contributors, How Task Scheduler Works.

parallelizing every query's alignment regions. The guard applies to these coarse regions, rather than prohibiting all nested oneTBB operations. Workload thresholds also avoid submitting very small ranges whose scheduling overhead could exceed their useful work.

Each participating thread needs its own installed computation context for temporary pipeline storage. For context-aware pipeline ranges, the pool supplies contexts scoped to the parallel region and reuses an already installed context when possible. The results outlive temporary scratch through their own owners. Consequently, work stealing changes which thread executes ready work, but does not grant a task access to another thread's mutable alignment scratch storage.

11.5.3. Admit memory before increasing overlap

A CPU limit alone cannot bound memory use. Four simultaneous target-index builds can require roughly four times the temporary storage of one build, even if each task uses only one processor. BioSeqDB therefore combines the arena's concurrency limit with a process-local memory-admission controller. Admission means obtaining a reservation, represented by a move-only permit, before starting work whose retained allocation is covered by that permit.

The controller uses the common memory policy's view of the process footprint and available host memory. It also applies an optional index-memory cap and accounts for bytes already promised to live permits. Reservations are updated atomically, so simultaneous tasks in the same process cannot each treat the same remaining reservation capacity as their own. A rejected request returns without waiting for another task to free memory. The caller can reduce the admitted wave, use a supported serial or streaming path, or report that the selected route cannot proceed.

A wave is the group of work items whose input and temporary storage may overlap. It is smaller than the logical batch when the whole batch would retain too much state. Preparation belongs inside this bound. Waiting until after every sequence has been copied would leave the copies outside the intended memory limit. Conversely, a lazy PostgreSQL source need not be flattened merely because its locator belongs to a batch. The selected reader and compute path determine which bytes must become resident.

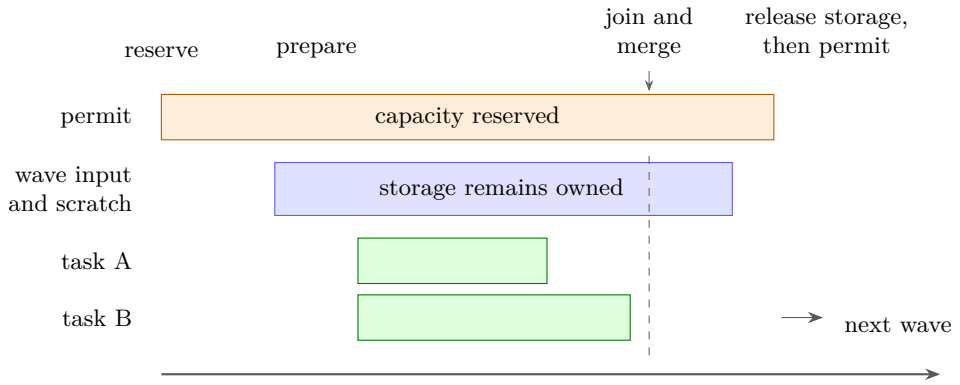


Figure 11.5: An admitted wave retains its input and temporary storage through the join. Completing one task is insufficient to release bytes another task still reads. In the ordinary permit-retaining path shown here, covered storage is released before its reservation becomes available to subsequent work. Output that must survive the wave has a separate owner.

Figure 11.5 distinguishes reservation from allocation. A permit records admitted capacity. It does not own the bytes or prove that the operating system has reserved physical pages.

The ordinary release order is therefore to destroy the covered allocation and then return its permit. Paths that reconcile an allocation with the measured process footprint can adjust reservation accounting, but still need an owner for the bytes and their readers.

Algorithm 20 summarizes the CPU scheduling boundary. It deliberately leaves biological implementation selection outside the loop. An unsupported requested implementation is handled by its admission policy, rather than silently replaced because fewer CPU tasks can run.

Algorithm 20 CPU work under backend ownership and bounded admission

```
Require: supported CPU request and a bounded batch
Backend adopts its scheduling configuration
for each next row or query wave do
    Validate source and index identities on the backend
    Obtain admission for required resident input and scratch
    if mode, work size, ownership, and admission allow parallel work then
        Prepare retained, worker-shareable input
        Run independent items in the local arena into separate result slots
        Join the submitted range
    else
        Execute the supported serial or streaming path
    Check PostgreSQL interrupts at backend-controlled boundaries
    Merge completed results into their continuing owners
    Release the wave's input and temporary storage
    Return remaining admission
```

These decisions bound controlled overlap within one process. They do not provide a cluster-wide memory reservation. Another backend can allocate after a host-memory sample, and unavailable host probes cannot establish how much physical memory remains. Persistent results and other database state also have their own lifetimes. The useful architectural guarantee is the explicit ownership and admission of controlled work, not a claim that any number of sessions or any input can never exhaust memory.

11.5.4. Completion, cancellation, and progress

Submission does not transfer PostgreSQL's API to a worker. Relation access, PostgreSQL allocation, and interrupt checks stay on the backend main thread. The worker runs the permitted computation and returns results or a failure to that thread. For explicitly tracked asynchronous tasks, the owner registers cleanup resources before submission. A bounded error record carries worker failure, and completion is published before joining can release the borrowed storage. The dispatch-buffer cleanup methods in Table 8.2 supply the enclosing statement cleanup. They need not be implemented again by every numeric algorithm.

This boundary limits cancellation responsiveness. The ordinary parallel range completes before the backend resumes its per-row interrupt checks. The native aligner does not continuously call PostgreSQL's cancellation API while computing an alignment. The explicitly tracked task wrapper can observe a cancellation request before invoking its callback, but that is not evidence of interruption at every point inside an already running kernel. A join protects ownership even when the caller no longer needs a result. It does not imply that already executing CPU work stops instantly.

The scheduler must also retain enough runnable work to make progress. oneTBB is intended primarily for computation that does not spend long periods blocked on IO or mu-

texes.[10] Nonblocking memory admission and the coarse-region guard avoid important forms of nested waiting and resource multiplication. They do not prove general deadlock freedom. BioSeqDB's input path, for example, can overlap parsing with backend writes through a bounded queue, so a claim that every pool task is nonblocking would be incorrect. Its producer and consumer require their own close-and-join protocol. The common arena supplies execution resources, while each operation retains responsibility for a valid dependency graph and bounded ownership.

11.6. Load sequence files through a reusable parser

Public sequence archives distribute FASTA or FASTQ text, often compressed with gzip. BioSeqDB's input layer turns that text into compact sequence chunks and raw headers. A PostgreSQL sink then turns the headers into typed columns and stores the chunks. Separating these jobs lets the same parser serve database loading and ordinary C++ consumers without importing PostgreSQL into the file-reading code.

11.6.1. Separate byte sources from record framing

A byte source fills a caller-owned buffer and reports bytes read, end of input, or an error. The parser is specialized for the source type, so refill calls need no virtual dispatch. Plain files can use a memory mapping or sequential reads. The SQL loading path uses zlib for gzip input. Biological format and compression are selected independently: the filename suffix chooses FASTA or FASTQ, while gzip magic bytes identify compression. The loader does not infer the biological grammar from an arbitrary first record.

The parser design was inspired by Helicase's vectorized parsing and compact DNA output, and by StringZilla's vectorized byte-search operations.[11] BioSeqDB implements its own kernels through Highway. These libraries are not runtime dependencies of the input path. In FASTA, the header begins with `>`, followed by a sequence body. For DNA4 packed into 64-bit units, the optimized body loop searches for the next `>` while encoding symbols. Boundary detection and packing thus share an input pass. Other alphabet and unit combinations use the same record states with their corresponding encoder and line scanner.

FASTQ adds a `+` separator and quality bytes. Quality lines can contain `@`, `+`, or `>`, so those characters cannot independently identify the next record. The parser instead counts quality bytes against the raw sequence length after removing line endings. This count is distinct from the number of symbols retained by the compact encoder.

11.6.2. Deliver chunks while a record remains open

The parser invokes sink functions, called callbacks, when a record begins, when packed chunks are available, and when the record ends. It keeps its current state and an unfinished storage word across refills. Complete words can be delivered before the record ends. The final chunk supplies the remaining word and its valid symbol count. Figure 11.6 shows why an input-buffer boundary need not be a record boundary.

The SQL sink runs on the PostgreSQL backend thread. Its default path consumes callbacks directly. Optional overlap moves owned events through a FIFO queue of fixed capacity: a worker parses, and the backend performs database writes. A full queue blocks the producer.

[10] UXL Foundation Contributors, When Task-Based Programming Is Inappropriate.

[11] Martayan et al., "Helicase: Vectorized parsing and bitpacking of genomic sequences"; Vardanian, StringZilla: Fast SIMD, SWAR, and GPGPU String Processing.

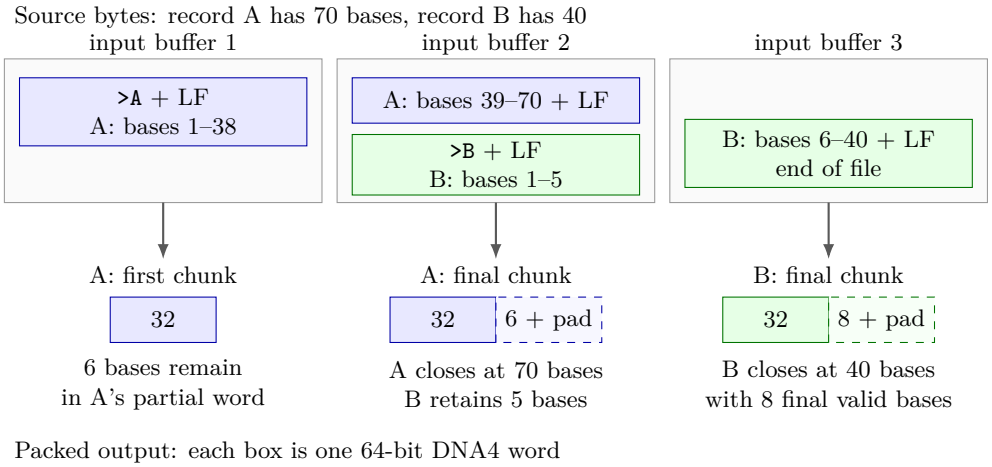


Figure 11.6: Two single-line FASTA records parsed with illustrative 41-byte input buffers. LF denotes a line-feed byte. Header and LF bytes occupy input space but are not sequence symbols. The first refill ends inside A: 32 bases are delivered, while six remain in its partial word. The next buffer finishes A and starts B. Source and packed widths are schematic. Word labels count valid bases, with padding in dashed words. Chunk views are borrowed for their callback and copied by the backend before reuse.

Closing it wakes blocked participants, and the worker is joined before a consumer error is propagated.

The reusable batch API has a different contract. It owns complete batches and can use direct mapped views, parallel parser shards, or a parallel gzip source. Those options do not describe SQL streaming, which refills a slab and uses the single-threaded zlib source for gzip. In particular, a batch parser's throughput is not a measurement of database loading.

11.6.3. Route header fields into SQL columns

Headers can be preserved verbatim or parsed using a named convention, such as GenBank, RefSeq, or UniProt. Their conventions distinguish identifiers from annotations carried in the same definition line.[12] Automatic selection samples a bounded decompressed prefix once per file. The loader also resolves field-to-column routes once, using destination names and any explicit mappings. Per-record parsing then passes field slices to the PostgreSQL sink, which applies the destination types. A header dialect describes metadata structure. It is not a strict validator for the sequence alphabet.

For example, a gzipped FASTA file may contain the header `>dna_new [organism=Homo sapiens] [chromosome=17] [moltype=mRNA]`. The following table gives those fields their relational types:

[12] National Center for Biotechnology Information, FASTA Format for Nucleotide Sequences; UniProt Consortium, FASTA headers.

Listing 11.4: Load a FASTA file with GenBank-style header fields.

```
CREATE TABLE sequences (
  id text,
  organism text,
  chromosome integer,
  moltype text,
  seq bioseq.seq('dna4', 64)
) USING bioseq;

SELECT bioseq.load('sequences'::regclass,
                   '/data/example.fasta.gz');
```

Here automatic routing produces identifier `dna_new`, organism `Homo sapiens`, integer chromosome `17`, and molecule type `mRNA`. The return value is the number of loaded records. Compression changes the byte source, while the SQL interface remains the same.

The sequence column selects the compact alphabet and storage-unit width before parsing starts. Once a record's fragments are ready, the sink writes the parent row and publishes its sequence locator. PostgreSQL controls transaction visibility and rollback. The sink evaluates defaults and `NOT NULL`/`CHECK` constraints. It does not run ordinary `INSERT` triggers or maintain ordinary secondary indexes. Normal trigger-based foreign-key checks and index-based unique or exclusion checks therefore do not run.

The memory guarantee concerns streamed packed bodies. Headers and FASTQ quality storage can grow with record length, and paths that retain unfinished physical lines remain subject to the input-buffer limit. Alphabet handling is permissive, and the declared strict-alphabet option is not enforced by the parser. FASTQ length checks also have an exception for excess quality bytes on an unterminated final line. These limits preclude treating successful loading as complete input validation.

11.7. Remote sequences and local persistence

An installation built with external-provider support exposes metadata, complete nucleotide records and sequence chunks through relations in the `bioseq_external` schema. These relations obtain their rows from remote services. An application can select a versioned accession and insert its sequence into a local BioSeqDB table before repeated analysis.

Metadata discovery and sequence retrieval use distinct requests. Generated HTTPS clients perform bounded metadata requests, and provider adapters normalize their responses. For NCBI, the catalog reads Datasets assembly reports.[13] A nucleotide accession such as `NC_012920.1` instead requests plain-text FASTA from the EFetch nucleotide endpoint.[14] Assembly accessions beginning with `GCF_` or `GCA_` select a Datasets ZIP package containing sequence files. An assembly accession and a constituent sequence accession therefore identify different retrieval units.

The remote relation returns an unbound sequence literal: sequence characters without a selected storage alphabet or packed layout. In Listing 11.5, assignment to the destination `bioseq.seq('dna5',64)` column validates and encodes those characters for the declared alphabet and storage width.

[13] National Center for Biotechnology Information, NCBI Datasets v2 REST API documentation.

[14] Sayers, "The E-utilities In-Depth: Parameters, Syntax and More."

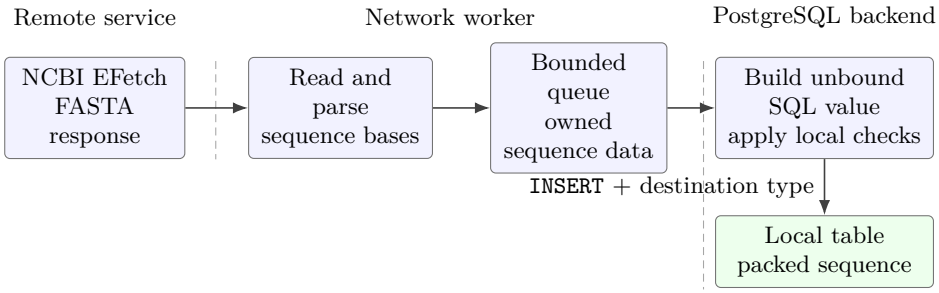


Figure 11.7: An exact NCBI sequence request followed by local insertion. Remote delivery and persistent sequence encoding have separate owners.

Listing 11.5: Importing one versioned NCBI sequence into local storage.

```
CREATE TABLE ncbi_sequences (
    accession text,
    sequence bioseq.seq('dna5', 64),
    retrieved_at timestamptz
) USING bioseq;

INSERT INTO ncbi_sequences
SELECT accession, sequence, retrieved_at
FROM bioseq_external.sequences
WHERE provider = 'ncbi' AND accession = 'NC_012920.1';

SELECT accession, bioseq.len(sequence) AS bases
FROM ncbi_sequences;
```

Ordinary SQL qualifications remain local checks even when equality predicates also constrain the remote request. Array expansion is capped at 64 query combinations, with a warning when combinations are omitted. A provider union may return partial results if one provider fails, and metadata scans do not automatically follow every page. Explicit accessions and a single provider make the import's requested scope clear. Selecting fewer columns from a sequence relation does not currently avoid sequence retrieval.

For the NCBI streaming route, a worker passes owned sequence data through a queue bounded by both bytes and item count. Chunk delivery limits temporary buffering, whereas a complete-record value still requires record-sized memory. Archive retrieval buffers a size-limited ZIP before decoding. The backend checks cancellation between bounded queue waits. Transient FASTA requests can be retried only before the first row is emitted, preventing replay of visible chunks.

Remote responses are not a durable cache, and request rates are not coordinated across backends. E-utilities usage requirements therefore remain relevant to the caller.[15] A transaction controls the local INSERT, but does not make separate HTTP requests one database snapshot. Retaining the accession version and retrieval time records which import supplied the stored sequence. Subsequent queries of the local table need no remote request.

[15] Sayers, "A General Introduction to the E-utilities."

11.8. Generation and validation

The shared computational core needs several interfaces: typed C++ calls, PostgreSQL functions, configuration settings, and benchmark adapters. Keeping these interfaces consistent is part of the architecture. Generation removes some repeated declarations, while assigning compilation work to modules limits its repetition.

11.8.1. Deriving the PostgreSQL boundary

A PostgreSQL routine repeats one contract in several places. Installation SQL declares its argument types, result and volatility. A C entry point must expose the corresponding symbol, decode those arguments and encode that result. BioSeqDB derives these artifacts from annotated C++ declarations. The following existing annotation fixture illustrates the API. Its declaration supplies the name and C++ types, while attributes supply SQL metadata:

Listing 11.6: A typed declaration with SQL metadata.

```
[[bioseq_sql::function({
    .volatility = bioseq::sql::volatility::immutable,
})]]
inline constexpr std::int32_t identity(
    [[bioseq_sql::param({.sql_type = bioseq::sql::pg::integer})]]
    std::int32_t value) {
    return value;
}
```

The generator uses Clang's C++ LibTooling and abstract syntax tree (AST) interfaces through `libclang-cpp`. Compiler actions inspect registered module owners under an explicit compilation profile.[16] AST facts describe declarations and types. Compiler token ranges associate annotations and documentation with those declarations. BioSeqDB then parses the annotation options as a restricted language and validates their admitted forms. Unknown options, conflicting annotations, duplicate defaults and unsupported C++/SQL conversions stop generation. Volatility and parallel safety remain author-supplied contracts: the generator does not prove them from an arbitrary function body.

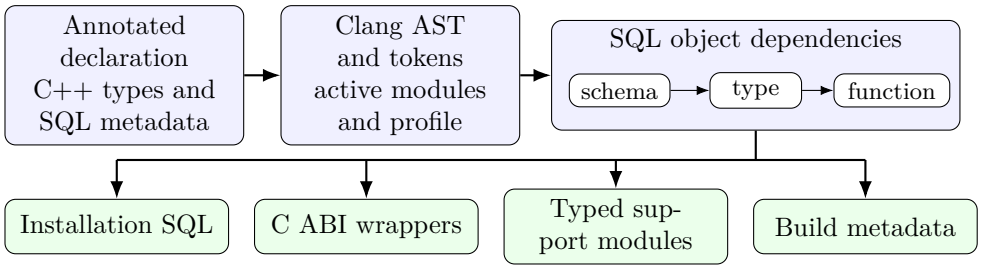


Figure 11.8: One checked interface produces related artifacts. The small dependency chain is illustrative. Typed support includes row codecs, configuration bindings and catalog handles where the declarations require them.

The graph records dependencies among declared SQL objects, each with a stable identity. Tarjan's algorithm identifies strongly connected components, after which a stable topological traversal orders the admitted component graph.[17] Unsupported cycles are rejected. Base

[16] The Clang Team, LibTooling; The Clang Team, Choosing the Right Interface for Your Application.
[17] Tarjan, "Depth-First Search and Linear Graph Algorithms."

types additionally need statement-level ordering between their shell declaration, input/output functions and completed definition.

Generated support code must exist before its consumers compile. The build therefore produces row codecs, configuration bindings and catalog handles before final SQL and wrappers. Final generation checks these intermediate outputs against the active declarations. Separate build profiles keep test-only declarations out of the production interface.

The generated C ABI also centralizes exception translation. C++ exceptions must not escape into PostgreSQL, while a PostgreSQL error can jump past C++ destructors.[18] The boundary preserves captured database diagnostics and leaves C++ exception scopes before raising the SQL error. Calls into PostgreSQL below that boundary still require an explicit bridge or a region without live nontrivial C++ objects.

Language reflection is a related alternative, but it is not the mechanism used here. Reflection was adopted for C++26.[19] Upstream Clang still lists it as unsupported.[20] BioSeqDB therefore uses an external compiler pass, including source locations, preprocessing state and module provenance, together with its own annotation validation.

11.8.2. Controlling specialization cost

Templates specialize kernels for concrete alphabets, storage layouts and algorithm choices. Independent parameters multiply the potential number of instantiations (Figure 11.9). A parameter can create new compiler work even when it does not change a downstream algorithm. Link-time removal of equivalent code cannot recover all the earlier parsing, instantiation and optimization work.

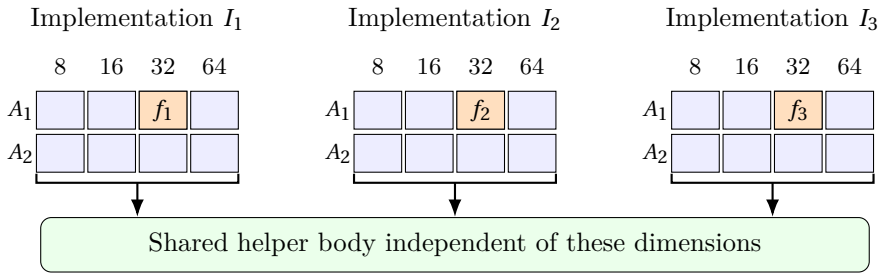


Figure 11.9: Illustrative specialization product: two alphabets, four storage widths in bits, and three implementations produce 24 cells. Each cell denotes one possible function body. Highlighted cells keep alphabet and width fixed while changing implementation. This is not a production support matrix.

BioSeqDB limits this cost by selecting length buckets after fixing the k-mer size, passing already selected input representations into scheduled work, and assigning expensive specializations to concrete module owners. Shared computational modules can be compiled once for compatible consumers. An ordinary benchmark and the extension can consequently reuse the same core without each owning its compilation again. This does not eliminate specialization: it gives the build graph explicit work units and avoids repeating irrelevant choices.

The cost has been substantial during development. Recorded May 2026 runs included a 32-minute debug benchmark build and PostgreSQL translation-unit compilations that exceeded a 50-minute timeout. These are historical observations, not current build timings or

[18] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Using C++ for Extensibility.
[19] Köppe et al., Editors' Report: Programming Languages—C++.
[20] The Clang Team, C++ Support in Clang.

measurements of SQL generation alone. Later investigations also found expensive chains of imported modules behind small function bodies. Controlling both template dimensions and module ownership is therefore necessary.

11.8.3. Benchmark specifications and validation

Benchmarking uses a generic framework and adapters for individual workloads. The framework owns configuration validation, matrix expansion, scheduling, measurement and report generation. An adapter defines the experiment: how to prepare its input, invoke an algorithm or SQL statement, evaluate the result and release temporary resources. The same orchestration serves in-process alignment, sequence input and output, and PostgreSQL experiments.

A JSON specification declares named inputs and algorithm parameter schemas. Within each benchmark, `matrix.for` assigns repetition counts to inputs, while `matrix.over` lists algorithms and parameter values. The framework expands the combinations separately for each algorithm and fills omitted parameters from declared defaults. Schema and reference checks reject inconsistent specifications before execution. JSON selects implementations already registered by the adapters.

Listing 11.7: Input and algorithm selection fields from an alignment benchmark specification.

```
"matrix": {
  "for": {"RMQ_REPRO_LONG_CCS": 100},
  "over": {
    "bioseq_ksw2_rmq_scalar_mm": {},
    "minimap2_rmq": {}
  }
}
```

This excerpt selects 100 repetitions of each registered alignment route on the same named input. The empty objects use the declared parameter defaults. Input definitions, metric selection and other run settings are omitted.

The measurement boundary is explicit. The framework times the execution callback, then passes its input and output to the evaluator. This permits reference computations outside that timer. However, adapters may also check results while constructing their output, so elapsed time must be interpreted together with the measured region. Alignment experiments through PostgreSQL check expected matches, coordinates within the configured tolerance and strand. Selected cases additionally validate CIGAR replay. These checks complement authored integration tests of SQL behavior.

Reports retain metric summaries and rule outcomes. Adapters can additionally emit durable raw samples. Rules compare selected configurations within one run, while regression checks use eligible historical captures. Eligibility requirements determine which runs can enter a historical comparison. The configured matrix still determines coverage: a successful timing run does not establish correctness for every query, setting or executor path.

11.9. Authorship, development and AI assistance

I developed BioSeqDB over approximately three years. Early work included Python aligner prototypes written approximately two years ago. I designed the system and implemented the BioSeqDB-specific algorithms, representations and optimizations presented as my contributions in the preceding chapters. Moving from those prototypes to a PostgreSQL extension also required substantial integration, testing and performance work.

These contributions build on published algorithms and existing software. The minimap2 mapping structure, prior SIMD minimizer work and adaptive block alignment are important foundations.[21] The implementation includes ports and adaptations, as well as new work. The basic columnar store comes from Citus and Hydra and retains code written by their authors and maintainers, as described in Section 9.1.[22] PostgreSQL, SeqAn, Highway and oneTBB also provide substantial components. The original contribution lies in the specific algorithms, changes and integration identified in this thesis. It does not imply authorship of the underlying published algorithms or of all source code in the repository.

11.9.1. Where AI tools were used

I used AI coding tools under requirements and designs that I supplied. Their work included:

- Source reorganization. Long files were divided into modules, functions were renamed, and code was adjusted for Clang-based checks. Many present file and function names were produced during this work.
- Development tools. AI helped implement benchmarking utilities and code generators according to my designs, and assisted with implementation and optimization tasks.
- Technical writing. AI generated working documentation.

The large reorganizations were intended to preserve the algorithms while changing names and file boundaries. They nevertheless introduced bugs. In my assessment, diagnosing and correcting regressions from AI-assisted work consumed approximately two months. Numerical code was particularly sensitive: a seemingly equivalent change to a mask, integer width or coordinate calculation could change results. A refactoring that compiles is therefore insufficient to establish numerical correctness.

AI assistance accelerated parts of development despite these costs. The research questions, project-specific designs and decisions about which ideas to pursue were mine. I directed and corrected the assisted work and retain responsibility for the final implementation and the claims made here.

Automatically generated documentation is not an authoritative specification. Some descriptions were incorrect, and others became stale after later changes. For this chapter, generated descriptions were checked against the implementation, the cited technical sources and the stated tests. Generated names and reports alone do not establish how a feature works.

11.10. Validation limits of the complete system

Validation of query-result caching, described in Section 8.5, remains incomplete. It has targeted tests, including comparisons with caching disabled, but these do not cover all combinations of plans, settings and transaction histories. Incorrect reuse and therefore incorrect query results remain a risk. The cache should be treated as experimental.

[21] Li, "Minimap2: pairwise alignment for nucleotide sequences"; Groot Koerkamp and Martayan, "SimdMinimizers: Computing Random Minimizers, fast"; Liu and Steinegger, "Block Aligner: an adaptive SIMD-accelerated aligner for sequences and position-specific scoring matrices."

[22] Citus Data contributors, Citus Columnar storage implementation; Hydra contributors, Hydra Columnar: PostgreSQL columnar storage extension.

Chapter 12

Benchmarking and performance evaluation

12.1. Benchmarking and performance evaluation

12.1.1. What the results show

The results distinguish two ways of using the same reference:

- Index: build the reference index first, restart PostgreSQL, then measure the query, including access to the stored index.
- No index: measure construction of a temporary reference index and the query together. The query batch shares that temporary index.

What an indexed minimizer comparison measures

Scalar and SIMD-ref have similar indexed times in this example, where they reuse the same kind of reference index and share most of the query pipeline.

Configuration	M7A / Index 2 Gbp / DNA4 $k = 15, w = 10$	M7I / Index 2 Gbp / DNA4 $k = 15, w = 10$
MPF / Scalar Scalar-DP / Block	3.8s ▲3.0× 3.1s ▲3.3×	3.5s ▲3.1× 2.7s ▲3.2×
MPF / SIMD-ref Scalar-DP / Block	3.6s ▲3.1× 2.9s ▲3.5×	3.4s ▲3.2× 2.6s ▲3.3×

Table 12.1: Two minimizer implementations with MPF, Scalar-DP and Block held fixed. Source: Table 12.26, rows R4, R3.

⚠ An indexed row does not measure reference-index construction

The benchmark requires a compatible persisted reference index. It builds that index before restarting PostgreSQL, and does not rebuild its minimizers inside the measured query. The query sequences themselves have no persisted minimizer index: their minimizers are extracted to search the reference. Changing the minimizer therefore does not measure reference construction again, but it need not produce an identical query execution.

The distinction follows the reference-read policy and query-side preparation described in Section 10.7.2. It is also checked by the campaign: the query fixture has no generated indexes, and accepted indexed records report query-sketch work alongside zero target-index recomputation. Scalar and SIMD-ref can share a persisted hash representation, whereas bounded ntHash can change both the query sketch and the reference lookup representation (Sections 4.13 and 4.12). Close timings are therefore consistent with substantial shared work, rather than proof that the complete execution was measured twice.

In each small table, the upper duration is A, the complete query measurement, and the lower duration is B, the residual after removing the recorded CIGAR/MAPQ intervals. Cells, comparison symbols and colours are copied from the full table. The detailed timing definition and remaining markers are explained in Section 12.1.2.

Paying for the reference once

The preferred BioseqDB configuration beats minimap2 when the reference index is available, but loses when temporary construction is included in this example.

Configuration	M7I / Index 2 Gbp / DNA4 $k = 15, w = 10$	M7I / No index 2 Gbp / DNA4 $k = 15, w = 10$
minimap2	10.7s ▼3.2× 8.6s ▼3.5×	56.6s ▲2.1× 54.6s ▲1.9×
MPF / Bounded SIMD-DP / Block	3.5s ▲3.0× 2.7s ▲3.2×	1m 56.2s ▼2.1× 1m 46.2s ▼1.9×

Table 12.2: The preferred configuration on M7I, with and without a prepared reference index. Source: Table 12.26, rows R1, R10.

A persistent index moves reference minimizer extraction and organization out of the query. BioseqDB can then read its prepared lookup structures, as shown in Figures 10.10 and 10.11. Without that index it must construct a transient representation before the batch can use it. This is consistent with the database use case in Section 10.1: preparation can be worthwhile when the same reference serves many later queries.

The difference between these columns is not an isolated measurement of building the persistent index. Temporary construction and persisted lookup use different paths. Durable publication, described in Section 10.6.3, is outside both query measurements. The result supports reuse of a prepared reference, but does not by itself give its construction cost or the number of queries needed to recover that cost.

A smaller reference can have a more expensive lookup path

The scalar MPF configuration takes much longer on the smaller reference with longer k-mers than on the larger reference with shorter k-mers.

Configuration	M7A / Index 4 Gbp / DNA4 $k = 15, w = 10$	M7I / Index 4 Gbp / DNA4 $k = 15, w = 10$
MPF / Scalar Scalar-DP / Block	5.4s ▲6.7× 4.6s ▲7.6×	4.9s ▲7.7× 4s ▲8.9×
MPF / Bounded SIMD-DP / Block	9.9s ▲3.6× 9s ▲3.9×	13s ▲2.9× 11.9s ▲3.0×

Table 12.3: Indexed DNA4 with the shorter k-mer profile. Source: Table 12.24, rows R4, R23.

Configuration	M7A / Index 2 Gbp / DNA4 $k = 20, w = 15$	M7I / Index 2 Gbp / DNA4 $k = 20, w = 15$
MPF / Scalar Scalar-DP / Block	5m 39.5s ▼ 17.3× 5m 39s ▼ 18.2×	12m 20.4s ▼ 40.7× 12m 19.8s ▼ 44.8×
MPF / Bounded SIMD-DP / Block	3s ▲ 6.6× 2.3s ▲ 8.1×	2.6s ▲ 6.9× 1.9s ▲ 8.6×

Table 12.4: Indexed DNA4 with the longer k-mer profile, despite a smaller reference. Source: Table 12.27, rows R38, R2.

The current implementation offers a concrete explanation through the hash representation and its reader. The integer-hash profile for shorter k-mers fits the compact direct lookup described in Section 10.6.3. The longer integer hashes require the wider representation and cannot use that compact reader. The alternative reader scans stored metadata to prepare and emit anchors. Successive groups of queries can revisit this metadata, as explained in Section 10.6.2. Mapping an index file therefore does not make every lookup equally cheap.

The bounded configuration is a useful counterexample: its compact hash profile retains eligibility for direct lookup, and the longer-k-mer case remains fast here. The bounded sketcher has a specialised default-profile kernel and a runtime parameter path (Section 4.13), but a generic sketch loop alone does not explain a large delay in a query that already has its reference index. These measurements also span campaign builds, so current source identifies a credible mechanism rather than a measured breakdown of every historical run.

A larger possible k-mer vocabulary does not mean an array of every possible k-mer is allocated. Index size depends on the observed minimizers, their stored width and their occurrence lists. Increasing the window can reduce minimizer density, while changing k also changes matches and repetition (Section 4.1.1 and Equation 4.3). Longer k-mers therefore do not imply either a larger index or swapping in every case.

SIMD and RMQ are workload-dependent choices

The preferred SIMD configuration is competitive, but the fastest chaining choice changes with the workload.

Workload	MPF / Bounded Scalar-DP / Block	MPF / Bounded SIMD-DP / Block	MPF / Bounded RMQ / Block
M7A / Index 2 Gbp / DNA4 $k = 15, w = 10$	3.6s ▲3.1× 2.8s ▲3.5×	3.7s ▲3.0× 3s ▲3.4×	3.7s ▲3.0× 2.9s ▲3.4×
M7I / Index 2 Gbp / DNA4 $k = 15, w = 10$	3.4s ▲3.1× 2.6s ▲3.3×	3.5s ▲3.0× 2.7s ▲3.2×	3.5s ▲3.0× 2.7s ▲3.2×

Table 12.5: Chaining choices with MPF, bounded minimizers and Block fixed. Source: Table 12.26, columns C1, C2.

Here the complete-query times are close. Scalar chaining already avoids much unproductive work through bounded predecessor searches and stopping rules (Algorithm 10). SIMD must also arrange coordinates and scores for vector processing, and its backtracking needs additional storage (Section 5.7 and Algorithm 14). Those costs can offset a cheaper score calculation. Shared index access and anchor generation can also hide a change in chaining time. The totals do not establish which effect dominates, or prove that compiler autovectorization explains a scalar win.

Workload	MPF / Bounded Scalar-DP / Block	MPF / Bounded SIMD-DP / Block	MPF / Bounded RMQ / Block
M7A / Index 4 Gbp / DNA4 $k = 15, w = 10$	5.3s ▲6.8× 4.6s ▲7.7×	9.9s ▲3.6× 9s ▲3.9×	5.4s ▲6.8× 4.6s ▲7.6×
M7I / Index 4 Gbp / DNA4 $k = 15, w = 10$	5.5s ▲6.9× 4.6s ▲7.8×	13s ▲2.9× 11.9s ▲3.0×	9.9s ▲3.9× 9.1s ▲4.0×

Table 12.6: The same choices on the larger DNA4 reference. Source: Table 12.24, columns C1, C2.

The larger case favours Scalar-DP. This is also a comparison of stopping policies: the SQL SIMD-DP choice uses bounded approximate batches, rather than the exact scalar predecessor traversal (Algorithm 11). RMQ replaces predecessor scanning with tree maintenance and range queries. Its benefit depends on how much scanning it saves. Its current candidate selection also differs from minimap2 RMQ (Section 5.11.3 and Table 5.7). These alternatives need not perform identical work or produce identical intermediate regions.

Workload	MPF / Bounded Scalar-DP / Block	MPF / Bounded SIMD-DP / Block	MPF / Bounded RMQ / Block
Local / Index 2 Gbp / DNA4 $k = 20, w = 15$	2.4s ▲ 12.0× 1.9s ▲ 13.8×	2s ▲ 15.0× 1.3s ▲ 20.4×	9.5s ▲ 3.1× 9s ▲ 2.9×

Table 12.7: A local case in which SIMD-DP wins and RMQ loses. Source: Table 12.23, column C7.

The local result prevents a general conclusion that SIMD helps less on the Apple machine than on AWS. For matched, complete five-repeat comparisons with MPF, bounded minimizers, Block and DNA4 fixed, the median AWS differences are small. Restricting the comparison to measurements from the same source file preserves that conclusion. The local coverage is too sparse for an equivalent aggregate. These slices support selecting a chaining method using matched workloads.

Vector width alone does not explain the CPU results

Scalar and reference SIMD minimizers can remain close on both the local machine and the AWS machines after reference construction is removed.

Workload	MPS / Scalar SIMD-DP / Block	MPS / SIMD-ref SIMD-DP / Block
Local / Index 2 Gbp / DNA4 $k = 15, w = 10$	4.4s ▲ 6.6×[s] 3.5s ▲ 7.5×[s]	4.4s ▲ 6.6×[s] 3.5s ▲ 7.4×[s]

Table 12.8: Local indexed minimizer choices with MPS, SIMD-DP and Block fixed. Both are single samples. Source: Table 12.23, column C5.

The local cells are close single observations, not a repeatability estimate. An indexed query extracts minimizers from the queries rather than sketching the entire reference again. A faster sketcher can therefore have little effect on complete-query time. Moreover, the reference SIMD implementation has shared rescan work when lanes lose their minima. Vector width does not remove that cost (Section 4.3). The bounded implementation uses different work organization, described in Section 4.13.

The current bounded kernel also interleaves two streams on the four-lane path and one on the eight-lane path, keeping the number of logical streams the same (Section 4.13.1). This changes instruction scheduling and register demand. It does not make the two processors equally fast. The earlier stream experiments in Section 4.9 explain the motivation. They do not establish a universal SIMD speedup across the different CPUs, index readers and memory conditions in this campaign.

Ambiguous bases expose work beyond minimizer extraction

The local DNA5 slowdown is much larger in the complete no-index measurement than in its residual measurement.

Configuration	Local / Index 2 Gbp / DNA4 $k = 15, w = 10$	Local / Index 2 Gbp / DNA5 $k = 15, w = 10$	Local / No index 2 Gbp / DNA4 $k = 15, w = 10$	Local / No index 2 Gbp / DNA5 $k = 15, w = 10$
MPF / Bounded SIMD-DP / Block	2.5s ▲ 11.5× 1.7s ▲ 15.6×	5.4s ▲ 2.8× 4.2s ▲ 3.1×	1m 54.3s ▼ 2.6× 1m 45.5s ▼ 2.5×	5m 17.3s ▼ 8.2× 1m 54.9s ▼ 3.0×

Table 12.9: The preferred local configuration on DNA4 and DNA5. Source: Table 12.23, row R2.

Ambiguous bases change the workload at several stages. The bounded DNA5 sketcher includes N in the k-mers it hashes. It does not simply skip every k-mer containing N like the minimap2-style ambiguity rule. Its hashes and selected anchors can therefore change along with the alphabet (Section 4.13). The CPU kernel supports this profile, so these rows are not evidence of an unnoticed scalar replacement. DNA5 also uses a different packed representation (Section 7.2).

However, the large gap that disappears from B points towards time covered by the alignment/CIGAR intervals, not solely towards an expensive N test in the sketcher. The fused alignment work described in Section 6.1.1 can change when anchors or exact matching runs change. The totals do not isolate that mechanism from other alignment work. They justify investigating the downstream alignment path before attributing the whole slowdown to DNA5 minimizer extraction.

A fast residual does not mean a fast complete query

This MPS configuration loses to minimap2 on complete time even though its residual time is lower.

Configuration	M7A / Index 2 Gbp / DNA4 $k = 20, w = 15$	M7I / Index 2 Gbp / DNA4 $k = 20, w = 15$
minimap2	19.6s ▼ 6.7× 18.6s ▼ 8.4×	18.2s ▼ 6.9× 16.5s ▼ 8.6×
MPS / Scalar SIMD-DP / Block	43.8s ▼ 2.2× 8.6s ▲ 2.2×	38s ▼ 2.1× 8.7s ▲ 1.9×

Table 12.10: An indexed comparison whose direction changes between A and B. Source: Table 12.27, rows R1, R18.

The CIGAR interval includes base dynamic programming, trace storage, traceback and result construction, not just conversion of an existing alignment into text (Section 6.1.1). A large A/B separation therefore identifies substantial wall time covered by that instrumented work. It is not evidence of a costly output copy alone.

There is a further limit under parallel execution: one query can be aligning while another is still seeding or chaining (Section 11.5.2). Removing the union of alignment intervals also removes wall time during which those other tasks ran. B is neither an isolated sum of early-stage costs nor a prediction of running the query with CIGAR and MAPQ disabled. Close A and B show little wall time covered by the removed intervals. They do not identify one remaining stage as the bottleneck.

The historical divergence-sort problem in Section 4.9.2 is a different issue. B removes the recorded CIGAR and MAPQ intervals. It does not subtract a separate timer for divergence estimation, which precedes final alignment for each query. Divergence work can nevertheless overlap another query's alignment interval. The current estimator uses prefix counts (Section 6.3.1). The reversal between A and B does not demonstrate a recurrence of the earlier sorting problem.

Block Aligner improves complete query time

Block Aligner improves complete time in this matched comparison, while the residual changes are smaller.

Configuration	M7A / Index 2 Gbp / DNA4 $k = 15, w = 10$	M7I / Index 2 Gbp / DNA4 $k = 15, w = 10$	M4 / Index 2 Gbp / DNA4 $k = 15, w = 10$
MPF / Bounded SIMD-DP / Block	3.7s ▲3.0× 3s ▲3.4×	3.5s ▲3.0× 2.7s ▲3.2×	6.7s ▲3.3× 4.5s ▲4.2×
MPF / Bounded SIMD-DP / KSW2	5.2s ▲2.1× 2.8s ▲3.6×	4.7s ▲2.3× 2.5s ▲3.5×	9.9s ▲2.3× 4.3s ▲4.4×

Table 12.11: Changing only the requested base aligner in the preferred configuration. Source: Table 12.26, rows R10, R14.

This pattern is consistent with a benefit in the fused alignment work. Block Aligner follows an adaptive block through the scoring matrix, whereas KSW2 uses its banded dynamic programming and stopping rules. Their gap models and extension cutoffs differ (Section 6.1.1). Fewer computed cells can matter as much as the speed of one cell update.

Matched five-repeat DNA4 comparisons favour Block on the shorter-k-mer AWS workload, but occasional reversals remain elsewhere. A reversal accompanied by a large change in B is weak evidence about the base aligner alone: residual wall time can change with concurrent overlap and execution conditions as well as other work. The current pipeline uses a small adaptive block range, described in Section 6.1.1. Changing that range may trade more score cells against less block-management work, but read length alone does not determine the best setting: the size and divergence of each region being extended matter. These tables do not contain a block-size sweep.

MPF and MPS change both lookup and downstream work

MPF is faster in this matched example, but the no-index comparison does not measure persistent-index construction in isolation.

Configuration	M7A / Index 2 Gbp / DNA4 $k = 15, w = 10$	M7A / No index 2 Gbp / DNA4 $k = 15, w = 10$
MPF / Bounded SIMD-DP / Block	3.7s ▲3.0× 3s ▲3.4×	1m 56.4s ▼2.2× 1m 48.1s ▼2.1×
MPS / Bounded SIMD-DP / Block	6.7s ▲1.7× 6s ▲1.7×	2m 20.8s ▼2.7× 1m 57.4s ▼2.3×

Table 12.12: MPF and MPS with the same minimizer, chaining and alignment choices. Source: Table 12.26, rows R10, R27.

MPF retains fragment-oriented lookup data. MPS groups occurrences over the sequence (Figure 10.5). Their readers expose different metadata, occurrence grouping and access patterns. The hash-major MPS reader can read selected streams rather than every stored position (Section 10.6), but that advantage does not guarantee a cheaper complete query than MPF direct lookup. Matched five-repeat bounded DNA4 comparisons usually favour MPF for indexed execution. The no-index differences are smaller and are sometimes close.

Persistent MPS construction includes sorting and may spill runs to temporary files (Figure 10.8). That explains a possible preparation cost, not a cost automatically charged to an indexed query. The sorter already keeps runs in memory when its memory budget allows and spills when needed. Temporary construction, persistent publication and later reading are different operations (Sections 10.5 and 10.6.3). An index/no-index subtraction cannot rank their durable build times.

Configuration	M7A / Index 4 Gbp / DNA5 $k = 20, w = 15$	M7I / Index 4 Gbp / DNA5 $k = 20, w = 15$
MPF / Bounded SIMD-DP / Block	18.2s ▲1.9× 16s ▲2.1×	13.2s ▲2.6× 11.4s ▲2.9×
MPS / Bounded SIMD-DP / Block	8m 5.8s ▼14.3×[s] 38.1s ▼16%[s]	5m 56.7s ▼10.4×[s] 35.4s ▼8%[s]

Table 12.13: The large DNA5 MPS slowdown appears on several machine classes. Source: Table 12.25, rows R2, R7.

The MPS anomaly is not unique to M7A. Much of its additional elapsed time is covered by the intervals removed from B, so persistent spill sorting does not explain the gap. Current DNA5 MPS uses the grouped reader and a serial query loop with fragmented target access, whereas MPF can use a shared resident target and parallel batch (Section 10.6.1). Alignment requests through the PostgreSQL view copy ranks through cached fragment access and per-

symbol extraction (Section 7.5.1). This is different work from reading a shared resident target, and provides a possible connection between the index route and the fused alignment interval. The packed fragment slots are separate from the decompressed stripe cache, which was disabled in the original benchmark campaign (Section 9.5.1).

Each MPS cell comes from one measured batch in the FAST fallback procedure, whereas the MPF cells have five repeats. The current source provides a plausible explanation, but there is no stage trace from each historical binary. The table cannot assign the whole gap to fragmentation, serial execution or memory pressure separately.

Machine comparisons include memory and I/O pressure

The minimap2 baseline itself slows sharply on C7G, so its large BioseqDB comparison factors also reflect a stressed reference run.

Configuration	M7A / Index 4 Gbp / DNA4 $k = 20, w = 15$	M7I / Index 4 Gbp / DNA4 $k = 20, w = 15$	M4 / Index 4 Gbp / DNA4 $k = 20, w = 15$	C7G / Index 4 Gbp / DNA4 $k = 20, w = 15$
minimap2	2m 48.1s ▼ 20.4× 2m 41.7s ▼ 22.6×	2m 50.3s ▼ 19.8× 2m 43s ▼ 22.2×	4m 54.3s ▼ 43.2× 4m 39s ▼ 54.6×	25m 5.5s ▼ 41.8× 25m 0.7s ▼ 46.6×
MPF / Bounded SIMD-DP / Block	9.2s ▲ 18.3× 8.5s ▲ 19.0×	11.4s ▲ 15.0× 10.7s ▲ 15.3×	6.8s ▲ 43.2× 5.1s ▲ 54.6×	39.1s ▲ 38.5×[8] 38s ▲ 39.5×[8]

Table 12.14: The same indexed workload across the AWS machine classes. Source: Table 12.25, rows R1, R2.

The C7G minimap2 record reports severe paging during measurement. Its available physical memory was nearly exhausted, and a large part of the helper's address space was swapped out. C7G also has less physical RAM than the other AWS classes in this campaign. The residency distinctions in Section 10.7.4 matter directly: accessing a stored index can wait for data even when no index is rebuilt.

M4 is slower in many comparisons, but is not uniformly the slowest machine. Its older CPU and lower instance-level EBS bandwidth can both matter when working data must be fetched from disk. The common provisioned volume settings do not make the effective I/O paths equal.[1] Neither the inventory nor the elapsed times measure a universal swap penalty for each processor. The C7G baseline measures execution under severe memory pressure. Its large comparison factor should therefore not be presented as a BioseqDB speedup that applies independently of the machine and its memory conditions.

[1] Amazon Web Services, Amazon EBS-optimized instance types.

An indexed query can lose to temporary construction

The C7G measurement is slower with a persisted index than without one.

Configuration	C7G / Index 2 Gbp / DNA4 $k = 20, w = 15$	C7G / No index 2 Gbp / DNA4 $k = 20, w = 15$
minimap2	6m 29.2s ▼ 92.8× 5m 56.9s ▼ 184.9×	4m 51.6s ▼ 2.9× 4m 46.6s ▼ 3.0×
MPF / SIMD-ref SIMD-DP / Block	13m 2.2s ▼ 2.0× 13m 1.5s ▼ 2.2×	2m 49.5s ▲ 1.7× 2m 21.9s ▲ 2.0×

Table 12.15: The apparently reversed index benefit for MPF and SIMD-ref. Source: Table 12.27, rows R1, R48.

The indexed record explicitly reports persisted MPF decoding and query-sketch jobs with no reference-index build or recomputation. Restarting PostgreSQL removes process-resident state. It does not remove the persisted index. The slow time must therefore be interpreted as querying and accessing that index, rather than silently constructing it again.

The two modes need not perform identical lookup work. A temporary index can use an in-memory representation, whereas a wide-hash persisted reader can repeatedly traverse stored controls (Section 10.6.2). Memory residency adds another possible difference (Section 10.7.4). These cells also come from different campaign replicas with different execution histories. The generator selects the lowest approved A independently for each cell. The table therefore shows a real recorded reversal, but does not establish a timing bug or show that removing the index repeatedly improves performance on one fixed deployment.

The longest runs need a resource explanation before a kernel ranking

Several wide-hash MPF observations remain extremely slow even after the alignment intervals are removed.

Workload	MPF / Scalar Scalar-DP / Block	MPF / SIMD-ref RMQ / Block
C7G / Index 4 Gbp / DNA4 $k = 20, w = 15$	3h 12m 16.6s ▼ 7.7×^{s} 3h 12m 14.4s ▼ 7.7×^{s}	2h 47m 50.9s ▼ 6.7×^{s} 2h 47m 48.7s ▼ 6.7×^{s}

Table 12.16: Examples of the longest indexed observations. Superscript s denotes a single sample. Source: Table 12.25, column C4.

Each of these very long measurements comes from a single FAST observation. Their close A and B place most of the delay outside the removed alignment intervals, but do not separate index traversal, anchor work, page faults and scheduling. The repeated control scans described in Section 10.6.2 can amplify slow memory or storage access: the same large metadata region is visited for successive query waves. Changing the final base aligner cannot remove that preceding work.

The campaign logs also contain a backend that appeared idle while its disk-read counter continued to advance and the kernel reported a page wait. That example occurred during untimed preparation. It explains why low CPU use alone is not evidence of a deadlock, but cannot diagnose these timed cells. The admission path in Section 11.5.3 and Algorithm 20 returns a decision rather than sleeping until memory becomes available. A denied reservation makes the caller reduce, serialize or reject work. This rules out treating the admission API itself as a demonstrated indefinite memory wait.

The available records contain no per-repeat trace proving a TBB deadlock in these completed runs, nor a uniform paging trace proving that every long cell had the same cause. A controlled repeat would need the actual reader, wave progress, page-fault/I/O counters and worker stacks together. Until then, these observations reveal a serious robustness limit of the measured path. They should not determine a general Scalar-DP versus RMQ ranking.

A larger GPU does not remove the host part of the pipeline

The local Vulkan route remains slower than minimap2 when the reference index is already available.

Configuration	Local / Index 2 Gbp / DNA4 $k = 15, w = 10$	Local / Index 2 Gbp / DNA4 $k = 20, w = 15$
minimap2	28.6s ▼11.5× 25.8s ▼15.6×	29.3s ▼15.0× 26.2s ▼22.9×
MPF / Vulkan Vulkan-DP / Block	6m 46.4s ▼14.2× 6m 34.1s ▼15.3×	3m 11.5s ▼6.5× 3m 2.2s ▼6.9×

Table 12.17: The local integrated GPU with Vulkan seeding and chaining. Source: Table 12.23, rows R1, R67.

The local benchmark used unified CPU/GPU memory, so its result cannot be explained simply as using CPU RAM instead of separate GPU VRAM. The published route includes host work, GPU submissions and synchronization. Its selected base aligner runs on the CPU, and the longer-k-mer local record also identifies CPU backtracking (Section 5.13.2). The staged minimizer schedule in Algorithm 8 illustrates why fast device computation need not imply fast complete-query execution.

Configuration	Vast.ai / Index 2 Gbp / DNA4 $k = 15, w = 10$	Vast.ai / Index 2 Gbp / DNA4 $k = 20, w = 15$
minimap2	14.1s ▲7.7×[s] 13s ▲4.5×[s]	1m 34.6s ▶ 4%[s] 1m 33.6s ▼2.1×[s]
MPS / Vulkan Vulkan-DP / Block	1m 48.7s ▼7.7×[s] 58.9s ▼4.5×[s]	1m 38.5s ▶ 4%[s] 47s ▲2.0×[s]

Table 12.18: The MPS Vulkan route and the same-host minimap2 baseline on an RTX 5090. Source: Table 12.32, rows R1, R3.

The discrete-GPU results are more mixed: the longer-k-mer MPS case approaches the baseline on complete time, while the other case remains much slower. These are single observations. They use a different host and software build from the local campaign, so they do not isolate the effect of replacing the GPU. The Vast.ai build places GPU working buffers in device-local memory but retains CPU-cached arrays for host consumers. The campaign diagnostics found that moving host-read chaining data towards the GPU could itself slow backtracking. The useful design rule is to place each array near the work that consumes it and reduce transfers between stages.

G3SA targets long-read alignment too. Its results do not imply that GPUs are useful only for short reads. Its released pipeline uses multiple kernels and retains major intermediate arrays on the device, rather than placing the whole mapper in one kernel.[2] Algorithm 7 describes one such stage. This suggests a direction for reducing host/device interaction, but its different pipeline does not predict the speedup of the hybrid PostgreSQL implementation measured here.

GPU capacity failures have more than one cause

The larger MPS GPU cases fail even though the MPF cases complete on the same device.

Configuration	Vast.ai / Index 4 Gbp / DNA4 $k = 15, w = 10$	Vast.ai / Index 4 Gbp / DNA4 $k = 20, w = 15$
MPF / Vulkan Vulkan-DP / Block	7m 7.5s ▼21.9×[s] 6m 8.1s ▼20.4×[s]	5m 48s ▼8.2×[s] 4m 48s ▼6.9×[s]
MPS / Vulkan Vulkan-DP / Block	×[2]	×[1]

Table 12.19: Completed MPF observations and MPS preparation failures on the RTX 5090. Source: Table 12.32, rows R5, R3.

The diagnosed MPS failure is a storage-buffer binding limit. Its combined hash/offset/count control array exceeds the device's allowed storage-buffer range, although the position buffer fits. These are index controls of the kind separated from position streams in

[2] Han et al., "G^{3}SA: A GPU-Accelerated Gold Standard Genomics Library for End-to-End Sequence Alignment"; Kim et al., G3SA: released long-read alignment implementation.

Figure 10.9, not just one raw sequence fragment. Available total GPU memory does not establish that one binding can represent the whole array. Splitting the work into bounded buffer windows is required even on a device with more VRAM. The table does not contain a successful full-workload measurement of that repair.

For the longer-k-mer profile, the detailed buffer diagnostics come from a companion KSW2 run using the same MPS index profile. The displayed Block run records a more general admission failure.

Other absent cells have different meanings. The measured Vast.ai build rejected the DNA5 sequence type during setup. Missing or failed execution evidence does not constitute a slow timing. These distinctions follow the admission principle in Section 11.4.3: a selected route must be supported and proved to have run before its elapsed time is compared.

Search dominates this BLAST measurement

BLAST remains much slower than the mapping-oriented tools even when its native reference database is prepared beforehand.

Workload	minimap2	BioseqDB	winnowmap	lexicmap	blastn
M7A / Index 2 Gbp / DNA4 $k = 15, w = 10$	11.2s ▼3.0× 10s ▼3.4×	3.7s ▲3.0× 3s ▲3.4×	12.9s ▼15% 11.1s ▼11%	2m 14.1s ▼12.0× 2m 14s ▼13.4×	5m 36.2s ▼30.1×[f] 5m 30.7s ▼33.0×
M7A / No index 2 Gbp / DNA4 $k = 15, w = 10$	53s ▲2.2× 51.9s ▲2.1×	1m 56.4s ▼2.2× 1m 48.1s ▼2.1×	3m 33.2s ▼4.0× 3m 31.5s ▼4.1×	3m 18.9s ▼3.8× 3m 18.7s ▼3.8×	5m 39.9s ▼6.4× 5m 34.5s ▼6.4×

Table 12.20: The preferred BioseqDB tuple and the external aligners on M7A. Source: Table 12.30, columns C1, C9.

The external timing includes launching the process and producing its output, but the BLAST stage evidence places most elapsed time inside word finding and preliminary search. The long indexed time is therefore not well explained as executable startup alone. BLAST's seed search and extension strategy differs from the minimizer/chain pipeline introduced in Section 2.3. Preparing a native database does not remove that query work. Increasing the word length can reduce it, so the longer-k-mer profile is not universally slower across tools.

Winnowmap also benefits strongly from a prepared index in this slice, while LexicMap's indexed search remains slower than minimap2 on this workload. The comparison rules in Section 12.1.4 remain important: native indexing, reference partitioning and ambiguity handling differ between tools. LexicMap has no equivalent minimizer-window parameter, and the B intervals do not all cover the same fused work as BioseqDB. The useful result is the complete time for the stated workload and validation rules. It does not establish interchangeable alignment semantics or an isolated kernel speed ranking.

12.1.2. Measurement procedure and reading the full tables

The following tables present the approved results available while the benchmark campaign is still running. They compare the BioseqDB mapping pipeline with minimap2 on one local Apple Silicon machine and four AWS machines. A separate Vast.ai table compares Vulkan configurations with a same-host minimap2 baseline. Each timing covers a batch of 1,024 queries. A missing measurement remains visibly missing, so the tables describe completed observations rather than an estimated full matrix. The machine inventory precedes the result tables.

- Index. A persisted index is constructed before measurement. Each repeat starts a fresh process and measures reading the index and mapping the queries. Construction of the persisted index is excluded from the current mapping interval. The index is available on disk, but is not retained in the previous process's memory. Operating-system page caches are not purged, so this is process-cold execution rather than a guaranteed cold disk read.

- No index. No pre-existing user index is available. The current measured interval includes transient target-index construction and mapping. The transient index can be reused within the batch, including by the remaining 1,023 queries after its initial construction. Fixture generation and process restart setup lie outside the current measured interval.

- Timing scopes. Historical outer-E2E timings are treated as mismeasurements and excluded from the tables, coverage counts and comparisons. Standard observations use the current framework mapping-iteration interval. FAST fallbacks retain their distinct single-sample timing scope and carry an [s] marker. They are never pooled with standard repeats. The external-aligner table identifies its separate timing boundary explicitly. In particular, "mapping interval" still includes transient construction in no-index mode.

- Two measurements per cell. The first duration, A, is the nearest-rank P50 of the complete measured interval. The second, B, is the nearest-rank P50 after subtracting the physical union of instrumented CIGAR and MAPQ intervals separately in each repeat. With five repeats the P50 is the third sorted observation. For FAST entries, P50 is the single observed value. If U_i is that interval union, the definitions are

$$A_{50} = \mathrm{P50}(A_i), \qquad B_{50} = \mathrm{P50}(A_i - |U_i|).$$

  Overlapping intervals are subtracted only once. The published medians are used directly, without subtracting independently aggregated medians. On both BioseqDB KSW2 and Block Aligner paths, the CIGAR interval contains fused base alignment, trace storage, traceback and result construction. B therefore does not universally represent seeding and chaining alone. Likewise, $A_{50} - B_{50}$ is not a measured median CIGAR/MAPQ cost. MAPQ is mapping confidence, not seed-divergence estimation (Section 6.3). The divergence estimator has no dedicated interval subtracted here. With parallel queries, however, one query's divergence or chaining work can overlap another query's CIGAR/MAPQ interval and lie inside the removed wall-time union. B is consequently residual elapsed time, not the runtime of an execution with those stages disabled.

Cache configuration. The original benchmark campaign disabled the optional cache of decompressed column and stripe chunks. Its setting must not be inferred from binary releases, where it is enabled by default. This cache is separate from the scanner's packed-fragment slots and the operating-system page cache (Table 9.1). The published summaries do not retain a per-observation snapshot of this GUC, so current source defaults cannot independently establish the setting of every historical build.

Input sequences and repetitions. The campaign configuration defines eight deterministic target fixtures, covering both target sizes, both alphabets and both minimizer parameter pairs. Each fixture contains one reference sequence and 1,024 derived queries. The same fixture is reused across the relevant algorithm configurations and repeats. The configured parameters are as follows.

- Target sizes and minimizers. References contain exactly 2×10^9 or 4×10^9 base pairs, denoted 2 and 4 decimal Gbp. The parameter pairs are $(k, w) = (15, 10)$ and $(20, 15)$, where k is the k-mer length and w is the number of k-mers in a minimizer window.

- Alphabets. DNA4 uses equiprobable canonical bases. The DNA5 generator assigns probability 1% to N and 24.75% to each canonical base.

- Query lengths and strand. The sampled pre-edit length has Gaussian mean 20,000 bp and standard deviation 500 bp, with rounding and clamping in the generator. Insertions and deletions can change the final length. A query is reverse complemented with probability 0.5.

- Divergence parameters. Substitution probability is 0.001. Insertion-event and deletion-event probabilities are each 0.0005. Their nominal sum is 0.002, or 0.2%, rather than a measured edit-distance rate. Indel lengths have a configured mean parameter of 1 bp and the generator's default standard-deviation parameter of 1 bp, followed by rounding and clamping. They are not fixed single-base edits.

- Occurrences. Each query originates from one known sampled reference interval. The fixture does not specify an occurrence-count distribution or an occurrence standard deviation. This provenance does not imply that an aligner returns exactly one hit. The original five summaries certify exact baseline match-count equality but intentionally omit the actual counts.

- Repetitions and variability. Standard observations have five accepted repeats. FAST observations have one measured batch and cannot establish repeat-to-repeat variability. A repeat measures the complete query batch, not a single query. The original five summaries contain medians and repeat counts, but no raw timing samples, so timing standard deviations and confidence intervals cannot be recovered from these files.

- FAST procedure. Compatible persisted indexes are reused across index-mode configurations. PostgreSQL restarts before measurement, and the operating-system filesystem cache is not cleared. Index preparation and restart are outside the timer. No-index measurements still include transient index construction. FAST remote runs omit the additional plan and alternative-setting controls, while retaining measured-statement correctness and timing checks.

Execution rows. BioseqDB uses task-parallel execution with prefiltering disabled and both CIGAR and MAPQ projection enabled. MPF and MPS denote minimizers per fragment and per sequence. Each row identifies the requested minimizer, chaining algorithm and base aligner in addition to this index representation. Scalar-DP and SIMD-DP refer to scalar minimap2 chaining and the SQL route's bounded SIMD-DP mode (Algorithm 11). RMQ denotes the separate range-minimum-query algorithm. SIMD-ref is the reference minimizer, Two-stack labels the `two_stack_nthash` setting described in Section 4.13, and Bounded is bounded ntHash SIMD. Block denotes Block Aligner. Vulkan rows can include CPU index handling, backtracking and base alignment, and do not imply an entirely GPU pipeline. If none of the Vulkan configurations has a measurement for the workloads and machines in a table, those configurations share one "Vulkan (all modes)" row. Recorded GPU unavailability is marked explicitly. A missing timing alone is not taken as evidence of unavailable hardware.

Reading comparisons and highlights. Each cell places the A duration above its comparison, followed by the B duration and its own comparison. Durations are rounded to tenths of a second for display. All comparisons and minima use the original integer nanoseconds.

- Minimap2 remains the first row. Local execution rows are ordered by their minimum available Index-mode A timing across all local sizes, minimizer parameters and alphabets, with all cases retained in the same table. AWS execution rows are ordered separately for each target-size and (k, w) group, taking the minimum across both alphabets and all four machine classes within that group. Each order is retained across continuation pages. No-index timings and B timings do not affect ordering. Rows without Index measurements in the relevant group appear last. Equal minima use the execution configuration as a deterministic tie-breaker. This ordering reflects the fastest available Index case, not average performance.

- A BioseqDB value is compared with minimap2 for the same machine class, fixture and index mode. Marked FAST fallbacks may be compared with standard observations, which have a different repeat count and execution procedure. The minimap2 row compares with the fastest measured eligible BioseqDB effective path in that same column, independently for A and B. Thus its two reference algorithms can differ. Baselines are repeated on continuation pages solely to assist reading.

- A green upward triangle means less elapsed time. A red downward triangle means more elapsed time. A gray rightward triangle indicates a tie when the ratio of the larger to the smaller duration is at most 1.05. This is a display convention, not a statistical significance test.

- For current duration t and reference r, the percentage is $100|t-r|/r$, describing relative elapsed-time reduction or increase. Differences above 30% use ratio notation instead: the larger duration divided by the smaller, rounded to one decimal place. A green 2.5× therefore means the reference takes 2.5 times as long, while a red 2.5× means the displayed execution takes 2.5 times as long. Differences through 30% retain percentage notation. Direction is always conveyed by the triangle.

- A blue background marks one smallest A and one smallest B down each machine/workload column, including minimap2 and all measured BioseqDB rows across continuation pages. Exact ties select the first displayed row. Repeated minimap2 baselines receive

blue marks only on the first page, so each column has two winner marks across the complete table when measurements are available. Each AWS workload has four machine subcolumns separated by vertical rules.

- Light gray identifies every minimap2 baseline row. Pale yellow identifies the preferred BioseqDB configuration: MPF / Bounded / SIMD-DP / Block. This is the author's selected configuration for comparison with the external aligners. The yellow tint does not assert that it wins every workload. Blue timing highlights identify the measured column winners and remain visible within either row tint.

- A dot means there is no approved observation in the snapshot. A horizontal dash with ^{g} identifies an explicitly recorded unavailable GPU configuration, which was not executed. Neither marker is a zero duration. The comparison reads n/a when no measured counterpart exists. In the local and AWS tables, only execution rows with an approved observation or an explicit unavailable cell in a panel are printed. Entirely unmeasured row families remain omitted.

Limits of the comparison. These observations were accumulated across builds. Some retain an older sealed minimap2 correctness control, which is not a new contemporaneous performance run. A correctness-only index control is never substituted for a no-index performance baseline. Comments in the source authorities also disclose missing process-priority attestation, recovery after post-measurement wrapper failures and historical callback-lifetime caveats. The ^{p} marker identifies the local observation retained by an explicit preservation decision without a run receipt. Approval of its measurement does not certify the safety of its historical build. No uniform elevated-priority or noise-free execution claim is made.

The remote C7G observations include severe paging, and other observations also report resource pressure. Cross-machine differences therefore combine hardware, resource availability and execution conditions. They are not an isolated test of instruction-set performance. The table records physical RAM and provisioned storage capacity. Selected row comments contain sampled available memory, but there is no uniform per-repeat series of free-memory and free-disk measurements. Missing C7G BioseqDB timings cannot establish either a win or a loss.

Matched configuration comparisons. The interpretation compares configurations within a machine class, workload and index mode. Its chaining summary uses indexed workloads, holds MPF, bounded minimizers, Block and DNA4 fixed, and requires a selected five-repeat observation for all three chaining choices. Its index-layout summary holds bounded minimizers and DNA4 fixed and pairs MPF with MPS at the same chaining and base-alignment settings. The base-aligner summary pairs Block with KSW2 for the same MPF DNA4 workload, CPU minimizer and chaining choice. Both sides of a pair must have five repeats and an exact requested CPU path. FAST observations and unsupported paths are excluded from these summaries; they remain visible in the source tables and are discussed separately when anomalous. Medians of matched elapsed-time ratios prevent a few extremely long observations from dominating a mean. No timing is discarded merely for being slow. Restricting chaining triples further to observations selected from the same YAML is a sensitivity check, not proof of identical binaries or cache conditions. Local coverage is reported separately from AWS coverage. The companion script `analyze_benchmark_results.py` reproduces these cohorts from the same selected observations and emits the paired values, source-file hashes and summary ratios.

Reproducible table generation. The standalone Python generator reads the normative `approved_results` rows and the remote `hardware_unavailable_cells`, with their embedded machine metadata. The Vast.ai table also reads explicit setup rejections and execution failures, which receive distinct markers rather than timings. It validates identities and timing scopes, rejects duplicates within a source, and records the SHA-256 of each input in the generated LaTeX source. Coverage totals are recomputed from approved rows rather than copied from historical progress summaries. Regeneration is manual and is not part of CMake. The fixture description was checked against the campaign JSON and its sequence generator, together with DOC0432 and DOC0433. The full row-specific qualifications remain in the source YAML authorities. Each cell selects the approved observation with the lowest A across all configured sources. Remote sources are the default summary, TEST3, TEST4, TEST7, TEST2 and FAST. Local sources are the default local summary and FAST LOCAL. The listed source order breaks exact A ties only. Both A and B, the repeat count and the timing scope come from the same selected observation. This reports the best observed timing across the available campaigns, not a pooled median or typical runtime. Selecting a lower timing does not by itself establish that a slower observation suffered swapping. The generator checks that every unique eligible tuple in these sources is rendered once logically, excluding the historical mismeasurements and accounting for repeated baseline rows on continuation pages. Missing fallback files are skipped. This selection also supplies the minimap2 and BioseqDB rows in the external-aligner comparison table. That table additionally reads the four external-aligner authorities and explains their separate timing and validation rules. The final GPU table reads the separate Vast.ai authority, including its own minimap2 baseline, and explains its single-sample procedure.

Table 12.21: Machine inventory captured with the benchmark summaries. Capacities are not free resources at execution time.

Property	Local	M7A	M7I	M4	C7G
Machine	MacBook Pro	m7a.xlarge	m7i.xlarge	m4.xlarge	c7g.xlarge
CPU	Apple M1 Pro	AMD EPYC 9R14	Xeon Platinum 8488C	Xeon E5-2686 v4 @ 2.30GHz	Neoverse-V1
Architecture	arm64	x86_64	x86_64	x86_64	aarch64
OS version / kernel	macOS 15.6 / Darwin 24.6.0	Linux / 7.0.0-1012-aws	Linux / 7.0.0-1012-aws	Linux / 7.0.0-1012-aws	Linux / 7.0.0-1012-aws
OS build / distribution	24G5054d	Distribution not recorded	Distribution not recorded	Distribution not recorded	Distribution not recorded
Logical / physical cores	8 / 8 (6P + 2E)	4 / 4	4 / 2	4 / 2	4 / 4
Physical RAM	16 GiB unified	16 GiB	16 GiB	16 GiB	8 GiB
GPU	Apple M1 Pro, 14 cores	None available	None available	None available	None available
Storage	APPLE SSD AP0512R	EBS gp3	EBS gp3	EBS gp3	EBS gp3
Disk capacity	500.28 GB	500 GiB	500 GiB	500 GiB	500 GiB
Provisioned disk I/O	Not recorded	6,000 IOPS / 500 MiB/s	6,000 IOPS / 500 MiB/s	6,000 IOPS / 500 MiB/s	6,000 IOPS / 500 MiB/s
Free disk / free RAM	No uniform per-repeat record	No uniform per-repeat record	No uniform per-repeat record	No uniform per-repeat record	No uniform per-repeat record

The AWS summaries record the Linux kernel but do not seal an OS distribution version. The local legacy capture does not seal the MoltenVK implementation identity. Unrecorded values are not inferred from the current machine.

Each cell selects the approved observation with the lowest A across its configured sources. Remote sources are the default summary, TEST3, TEST4, TEST7, TEST2 and FAST. Local sources are the default summary and FAST LOCAL. This source order breaks exact A ties only. B, repeat count and timing scope come from the same selected observation. These are best observed timings across campaigns, not estimates of typical execution time. The inventory describes the default hosts. Other measurements may come from different instances of the same machine class.

Table 12.22: Published snapshot coverage after excluding mismeasured historical timings.

Host	Logical tuples	Observations	Repeats	BioseqDB obs.	GPU unavailable
Local	355	355	767	339	0
M7A	657	657	1713	641	512
M7I	631	631	1671	615	512
M4	262	262	630	246	512
C7G	254	254	834	238	512

The published BioseqDB campaign snapshot contains 2159 timing observations and 5,615 accepted repeats. FAST entries retain their separate single-sample timing scope and are marked with a superscript s. The 66 historical outer-E2E observations are treated as mismeasurements and excluded from coverage and comparisons. Standard observations have five accepted repeats. FAST observations have one. The planned local matrix has 1,296 logical cells and 6,480 repeats. It remains incomplete. Unavailable-GPU counts refer only to the remote authorities' recorded eligible subsets.

Engine	Execution	Index								No index							
		4 Gbp				2 Gbp				4 Gbp				2 Gbp			
		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15	
		DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	39.7s ▼2.2× 36.8s ▼2.2×	25.2s ▼1.5× 22.7s ▼1.5×	1m 56.9s ▼29.1× 1m 53.2s ▼32.9×	24.9s ▼1.6× 22.2s ▼1.6×	28.6s ▼11.5× 25.8s ▼15.6×	15.3s ▼2.8× 12.7s ▼3.3×	29.3s ▼15.0× 26.2s ▼22.9×	8.7s ▼1.7× 8.1s ▼2.3×	2m 52.4s ▲19% 2m 48.5s ▲15%	2m 17.1s ▲3.9× 2m 15.9s ▲1.7×	2m 55s ▼6% 2m 51s ▼12%	55.9s ▲8.6× 55.1s ▲3.1×	43.3s ▲2.6× 42.5s ▲2.4×	38.9s ▲8.0× 38.2s ▲2.8×	53.1s ▲1.6× 48.2s ▲1.5×	28.7s ▲9.8× 28.2s ▲2.9×
R2 BioseqDB	MPF / Bounded SIMD-DP / Block	17.7s ▲2.2× 17s ▲2.2×	21.5s ▲15% 20.2s ▲11%	14.6s ▲8.0× 13.8s ▲8.2×	21s ▲16% 19.8s ▲11%	2.5s ▲11.5× 1.7s ▲15.6×	5.4s ▲2.8× 4.2s ▲3.1×	2s ▲15.0× 1.3s ▲20.4×	5.3s ▲1.6× 3.9s ▲2.1×	3m 48.1s ▼1.3× 3m 35s ▼28%	8m 57.6s ▼3.9× 3m 49.2s ▼1.7×	2m 52s ▶ 2% 2m 39.3s ▲7%	8m 13.1s ▼8.8× 2m 54.5s ▼3.2×	1m 54.3s ▼2.6× 1m 45.5s ▼2.5×	5m 17.3s ▼8.2× 1m 54.9s ▼3.0×	1m 28.3s ▼1.7× 1m 16.8s ▼1.6×	4m 45.7s ▼9.9× 1m 26.6s ▼3.1×
R3	MPF / Bounded Scalar-DP / KSW2	·	36.5s ▼1.4×[s] 34.8s ▼1.5×[s]	27.1s ▲4.3×[s] 25.6s ▲4.4×[s]	30.3s ▼22%[s] 28.7s ▼29%[s]	·	5.8s ▲2.6×[s] 4.2s ▲3.1×[s]	2.4s ▲12.3× 1.1s ▲22.9×	5.4s ▲1.6×[s] 3.8s ▲2.2×[s]	·	·	·	·	·	·	·	·
R4	MPF / Bounded Scalar-DP / Block	·	34.8s ▼1.4×[s] 33.5s ▼1.5×[s]	26s ▲4.5×[s] 25.4s ▲4.5×[s]	30.7s ▼23%[s] 29.5s ▼1.3×[s]	·	5.4s ▲2.8×[s] 4.3s ▲3.0×[s]	2.4s ▲12.0× 1.9s ▲13.8×	5.1s ▲1.7×[s] 3.9s ▲2.1×[s]	·	·	·	·	·	·	·	·
R5	MPF / Bounded SIMD-DP / KSW2	20.8s ▲1.9× 19.1s ▲1.9×	16.9s ▲1.5×[s] 15.3s ▲1.5×[s]	19.8s ▲5.9× 18.2s ▲6.2×	30.1s ▼21% 28.5s ▼28%	3.7s ▲7.8× 2.4s ▲10.7×	6s ▲2.5× 4.5s ▲2.9×	2.5s ▲11.7× 1.2s ▲22.6×	5s ▲1.7× 3.6s ▲2.3×	3m 32.7s ▼23% 3m 17.7s ▼17%	·	2m 44.9s ▲6% 2m 32.2s ▲11%	·	1m 52.7s ▼2.6× 1m 40s ▼2.4×	5m 12.1s ▼8.0× 1m 46.7s ▼2.8×	1m 26.1s ▼1.6× 1m 13.9s ▼1.5×	4m 41.4s ▼9.8× 1m 21.9s ▼2.9×
R6	MPF / Bounded RMQ / KSW2	·	34.8s ▼1.4×[s] 33.2s ▼1.5×[s]	27.4s ▲4.3×[s] 25.9s ▲4.4×[s]	31.7s ▼27%[s] 30.1s ▼1.4×[s]	·	5.8s ▲2.6×[s] 3.9s ▲3.3×[s]	2.8s ▲10.6×[s] 1.2s ▲21.1×[s]	5.4s ▲1.6×[s] 3.7s ▲2.2×[s]	·	·	·	·	·	·	·	·
R7	MPF / Bounded RMQ / Block	·	34s ▼1.3×[s] 32.7s ▼1.4×[s]	4s ▲29.1×[s] 3.4s ▲32.9×[s]	15.4s ▲1.6×[s] 14.2s ▲1.6×[s]	·	5.9s ▲2.6×[s] 4.6s ▲2.8×[s]	9.5s ▲3.1× 9s ▲2.9×	14s ▼1.6×[s] 12.7s ▼1.6×[s]	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Minimap2 stays first. Other rows follow increasing minima of available Index A times across all local sizes, k/w pairs and alphabets. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index								No index							
		4 Gbp				2 Gbp				4 Gbp				2 Gbp			
		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15	
		DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	39.7s ▼2.2× 36.8s ▼2.2×	25.2s ▼1.5× 22.7s ▼1.5×	1m 56.9s ▼29.1× 1m 53.2s ▼32.9×	24.9s ▼1.6× 22.2s ▼1.6×	28.6s ▼11.5× 25.8s ▼15.6×	15.3s ▼2.8× 12.7s ▼3.3×	29.3s ▼15.0× 26.2s ▼22.9×	8.7s ▼1.7× 8.1s ▼2.3×	2m 52.4s ▲19% 2m 48.5s ▲15%	2m 17.1s ▲3.9× 2m 15.9s ▲1.7×	2m 55s ▼6% 2m 51s ▼12%	55.9s ▲8.6× 55.1s ▲3.1×	43.3s ▲2.6× 42.5s ▲2.4×	38.9s ▲8.0× 38.2s ▲2.8×	53.1s ▲1.6× 48.2s ▲1.5×	28.7s ▲9.8× 28.2s ▲2.9×
R8 BioseqDB	MPS / Bounded Scalar-DP / Block	·	·	·	·	4.1s ▲7.0×[s] 3.2s ▲8.1×[s]	3m 47.5s ▼14.9×[s] 21.8s ▼1.7×[s]	4.5s ▲6.5×[s] 3.7s ▲7.1×[s]	3m 29.2s ▼24.1×[s] 13.3s ▼1.6×[s]	·	·	·	·	·	·	·	·
R9	MPS / Bounded RMQ / Block	·	·	·	·	4.3s ▲6.7×[s] 3.2s ▲8.0×[s]	3m 45.3s ▼14.8×[s] 21.8s ▼1.7×[s]	4.4s ▲6.6×[s] 3.8s ▲7.0×[s]	3m 30s ▼24.2×[s] 13.4s ▼1.6×[s]	·	·	·	·	·	·	·	·
R10	MPS / Bounded SIMD-DP / Block	·	·	·	·	4.3s ▲6.7×[s] 3.3s ▲7.9×[s]	3m 45.2s ▼14.7×[s] 22.3s ▼1.8×[s]	4.5s ▲6.6× 3.8s ▲6.9×	3m 27.4s ▼23.9×[s] 14.1s ▼1.7×[s]	3m 53.6s ▼1.4× 3m 40.3s ▼1.3×	8m 50.7s ▼3.9× 3m 46s ▼1.7×	2m 51.3s ▶2% 2m 38.1s ▲8%	7m 58.5s ▼8.6× 2m 48.9s ▼3.1×	1m 59.6s ▼2.8× 1m 49s ▼2.6×	5m 15.7s ▼8.1× 1m 54.7s ▼3.0×	1m 28s ▼1.7× 1m 18.3s ▼1.6×	4m 46.9s ▼10.0× 1m 25.3s ▼3.0×
R11	MPS / Two-stack Scalar-DP / Block	·	·	·	·	4.3s ▲6.6×[s] 3.4s ▲7.7×[s]	3m 32.4s ▼13.9×[s] 16.6s ▼1.3×[s]	11.8s ▲2.5×[s] 4.5s ▲5.8×[s]	3m 21.7s ▼23.2×[s] 8.6s ▼5%[s]	·	·	·	·	·	·	·	·
R12	MPS / Two-stack RMQ / Block	·	·	·	·	4.3s ▲6.6×[s] 3.4s ▲7.7×[s]	3m 35.1s ▼14.1×[s] 17.4s ▼1.4×[s]	11.8s ▲2.5×[s] 4.6s ▲5.7×[s]	3m 22s ▼23.3×[s] 8.6s ▼6%[s]	·	·	·	·	·	·	·	·
R13	MPS / SIMD-ref RMQ / Block	·	·	·	·	4.3s ▲6.6×[s] 3.5s ▲7.4×[s]	3m 51.3s ▼15.1×[s] 27.3s ▼2.1×[s]	12.9s ▲2.3×[s] 5.8s ▲4.5×[s]	3m 38.6s ▼25.2×[s] 20.6s ▼2.5×[s]	·	·	·	·	·	·	·	·
R14	MPS / SIMD-ref Scalar-DP / Block	·	·	·	·	4.4s ▲6.6×[s] 3.5s ▲7.5×[s]	3m 50.5s ▼15.1×[s] 26.1s ▼2.1×[s]	12.5s ▲2.3×[s] 5.6s ▲4.7×[s]	3m 39.6s ▼25.3×[s] 20.2s ▼2.5×[s]	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Minimap2 stays first. Other rows follow increasing minima of available Index A times across all local sizes, k/w pairs and alphabets. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index								No index							
		4 Gbp				2 Gbp				4 Gbp				2 Gbp			
		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15	
		DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	39.7s ▼2.2× 36.8s ▼2.2×	25.2s ▼1.5× 22.7s ▼1.5×	1m 56.9s ▼29.1× 1m 53.2s ▼32.9×	24.9s ▼1.6× 22.2s ▼1.6×	28.6s ▼11.5× 25.8s ▼15.6×	15.3s ▼2.8× 12.7s ▼3.3×	29.3s ▼15.0× 26.2s ▼22.9×	8.7s ▼1.7× 8.1s ▼2.3×	2m 52.4s ▲19% 2m 48.5s ▲15%	2m 17.1s ▲3.9× 2m 15.9s ▲1.7×	2m 55s ▼6% 2m 51s ▼12%	55.9s ▲8.6× 55.1s ▲3.1×	43.3s ▲2.6× 42.5s ▲2.4×	38.9s ▲8.0× 38.2s ▲2.8×	53.1s ▲1.6× 48.2s ▲1.5×	28.7s ▲9.8× 28.2s ▲2.9×
R15 BioseqDB	MPS / Scalar SIMD-DP / Block	·	·	·	·	4.4s ▲6.6×[s] 3.5s ▲7.5×[s]	3m 46.2s ▼14.8×[s] 22.6s ▼1.8×[s]	13.3s ▲2.2×[s] 5.5s ▲4.7×[s]	3m 35s ▼24.8×[s] 16.3s ▼2.0×[s]	·	·	·	·	·	·	·	·
R16	MPS / SIMD-ref SIMD-DP / Block	·	·	·	·	4.4s ▲6.6×[s] 3.5s ▲7.4×[s]	3m 53.3s ▼15.3×[s] 27s ▼2.1×[s]	12.7s ▲2.3×[s] 5.6s ▲4.7×[s]	3m 41.1s ▼25.5×[s] 21s ▼2.6×[s]	·	·	·	·	·	·	·	·
R17	MPS / Two-stack SIMD-DP / Block	·	·	·	·	4.4s ▲6.5×[s] 3.5s ▲7.5×[s]	3m 39s ▼14.3×[s] 18.2s ▼1.4×[s]	13.3s ▲2.2×[s] 4.5s ▲5.9×[s]	3m 26s ▼23.7×[s] 9.6s ▼18%[s]	·	·	·	·	·	·	·	·
R18	MPS / Scalar Scalar-DP / Block	·	·	·	·	4.4s ▲6.5×[s] 3.5s ▲7.5×[s]	3m 47.4s ▼14.9×[s] 22.4s ▼1.8×[s]	12.5s ▲2.4×[s] 5.6s ▲4.7×[s]	3m 34.3s ▼24.7×[s] 14.9s ▼1.8×[s]	·	·	·	·	·	·	·	·
R19	MPS / Scalar RMQ / Block	·	·	·	·	4.4s ▲6.5×[s] 3.5s ▲7.5×[s]	3m 43.8s ▼14.6×[s] 21.9s ▼1.7×[s]	12.9s ▲2.3×[s] 5.8s ▲4.5×[s]	3m 35.1s ▼24.8×[s] 15.1s ▼1.9×[s]	·	·	·	·	·	·	·	·
R20	MPS / Bounded Scalar-DP / KSW2	·	·	·	·	5s ▲5.7×[s] 3.2s ▲8.1×[s]	3m 48.5s ▼15.0×[s] 22s ▼1.7×[s]	5.5s ▲5.3× 3.9s ▲6.7×	3m 32s ▼24.4×[s] 13.3s ▼1.6×[s]	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Minimap2 stays first. Other rows follow increasing minima of available Index A times across all local sizes, k/w pairs and alphabets. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index								No index							
		4 Gbp				2 Gbp				4 Gbp				2 Gbp			
		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15	
		DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	39.7s ▼2.2× 36.8s ▼2.2×	25.2s ▼1.5× 22.7s ▼1.5×	1m 56.9s ▼29.1× 1m 53.2s ▼32.9×	24.9s ▼1.6× 22.2s ▼1.6×	28.6s ▼11.5× 25.8s ▼15.6×	15.3s ▼2.8× 12.7s ▼3.3×	29.3s ▼15.0× 26.2s ▼22.9×	8.7s ▼1.7× 8.1s ▼2.3×	2m 52.4s ▲19% 2m 48.5s ▲15%	2m 17.1s ▲3.9× 2m 15.9s ▲1.7×	2m 55s ▼6% 2m 51s ▼12%	55.9s ▲8.6× 55.1s ▲3.1×	43.3s ▲2.6× 42.5s ▲2.4×	38.9s ▲8.0× 38.2s ▲2.8×	53.1s ▲1.6× 48.2s ▲1.5×	28.7s ▲9.8× 28.2s ▲2.9×
R21 BioseqDB	MPS / Bounded RMQ / KSW2	·	·	·	·	5.1s ▲5.6×[s] 3.2s ▲8.1×[s]	3m 47.8s ▼14.9×[s] 22.2s ▼1.7×[s]	5.3s ▲5.6×[s] 3.7s ▲7.1×[s]	3m 31.9s ▼24.4×[s] 13.4s ▼1.6×[s]	·	·	·	·	·	·	·	·
R22	MPS / SIMD-ref Scalar-DP / KSW2	·	·	·	·	5.1s ▲5.6×[s] 3.4s ▲7.5×[s]	3m 51.5s ▼15.2×[s] 25.9s ▼2.0×[s]	14.7s ▲2.0×[s] 5.7s ▲4.6×[s]	3m 44.9s ▼25.9×[s] 21.1s ▼2.6×[s]	·	·	·	·	·	·	·	·
R23	MPS / Bounded SIMD-DP / KSW2	·	·	·	·	5.2s ▲5.5× 3.2s ▲8.1×	3m 47s ▼14.9×[s] 21.5s ▼1.7×[s]	5.3s ▲5.5× 3.8s ▲7.0×	3m 32.1s ▼24.4×[s] 13.2s ▼1.6×[s]	3m 54s ▼1.4× 3m 39s ▼30%	·	2m 52.6s ▶1% 2m 36.8s ▲8%	·	2m 3.1s ▼2.8× 1m 50.4s ▼2.6×	5m 13.3s ▼8.1× 1m 50.9s ▼2.9×	1m 30.6s ▼1.7× 1m 17.7s ▼1.6×	4m 44.7s ▼9.9× 1m 25.3s ▼3.0×
R24	MPS / Two-stack RMQ / KSW2	·	·	·	·	5.2s ▲5.5×[s] 3.4s ▲7.6×[s]	3m 34.7s ▼14.1×[s] 16.7s ▼1.3×[s]	16.7s ▲1.8×[s] 4.6s ▲5.7×[s]	3m 27.7s ▼23.9×[s] 8.4s ▶3%[s]	·	·	·	·	·	·	·	·
R25	MPS / Two-stack Scalar-DP / KSW2	·	·	·	·	5.2s ▲5.5×[s] 3.4s ▲7.6×[s]	3m 34.2s ▼14.0×[s] 16.5s ▼30%[s]	21.8s ▲26% 4.8s ▲5.5×	3m 28.2s ▼24.0×[s] 8.6s ▼6%[s]	·	·	·	·	·	·	·	·
R26	MPS / Two-stack SIMD-DP / KSW2	·	·	·	·	5.2s ▲5.5×[s] 3.4s ▲7.6×[s]	3m 40.4s ▼14.4×[s] 17.5s ▼1.4×[s]	16.6s ▲1.8×[s] 4.5s ▲5.8×[s]	3m 28s ▼24.0×[s] 8.5s ▼5%[s]	·	·	·	·	·	·	·	·
R27	MPS / SIMD-ref RMQ / KSW2	·	·	·	·	5.2s ▲5.5×[s] 3.5s ▲7.3×[s]	3m 52.6s ▼15.2×[s] 27.3s ▼2.2×[s]	14.8s ▲2.0×[s] 5.7s ▲4.6×[s]	3m 42.9s ▼25.7×[s] 20.5s ▼2.5×[s]	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Minimap2 stays first. Other rows follow increasing minima of available Index A times across all local sizes, k/w pairs and alphabets. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index								No index							
		4 Gbp				2 Gbp				4 Gbp				2 Gbp			
		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15	
		DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	39.7s ▼2.2× 36.8s ▼2.2×	25.2s ▼1.5× 22.7s ▼1.5×	1m 56.9s ▼29.1× 1m 53.2s ▼32.9×	24.9s ▼1.6× 22.2s ▼1.6×	28.6s ▼11.5× 25.8s ▼15.6×	15.3s ▼2.8× 12.7s ▼3.3×	29.3s ▼15.0× 26.2s ▼22.9×	8.7s ▼1.7× 8.1s ▼2.3×	2m 52.4s ▲19% 2m 48.5s ▲15%	2m 17.1s ▲3.9× 2m 15.9s ▲1.7×	2m 55s ▼6% 2m 51s ▼12%	55.9s ▲8.6× 55.1s ▲3.1×	43.3s ▲2.6× 42.5s ▲2.4×	38.9s ▲8.0× 38.2s ▲2.8×	53.1s ▲1.6× 48.2s ▲1.5×	28.7s ▲9.8× 28.2s ▲2.9×
R28 BioseqDB	MPS / Scalar SIMD-DP / KSW2	·	·	·	·	5.3s ▲5.4×[s] 3.4s ▲7.6×[s]	3m 46.1s ▼14.8×[s] 21.5s ▼1.7×[s]	17.6s ▲1.7×[s] 5.5s ▲4.7×[s]	3m 36.6s ▼24.9×[s] 15.3s ▼1.9×[s]	·	·	·	·	·	·	·	·
R29	MPS / SIMD-ref SIMD-DP / KSW2	·	·	·	·	5.3s ▲5.4×[s] 3.5s ▲7.4×[s]	3m 52.8s ▼15.2×[s] 26.4s ▼2.1×[s]	14.7s ▲2.0×[s] 5.6s ▲4.7×[s]	3m 42.9s ▼25.7×[s] 20.3s ▼2.5×[s]	·	·	·	·	·	·	·	·
R30	MPS / Scalar RMQ / KSW2	·	·	·	·	5.3s ▲5.4×[s] 3.4s ▲7.6×[s]	3m 49.4s ▼15.0×[s] 21.8s ▼1.7×[s]	14.6s ▲2.0×[s] 5.7s ▲4.6×[s]	3m 35.5s ▼24.8×[s] 15.1s ▼1.9×[s]	·	·	·	·	·	·	·	·
R31	MPS / Scalar Scalar-DP / KSW2	·	·	·	·	5.3s ▲5.4×[s] 3.4s ▲7.6×[s]	3m 47s ▼14.9×[s] 22s ▼1.7×[s]	16.3s ▲1.8×[s] 5.6s ▲4.7×[s]	3m 35.8s ▼24.9×[s] 15s ▼1.8×[s]	·	·	·	·	·	·	·	·
R32	MPF / Two-stack SIMD-DP / Block	·	·	·	·	·	5.4s ▲2.8×[s] 4.2s ▲3.0×[s]	1m 45s ▼3.6× 1m 44.4s ▼4.0×	59.1s ▼6.8×[s] 58.5s ▼7.2×[s]	·	·	·	·	·	·	·	·
R33	MPF / Scalar SIMD-DP / Block	·	·	·	·	·	6.1s ▲2.5×[s] 4.7s ▲2.7×[s]	1m 52.3s ▼3.8× 1m 51.9s ▼4.3×	3m 2.6s ▼21.0×[s] 3m 1.6s ▼22.3×[s]	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Minimap2 stays first. Other rows follow increasing minima of available Index A times across all local sizes, k/w pairs and alphabets. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index								No index							
		4 Gbp				2 Gbp				4 Gbp				2 Gbp			
		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15	
		DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	39.7s ▼2.2× 36.8s ▼2.2×	25.2s ▼1.5× 22.7s ▼1.5×	1m 56.9s ▼29.1× 1m 53.2s ▼32.9×	24.9s ▼1.6× 22.2s ▼1.6×	28.6s ▼11.5× 25.8s ▼15.6×	15.3s ▼2.8× 12.7s ▼3.3×	29.3s ▼15.0× 26.2s ▼22.9×	8.7s ▼1.7× 8.1s ▼2.3×	2m 52.4s ▲19% 2m 48.5s ▲15%	2m 17.1s ▲3.9× 2m 15.9s ▲1.7×	2m 55s ▼6% 2m 51s ▼12%	55.9s ▲8.6× 55.1s ▲3.1×	43.3s ▲2.6× 42.5s ▲2.4×	38.9s ▲8.0× 38.2s ▲2.8×	53.1s ▲1.6× 48.2s ▲1.5×	28.7s ▲9.8× 28.2s ▲2.9×
R34 BioseqDB	MPF / Scalar RMQ / Block	·	·	·	·	·	8s ▲1.9×[s] 6.7s ▲1.9×[s]	1m 50s ▼3.7× 1m 49.6s ▼4.2×	3m 3.4s ▼21.1×[s] 3m 2.4s ▼22.4×[s]	·	·	·	·	·	·	·	·
R35	MPF / Scalar Scalar-DP / KSW2	·	·	·	·	·	8.5s ▲1.8×[s] 7s ▲1.8×[s]	1m 43.2s ▼3.5×[s] 1m 41.9s ▼3.9×[s]	3m 8.6s ▼21.7×[s] 3m 7.3s ▼23.0×[s]	·	·	·	·	·	·	·	·
R36	MPF / Scalar SIMD-DP / KSW2	·	·	·	·	·	9s ▲1.7×[s] 7.5s ▲1.7×[s]	1m 43s ▼3.5×[s] 1m 41.7s ▼3.9×[s]	3m 8.9s ▼21.8×[s] 3m 7.6s ▼23.1×[s]	·	·	·	·	·	·	·	·
R37	MPF / Scalar Scalar-DP / Block	·	·	·	·	·	10.1s ▲1.5×[s] 8.9s ▲30%[s]	1m 48.8s ▼3.7× 1m 48.4s ▼4.1×	3m 4.8s ▼21.3×[s] 3m 3.8s ▼22.6×[s]	·	·	·	·	·	·	·	·
R38	MPF / Two-stack SIMD-DP / KSW2	·	·	·	·	·	10.4s ▲1.5×[s] 8.8s ▲1.4×[s]	1m 21.3s ▼2.8× 1m 19.5s ▼3.0×	1m 1.1s ▼7.0×[s] 58.6s ▼7.2×[s]	·	·	·	·	·	·	·	·
R39	MPF / SIMD-ref RMQ / Block	·	·	·	·	·	10.5s ▲1.4×[s] 9.3s ▲26%[s]	1m 40.4s ▼3.4× 1m 40s ▼3.8×	1m 36.1s ▼11.1×[s] 1m 35.1s ▼11.7×[s]	·	·	·	·	·	·	·	·
R40	MPF / SIMD-ref SIMD-DP / Block	·	·	·	·	·	10.8s ▲29%[s] 9.6s ▲25%[s]	1m 42.9s ▼3.5× 1m 42.6s ▼3.9×	1m 57.8s ▼13.6×[s] 1m 56.7s ▼14.4×[s]	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Minimap2 stays first. Other rows follow increasing minima of available Index A times across all local sizes, k/w pairs and alphabets. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index								No index							
		4 Gbp				2 Gbp				4 Gbp				2 Gbp			
		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15	
		DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	39.7s ▼2.2× 36.8s ▼2.2×	25.2s ▼1.5× 22.7s ▼1.5×	1m 56.9s ▼29.1× 1m 53.2s ▼32.9×	24.9s ▼1.6× 22.2s ▼1.6×	28.6s ▼11.5× 25.8s ▼15.6×	15.3s ▼2.8× 12.7s ▼3.3×	29.3s ▼15.0× 26.2s ▼22.9×	8.7s ▼1.7× 8.1s ▼2.3×	2m 52.4s ▲19% 2m 48.5s ▲15%	2m 17.1s ▲3.9× 2m 15.9s ▲1.7×	2m 55s ▼6% 2m 51s ▼12%	55.9s ▲8.6× 55.1s ▲3.1×	43.3s ▲2.6× 42.5s ▲2.4×	38.9s ▲8.0× 38.2s ▲2.8×	53.1s ▲1.6× 48.2s ▲1.5×	28.7s ▲9.8× 28.2s ▲2.9×
R41 BioseqDB	MPF / SIMD-ref SIMD-DP / KSW2	·	·	·	·	·	10.9s ▲29%[s] 9.3s ▲27%[s]	1m 44.7s ▼3.6×[s] 1m 43.3s ▼3.9×[s]	1m 37.2s ▼11.2×[s] 1m 35.9s ▼11.8×[s]	·	·	·	·	·	·	·	·
R42	MPF / Two-stack Scalar-DP / Block	·	·	·	·	·	10.9s ▲28%[s] 9.7s ▲24%[s]	1m 21s ▼2.8× 1m 20.6s ▼3.1×	57.4s ▼6.6×[s] 56.7s ▼7.0×[s]	·	·	·	·	·	·	·	·
R43	MPF / Two-stack Scalar-DP / KSW2	·	·	·	·	·	11.4s ▲25%[s] 9.9s ▲22%[s]	1m 21.1s ▼2.8× 1m 19.3s ▼3.0×	58.7s ▼6.8×[s] 56.2s ▼6.9×[s]	·	·	·	·	·	·	·	·
R44	MPF / Scalar RMQ / KSW2	·	·	·	·	·	12s ▲22%[s] 10.5s ▲17%[s]	1m 44.5s ▼3.6×[s] 1m 43.2s ▼3.9×[s]	3m 0.8s ▼20.8×[s] 2m 59.5s ▼22.1×[s]	·	·	·	·	·	·	·	·
R45	MPF / Two-stack RMQ / KSW2	·	·	·	·	·	12.4s ▲19%[s] 10.8s ▲15%[s]	1m 12.2s ▼2.5× 1m 10.5s ▼2.7×	59.7s ▼6.9×[s] 57.2s ▼7.0×[s]	·	·	·	·	·	·	·	·
R46	MPF / Two-stack RMQ / Block	·	·	·	·	·	12.7s ▲17%[s] 11.5s ▲10%[s]	1m 35.2s ▼3.2× 1m 34.5s ▼3.6×	58s ▼6.7×[s] 57.2s ▼7.0×[s]	·	·	·	·	·	·	·	·
R47	MPS / Two-stack Vulkan-DP / Block	·	·	·	·	26.2s ▲8%[s] 19s ▲26%[s]	3m 49.5s ▼15.0×[s] 33.7s ▼2.6×[s]	13.1s ▲2.2×[s] 5.8s ▲4.5×[s]	3m 23.1s ▼23.4×[s] 9.9s ▼22%[s]	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Minimap2 stays first. Other rows follow increasing minima of available Index A times across all local sizes, k/w pairs and alphabets. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index								No index							
		4 Gbp				2 Gbp				4 Gbp				2 Gbp			
		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15	
		DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	39.7s ▼2.2× 36.8s ▼2.2×	25.2s ▼1.5× 22.7s ▼1.5×	1m 56.9s ▼29.1× 1m 53.2s ▼32.9×	24.9s ▼1.6× 22.2s ▼1.6×	28.6s ▼11.5× 25.8s ▼15.6×	15.3s ▼2.8× 12.7s ▼3.3×	29.3s ▼15.0× 26.2s ▼22.9×	8.7s ▼1.7× 8.1s ▼2.3×	2m 52.4s ▲19% 2m 48.5s ▲15%	2m 17.1s ▲3.9× 2m 15.9s ▲1.7×	2m 55s ▼6% 2m 51s ▼12%	55.9s ▲8.6× 55.1s ▲3.1×	43.3s ▲2.6× 42.5s ▲2.4×	38.9s ▲8.0× 38.2s ▲2.8×	53.1s ▲1.6× 48.2s ▲1.5×	28.7s ▲9.8× 28.2s ▲2.9×
R48 BioseqDB	MPS / Two-stack Vulkan-DP / KSW2	·	·	·	·	27.6s ▶ 4%[s] 18.6s ▲28%[s]	3m 51.3s ▼15.1×[s] 33.5s ▼2.6×[s]	18.1s ▲1.6×[s] 5.8s ▲4.5×[s]	3m 29.5s ▼24.1×[s] 9.9s ▼21%[s]	·	·	·	·	·	·	·	·
R49	MPS / Bounded Vulkan-DP / Block	·	·	·	·	25.5s ▲11%[s] 18.2s ▲29%[s]	4m 7.7s ▼16.2×[s] 43.2s ▼3.4×[s]	19.8s ▲1.5×[s] 12.9s ▲2.0×[s]	3m 47.2s ▼26.2×[s] 29s ▼3.6×[s]	·	·	·	·	·	·	·	·
R50	MPS / Scalar Vulkan-DP / Block	·	·	·	·	28.8s ▶ 1%[s] 18.6s ▲28%[s]	4m 7s ▼16.2×[s] 44.1s ▼3.5×[s]	20.9s ▲29%[s] 13.8s ▲1.9×[s]	3m 50.9s ▼26.6×[s] 31.3s ▼3.8×[s]	·	·	·	·	·	·	·	·
R51	MPS / SIMD-ref Vulkan-DP / Block	·	·	·	·	25.2s ▲12%[s] 17.9s ▲1.4×[s]	3m 59.3s ▼15.7×[s] 35.3s ▼2.8×[s]	21.4s ▲27%[s] 14.2s ▲1.8×[s]	3m 57.1s ▼27.3×[s] 37.5s ▼4.6×[s]	·	·	·	·	·	·	·	·
R52	MPS / Bounded Vulkan-DP / KSW2	·	·	·	·	28.4s ▶ 0%[s] 19.2s ▲26%[s]	4m 4.9s ▼16.0×[s] 42.5s ▼3.3×[s]	23s ▲21%[s] 14s ▲1.9×[s]	3m 50.4s ▼26.5×[s] 28.8s ▼3.5×[s]	·	·	·	·	·	·	·	·
R53	MPS / SIMD-ref Vulkan-DP / KSW2	·	·	·	·	28.4s ▶ 0%[s] 18.2s ▲30%[s]	4m 1.2s ▼15.8×[s] 35.5s ▼2.8×[s]	23.8s ▲19%[s] 14.5s ▲1.8×[s]	3m 57.5s ▼27.4×[s] 37.7s ▼4.6×[s]	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Minimap2 stays first. Other rows follow increasing minima of available Index A times across all local sizes, k/w pairs and alphabets. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index								No index							
		4 Gbp				2 Gbp				4 Gbp				2 Gbp			
		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15	
		DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	39.7s ▼2.2× 36.8s ▼2.2×	25.2s ▼1.5× 22.7s ▼1.5×	1m 56.9s ▼29.1× 1m 53.2s ▼32.9×	24.9s ▼1.6× 22.2s ▼1.6×	28.6s ▼11.5× 25.8s ▼15.6×	15.3s ▼2.8× 12.7s ▼3.3×	29.3s ▼15.0× 26.2s ▼22.9×	8.7s ▼1.7× 8.1s ▼2.3×	2m 52.4s ▲19% 2m 48.5s ▲15%	2m 17.1s ▲3.9× 2m 15.9s ▲1.7×	2m 55s ▼6% 2m 51s ▼12%	55.9s ▲8.6× 55.1s ▲3.1×	43.3s ▲2.6× 42.5s ▲2.4×	38.9s ▲8.0× 38.2s ▲2.8×	53.1s ▲1.6× 48.2s ▲1.5×	28.7s ▲9.8× 28.2s ▲2.9×
R54 BioseqDB	MPS / Scalar Vulkan-DP / KSW2	·	·	·	·	31s ▼9%[s] 18.4s ▲29%[s]	4m 7.9s ▼16.2×[s] 43.3s ▼3.4×[s]	24.2s ▲18%[s] 15s ▲1.7×[s]	3m 55.7s ▼27.1×[s] 32.7s ▼4.0×[s]	·	·	·	·	·	·	·	·
R55	MPF / Bounded Vulkan-DP / KSW2	57.1s ▼1.4×[s] 40.2s ▼9%[s]	5m 53.3s ▼14.0×[s] 51.2s ▼2.3×[s]	1m 3.5s ▲1.8×[s] 46.7s ▲2.4×[s]	6m 14.8s ▼15.1×[s] 1m 3.1s ▼2.8×[s]	42s ▼1.5×[s] 27.9s ▼8%[s]	3m 59.2s ▼15.7×[s] 30.8s ▼2.4×[s]	32.3s ▼10%[s] 17.8s ▲1.5×[s]	3m 47.2s ▼26.2×[s] 23.7s ▼2.9×[s]	·	·	·	·	·	·	·	·
R56	MPF / Bounded Vulkan-DP / Block	44.5s ▼12%[s] 29.5s ▲20%[s]	6m 10.7s ▼14.7×[s] 1m 8.5s ▼3.0×[s]	1m 0.8s ▲1.9×[s] 45.9s ▲2.5×[s]	6m 29.4s ▼15.6×[s] 1m 21.6s ▼3.7×[s]	37.9s ▼1.3×[s] 25.3s ▶ 2%[s]	4m 4.2s ▼16.0×[s] 38.6s ▼3.0×[s]	37s ▼26%[s] 24.7s ▲6%[s]	3m 48.2s ▼26.3×[s] 23.4s ▼2.9×[s]	·	·	·	·	·	·	·	·
R57	MPF / SIMD-ref Vulkan-DP / Block	·	·	·	·	39s ▼1.4×[s] 26.4s ▶ 2%[s]	3m 49.5s ▼15.0×[s] 24s ▼1.9×[s]	13m 49.2s ▼28.3×[s] 13m 37.6s ▼31.2×[s]	14m 54.7s ▼103.0×[s] 11m 32.8s ▼85.2×[s]	·	·	·	·	·	·	·	·
R58	MPF / Scalar Vulkan-DP / Block	43.3s ▼9%[s] 28.9s ▲21%[s]	·	·	·	39s ▼1.4×[s] 26.5s ▶ 3%[s]	4m 5.4s ▼16.1×[s] 39.5s ▼3.1×[s]	12m 37.6s ▼25.8× 12m 26.5s ▼28.5×	27m 27.3s ▼189.7×[s] 24m 4.7s ▼177.7×[s]	·	·	·	·	·	·	·	·
R59	MPF / Two-stack Vulkan-DP / KSW2	·	·	·	·	40.9s ▼1.4×[s] 27.2s ▼5%[s]	4m 6.8s ▼16.2×[s] 36.8s ▼2.9×[s]	13m 56.9s ▼28.5× 13m 25.1s ▼30.7×	10m 19.9s ▼71.4×[s] 6m 50.6s ▼50.5×[s]	·	·	·	·	·	·	·	·
R60	MPF / Scalar Vulkan-DP / KSW2	59.4s ▼1.5×[s] 42s ▼14%[s]	·	·	·	40.9s ▼1.4×[s] 26.8s ▶ 4%[s]	3m 59.5s ▼15.7×[s] 31.6s ▼2.5×[s]	13m 53.7s ▼28.4×[s] 13m 39.4s ▼31.3×[s]	·	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Minimap2 stays first. Other rows follow increasing minima of available Index A times across all local sizes, k/w pairs and alphabets. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index								No index							
		4 Gbp				2 Gbp				4 Gbp				2 Gbp			
		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15	
		DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	39.7s ▼2.2× 36.8s ▼2.2×	25.2s ▼1.5× 22.7s ▼1.5×	1m 56.9s ▼29.1× 1m 53.2s ▼32.9×	24.9s ▼1.6× 22.2s ▼1.6×	28.6s ▼11.5× 25.8s ▼15.6×	15.3s ▼2.8× 12.7s ▼3.3×	29.3s ▼15.0× 26.2s ▼22.9×	8.7s ▼1.7× 8.1s ▼2.3×	2m 52.4s ▲19% 2m 48.5s ▲15%	2m 17.1s ▲3.9× 2m 15.9s ▲1.7×	2m 55s ▼6% 2m 51s ▼12%	55.9s ▲8.6× 55.1s ▲3.1×	43.3s ▲2.6× 42.5s ▲2.4×	38.9s ▲8.0× 38.2s ▲2.8×	53.1s ▲1.6× 48.2s ▲1.5×	28.7s ▲9.8× 28.2s ▲2.9×
R61 BioseqDB	MPF / SIMD-ref Vulkan-DP / KSW2	·	·	·	·	41.8s ▼1.5×[s] 27.5s ▼6%[s]	3m 50.7s ▼15.1×[s] 23.9s ▼1.9×[s]	13m 53s ▼28.4×[s] 13m 39.1s ▼31.2×[s]	15m 18.3s ▼105.7×[s] 11m 51.5s ▼87.5×[s]	·	·	·	·	·	·	·	·
R62	MPF / Two-stack Vulkan-DP / Block	·	·	·	·	43.6s ▼1.5×[s] 31.6s ▼22%[s]	4m 5.1s ▼16.0×[s] 36.3s ▼2.9×[s]	12m 53.5s ▼26.4× 12m 25.3s ▼28.4×	10m 19.7s ▼71.4×[s] 6m 54.6s ▼51.0×[s]	·	·	·	·	·	·	·	·
R63	MPF / SIMD-ref Scalar-DP / Block	·	·	·	·	·	·	1m 34.2s ▼3.2× 1m 34s ▼3.6×	1m 37.5s ▼11.2×[s] 1m 36.6s ▼11.9×[s]	·	·	·	·	·	·	·	·
R64	MPF / SIMD-ref RMQ / KSW2	·	·	·	·	·	·	1m 44s ▼3.5×[s] 1m 42.7s ▼3.9×[s]	1m 34.8s ▼10.9×[s] 1m 33.5s ▼11.5×[s]	·	·	·	·	·	·	·	·
R65	MPF / SIMD-ref Scalar-DP / KSW2	·	·	·	·	·	·	1m 44.3s ▼3.6×[s] 1m 43s ▼3.9×[s]	1m 39.6s ▼11.5×[s] 1m 38.4s ▼12.1×[s]	·	·	·	·	·	·	·	·
R66	MPF / Vulkan RMQ / Block	·	·	·	·	5m 41.1s ▼11.9×[s] 5m 31.7s ▼12.8×[s]	8m 50.1s ▼34.7×[s] 5m 27s ▼25.7×[s]	3m 9.8s ▼6.5×[s] 3m 1.4s ▼6.9×[s]	6m 26.4s ▼44.5×[s] 3m 9.2s ▼23.3×[s]	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Minimap2 stays first. Other rows follow increasing minima of available Index A times across all local sizes, k/w pairs and alphabets. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index								No index							
		4 Gbp				2 Gbp				4 Gbp				2 Gbp			
		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15	
		DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	39.7s ▼2.2× 36.8s ▼2.2×	25.2s ▼1.5× 22.7s ▼1.5×	1m 56.9s ▼29.1× 1m 53.2s ▼32.9×	24.9s ▼1.6× 22.2s ▼1.6×	28.6s ▼11.5× 25.8s ▼15.6×	15.3s ▼2.8× 12.7s ▼3.3×	29.3s ▼15.0× 26.2s ▼22.9×	8.7s ▼1.7× 8.1s ▼2.3×	2m 52.4s ▲19% 2m 48.5s ▲15%	2m 17.1s ▲3.9× 2m 15.9s ▲1.7×	2m 55s ▼6% 2m 51s ▼12%	55.9s ▲8.6× 55.1s ▲3.1×	43.3s ▲2.6× 42.5s ▲2.4×	38.9s ▲8.0× 38.2s ▲2.8×	53.1s ▲1.6× 48.2s ▲1.5×	28.7s ▲9.8× 28.2s ▲2.9×
R67 BioseqDB	MPF / Vulkan Vulkan-DP / Block	9m 36.5s ▼14.5× 9m 22.3s ▼15.3×	13m 49.8s ▼32.9× 8m 45.2s ▼23.2×	4m 57.7s ▼2.5× 4m 42.7s ▼2.5×	10m 50.3s ▼26.1× 5m 42.9s ▼15.4×	6m 46.4s ▼14.2× 6m 34.1s ▼15.3×	8m 46.4s ▼34.5× 5m 22.9s ▼25.4×	3m 11.5s ▼6.5× 3m 2.2s ▼6.9×	6m 30.8s ▼45.0× 3m 13.4s ▼23.8×	9m 50.5s ▼3.4× 9m 37.9s ▼3.4×	14m 21.8s ▼6.3× 9m 10.7s ▼4.1×	4m 10.1s ▼1.4× 4m 1s ▼1.4×	9m 51.7s ▼10.6× 4m 35.5s ▼5.0×	6m 16.9s ▼8.7× 6m 6.9s ▼8.6×	9m 20.4s ▼14.4× 5m 57.2s ▼9.4×	3m 10.7s ▼3.6× 3m 0.2s ▼3.7×	6m 41.2s ▼14.0× 3m 21.5s ▼7.1×
R68	MPF / Vulkan RMQ / KSW2	·	·	·	·	5m 51.9s ▼12.3×[s] 5m 40.4s ▼13.2×[s]	8m 46.4s ▼34.5×[s] 5m 22.2s ▼25.3×[s]	3m 12.6s ▼6.6×[s] 2m 58.9s ▼6.8×[s]	6m 29.8s ▼44.9×[s] 3m 8.7s ▼23.2×[s]	·	·	·	·	·	·	·	·
R69	MPF / Vulkan Scalar-DP / KSW2	·	·	·	·	5m 38.2s ▼11.8×[s] 5m 28s ▼12.7×[s]	8m 37.9s ▼33.9×[s] 5m 16s ▼24.9×[s]	3m 25.8s ▼7.0× 3m 14.4s ▼7.4×	6m 24.9s ▼44.3×[s] 3m 8.7s ▼23.2×[s]	·	·	·	·	·	·	·	·
R70	MPF / Vulkan SIMD-DP / KSW2	·	·	·	·	5m 44.6s ▼12.1×[s] 5m 34.2s ▼12.9×[s]	8m 41.2s ▼34.1×[s] 5m 20s ▼25.2×[s]	3m 33.4s ▼7.3× 3m 19s ▼7.6×	6m 39.3s ▼46.0×[s] 3m 17.5s ▼24.3×[s]	·	·	·	·	·	·	·	·
R71	MPF / Vulkan Scalar-DP / Block	·	·	·	·	5m 40.3s ▼11.9×[s] 5m 29.9s ▼12.8×[s]	8m 42.9s ▼34.2×[s] 5m 22.1s ▼25.3×[s]	3m 41.9s ▼7.6× 3m 31.8s ▼8.1×	6m 25.4s ▼44.4×[s] 3m 9.3s ▼23.3×[s]	·	·	·	·	·	·	·	·
R72	MPF / Vulkan Vulkan-DP / KSW2	·	·	·	·	6m 5.5s ▼12.8×[s] 5m 53.5s ▼13.7×[s]	8m 57.9s ▼35.2×[s] 5m 38.3s ▼26.6×[s]	3m 46.3s ▼7.7× 3m 33.1s ▼8.1×	6m 39s ▼45.9×[s] 3m 23.4s ▼25.0×[s]	·	·	·	·	·	·	·	·
R73	MPF / Vulkan SIMD-DP / Block	·	·	·	·	5m 43.9s ▼12.0×[s] 5m 34.1s ▼12.9×[s]	8m 37.5s ▼33.9×[s] 5m 14.9s ▼24.8×[s]	4m 42.2s ▼9.6× 4m 30.5s ▼10.3×	6m 35.3s ▼45.5×[s] 3m 19.5s ▼24.5×[s]	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Minimap2 stays first. Other rows follow increasing minima of available Index A times across all local sizes, k/w pairs and alphabets. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index								No index							
		4 Gbp				2 Gbp				4 Gbp				2 Gbp			
		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15	
		DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	39.7s ▼2.2× 36.8s ▼2.2×	25.2s ▼1.5× 22.7s ▼1.5×	1m 56.9s ▼29.1× 1m 53.2s ▼32.9×	24.9s ▼1.6× 22.2s ▼1.6×	28.6s ▼11.5× 25.8s ▼15.6×	15.3s ▼2.8× 12.7s ▼3.3×	29.3s ▼15.0× 26.2s ▼22.9×	8.7s ▼1.7× 8.1s ▼2.3×	2m 52.4s ▲19% 2m 48.5s ▲15%	2m 17.1s ▲3.9× 2m 15.9s ▲1.7×	2m 55s ▼6% 2m 51s ▼12%	55.9s ▲8.6× 55.1s ▲3.1×	43.3s ▲2.6× 42.5s ▲2.4×	38.9s ▲8.0× 38.2s ▲2.8×	53.1s ▲1.6× 48.2s ▲1.5×	28.7s ▲9.8× 28.2s ▲2.9×
R74 BioseqDB	MPS / Vulkan RMQ / KSW2	·	·	·	·	1h 56m 53.5s ▼245.5×[s] 1h 56m 41.7s ▼271.1×[s]	·	·	·	·	·	·	·	·	·	·	·
R75	MPS / Vulkan RMQ / Block	·	·	·	·	1h 56m 56.6s ▼245.6×[s] 1h 56m 45.8s ▼271.2×[s]	·	·	·	·	·	·	·	·	·	·	·
R76	MPS / Vulkan SIMD-DP / KSW2	·	·	·	·	1h 56m 58.7s ▼245.7×[s] 1h 56m 45.6s ▼271.2×[s]	·	·	·	·	·	·	·	·	·	·	·
R77	MPS / Vulkan Scalar-DP / Block	·	·	·	·	1h 57m 18.3s ▼246.4×[s] 1h 57m 8.7s ▼272.1×[s]	·	·	·	·	·	·	·	·	·	·	·
R78	MPS / Vulkan SIMD-DP / Block	·	·	·	·	1h 57m 20.2s ▼246.4×[s] 1h 57m 9.8s ▼272.2×[s]	·	·	·	·	·	·	·	·	·	·	·
R79	MPS / Vulkan Scalar-DP / KSW2	·	·	·	·	1h 57m 21.3s ▼246.5×[s] 1h 57m 7.9s ▼272.1×[s]	·	·	·	·	·	·	·	·	·	·	·
R80	MPS / Vulkan Vulkan-DP / Block	·	·	·	·	1h 57m 33.2s ▼246.9×[s] 1h 57m 22.2s ▼272.6×[s]	·	·	·	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Minimap2 stays first. Other rows follow increasing minima of available Index A times across all local sizes, k/w pairs and alphabets. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 4 Gbp k=15, w=10								No index 4 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	36.2s ▼6.9× 35s ▼8.0×	38.1s ▼7.8× 36s ▼9.3×	1m 32.4s ▼10.7× 1m 28.5s ▼13.5×	6m 31.5s ▼11.5× 6m 26.3s ▼12.7×	31.6s ▼2.1× 30.3s ▼3.0×	34.1s ▼2.9× 32s ▼3.7×	1m 9.5s ▼3.0× 1m 6.1s ▼4.3×	3m 1s ▼7.9× 2m 51.5s ▼8.6×	2m 55.5s ▲20% 2m 54s ▲17%	2m 50.4s ▲23% 2m 48.2s ▲20%	5m 35.7s ▲18% 5m 31.9s ▲14%	44m 20.5s n/a 44m 15.6s n/a	1m 47.1s ▲5.5× 1m 45.8s ▲2.1×	1m 47.2s ▲4.6× 1m 45.1s ▲2.1×	3m 40.3s ▲4.2× 3m 36.6s ▲1.7×	29m 43.4s n/a 29m 38.5s n/a
R2 BioseqDB	MPF / SIMD-ref Scalar-DP / Block	5.3s ▲6.9× 4.5s ▲7.8×	4.9s ▲7.8× 4s ▲9.0×	11.6s ▲7.9×[s] 9s ▲9.9×[s]	35.1s ▲11.2×[s] 34s ▲11.4×[s]	14.8s ▲2.1×[s] 10.3s ▲2.9×[s]	15s ▲2.3×[s] 10.9s ▲2.9×[s]	26.1s ▲2.7×[s] 17.6s ▲3.8×[s]	22.9s ▲7.9×[s] 20.4s ▲8.4×[s]	4m 50.9s ▼1.7×[s] 4m 39.8s ▼1.6×[s]	4m 54.9s ▼1.7×[s] 4m 41.8s ▼1.7×[s]	·	·	·	·	·	·
R3	MPF / Two-stack Scalar-DP / Block	5.3s ▲6.8× 4.5s ▲7.7×	4.9s ▲7.8× 4s ▲8.9×	10.6s ▲8.7×[s] 7.4s ▲11.9×[s]	34.5s ▲11.3×[s] 33.4s ▲11.6×[s]	15.4s ▲2.0×[s] 11.3s ▲2.7×[s]	11.8s ▲2.9×[s] 8.8s ▲3.7×[s]	23.4s ▲3.0×[s] 16.1s ▲4.1×[s]	·	5m 6.4s ▼1.7×[s] 4m 54.3s ▼1.7×[s]	4m 10.4s ▼1.5×[s] 3m 58.2s ▼1.4×[s]	·	·	10m 12.8s ▼5.7×[s] 4m 6.6s ▼2.3×[s]	8m 47.2s ▼4.9×[s] 3m 42.1s ▼2.1×[s]	·	·
R4	MPF / Scalar Scalar-DP / Block	5.4s ▲6.7× 4.6s ▲7.6×	4.9s ▲7.7× 4s ▲8.9×	11.6s ▲8.0×[s] 8.6s ▲10.3×[s]	37.7s ▲10.4×[s] 36.6s ▲10.6×[s]	19.1s ▲1.7×[s] 14.6s ▲2.1×[s]	16.9s ▲2.0×[s] 13.7s ▲2.3×[s]	28s ▲2.5×[s] 20.6s ▲3.2×[s]	49.5s ▲3.7×[s] 45.9s ▲3.7×[s]	5m 20.4s ▼1.8×[s] 5m 8s ▼1.8×[s]	5m 8.8s ▼1.8×[s] 4m 57s ▼1.8×[s]	·	·	11m 9.4s ▼6.3× 5m 3.9s ▼2.9×	9m 52.5s ▼5.5× 5m 21.2s ▼3.1×	·	·
R5	MPF / SIMD-ref RMQ / Block	5.4s ▲6.7× 4.5s ▲7.8×	5s ▲7.7× 4.1s ▲8.8×	1m 11.9s ▲22%[s] 1m 8.9s ▲22%[s]	36s ▲10.9×[s] 34.6s ▲11.2×[s]	26.6s ▲16%[s] 21.2s ▲30%[s]	22.9s ▲1.5×[s] 17.8s ▲1.8×[s]	27.7s ▲2.5×[s] 17.3s ▲3.8×[s]	50.2s ▲3.6×[s] 47.7s ▲3.6×[s]	5m 0.9s ▼1.7×[s] 4m 49.9s ▼1.7×[s]	5m 32s ▼1.9×[s] 5m 18.9s ▼1.9×[s]	·	·	·	·	·	·
R6	MPF / Two-stack RMQ / Block	5.3s ▲6.8× 4.4s ▲7.9×	5s ▲7.7× 4.1s ▲8.9×	1m 21s ▲12%[s] 1m 17.7s ▲12%[s]	35s ▲11.2×[s] 33.7s ▲11.5×[s]	21.2s ▲1.5×[s] 16.9s ▲1.8×[s]	21.7s ▲1.6×[s] 18.6s ▲1.7×[s]	30.6s ▲2.3×[s] 23s ▲2.9×[s]	·	4m 31.5s ▼1.5× 4m 22.4s ▼1.5×	4m 45s ▼1.7× 4m 33.2s ▼1.6×	·	·	10m 15.4s ▼5.7×[s] 4m 7.2s ▼2.3×[s]	8m 43.3s ▼4.9×[s] 3m 39s ▼2.1×[s]	·	·
R7	MPF / Scalar RMQ / Block	5.5s ▲6.6× 4.6s ▲7.6×	5s ▲7.6× 4.1s ▲8.8×	22.8s ▲4.0×[s] 19.7s ▲4.5×[s]	35.2s ▲11.1×[s] 34s ▲11.4×[s]	35s ▼11%[s] 30.5s ▶ 1%[s]	35.6s ▶ 4%[s] 32.4s ▶ 1%[s]	34s ▲2.0×[s] 26.4s ▲2.5×[s]	56.3s ▲3.2×[s] 53s ▲3.2×[s]	5m 26.3s ▼1.9× 5m 17.1s ▼1.8×	5m 17.7s ▼1.9× 5m 5.9s ▼1.8×	·	·	11m 28.1s ▼6.4×[s] 5m 21.3s ▼3.0×[s]	10m 39.7s ▼6.0×[s] 5m 34.9s ▼3.2×[s]	·	·
R8	MPF / Two-stack SIMD-DP / Block	5.4s ▲6.7× 4.5s ▲7.8×	5s ▲7.6× 4s ▲8.9×	43.8s ▲2.1×[s] 40.6s ▲2.2×[s]	35.7s ▲11.0×[s] 34.6s ▲11.2×[s]	17.7s ▲1.8×[s] 13.5s ▲2.2×[s]	22s ▲1.6×[s] 18.6s ▲1.7×[s]	23.1s ▲3.0×[s] 15.6s ▲4.2×[s]	36.6s ▲5.0×[s] 33.2s ▲5.2×[s]	4m 54.2s ▼1.7×[s] 4m 41.8s ▼1.6×[s]	4m 16.1s ▼1.5×[s] 4m 3.7s ▼1.4×[s]	·	·	10m 20.1s ▼5.8×[s] 4m 12.8s ▼2.4×[s]	8m 41.2s ▼4.9×[s] 3m 38.4s ▼2.1×[s]	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Minimap2 stays first. Other rows follow increasing minima of available Index A times within this size/k/w group, across alphabets and all displayed machines. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 4 Gbp k=15, w=10								No index 4 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	36.2s ▼6.9× 35s ▼8.0×	38.1s ▼7.8× 36s ▼9.3×	1m 32.4s ▼10.7× 1m 28.5s ▼13.5×	6m 31.5s ▼11.5× 6m 26.3s ▼12.7×	31.6s ▼2.1× 30.3s ▼3.0×	34.1s ▼2.9× 32s ▼3.7×	1m 9.5s ▼3.0× 1m 6.1s ▼4.3×	3m 1s ▼7.9× 2m 51.5s ▼8.6×	2m 55.5s ▲20% 2m 54s ▲17%	2m 50.4s ▲23% 2m 48.2s ▲20%	5m 35.7s ▲18% 5m 31.9s ▲14%	44m 20.5s n/a 44m 15.6s n/a	1m 47.1s ▲5.5× 1m 45.8s ▲2.1×	1m 47.2s ▲4.6× 1m 45.1s ▲2.1×	3m 40.3s ▲4.2× 3m 36.6s ▲1.7×	29m 43.4s n/a 29m 38.5s n/a
R9 BioseqDB	MPF / Scalar SIMD-DP / Block	5.5s ▲6.6× 4.6s ▲7.6×	5.1s ▲7.5× 4.1s ▲8.8×	32.2s ▲2.9×ˢ 29.2s ▲3.0×ˢ	35.7s ▲11.0×ˢ 34.6s ▲11.2×ˢ	23.2s ▲27%ˢ 18.8s ▲1.6×ˢ	32.3s ▲5%ˢ 28.5s ▲11%ˢ	1m 28.5s ▼27%ˢ 1m 20.8s ▼22%ˢ	57.3s ▲3.2×ˢ 53.9s ▲3.2×ˢ	5m 24.9s ▼1.9× 5m 16s ▼1.8×	5m 22s ▼1.9×ˢ 5m 9.2s ▼1.8×ˢ	·	·	11m 28.2s ▼6.4×ˢ 5m 22.9s ▼3.1×ˢ	9m 48.5s ▼5.5× 5m 17.7s ▼3.0×	·	·
R10	MPF / Bounded Scalar-DP / Block	5.3s ▲6.8× 4.6s ▲7.7×	5.5s ▲6.9× 4.6s ▲7.8×	10.8s ▲8.5×ˢ 7.8s ▲11.4×ˢ	34.1s ▲11.5×ˢ 33s ▲11.7×ˢ	18.7s ▲1.7×ˢ 14.3s ▲2.1×ˢ	18.3s ▲1.9×ˢ 14.8s ▲2.2×ˢ	23.6s ▲3.0×ˢ 16s ▲4.1×ˢ	54s ▲3.3×ˢ 50.6s ▲3.4×ˢ	3m 40.6s ▼26% 3m 31.8s ▼22%	4m 25.9s ▼1.6× 4m 13.9s ▼1.5×	·	·	10m 4.6s ▼5.6×ˢ 3m 56.7s ▼2.2×ˢ	14m 49s ▼8.3×ˢ 8m 18.7s ▼4.7×ˢ	17m 59.1s ▼4.9× 9m 9.4s ▼2.5×	·
R11	MPF / Bounded RMQ / Block	5.4s ▲6.8× 4.6s ▲7.6×	9.9s ▲3.9× 9.1s ▲4.0×	1m 5.5s ▲29%ˢ 1m 2.5s ▲29%ˢ	37.3s ▲10.5×ˢ 36.1s ▲10.7×ˢ	25.7s ▲19%ˢ 21.3s ▲30%ˢ	25.7s ▲25%ˢ 22.1s ▲1.5×ˢ	23.6s ▲2.9×ˢ 15.9s ▲4.2×ˢ	53s ▲3.4×ˢ 49.5s ▲3.5×ˢ	4m 2.1s ▼1.4× 3m 53.3s ▼1.3×	3m 42.6s ▼1.3× 3m 31.1s ▼25%	·	·	10m 3.7s ▼5.6×ˢ 3m 53.5s ▼2.2×ˢ	14m 40.9s ▼8.2×ˢ 8m 18.9s ▼4.7×ˢ	·	·
R12	MPF / SIMD-ref SIMD-DP / Block	5.4s ▲6.7× 4.5s ▲7.8×	17.6s ▲2.2× 16.5s ▲2.2×	8.7s ▲10.7× 6.5s ▲13.5×	35.9s ▲10.9×ˢ 34.7s ▲11.1×ˢ	29.2s ▲8%ˢ 22.8s ▲25%ˢ	33s ▶3%ˢ 27s ▲16%ˢ	54.5s ▲22%ˢ 43.2s ▲1.5×ˢ	37.7s ▲4.8×ˢ 35.3s ▲4.9×ˢ	4m 54.9s ▼1.7×ˢ 4m 43.5s ▼1.6×ˢ	4m 42.7s ▼1.7×ˢ 4m 29.7s ▼1.6×ˢ	·	·	·	·	·	·
R13	MPF / SIMD-ref Scalar-DP / KSW2	13.8s ▲2.6× 11.3s ▲3.1×	6.3s ▲6.0× 3.9s ▲9.3×	17.9s ▲5.2×ˢ 10.1s ▲8.8×ˢ	39s ▲10.0×ˢ 35.1s ▲11.0×ˢ	14.8s ▲2.1×ˢ 10.2s ▲3.0×ˢ	16s ▲2.1×ˢ 11.5s ▲2.8×ˢ	26.6s ▲2.6×ˢ 17.2s ▲3.8×ˢ	23.6s ▲7.7×ˢ 20.4s ▲8.4×ˢ	4m 52.7s ▼1.7×ˢ 4m 39.7s ▼1.6×ˢ	5m 1.6s ▼1.8×ˢ 4m 46.5s ▼1.7×ˢ	·	·	·	·	·	·
R14	MPF / Two-stack RMQ / KSW2	6.9s ▲5.2× 4.4s ▲8.0×	6.4s ▲5.9× 3.9s ▲9.3×	32.1s ▲2.9×ˢ 23.8s ▲3.7×ˢ	35.5s ▲11.0×ˢ 31.4s ▲12.3×ˢ	17.2s ▲1.8×ˢ 13s ▲2.3×ˢ	17.3s ▲2.0×ˢ 13.9s ▲2.3×ˢ	24.4s ▲2.8×ˢ 16.4s ▲4.0×ˢ	·	4m 33.3s ▼1.6×ˢ 4m 19.4s ▼1.5×ˢ	4m 13.5s ▼1.5×ˢ 4m 0s ▼1.4×ˢ	·	·	10m 11.1s ▼5.7×ˢ 4m 1.3s ▼2.3×ˢ	8m 47.2s ▼4.9×ˢ 3m 42.1s ▼2.1×ˢ	·	·
R15	MPF / SIMD-ref RMQ / KSW2	6.9s ▲5.3× 4.4s ▲8.0×	6.5s ▲5.9× 3.9s ▲9.3×	19.3s ▲4.8×ˢ 11.4s ▲7.8×ˢ	39.9s ▲9.8×ˢ 35.7s ▲10.8×ˢ	15.9s ▲2.0×ˢ 10.4s ▲2.9×ˢ	17.9s ▲1.9×ˢ 12.3s ▲2.6×ˢ	29.4s ▲2.4×ˢ 17.7s ▲3.7×ˢ	23.4s ▲7.7×ˢ 20s ▲8.6×ˢ	4m 58.1s ▼1.7×ˢ 4m 45.3s ▼1.6×ˢ	5m 2.1s ▼1.8×ˢ 4m 47.2s ▼1.7×ˢ	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —ᵍ = GPU unavailable, not executed. n/a = no measured comparison.
ᶠSIMD-ref request observed as scalar, compared only as that effective path. ᴾAccepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Minimap2 stays first. Other rows follow increasing minima of available Index A times within this size/k/w group, across alphabets and all displayed machines. No Index results = last. Coverage varies.
ˢFAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 4 Gbp k=15, w=10								No index 4 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	36.2s ▼6.9× 35s ▼8.0×	38.1s ▼7.8× 36s ▼9.3×	1m 32.4s ▼10.7× 1m 28.5s ▼13.5×	6m 31.5s ▼11.5× 6m 26.3s ▼12.7×	31.6s ▼2.1× 30.3s ▼3.0×	34.1s ▼2.9× 32s ▼3.7×	1m 9.5s ▼3.0× 1m 6.1s ▼4.3×	3m 1s ▼7.9× 2m 51.5s ▼8.6×	2m 55.5s ▲20% 2m 54s ▲17%	2m 50.4s ▲23% 2m 48.2s ▲20%	5m 35.7s ▲18% 5m 31.9s ▲14%	44m 20.5s n/a 44m 15.6s n/a	1m 47.1s ▲5.5× 1m 45.8s ▲2.1×	1m 47.2s ▲4.6× 1m 45.1s ▲2.1×	3m 40.3s ▲4.2× 3m 36.6s ▲1.7×	29m 43.4s n/a 29m 38.5s n/a
R16 BioseqDB	MPF / Scalar SIMD-DP / KSW2	8.2s ▲4.4× 5.6s ▲6.2×	6.5s ▲5.9× 3.9s ▲9.1×	2m 15.5s ▼1.5×[s] 2m 7.7s ▼1.4×[s]	37.8s ▲10.3×[s] 33.9s ▲11.4×[s]	42.2s ▼1.3×[s] 37.6s ▼24%[s]	26.9s ▲21%[s] 23.3s ▲27%[s]	1m 59.3s ▼1.7×[s] 1m 51.3s ▼1.7×[s]	55s ▲3.3×[s] 50.9s ▲3.4×[s]	5m 27.2s ▼1.9× 5m 16.6s ▼1.8×	5m 50s ▼2.1× 5m 36.1s ▼2.0×	·	·	11m 14.3s ▼6.3× 5m 8.1s ▼2.9×	9m 48.4s ▼5.5× 5m 16.4s ▼3.0×	·	·
R17	MPF / SIMD-ref SIMD-DP / KSW2	6.9s ▲5.2× 4.4s ▲8.0×	6.5s ▲5.9× 3.9s ▲9.2×	53.6s ▲1.7×[s] 45.3s ▲2.0×[s]	39.8s ▲9.8×[s] 35.9s ▲10.8×[s]	23.3s ▲26%[s] 17.1s ▲1.8×[s]	38.6s ▼13%[s] 32.2s ▶ 1%[s]	46.7s ▲1.5×[s] 34.1s ▲1.9×[s]	51.3s ▲3.5×[s] 47.9s ▲3.6×[s]	4m 57.4s ▼1.7×[s] 4m 44.6s ▼1.6×[s]	5m 23.4s ▼1.9×[s] 5m 7.5s ▼1.8×[s]	·	·	·	·	·	·
R18	MPF / Scalar RMQ / KSW2	7s ▲5.1× 4.5s ▲7.7×	6.5s ▲5.9× 3.9s ▲9.1×	17.7s ▲5.2×[s] 9.5s ▲9.3×[s]	36.3s ▲10.8×[s] 32.3s ▲12.0×[s]	26.4s ▲17%[s] 21.9s ▲28%[s]	21s ▲1.6×[s] 17.6s ▲1.8×[s]	35.4s ▲2.0×[s] 27.1s ▲2.4×[s]	51.9s ▲3.5×[s] 48s ▲3.6×[s]	5m 15.8s ▼1.8×[s] 5m 2.5s ▼1.7×[s]	5m 19.7s ▼1.9×[s] 5m 5.9s ▼1.8×[s]	·	·	11m 18.9s ▼6.3× 5m 12.1s ▼2.9×	9m 52.6s ▼5.5× 5m 20s ▼3.0×	·	·
R19	MPF / Bounded Scalar-DP / KSW2	6.9s ▲5.3× 4.4s ▲7.9×	7.6s ▲5.0× 5.2s ▲6.9×	14.7s ▲6.3×[s] 7s ▲12.7×[s]	39.1s ▲10.0×[s] 35.3s ▲11.0×[s]	22.4s ▲29%[s] 18.1s ▲1.7×[s]	14.1s ▲2.4×[s] 10s ▲3.2×[s]	24.1s ▲2.9×[s] 16s ▲4.1×[s]	52s ▲3.5×[s] 48.1s ▲3.6×[s]	3m 56.3s ▼1.3× 3m 44.9s ▼29%	4m 6.8s ▼1.4× 3m 52.3s ▼1.4×	·	·	10m 13.7s ▼5.7×[s] 4m 6.2s ▼2.3×[s]	14m 38.1s ▼8.2×[s] 8m 47s ▼5.0×[s]	·	·
R20	MPF / Two-stack SIMD-DP / KSW2	6.9s ▲5.3× 4.4s ▲8.0×	7.3s ▲5.2× 4.8s ▲7.5×	12.5s ▲7.4× 6.8s ▲13.1×	41.9s ▲9.4×[s] 37.8s ▲10.2×[s]	28.6s ▲9%[s] 24.4s ▲19%[s]	24s ▲30%[s] 20.6s ▲1.6×[s]	57.1s ▲18%[s] 49.1s ▲26%[s]	·	4m 37.5s ▼1.6× 4m 25.9s ▼1.5×	4m 48.3s ▼1.7× 4m 34.5s ▼1.6×	·	·	10m 6.4s ▼5.7×[s] 3m 55.9s ▼2.2×[s]	8m 51.6s ▼5.0×[s] 3m 47s ▼2.2×[s]	·	·
R21	MPF / Bounded RMQ / KSW2	6.9s ▲5.2× 4.5s ▲7.9×	9.6s ▲3.9× 7.2s ▲5.0×	14.6s ▲6.3×[s] 6.7s ▲13.1×[s]	39.9s ▲9.8×[s] 35.8s ▲10.8×[s]	30.9s ▶ 2%[s] 26.5s ▲12%[s]	37.1s ▼9%[s] 33.3s ▶ 4%[s]	24.1s ▲2.9×[s] 15.8s ▲4.2×[s]	58.6s ▲3.1×[s] 54.5s ▲3.1×[s]	3m 41.3s ▼26% 3m 30.7s ▼21%	3m 59.5s ▼1.4× 3m 45.5s ▼1.3×	·	·	10m 2.7s ▼5.6×[s] 3m 55.6s ▼2.2×[s]	15m 4.3s ▼8.4×[s] 8m 30.7s ▼4.9×[s]	·	·
R22	MPF / Bounded SIMD-DP / KSW2	14.7s ▲2.5× 12.1s ▲2.9×	8.1s ▲4.7× 5.5s ▲6.5×	24.9s ▲3.7×[s] 16.9s ▲5.2×[s]	34.6s ▲11.3×[s] 30.4s ▲12.7×[s]	30s ▲5%[s] 25.5s ▲16%[s]	38.5s ▼13%[s] 34.3s ▼7%[s]	51.7s ▲26%[s] 43.6s ▲1.5×[s]	55.6s ▲3.3×[s] 51.7s ▲3.3×[s]	3m 57.7s ▼1.4× 3m 46.9s ▼1.3×	4m 0.1s ▼1.4× 3m 46s ▼1.3×	·	·	10m 21.9s ▼5.8×[s] 4m 12s ▼2.4×[s]	11m 12.6s ▼6.3×[s] 5m 0.9s ▼2.9×[s]	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Minimap2 stays first. Other rows follow increasing minima of available Index A times within this size/k/w group, across alphabets and all displayed machines. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 4 Gbp k=15, w=10								No index 4 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	36.2s ▼6.9× 35s ▼8.0×	38.1s ▼7.8× 36s ▼9.3×	1m 32.4s ▼10.7× 1m 28.5s ▼13.5×	6m 31.5s ▼11.5× 6m 26.3s ▼12.7×	31.6s ▼2.1× 30.3s ▼3.0×	34.1s ▼2.9× 32s ▼3.7×	1m 9.5s ▼3.0× 1m 6.1s ▼4.3×	3m 1s ▼7.9× 2m 51.5s ▼8.6×	2m 55.5s ▲20% 2m 54s ▲17%	2m 50.4s ▲23% 2m 48.2s ▲20%	5m 35.7s ▲18% 5m 31.9s ▲14%	44m 20.5s n/a 44m 15.6s n/a	1m 47.1s ▲5.5× 1m 45.8s ▲2.1×	1m 47.2s ▲4.6× 1m 45.1s ▲2.1×	3m 40.3s ▲4.2× 3m 36.6s ▲1.7×	29m 43.4s n/a 29m 38.5s n/a
R23 BioseqDB	MPF / Bounded SIMD-DP / Block	9.9s ▲3.6× 9s ▲3.9×	13s ▲2.9× 11.9s ▲3.0×	9.4s ▲9.8× 7s ▲12.7×	35s ▲11.2×[s] 33.8s ▲11.4×[s]	23.1s ▲27%[s] 18.6s ▲1.6×[s]	25.2s ▲26%[s] 21.2s ▲1.5×[s]	54.4s ▲22%[s] 46.7s ▲29%[s]	49.8s ▲3.6×[s] 46.4s ▲3.7×[s]	3m 44.5s ▼28% 3m 35.9s ▼24%	3m 57.4s ▼1.4× 3m 45.7s ▼1.3×	6m 50.6s ▼22% 6m 30.7s ▼18%	·	9m 45.5s ▼5.5× 3m 41.5s ▼2.1×	8m 25.6s ▼4.7× 3m 56.7s ▼2.3×	15m 24.9s ▼4.2× 6m 0.2s ▼1.7×	·
R24	MPF / Scalar Scalar-DP / KSW2	15s ▲2.4× 12.5s ▲2.8×	10.6s ▲3.6× 8.2s ▲4.4×	15.3s ▲6.1×[s] 7.3s ▲12.1×[s]	36.2s ▲10.8×[s] 32.2s ▲12.0×[s]	20s ▲1.6×[s] 15.6s ▲1.9×[s]	12.4s ▲2.8×[s] 9.1s ▲3.5×[s]	25.3s ▲2.7×[s] 17.1s ▲3.9×[s]	57.6s ▲3.1×[s] 53.6s ▲3.2×[s]	5m 25.6s ▼1.9×[s] 5m 12.1s ▼1.8×[s]	5m 5.9s ▼1.8×[s] 4m 52.6s ▼1.7×[s]	·	·	11m 14.1s ▼6.3× 5m 5.9s ▼2.9×	9m 54.4s ▼5.5× 5m 20.4s ▼3.0×	·	·
R25	MPS / Bounded Scalar-DP / Block	10.9s ▲3.3× 10.3s ▲3.4×	11.1s ▲3.4× 10.3s ▲3.5×	21.5s ▲4.3×[s] 18.9s ▲4.7×[s]	·	8m 24.2s ▼16.0×[s] 59.7s ▼2.0×[s]	6m 45.3s ▼11.9×[s] 1m 3.2s ▼2.0×[s]	·	·	4m 17.5s ▼1.5× 4m 8.2s ▼1.4×	4m 7.1s ▼1.4× 3m 55.3s ▼1.4×	·	·	10m 38s ▼6.0×[s] 4m 13.2s ▼2.4×[s]	9m 41.8s ▼5.4×[s] 4m 18.5s ▼2.5×[s]	·	·
R26	MPS / SIMD-ref Scalar-DP / Block	11.9s ▲3.0×[s] 10.9s ▲3.2×[s]	10.9s ▲3.5×[s] 10s ▲3.6×[s]	·	·	6m 13.5s ▼11.8×[s] 39.8s ▼1.3×[s]	5m 21s ▼9.4×[s] 35.2s ▼10%[s]	·	·	·	·	·	·	·	·	·	·
R27	MPS / SIMD-ref RMQ / Block	12s ▲3.0×[s] 11s ▲3.2×[s]	11s ▲3.5×[s] 10s ▲3.6×[s]	·	·	6m 15.3s ▼11.9×[s] 41.2s ▼1.4×[s]	5m 22.7s ▼9.5×[s] 37s ▼15%[s]	·	·	·	·	·	·	·	·	·	·
R28	MPS / Bounded RMQ / Block	11s ▲3.3× 10.3s ▲3.4×	11.5s ▲3.3× 10.7s ▲3.4×	21.8s ▲4.2×[s] 18.8s ▲4.7×[s]	·	8m 23.3s ▼15.9×[s] 59.1s ▼1.9×[s]	7m 3.6s ▼12.4×[s] 1m 6.2s ▼2.1×[s]	·	·	4m 10.2s ▼1.4× 4m 1s ▼1.4×	4m 11.6s ▼1.5× 3m 59.9s ▼1.4×	6m 51.1s ▼22% 6m 31.1s ▼18%	·	10m 27.7s ▼5.9×[s] 4m 6s ▼2.3×[s]	9m 42.6s ▼5.4×[s] 4m 14.7s ▼2.4×[s]	·	·
R29	MPS / Bounded SIMD-DP / Block	11.1s ▲3.3× 10.5s ▲3.3×	11.4s ▲3.4× 10.5s ▲3.4×	30.1s ▲3.1×[s] 27.5s ▲3.2×[s]	·	8m 12s ▼15.6×[s] 1m 0.9s ▼2.0×[s]	6m 45.6s ▼11.9×[s] 1m 1.4s ▼1.9×[s]	·	·	4m 19.3s ▼1.5× 4m 10.3s ▼1.4×	4m 29.2s ▼1.6× 4m 17.1s ▼1.5×	·	·	10m 25.7s ▼5.8×[s] 4m 1.1s ▼2.3×[s]	9m 26.4s ▼5.3×[s] 4m 6.2s ▼2.3×[s]	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash. Minimap2 stays first. Other rows follow increasing minima of available Index A times within this size/k/w group, across alphabets and all displayed machines. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Table 12.24: (B): AWS: 4 Gbp, k=15, w=10 (mapping interval)

Engine	Execution	Index 4 Gbp k=15, w=10								No index 4 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	36.2s ▼6.9× 35s ▼8.0×	38.1s ▼7.8× 36s ▼9.3×	1m 32.4s ▼10.7× 1m 28.5s ▼13.5×	6m 31.5s ▼11.5× 6m 26.3s ▼12.7×	31.6s ▼2.1× 30.3s ▼3.0×	34.1s ▼2.9× 32s ▼3.7×	1m 9.5s ▼3.0× 1m 6.1s ▼4.3×	3m 1s ▼7.9× 2m 51.5s ▼8.6×	2m 55.5s ▲20% 2m 54s ▲17%	2m 50.4s ▲23% 2m 48.2s ▲20%	5m 35.7s ▲18% 5m 31.9s ▲14%	44m 20.5s n/a 44m 15.6s n/a	1m 47.1s ▲5.5× 1m 45.8s ▲2.1×	1m 47.2s ▲4.6× 1m 45.1s ▲2.1×	3m 40.3s ▲4.2× 3m 36.6s ▲1.7×	29m 43.4s n/a 29m 38.5s n/a
R30 BioseqDB	MPS / Two-stack SIMD-DP / Block	11.5s ▲3.1× 10.9s ▲3.2×	11.2s ▲3.4× 10.4s ▲3.5×	·	·	8m 2s ▼15.3×[s] 48.3s ▼1.6×[s]	7m 1.6s ▼12.4×[s] 48s ▼1.5×[s]	·	·	4m 39.5s ▼1.6× 4m 30.1s ▼1.6×	4m 24.7s ▼1.6× 4m 13s ▼1.5×	·	·	·	·	·	·
R31	MPS / Scalar SIMD-DP / Block	11.3s ▲3.2× 10.7s ▲3.3×	11.4s ▲3.3× 10.7s ▲3.4×	·	·	8m 11.6s ▼15.6×[s] 1m 1.8s ▼2.0×[s]	6m 34.1s ▼11.6×[s] 58.8s ▼1.8×[s]	·	·	5m 48.1s ▼2.0×[s] 5m 36.4s ▼1.9×[s]	5m 51.7s ▼2.1× 5m 39.9s ▼2.0×	·	·	11m 45.1s ▼6.6× 5m 20.1s ▼3.0×	10m 10.3s ▼5.7× 5m 30.2s ▼3.1×	·	·
R32	MPS / Two-stack RMQ / Block	11.4s ▲3.2× 10.9s ▲3.2×	11.8s ▲3.2× 10.9s ▲3.3×	·	·	8m 8.4s ▼15.5×[s] 45.9s ▼1.5×[s]	7m 22.7s ▼13.0×[s] 47.8s ▼1.5×[s]	·	·	4m 44.1s ▼1.6× 4m 35.1s ▼1.6×	4m 42.8s ▼1.7× 4m 30.9s ▼1.6×	·	·	·	·	·	·
R33	MPS / Scalar RMQ / Block	11.5s ▲3.2× 10.9s ▲3.2×	11.9s ▲3.2× 11s ▲3.3×	·	·	7m 41.6s ▼14.6×[s] 59.8s ▼2.0×[s]	6m 41.2s ▼11.8×[s] 59.4s ▼1.9×[s]	·	·	5m 40.1s ▼1.9× 5m 31s ▼1.9×	6m 11.3s ▼2.2× 5m 59.5s ▼2.1×	·	·	11m 59.3s ▼6.7×[s] 5m 36.2s ▼3.2×[s]	·	·	·
R34	MPS / Two-stack Scalar-DP / Block	12s ▲3.0×[s] 11s ▲3.2×[s]	11.6s ▲3.3×[s] 10.6s ▲3.4×[s]	·	·	8m 0.3s ▼15.2×[s] 45.7s ▼1.5×[s]	6m 34.2s ▼11.6×[s] 45.6s ▼1.4×[s]	·	·	·	·	·	·	·	·	·	·
R35	MPS / Scalar Scalar-DP / Block	15.6s ▲2.3×[s] 14.7s ▲2.4×[s]	12.1s ▲3.1×[s] 11s ▲3.3×[s]	·	·	7m 42.1s ▼14.6×[s] 58.9s ▼1.9×[s]	6m 19.9s ▼11.1×[s] 54.2s ▼1.7×[s]	·	·	5m 51.9s ▼2.0×[s] 5m 39.9s ▼2.0×[s]	·	·	·	11m 44.1s ▼6.6× 5m 20.6s ▼3.0×	10m 13.1s ▼5.7× 5m 32.9s ▼3.2×	·	·
R36	MPS / Bounded SIMD-DP / KSW2	12.3s ▲3.0× 9.7s ▲3.6×	12.1s ▲3.1× 9.1s ▲4.0×	26.1s ▲3.5×[s] 16.6s ▲5.3×[s]	·	8m 21.9s ▼15.9×[s] 57.7s ▼1.9×[s]	7m 3s ▼12.4×[s] 1m 0.6s ▼1.9×[s]	·	·	4m 19.7s ▼1.5× 4m 8.9s ▼1.4×	3m 58.7s ▼1.4× 3m 45.4s ▼1.3×	10m 17.4s ▼1.8× 9m 56s ▼1.8×	·	10m 43.5s ▼6.0×[s] 4m 18.3s ▼2.4×[s]	9m 31.6s ▼5.3×[s] 4m 7.6s ▼2.4×[s]	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Minimap2 stays first. Other rows follow increasing minima of available Index A times within this size/k/w group, across alphabets and all displayed machines. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 4 Gbp k=15, w=10								No index 4 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	36.2s ▼6.9× 35s ▼8.0×	38.1s ▼7.8× 36s ▼9.3×	1m 32.4s ▼10.7× 1m 28.5s ▼13.5×	6m 31.5s ▼11.5× 6m 26.3s ▼12.7×	31.6s ▼2.1× 30.3s ▼3.0×	34.1s ▼2.9× 32s ▼3.7×	1m 9.5s ▼3.0× 1m 6.1s ▼4.3×	3m 1s ▼7.9× 2m 51.5s ▼8.6×	2m 55.5s ▲20% 2m 54s ▲17%	2m 50.4s ▲23% 2m 48.2s ▲20%	5m 35.7s ▲18% 5m 31.9s ▲14%	44m 20.5s n/a 44m 15.6s n/a	1m 47.1s ▲5.5× 1m 45.8s ▲2.1×	1m 47.2s ▲4.6× 1m 45.1s ▲2.1×	3m 40.3s ▲4.2× 3m 36.6s ▲1.7×	29m 43.4s n/a 29m 38.5s n/a
R37 BioseqDB	MPS / Bounded Scalar-DP / KSW2	12.3s ▲2.9× 9.7s ▲3.6×	13.5s ▲2.8× 9.6s ▲3.7×	26.1s ▲3.5×[s] 16.5s ▲5.4×[s]	·	8m 23.3s ▼15.9×[s] 58.5s ▼1.9×[s]	6m 59.2s ▼12.3×[s] 1m 4.7s ▼2.0×[s]	·	·	4m 19.1s ▼1.5× 4m 8.7s ▼1.4×	4m 29.2s ▼1.6× 4m 15s ▼1.5×	·	·	10m 32.3s ▼5.9×[s] 4m 7.7s ▼2.3×[s]	9m 53.4s ▼5.5×[s] 4m 23.7s ▼2.5×[s]	·	·
R38	MPS / SIMD-ref SIMD-DP / KSW2	14.6s ▲2.5×[s] 9.8s ▲3.6×[s]	12.6s ▲3.0×[s] 8.7s ▲4.1×[s]	·	·	6m 16.2s ▼11.9×[s] 40.5s ▼1.3×[s]	5m 23.1s ▼9.5×[s] 36.2s ▼13%[s]	·	·	·	·	·	·	·	·	·	·
R39	MPS / Bounded RMQ / KSW2	12.6s ▲2.9× 9.7s ▲3.6×	13.7s ▲2.8× 9.5s ▲3.8×	26.4s ▲3.5×[s] 16.6s ▲5.3×[s]	·	8m 35.7s ▼16.3×[s] 58.8s ▼1.9×[s]	6m 49.4s ▼12.0×[s] 1m 4.5s ▼2.0×[s]	·	·	4m 20.8s ▼1.5× 4m 9.9s ▼1.4×	3m 58.2s ▼1.4× 3m 44.9s ▼1.3×	6m 49.2s ▼22% 6m 25.6s ▼16%	·	10m 29.6s ▼5.9×[s] 4m 5.4s ▼2.3×[s]	9m 40.1s ▼5.4×[s] 4m 14.5s ▼2.4×[s]	·	·
R40	MPS / SIMD-ref Scalar-DP / KSW2	14.5s ▲2.5×[s] 10s ▲3.5×[s]	12.7s ▲3.0×[s] 8.7s ▲4.1×[s]	·	·	6m 13.5s ▼11.8×[s] 39.6s ▼1.3×[s]	5m 21.7s ▼9.4×[s] 35.3s ▼10%[s]	·	·	·	·	·	·	·	·	·	·
R41	MPS / Scalar SIMD-DP / KSW2	12.7s ▲2.9× 9.9s ▲3.6×	13.2s ▲2.9× 9.7s ▲3.7×	·	·	8m 31.6s ▼16.2×[s] 1m 3.7s ▼2.1×[s]	6m 40.8s ▼11.8×[s] 56.5s ▼1.8×[s]	·	·	5m 39.7s ▼1.9× 5m 28.2s ▼1.9×	5m 42.5s ▼2.0× 5m 29s ▼2.0×	·	·	11m 45s ▼6.6× 5m 19.1s ▼3.0×	10m 13.5s ▼5.7× 5m 31.5s ▼3.2×	·	·
R42	MPS / SIMD-ref RMQ / KSW2	14.5s ▲2.5×[s] 10s ▲3.5×[s]	12.8s ▲3.0×[s] 8.7s ▲4.1×[s]	·	·	6m 21.1s ▼12.1×[s] 41.4s ▼1.4×[s]	5m 23.8s ▼9.5×[s] 37s ▼16%[s]	·	·	·	·	·	·	·	·	·	·
R43	MPS / Two-stack SIMD-DP / KSW2	13.1s ▲2.8× 10.5s ▲3.3×	13.4s ▲2.8× 9.7s ▲3.7×	·	·	8m 20.3s ▼15.8×[s] 45.6s ▼1.5×[s]	6m 45s ▼11.9×[s] 45.4s ▼1.4×[s]	·	·	4m 54.4s ▼1.7× 4m 44.1s ▼1.6×	4m 39.9s ▼1.6× 4m 26.4s ▼1.6×	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash. Minimap2 stays first. Other rows follow increasing minima of available Index A times within this size/k/w group, across alphabets and all displayed machines. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 4 Gbp k=15, w=10								No index 4 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	36.2s ▼6.9× 35s ▼8.0×	38.1s ▼7.8× 36s ▼9.3×	1m 32.4s ▼10.7× 1m 28.5s ▼13.5×	6m 31.5s ▼11.5× 6m 26.3s ▼12.7×	31.6s ▼2.1× 30.3s ▼3.0×	34.1s ▼2.9× 32s ▼3.7×	1m 9.5s ▼3.0× 1m 6.1s ▼4.3×	3m 1s ▼7.9× 2m 51.5s ▼8.6×	2m 55.5s ▲20% 2m 54s ▲17%	2m 50.4s ▲23% 2m 48.2s ▲20%	5m 35.7s ▲18% 5m 31.9s ▲14%	44m 20.5s n/a 44m 15.6s n/a	1m 47.1s ▲5.5× 1m 45.8s ▲2.1×	1m 47.2s ▲4.6× 1m 45.1s ▲2.1×	3m 40.3s ▲4.2× 3m 36.6s ▲1.7×	29m 43.4s n/a 29m 38.5s n/a
R44 BioseqDB	MPS / Two-stack Scalar-DP / KSW2	14.6s ▲2.5×[s] 9.9s ▲3.5×[s]	13.2s ▲2.9×[s] 9.2s ▲3.9×[s]	·	·	8m 22s ▼15.9×[s] 46.5s ▼1.5×[s]	6m 54.6s ▼12.2×[s] 45.8s ▼1.4×[s]	·	·	·	·	·	·	10m 24.5s ▼5.8× 4m 4.1s ▼2.3×	8m 30.6s ▼4.8× 3m 55.3s ▼2.2×	·	·
R45	MPS / Scalar RMQ / KSW2	13.3s ▲2.7× 10.8s ▲3.3×	18.6s ▲2.0×[s] 13.7s ▲2.6×[s]	·	·	7m 41.4s ▼14.6×[s] 59.5s ▼2.0×[s]	6m 30.2s ▼11.5×[s] 54.7s ▼1.7×[s]	·	·	5m 34.3s ▼1.9×[s] 5m 20.2s ▼1.8×[s]	·	·	·	12m 14s ▼6.9×[s] 5m 48.2s ▼3.3×[s]	·	·	·
R46	MPS / Two-stack RMQ / KSW2	13.3s ▲2.7× 10.8s ▲3.2×	17.9s ▲2.1×[s] 13.5s ▲2.7×[s]	·	·	8m 11.3s ▼15.6×[s] 46s ▼1.5×[s]	6m 46s ▼11.9×[s] 45.8s ▼1.4×[s]	·	·	·	·	·	·	·	·	·	·
R47	MPF / Two-stack Scalar-DP / KSW2	20.1s ▲1.8× 17.6s ▲2.0×	16.8s ▲2.3× 14.4s ▲2.5×	17.5s ▲5.3×[s] 9.2s ▲9.6×[s]	35.7s ▲11.0×[s] 31.7s ▲12.2×[s]	15.2s ▲2.1×[s] 11s ▲2.7×[s]	14.2s ▲2.4×[s] 10.7s ▲3.0×[s]	23.3s ▲3.0×[s] 15.5s ▲4.3×[s]	·	4m 31.8s ▼1.5×[s] 4m 18.1s ▼1.5×[s]	4m 18s ▼1.5×[s] 4m 3.8s ▼1.4×[s]	·	·	10m 1.7s ▼5.6× 3m 53.1s ▼2.2×	8m 16.7s ▼4.6× 3m 46.2s ▼2.2×	·	·
R48	MPS / Scalar Scalar-DP / KSW2	14.5s ▲2.5×[s] 10.1s ▲3.5×[s]	14.3s ▲2.7×[s] 9.5s ▲3.8×[s]	·	·	7m 42s ▼14.6×[s] 58.8s ▼1.9×[s]	6m 31.7s ▼11.5×[s] 54.5s ▼1.7×[s]	·	·	5m 38.2s ▼1.9×[s] 5m 24.8s ▼1.9×[s]	·	·	·	12m 12s ▼6.8×[s] 5m 47s ▼3.3×[s]	·	·	·
R49	MPS / SIMD-ref SIMD-DP / Block	15.4s ▲2.3×[s] 14.5s ▲2.4×[s]	15.1s ▲2.5×[s] 14.2s ▲2.5×[s]	·	·	6m 17.1s ▼11.9×[s] 42.5s ▼1.4×[s]	5m 21.5s ▼9.4×[s] 37.9s ▼18%[s]	·	·	·	·	8m 12.5s ▼1.5× 7m 55.2s ▼1.4×	·	·	·	·	·
R50 BioseqDB	Vulkan (all modes)	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Minimap2 stays first. Other rows follow increasing minima of available Index A times within this size/k/w group, across alphabets and all displayed machines. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 4 Gbp k=20, w=15								No index 4 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	2m 48.1s ▼20.4× 2m 41.7s ▼22.6×	2m 50.3s ▼19.8× 2m 43s ▼22.2×	4m 54.3s ▼43.2× 4m 39s ▼54.6×	25m 5.5s ▼41.8× 25m 0.7s ▼46.6×	33.9s ▼2.3× 32.9s ▼3.1×	34.4s ▼2.6× 32.7s ▼2.9×	1m 8.2s ▼3.7× 1m 5.1s ▼4.6×	3m 9.8s ▼3.9× 3m 3.7s ▼4.1×	3m 28.3s ▼1.3× 3m 22.8s ▼1.4×	3m 53.2s ▼1.4× 3m 43.5s ▼1.5×	7m 14.6s ▼1.5× 6m 57.2s ▼1.6×	51m 22s n/a 51m 17.2s n/a	1m 10.3s ▲7.6× 1m 9s ▲2.4×	1m 8.6s ▲6.4× 1m 6.9s ▲2.3×	2m 21.5s ▲6.0× 2m 18.4s ▲2.2×	15m 56.8s n/a 15m 51.9s n/a
R2 BioseqDB	MPF / Bounded SIMD-DP / Block	9.2s ▲18.3× 8.5s ▲19.0×	11.4s ▲15.0× 10.7s ▲15.3×	6.8s ▲43.2× 5.1s ▲54.6×	39.1s ▲38.5×[s] 38s ▲39.5×[s]	18.2s ▲1.9× 16s ▲2.1×	13.2s ▲2.6× 11.4s ▲2.9×	18.6s ▲3.7× 14.3s ▲4.6×	57.2s ▲3.3×[s] 53.9s ▲3.4×[s]	2m 38.5s ▲24% 2m 29.8s ▲26%	2m 49s ▲28% 2m 37s ▲30%	5m 3.3s ▲1.4× 4m 42.7s ▲1.5×	·	8m 52.3s ▼7.6× 2m 45.5s ▼2.4×	7m 22.2s ▼6.4× 2m 48.2s ▼2.5×	14m 43.3s ▼6.2× 5m 9.5s ▼2.2×	·
R3	MPF / Bounded Scalar-DP / Block	10.7s ▲15.7× 10.1s ▲16.1×	11.5s ▲14.8× 10.8s ▲15.2×	7.6s ▲38.5×[s] 5.4s ▲51.4×[s]	36.6s ▲41.1×[s] 35.5s ▲42.2×[s]	17s ▲2.0×[s] 12.8s ▲2.6×[s]	17.3s ▲2.0×[s] 14.1s ▲2.3×[s]	23s ▲3.0×[s] 15.8s ▲4.1×[s]	51.2s ▲3.7×[s] 47.8s ▲3.8×[s]	2m 50s ▲18% 2m 41s ▲21%	2m 48.3s ▲28% 2m 35.9s ▲1.4×	4m 47.5s ▲1.5× 4m 26.9s ▲1.6×	·	9m 21.4s ▼8.0×[s] 3m 11.2s ▼2.8×[s]	7m 45.7s ▼6.8×[s] 2m 38.2s ▼2.4×[s]	·	·
R4	MPF / Bounded RMQ / Block	8.3s ▲20.4× 7.6s ▲21.3×	8.6s ▲19.8× 7.8s ▲20.8×	35s ▲8.4×[s] 32.9s ▲8.5×[s]	38.3s ▲39.3×[s] 37.2s ▲40.3×[s]	35.9s ▼6%[s] 31.8s ▶ 3%[s]	33.2s ▶ 4%[s] 30.1s ▲8%[s]	46.2s ▲1.5×[s] 39.4s ▲1.7×[s]	49.2s ▲3.9×[s] 45.8s ▲4.0×[s]	2m 53.8s ▲17% 2m 43.7s ▲19%	3m 4.5s ▲21% 2m 52.1s ▲23%	·	·	9m 6.6s ▼7.8×[s] 2m 56.5s ▼2.6×[s]	7m 59.6s ▼7.0×[s] 2m 51.2s ▼2.6×[s]	·	·
R5	MPF / Bounded SIMD-DP / KSW2	9.6s ▲17.6× 7.1s ▲22.6×	9.6s ▲17.7× 7.3s ▲22.2×	38.3s ▲7.7×[s] 30.9s ▲9.0×[s]	40.5s ▲37.2×[s] 36.4s ▲41.2×[s]	34.6s ▶ 2%[s] 30.4s ▲8%[s]	21.3s ▲1.6×[s] 17.7s ▲1.8×[s]	37s ▲1.8×[s] 29.3s ▲2.2×[s]	55.9s ▲3.4×[s] 52s ▲3.5×[s]	2m 51.2s ▲18% 2m 39.2s ▲21%	3m 0s ▲23% 2m 45.6s ▲26%	·	·	9m 4.2s ▼7.7×[s] 2m 55.2s ▼2.5×[s]	7m 44.4s ▼6.8×[s] 2m 37.2s ▼2.3×[s]	·	·
R6	MPF / Bounded RMQ / KSW2	10.6s ▲15.9× 8.3s ▲19.5×	14.6s ▲11.7× 12.5s ▲13.0×	12.5s ▲23.5×[s] 5.1s ▲54.4×[s]	36s ▲41.8×[s] 32.2s ▲46.6×[s]	26.8s ▲21%[s] 22.7s ▲1.5×[s]	34.6s ▶ 1%[s] 31.2s ▶ 4%[s]	26s ▲2.6×[s] 18.5s ▲3.5×[s]	53.3s ▲3.6×[s] 49.4s ▲3.7×[s]	2m 51.8s ▲18% 2m 40.6s ▲21%	2m 50.2s ▲27% 2m 36.4s ▲1.4×	·	·	9m 12.4s ▼7.9×[s] 2m 59.4s ▼2.6×[s]	7m 51.1s ▼6.9×[s] 2m 41.8s ▼2.4×[s]	·	·
R7	MPS / Bounded SIMD-DP / Block	11.7s ▲14.3× 11s ▲14.7×	11.9s ▲14.3× 11.2s ▲14.6×	·	·	8m 5.8s ▼14.3×[s] 38.1s ▼16%[s]	5m 56.7s ▼10.4×[s] 35.4s ▼8%[s]	·	·	2m 43.8s ▲21% 2m 34.7s ▲24%	2m 58.9s ▲23% 2m 46.3s ▲26%	·	·	9m 21.2s ▼8.0×[s] 3m 2.3s ▼2.6×[s]	8m 1.2s ▼7.0×[s] 2m 44.6s ▼2.5×[s]	·	·
R8	MPS / Bounded RMQ / Block	11.7s ▲14.3× 11s ▲14.6×	11.8s ▲14.5× 11.1s ▲14.7×	·	·	7m 41.1s ▼13.6×[s] 35.7s ▼8%[s]	5m 57.8s ▼10.4×[s] 34.4s ▼5%[s]	·	·	2m 43.8s ▲21% 2m 34.7s ▲24%	2m 52.1s ▲26% 2m 39.6s ▲29%	·	·	9m 17.3s ▼7.9×[s] 2m 59.8s ▼2.6×[s]	·	14m 16s ▼6.0× 5m 26.4s ▼2.4×	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Minimap2 stays first. Other rows follow increasing minima of available Index A times within this size/k/w group, across alphabets and all displayed machines. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 4 Gbp k=20, w=15								No index 4 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	2m 48.1s ▼20.4× 2m 41.7s ▼22.6×	2m 50.3s ▼19.8× 2m 43s ▼22.2×	4m 54.3s ▼43.2× 4m 39s ▼54.6×	25m 5.5s ▼41.8× 25m 0.7s ▼46.6×	33.9s ▼2.3× 32.9s ▼3.1×	34.4s ▼2.6× 32.7s ▼2.9×	1m 8.2s ▼3.7× 1m 5.1s ▼4.6×	3m 9.8s ▼3.9× 3m 3.7s ▼4.1×	3m 28.3s ▼1.3× 3m 22.8s ▼1.4×	3m 53.2s ▼1.4× 3m 43.5s ▼1.5×	7m 14.6s ▼1.5× 6m 57.2s ▼1.6×	51m 22s n/a 51m 17.2s n/a	1m 10.3s ▲7.6× 1m 9s ▲2.4×	1m 8.6s ▲6.4× 1m 6.9s ▲2.3×	2m 21.5s ▲6.0× 2m 18.4s ▲2.2×	15m 56.8s n/a 15m 51.9s n/a
R9 BioseqDB	MPS / Bounded Scalar-DP / Block	11.8s ▲14.2× 11.1s ▲14.6×	12.7s ▲13.4× 11.9s ▲13.7×	·	·	7m 27.3s ▼13.2×[s] 36.3s ▼10%[s]	5m 57.5s ▼10.4×[s] 34.3s ▶ 5%[s]	·	·	2m 52.6s ▲17% 2m 42.8s ▲20%	2m 58.9s ▲23% 2m 46.9s ▲25%	·	·	9m 15.5s ▼7.9×[s] 3m 0.5s ▼2.6×[s]	·	·	·
R10	MPF / Bounded Scalar-DP / KSW2	14.5s ▲11.6× 12.2s ▲13.3×	14.2s ▲12.0× 11.9s ▲13.7×	12.7s ▲23.2×[s] 5.2s ▲53.2×[s]	36.8s ▲40.9×[s] 32.8s ▲45.7×[s]	14.8s ▲2.3×[s] 10.7s ▲3.1×[s]	17.7s ▲1.9×[s] 14.3s ▲2.3×[s]	27s ▲2.5×[s] 19.4s ▲3.4×[s]	48.6s ▲3.9×[s] 44.8s ▲4.1×[s]	2m 40.6s ▲23% 2m 29.9s ▲26%	3m 2.9s ▲22% 2m 48.3s ▲25%	·	·	9m 11s ▼7.8×[s] 2m 56.6s ▼2.6×[s]	7m 47.4s ▼6.8×[s] 2m 37.9s ▼2.4×[s]	·	·
R11	MPS / Bounded RMQ / KSW2	13.5s ▲12.4× 11s ▲14.7×	12.8s ▲13.3× 10.6s ▲15.4×	·	·	7m 20.4s ▼13.0×[s] 35.7s ▼8%[s]	6m 8.8s ▼10.7×[s] 34.6s ▼6%[s]	·	·	2m 56.8s ▲15% 2m 44.8s ▲19%	3m 11.4s ▲18% 2m 56.8s ▲21%	·	·	9m 19.1s ▼8.0×[s] 3m 4s ▼2.7×[s]	·	·	·
R12	MPS / Bounded Scalar-DP / KSW2	13.4s ▲12.6× 10.9s ▲14.9×	13s ▲13.1× 10.7s ▲15.2×	·	·	7m 44.9s ▼13.7×[s] 35.9s ▼9%[s]	5m 59.4s ▼10.4×[s] 34s ▶ 4%[s]	·	·	2m 59.1s ▲14% 2m 47.7s ▲17%	2m 50.8s ▲27% 2m 36.9s ▲30%	·	·	9m 18.6s ▼8.0×[s] 3m 3.8s ▼2.7×[s]	·	·	·
R13	MPS / Bounded SIMD-DP / KSW2	13.3s ▲12.7× 10.8s ▲15.0×	13.6s ▲12.5× 11.1s ▲14.7×	·	·	7m 20.3s ▼13.0×[s] 35.3s ▼7%[s]	5m 56.2s ▼10.3×[s] 34s ▶ 4%[s]	·	·	2m 59s ▲14% 2m 48.5s ▲17%	2m 52.5s ▲26% 2m 38.8s ▲29%	·	·	9m 21.1s ▼8.0×[s] 3m 3.5s ▼2.7×[s]	8m 2.9s ▼7.0×[s] 2m 44s ▼2.5×[s]	·	·
R14	MPS / Two-stack SIMD-DP / Block	51.6s ▲3.3× 9.8s ▲16.6×	44.7s ▲3.8× 9.6s ▲17.0×	1m 16.1s ▲3.9× 15.7s ▲17.7×	·	·	·	·	·	·	·	·	·	·	·	·	·
R15	MPS / Two-stack RMQ / Block	50.7s ▲3.3× 9.4s ▲17.2×	44.9s ▲3.8× 9.9s ▲16.5×	·	·	·	·	·	·	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash. Minimap2 stays first. Other rows follow increasing minima of available Index A times within this size/k/w group, across alphabets and all displayed machines. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 4 Gbp k=20, w=15								No index 4 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	2m 48.1s ▼20.4× 2m 41.7s ▼22.6×	2m 50.3s ▼19.8× 2m 43s ▼22.2×	4m 54.3s ▼43.2× 4m 39s ▼54.6×	25m 5.5s ▼41.8× 25m 0.7s ▼46.6×	33.9s ▼2.3× 32.9s ▼3.1×	34.4s ▼2.6× 32.7s ▼2.9×	1m 8.2s ▼3.7× 1m 5.1s ▼4.6×	3m 9.8s ▼3.9× 3m 3.7s ▼4.1×	3m 28.3s ▼1.3× 3m 22.8s ▼1.4×	3m 53.2s ▼1.4× 3m 43.5s ▼1.5×	7m 14.6s ▼1.5× 6m 57.2s ▼1.6×	51m 22s n/a 51m 17.2s n/a	1m 10.3s ▲7.6× 1m 9s ▲2.4×	1m 8.6s ▲6.4× 1m 6.9s ▲2.3×	2m 21.5s ▲6.0× 2m 18.4s ▲2.2×	15m 56.8s n/a 15m 51.9s n/a
R16 BioseqDB	MPS / Two-stack RMQ / KSW2	53.5s ▲3.1× 9.3s ▲17.4×	47.1s ▲3.6×[s] 9s ▲18.2×[s]	·	·	·	·	·	·	·	·	·	·	·	·	·	·
R17	MPS / Two-stack SIMD-DP / KSW2	55.3s ▲3.0× 9.6s ▲16.8×	48.6s ▲3.5× 9.6s ▲17.0×	·	·	·	·	·	·	·	·	·	·	·	·	·	·
R18	MPS / Two-stack Scalar-DP / KSW2	57.2s ▲2.9×[s] 9.8s ▲16.5×[s]	49.4s ▲3.4×[s] 9.2s ▲17.8×[s]	·	·	·	·	·	·	·	·	·	·	·	·	·	·
R19	MPS / Scalar SIMD-DP / Block	57s ▲3.0× 14.3s ▲11.3×	50.6s ▲3.4× 15.1s ▲10.8×	·	·	7m 14.5s ▼12.8×[s] 41.9s ▼27%[s]	6m 19.3s ▼11.0×[s] 41.3s ▼26%[s]	·	·	5m 10.1s ▼1.5× 4m 27.6s ▼1.3×	5m 3.5s ▼1.3× 4m 28.9s ▼20%	·	·	·	·	·	·
R20	MPS / SIMD-ref Scalar-DP / Block	1m 1.1s ▲2.7×[s] 14.9s ▲10.8×[s]	50.8s ▲3.4×[s] 14.6s ▲11.2×[s]	·	·	7m 13.1s ▼12.8×[s] 52.7s ▼1.6×[s]	6m 7.7s ▼10.7×[s] 50.4s ▼1.5×[s]	·	·	·	·	·	·	·	·	·	·
R21	MPS / SIMD-ref RMQ / Block	1m 0.8s ▲2.8×[s] 15s ▲10.8×[s]	50.9s ▲3.3×[s] 14.7s ▲11.1×[s]	·	·	7m 13.3s ▼12.8×[s] 53.4s ▼1.6×[s]	6m 11.5s ▼10.8×[s] 54s ▼1.7×[s]	·	·	·	·	·	·	·	·	·	·
R22	MPS / Scalar RMQ / Block	57.4s ▲2.9× 14.6s ▲11.1×	51.1s ▲3.3× 15.9s ▲10.3×	·	·	7m 2.9s ▼12.5×[s] 38.9s ▼18%[s]	6m 25.9s ▼11.2×[s] 44s ▼1.3×[s]	·	·	5m 10.6s ▼1.5× 4m 28.1s ▼1.3×	5m 37s ▼1.4× 5m 0.7s ▼1.3×	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash. Minimap2 stays first. Other rows follow increasing minima of available Index A times within this size/k/w group, across alphabets and all displayed machines. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 4 Gbp k=20, w=15								No index 4 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	2m 48.1s ▼20.4× 2m 41.7s ▼22.6×	2m 50.3s ▼19.8× 2m 43s ▼22.2×	4m 54.3s ▼43.2× 4m 39s ▼54.6×	25m 5.5s ▼41.8× 25m 0.7s ▼46.6×	33.9s ▼2.3× 32.9s ▼3.1×	34.4s ▼2.6× 32.7s ▼2.9×	1m 8.2s ▼3.7× 1m 5.1s ▼4.6×	3m 9.8s ▼3.9× 3m 3.7s ▼4.1×	3m 28.3s ▼1.3× 3m 22.8s ▼1.4×	3m 53.2s ▼1.4× 3m 43.5s ▼1.5×	7m 14.6s ▼1.5× 6m 57.2s ▼1.6×	51m 22s n/a 51m 17.2s n/a	1m 10.3s ▲7.6× 1m 9s ▲2.4×	1m 8.6s ▲6.4× 1m 6.9s ▲2.3×	2m 21.5s ▲6.0× 2m 18.4s ▲2.2×	15m 56.8s n/a 15m 51.9s n/a
R23 BioseqDB	MPS / SIMD-ref SIMD-DP / Block	1m 5.1s ▲2.6×[s] 15s ▲10.8×[s]	51.5s ▲3.3×[s] 14.6s ▲11.1×[s]	·	·	7m 16s ▼12.8×[s] 56.4s ▼1.7×[s]	6m 11.8s ▼10.8×[s] 53s ▼1.6×[s]	·	·	·	·	6m 45.8s ▲7% 5m 46.6s ▲17%	·	·	·	·	·
R24	MPS / Scalar Scalar-DP / Block	1m 0s ▲2.8×[s] 14.7s ▲11.0×[s]	51.7s ▲3.3×[s] 14.7s ▲11.1×[s]	·	·	7m 3s ▼12.5×[s] 38.5s ▼17%[s]	6m 18s ▼11.0×[s] 41.1s ▼26%[s]	·	·	5m 22.6s ▼1.5×[s] 4m 36.5s ▼1.4×[s]	·	·	·	·	·	·	·
R25	MPS / Two-stack Scalar-DP / Block	56.5s ▲3.0×[s] 9.7s ▲16.7×[s]	51.7s ▲3.3×[s] 9.9s ▲16.5×[s]	·	·	·	·	·	·	·	·	·	·	·	·	·	·
R26	MPS / SIMD-ref SIMD-DP / KSW2	1m 2.9s ▲2.7×[s] 14.9s ▲10.8×[s]	52s ▲3.3×[s] 14.4s ▲11.3×[s]	·	·	7m 15.8s ▼12.8×[s] 53.3s ▼1.6×[s]	6m 10.8s ▼10.8×[s] 51.3s ▼1.6×[s]	·	·	·	·	·	·	·	·	·	·
R27	MPS / Scalar SIMD-DP / KSW2	58.6s ▲2.9× 14.4s ▲11.2×	52.3s ▲3.3× 15.4s ▲10.6×	·	·	7m 14.6s ▼12.8×[s] 39s ▼19%[s]	6m 25.1s ▼11.2×[s] 39s ▼19%[s]	·	·	5m 10.5s ▼1.5× 4m 25.5s ▼1.3×	4m 45.4s ▼22% 4m 9.3s ▼12%	·	·	·	·	·	·
R28	MPS / Scalar Scalar-DP / KSW2	1m 1.8s ▲2.7×[s] 14.7s ▲11.0×[s]	52.7s ▲3.2×[s] 14.6s ▲11.2×[s]	·	·	7m 1.1s ▼12.4×[s] 38.9s ▼18%[s]	6m 4.9s ▼10.6×[s] 39.9s ▼22%[s]	·	·	5m 9.5s ▼1.5×[s] 4m 23s ▼30%[s]	·	·	·	·	·	·	·
R29	MPS / SIMD-ref RMQ / KSW2	1m 2.5s ▲2.7×[s] 15s ▲10.8×[s]	53.6s ▲3.2×[s] 15.3s ▲10.7×[s]	·	·	7m 16s ▼12.8×[s] 53.4s ▼1.6×[s]	6m 11.5s ▼10.8×[s] 52.1s ▼1.6×[s]	·	·	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash. Minimap2 stays first. Other rows follow increasing minima of available Index A times within this size/k/w group, across alphabets and all displayed machines. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Table 12.25: (C): AWS: 4 Gbp, k=20, w=15 (mapping interval)

Engine	Execution	Index 4 Gbp k=20, w=15 DNA4 M7A C1	Index 4 Gbp k=20, w=15 DNA4 M7I C2	Index 4 Gbp k=20, w=15 DNA4 M4 C3	Index 4 Gbp k=20, w=15 DNA4 C7G C4	Index 4 Gbp k=20, w=15 DNA5 M7A C5	Index 4 Gbp k=20, w=15 DNA5 M7I C6	Index 4 Gbp k=20, w=15 DNA5 M4 C7	Index 4 Gbp k=20, w=15 DNA5 C7G C8	No index 4 Gbp k=20, w=15 DNA4 M7A C9	No index 4 Gbp k=20, w=15 DNA4 M7I C10	No index 4 Gbp k=20, w=15 DNA4 M4 C11	No index 4 Gbp k=20, w=15 DNA4 C7G C12	No index 4 Gbp k=20, w=15 DNA5 M7A C13	No index 4 Gbp k=20, w=15 DNA5 M7I C14	No index 4 Gbp k=20, w=15 DNA5 M4 C15	No index 4 Gbp k=20, w=15 DNA5 C7G C16
R1 minimap2	Baseline	2m 48.1s ▼20.4× 2m 41.7s ▼22.6×	2m 50.3s ▼19.8× 2m 43s ▼22.2×	4m 54.3s ▼43.2× 4m 39s ▼54.6×	25m 5.5s ▼41.8× 25m 0.7s ▼46.6×	33.9s ▼2.3× 32.9s ▼3.1×	34.4s ▼2.6× 32.7s ▼2.9×	1m 8.2s ▼3.7× 1m 5.1s ▼4.6×	3m 9.8s ▼3.9× 3m 3.7s ▼4.1×	3m 28.3s ▼1.3× 3m 22.8s ▼1.4×	3m 53.2s ▼1.4× 3m 43.5s ▼1.5×	7m 14.6s ▼1.5× 6m 57.2s ▼1.6×	51m 22s n/a 51m 17.2s n/a	1m 10.3s ▲7.6× 1m 9s ▲2.4×	1m 8.6s ▲6.4× 1m 6.9s ▲2.3×	2m 21.5s ▲6.0× 2m 18.4s ▲2.2×	15m 56.8s n/a 15m 51.9s n/a
R30 BioseqDB	MPS / SIMD-ref Scalar-DP / KSW2	1m 2.8s ▲2.7×[s] 15s ▲10.8×[s]	53.9s ▲3.2×[s] 15.2s ▲10.7×[s]	·	·	7m 17.8s ▼12.9×[s] 53.3s ▼1.6×[s]	6m 9.3s ▼10.7×[s] 50.2s ▼1.5×[s]	·	·	·	·	·	·	·	·	·	·
R31	MPS / Scalar RMQ / KSW2	59.3s ▲2.8× 14.7s ▲11.0×	56.8s ▲3.0×[s] 14.7s ▲11.1×[s]	·	·	7m 5s ▼12.5×[s] 39.2s ▼19%[s]	6m 26.9s ▼11.2×[s] 41.2s ▼26%[s]	·	·	5m 14.2s ▼1.5×[s] 4m 26.6s ▼1.3×[s]	·	·	·	·	·	·	·
R32	MPF / Two-stack RMQ / KSW2	9m 9.8s ▼3.3× 9m 7.7s ▼3.4×	15m 43.9s ▼5.5×[s] 15m 40.9s ▼5.8×[s]	33m 26.8s ▼6.8×[s] 33m 20.7s ▼7.2×[s]	·	·	·	·	·	3m 22.2s ▶ 3%[s] 2m 35.6s ▲23%[s]	3m 3.8s ▲21%[s] 2m 25s ▲1.5×[s]	·	·	·	·	·	·
R33	MPF / Two-stack RMQ / Block	9m 11s ▼3.3× 9m 10.5s ▼3.4×	13m 20.9s ▼4.7× 13m 20.1s ▼4.9×	33m 23s ▼6.8×[s] 33m 21.5s ▼7.2×[s]	·	·	·	·	·	3m 21.7s ▶ 3%[s] 2m 38.8s ▲22%[s]	3m 0.8s ▲22%[s] 2m 25s ▲1.5×[s]	·	·	·	·	·	·
R34	MPF / Two-stack SIMD-DP / Block	9m 11.2s ▼3.3× 9m 10.7s ▼3.4×	15m 34.3s ▼5.5×[s] 15m 33.4s ▼5.7×[s]	33m 48.6s ▼6.9×[s] 33m 47.1s ▼7.3×[s]	·	·	·	·	·	3m 19s ▶ 4%[s] 2m 36.1s ▲23%[s]	3m 6.1s ▲20%[s] 2m 30.7s ▲1.5×[s]	·	·	·	·	·	·
R35	MPF / Two-stack SIMD-DP / KSW2	9m 11.4s ▼3.3× 9m 9.5s ▼3.4×	13m 22s ▼4.7× 13m 19.3s ▼4.9×	33m 35.9s ▼6.9×[s] 33m 29.9s ▼7.2×[s]	·	·	·	·	·	3m 21.5s ▶ 3%[s] 2m 35s ▲24%[s]	3m 4.9s ▲21%[s] 2m 25.8s ▲1.5×[s]	·	·	·	·	·	·
R36	MPF / Two-stack Scalar-DP / Block	9m 15.8s ▼3.3×[s] 9m 15s ▼3.4×[s]	15m 26.5s ▼5.4×[s] 15m 25.5s ▼5.7×[s]	33m 22.2s ▼6.8×[s] 33m 20.6s ▼7.2×[s]	·	·	·	·	·	3m 18.4s ▶ 5%[s] 2m 36s ▲23%[s]	3m 4.1s ▲21%[s] 2m 28.5s ▲1.5×[s]	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash. Minimap2 stays first. Other rows follow increasing minima of available Index A times within this size/k/w group, across alphabets and all displayed machines. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 4 Gbp k=20, w=15								No index 4 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	2m 48.1s ▼20.4× 2m 41.7s ▼22.6×	2m 50.3s ▼19.8× 2m 43s ▼22.2×	4m 54.3s ▼43.2× 4m 39s ▼54.6×	25m 5.5s ▼41.8× 25m 0.7s ▼46.6×	33.9s ▼2.3× 32.9s ▼3.1×	34.4s ▼2.6× 32.7s ▼2.9×	1m 8.2s ▼3.7× 1m 5.1s ▼4.6×	3m 9.8s ▼3.9× 3m 3.7s ▼4.1×	3m 28.3s ▼1.3× 3m 22.8s ▼1.4×	3m 53.2s ▼1.4× 3m 43.5s ▼1.5×	7m 14.6s ▼1.5× 6m 57.2s ▼1.6×	51m 22s n/a 51m 17.2s n/a	1m 10.3s ▲7.6× 1m 9s ▲2.4×	1m 8.6s ▲6.4× 1m 6.9s ▲2.3×	2m 21.5s ▲6.0× 2m 18.4s ▲2.2×	15m 56.8s n/a 15m 51.9s n/a
R37 BioseqDB	MPF / Two-stack Scalar-DP / KSW2	9m 16.3s ▼3.3×[s] 9m 13.5s ▼3.4×[s]	16m 8.5s ▼5.7×[s] 16m 5.1s ▼5.9×[s]	33m 26.6s ▼6.8×[s] 33m 20.5s ▼7.2×[s]	·	·	·	·	·	3m 21.4s ▶ 3%[s] 2m 35s ▲24%[s]	3m 4.9s ▲21%[s] 2m 25.7s ▲1.5×[s]	·	·	·	·	·	·
R38	MPF / Scalar Scalar-DP / KSW2	11m 11.3s ▼4.0×[s] 11m 7.6s ▼4.1×[s]	24m 35.8s ▼8.7×[s] 24m 32.7s ▼9.0×[s]	49m 58.6s ▼10.2×[s] 49m 52.4s ▼10.7×[s]	3h 11m 57.8s ▼7.7×[s] 3h 11m 53.5s ▼7.7×[s]	16m 14.9s ▼28.7×[s] 16m 11.2s ▼29.5×[s]	38m 2.4s ▼66.3×[s] 37m 59.5s ▼69.7×[s]	1h 11m 44.1s ▼63.1×[s] 1h 11m 37.9s ▼66.0×[s]	2h 23m 50.8s ▼45.5×[s] 2h 23m 46.4s ▼47.0×[s]	5m 8.2s ▼1.5×[s] 4m 20.4s ▼28%[s]	4m 17.9s ▼11%[s] 3m 42.6s ▶ 0%[s]	·	·	12m 31.3s ▼10.7×[s] 4m 43.5s ▼4.1×[s]	11m 42.9s ▼10.2×[s] 5m 21.5s ▼4.8×[s]	·	·
R39	MPF / Scalar Scalar-DP / Block	11m 13.8s ▼4.0×[s] 11m 13s ▼4.2×[s]	26m 53.1s ▼9.5×[s] 26m 52.3s ▼9.9×[s]	50m 56.1s ▼10.4×[s] 50m 54.6s ▼10.9×[s]	3h 12m 16.6s ▼7.7×[s] 3h 12m 14.4s ▼7.7×[s]	14m 50.9s ▼26.2×[s] 14m 47.2s ▼26.9×[s]	29m 53.2s ▼52.1×[s] 29m 50.6s ▼54.7×[s]	1h 12m 1.2s ▼63.4×[s] 1h 11m 55.6s ▼66.3×[s]	2h 25m 29.9s ▼46.0×[s] 2h 25m 25.6s ▼47.5×[s]	5m 21.8s ▼1.5×[s] 4m 34.8s ▼1.4×[s]	4m 17.1s ▼10%[s] 3m 43.8s ▶ 0%[s]	·	·	12m 28.5s ▼10.7×[s] 4m 39.8s ▼4.1×[s]	11m 56.7s ▼10.4×[s] 5m 31.3s ▼5.0×[s]	·	·
R40	MPF / Scalar SIMD-DP / KSW2	11m 14.3s ▼4.0× 11m 12.2s ▼4.2×	24m 37.5s ▼8.7× 24m 35.6s ▼9.1×	44m 15.3s ▼9.0×[s] 44m 9.6s ▼9.5×[s]	1h 34m 25.1s ▼3.8×[s] 1h 34m 20.6s ▼3.8×[s]	14m 52.8s ▼26.3×[s] 14m 49.1s ▼27.0×[s]	32m 52.9s ▼57.3×[s] 32m 50s ▼60.2×[s]	1h 8m 24.8s ▼60.2×[s] 1h 8m 18.7s ▼62.9×[s]	1h 26m 14.3s ▼27.3×[s] 1h 26m 9.8s ▼28.1×[s]	4m 53s ▼1.4× 4m 8.2s ▼22%	5m 9.3s ▼1.3× 4m 31.7s ▼22%	·	·	12m 34.5s ▼10.7×[s] 4m 44.3s ▼4.1×[s]	11m 58.3s ▼10.5×[s] 5m 31.6s ▼5.0×[s]	·	·
R41	MPF / Scalar RMQ / KSW2	11m 14.6s ▼4.0× 11m 12.5s ▼4.2×	25m 12.4s ▼8.9× 25m 10.5s ▼9.3×	53m 45.8s ▼11.0×[s] 53m 39.5s ▼11.5×[s]	3h 11m 29.7s ▼7.6×[s] 3h 11m 25.2s ▼7.7×[s]	15m 59.1s ▼28.3×[s] 15m 55.5s ▼29.0×[s]	29m 55.2s ▼52.2×[s] 29m 52.6s ▼54.8×[s]	1h 11m 28.9s ▼62.9×[s] 1h 11m 22.8s ▼65.8×[s]	1h 56m 30.8s ▼36.8×[s] 1h 56m 26.1s ▼38.0×[s]	5m 17.4s ▼1.5×[s] 4m 30.6s ▼1.3×[s]	4m 16.1s ▼10%[s] 3m 41.1s ▶ 1%[s]	·	·	12m 25.5s ▼10.6×[s] 4m 33.9s ▼4.0×[s]	12m 4.7s ▼10.6×[s] 5m 35.2s ▼5.0×[s]	·	·
R42	MPF / SIMD-ref RMQ / Block	11m 19.5s ▼4.0×[s] 11m 18.8s ▼4.2×[s]	23m 45.6s ▼8.4×[s] 23m 44.9s ▼8.7×[s]	45m 51.5s ▼9.3×[s] 45m 50.2s ▼9.9×[s]	2h 47m 50.9s ▼6.7×[s] 2h 47m 48.7s ▼6.7×[s]	12m 10.8s ▼21.5×[s] 12m 7.1s ▼22.1×[s]	31m 7.5s ▼54.3×[s] 31m 4.5s ▼57.0×[s]	47m 1.1s ▼41.4×[s] 46m 55.3s ▼43.2×[s]	·	4m 39.8s ▼1.3×[s] 3m 55.2s ▼16%[s]	4m 28.8s ▼15%[s] 3m 52s ▶ 4%[s]	·	·	10m 49.3s ▼9.2×[s] 4m 37.1s ▼4.0×[s]	10m 38.4s ▼9.3×[s] 5m 1.8s ▼4.5×[s]	·	·
R43	MPF / SIMD-ref RMQ / KSW2	11m 21.2s ▼4.1×[s] 11m 17.6s ▼4.2×[s]	23m 50.3s ▼8.4×[s] 23m 47.4s ▼8.8×[s]	47m 6.3s ▼9.6×[s] 47m 0.3s ▼10.1×[s]	3h 11m 12.9s ▼7.6×[s] 3h 11m 8.5s ▼7.6×[s]	12m 7.2s ▼21.4×[s] 12m 3.6s ▼22.0×[s]	25m 37s ▼44.7×[s] 25m 34.2s ▼46.9×[s]	47m 19.6s ▼41.6×[s] 47m 13.3s ▼43.5×[s]	·	4m 43.9s ▼1.4×[s] 3m 57.8s ▼17%[s]	4m 55.1s ▼27%[s] 4m 16.8s ▼15%[s]	·	·	10m 52.3s ▼9.3×[s] 4m 28.1s ▼3.9×[s]	10m 29.8s ▼9.2×[s] 4m 56.7s ▼4.4×[s]	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Minimap2 stays first. Other rows follow increasing minima of available Index A times within this size/k/w group, across alphabets and all displayed machines. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Table 12.25: (C): AWS: 4 Gbp, k=20, w=15 (mapping interval)

Engine	Execution	Index 4 Gbp k=20, w=15								No index 4 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	2m 48.1s ▼20.4× 2m 41.7s ▼22.6×	2m 50.3s ▼19.8× 2m 43s ▼22.2×	4m 54.3s ▼43.2× 4m 39s ▼54.6×	25m 5.5s ▼41.8× 25m 0.7s ▼46.6×	33.9s ▼2.3× 32.9s ▼3.1×	34.4s ▼2.6× 32.7s ▼2.9×	1m 8.2s ▼3.7× 1m 5.1s ▼4.6×	3m 9.8s ▼3.9× 3m 3.7s ▼4.1×	3m 28.3s ▼1.3× 3m 22.8s ▼1.4×	3m 53.2s ▼1.4× 3m 43.5s ▼1.5×	7m 14.6s ▼1.5× 6m 57.2s ▼1.6×	51m 22s n/a 51m 17.2s n/a	1m 10.3s ▲7.6× 1m 9s ▲2.4×	1m 8.6s ▲6.4× 1m 6.9s ▲2.3×	2m 21.5s ▲6.0× 2m 18.4s ▲2.2×	15m 56.8s n/a 15m 51.9s n/a
R44 BioseqDB	MPF / SIMD-ref SIMD-DP / KSW2	11m 21.3s ▼4.1×[s] 11m 17.7s ▼4.2×[s]	24m 2.7s ▼8.5×[s] 23m 59.7s ▼8.8×[s]	46m 8s ▼9.4×[s] 46m 1.9s ▼9.9×[s]	1h 35m 48.4s ▼3.8×[s] 1h 35m 44.2s ▼3.8×[s]	12m 1.2s ▼21.2×[s] 11m 57.5s ▼21.8×[s]	35m 19s ▼61.6×[s] 35m 15.4s ▼64.7×[s]	46m 59.7s ▼41.3×[s] 46m 53.4s ▼43.2×[s]	·	4m 42.6s ▼1.4×[s] 3m 56.3s ▼17%[s]	4m 36.4s ▼19%[s] 3m 58.1s ▼7%[s]	·	·	11m 3.4s ▼9.4×[s] 4m 43.3s ▼4.1×[s]	10m 45.6s ▼9.4×[s] 5m 9.2s ▼4.6×[s]	·	·
R45	MPF / SIMD-ref Scalar-DP / Block	11m 22.1s ▼4.1×[s] 11m 21.3s ▼4.2×[s]	24m 22.6s ▼8.6×[s] 24m 21.8s ▼9.0×[s]	46m 17.2s ▼9.4×[s] 46m 15.7s ▼9.9×[s]	2h 59m 10.5s ▼7.1×[s] 2h 59m 8.2s ▼7.2×[s]	12m 25.5s ▼22.0×[s] 12m 21.9s ▼22.5×[s]	25m 32.6s ▼44.5×[s] 25m 30s ▼46.8×[s]	47m 21.3s ▼41.7×[s] 47m 15.6s ▼43.5×[s]	·	4m 45.1s ▼1.4×[s] 4m 0.5s ▼19%[s]	4m 33.4s ▼17%[s] 3m 57.3s ▼6%[s]	·	·	11m 4.3s ▼9.5×[s] 4m 42.4s ▼4.1×[s]	10m 27.9s ▼9.2×[s] 4m 57.1s ▼4.4×[s]	·	·
R46	MPF / Scalar RMQ / Block	11m 22.5s ▼4.1× 11m 22s ▼4.2×	24m 52.9s ▼8.8× 24m 52.4s ▼9.2×	46m 1.7s ▼9.4×[s] 46m 0.4s ▼9.9×[s]	2h 57m 27.5s ▼7.1×[s] 2h 57m 25.3s ▼7.1×[s]	14m 45.6s ▼26.1×[s] 14m 42s ▼26.8×[s]	31m 31s ▼54.9×[s] 31m 28.4s ▼57.7×[s]	1h 7m 40.8s ▼59.5×[s] 1h 7m 35.3s ▼62.3×[s]	1h 26m 25.6s ▼27.3×[s] 1h 26m 20.7s ▼28.2×[s]	5m 16.6s ▼1.5× 4m 33.9s ▼1.4×	4m 53.5s ▼26% 4m 17.9s ▼15%	·	·	12m 47.3s ▼10.9×[s] 4m 55.6s ▼4.3×[s]	12m 11.3s ▼10.7×[s] 5m 42.7s ▼5.1×[s]	·	·
R47	MPF / SIMD-ref Scalar-DP / KSW2	11m 28.4s ▼4.1×[s] 11m 24.7s ▼4.2×[s]	24m 37.6s ▼8.7×[s] 24m 34.9s ▼9.0×[s]	46m 9.8s ▼9.4×[s] 46m 3.7s ▼9.9×[s]	·	12m 9.4s ▼21.5×[s] 12m 5.8s ▼22.0×[s]	25m 52.5s ▼45.1×[s] 25m 49.6s ▼47.4×[s]	47m 13.9s ▼41.6×[s] 47m 7.6s ▼43.4×[s]	·	4m 48.9s ▼1.4×[s] 4m 2.5s ▼20%[s]	4m 34s ▼17%[s] 3m 55.9s ▼6%[s]	·	·	10m 57s ▼9.4×[s] 4m 39.3s ▼4.0×[s]	10m 35.2s ▼9.3×[s] 5m 2s ▼4.5×[s]	·	·
R48	MPF / Scalar SIMD-DP / Block	11m 30.6s ▼4.1×[s] 11m 29.9s ▼4.3×[s]	25m 2.6s ▼8.8×[s] 25m 1.8s ▼9.2×[s]	45m 36s ▼9.3×[s] 45m 34.7s ▼9.8×[s]	1h 36m 16.5s ▼3.8×[s] 1h 36m 14.3s ▼3.8×[s]	15m 49.6s ▼28.0×[s] 15m 45.8s ▼28.7×[s]	32m 50.4s ▼57.2×[s] 32m 47.7s ▼60.2×[s]	1h 6m 46.8s ▼58.8×[s] 1h 6m 41.3s ▼61.4×[s]	1h 30m 32.9s ▼28.6×[s] 1h 30m 28.4s ▼29.6×[s]	4m 51s ▼1.4× 4m 8.2s ▼22%	4m 40s ▼20% 4m 5.7s ▼10%	·	·	12m 25.7s ▼10.6×[s] 4m 36.4s ▼4.0×[s]	11m 44.4s ▼10.3×[s] 5m 24.7s ▼4.9×[s]	·	·
R49	MPF / SIMD-ref SIMD-DP / Block	12m 1.2s ▼4.3×[s] 12m 0.4s ▼4.5×[s]	24m 25.1s ▼8.6×[s] 24m 24.2s ▼9.0×[s]	46m 40.1s ▼9.5×[s] 46m 38.8s ▼10.0×[s]	1h 35m 35.8s ▼3.8×[s] 1h 35m 33.7s ▼3.8×[s]	12m 28.9s ▼22.1×[s] 12m 25.2s ▼22.6×[s]	34m 56.4s ▼60.9×[s] 34m 53.2s ▼64.0×[s]	49m 6.1s ▼43.2×[s] 49m 0.3s ▼45.2×[s]	·	4m 45.5s ▼1.4×[s] 4m 1.2s ▼19%[s]	4m 32.2s ▼17%[s] 3m 55.6s ▼5%[s]	·	·	10m 51.7s ▼9.3×[s] 4m 28.8s ▼3.9×[s]	10m 57.3s ▼9.6×[s] 5m 24.8s ▼4.9×[s]	·	·
R50 BioseqDB	Vulkan (all modes)	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Minimap2 stays first. Other rows follow increasing minima of available Index A times within this size/k/w group, across alphabets and all displayed machines. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 2 Gbp k=15, w=10								No index 2 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	11.2s ▼3.1× 10s ▼3.6×	10.7s ▼3.2× 8.6s ▼3.5×	22.5s ▼3.6× 18.8s ▼4.4×	1m 13.1s ▼22.5× 1m 5.7s ▼27.7×	9.9s ▼21% 8.8s ▼1.6×	9.5s ▼1.4× 7.6s ▼1.6×	16.5s ▼1.7× 12.9s ▼2.0×	28.2s ▼2.3× 26.2s ▼3.0×	53s ▲2.2× 51.9s ▲2.1×	56.6s ▲2.1× 54.6s ▲1.9×	1m 55.5s ▲1.8× 1m 51.8s ▲1.5×	6m 1.3s ▼3.0× 5m 55.2s ▼3.1×	48.1s ▲7.3× 47s ▲2.4×	44.5s ▲6.5× 42.6s ▲2.7×	1m 30.2s ▲6.4× 1m 26.7s ▲2.2×	2m 46.4s ▲4.0× 2m 42.7s ▼27%
R2 BioseqDB	MPF / Bounded Scalar-DP / Block	3.6s ▲3.1× 2.8s ▲3.5×	3.4s ▲3.1× 2.6s ▲3.3×	6.4s ▲3.5× 4.4s ▲4.2×	3.2s ▲22.5× 2.4s ▲27.7×	10.2s ▶ 3%[s] 6s ▲1.5×[s]	8.6s ▲10%[s] 5.5s ▲28%[s]	14.6s ▲11%[s] 8.4s ▲1.5×[s]	12.4s ▲2.3×[s] 9.2s ▲2.9×[s]	1m 56.2s ▼2.2× 1m 47.8s ▼2.1×	2m 7.4s ▼2.3× 1m 56.8s ▼2.1×	·	2m 1.9s ▲3.0× 1m 56.1s ▲3.1×	6m 7.7s ▼7.7×[s] 2m 6.1s ▼2.7×[s]	6m 48.2s ▼9.2×[s] 3m 14.7s ▼4.6×[s]	·	11m 10.4s ▼4.0× 2m 8s ▲21%
R3	MPF / SIMD-ref Scalar-DP / Block	3.6s ▲3.1× 2.9s ▲3.5×	3.4s ▲3.2× 2.6s ▲3.3×	6.5s ▲3.5× 4.4s ▲4.2×	3.3s ▲22.1× 2.5s ▲26.2×	12.7s ▼29%[s] 7.4s ▲15%[s]	13s ▼1.4×[s] 7.4s ▶ 3%[s]	24s ▼1.5×[s] 13.6s ▼5%[s]	16.5s ▲1.7×[s] 13.6s ▲1.9×[s]	2m 51.6s ▼3.2×[s] 2m 26.6s ▼2.8×[s]	2m 23.6s ▼2.5×[s] 2m 2.9s ▼2.3×[s]	·	·	·	·	·	·
R4	MPF / Scalar Scalar-DP / Block	3.8s ▲3.0× 3.1s ▲3.3×	3.5s ▲3.1× 2.7s ▲3.2×	6.5s ▲3.4× 4.8s ▲3.9×	3.4s ▲21.8× 2.5s ▲26.2×	10.1s ▶ 2%[s] 5.9s ▲1.5×[s]	8.3s ▲13%[s] 5.2s ▲1.5×[s]	15.9s ▶ 4%[s] 9.6s ▲26%[s]	14.5s ▲1.9×[s] 11.3s ▲2.3×[s]	2m 51.4s ▼3.2× 2m 29.1s ▼2.9×	2m 59.3s ▼3.2× 2m 39.2s ▼2.9×	·	·	7m 9.6s ▼8.9×[s] 2m 48s ▼3.6×[s]	6m 28.6s ▼8.7×[s] 2m 48.4s ▼4.0×[s]	·	·
R5	MPF / Two-stack Scalar-DP / Block	3.6s ▲3.1× 2.9s ▲3.4×	3.4s ▲3.2× 2.6s ▲3.3×	6.3s ▲3.6× 4.6s ▲4.1×	7.9s ▲9.2× 7.1s ▲9.3×	9.7s ▶ 3%[s] 5.5s ▲1.6×[s]	7.6s ▲20%[s] 4.7s ▲1.6×[s]	15.8s ▶ 4%[s] 8.8s ▲1.5×[s]	12.4s ▲2.3×[s] 8.9s ▲3.0×[s]	2m 41.5s ▼3.0×[s] 2m 17s ▼2.6×[s]	2m 37.7s ▼2.8×[s] 2m 16.7s ▼2.5×[s]	·	·	6m 32.3s ▼8.2×[s] 2m 11.7s ▼2.8×[s]	5m 45.5s ▼7.8×[s] 2m 3.6s ▼2.9×[s]	·	·
R6	MPF / Two-stack SIMD-DP / Block	3.7s ▲3.0× 3s ▲3.4×	3.5s ▲3.0× 2.6s ▲3.3×	6.7s ▲3.4× 4.7s ▲4.0×	3.4s ▲21.5× 2.5s ▲25.8×	11.8s ▼19%[s] 7.6s ▲14%[s]	8.6s ▲10%[s] 5s ▲1.5×[s]	16s ▶ 3%[s] 9.1s ▲30%[s]	13.5s ▲2.1× 11s ▲2.4×	2m 38.5s ▼3.0× 2m 16s ▼2.6×	2m 34.1s ▼2.7× 2m 13.1s ▼2.4×	·	3m 52.6s ▲1.6× 3m 35.8s ▲1.6×	6m 24.3s ▼8.0×[s] 2m 4.9s ▼2.7×[s]	5m 40.8s ▼7.7×[s] 1m 59.2s ▼2.8×[s]	·	13m 14.7s ▼4.8× 3m 54.6s ▼1.4×
R7	MPF / SIMD-ref SIMD-DP / Block	3.7s ▲3.0× 2.9s ▲3.4×	3.5s ▲3.0× 2.7s ▲3.2×	6.9s ▲3.3× 4.5s ▲4.2×	3.4s ▲21.4× 2.5s ▲26.3×	13.2s ▼1.3×[s] 7.7s ▲13%[s]	13.9s ▼1.5×[s] 8s ▶ 5%[s]	28.7s ▼1.7×[s] 14.5s ▼13%[s]	26.2s ▲7% 24.4s ▲7%	2m 47.2s ▼3.2×[s] 2m 22.2s ▼2.7×[s]	2m 32.6s ▼2.7× 2m 12.9s ▼2.4×	·	3m 6.8s ▲1.9× 2m 49.2s ▲2.1×	·	·	·	·
R8	MPF / Bounded RMQ / Block	3.7s ▲3.0× 2.9s ▲3.4×	3.5s ▲3.0× 2.7s ▲3.2×	6.5s ▲3.5× 4.5s ▲4.2×	3.4s ▲21.2× 2.5s ▲26.1×	18.7s ▼1.9×[s] 14.5s ▼1.6×[s]	20s ▼2.1×[s] 16.6s ▼2.2×[s]	21.3s ▼29%[s] 14.6s ▼13%[s]	12.4s ▲2.3×[s] 9s ▲2.9×[s]	1m 55.5s ▼2.2× 1m 47.1s ▼2.1×	2m 21s ▼2.5× 2m 10.3s ▼2.4×	·	2m 4s ▲2.9× 1m 58.1s ▲3.0×	6m 8.6s ▼7.7×[s] 2m 7s ▼2.7×[s]	5m 25.3s ▼7.3×[s] 1m 58.4s ▼2.8×[s]	·	11m 12.6s ▼4.0× 2m 10.3s ▲20%

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Minimap2 stays first. Other rows follow increasing minima of available Index A times within this size/k/w group, across alphabets and all displayed machines. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 2 Gbp k=15, w=10								No index 2 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	11.2s ▼3.1× 10s ▼3.6×	10.7s ▼3.2× 8.6s ▼3.5×	22.5s ▼3.6× 18.8s ▼4.4×	1m 13.1s ▼22.5× 1m 5.7s ▼27.7×	9.9s ▼21% 8.8s ▼1.6×	9.5s ▼1.4× 7.6s ▼1.6×	16.5s ▼1.7× 12.9s ▼2.0×	28.2s ▼2.3× 26.2s ▼3.0×	53s ▲2.2× 51.9s ▲2.1×	56.6s ▲2.1× 54.6s ▲1.9×	1m 55.5s ▲1.8× 1m 51.8s ▲1.5×	6m 1.3s ▼3.0× 5m 55.2s ▼3.1×	48.1s ▲7.3× 47s ▲2.4×	44.5s ▲6.5× 42.6s ▲2.7×	1m 30.2s ▲6.4× 1m 26.7s ▲2.2×	2m 46.4s ▲4.0× 2m 42.7s ▼27%
R9 BioseqDB	MPF / SIMD-ref RMQ / Block	3.7s ▲3.0× 3s ▲3.4×	3.5s ▲3.0× 2.7s ▲3.2×	6.7s ▲3.4× 4.6s ▲4.1×	3.5s ▲21.0× 2.6s ▲25.5×	13.6s ▼1.4×[s] 7.8s ▲12%[s]	13.2s ▼1.4×[s] 8.1s ▼6%[s]	25.7s ▼1.6×[s] 14s ▼9%[s]	18.3s ▲1.5×[s] 15.5s ▲1.7×[s]	2m 42.5s ▼3.1×[s] 2m 17.6s ▼2.7×[s]	2m 24.5s ▼2.6×[s] 2m 4.8s ▼2.3×[s]	·	·	·	·	·	·
R10	MPF / Bounded SIMD-DP / Block	3.7s ▲3.0× 3s ▲3.4×	3.5s ▲3.0× 2.7s ▲3.2×	6.7s ▲3.3× 4.5s ▲4.2×	8.8s ▲8.3× 7.9s ▲8.3×	8.2s ▲17% 5.8s ▲1.5×	6.6s ▲1.4× 4.6s ▲1.6×	13.7s ▲17% 8.8s ▲1.5×	20.2s ▲28%[s] 16.8s ▲1.6×[s]	1m 56.4s ▼2.2× 1m 48.1s ▼2.1×	1m 56.2s ▼2.1× 1m 46.2s ▼1.9×	3m 42.9s ▼1.9× 3m 25s ▼1.8×	2m 5.6s ▲2.9× 1m 59.7s ▲3.0×	5m 52.1s ▼7.3× 1m 53.5s ▼2.4×	4m 49.2s ▼6.5× 1m 56.8s ▼2.7×	9m 39.5s ▼6.4× 3m 18.1s ▼2.3×	11m 24.8s ▼4.1× 2m 20.4s ▲14%
R11	MPF / Scalar RMQ / Block	4s ▲2.8× 3.1s ▲3.2×	3.7s ▲2.9× 2.8s ▲3.1×	7.1s ▲3.2× 4.8s ▲3.9×	3.5s ▲20.6× 2.6s ▲25.3×	10s ▶ 1%[s] 5.9s ▲1.5×[s]	8.7s ▲8%[s] 5.7s ▲25%[s]	9.5s ▲1.7× 6.5s ▲2.0×	23s ▲18%[s] 19.8s ▲25%[s]	2m 51.5s ▼3.2× 2m 29.1s ▼2.9×	3m 4.8s ▼3.3× 2m 44.5s ▼3.0×	·	·	7m 6.3s ▼8.9×[s] 2m 44.6s ▼3.5×[s]	6m 52.5s ▼9.3×[s] 3m 7.3s ▼4.4×[s]	·	·
R12	MPF / Two-stack RMQ / Block	3.8s ▲2.9× 3s ▲3.3×	3.6s ▲3.0× 2.7s ▲3.2×	6.8s ▲3.3× 4.7s ▲4.0×	3.6s ▲20.1× 2.7s ▲24.2×	9.6s ▶ 3%[s] 5.5s ▲1.6×[s]	8.2s ▲14%[s] 5s ▲1.5×[s]	12.8s ▲23% 8.6s ▲1.5×	14.3s ▲2.0×[s] 11.1s ▲2.4×[s]	2m 38.4s ▼3.0× 2m 16.3s ▼2.6×	2m 38s ▼2.8× 2m 17.8s ▼2.5×	·	·	6m 27.6s ▼8.1×[s] 2m 7.9s ▼2.7×[s]	6m 2.1s ▼8.1×[s] 2m 19.9s ▼3.3×[s]	·	·
R13	MPF / Scalar SIMD-DP / Block	3.9s ▲2.9× 3.1s ▲3.3×	4.5s ▲2.4× 3.6s ▲2.4×	6.6s ▲3.4× 4.7s ▲4.0×	9.1s ▲8.1× 8.1s ▲8.1×	10.4s ▼5%[s] 6s ▲1.5×[s]	11.1s ▼17%[s] 7.8s ▶ 2%[s]	16.6s ▶ 1%[s] 10.1s ▲22%[s]	14.3s ▲2.0× 12.1s ▲2.2×	3m 8.5s ▼3.6× 2m 46.3s ▼3.2×	3m 1.2s ▼3.2× 2m 41.3s ▼3.0×	·	3m 2.9s ▲2.0× 2m 46s ▲2.1×	7m 10.9s ▼9.0×[s] 2m 49.7s ▼3.6×[s]	6m 59s ▼9.4×[s] 3m 11.3s ▼4.5×[s]	·	12m 21.6s ▼4.5× 3m 1s ▼11%
R14	MPF / Bounded SIMD-DP / KSW2	5.2s ▲2.1× 2.8s ▲3.6×	4.7s ▲2.3× 2.5s ▲3.5×	9.9s ▲2.3× 4.3s ▲4.4×	7.4s ▲9.9× 4.4s ▲15.1×	10.2s ▶ 3%[s] 5.9s ▲1.5×[s]	12.5s ▼1.3×[s] 8.7s ▼14%[s]	14.1s ▲14% 8.5s ▲1.5×	21.5s ▲24%[s] 17.6s ▲1.5×[s]	2m 11.6s ▼2.5× 2m 1.1s ▼2.3×	2m 7.5s ▼2.3× 1m 54.8s ▼2.1×	·	2m 20.7s ▲2.6× 2m 10.4s ▲2.7×	6m 9.1s ▼7.7×[s] 2m 5.6s ▼2.7×[s]	5m 32.3s ▼7.5×[s] 1m 56.1s ▼2.7×[s]	·	11m 23.6s ▼4.1× 2m 22.7s ▲12%
R15	MPF / SIMD-ref Scalar-DP / KSW2	5.1s ▲2.2× 2.8s ▲3.6×	4.7s ▲2.3× 2.5s ▲3.4×	9.8s ▲2.3× 4.3s ▲4.3×	7.2s ▲10.1× 4.4s ▲15.1×	12.8s ▼29%[s] 7.3s ▲17%[s]	13s ▼1.4×[s] 7.3s ▶ 4%[s]	24.5s ▼1.5×[s] 13.4s ▶ 4%[s]	16.2s ▲1.7×[s] 12.9s ▲2.0×[s]	2m 52.8s ▼3.3×[s] 2m 25.9s ▼2.8×[s]	2m 31.6s ▼2.7×[s] 2m 9.7s ▼2.4×[s]	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Minimap2 stays first. Other rows follow increasing minima of available Index A times within this size/k/w group, across alphabets and all displayed machines. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 2 Gbp k=15, w=10								No index 2 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	11.2s ▼3.1× 10s ▼3.6×	10.7s ▼3.2× 8.6s ▼3.5×	22.5s ▼3.6× 18.8s ▼4.4×	1m 13.1s ▼22.5× 1m 5.7s ▼27.7×	9.9s ▼21% 8.8s ▼1.6×	9.5s ▼1.4× 7.6s ▼1.6×	16.5s ▼1.7× 12.9s ▼2.0×	28.2s ▼2.3× 26.2s ▼3.0×	53s ▲2.2× 51.9s ▲2.1×	56.6s ▲2.1× 54.6s ▲1.9×	1m 55.5s ▲1.8× 1m 51.8s ▲1.5×	6m 1.3s ▼3.0× 5m 55.2s ▼3.1×	48.1s ▲7.3× 47s ▲2.4×	44.5s ▲6.5× 42.6s ▲2.7×	1m 30.2s ▲6.4× 1m 26.7s ▲2.2×	2m 46.4s ▲4.0× 2m 42.7s ▼27%
R16 BioseqDB	MPF / Bounded Scalar-DP / KSW2	5.2s ▲2.1× 2.8s ▲3.6×	4.7s ▲2.3× 2.5s ▲3.5×	9.7s ▲2.3× 4.3s ▲4.4×	7.2s ▲10.1× 4.3s ▲15.4×	9.9s ▶ 0%[s] 5.8s ▲1.5×[s]	8.5s ▲11%[s] 5.1s ▲1.5×[s]	15.2s ▲8%[s] 8.5s ▲1.5×[s]	13s ▲2.2×[s] 9.4s ▲2.8×[s]	2m 8.2s ▼2.4× 1m 58.3s ▼2.3×	2m 3.7s ▼2.2× 1m 51.2s ▼2.0×	·	2m 25.1s ▲2.5× 2m 14.9s ▲2.6×	6m 7.9s ▼7.7×[s] 2m 3.4s ▼2.6×[s]	5m 36.8s ▼7.6×[s] 2m 6.4s ▼3.0×[s]	9m 39.3s ▼6.4× 3m 13.2s ▼2.2×	11m 21s ▼4.1× 2m 19.3s ▲14%
R17	MPF / Two-stack Scalar-DP / KSW2	5.2s ▲2.1× 2.8s ▲3.5×	4.7s ▲2.3× 2.5s ▲3.4×	9.5s ▲2.4× 4.5s ▲4.2×	7.2s ▲10.1× 4.4s ▲14.9×	9.7s ▶ 2%[s] 5.6s ▲1.6×[s]	7.8s ▲18%[s] 4.7s ▲1.6×[s]	16.5s ▶ 0%[s] 8.9s ▲1.5×[s]	12.8s ▲2.2×[s] 9s ▲2.9×[s]	2m 46.6s ▼3.1×[s] 2m 20.5s ▼2.7×[s]	2m 46.1s ▼2.9×[s] 2m 22.8s ▼2.6×[s]	·	·	6m 29s ▼8.1×[s] 2m 7.9s ▼2.7×[s]	5m 48.7s ▼7.8×[s] 2m 5.1s ▼2.9×[s]	·	·
R18	MPF / Bounded RMQ / KSW2	5.3s ▲2.1× 2.8s ▲3.6×	4.8s ▲2.2× 2.5s ▲3.5×	9.8s ▲2.3× 4.2s ▲4.4×	7.7s ▲9.5× 4.6s ▲14.2×	12s ▼21%[s] 7.7s ▲13%[s]	11.5s ▼21%[s] 7.6s ▶ 0%[s]	17.2s ▶ 4%[s] 10s ▲22%[s]	13.2s ▲2.1×[s] 9.4s ▲2.8×[s]	2m 17.7s ▼2.6× 2m 7s ▼2.4×	1m 59.4s ▼2.1× 1m 47.5s ▼2.0×	·	2m 24s ▲2.5× 2m 13.9s ▲2.7×	6m 7.2s ▼7.6×[s] 2m 3.6s ▼2.6×[s]	5m 26.8s ▼7.3×[s] 1m 56.2s ▼2.7×[s]	·	·
R19	MPF / Scalar Scalar-DP / KSW2	5.2s ▲2.1× 2.9s ▲3.5×	4.8s ▲2.2× 2.6s ▲3.3×	9.5s ▲2.4× 4.5s ▲4.2×	8.6s ▲8.5× 5.7s ▲11.6×	10.1s ▶ 2%[s] 5.9s ▲1.5×[s]	8.8s ▲8%[s] 5.3s ▲1.4×[s]	15.9s ▶ 4%[s] 9.2s ▲29%[s]	13.1s ▲2.1×[s] 9.4s ▲2.8×[s]	2m 52.3s ▼3.3× 2m 28.6s ▼2.9×	3m 4.9s ▼3.3× 2m 43.4s ▼3.0×	·	·	7m 3.4s ▼8.8×[s] 2m 40.4s ▼3.4×[s]	6m 32.5s ▼8.8×[s] 2m 49.9s ▼4.0×[s]	·	·
R20	MPF / SIMD-ref SIMD-DP / KSW2	5.3s ▲2.1× 2.8s ▲3.6×	4.8s ▲2.2× 2.5s ▲3.4×	9.7s ▲2.3× 4.3s ▲4.4×	9s ▲8.1× 6.2s ▲10.6×	16.7s ▼1.7×[s] 11.2s ▼27%[s]	16.4s ▼1.7×[s] 9.7s ▼27%[s]	30.5s ▼1.8×[s] 14.9s ▼15%[s]	17.3s ▲1.6×[s] 13.9s ▲1.9×[s]	2m 51.2s ▼3.2×[s] 2m 24.3s ▼2.8×[s]	2m 22.1s ▼2.5×[s] 2m 1.6s ▼2.2×[s]	3m 23.3s ▼1.8× 2m 50.3s ▼1.5×	·	·	·	·	·
R21	MPF / Two-stack SIMD-DP / KSW2	5.3s ▲2.1× 2.9s ▲3.5×	4.8s ▲2.2× 2.5s ▲3.4×	9.8s ▲2.3× 4.5s ▲4.2×	7.1s ▲10.3× 4.2s ▲15.6×	9.7s ▶ 2%[s] 5.5s ▲1.6×[s]	9.6s ▶ 1%[s] 5.2s ▲1.5×[s]	16.4s ▶ 0%[s] 8.8s ▲1.5×[s]	14.9s ▲1.9×[s] 11s ▲2.4×[s]	2m 27.4s ▼2.8× 2m 3.1s ▼2.4×	2m 26.8s ▼2.6× 2m 5.2s ▼2.3×	·	·	6m 28.7s ▼8.1×[s] 2m 7.4s ▼2.7×[s]	5m 46.6s ▼7.8×[s] 2m 1.5s ▼2.9×[s]	·	·
R22	MPF / SIMD-ref RMQ / KSW2	5.3s ▲2.1× 2.8s ▲3.6×	4.9s ▲2.2× 2.5s ▲3.4×	9.9s ▲2.3× 4.3s ▲4.4×	5.5s ▲13.3× 2.4s ▲26.8×	13.6s ▼1.4×[s] 7.5s ▲15%[s]	13.8s ▼1.5×[s] 7.7s ▶ 0%[s]	26.7s ▼1.6×[s] 13.8s ▼7%[s]	31.9s ▼13%[s] 28.3s ▼8%[s]	2m 45.3s ▼3.1×[s] 2m 18.5s ▼2.7×[s]	2m 24.6s ▼2.6×[s] 2m 3.8s ▼2.3×[s]	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Minimap2 stays first. Other rows follow increasing minima of available Index A times within this size/k/w group, across alphabets and all displayed machines. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 2 Gbp k=15, w=10								No index 2 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	11.2s ▼3.1× 10s ▼3.6×	10.7s ▼3.2× 8.6s ▼3.5×	22.5s ▼3.6× 18.8s ▼4.4×	1m 13.1s ▼22.5× 1m 5.7s ▼27.7×	9.9s ▼21% 8.8s ▼1.6×	9.5s ▼1.4× 7.6s ▼1.6×	16.5s ▼1.7× 12.9s ▼2.0×	28.2s ▼2.3× 26.2s ▼3.0×	53s ▲2.2× 51.9s ▲2.1×	56.6s ▲2.1× 54.6s ▲1.9×	1m 55.5s ▲1.8× 1m 51.8s ▲1.5×	6m 1.3s ▼3.0× 5m 55.2s ▼3.1×	48.1s ▲7.3× 47s ▲2.4×	44.5s ▲6.5× 42.6s ▲2.7×	1m 30.2s ▲6.4× 1m 26.7s ▲2.2×	2m 46.4s ▲4.0× 2m 42.7s ▼27%
R23 BioseqDB	MPF / Scalar SIMD-DP / KSW2	5.4s ▲2.1× 2.9s ▲3.5×	4.9s ▲2.2× 2.6s ▲3.3×	10.3s ▲2.2× 4.6s ▲4.1×	7.8s ▲9.3× 5s ▲13.2×	10.2s ▶ 3%[s] 5.9s ▲1.5×[s]	8.9s ▲6%[s] 5.3s ▲1.5×[s]	17.8s ▼8%[s] 10.8s ▲16%[s]	15.6s ▲1.8×[s] 12.5s ▲2.1×[s]	3m 4.9s ▼3.5× 2m 40.5s ▼3.1×	3m 7.6s ▼3.3× 2m 46s ▼3.0×	·	·	7m 6.6s ▼8.9×[s] 2m 43.2s ▼3.5×[s]	6m 54s ▼9.3×[s] 3m 6.6s ▼4.4×[s]	·	·
R24	MPF / Two-stack RMQ / KSW2	5.3s ▲2.1× 2.9s ▲3.5×	5s ▲2.1× 2.5s ▲3.4×	10.1s ▲2.2× 4.5s ▲4.2×	5.6s ▲12.9× 2.6s ▲25.5×	9.6s ▶ 4%[s] 5.5s ▲1.6×[s]	8.9s ▲6%[s] 5.1s ▲1.5×[s]	16.4s ▶ 1%[s] 8.9s ▲1.5×[s]	12.7s ▲2.2×[s] 8.8s ▲3.0×[s]	2m 37s ▼3.0× 2m 12.9s ▼2.6×	2m 34.6s ▼2.7× 2m 12.9s ▼2.4×	·	·	6m 27.3s ▼8.1×[s] 2m 5.7s ▼2.7×[s]	5m 42s ▼7.7×[s] 1m 58.6s ▼2.8×[s]	·	·
R25	MPF / Scalar RMQ / KSW2	5.5s ▲2.0× 2.9s ▲3.4×	5.1s ▲2.1× 2.7s ▲3.2×	10.3s ▲2.2× 4.6s ▲4.1×	5.6s ▲13.1× 2.5s ▲26.3×	10s ▶ 1%[s] 5.9s ▲1.5×[s]	8.7s ▲8%[s] 5.3s ▲1.4×[s]	16s ▶ 3%[s] 9.3s ▲28%[s]	21.1s ▲25%[s] 17.5s ▲1.5×[s]	3m 1.5s ▼3.4× 2m 37.7s ▼3.0×	2m 54.6s ▼3.1× 2m 33.3s ▼2.8×	·	·	7m 3s ▼8.8×[s] 2m 38.7s ▼3.4×[s]	6m 59.2s ▼9.4×[s] 3m 13.2s ▼4.5×[s]	·	·
R26	MPS / Bounded Scalar-DP / Block	6.4s ▲1.7× 5.7s ▲1.8×	6.6s ▲1.6× 5.7s ▲1.5×	12.5s ▲1.8×[s] 10.1s ▲1.9×[s]	5.9s ▲12.3× 4.9s ▲13.4×	4m 54s ▼29.7×[s] 32s ▼3.6×[s]	4m 0.6s ▼25.4×[s] 28.5s ▼3.7×[s]	7m 33.3s ▼27.5×[s] 53.3s ▼4.1×[s]	·	2m 24.9s ▼2.7× 2m 2.4s ▼2.4×	2m 39.7s ▼2.8× 2m 18.8s ▼2.5×	·	2m 43s ▲2.2× 2m 26.5s ▲2.4×	6m 32.4s ▼8.2×[s] 2m 10.2s ▼2.8×[s]	5m 33.1s ▼7.5×[s] 1m 56s ▼2.7×[s]	·	11m 41.8s ▼4.2× 2m 22.1s ▲13%
R27	MPS / Bounded SIMD-DP / Block	6.7s ▲1.7× 6s ▲1.7×	6.9s ▲1.5× 6s ▲1.4×	12.5s ▲1.8× 10.4s ▲1.8×	6s ▲12.1× 5s ▲13.0×	4m 54.5s ▼29.7×[s] 33.3s ▼3.8×[s]	4m 26.3s ▼28.1×[s] 33.8s ▼4.4×[s]	8m 17s ▼30.1×[s] 54.4s ▼4.2×[s]	·	2m 20.8s ▼2.7× 1m 57.4s ▼2.3×	2m 25.9s ▼2.6× 2m 5.5s ▼2.3×	·	2m 42.2s ▲2.2× 2m 23.6s ▲2.5×	6m 30.1s ▼8.1×[s] 2m 18.5s ▼2.9×[s]	5m 34.2s ▼7.5×[s] 1m 56.9s ▼2.7×[s]	·	11m 44s ▼4.2× 2m 21.6s ▲13%
R28	MPS / Bounded RMQ / Block	6.5s ▲1.7× 5.7s ▲1.8×	6.5s ▲1.6× 5.8s ▲1.5×	12.7s ▲1.8×[s] 10.2s ▲1.8×[s]	6.1s ▲12.0× 5.1s ▲12.8×	4m 55.9s ▼29.9×[s] 33.3s ▼3.8×[s]	4m 1.2s ▼25.4×[s] 29.1s ▼3.8×[s]	7m 32.9s ▼27.5×[s] 53.3s ▼4.1×[s]	·	2m 31.3s ▼2.9× 2m 9.3s ▼2.5×	2m 16.4s ▼2.4× 1m 56.6s ▼2.1×	·	2m 44.7s ▲2.2× 2m 26.9s ▲2.4×	6m 30.9s ▼8.1×[s] 2m 9.4s ▼2.8×[s]	5m 39.3s ▼7.6×[s] 2m 0.7s ▼2.8×[s]	·	11m 44.3s ▼4.2× 2m 23.5s ▲12%
R29	MPS / SIMD-ref SIMD-DP / Block	7.2s ▲1.5×[s] 6.1s ▲1.6×[s]	6.7s ▲1.6×[s] 5.7s ▲1.5×[s]	·	6.4s ▲11.4× 5.5s ▲12.0×	4m 55.8s ▼29.9×[s] 34.7s ▼3.9×[s]	4m 10.3s ▼26.4×[s] 31.6s ▼4.1×[s]	·	·	·	·	4m 15.9s ▼2.2× 3m 41.9s ▼2.0×	3m 19.6s ▲1.8× 3m 1.1s ▲2.0×	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Minimap2 stays first. Other rows follow increasing minima of available Index A times within this size/k/w group, across alphabets and all displayed machines. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 2 Gbp k=15, w=10								No index 2 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	11.2s ▼3.1× 10s ▼3.6×	10.7s ▼3.2× 8.6s ▼3.5×	22.5s ▼3.6× 18.8s ▼4.4×	1m 13.1s ▼22.5× 1m 5.7s ▼27.7×	9.9s ▼21% 8.8s ▼1.6×	9.5s ▼1.4× 7.6s ▼1.6×	16.5s ▼1.7× 12.9s ▼2.0×	28.2s ▼2.3× 26.2s ▼3.0×	53s ▲2.2× 51.9s ▲2.1×	56.6s ▲2.1× 54.6s ▲1.9×	1m 55.5s ▲1.8× 1m 51.8s ▲1.5×	6m 1.3s ▼3.0× 5m 55.2s ▼3.1×	48.1s ▲7.3× 47s ▲2.4×	44.5s ▲6.5× 42.6s ▲2.7×	1m 30.2s ▲6.4× 1m 26.7s ▲2.2×	2m 46.4s ▲4.0× 2m 42.7s ▼27%
R30 BioseqDB	MPS / Two-stack SIMD-DP / Block	6.8s ▲1.6× 6.1s ▲1.7×	6.8s ▲1.6× 6s ▲1.4×	12.6s ▲1.8× 10.6s ▲1.8×	6.5s ▲11.2× 5.7s ▲11.6×	4m 47.8s ▼29.1×[s] 26.2s ▼3.0×[s]	3m 58.5s ▼25.1×[s] 24.4s ▼3.2×[s]	·	·	2m 46.7s ▼3.1× 2m 24.6s ▼2.8×	2m 47.4s ▼3.0× 2m 27.3s ▼2.7×	·	3m 50.1s ▲1.6× 3m 32.7s ▲1.7×	·	·	·	·
R31	MPS / Scalar SIMD-DP / Block	6.8s ▲1.6× 6.2s ▲1.6×	7.1s ▲1.5× 6.2s ▲28%	·	6.6s ▲11.1× 5.6s ▲11.7×	4m 53.5s ▼29.6×[s] 32.6s ▼3.7×[s]	4m 10s ▼26.4×[s] 31.4s ▼4.1×[s]	·	·	3m 9.8s ▼3.6× 2m 47.5s ▼3.2×	3m 6.4s ▼3.3× 2m 46.4s ▼3.0×	·	3m 9.9s ▲1.9× 2m 52.7s ▲2.1×	7m 6.1s ▼8.9×[s] 2m 43.9s ▼3.5×[s]	·	·	12m 35.2s ▼4.5× 3m 14.5s ▼20%
R32	MPS / SIMD-ref Scalar-DP / Block	7.2s ▲1.6×[s] 6.1s ▲1.6×[s]	6.8s ▲1.6×[s] 5.8s ▲1.5×[s]	·	·	4m 54.2s ▼29.7×[s] 32.4s ▼3.7×[s]	4m 9.1s ▼26.3×[s] 31.3s ▼4.1×[s]	·	·	·	·	·	·	·	·	·	·
R33	MPS / SIMD-ref RMQ / Block	7.1s ▲1.6×[s] 6s ▲1.7×[s]	6.8s ▲1.6×[s] 5.8s ▲1.5×[s]	·	·	4m 56.7s ▼29.9×[s] 35.7s ▼4.1×[s]	4m 11.8s ▼26.6×[s] 33.7s ▼4.4×[s]	·	·	·	·	3m 24.8s ▼1.8× 2m 59.9s ▼1.6×	·	·	·	·	·
R34	MPS / Two-stack RMQ / Block	6.9s ▲1.6× 6.2s ▲1.6×	7.5s ▲29% 6.6s ▲23%	·	·	4m 45.1s ▼28.8×[s] 25.1s ▼2.8×[s]	3m 56.8s ▼25.0×[s] 23.9s ▼3.1×[s]	·	·	2m 47.3s ▼3.2× 2m 24.6s ▼2.8×	2m 30.3s ▼2.7× 2m 10.8s ▼2.4×	4m 45s ▼2.5× 4m 11.3s ▼2.2×	·	·	·	·	·
R35	MPS / Two-stack Scalar-DP / Block	6.9s ▲1.6× 6.2s ▲1.6×	7.2s ▲1.5× 6.3s ▲26%	·	·	4m 44.1s ▼28.7×[s] 24.5s ▼2.8×[s]	3m 57.3s ▼25.0×[s] 23.7s ▼3.1×[s]	·	·	·	·	·	·	·	·	·	·
R36	MPS / Scalar RMQ / Block	6.9s ▲1.6× 6.2s ▲1.6×	7s ▲1.5× 6.2s ▲28%	·	·	4m 52.3s ▼29.5×[s] 31.8s ▼3.6×[s]	4m 9.2s ▼26.3×[s] 30.9s ▼4.0×[s]	·	·	3m 15.8s ▼3.7× 2m 53.2s ▼3.3×	3m 13.2s ▼3.4× 2m 53s ▼3.2×	·	·	7m 7.4s ▼8.9×[s] 2m 45.6s ▼3.5×[s]	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash. Minimap2 stays first. Other rows follow increasing minima of available Index A times within this size/k/w group, across alphabets and all displayed machines. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 2 Gbp k=15, w=10								No index 2 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	11.2s ▼3.1× 10s ▼3.6×	10.7s ▼3.2× 8.6s ▼3.5×	22.5s ▼3.6× 18.8s ▼4.4×	1m 13.1s ▼22.5× 1m 5.7s ▼27.7×	9.9s ▼21% 8.8s ▼1.6×	9.5s ▼1.4× 7.6s ▼1.6×	16.5s ▼1.7× 12.9s ▼2.0×	28.2s ▼2.3× 26.2s ▼3.0×	53s ▲2.2× 51.9s ▲2.1×	56.6s ▲2.1× 54.6s ▲1.9×	1m 55.5s ▲1.8× 1m 51.8s ▲1.5×	6m 1.3s ▼3.0× 5m 55.2s ▼3.1×	48.1s ▲7.3× 47s ▲2.4×	44.5s ▲6.5× 42.6s ▲2.7×	1m 30.2s ▲6.4× 1m 26.7s ▲2.2×	2m 46.4s ▲4.0× 2m 42.7s ▼27%
R37 BioseqDB	MPS / Scalar Scalar-DP / Block	7.5s ▲1.5× 6.3s ▲1.6×	7.2s ▲1.5× 6.4s ▲26%	·	·	4m 52s ▼29.5×[s] 31.3s ▼3.6×[s]	4m 9.3s ▼26.3×[s] 30.8s ▼4.0×[s]	·	·	3m 13.5s ▼3.6×[s] 2m 47.5s ▼3.2×[s]	3m 16.5s ▼3.5×[s] 2m 55.3s ▼3.2×[s]	·	·	7m 3.9s ▼8.8×[s] 2m 41.8s ▼3.4×[s]	·	·	·
R38	MPS / Bounded Scalar-DP / KSW2	7.9s ▲29% 4.9s ▲2.0×	8.6s ▲20% 5s ▲1.7×	17.2s ▲24%[s] 8.9s ▲2.1×[s]	8s ▲9.2× 4.6s ▲14.2×	4m 56.3s ▼29.9×[s] 32.9s ▼3.7×[s]	4m 2.1s ▼25.5×[s] 28.6s ▼3.7×[s]	8m 11.2s ▼29.8×[s] 53.8s ▼4.2×[s]	·	2m 37s ▼3.0× 2m 12s ▼2.5×	2m 30.6s ▼2.7× 2m 7.1s ▼2.3×	·	2m 29s ▲2.4× 2m 7.2s ▲2.8×	6m 26.9s ▼8.1×[s] 2m 3.3s ▼2.6×[s]	5m 37.5s ▼7.6×[s] 1m 58.7s ▼2.8×[s]	·	12m 0.3s ▼4.3× 2m 37s ▶ 4%
R39	MPS / Bounded RMQ / KSW2	7.9s ▲29% 4.9s ▲2.0×	8.5s ▲20% 4.9s ▲1.7×	17.4s ▲23%[s] 8.8s ▲2.1×[s]	8.1s ▲9.1× 4.6s ▲14.2×	4m 56.6s ▼29.9×[s] 32.7s ▼3.7×[s]	4m 2.8s ▼25.6×[s] 28.9s ▼3.8×[s]	7m 36.6s ▼27.7×[s] 53.2s ▼4.1×[s]	·	2m 21.9s ▼2.7× 1m 57.6s ▼2.3×	2m 27.9s ▼2.6× 2m 5.4s ▼2.3×	·	2m 29.1s ▲2.4× 2m 8.2s ▲2.8×	6m 41.2s ▼8.3×[s] 2m 18.2s ▼2.9×[s]	5m 35.7s ▼7.5×[s] 1m 59.6s ▼2.8×[s]	·	·
R40	MPS / Two-stack RMQ / KSW2	8.3s ▲26% 5.3s ▲1.9×	8.1s ▲23% 5.1s ▲1.7×	·	·	4m 47.6s ▼29.0×[s] 25.3s ▼2.9×[s]	3m 58.5s ▼25.2×[s] 23.4s ▼3.1×[s]	·	·	2m 51.1s ▼3.2× 2m 26.7s ▼2.8×	2m 59.5s ▼3.2× 2m 36.8s ▼2.9×	·	·	·	·	·	·
R41	MPS / Bounded SIMD-DP / KSW2	8.2s ▲26% 5.4s ▲1.8×	8.4s ▲21% 4.9s ▲1.8×	17.8s ▲21%[s] 8.9s ▲2.1×[s]	9.6s ▲7.6× 6s ▲10.9×	4m 56.1s ▼29.9×[s] 32.4s ▼3.7×[s]	4m 12.1s ▼26.6×[s] 30.4s ▼4.0×[s]	8m 4.9s ▼29.4×[s] 52.6s ▼4.1×[s]	·	2m 20.7s ▼2.7× 1m 56.8s ▼2.3×	2m 19.7s ▼2.5× 1m 57.9s ▼2.2×	·	2m 42.4s ▲2.2× 2m 21.1s ▲2.5×	6m 37.2s ▼8.3×[s] 2m 13.5s ▼2.8×[s]	5m 36.2s ▼7.6×[s] 1m 59.7s ▼2.8×[s]	·	11m 54s ▼4.3× 2m 27.8s ▲9%
R42	MPS / Scalar RMQ / KSW2	8.2s ▲26% 5.4s ▲1.9×	8.7s ▲18% 5.4s ▲1.6×	·	·	4m 53.7s ▼29.7×[s] 31.6s ▼3.6×[s]	4m 11.2s ▼26.5×[s] 31.2s ▼4.1×[s]	·	·	3m 14.2s ▼3.7× 2m 49.7s ▼3.3×	3m 39.1s ▼3.9× 3m 16.1s ▼3.6×	·	·	7m 9s ▼8.9×[s] 2m 43.5s ▼3.5×[s]	·	·	·
R43	MPS / Two-stack SIMD-DP / KSW2	8.3s ▲26% 5.4s ▲1.8×	8.4s ▲21% 5.2s ▲1.7×	16.2s ▲28% 9.2s ▲2.0×	·	4m 46.4s ▼28.9×[s] 24.9s ▼2.8×[s]	3m 58.8s ▼25.2×[s] 23.6s ▼3.1×[s]	·	·	2m 40.2s ▼3.0× 2m 15.4s ▼2.6×	2m 42.5s ▼2.9× 2m 20.9s ▼2.6×	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash. Minimap2 stays first. Other rows follow increasing minima of available Index A times within this size/k/w group, across alphabets and all displayed machines. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 2 Gbp k=15, w=10								No index 2 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	11.2s ▼3.1× 10s ▼3.6×	10.7s ▼3.2× 8.6s ▼3.5×	22.5s ▼3.6× 18.8s ▼4.4×	1m 13.1s ▼22.5× 1m 5.7s ▼27.7×	9.9s ▼21% 8.8s ▼1.6×	9.5s ▼1.4× 7.6s ▼1.6×	16.5s ▼1.7× 12.9s ▼2.0×	28.2s ▼2.3× 26.2s ▼3.0×	53s ▲2.2× 51.9s ▲2.1×	56.6s ▲2.1× 54.6s ▲1.9×	1m 55.5s ▲1.8× 1m 51.8s ▲1.5×	6m 1.3s ▼3.0× 5m 55.2s ▼3.1×	48.1s ▲7.3× 47s ▲2.4×	44.5s ▲6.5× 42.6s ▲2.7×	1m 30.2s ▲6.4× 1m 26.7s ▲2.2×	2m 46.4s ▲4.0× 2m 42.7s ▼27%
R44 BioseqDB	MPS / Scalar SIMD-DP / KSW2	8.3s ▲26% 5.5s ▲1.8×	8.3s ▲22% 5.2s ▲1.6×	·	·	4m 52.9s ▼29.6×[s] 31.1s ▼3.5×[s]	4m 10.6s ▼26.4×[s] 30.7s ▼4.0×[s]	·	·	3m 1.5s ▼3.4× 2m 37.6s ▼3.0×	3m 21.1s ▼3.6× 2m 58.7s ▼3.3×	·	·	7m 6.4s ▼8.9×[s] 2m 43.1s ▼3.5×[s]	·	·	·
R45	MPS / Scalar Scalar-DP / KSW2	8.3s ▲26% 5.4s ▲1.8×	8.7s ▲18% 5.3s ▲1.6×	·	·	4m 54.2s ▼29.7×[s] 31.5s ▼3.6×[s]	4m 11s ▼26.5×[s] 31s ▼4.1×[s]	·	·	3m 21.3s ▼3.8×[s] 2m 53.7s ▼3.3×[s]	·	·	·	7m 19.5s ▼9.1×[s] 2m 56s ▼3.7×[s]	·	·	·
R46	MPS / Two-stack Scalar-DP / KSW2	8.4s ▲25% 5.4s ▲1.8×	9.4s ▲12% 5.5s ▲1.6×	·	·	4m 46.5s ▼28.9×[s] 24.8s ▼2.8×[s]	3m 58s ▼25.1×[s] 23s ▼3.0×[s]	·	·	·	·	·	·	·	·	·	·
R47	MPS / SIMD-ref SIMD-DP / KSW2	9.8s ▲13%[s] 5.3s ▲1.9×[s]	8.6s ▲20%[s] 5s ▲1.7×[s]	15.7s ▲1.4× 9.3s ▲2.0×	·	4m 55.9s ▼29.9×[s] 33.3s ▼3.8×[s]	4m 11.3s ▼26.5×[s] 31.3s ▼4.1×[s]	·	·	·	·	·	·	·	·	·	·
R48	MPS / SIMD-ref RMQ / KSW2	9.8s ▲12%[s] 5.2s ▲1.9×[s]	8.6s ▲19%[s] 4.9s ▲1.7×[s]	·	·	4m 58.8s ▼30.2×[s] 36.1s ▼4.1×[s]	4m 12.9s ▼26.7×[s] 33.1s ▼4.3×[s]	·	·	·	·	·	·	·	·	·	·
R49	MPS / SIMD-ref Scalar-DP / KSW2	9.8s ▲12%[s] 5.2s ▲1.9×[s]	8.7s ▲19%[s] 5s ▲1.7×[s]	16.4s ▲27% 8.9s ▲2.1×	·	4m 55s ▼29.8×[s] 32s ▼3.6×[s]	4m 9.4s ▼26.3×[s] 29.7s ▼3.9×[s]	·	·	·	·	·	·	·	·	·	·
R50 BioseqDB	Vulkan (all modes)	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Minimap2 stays first. Other rows follow increasing minima of available Index A times within this size/k/w group, across alphabets and all displayed machines. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 2 Gbp k=20, w=15								No index 2 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	19.6s ▼6.7× 18.6s ▼8.4×	18.2s ▼6.9× 16.5s ▼8.6×	42.5s ▼8.2× 39.4s ▼12.6×	6m 29.2s ▼92.8× 5m 56.9s ▼184.9×	9.3s ▼24% 8.4s ▼1.6×	8.7s ▼1.5× 7s ▼1.7×	15.8s ▼26% 12.7s ▼1.7×	18.5s ▼1.6× 17s ▼1.8×	42.4s ▲2.0× 41.4s ▲1.9×	42.9s ▲2.0× 41.1s ▲1.8×	1m 26.9s ▲1.9× 1m 23.7s ▲1.7×	4m 51.6s ▼2.9× 4m 46.6s ▼3.0×	35s ▲8.6× 34s ▲2.3×	35.9s ▲6.7× 34.2s ▲2.1×	1m 14.1s ▲6.9× 1m 11.1s ▲2.2×	1m 0.1s ▲10.4× 58.6s ▲1.8×
R2 BioseqDB	MPF / Bounded SIMD-DP / Block	3s ▲6.6× 2.3s ▲8.1×	2.6s ▲6.9× 1.9s ▲8.6×	5.4s ▲7.9× 3.4s ▲11.7×	4.5s ▲86.7× 3.6s ▲97.9×	7.5s ▲19% 5.2s ▲1.6×	5.8s ▲1.5× 4s ▲1.7×	12.6s ▲21% 7.9s ▲1.6×	12.9s ▲1.4× 10.6s ▲1.6×	1m 24.8s ▼2.0× 1m 16.9s ▼1.9×	1m 25.4s ▼2.0× 1m 15.7s ▼1.8×	2m 44s ▼1.9× 2m 26.2s ▼1.7×	1m 42s ▲2.9× 1m 35.7s ▲3.0×	5m 1.5s ▼8.6× 1m 24.2s ▼2.5×	4m 0.6s ▼6.7× 1m 24.3s ▼2.5×	8m 31.7s ▼6.9× 2m 34.1s ▼2.2×	10m 27.6s ▼10.4× 1m 51.4s ▼1.9×
R3	MPF / Bounded RMQ / Block	2.9s ▲6.7× 2.2s ▲8.3×	2.7s ▲6.7× 2s ▲8.3×	5.2s ▲8.2×[s] 3.3s ▲12.1×[s]	4.7s ▲82.6× 3.9s ▲92.2×	11.1s ▼19%[s] 7.1s ▲16%[s]	8.8s ▶ 1%[s] 5.7s ▲19%[s]	13.5s ▲15%[s] 7.5s ▲1.7×[s]	11.6s ▲1.6× 9.4s ▲1.8×	1m 34s ▼2.2× 1m 24.9s ▼2.1×	1m 33s ▼2.2× 1m 22.3s ▼2.0×	·	1m 44.1s ▲2.8× 1m 38.1s ▲2.9×	5m 11.1s ▼8.9×[s] 1m 29.9s ▼2.6×[s]	4m 41.1s ▼7.8×[s] 1m 27.4s ▼2.6×[s]	·	10m 23.5s ▼10.4× 1m 47.3s ▼1.8×
R4	MPF / Bounded Scalar-DP / Block	3s ▲6.6× 2.3s ▲8.1×	2.8s ▲6.5× 2s ▲8.2×	5.3s ▲8.0×[s] 3.3s ▲11.8×[s]	4.2s ▲92.8× 3.3s ▲106.7×	9.5s ▶ 2%[s] 5.3s ▲1.6×[s]	8.1s ▲7%[s] 4.7s ▲1.5×[s]	13.7s ▲14%[s] 7.5s ▲1.7×[s]	12.8s ▲1.4× 10.6s ▲1.6×	1m 32.9s ▼2.2× 1m 23.4s ▼2.0×	1m 31.2s ▼2.1× 1m 21s ▼2.0×	·	1m 43.1s ▲2.8× 1m 36.4s ▲3.0×	5m 11.8s ▼8.9×[s] 1m 30.8s ▼2.7×[s]	4m 49.9s ▼8.1×[s] 1m 33.6s ▼2.7×[s]	·	10m 26.2s ▼10.4× 1m 48.4s ▼1.9×
R5	MPF / Bounded SIMD-DP / KSW2	4.6s ▲4.3× 2.2s ▲8.3×	4.1s ▲4.5× 1.9s ▲8.5×	9.9s ▲4.3×[s] 3.3s ▲12.1×[s]	4.7s ▲83.2× 1.9s ▲184.9×	11.5s ▼23%[s] 7.4s ▲12%[s]	10.8s ▼23%[s] 7.1s ▶ 1%[s]	15.4s ▶ 2%[s] 8.6s ▲1.5×[s]	12.5s ▲1.5× 9.7s ▲1.8×	1m 34.1s ▼2.2× 1m 24s ▼2.0×	1m 32.2s ▼2.2× 1m 20s ▼1.9×	·	1m 46.2s ▲2.7× 1m 35.2s ▲3.0×	5m 10.2s ▼8.9×[s] 1m 27.3s ▼2.6×[s]	4m 45.4s ▼7.9×[s] 1m 30.5s ▼2.6×[s]	·	10m 25.5s ▼10.4× 1m 44.7s ▼1.8×
R6	MPF / Bounded Scalar-DP / KSW2	4.6s ▲4.3× 2.2s ▲8.3×	4.1s ▲4.4× 1.9s ▲8.6×	9.7s ▲4.4×[s] 3.1s ▲12.6×[s]	5.8s ▲67.2× 3s ▲119.5×	9.5s ▶ 2%[s] 5.3s ▲1.6×[s]	8.3s ▶ 5%[s] 4.6s ▲1.5×[s]	14.2s ▲10%[s] 7.5s ▲1.7×[s]	13.3s ▲28% 10.5s ▲1.6×	1m 27.2s ▼2.1× 1m 17s ▼1.9×	1m 39.6s ▼2.3× 1m 26.7s ▼2.1×	·	1m 46.4s ▲2.7× 1m 36.3s ▲3.0×	5m 10.4s ▼8.9×[s] 1m 27.2s ▼2.6×[s]	4m 48.1s ▼8.0×[s] 1m 29.7s ▼2.6×[s]	·	10m 29.2s ▼10.5× 1m 47.6s ▼1.8×
R7	MPF / Bounded RMQ / KSW2	4.5s ▲4.3× 2.2s ▲8.4×	4.2s ▲4.3× 2s ▲8.4×	9.7s ▲4.4×[s] 3.2s ▲12.4×[s]	6.2s ▲63.0× 3.4s ▲104.6×	9.4s ▶ 1%[s] 5.3s ▲1.6×[s]	8.1s ▲8%[s] 4.6s ▲1.5×[s]	15.6s ▶ 1%[s] 9.1s ▲28%[s]	12.6s ▲1.5× 9.8s ▲1.7×	1m 32.9s ▼2.2× 1m 23.1s ▼2.0×	1m 41.2s ▼2.4× 1m 28.4s ▼2.2×	·	1m 46.6s ▲2.7× 1m 35.5s ▲3.0×	5m 9.8s ▼8.9×[s] 1m 27.1s ▼2.6×[s]	4m 52.9s ▼8.2×[s] 1m 32.9s ▼2.7×[s]	·	10m 33.7s ▼10.5× 1m 55.3s ▼2.0×
R8	MPS / Bounded Scalar-DP / Block	6.6s ▲3.0× 5.9s ▲3.2×	6.6s ▲2.7× 5.9s ▲2.8×	12.8s ▲3.3×[s] 10.5s ▲3.8×[s]	6.3s ▲61.6× 5.4s ▲65.7×	4m 37.1s ▼29.7×[s] 20s ▼2.4×[s]	3m 55.5s ▼27.0×[s] 19.7s ▼2.8×[s]	·	·	1m 42.1s ▼2.4× 1m 20s ▼1.9×	1m 37.4s ▼2.3× 1m 18s ▼1.9×	·	1m 56.9s ▲2.5× 1m 39.6s ▲2.9×	5m 28.5s ▼9.4×[s] 1m 29.3s ▼2.6×[s]	4m 45.1s ▼7.9×[s] 1m 22.8s ▼2.4×[s]	·	10m 46.5s ▼10.8× 1m 52.9s ▼1.9×

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Minimap2 stays first. Other rows follow increasing minima of available Index A times within this size/k/w group, across alphabets and all displayed machines. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 2 Gbp k=20, w=15								No index 2 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	19.6s ▼6.7× 18.6s ▼8.4×	18.2s ▼6.9× 16.5s ▼8.6×	42.5s ▼8.2× 39.4s ▼12.6×	6m 29.2s ▼92.8× 5m 56.9s ▼184.9×	9.3s ▼24% 8.4s ▼1.6×	8.7s ▼1.5× 7s ▼1.7×	15.8s ▼26% 12.7s ▼1.7×	18.5s ▼1.6× 17s ▼1.8×	42.4s ▲2.0× 41.4s ▲1.9×	42.9s ▲2.0× 41.1s ▲1.8×	1m 26.9s ▲1.9× 1m 23.7s ▲1.7×	4m 51.6s ▼2.9× 4m 46.6s ▼3.0×	35s ▲8.6× 34s ▲2.3×	35.9s ▲6.7× 34.2s ▲2.1×	1m 14.1s ▲6.9× 1m 11.1s ▲2.2×	1m 0.1s ▲10.4× 58.6s ▲1.8×
R9 BioseqDB	MPS / Bounded RMQ / Block	6.6s ▲3.0× 5.9s ▲3.2×	6.5s ▲2.8× 5.8s ▲2.8×	12.6s ▲3.4×[s] 10.3s ▲3.8×[s]	6.4s ▲61.0× 5.5s ▲65.2×	4m 37.8s ▼29.8×[s] 20.1s ▼2.4×[s]	3m 55.4s ▼26.9×[s] 19.8s ▼2.8×[s]	·	·	1m 43.2s ▼2.4× 1m 21s ▼2.0×	1m 41.8s ▼2.4× 1m 21.9s ▼2.0×	2m 55s ▼2.0× 2m 21s ▼1.7×	1m 57.1s ▲2.5× 1m 39.5s ▲2.9×	5m 28.1s ▼9.4×[s] 1m 28.8s ▼2.6×[s]	4m 44.5s ▼7.9×[s] 1m 22.4s ▼2.4×[s]	·	10m 46.5s ▼10.8× 1m 51.9s ▼1.9×
R10	MPS / Bounded SIMD-DP / Block	6.5s ▲3.0× 5.8s ▲3.2×	6.5s ▲2.8× 5.8s ▲2.8×	16.8s ▲2.5×[s] 14.2s ▲2.8×[s]	6.4s ▲60.5× 5.5s ▲64.7×	4m 39s ▼29.9×[s] 20.9s ▼2.5×[s]	3m 55.7s ▼27.0×[s] 20.1s ▼2.9×[s]	7m 43.9s ▼29.3×[s] 35.6s ▼2.8×[s]	·	1m 54.2s ▼2.7×[s] 1m 29.4s ▼2.2×[s]	1m 40.7s ▼2.4×[s] 1m 20.8s ▼2.0×[s]	·	1m 51.5s ▲2.6× 1m 35.2s ▲3.0×	5m 38.6s ▼9.7×[s] 1m 39.5s ▼2.9×[s]	4m 46.5s ▼8.0×[s] 1m 23.5s ▼2.4×[s]	8m 50.5s ▼7.2× 2m 35s ▼2.2×	10m 48.8s ▼10.8× 1m 54.4s ▼2.0×
R11	MPS / Bounded RMQ / KSW2	8.2s ▲2.4× 5.7s ▲3.3×	8s ▲2.3× 5.6s ▲2.9×	17.4s ▲2.4×[s] 10.1s ▲3.9×[s]	8.3s ▲46.6× 5.3s ▲67.2×	4m 39.5s ▼30.0×[s] 20.2s ▼2.4×[s]	3m 57.2s ▼27.2×[s] 19.8s ▼2.8×[s]	·	·	1m 48.4s ▼2.6× 1m 24.7s ▼2.0×	1m 39.1s ▼2.3× 1m 18s ▼1.9×	·	2m 0.6s ▲2.4× 1m 39.5s ▲2.9×	5m 38.2s ▼9.7×[s] 1m 37.1s ▼2.9×[s]	4m 45.8s ▼8.0×[s] 1m 22.4s ▼2.4×[s]	·	10m 55.2s ▼10.9× 1m 53.8s ▼1.9×
R12	MPS / Bounded SIMD-DP / KSW2	8.4s ▲2.3× 5.7s ▲3.3×	8s ▲2.3× 5.6s ▲2.9×	17.5s ▲2.4×[s] 10.1s ▲3.9×[s]	8.2s ▲47.3× 5.3s ▲67.7×	4m 39.1s ▼29.9×[s] 20.1s ▼2.4×[s]	3m 56.9s ▼27.1×[s] 19.4s ▼2.8×[s]	7m 26s ▼28.2×[s] 34.2s ▼2.7×[s]	·	1m 43.1s ▼2.4× 1m 19.2s ▼1.9×	1m 39.2s ▼2.3× 1m 17.9s ▼1.9×	·	1m 59.9s ▲2.4× 1m 38.2s ▲2.9×	5m 30.8s ▼9.5×[s] 1m 29.4s ▼2.6×[s]	4m 45.4s ▼7.9×[s] 1m 22.1s ▼2.4×[s]	·	10m 48.3s ▼10.8× 1m 48.9s ▼1.9×
R13	MPS / Bounded Scalar-DP / KSW2	8.2s ▲2.4× 5.7s ▲3.3×	8.1s ▲2.3× 5.6s ▲2.9×	17.7s ▲2.4×[s] 10.1s ▲3.9×[s]	8.4s ▲46.5× 5.3s ▲66.8×	4m 39.7s ▼30.0×[s] 20.4s ▼2.4×[s]	3m 56.6s ▼27.1×[s] 19.5s ▼2.8×[s]	·	·	1m 50.8s ▼2.6× 1m 27.1s ▼2.1×	1m 43.6s ▼2.4× 1m 21.8s ▼2.0×	·	2m 0.5s ▲2.4× 1m 38.7s ▲2.9×	5m 29.2s ▼9.4×[s] 1m 28s ▼2.6×[s]	·	·	10m 47.5s ▼10.8× 1m 51.8s ▼1.9×
R14	MPS / Two-stack SIMD-DP / Block	39.5s ▼2.0× 6.8s ▲2.7×	35.7s ▼2.0× 7.2s ▲2.3×	·	33.9s ▲11.5× 6.9s ▲51.8×	4m 25.9s ▼28.5×[s] 13.3s ▼1.6×[s]	3m 42s ▼25.4×[s] 12s ▼1.7×[s]	·	·	·	·	·	·	·	·	·	·
R15	MPS / SIMD-ref SIMD-DP / Block	46s ▼2.3×[s] 8.9s ▲2.1×[s]	40.5s ▼2.2×[s] 8.8s ▲1.9×[s]	·	34.8s ▲11.2× 8.2s ▲43.6×	4m 48.1s ▼30.9×[s] 30.2s ▼3.6×[s]	4m 3.1s ▼27.8×[s] 28.3s ▼4.0×[s]	·	·	·	·	·	2m 54.8s ▲1.7× 2m 26.4s ▲2.0×	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash. Minimap2 stays first. Other rows follow increasing minima of available Index A times within this size/k/w group, across alphabets and all displayed machines. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 2 Gbp k=20, w=15								No index 2 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	19.6s ▼6.7× 18.6s ▼8.4×	18.2s ▼6.9× 16.5s ▼8.6×	42.5s ▼8.2× 39.4s ▼12.6×	6m 29.2s ▼92.8× 5m 56.9s ▼184.9×	9.3s ▼24% 8.4s ▼1.6×	8.7s ▼1.5× 7s ▼1.7×	15.8s ▼26% 12.7s ▼1.7×	18.5s ▼1.6× 17s ▼1.8×	42.4s ▲2.0× 41.4s ▲1.9×	42.9s ▲2.0× 41.1s ▲1.8×	1m 26.9s ▲1.9× 1m 23.7s ▲1.7×	4m 51.6s ▼2.9× 4m 46.6s ▼3.0×	35s ▲8.6× 34s ▲2.3×	35.9s ▲6.7× 34.2s ▲2.1×	1m 14.1s ▲6.9× 1m 11.1s ▲2.2×	1m 0.1s ▲10.4× 58.6s ▲1.8×
R16 BioseqDB	MPS / Two-stack RMQ / Block	39.9s ▼2.0× 6.8s ▲2.7×	35.5s ▼1.9× 7s ▲2.4×	·	·	4m 24.9s ▼28.4×[s] 12.5s ▼1.5×[s]	3m 41.4s ▼25.3×[s] 11.5s ▼1.6×[s]	·	·	·	·	·	·	·	·	·	·
R17	MPS / Two-stack Scalar-DP / Block	39.8s ▼2.0× 6.8s ▲2.7×	35.8s ▼2.0× 6.5s ▲2.5×	·	·	4m 25.7s ▼28.5×[s] 12.7s ▼1.5×[s]	3m 41.4s ▼25.3×[s] 11.5s ▼1.6×[s]	·	·	·	·	·	·	·	·	·	·
R18	MPS / Scalar SIMD-DP / Block	43.8s ▼2.2× 8.6s ▲2.2×	38s ▼2.1× 8.7s ▲1.9×	·	36.6s ▲10.6× 8.2s ▲43.4×	4m 40.3s ▼30.1×[s] 23.1s ▼2.8×[s]	4m 0.1s ▼27.5×[s] 22.3s ▼3.2×[s]	·	·	2m 52.9s ▼4.1× 2m 16.2s ▼3.3×	2m 34.3s ▼3.6× 2m 5.6s ▼3.1×	·	2m 43.8s ▲1.8× 2m 16.9s ▲2.1×	6m 48.5s ▼11.7×[s] 2m 32s ▼4.5×[s]	·	·	·
R19	MPS / Scalar RMQ / Block	43.6s ▼2.2× 8.8s ▲2.1×	37.4s ▼2.1× 8.8s ▲1.9×	·	·	4m 39.7s ▼30.0×[s] 22s ▼2.6×[s]	3m 57.3s ▼27.2×[s] 21.1s ▼3.0×[s]	·	·	2m 53.3s ▼4.1× 2m 17.7s ▼3.3×	2m 48s ▼3.9× 2m 18.9s ▼3.4×	3m 24s ▼2.3× 2m 47.8s ▼2.0×	·	6m 48.4s ▼11.7×[s] 2m 33s ▼4.5×[s]	·	·	·
R20	MPS / Scalar Scalar-DP / Block	43.3s ▼2.2× 8.7s ▲2.2×	37.5s ▼2.1× 8.9s ▲1.9×	·	·	4m 39.8s ▼30.0×[s] 22s ▼2.6×[s]	3m 57.2s ▼27.1×[s] 20.8s ▼3.0×[s]	·	·	2m 57.9s ▼4.2×[s] 2m 19.7s ▼3.4×[s]	·	·	·	·	·	·	·
R21	MPS / Scalar SIMD-DP / KSW2	45.1s ▼2.3× 8.4s ▲2.2×	37.9s ▼2.1× 8.2s ▲2.0×	1m 6.6s ▼1.6× 14.5s ▲2.7×	40.5s ▲9.6× 8.3s ▲43.2×	4m 40.4s ▼30.1×[s] 21.8s ▼2.6×[s]	3m 58.8s ▼27.3×[s] 20.9s ▼3.0×[s]	·	·	2m 54.8s ▼4.1× 2m 18.1s ▼3.3×	2m 39.7s ▼3.7× 2m 8.9s ▼3.1×	4m 27.6s ▼3.1× 3m 34.6s ▼2.6×	·	6m 52.2s ▼11.8×[s] 2m 34.3s ▼4.5×[s]	·	·	·
R22	MPS / SIMD-ref RMQ / Block	46.5s ▼2.4×[s] 9.2s ▲2.0×[s]	38.3s ▼2.1×[s] 8.8s ▲1.9×[s]	·	·	4m 47s ▼30.8×[s] 28.9s ▼3.4×[s]	4m 2.7s ▼27.8×[s] 28.4s ▼4.0×[s]	·	·	·	·	3m 51.4s ▼2.7× 3m 1.9s ▼2.2×	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash. Minimap2 stays first. Other rows follow increasing minima of available Index A times within this size/k/w group, across alphabets and all displayed machines. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 2 Gbp k=20, w=15								No index 2 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	19.6s ▼6.7× 18.6s ▼8.4×	18.2s ▼6.9× 16.5s ▼8.6×	42.5s ▼8.2× 39.4s ▼12.6×	6m 29.2s ▼92.8× 5m 56.9s ▼184.9×	9.3s ▼24% 8.4s ▼1.6×	8.7s ▼1.5× 7s ▼1.7×	15.8s ▼26% 12.7s ▼1.7×	18.5s ▼1.6× 17s ▼1.8×	42.4s ▲2.0× 41.4s ▲1.9×	42.9s ▲2.0× 41.1s ▲1.8×	1m 26.9s ▲1.9× 1m 23.7s ▲1.7×	4m 51.6s ▼2.9× 4m 46.6s ▼3.0×	35s ▲8.6× 34s ▲2.3×	35.9s ▲6.7× 34.2s ▲2.1×	1m 14.1s ▲6.9× 1m 11.1s ▲2.2×	1m 0.1s ▲10.4× 58.6s ▲1.8×
R23 BioseqDB	MPS / SIMD-ref Scalar-DP / Block	46.3s ▼2.4×[s] 9.2s ▲2.0×[s]	38.4s ▼2.1×[s] 8.8s ▲1.9×[s]	·	·	4m 46.5s ▼30.7×[s] 28.7s ▼3.4×[s]	4m 0.9s ▼27.6×[s] 26.6s ▼3.8×[s]	·	·	·	·	·	·	·	·	·	·
R24	MPS / Scalar Scalar-DP / KSW2	45.2s ▼2.3× 8.7s ▲2.1×	39.2s ▼2.2× 8.8s ▲1.9×	1m 6.9s ▼1.6× 14.8s ▲2.7×	·	4m 40.9s ▼30.1×[s] 21.8s ▼2.6×[s]	3m 58.9s ▼27.3×[s] 20.7s ▼2.9×[s]	·	·	2m 57.6s ▼4.2×[s] 2m 18.2s ▼3.3×[s]	·	·	·	·	·	·	·
R25	MPS / Two-stack Scalar-DP / KSW2	43.6s ▼2.2× 6.8s ▲2.7×	39.5s ▼2.2× 7.2s ▲2.3×	·	·	4m 31.6s ▼29.1×[s] 12.4s ▼1.5×[s]	3m 44.6s ▼25.7×[s] 11.3s ▼1.6×[s]	·	·	·	·	·	·	·	·	·	·
R26	MPS / Two-stack SIMD-DP / KSW2	43.7s ▼2.2× 6.8s ▲2.7×	39.6s ▼2.2× 7.2s ▲2.3×	·	·	4m 30s ▼29.0×[s] 12.4s ▼1.5×[s]	3m 45.3s ▼25.8×[s] 11.5s ▼1.6×[s]	·	·	·	·	·	·	·	·	·	·
R27	MPS / Two-stack RMQ / KSW2	43.4s ▼2.2× 6.8s ▲2.7×	39.6s ▼2.2× 6.7s ▲2.4×	·	·	4m 29.8s ▼28.9×[s] 12.6s ▼1.5×[s]	3m 45.3s ▼25.8×[s] 11.5s ▼1.6×[s]	·	·	·	·	·	·	·	·	·	·
R28	MPS / Scalar RMQ / KSW2	45.4s ▼2.3× 8.8s ▲2.1×	39.7s ▼2.2× 9s ▲1.8×	·	·	4m 41.5s ▼30.2×[s] 22.4s ▼2.7×[s]	4m 0.1s ▼27.5×[s] 21.1s ▼3.0×[s]	·	·	2m 52.9s ▼4.1× 2m 15.4s ▼3.3×	2m 47.4s ▼3.9× 2m 15.8s ▼3.3×	·	·	·	·	·	·
R29	MPS / SIMD-ref Scalar-DP / KSW2	47.8s ▼2.4×[s] 9.1s ▲2.0×[s]	39.9s ▼2.2×[s] 8.7s ▲1.9×[s]	·	·	4m 48.7s ▼31.0×[s] 28.7s ▼3.4×[s]	4m 3.3s ▼27.8×[s] 27s ▼3.8×[s]	·	·	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash. Minimap2 stays first. Other rows follow increasing minima of available Index A times within this size/k/w group, across alphabets and all displayed machines. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 2 Gbp k=20, w=15								No index 2 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	19.6s ▼6.7× 18.6s ▼8.4×	18.2s ▼6.9× 16.5s ▼8.6×	42.5s ▼8.2× 39.4s ▼12.6×	6m 29.2s ▼92.8× 5m 56.9s ▼184.9×	9.3s ▼24% 8.4s ▼1.6×	8.7s ▼1.5× 7s ▼1.7×	15.8s ▼26% 12.7s ▼1.7×	18.5s ▼1.6× 17s ▼1.8×	42.4s ▲2.0× 41.4s ▲1.9×	42.9s ▲2.0× 41.1s ▲1.8×	1m 26.9s ▲1.9× 1m 23.7s ▲1.7×	4m 51.6s ▼2.9× 4m 46.6s ▼3.0×	35s ▲8.6× 34s ▲2.3×	35.9s ▲6.7× 34.2s ▲2.1×	1m 14.1s ▲6.9× 1m 11.1s ▲2.2×	1m 0.1s ▲10.4× 58.6s ▲1.8×
R30 BioseqDB	MPS / SIMD-ref RMQ / KSW2	48.4s ▼2.5×[s] 9.1s ▲2.0×[s]	40.4s ▼2.2×[s] 8.8s ▲1.9×[s]	·	·	4m 48.1s ▼30.9×[s] 28.9s ▼3.4×[s]	4m 3.5s ▼27.9×[s] 27.1s ▼3.9×[s]	·	·	·	·	·	·	·	·	·	·
R31	MPS / SIMD-ref SIMD-DP / KSW2	47.5s ▼2.4×[s] 8.9s ▲2.1×[s]	40.7s ▼2.2×[s] 8.9s ▲1.9×[s]	1m 6.7s ▼1.6× 14.4s ▲2.7×	·	4m 47.7s ▼30.9×[s] 28.5s ▼3.4×[s]	4m 3.9s ▼27.9×[s] 27.7s ▼3.9×[s]	·	·	·	·	·	·	·	·	·	·
R32	MPF / Two-stack RMQ / Block	4m 37.6s ▼14.2× 4m 37.1s ▼14.9×	7m 30s ▼24.7× 7m 29.2s ▼27.2×	17m 24.5s ▼24.6×[s] 17m 22.9s ▼26.5×[s]	·	3m 23s ▼21.8×[s] 3m 21.6s ▼24.0×[s]	5m 46.9s ▼39.7×[s] 5m 45.4s ▼49.2×[s]	11m 57.4s ▼45.4×[s] 11m 54.8s ▼56.1×[s]	·	1m 54.6s ▼2.7×[s] 1m 19s ▼1.9×[s]	1m 45.2s ▼2.5×[s] 1m 14.4s ▼1.8×[s]	·	·	5m 31.5s ▼9.5×[s] 1m 18.5s ▼2.3×[s]	4m 59.4s ▼8.3×[s] 1m 13.4s ▼2.1×[s]	·	·
R33	MPF / Two-stack RMQ / KSW2	4m 34.2s ▼14.0× 4m 32.2s ▼14.6×	6m 52.3s ▼22.7× 6m 49.6s ▼24.8×	17m 29.3s ▼24.7×[s] 17m 23.1s ▼26.5×[s]	·	3m 23.9s ▼21.9×[s] 3m 21.3s ▼24.0×[s]	5m 43s ▼39.3×[s] 5m 39.6s ▼48.3×[s]	12m 2.7s ▼45.7×[s] 11m 55.6s ▼56.2×[s]	·	1m 59.6s ▼2.8×[s] 1m 19.6s ▼1.9×[s]	1m 48.5s ▼2.5×[s] 1m 14.5s ▼1.8×[s]	·	·	5m 35.5s ▼9.6×[s] 1m 17.7s ▼2.3×[s]	5m 2s ▼8.4×[s] 1m 12.5s ▼2.1×[s]	·	·
R34	MPF / Two-stack SIMD-DP / Block	4m 36.1s ▼14.1× 4m 35.5s ▼14.8×	7m 3.9s ▼23.3× 7m 3s ▼25.6×	17m 28.7s ▼24.7×[s] 17m 27.2s ▼26.6×[s]	5m 27s ▲16% 5m 26s ▲9%	3m 24.2s ▼21.9×[s] 3m 22.7s ▼24.2×[s]	5m 44.1s ▼39.4×[s] 5m 42.7s ▼48.8×[s]	11m 58.9s ▼45.5×[s] 11m 56.2s ▼56.2×[s]	4m 0.4s ▼13.0× 3m 59s ▼14.0×	1m 57.3s ▼2.8×[s] 1m 21.6s ▼2.0×[s]	1m 45.1s ▼2.5×[s] 1m 15.1s ▼1.8×[s]	·	·	5m 30.7s ▼9.5×[s] 1m 17.9s ▼2.3×[s]	4m 57.2s ▼8.3×[s] 1m 13.2s ▼2.1×[s]	·	·
R35	MPF / Two-stack SIMD-DP / KSW2	4m 37.8s ▼14.2× 4m 35.9s ▼14.8×	7m 37.6s ▼25.1× 7m 34.4s ▼27.5×	17m 30.1s ▼24.7×[s] 17m 23.9s ▼26.5×[s]	·	3m 24.7s ▼22.0×[s] 3m 22.1s ▼24.1×[s]	5m 44.4s ▼39.4×[s] 5m 40.9s ▼48.5×[s]	12m 4s ▼45.8×[s] 11m 56.8s ▼56.3×[s]	·	1m 58.9s ▼2.8×[s] 1m 18.9s ▼1.9×[s]	1m 47.5s ▼2.5×[s] 1m 13.8s ▼1.8×[s]	·	·	5m 34.8s ▼9.6×[s] 1m 17.6s ▼2.3×[s]	5m 3.8s ▼8.5×[s] 1m 12.8s ▼2.1×[s]	·	·
R36	MPF / Two-stack Scalar-DP / KSW2	4m 38.1s ▼14.2× 4m 36.2s ▼14.8×	7m 34.7s ▼25.0× 7m 31.6s ▼27.3×	17m 28.2s ▼24.7×[s] 17m 22.1s ▼26.5×[s]	·	3m 24.7s ▼22.0×[s] 3m 22.1s ▼24.1×[s]	5m 43.5s ▼39.3×[s] 5m 40.1s ▼48.4×[s]	12m 2.2s ▼45.7×[s] 11m 55s ▼56.1×[s]	·	1m 59.1s ▼2.8×[s] 1m 19.3s ▼1.9×[s]	1m 47.1s ▼2.5×[s] 1m 13.3s ▼1.8×[s]	·	·	5m 34.9s ▼9.6×[s] 1m 17.2s ▼2.3×[s]	4m 58.5s ▼8.3×[s] 1m 10.5s ▼2.1×[s]	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash. Minimap2 stays first. Other rows follow increasing minima of available Index A times within this size/k/w group, across alphabets and all displayed machines. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 2 Gbp k=20, w=15								No index 2 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	19.6s ▼6.7× 18.6s ▼8.4×	18.2s ▼6.9× 16.5s ▼8.6×	42.5s ▼8.2× 39.4s ▼12.6×	6m 29.2s ▼92.8× 5m 56.9s ▼184.9×	9.3s ▼24% 8.4s ▼1.6×	8.7s ▼1.5× 7s ▼1.7×	15.8s ▼26% 12.7s ▼1.7×	18.5s ▼1.6× 17s ▼1.8×	42.4s ▲2.0× 41.4s ▲1.9×	42.9s ▲2.0× 41.1s ▲1.8×	1m 26.9s ▲1.9× 1m 23.7s ▲1.7×	4m 51.6s ▼2.9× 4m 46.6s ▼3.0×	35s ▲8.6× 34s ▲2.3×	35.9s ▲6.7× 34.2s ▲2.1×	1m 14.1s ▲6.9× 1m 11.1s ▲2.2×	1m 0.1s ▲10.4× 58.6s ▲1.8×
R37 BioseqDB	MPF / Two-stack Scalar-DP / Block	4m 36s ▼14.1× 4m 35.5s ▼14.8×	6m 53.8s ▼22.7× 6m 52.9s ▼25.0×	17m 25.1s ▼24.6×[s] 17m 23.5s ▼26.5×[s]	·	3m 24.8s ▼22.0×[s] 3m 23.4s ▼24.3×[s]	5m 38s ▼38.7×[s] 5m 36.6s ▼47.9×[s]	11m 57.4s ▼45.4×[s] 11m 54.9s ▼56.1×[s]	·	1m 55.2s ▼2.7×[s] 1m 18.8s ▼1.9×[s]	1m 43.5s ▼2.4×[s] 1m 13.7s ▼1.8×[s]	·	·	5m 30.7s ▼9.5×[s] 1m 17.1s ▼2.3×[s]	4m 55s ▼8.2×[s] 1m 11.7s ▼2.1×[s]	·	·
R38	MPF / Scalar Scalar-DP / Block	5m 39.5s ▼17.3× 5m 39s ▼18.2×	12m 20.4s ▼40.7× 12m 19.8s ▼44.8×	22m 35.5s ▼31.9× 22m 34.5s ▼34.4×	12m 31.2s ▼1.9× 12m 30.6s ▼2.1×	7m 58.4s ▼51.3×[s] 7m 54.7s ▼56.6×[s]	17m 10s ▼117.9×[s] 17m 7.3s ▼146.2×[s]	29m 8.1s ▼110.6×[s] 29m 2.5s ▼136.8×[s]	15m 18.5s ▼49.7×[s] 15m 15.3s ▼53.8×[s]	2m 53.1s ▼4.1×[s] 2m 14.9s ▼3.3×[s]	2m 43.4s ▼3.8×[s] 2m 13.1s ▼3.2×[s]	·	·	6m 44s ▼11.6×[s] 2m 28s ▼4.4×[s]	6m 4.7s ▼10.2×[s] 2m 28s ▼4.3×[s]	·	·
R39	MPF / Scalar Scalar-DP / KSW2	5m 39.7s ▼17.3× 5m 37.2s ▼18.1×	12m 8.9s ▼40.0× 12m 7.2s ▼44.0×	22m 21.6s ▼31.6×[s] 22m 15.8s ▼33.9×[s]	10m 39.3s ▼1.6× 10m 36.8s ▼1.8×	7m 46.7s ▼50.1×[s] 7m 43s ▼55.2×[s]	16m 59.7s ▼116.7×[s] 16m 56.8s ▼144.7×[s]	28m 56.8s ▼109.8×[s] 28m 50.8s ▼135.9×[s]	15m 4.5s ▼48.9×[s] 15m 0.9s ▼52.9×[s]	2m 57.1s ▼4.2×[s] 2m 16.9s ▼3.3×[s]	3m 14.4s ▼4.5×[s] 2m 40.8s ▼3.9×[s]	·	·	6m 47s ▼11.6×[s] 2m 28.7s ▼4.4×[s]	6m 3s ▼10.1×[s] 2m 24.7s ▼4.2×[s]	·	·
R40	MPF / Scalar SIMD-DP / Block	5m 39.7s ▼17.3× 5m 39.2s ▼18.2×	12m 5.2s ▼39.8× 12m 4.6s ▼43.9×	22m 32.6s ▼31.8× 22m 31.6s ▼34.3×	12m 48.6s ▼2.0× 12m 47.9s ▼2.2×	7m 52s ▼50.6×[s] 7m 48.3s ▼55.9×[s]	16m 56.1s ▼116.3×[s] 16m 53.4s ▼144.2×[s]	29m 42.8s ▼112.8×[s] 29m 37.4s ▼139.5×[s]	15m 28s ▼50.2×[s] 15m 24.8s ▼54.3×[s]	2m 45.2s ▼3.9× 2m 8.9s ▼3.1×	2m 44.3s ▼3.8× 2m 14.4s ▼3.3×	·	2m 41s ▲1.8× 2m 13.3s ▲2.2×	6m 49.3s ▼11.7×[s] 2m 33s ▼4.5×[s]	6m 44.7s ▼11.3×[s] 2m 51.6s ▼5.0×[s]	·	11m 45.1s ▼11.7× 2m 31.9s ▼2.6×
R41	MPF / Scalar RMQ / Block	5m 39.9s ▼17.3× 5m 39.5s ▼18.2×	12m 19.6s ▼40.6× 12m 19.1s ▼44.8×	22m 41.7s ▼32.0× 22m 40.7s ▼34.6×	12m 57s ▼2.0× 12m 56.4s ▼2.2×	7m 52.2s ▼50.7×[s] 7m 48.5s ▼55.9×[s]	17m 9.4s ▼117.8×[s] 17m 6.8s ▼146.1×[s]	21m 1.2s ▼79.8× 20m 58.9s ▼98.8×	15m 4.2s ▼48.9×[s] 15m 1.3s ▼52.9×[s]	2m 49.9s ▼4.0× 2m 13.4s ▼3.2×	2m 29.9s ▼3.5× 2m 1.1s ▼2.9×	·	·	6m 44.4s ▼11.6×[s] 2m 28.2s ▼4.4×[s]	6m 31.8s ▼10.9×[s] 2m 38.1s ▼4.6×[s]	·	·
R42	MPF / SIMD-ref Scalar-DP / KSW2	5m 40.6s ▼17.4×[s] 5m 36.9s ▼18.1×[s]	12m 16.8s ▼40.5×[s] 12m 13.7s ▼44.4×[s]	22m 52.9s ▼32.3×[s] 22m 46.8s ▼34.7×[s]	12m 8.2s ▼1.9×[s] 12m 4.8s ▼2.0×[s]	6m 5.2s ▼39.2×[s] 6m 1.5s ▼43.1×[s]	13m 19.6s ▼91.5×[s] 13m 16.8s ▼113.4×[s]	24m 19.3s ▼92.3×[s] 24m 13.1s ▼114.1×[s]	11m 40.3s ▼37.9×[s] 11m 36.8s ▼40.9×[s]	2m 42s ▼3.8×[s] 2m 2.1s ▼2.9×[s]	2m 30.1s ▼3.5×[s] 1m 57.8s ▼2.9×[s]	·	·	6m 17.7s ▼10.8×[s] 2m 16.5s ▼4.0×[s]	6m 21.8s ▼10.6×[s] 2m 44.5s ▼4.8×[s]	·	·
R43	MPF / Scalar SIMD-DP / KSW2	5m 42.8s ▼17.5× 5m 40.6s ▼18.3×	12m 19.8s ▼40.6× 12m 18s ▼44.7×	22m 43.4s ▼32.1× 22m 39.5s ▼34.5×	11m 32.1s ▼1.8× 11m 29.4s ▼1.9×	8m 18.5s ▼53.5×[s] 8m 14.8s ▼59.0×[s]	16m 50.3s ▼115.6×[s] 16m 47.4s ▼143.4×[s]	29m 22.4s ▼111.5×[s] 29m 16.4s ▼137.9×[s]	15m 17.5s ▼49.6×[s] 15m 13.9s ▼53.7×[s]	2m 52.8s ▼4.1× 2m 15.7s ▼3.3×	2m 32.2s ▼3.6× 2m 0.8s ▼2.9×	·	·	6m 48.6s ▼11.7×[s] 2m 28.4s ▼4.4×[s]	6m 49.3s ▼11.4×[s] 2m 53.7s ▼5.1×[s]	·	·

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash. Minimap2 stays first. Other rows follow increasing minima of available Index A times within this size/k/w group, across alphabets and all displayed machines. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

Engine	Execution	Index 2 Gbp k=20, w=15								No index 2 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	19.6s ▼6.7× 18.6s ▼8.4×	18.2s ▼6.9× 16.5s ▼8.6×	42.5s ▼8.2× 39.4s ▼12.6×	6m 29.2s ▼92.8× 5m 56.9s ▼184.9×	9.3s ▼24% 8.4s ▼1.6×	8.7s ▼1.5× 7s ▼1.7×	15.8s ▼26% 12.7s ▼1.7×	18.5s ▼1.6× 17s ▼1.8×	42.4s ▲2.0× 41.4s ▲1.9×	42.9s ▲2.0× 41.1s ▲1.8×	1m 26.9s ▲1.9× 1m 23.7s ▲1.7×	4m 51.6s ▼2.9× 4m 46.6s ▼3.0×	35s ▲8.6× 34s ▲2.3×	35.9s ▲6.7× 34.2s ▲2.1×	1m 14.1s ▲6.9× 1m 11.1s ▲2.2×	1m 0.1s ▲10.4× 58.6s ▲1.8×
R44 BioseqDB	MPF / SIMD-ref RMQ / Block	5m 44s ▼17.6×[s] 5m 43.3s ▼18.4×[s]	12m 19.6s ▼40.6×[s] 12m 18.9s ▼44.7×[s]	23m 12.3s ▼32.7×[s] 23m 11s ▼35.3×[s]	11m 15.2s ▼1.7×[s] 11m 14.4s ▼1.9×[s]	6m 4.6s ▼39.1×[s] 6m 1s ▼43.1×[s]	13m 18.8s ▼91.4×[s] 13m 16.2s ▼113.3×[s]	23m 19.7s ▼88.5× 23m 16.1s ▼109.6×	12m 38s ▼41.0×[s] 12m 35s ▼44.4×[s]	2m 40.3s ▼3.8×[s] 2m 2.4s ▼3.0×[s]	2m 36.1s ▼3.6×[s] 2m 4.7s ▼3.0×[s]	·	·	6m 15.1s ▼10.7×[s] 2m 16.3s ▼4.0×[s]	6m 12.9s ▼10.4×[s] 2m 40.4s ▼4.7×[s]	·	·
R45	MPF / Scalar RMQ / KSW2	5m 46.6s ▼17.7× 5m 44.5s ▼18.5×	12m 26.1s ▼41.0× 12m 24.3s ▼45.1×	22m 22.5s ▼31.6×[s] 22m 16.8s ▼33.9×[s]	11m 45.8s ▼1.8× 11m 43.3s ▼2.0×	7m 49.2s ▼50.3×[s] 7m 45.6s ▼55.5×[s]	17m 0.3s ▼116.8×[s] 16m 57.4s ▼144.8×[s]	29m 26.3s ▼111.7×[s] 29m 20.3s ▼138.2×[s]	15m 2.9s ▼48.8×[s] 14m 59.6s ▼52.8×[s]	2m 53.6s ▼4.1× 2m 15.9s ▼3.3×	2m 44s ▼3.8× 2m 12.1s ▼3.2×	·	·	6m 46.2s ▼11.6×[s] 2m 28.4s ▼4.4×[s]	6m 33.5s ▼11.0×[s] 2m 49.4s ▼5.0×[s]	·	·
R46	MPF / SIMD-ref SIMD-DP / KSW2	5m 50.5s ▼17.9×[s] 5m 46.9s ▼18.6×[s]	12m 30.9s ▼41.3×[s] 12m 27.6s ▼45.3×[s]	22m 56s ▼32.4×[s] 22m 50s ▼34.8×[s]	11m 32.8s ▼1.8×[s] 11m 29.4s ▼1.9×[s]	6m 4.7s ▼39.1×[s] 6m 1.1s ▼43.1×[s]	13m 22.2s ▼91.8×[s] 13m 19.2s ▼113.7×[s]	25m 24.5s ▼96.4×[s] 25m 18.3s ▼119.2×[s]	12m 52.2s ▼41.8×[s] 12m 48.8s ▼45.2×[s]	2m 40.4s ▼3.8×[s] 2m 0.9s ▼2.9×[s]	2m 41.5s ▼3.8×[s] 2m 8.1s ▼3.1×[s]	·	·	6m 16.9s ▼10.8×[s] 2m 16s ▼4.0×[s]	6m 11s ▼10.3×[s] 2m 39.5s ▼4.7×[s]	·	·
R47	MPF / SIMD-ref RMQ / KSW2	5m 50.6s ▼17.9×[s] 5m 47s ▼18.6×[s]	12m 12.8s ▼40.3×[s] 12m 9.8s ▼44.2×[s]	23m 7.9s ▼32.6×[s] 23m 1.9s ▼35.1×[s]	10m 52.1s ▼1.7× 10m 49.7s ▼1.8×	6m 6.7s ▼39.3×[s] 6m 3s ▼43.3×[s]	13m 19.1s ▼91.5×[s] 13m 16.2s ▼113.3×[s]	24m 33.5s ▼93.2×[s] 24m 27.3s ▼115.2×[s]	12m 1.2s ▼39.0×[s] 11m 57.8s ▼42.2×[s]	2m 41.8s ▼3.8×[s] 2m 2.1s ▼2.9×[s]	2m 37.2s ▼3.7×[s] 2m 3.8s ▼3.0×[s]	·	·	6m 18.7s ▼10.8×[s] 2m 18s ▼4.1×[s]	6m 21.7s ▼10.6×[s] 2m 44.6s ▼4.8×[s]	·	·
R48	MPF / SIMD-ref SIMD-DP / Block	5m 50.7s ▼17.9×[s] 5m 49.9s ▼18.8×[s]	12m 26.8s ▼41.0×[s] 12m 25.9s ▼45.2×[s]	22m 58.7s ▼32.4×[s] 22m 57.3s ▼35.0×[s]	13m 2.2s ▼2.0× 13m 1.5s ▼2.2×	6m 8.9s ▼39.6×[s] 6m 5.3s ▼43.6×[s]	13m 31.2s ▼92.8×[s] 13m 28.4s ▼115.0×[s]	24m 58.1s ▼94.7×[s] 24m 52.4s ▼117.1×[s]	13m 3.8s ▼42.4×[s] 13m 0.9s ▼45.9×[s]	2m 38.4s ▼3.7×[s] 2m 0.1s ▼2.9×[s]	2m 35.4s ▼3.6×[s] 2m 4.3s ▼3.0×[s]	·	2m 49.5s ▲1.7× 2m 21.9s ▲2.0×	6m 15.5s ▼10.7×[s] 2m 16.4s ▼4.0×[s]	5m 52.6s ▼9.8×[s] 2m 26.6s ▼4.3×[s]	·	12m 1.9s ▼12.0× 3m 7.8s ▼3.2×
R49	MPF / SIMD-ref Scalar-DP / Block	5m 51.9s ▼18.0×[s] 5m 51.1s ▼18.9×[s]	12m 10.6s ▼40.1×[s] 12m 9.8s ▼44.2×[s]	23m 42.4s ▼33.5×[s] 23m 41s ▼36.1×[s]	11m 56.4s ▼1.8×[s] 11m 55.6s ▼2.0×[s]	6m 3.8s ▼39.0×[s] 6m 0.1s ▼43.0×[s]	13m 15.2s ▼91.0×[s] 13m 12.6s ▼112.8×[s]	24m 21.9s ▼92.5×[s] 24m 16.2s ▼114.3×[s]	11m 53.1s ▼38.6×[s] 11m 50s ▼41.7×[s]	2m 40s ▼3.8×[s] 2m 1.7s ▼2.9×[s]	2m 29.9s ▼3.5×[s] 1m 58.9s ▼2.9×[s]	·	·	6m 18.4s ▼10.8×[s] 2m 19.3s ▼4.1×[s]	6m 19.9s ▼10.6×[s] 2m 45.3s ▼4.8×[s]	·	·
R50 BioseqDB	Vulkan (all modes)	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]

Each cell: A, comparison, B, comparison. Times cover 1,024 queries. · = no approved observation. —[g] = GPU unavailable, not executed. n/a = no measured comparison.
[f]SIMD-ref request observed as scalar, compared only as that effective path. [p]Accepted by explicit preservation after wrapper postflight failure, no run receipt claimed.
MPF/MPS = minimizers per fragment/sequence. Labels: index representation / minimizer, then chaining / base alignment. SIMD-ref is the reference minimizer, Bounded is bounded ntHash.
Minimap2 stays first. Other rows follow increasing minima of available Index A times within this size/k/w group, across alphabets and all displayed machines. No Index results = last. Coverage varies.
[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.
Blue = one lowest A and one lowest B down each machine/workload column, across continuation pages. Exact ties select the first row. Repeated baselines are marked only on page 1.
Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column.
When all Vulkan modes are unmeasured, they share one row. Baseline comparisons use the fastest measured BioseqDB effective path separately for A and B.

12.1.4. Comparison with other aligners

The external-aligner table places minimap2 and one fixed BioseqDB configuration alongside blastn, Winnowmap and LexicMap. Both minimap2 and BioseqDB timings come from the corresponding machine's non-external remote summary, matched by machine class, target size, alphabet, (k, w) and index mode. External-campaign minimap2 timings are excluded. BioseqDB always uses minimizers per fragment (MPF), bounded ntHash SIMD minimizers, SIMD-DP chaining and Block Aligner. Missing observations are marked with a dot. Both rows select the lowest-A observation across default / TEST3 / TEST4 / TEST7 / TEST2 / FAST sources, retaining its B and sample count. Source order breaks exact A ties only. FAST entries carry a single-sample footnote. No other BioseqDB configuration or historical timing scope is substituted.

The table contains 316 distinct observations and 1,548 accepted repeats. Only approved results are included. Qualification runs, interrupted attempts and the small-query checks used before the full runs are excluded.

Measurement procedure.

- Each external observation has five fresh-process repeats of 1,024 queries. A is the median wall time, including executable startup, index loading, mapping and writing output. Fixture generation and validation are outside the timer. The index mode prepares the native on-disk index beforehand. No-index mode creates it inside the measured interval, including tool-specific preparation such as counting frequent k-mers or partitioning the reference. Operating-system caches are not purged.

- The external campaign checks one primary occurrence per query, its position and strand, and an edit script reconstructed from the gapped output. These files include the five raw timing samples and returned-count vectors. This is additional evidence beyond the median-only summaries used for the earlier tables. MAPQ is checked only where the tool provides it. No MAPQ value is invented for blastn or LexicMap.

- B is computed separately in each repeat by subtracting the union of the tool's instrumented alignment and available MAPQ intervals, then taking the median. BLAST subtracts traceback-search regions. LexicMap subtracts wavefront-alignment calls. Winnowmap subtracts its native alignment and MAPQ intervals. These are different boundaries, so B is a tool-specific remaining time, not the duration of a common seeding-and-chaining stage.

How the tools were configured.

- blastn 2.17.0+. The word size equals the requested k. There is no minimizer-window setting, so w labels the fixture only. Low-complexity masking is disabled, and each reported alignment must cover at least 80% of the query to remove short chance matches. Match reward 2, mismatch penalty 4, gap-open penalty 4 and gap-extension penalty 2 reproduce the baseline's first affine-gap branch. BLAST has no second branch. The default E-value threshold of 10 is retained. DNA5 cells are marked because BLAST scores N differently.

- Winnowmap. Both k and w are retained. Frequent k-mers are counted with Meryl to build weighted minimizers. This preparation is included in no-index time. Baseline scoring and first-stage bandwidth are retained, while Winnowmap's additional

structural-variant-aware stage keeps its native settings. Secondary alignments are suppressed. The benchmark build restores persisted-index output and reconstructs the weighting filter on loading. It also initializes a repeat-length variable to prevent undefined MAPQ/output behaviour. Memory-limited and multipart runs carry cell-specific notes.

- LexicMap. The requested k is retained, but LexicHash masks replace minimizer windows, so w is inapplicable. The native 20,000 masks, gap filling between sparsely seeded regions and low-complexity rejection remain enabled. Query coverage is at least 80%. Its native wavefront-alignment costs are match 0, mismatch 4 and gap open/extension 6/2. Its separate reported-score calculation uses match 2, mismatch -3 and gaps 5/2. These settings are not rewritten to mimic minimap2. References are divided into records of at most 268,435,455 bases with overlap equal to the longest query, and output coordinates are restored to the logical reference. DNA5 cells are marked because its native reference encoding replaces N with A.

What the comparison establishes. The external runs used new instances of the same EC2 types, not the physical instances used for the retained minimap2 and BioseqDB results. External volumes are 300 GiB unencrypted gp3, compared with 500 GiB encrypted gp3 in the earlier campaign. Both provision 6,000 IOPS and 500 MiB/s. The external host bindings record four benchmark threads, nice -20 and Clang 18.1.3, whereas the earlier campaign used Clang 22.1.8. Compressed-RAM swap is recorded for many external runs. Its presence does not prove how much swapping occurred during each measurement.

The workload dimensions agree, but the timing boundaries, reference layouts, scoring and resource conditions are not identical. In particular, the retained framework timings exclude process-startup setup, and BioseqDB no-index mode constructs a transient index rather than the external tools' on-disk indexes. The summaries do not establish cross-campaign byte-for-byte fixture identity. Ratios therefore describe the published elapsed times under these qualified conditions, rather than an isolated causal effect of the aligner.

The table retains the green/red/gray triangles, 5% tie band and × notation above 30%. Every non-baseline row compares with minimap2 in the same machine-class/workload column. Minimap2 compares only with the displayed fixed BioseqDB configuration, not another faster BioseqDB row. Blue marks one lowest reported A and one lowest reported B down the five rows in each column. Exact ties select the first displayed row. Light gray identifies the minimap2 baseline. Pale yellow identifies the preferred MPF / Bounded / SIMD-DP / Block configuration, independently of the measured winners. For B, both the ratio and the highlight compare native residual times with different work removed, and do not establish superiority at an equivalent processing stage. Dots mean no approved timing, not failure or hardware unavailability.

Different campaigns and native timing boundaries. A/B ratios are descriptive, subject to the measurement notes.

Aligner	Configuration	Index 4 Gbp k=15, w=10 DNA4 M7A C1	Index 4 Gbp k=15, w=10 DNA4 M7I C2	Index 4 Gbp k=15, w=10 DNA4 M4 C3	Index 4 Gbp k=15, w=10 DNA4 C7G C4	Index 4 Gbp k=15, w=10 DNA5 M7A C5	Index 4 Gbp k=15, w=10 DNA5 M7I C6	Index 4 Gbp k=15, w=10 DNA5 M4 C7	Index 4 Gbp k=15, w=10 DNA5 C7G C8	No index 4 Gbp k=15, w=10 DNA4 M7A C9	No index 4 Gbp k=15, w=10 DNA4 M7I C10	No index 4 Gbp k=15, w=10 DNA4 M4 C11	No index 4 Gbp k=15, w=10 DNA4 C7G C12	No index 4 Gbp k=15, w=10 DNA5 M7A C13	No index 4 Gbp k=15, w=10 DNA5 M7I C14	No index 4 Gbp k=15, w=10 DNA5 M4 C15	No index 4 Gbp k=15, w=10 DNA5 C7G C16
R1 minimap2	Retained baseline	36.2s ▼3.6× 35s ▼3.9×	38.1s ▼2.9× 36s ▼3.0×	1m 32.4s ▼9.8× 1m 28.5s ▼12.7×	6m 31.5s ▼11.2×[g] 6m 26.3s ▼11.4×	31.6s ▼1.4× 30.3s ▼1.6×	34.1s ▼1.4× 32s ▼1.5×	1m 9.5s ▼28% 1m 6.1s ▼1.4×	3m 1s ▼3.6×[g] 2m 51.5s ▼3.7×	2m 55.5s ▲22% 2m 54s ▲19%	2m 50.4s ▲28% 2m 48.2s ▲25%	5m 35.7s ▲18% 5m 31.9s ▲15%	44m 20.5s n/a[g] 44m 15.6s n/a	1m 47.1s ▲5.5× 1m 45.8s ▲2.1×	1m 47.2s ▲4.7× 1m 45.1s ▲2.3×	3m 40.3s ▲4.2× 3m 36.6s ▲1.7×	29m 43.4s n/a[g] 29m 38.5s n/a
R2 BioseqDB	MPF / Bounded SIMD-DP / Block	9.9s ▲3.6× 9s ▲3.9×	13s ▲2.9× 11.9s ▲3.0×	9.4s ▲9.8× 7s ▲12.7×	35s ▲11.2×[s] 33.8s ▲11.4×	23.1s ▲27%[s] 18.6s ▲1.6×	25.2s ▲26%[s] 21.2s ▲1.5×	54.4s ▲22%[s] 46.7s ▲29%	49.8s ▲3.6×[s] 46.4s ▲3.7×	3m 44.5s ▼28% 3m 35.9s ▼24%	3m 57.4s ▼1.4× 3m 45.7s ▼1.3×	6m 50.6s ▼22% 6m 30.7s ▼18%	·	9m 45.5s ▼5.5× 3m 41.5s ▼2.1×	8m 25.6s ▼4.7× 3m 56.7s ▼2.3×	15m 24.9s ▼4.2× 6m 0.2s ▼1.7×	·
R3 blastn	Native[*]	11m 14.7s ▼18.6×[c] 11m 9.5s ▼19.1×	16m 40.6s ▼26.3×[c] 16m 29.1s ▼27.5×	22m 6.7s ▼14.4×[c] 21m 51.5s ▼14.8×	12m 25.4s ▼1.9×[c] 12m 19.3s ▼1.9×	11m 40.1s ▼22.2×[a,c] 9m 50.3s ▼19.5×	16m 52.8s ▼29.7×[a,c] 13m 48.2s ▼25.9×	23m 30.3s ▼20.3×[a,c] 18m 49.3s ▼17.1×	12m 50.3s ▼4.3×[a,c] 10m 46.6s ▼3.8×	11m 28s ▼3.9×[c] 11m 22.8s ▼3.9×	16m 55s ▼6.0×[c] 16m 43.4s ▼6.0×	22m 36.2s ▼4.0×[c] 22m 20.8s ▼4.0×	13m 1.6s ▲3.4×[c] 12m 55.8s ▲3.4×	12m 18.6s ▼6.9×[a,c] 10m 29.2s ▼5.9×	17m 25.7s ▼9.8×[a,c] 14m 22.8s ▼8.2×	24m 29.4s ▼6.7×[a,c] 19m 49.6s ▼5.5×	13m 43.9s ▲2.2×[a,c] 11m 39.5s ▲2.5×
R4 winnowmap	Native[*]	52.9s ▼1.5×[c,e] 50.6s ▼1.4×	59.4s ▼1.6×[c,e] 55.2s ▼1.5×	1m 49.1s ▼18%[c,e] 1m 41.3s ▼14%	1m 16.1s ▲5.1×[c,d] 1m 8.2s ▲5.7×	31.6s ▶ 0%[c,e] 29.4s ▶ 3%	44.2s ▼30%[c,e] 40.3s ▼26%	59s ▲15%[c,e] 51.8s ▲22%	1m 8s ▲2.7×[c,d] 1m 0.3s ▲2.8×	8m 6s ▼2.8×[c,e] 8m 3.7s ▼2.8×	7m 59.6s ▼2.8×[c,e] 7m 55.3s ▼2.8×	15m 50.3s ▼2.8×[c,e] 15m 42.3s ▼2.8×	7m 57s ▲5.6×[c,d] 7m 48.9s ▲5.7×	6m 51.2s ▼3.8×[c,e] 6m 49.1s ▼3.9×	6m 41.1s ▼3.7×[c,e] 6m 37.2s ▼3.8×	12m 42.6s ▼3.5×[c,e] 12m 35.1s ▼3.5×	7m 10.6s ▲4.1×[c,d] 7m 3.1s ▲4.2×
R5 lexicmap	Native[*]	4m 7.1s ▼6.8×[c] 4m 7s ▼7.0×	6m 29.9s ▼10.2×[c] 6m 29.7s ▼10.8×	9m 24.6s ▼6.1×[c] 9m 24.4s ▼6.4×	7m 16.7s ▼12%[c] 7m 16.4s ▼13%	4m 7.8s ▼7.9×[b,c] 4m 7.5s ▼8.2×	6m 29.2s ▼11.4×[b,c] 6m 28.8s ▼12.1×	11m 26.4s ▼9.9×[b,c] 11m 25.5s ▼10.4×	7m 15.7s ▼2.4×[b,c] 7m 13s ▼2.5×	7m 34.2s ▼2.6×[c] 7m 34.1s ▼2.6×	11m 38.9s ▼4.1×[c] 11m 38.7s ▼4.2×	17m 11.5s ▼3.1×[c] 17m 11.2s ▼3.1×	15m 28s ▲2.9×[c] 15m 27.2s ▲2.9×	7m 48.7s ▼4.4×[b,c] 7m 48.4s ▼4.4×	11m 34.4s ▼6.5×[b,c] 11m 34s ▼6.6×	17m 36.1s ▼4.8×[b,c] 17m 35.3s ▼4.9×	16m 6s ▲1.8×[b,c] 16m 3.7s ▲1.8×

Each cell: A, comparison, B, comparison. Batches of 1,024 queries. Five repeats unless marked s (FAST: one sample). Blue = one lowest A and one lowest B down each machine/workload column.

Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column. Exact ties select the first row.

Ratios use minimap2 as reference, except minimap2 itself uses the fixed BioseqDB row. · = no approved timing. n/a = no measured counterpart.

Both minimap2 and BioseqDB use the matching non-external remote measurements.

[*]External A includes process startup and output. B removes different native stages and is not an equivalent-work comparison. BLAST and LexicMap have no *w* or MAPQ equivalent.

[a]BLAST DNA5 uses an N score of -3, compared with -1 in the baseline.

[b]LexicMap stores reference N as A. Queries retain their bases, and edit-script checks use that native reference encoding.

[c]External input uses two reference records for the 4 Gbp sequence to stay within signed 32-bit coordinates.

[d]Winnowmap uses overlapping 250 Mb records and 499M index batches after an out-of-memory failure. Results are merged globally. This can affect MAPQ.

[e]Winnowmap uses 1999M index batches and merges the results from multiple index parts.

[g]Retained C7G minimap2 measurement comes from the earlier campaign with documented paging pressure.

[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.

Different campaigns and native timing boundaries. A/B ratios are descriptive, subject to the measurement notes.

Aligner	Configuration	Index 4 Gbp k=20, w=15 DNA4 M7A C1	Index 4 Gbp k=20, w=15 DNA4 M7I C2	Index 4 Gbp k=20, w=15 DNA4 M4 C3	Index 4 Gbp k=20, w=15 DNA4 C7G C4	Index 4 Gbp k=20, w=15 DNA5 M7A C5	Index 4 Gbp k=20, w=15 DNA5 M7I C6	Index 4 Gbp k=20, w=15 DNA5 M4 C7	Index 4 Gbp k=20, w=15 DNA5 C7G C8	No index 4 Gbp k=20, w=15 DNA4 M7A C9	No index 4 Gbp k=20, w=15 DNA4 M7I C10	No index 4 Gbp k=20, w=15 DNA4 M4 C11	No index 4 Gbp k=20, w=15 DNA4 C7G C12	No index 4 Gbp k=20, w=15 DNA5 M7A C13	No index 4 Gbp k=20, w=15 DNA5 M7I C14	No index 4 Gbp k=20, w=15 DNA5 M4 C15	No index 4 Gbp k=20, w=15 DNA5 C7G C16
R1 minimap2	Retained baseline	2m 48.1s ▼18.3× 2m 41.7s ▼19.0×	2m 50.3s ▼15.0× 2m 43s ▼15.3×	4m 54.3s ▼43.2× 4m 39s ▼54.6×	25m 5.5s ▼38.5×[g] 25m 0.7s ▼39.5×	33.9s ▼1.9× 32.9s ▼2.1×	34.4s ▼2.6× 32.7s ▼2.9×	1m 8.2s ▼3.7× 1m 5.1s ▼4.6×	3m 9.8s ▼3.3×[g] 3m 3.7s ▼3.4×	3m 28.3s ▼1.3× 3m 22.8s ▼1.4×	3m 53.2s ▼1.4× 3m 43.5s ▼1.4×	7m 14.6s ▼1.4× 6m 57.2s ▼1.5×	51m 22s n/a[g] 51m 17.2s n/a	1m 10.3s ▲7.6× 1m 9s ▲2.4×	1m 8.6s ▲6.4× 1m 6.9s ▲2.5×	2m 21.5s ▲6.2× 2m 18.4s ▲2.2×	15m 56.8s n/a[g] 15m 51.9s n/a
R2 BioseqDB	MPF / Bounded SIMD-DP / Block	9.2s ▲18.3× 8.5s ▲19.0×	11.4s ▲15.0× 10.7s ▲15.3×	6.8s ▲43.2× 5.1s ▲54.6×	39.1s ▲38.5×[s] 38s ▲39.5×	18.2s ▲1.9× 16s ▲2.1×	13.2s ▲2.6× 11.4s ▲2.9×	18.6s ▲3.7× 14.3s ▲4.6×	57.2s ▲3.3×[s] 53.9s ▲3.4×	2m 38.5s ▲24% 2m 29.8s ▲26%	2m 49s ▲28% 2m 37s ▲30%	5m 3.3s ▲1.4× 4m 42.7s ▲1.5×	·	8m 52.3s ▼7.6× 2m 45.5s ▼2.4×	7m 22.2s ▼6.4× 2m 48.2s ▼2.5×	14m 43.3s ▼6.2× 5m 9.5s ▼2.2×	·
R3 blastn	Native*	4m 35.4s ▼1.6×[c] 4m 30.3s ▼1.7×	6m 21.2s ▼2.2×[c] 6m 10.8s ▼2.3×	8m 55.3s ▼1.8×[c] 8m 41.3s ▼1.9×	5m 9.4s ▲4.9×[c] 5m 4.2s ▲4.9×	5m 39.4s ▼10.0×[a,c] 4m 13.6s ▼7.7×	7m 32.4s ▼13.1×[a,c] 5m 27.1s ▼10.0×	11m 29.3s ▼10.1×[a,c] 8m 4s ▼7.4×	6m 10.2s ▼2.0×[a,c] 4m 39s ▼1.5×	4m 58.3s ▼1.4×[c] 4m 53.3s ▼1.4×	6m 40.8s ▼1.7×[c] 6m 30.3s ▼1.7×	9m 34s ▼1.3×[c] 9m 19.8s ▼1.3×	5m 40.5s ▲9.1×[c] 5m 35.1s ▲9.2×	6m 15.6s ▼5.3×[a,c] 4m 49.2s ▼4.2×	8m 11.1s ▼7.2×[a,c] 6m 2.7s ▼5.4×	12m 29.6s ▼5.3×[a,c] 9m 3.4s ▼3.9×	7m 4.7s ▲2.3×[a,c] 5m 34.7s ▲2.8×
R4 winnowmap	Native*	55.6s ▲3.0×[c,e] 53.8s ▲3.0×	59.6s ▲2.9×[c,e] 56.2s ▲2.9×	1m 43s ▲2.9×[c,e] 1m 36.3s ▲2.9×	1m 0.7s ▲24.8×[c,d] 54.1s ▲27.7×	24.6s ▲28%[c,e] 22.8s ▲1.4×	23.7s ▲1.4×[c,e] 20.4s ▲1.6×	36.1s ▲1.9×[c,e] 29.6s ▲2.2×	41.4s ▲4.6×[c,d] 34.7s ▲5.3×	10m 41.8s ▼3.1×[c,e] 10m 40s ▼3.2×	11m 28.6s ▼3.0×[c,e] 11m 25.2s ▼3.1×	33m 6.8s ▼4.6×[c,e] 33m 0.1s ▼4.7×	12m 28.4s ▲4.1×[c,d] 12m 21.9s ▲4.1×	8m 28.7s ▼7.2×[c,e] 8m 26.9s ▼7.3×	8m 59.5s ▼7.9×[c,e] 8m 56.1s ▼8.0×	24m 58.6s ▼10.6×[c,e] 24m 52s ▼10.8×	10m 19s ▲1.5×[c,d] 10m 12.1s ▲1.6×
R5 lexicmap	Native*	56.9s ▲3.0×[c] 56.7s ▲2.9×	1m 21.3s ▲2.1×[c] 1m 21.1s ▲2.0×	2m 1.2s ▲2.4×[c] 2m 0.9s ▲2.3×	1m 16.5s ▲19.7×[c] 1m 16.1s ▲19.7×	56.4s ▼1.7×[b,c] 55.8s ▼1.7×	1m 20.1s ▼2.3×[b,c] 1m 19.6s ▼2.4×	2m 9.7s ▼1.9×[b,c] 2m 8.5s ▼2.0×	1m 14.6s ▲2.5×[b,c] 1m 12.8s ▲2.5×	4m 52.5s ▼1.4×[c] 4m 52.3s ▼1.4×	6m 45.3s ▼1.7×[c] 6m 45.1s ▼1.8×	10m 57.1s ▼1.5×[c] 10m 56.8s ▼1.6×	9m 46.1s ▲5.3×[c] 9m 45.6s ▲5.3×	4m 52.2s ▼4.2×[b,c] 4m 51.8s ▼4.2×	6m 46.8s ▼5.9×[b,c] 6m 46.3s ▼6.1×	10m 14.1s ▼4.3×[b,c] 10m 13.1s ▼4.4×	9m 49.2s ▲1.6×[b,c] 9m 47.7s ▲1.6×

Each cell: A, comparison, B, comparison. Batches of 1,024 queries. Five repeats unless marked s (FAST: one sample). Blue = one lowest A and one lowest B down each machine/workload column.

Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column. Exact ties select the first row.

Ratios use minimap2 as reference, except minimap2 itself uses the fixed BioseqDB row. · = no approved timing. n/a = no measured counterpart.

Both minimap2 and BioseqDB use the matching non-external remote measurements.

*External A includes process startup and output. B removes different native stages and is not an equivalent-work comparison. BLAST and LexicMap have no *w* or MAPQ equivalent.

[a]BLAST DNA5 uses an N score of -3, compared with -1 in the baseline.

[b]LexicMap stores reference N as A. Queries retain their bases, and edit-script checks use that native reference encoding.

[c]External input uses two reference records for the 4 Gbp sequence to stay within signed 32-bit coordinates.

[d]Winnowmap uses overlapping 250 Mb records and 499M index batches after an out-of-memory failure. Results are merged globally. This can affect MAPQ.

[e]Winnowmap uses 1999M index batches and merges the results from multiple index parts.

[g]Retained C7G minimap2 measurement comes from the earlier campaign with documented paging pressure.

[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.

Different campaigns and native timing boundaries. A/B ratios are descriptive, subject to the measurement notes.

Aligner	Configuration	Index 2 Gbp k=15, w=10 DNA4 M7A C1	Index 2 Gbp k=15, w=10 DNA4 M7I C2	Index 2 Gbp k=15, w=10 DNA4 M4 C3	Index 2 Gbp k=15, w=10 DNA4 C7G C4	Index 2 Gbp k=15, w=10 DNA5 M7A C5	Index 2 Gbp k=15, w=10 DNA5 M7I C6	Index 2 Gbp k=15, w=10 DNA5 M4 C7	Index 2 Gbp k=15, w=10 DNA5 C7G C8	No index 2 Gbp k=15, w=10 DNA4 M7A C9	No index 2 Gbp k=15, w=10 DNA4 M7I C10	No index 2 Gbp k=15, w=10 DNA4 M4 C11	No index 2 Gbp k=15, w=10 DNA4 C7G C12	No index 2 Gbp k=15, w=10 DNA5 M7A C13	No index 2 Gbp k=15, w=10 DNA5 M7I C14	No index 2 Gbp k=15, w=10 DNA5 M4 C15	No index 2 Gbp k=15, w=10 DNA5 C7G C16
R1 minimap2	Retained baseline	11.2s ▼3.0× 10s ▼3.4×	10.7s ▼3.0× 8.6s ▼3.2×	22.5s ▼3.3× 18.8s ▼4.2×	1m 13.1s ▼8.3×[g] 1m 5.7s ▼8.3×	9.9s ▼21% 8.8s ▼1.5×	9.5s ▼1.4× 7.6s ▼1.6×	16.5s ▼20% 12.9s ▼1.5×	28.2s ▼1.4×[g] 26.2s ▼1.6×	53s ▲2.2× 51.9s ▲2.1×	56.6s ▲2.1× 54.6s ▲1.9×	1m 55.5s ▲1.9× 1m 51.8s ▲1.8×	6m 1.3s ▼2.9×[g] 5m 55.2s ▼3.0×	48.1s ▲7.3× 47s ▲2.4×	44.5s ▲6.5× 42.6s ▲2.7×	1m 30.2s ▲6.4× 1m 26.7s ▲2.3×	2m 46.4s ▲4.1×[g] 2m 42.7s ▼16%
R2 BioseqDB	MPF / Bounded SIMD-DP / Block	3.7s ▲3.0× 3s ▲3.4×	3.5s ▲3.0× 2.7s ▲3.2×	6.7s ▲3.3× 4.5s ▲4.2×	8.8s ▲8.3×[g] 7.9s ▲8.3×	8.2s ▲17% 5.8s ▲1.5×	6.6s ▲1.4× 4.6s ▲1.6×	13.7s ▲17% 8.8s ▲1.5×	20.2s ▲28%[s] 16.8s ▲1.6×	1m 56.4s ▼2.2× 1m 48.1s ▼2.1×	1m 56.2s ▼2.1× 1m 46.2s ▼1.9×	3m 42.9s ▼1.9× 3m 25s ▼1.8×	2m 5.6s ▲2.9×[g] 1m 59.7s ▲3.0×	5m 52.1s ▼7.3× 1m 53.5s ▼2.4×	4m 49.2s ▼6.5× 1m 56.8s ▼2.7×	9m 39.5s ▼6.4× 3m 18.1s ▼2.3×	11m 24.8s ▼4.1× 2m 20.4s ▲14%
R3 blastn	Native[*]	5m 36.2s ▼30.1×[f] 5m 30.7s ▼33.0×	8m 14.3s ▼46.4×[f] 8m 2.4s ▼56.0×	10m 51.3s ▼28.9× 10m 35.3s ▼33.8×	5m 54.4s ▼4.9×[f] 5m 48.6s ▼5.3×	6m 37.9s ▼40.2×[a] 4m 57.9s ▼33.9×	9m 32.3s ▼60.3×[a] 6m 18s ▼49.6×	13m 27.4s ▼49.0×[a] 8m 52.1s ▼41.2×	6m 53.6s ▼14.7×[a] 4m 51.8s ▼11.1×	5m 39.9s ▼6.4× 5m 34.5s ▼6.4×	8m 22.6s ▼8.9× 8m 10.6s ▼9.0×	11m 6.9s ▼5.8× 10m 50.9s ▼5.8×	6m 6.4s ▶ 1% 6m 0.6s ▶ 2%	6m 51.8s ▼8.6×[a] 5m 12.2s ▼6.6×	9m 48.4s ▼13.2×[a] 6m 35.3s ▼9.3×	13m 55.3s ▼9.3×[a] 9m 23.3s ▼6.5×	7m 14.5s ▼2.6×[a] 5m 12.1s ▼1.9×
R4 winnowmap	Native[*]	12.9s ▼15% 11.1s ▼11%	13.5s ▼27% 10.3s ▼20%	23.2s ▶ 3% 17.1s ▲9%	44.3s ▲1.7×[d] 37.8s ▲1.7×	12.3s ▼24% 10.6s ▼21%	13s ▼1.4× 10s ▼1.3×	23.8s ▼1.4× 17.6s ▼1.4×	37.3s ▼1.3×[d] 31.4s ▼20%	3m 33.2s ▼4.0× 3m 31.5s ▼4.1×	3m 22.6s ▼3.6× 3m 19.3s ▼3.7×	6m 21.3s ▼3.3× 6m 15s ▼3.4×	3m 56.6s ▲1.5×[d] 3m 50.3s ▲1.5×	3m 7.4s ▼3.9× 3m 5.8s ▼4.0×	3m 2.2s ▼4.1× 2m 59.3s ▼4.2×	6m 35.9s ▼4.4× 6m 29.8s ▼4.5×	3m 33.9s ▼29%[d] 3m 28.1s ▼28%
R5 lexicmap	Native[*]	2m 14.1s ▼12.0× 2m 14s ▼13.4×	3m 26.7s ▼19.4× 3m 26.6s ▼24.0×	4m 40.1s ▼12.4× 4m 39.9s ▼14.9×	3m 24.3s ▼2.8× 3m 24.2s ▼3.1×	2m 12.9s ▼13.4×[b] 2m 12.5s ▼15.1×	3m 24.1s ▼21.5×[b] 3m 23.6s ▼26.7×	4m 56.4s ▼18.0×[b] 4m 55.6s ▼22.9×	3m 22.8s ▼7.2×[b] 3m 22.5s ▼7.7×	3m 18.9s ▼3.8× 3m 18.7s ▼3.8×	5m 10.5s ▼5.5× 5m 10.3s ▼5.7×	8m 19.4s ▼4.3× 8m 19.2s ▼4.5×	5m 43.1s ▲5% 5m 43s ▶ 3%	3m 16.5s ▼4.1×[b] 3m 16.1s ▼4.2×	5m 5s ▼6.9×[b] 5m 4.5s ▼7.2×	7m 40.8s ▼5.1×[b] 7m 39.9s ▼5.3×	5m 52.3s ▼2.1×[b] 5m 51.9s ▼2.2×

Each cell: A, comparison, B, comparison. Batches of 1,024 queries. Five repeats unless marked s (FAST: one sample). Blue = one lowest A and one lowest B down each machine/workload column.

Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column. Exact ties select the first row.

Ratios use minimap2 as reference, except minimap2 itself uses the fixed BioseqDB row. · = no approved timing. n/a = no measured counterpart.

Both minimap2 and BioseqDB use the matching non-external remote measurements.

[*]External A includes process startup and output. B removes different native stages and is not an equivalent-work comparison. BLAST and LexicMap have no *w* or MAPQ equivalent.

[a]BLAST DNA5 uses an N score of -3, compared with -1 in the baseline.

[b]LexicMap stores reference N as A. Queries retain their bases, and edit-script checks use that native reference encoding.

[d]Winnowmap uses overlapping 250 Mb records and 499M index batches after an out-of-memory failure. Results are merged globally. This can affect MAPQ.

[f]Swap configuration was not recorded before this external measurement. Later host snapshots do not establish its earlier swap state.

[g]Retained C7G minimap2 measurement comes from the earlier campaign with documented paging pressure.

[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.

Different campaigns and native timing boundaries. A/B ratios are descriptive, subject to the measurement notes.

Aligner	Configuration	Index 2 Gbp k=20, w=15 DNA4 M7A C1	Index 2 Gbp k=20, w=15 DNA4 M7I C2	Index 2 Gbp k=20, w=15 DNA4 M4 C3	Index 2 Gbp k=20, w=15 DNA4 C7G C4	Index 2 Gbp k=20, w=15 DNA5 M7A C5	Index 2 Gbp k=20, w=15 DNA5 M7I C6	Index 2 Gbp k=20, w=15 DNA5 M4 C7	Index 2 Gbp k=20, w=15 DNA5 C7G C8	No index 2 Gbp k=20, w=15 DNA4 M7A C9	No index 2 Gbp k=20, w=15 DNA4 M7I C10	No index 2 Gbp k=20, w=15 DNA4 M4 C11	No index 2 Gbp k=20, w=15 DNA4 C7G C12	No index 2 Gbp k=20, w=15 DNA5 M7A C13	No index 2 Gbp k=20, w=15 DNA5 M7I C14	No index 2 Gbp k=20, w=15 DNA5 M4 C15	No index 2 Gbp k=20, w=15 DNA5 C7G C16
R1 minimap2	Retained baseline	19.6s ▼6.6× 18.6s ▼8.1×	18.2s ▼6.9× 16.5s ▼8.6×	42.5s ▼7.9× 39.4s ▼11.7×	6m 29.2s ▼86.7× 5m 56.9s ▼97.9×	9.3s ▼24% 8.4s ▼1.6×	8.7s ▼1.5× 7s ▼1.7×	15.8s ▼26% 12.7s ▼1.6×	18.5s ▼1.4×[g] 17s ▼1.6×	42.4s ▲2.0× 41.4s ▲1.9×	42.9s ▲2.0× 41.1s ▲1.8×	1m 26.9s ▲1.9× 1m 23.7s ▲1.7×	4m 51.6s ▼2.9×[g] 4m 46.6s ▼3.0×	35s ▲8.6× 34s ▲2.5×	35.9s ▲6.7× 34.2s ▲2.5×	1m 14.1s ▲6.9× 1m 11.1s ▲2.2×	1m 0.1s ▲10.4×[g] 58.6s ▲1.9×
R2 BioseqDB	MPF / Bounded SIMD-DP / Block	3s ▲6.6× 2.3s ▲8.1×	2.6s ▲6.9× 1.9s ▲8.6×	5.4s ▲7.9× 3.4s ▲11.7×	4.5s ▲86.7×[g] 3.6s ▲97.9×	7.5s ▲19% 5.2s ▲1.6×	5.8s ▲1.5× 4s ▲1.7×	12.6s ▲21% 7.9s ▲1.6×	12.9s ▲1.4× 10.6s ▲1.6×	1m 24.8s ▼2.0× 1m 16.9s ▼1.9×	1m 25.4s ▼2.0× 1m 15.7s ▼1.8×	2m 44s ▼1.9× 2m 26.2s ▼1.7×	1m 42s ▲2.9× 1m 35.7s ▲3.0×	5m 1.5s ▼8.6× 1m 24.2s ▼2.5×	4m 0.6s ▼6.7× 1m 24.3s ▼2.5×	8m 31.7s ▼6.9× 2m 34.1s ▼2.2×	10m 27.6s ▼10.4× 1m 51.4s ▼1.9×
R3 blastn	Native*	2m 19.5s ▼7.1× 2m 14.5s ▼7.2×	3m 12.3s ▼10.6× 3m 1.8s ▼11.0×	4m 30.1s ▼6.4× 4m 16s ▼6.5×	2m 27.9s ▲2.6× 2m 22.7s ▲2.5×	3m 27.6s ▼22.3×[a] 2m 22.7s ▼17.0×	4m 37.9s ▼31.8×[a] 2m 47s ▼23.8×	7m 14.2s ▼27.5×[a] 4m 26.5s ▼20.9×	3m 30.4s ▼11.4×[a] 2m 14.4s ▼7.9×	2m 28.5s ▼3.5× 2m 23.4s ▼3.5×	3m 20.9s ▼4.7× 3m 10.4s ▼4.6×	4m 46.7s ▼3.3× 4m 32.6s ▼3.3×	2m 38.7s ▲1.8× 2m 33.5s ▲1.9×	3m 43.6s ▼6.4×[a] 2m 37.7s ▼4.6×	4m 51.4s ▼8.1×[a] 2m 59.6s ▼5.3×	7m 39.4s ▼6.2×[a] 4m 49.6s ▼4.1×	3m 50.8s ▼3.8×[a] 2m 36.2s ▼2.7×
R4 winnowmap	Native*	28.4s ▼1.4× 26.9s ▼1.4×	27.6s ▼1.5× 24.8s ▼1.5×	1m 2.8s ▼1.5× 55s ▼1.4×	8m 35.8s ▼1.3× 8m 4.4s ▼1.4×	10.8s ▼16% 9.4s ▼12%	10.9s ▼24% 8s ▼14%	20.1s ▼27% 14.5s ▼14%	21s ▼14% 17.1s ▶ 0%	4m 16.8s ▼6.1× 4m 15.2s ▼6.2×	5m 0.6s ▼7.0× 4m 57.9s ▼7.2×	10m 23.6s ▼7.2× 10m 18s ▼7.4×	16m 47.8s ▼3.5× 16m 11s ▼3.4×	3m 21.4s ▼5.8× 3m 19.9s ▼5.9×	4m 0.5s ▼6.7× 3m 57.6s ▼7.0×	8m 56.2s ▼7.2× 8m 50.6s ▼7.5×	4m 43.2s ▼4.7× 4m 39.3s ▼4.8×
R5 lexicmap	Native*	38.8s ▼2.0× 38.6s ▼2.1×	54.3s ▼3.0× 54s ▼3.3×	1m 29.8s ▼2.1× 1m 29.4s ▼2.3×	49.9s ▲7.8× 49.7s ▲7.2×	37.8s ▼4.1×[b] 37.3s ▼4.4×	53.5s ▼6.1×[b] 52.8s ▼7.5×	1m 18.6s ▼5.0×[b] 1m 17.5s ▼6.1×	49.7s ▼2.7×[b] 49.3s ▼2.9×	1m 41.8s ▼2.4× 1m 41.7s ▼2.5×	2m 35.6s ▼3.6× 2m 35.4s ▼3.8×	4m 6.7s ▼2.8× 4m 6.3s ▼2.9×	3m 25.5s ▲30% 3m 25.3s ▲28%	1m 43.4s ▼3.0×[b] 1m 42.6s ▼3.0×	2m 37.7s ▼4.4×[b] 2m 37.1s ▼4.6×	3m 45.3s ▼3.0×[b] 3m 43.9s ▼3.2×	3m 16.3s ▼3.3×[b] 3m 15.8s ▼3.3×

Each cell: A, comparison, B, comparison. Batches of 1,024 queries. Five repeats unless marked s (FAST: one sample). Blue = one lowest A and one lowest B down each machine/workload column.

Light gray = minimap2 baseline. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration, not a claim of winning every column. Exact ties select the first row.

Ratios use minimap2 as reference, except minimap2 itself uses the fixed BioseqDB row. · = no approved timing. n/a = no measured counterpart.

Both minimap2 and BioseqDB use the matching non-external remote measurements.

*External A includes process startup and output. B removes different native stages and is not an equivalent-work comparison. BLAST and LexicMap have no *w* or MAPQ equivalent.

[a]BLAST DNA5 uses an N score of -3, compared with -1 in the baseline.

[b]LexicMap stores reference N as A. Queries retain their bases, and edit-script checks use that native reference encoding.

[g]Retained C7G minimap2 measurement comes from the earlier campaign with documented paging pressure.

12.1.5. Vulkan on a Vast.ai GPU instance

This separate campaign compares Vulkan configurations with minimap2 on the same Vast.ai instance. The machine has one NVIDIA GeForce RTX 5090 (32,607 MiB VRAM) and an AMD EPYC 7282 16-Core Processor. The container has a 7.68-core CPU quota and 120.8 GiB memory limit. It runs Ubuntu 24.04 with NVIDIA driver 595.84 and Vulkan API 1.4.329.

Each accepted observation measures one batch of 1,024 queries. P50 is this single sample, with no estimate of repeat-to-repeat variability. PostgreSQL restarts before measurement, but operating-system caches are uncontrolled. Index preparation is untimed, and compatible validated persisted indexes can be reused. No-index A includes transient index construction. B subtracts the same-sample union of fused base-alignment/CIGAR and MAPQ intervals. It is not a pure seeding-and-chaining time.

Measured statements retain correctness checks against the deterministic query truth and ordered match-count parity with same-host minimap2. Prefiltering is disabled. Physical GPU execution and timing evidence are required, while the additional plan and alternative-setting controls are omitted. Vulkan rows still include CPU work such as index handling, backtracking and base alignment. The measured build includes campaign-specific GPU memory-placement and native-Vulkan validation changes. These results therefore describe that build and host.

The source contains 20 approved observations (12 Vulkan and 8 minimap2), 16 setup rejections and 5 execution failures. The table retains only workload columns with an approved Vulkan timing, displaying 16 timings and 4 rejection or failure markers. Empty columns and columns containing only baseline timings, failures or rejections are omitted. The remaining columns keep their original widths. Superseded timings and resolved failures are excluded.

Red circles identify explicit setup rejections. Red crosses identify execution failures. Neither has an approved duration. Dots identify cells with no recorded outcome. Blue marks the lowest available A and B separately, and light gray identifies minimap2. Comparisons retain the earlier triangle directions, 5% tie band and ratio notation above 30%.

Engine	Execution	Index			
		4 Gbp		2 Gbp	
		k=15 w=10 DNA4	k=20 w=15 DNA4	k=15 w=10 DNA4	k=20 w=15 DNA4
		C1	C2	C3	C4
R1 minimap2	Baseline	19.5s ▲21.1×[s] 18s ▲19.3×[s]	42.5s ▲7.6×[s] 41.5s ▲6.3×[s]	14.1s ▲7.7×[s] 13s ▲4.5×[s]	1m 34.6s ▶ 4%[s] 1m 33.6s ▼2.1×[s]
R2 BioseqDB	MPS / Vulkan Vulkan-DP / KSW2	×[2]	×[2]	1m 54.9s ▼8.2×[s] 1m 2.3s ▼4.8×[s]	1m 38.5s ▶ 4%[s] 44.1s ▲2.1×[s]
R3 BioseqDB	MPS / Vulkan Vulkan-DP / Block	×[2]	×[1]	1m 48.7s ▼7.7×[s] 58.9s ▼4.5×[s]	1m 38.5s ▶ 4%[s] 47s ▲2.0×[s]
R4 BioseqDB	MPF / Vulkan Vulkan-DP / KSW2	6m 50.8s ▼21.1×[s] 5m 48.5s ▼19.3×[s]	5m 24.2s ▼7.6×[s] 4m 22.1s ▼6.3×[s]	3m 38.5s ▼15.5×[s] 2m 49.8s ▼13.0×[s]	2m 58.3s ▼1.9×[s] 2m 9.4s ▼1.4×[s]
R5 BioseqDB	MPF / Vulkan Vulkan-DP / Block	7m 7.5s ▼21.9×[s] 6m 8.1s ▼20.4×[s]	5m 48s ▼8.2×[s] 4m 48s ▼6.9×[s]	4m 37s ▼19.7×[s] 3m 47s ▼17.4×[s]	3m 8.4s ▼2.0×[s] 2m 21.2s ▼1.5×[s]

Each cell: A, comparison, B, comparison. Light gray = minimap2 baseline. Blue = one lowest A and one lowest B per workload column across all rows. Exact ties select the first row. Repeated baselines are marked only on page 1.

[s]FAST: one measured batch of 1,024 queries. The reported P50 is that single datapoint, not a five-repeat estimate. Samples are not pooled with standard runs.

Ratios compare with same-host minimap2. Minimap2 compares with the fastest measured Vulkan row separately for A and B. n/a = no measured counterpart.

Only columns with an approved Vulkan timing are shown. Columns with only baseline timings, failures, rejections or no outcomes are omitted.

MPF/MPS = minimizers per fragment/sequence. Labels give index representation / minimizer, then chaining / base alignment. Rows follow increasing minimum Index A across the displayed cases.

· = no recorded outcome. ◯ = setup rejection. × = execution failure. Neither marker is a timing.

[1]Execution failed because the persisted GPU index could not be admitted for query execution.

[2]Execution failed before GPU query dispatch: the persisted MPS control buffer could not be admitted within the Vulkan storage-buffer limit.

Chapter 13

Conclusions

BioSeqDB demonstrates that sequence search can be represented as a PostgreSQL execution operation rather than as a scalar call to an external mapper. The system stores packed biological sequences, constructs persistent minimizer indexes, and runs a minimap2-derived pipeline for seeding, chaining, filtering, and final extension. Planner support and custom scans preserve the batch structure required by SIMD and GPU implementations while presenting typed SQL results that can participate in filters, projections, and joins.

The first research question concerned a shared representation for the algorithmic stages. Alphabet-generic compact ranges cover resident values, while lazy sequence views represent fragmented, sliced, and composed sources. Algorithms read the required region without first converting the complete value to a byte array. The second question concerned heterogeneous execution. Minimizer extraction and chaining provide regular work for the implemented SIMD and GPU routes. Dispatch follows explicit algorithm settings, input compatibility, index form, and resource availability. Final alignment uses banded or unbanded KSW2, adaptive Block Aligner, and GPU routes that may return specialized scripts before device dynamic programming. Optional prefilters may reject unsuitable candidates but do not replace alignment.

The third question concerned database execution. A custom scan owns the search expression, batches input rows, and substitutes match fields into planned expressions. This design also permits multiple alignments for one input row. The fourth question concerned the surrounding data system. Physical sequence storage, persistent indexes, bounded ingestion, external-provider relation definitions, cache coordination, and source-driven SQL generation give the kernels explicit storage and lifecycle contracts.

The remaining limits are operational and algorithmic. Some remote retrieval paths still require record-sized or archive-sized memory. Provider quotas are not coordinated across all PostgreSQL backends. Hardware-specific routes have batch-size thresholds and require broader evaluation on additional devices. GPU chaining may examine a bounded predecessor set, and seed and alignment backends retain route-specific contracts. The present mapper is centered on linear references rather than pangenome graphs. A relational database can nevertheless plan and execute batched sequence analysis while retaining typed data, bounded resources, persistent derived state, and explicit algorithm selection.

References

Aasna, Evelin et al. “DREAM-Stellar: parallel and space efficient exact local alignment.” In: BMC Bioinformatics 27.1, 67 (Feb. 18, 2026). doi: 10.1186/s12859-026-06389-0. url: https://link.springer.com/article/10.1186/s12859-026-06389-0.

Abadi, Daniel, Samuel Madden, and Miguel Ferreira. “Integrating compression and execution in column-oriented database systems.” In: Proceedings of the 2006 ACM SIGMOD International Conference on Management of Data. SIGMOD ’06. Association for Computing Machinery, 2006, pp. 671–682. doi: 10.1145/1142473.1142548. url: https://doi.org/10.1145/1142473.1142548.

cp-algorithms contributors. Range Minimum Query. url: https://cp-algorithms.com/sequences/rmq.html (visited on 09/14/2026).

— Segment Tree. url: https://cp-algorithms.com/data_structures/segment_tree.html (visited on 09/14/2026).

Amazon Web Services. Amazon EBS-optimized instance types. url: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ebs-optimized.html (visited on 09/25/2026).

Chernishev, George A. et al. “A Comprehensive Study of Late Materialization Strategies for a Disk-Based Column-Store.” In: Proceedings of the 24th International Workshop on Design, Optimization, Languages and Analytical Processing of Big Data. Vol. 3130. CEUR Workshop Proceedings. 2022, pp. 21–30. url: https://ceur-ws.org/Vol-3130/paper3.pdf.

Citus Data contributors. Citus Columnar storage implementation. Version 14.0.0. AGPL-3.0. Feb. 17, 2026. url: https://github.com/citusdata/citus (visited on 07/28/2026).

ClickHouse contributors. ClickBench: analytical database benchmark. Historical PostgreSQL, Citus, and Hydra submissions. Sept. 20, 2023. url: https://github.com/ClickHouse/ClickBench/tree/2e8146c397f81e3e98937205d09e331313b991e3 (visited on 09/09/2026).

Conway, Neil. Inside the PostgreSQL Query Optimizer. Slide deck. Undated online source. Fujitsu Australia Software Technology. url: https://www.neilconway.org/talks/optimizer/optimizer.pdf (visited on 09/17/2026).

cppreference.com contributors. C++ named requirements: StandardLayoutType. Undated online source. url: https://en.cppreference.com/cpp/named_req/StandardLayoutType (visited on 09/15/2026).

Crick, Francis. “Central Dogma of Molecular Biology.” In: Nature 227.5258 (Aug. 8, 1970), pp. 561–563. doi: 10.1038/227561a0. url: https://doi.org/10.1038/227561a0.

Cubukcu, Umur et al. “Citus: Distributed PostgreSQL for Data-Intensive Applications.” In: Proceedings of the 2021 International Conference on Management of Data. SIGMOD ’21. Association for Computing Machinery, 2021, pp. 2490–2502. doi: 10.1145/3448016.3457551. url: https://doi.org/10.1145/3448016.3457551.

Davis, Jeff. Citus 10 brings columnar compression to Postgres. Mar. 6, 2021. url: https://www.citusdata.com/blog/2021/03/06/citus-10-columnar-compression-for-postgres/ (visited on 09/08/2026).

Feintzeig, Jake. SIMD + Parallelism: Ray Tracer Optimization Part 3. Undated online source. url: https://www.jakef.science/posts/simd-parallelism/ (visited on 08/19/2026).

Gotoh, Osamu. “An improved algorithm for matching biological sequences.” In: Journal of Molecular Biology 162.3 (1982), pp. 705–708. doi: 10.1016/0022-2836(82)90398-9. url: https://doi.org/10.1016/0022-2836(82)90398-9.

Groot Koerkamp, Ragnar and Igor Martayan. packed-seq: Bitpacked sequence trait and implementation. Version 5.0.0. MIT license. Rust crate developed for SimdMinimizers (SEA 2025). url: https://github.com/rust-seq/packed-seq (visited on 09/15/2026).

— packed_seq crate documentation. Version 5.0.0. July 7, 2026. url: https://docs.rs/packed-seq/latest/packed_seq/ (visited on 09/15/2026).

— “SimdMinimizers: Computing Random Minimizers, fast.” In: 23rd International Symposium on Experimental Algorithms. Vol. 338. Leibniz International Proceedings in Informatics. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025, 20:1–20:19. doi: 10.4230/LIPIcs.SEA.2025.20. url: https://doi.org/10.4230/LIPIcs.SEA.2025.20.

Han, Yeejoo et al. “G^{3}SA: A GPU-Accelerated Gold Standard Genomics Library for End-to-End Sequence Alignment.” In: Proceedings of the 39th ACM International Conference on Supercomputing. ICS ’25. Salt Lake City, Utah, USA: Association for Computing Machinery, 2025, pp. 173–188. doi: 10.1145/3721145.3729516. url: https://doi.org/10.1145/3721145.3729516.

Hydra contributors. Hydra Columnar: PostgreSQL columnar storage extension. Source snapshot d1f7657ee822. Feb. 10, 2025. url: https://github.com/hydradatabase/columnar (visited on 09/09/2026).

Institute, National Human Genome Research. The Cost of Sequencing a Human Genome. Version 17. Undated online source. url: https://www.genome.gov/about-genomics/fact-sheets/Sequencing-Human-Genome-cost (visited on 07/28/2026).

Juliusz K Mateusz C, Szymon D and Damian W. “Master thesis: An integrated and practical database solution for storage and efficient processing of large genomic datasets.” In: MIM UW 1.1 (2022). url: https://mimuw.edu.pl/en/.

Kallenborn, Felix et al. “GPU-accelerated homology search with MMseqs2.” In: Nature Methods 22.10 (Oct. 2025), pp. 2024–2027. issn: 1548-7105. doi: 10.1038/s41592-025-02819-8. url: https://doi.org/10.1038/s41592-025-02819-8.

— “GPU-accelerated homology search with MMseqs2.” In: Nature Methods 22.10 (Sept. 18, 2025), pp. 2024–2027. doi: 10.1038/s41592-025-02819-8. url: https://doi.org/10.1038/s41592-025-02819-8.

Kim, Sunwoo et al. G3SA: released long-read alignment implementation. The minimap2 directory contains the long-read pipeline and its kernel orchestration. url: https://github.com/sunwookim028/G3SA (visited on 09/25/2026).

Knuth, Donald E. The Art of Computer Programming, Volume 3: Sorting and Searching. 2nd ed. Reading, Massachusetts: Addison-Wesley, 1998. isbn: 0-201-89685-0. url: https://cs.stanford.edu/~knuth/taocp.html.

Editors' Report: Programming Languages—C++. WG21 paper N5015. ISO/IEC JTC 1/SC 22/WG 21, Aug. 14, 2025. url: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5015.html (visited on 09/09/2026).

Kreis, Lara et al. GPU-RMQ: Accelerating Range Minimum Queries on Modern GPUs. Preprint. Apr. 2, 2026. doi: 10.48550/arXiv.2604.01811. arXiv: 2604.01811.

Kryukov, Kirill et al. "Nucleotide Archival Format (NAF) enables efficient lossless reference-free compression of DNA sequences." In: Bioinformatics 35.19 (Oct. 2019), pp. 3826–3828. issn: 1367-4803. doi: 10.1093/bioinformatics/btz144. eprint: https://academic.oup.com/bioinformatics/article-pdf/35/19/3826/48977341/bioinformatics_35_19_3826.pdf. url: https://doi.org/10.1093/bioinformatics/btz144.

Lane, Tom. Re: what exactly is a PlaceHolderVar? Reply to Robert Haas. Message-ID: 23076.1277179362@sss.pgh.pa.us. PostgreSQL Hackers mailing list. June 22, 2010. url: https://www.postgresql.org/message-id/23076.1277179362@sss.pgh.pa.us (visited on 09/17/2026).

Lanius, Christian. parse_mmi.py: Python parser and lookup functions for minimap2 indexes. Sept. 27, 2021. url: https://gist.github.com/christian-lanius/8b8d6a38e35b93783ff7d6236211ff5e (visited on 09/08/2026).

Li, Heng. minimap2. Version 2.30. url: https://github.com/lh3/minimap2/tree/79c9cc186b95f50bd899f69b48eba995ced810c6 (visited on 07/28/2026).

— Minimap2 manual. url: https://lh3.github.io/minimap2/minimap2.html (visited on 09/08/2026).

— "Minimap2: pairwise alignment for nucleotide sequences." In: Bioinformatics 34.18 (2018), pp. 3094–3100. doi: 10.1093/bioinformatics/bty191. arXiv: 1708.01492 [q-bio.GN]. url: https://doi.org/10.1093/bioinformatics/bty191.

— "Minimap2: pairwise alignment for nucleotide sequences." In: Bioinformatics 34.18 (Sept. 2018), pp. 3094–3100. issn: 1367-4803. doi: 10.1093/bioinformatics/bty191. eprint: https://academic.oup.com/bioinformatics/article-pdf/34/18/3094/48919122/bioinformatics_34_18_3094.pdf. url: https://doi.org/10.1093/bioinformatics/bty191.

Li, Heng, Xiaowen Feng, and Chong Chu. "The design and construction of reference pangenome graphs with minigraph." In: Genome Biology 21 (Oct. 2020), p. 265. doi: 10.1186/s13059-020-02168-z.

Liu, Daniel and Martin Steinegger. "Block Aligner: an adaptive SIMD-accelerated aligner for sequences and position-specific scoring matrices." In: Bioinformatics 39.8, btad487 (2023). doi: 10.1093/bioinformatics/btad487. url: https://doi.org/10.1093/bioinformatics/btad487.

Martayan, Igor et al. "Helicase: Vectorized parsing and bitpacking of genomic sequences." In: bioRxiv (Mar. 2026). Preprint. doi: 10.64898/2026.03.19.712912. url: https://github.com/imartayan/helicase (visited on 09/09/2026).

minimap2 contributors. How to interpret the mmi index file? Issue 820. Discussion updated in 2025. Sept. 26, 2021. url: https://github.com/lh3/minimap2/issues/820 (visited on 09/08/2026).

Mirdita, M. et al. "Fast and sensitive taxonomic assignment to metagenomic contigs." In: Bioinformatics 37.18 (2021), pp. 3029–3031. doi: 10.1093/bioinformatics/btab184. url: https://doi.org/10.1093/bioinformatics/btab184.

Mirdita, Milot, Martin Steinegger, and Johannes Söding. “MMseqs2 desktop and local web server app for fast, interactive sequence searches.” In: Bioinformatics 35.16 (2019), pp. 2856–2858. doi: 10.1093/bioinformatics/bty1057. url: https://doi.org/10.1093/bioinformatics/bty1057.

National Center for Biotechnology Information. FASTA Format for Nucleotide Sequences. June 18, 2025. url: https://www.ncbi.nlm.nih.gov/genbank/fastaformat/ (visited on 09/09/2026).

— NCBI Datasets v2 REST API documentation. Undated online source. url: https://www.ncbi.nlm.nih.gov/datasets/docs/v2/api/ (visited on 07/28/2026).

Needleman, Saul B. and Christian D. Wunsch. “A general method applicable to the search for similarities in the amino acid sequence of two proteins.” In: Journal of Molecular Biology 48.3 (1970), pp. 443–453. doi: 10.1016/0022-2836(70)90057-4. url: https://doi.org/10.1016/0022-2836(70)90057-4.

Neon. The pg_search extension. Undated online source. Documents ParadeDB pg_search on Neon. url: https://neon.com/docs/extensions/pg_search (visited on 09/17/2026).

Pigsty. pg_search: full text search for PostgreSQL using BM25. Version 0.25.2. Undated online source. Last modified 2026-08-21. Aug. 21, 2026. url: https://pigsty.io/ext/e/pg_search/ (visited on 09/17/2026).

PostgreSQL Global Development Group. PostgreSQL 12 Release Notes. Version 12. url: https://www.postgresql.org/docs/12/release-12.html (visited on 09/16/2026).

— PostgreSQL 17 Documentation: Appendix K. PostgreSQL Limits. Version 17. url: https://www.postgresql.org/docs/17/limits.html (visited on 09/21/2026).

— PostgreSQL 17 Documentation: Architectural Fundamentals. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/tutorial-arch.html (visited on 09/16/2026).

— PostgreSQL 17 Documentation: Background Worker Processes. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/bgworker.html (visited on 09/16/2026).

— PostgreSQL 17 Documentation: Behavior in Threaded Programs. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/libpq-threading.html (visited on 09/16/2026).

— PostgreSQL 17 Documentation: Database Page Layout. Version 17. url: https://www.postgresql.org/docs/17/storage-page-layout.html (visited on 09/08/2026).

— PostgreSQL 17 Documentation: Foreign Data. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/ddl-foreign-data.html (visited on 09/16/2026).

— PostgreSQL 17 Documentation: Function Optimization Information. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/xfunc-optimization.html (visited on 07/28/2026).

— PostgreSQL 17 Documentation: Generic WAL Records. Version 17. url: https://www.postgresql.org/docs/17/generic-wal.html (visited on 09/08/2026).

— PostgreSQL 17 Documentation: How Parallel Query Works. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/how-parallel-query-works.html (visited on 09/16/2026).

— PostgreSQL 17 Documentation: Introduction to Concurrency Control. Version 17. url: https://www.postgresql.org/docs/17/mvcc-intro.html (visited on 09/08/2026).

— PostgreSQL 17 Documentation: pg_settings. Version 17. url: https://www.postgresql.org/docs/17/view-pg-settings.html (visited on 09/09/2026).
— PostgreSQL 17 Documentation: *postgres_fdw*. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/postgres-fdw.html (visited on 09/16/2026).
— PostgreSQL 17 Documentation: Resource Consumption. Version 17. url: https://www.postgresql.org/docs/17/runtime-config-resource.html#GUC-MAX-PARALLEL-WORKERS-PER-GATHER (visited on 09/09/2026).
— PostgreSQL 17 Documentation: Set Returning Functions. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/functions-srf.html (visited on 07/28/2026).
— PostgreSQL 17 Documentation: Table Access Method Interface Definition. Version 17. url: https://www.postgresql.org/docs/17/tableam.html (visited on 09/08/2026).
— PostgreSQL 17 Documentation: TOAST. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/storage-toast.html (visited on 07/28/2026).
— PostgreSQL 17 Documentation: Using C++ for Extensibility. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/xfunc-c.html (visited on 07/28/2026).
— PostgreSQL 17 Documentation: Writing a Custom Scan Provider. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/custom-scan.html (visited on 07/28/2026).
— PostgreSQL 17 source: tuple table slots. Version 17. Header `src/include/executor/tuptable.h` on tag `REL_17_10`. url: https://github.com/postgres/postgres/blob/REL_17_10/src/include/executor/tuptable.h (visited on 09/18/2026).
— PostgreSQL 17: Configuration Inspection Functions. url: https://github.com/postgres/postgres/blob/REL_17_STABLE/src/backend/utils/misc/guc_funcs.c (visited on 09/09/2026).
— PostgreSQL 9.1 Release Notes. Version 9.1. url: https://www.postgresql.org/docs/9.1/release-9-1.html (visited on 09/16/2026).
— PostgreSQL 9.3 Release Notes. Version 9.3. url: https://www.postgresql.org/docs/9.3/release-9-3.html (visited on 09/16/2026).
— PostgreSQL 9.5 Release Notes. Version 9.5. url: https://www.postgresql.org/docs/9.5/release-9-5.html (visited on 09/16/2026).
— PostgreSQL Documentation: Planner/Optimizer. Undated online source. Section on generating possible plans, including the Genetic Query Optimizer. url: https://www.postgresql.org/docs/current/planner-optimizer.html#PLANNER-OPTIMIZER-GENERATING-POSSIBLE-PLANS (visited on 09/17/2026).
Python Software Foundation. Python 3 Documentation: Common Sequence Operations. Version 3.14.7. Slice clamping rules are given in note 4 of the operation table. Sept. 15, 2026. url: https://docs.python.org/3/builtins/stdtypes.html#common-sequence-operations (visited on 09/15/2026).
Reinert, Knut et al. "The SeqAn C++ template library for efficient sequence analysis: A resource for programmers." In: Journal of Biotechnology 261 (Nov. 10, 2017), pp. 157–168. doi: 10.1016/j.jbiotec.2017.07.017. url: https://www.sciencedirect.com/science/article/pii/S0168165617315420.
Roberts, Michael et al. "Reducing storage requirements for biological sequence comparison." In: Bioinformatics 20.18 (2004), pp. 3363–3369. doi: 10.1093/bioinformatics/bth408. url: https://doi.org/10.1093/bioinformatics/bth408.

Sadasivan H Maric M, Dawson E et all. mm2-gpu: Accelerating Minimap2 for Accurate Long Read Alignment on GPUs. Undated online source. doi: https://doi.org/10.26502/jbb.2642-91280067. url: https://github.com/Minimap2onGPU/mm2-gb (visited on 08/19/2026).

SAM/BAM Format Specification Working Group. Sequence Alignment/Map Format Specification. Version 1.6. Undated online source. url: https://samtools.github.io/hts-specs/SAMv1.pdf (visited on 07/28/2026).

Saurabh Kalikar Chirag Jain, Md Vasimuddin et all. mm2-fast: Accelerating minimap2 for long-read sequencing applications on modern CPUs. Undated online source. doi: https://doi.org/10.1038/s43588-022-00201-8. url: https://github.com/bwa-mem2/mm2-fast (visited on 08/19/2026).

Sayers, Eric. "A General Introduction to the E-utilities." In: Entrez Programming Utilities Help. Updated 2022-11-17. Bethesda, Maryland: National Center for Biotechnology Information, May 26, 2009. url: https://www.ncbi.nlm.nih.gov/books/NBK25497/ (visited on 09/09/2026).

— "The E-utilities In-Depth: Parameters, Syntax and More." In: Entrez Programming Utilities Help. Updated 2026-03-04. Bethesda, Maryland: National Center for Biotechnology Information, May 29, 2009. url: https://www.ncbi.nlm.nih.gov/books/NBK25499/ (visited on 09/09/2026).

SeqAn Project. Alphabets in SeqAn. Version 3.4.0. url: https://docs.seqan.de/seqan3/3.4.0/tutorial_alphabets.html (visited on 09/09/2026).

— SeqAn Manual. url: https://seqan.readthedocs.io/en/main/ (visited on 09/09/2026).

— SeqAn Tutorial: Sequences In-Depth. Version 1.4.2. String specializations, including packed and external strings. url: https://seqan.readthedocs.io/en/seqan-v1.4.2/Tutorial/SequencesInDepth.html (visited on 09/15/2026).

Shen, Wei. LexicMap FAQs. Undated online source. url: https://bioinf.shenwei.me/LexicMap/faqs/ (visited on 08/19/2026).

Shen, Wei, Botond Sipos, and Liuyang Zhao. SeqKit usage documentation: the subseq subcommand. Undated online source. Documentation for the tool described in SeqKit2, iMeta 3(3):e191, 2024. url: https://bioinf.shenwei.me/seqkit/usage/#subseq (visited on 09/15/2026).

Shur, Arseny. On Minimizers of Minimum Density. June 5, 2025. doi: 10.48550/arXiv.2506.05277. arXiv: 2506.05277 [cs.DS]. url: https://arxiv.org/abs/2506.05277 (visited on 08/31/2026).

Siragusa, Enrico, David Weese, and Knut Reinert. Fast and sensitive read mapping with approximate seeds and multiple backtracking. Version 1. Preprint. Aug. 21, 2012. doi: 10.48550/arXiv.1208.4238. arXiv: 1208.4238 [cs.DS]. url: https://arxiv.org/abs/1208.4238v1 (visited on 09/09/2026).

Smith, Temple F. and Michael S. Waterman. "Identification of common molecular subsequences." In: Journal of Molecular Biology 147.1 (1981), pp. 195–197. doi: 10.1016/0022-2836(81)90087-5. url: https://doi.org/10.1016/0022-2836(81)90087-5.

Steinegger, Martin and Johannes Söding. "Clustering huge protein sequence sets in linear time." In: Nature Communications 9, 2542 (June 29, 2018). doi: 10.1038/s41467-018-04964-5. url: https://doi.org/10.1038/s41467-018-04964-5.

— "MMseqs2 enables sensitive protein sequence searching for the analysis of massive data sets." In: Nature Biotechnology 35.11 (Oct. 16, 2017), pp. 1026–1028. doi: 10.1038/nbt.3988. url: https://doi.org/10.1038/nbt.3988.
— "MMseqs2: sensitive protein sequence searching for the analysis of massive data sets." Version 5. In: bioRxiv, 079681 (June 7, 2017). Preprint. doi: 10.1101/079681. url: https://www.biorxiv.org/content/early/2017/06/07/079681.
Suzuki, Hajime and Masahiro Kasahara. "Introducing difference recurrence relations for faster semi-global alignment of long sequences." In: BMC Bioinformatics 19, 45 (2018). doi: 10.1186/s12859-018-2014-8. url: https://doi.org/10.1186/s12859-018-2014-8.
Sweden, NBIS - National Bioinformatics Infrastructure. The Cost of Sequencing a Human Genome. Version 1. Undated online source. url: https://github.com/NBISweden/GAAS/blob/master/annotation/knowledge/cigar.md (visited on 07/28/2026).
Tarjan, Robert Endre. "Depth-First Search and Linear Graph Algorithms." In: SIAM Journal on Computing 1.2 (1972), pp. 146–160. doi: 10.1137/0201010. url: https://doi.org/10.1137/0201010.
The Clang Team. C++ Support in Clang. url: https://clang.llvm.org/cxx_status.html (visited on 09/09/2026).
— Choosing the Right Interface for Your Application. url: https://clang.llvm.org/docs/Tooling.html (visited on 09/09/2026).
— LibTooling. url: https://clang.llvm.org/docs/LibTooling.html (visited on 09/09/2026).
Uhl, Tim Niklas, Matthias Schimek, and Daniel Brommer. Concepts in Practice: C++ MPI Bindings for the HPC Ecosystem. From a Standardizable Core to a Composable Interface. 2026. arXiv: 2606.09102 [cs.DC]. url: https://arxiv.org/abs/2606.09102.
UniProt Consortium. FASTA headers. June 10, 2026. url: https://www.uniprot.org/help/fasta-headers (visited on 09/09/2026).
UXL Foundation Contributors. How Task Scheduler Works. url: https://uxlfoundation.github.io/oneTBB/main/tbb_userguide/How_Task_Scheduler_Works.html (visited on 09/09/2026).
— oneTBB Benefits. url: https://uxlfoundation.github.io/oneTBB/main/intro/Benefits.html (visited on 09/09/2026).
— oneTBB Specification: task_arena. url: https://uxlfoundation.github.io/oneTBB/main/specification/source/task_scheduler/task_arena/task_arena_cls.html (visited on 09/09/2026).
— When Task-Based Programming Is Inappropriate. url: https://uxlfoundation.github.io/oneTBB/main/tbb_userguide/When_Task-Based_Programming_Is_Inappropriate.html (visited on 09/09/2026).
Vardanian, Ash. StringZilla: Fast SIMD, SWAR, and GPGPU String Processing. url: https://github.com/ashvardanian/StringZilla (visited on 09/09/2026).
Youngblut, Nick. MMseqs2 Github issue 338. Undated online source. url: https://github.com/soedinglab/MMseqs2/issues/338 (visited on 08/19/2026).
Zhan, Xiaowei. MMseqs2 Github issue 517. Undated online source. url: https://github.com/soedinglab/MMseqs2/issues/517 (visited on 08/19/2026).

Appendix 1

A runnable SQL walkthrough

This appendix follows one small dataset from insertion to alignment and index maintenance. Read the left column from top to bottom. SQL statements are executable, while the differently coloured plan blocks are output from `EXPLAIN ANALYZE`. The right column explains each block.

The accompanying `appendices/02_sql_walkthrough/run.sql` executes the same SQL files used to typeset this listing. Run it with `psql -X -d bioseq_walkthrough -f run.sql` in a new, empty local database created with `createdb bioseq_walkthrough`. The BioSeqDB library must already be installed on the server. Use a database administrator account for this exercise, since it installs the extension and writes a small FASTA fixture to `/tmp/bioseq-walkthrough.fasta` on the server. Each statement commits separately under the default `psql` behaviour. Do not wrap the whole script in one transaction.

No alignment GUC is changed in this walkthrough. It uses the defaults listed in Appendix 2, with explicit per-call options where they are being demonstrated. Names and dates are fictional. The DNA is generated deterministically to keep long strings out of the listing.

⚠ Alpha-version limitations

Please note that the current software is an alpha version and still has numerous issues. Its memory checks can be too restrictive. They keep at least 2 GiB available before admitting more work and also limit the process's memory use. Leave several GiB available when trying these examples, especially when building grouped MPF indexes.
One error-reporting bug can turn a failed query-minimizer allocation into `transient query execution stats disagree with its generated backend`. Free memory and retry before assuming that an index is missing. Building a stored target index does not remove the need to generate query minimizers. The message does not establish that the query rejected that index. Warning 10.7.2 explains this distinction.
Keep `bioseq.index.insert_maintenance.enabled` off, as it is by default. Ordinary per-row minimizer maintenance is unsupported with the default grouped format. Build it explicitly after inserting rows, as shown below. Turning maintenance off does not disable memory checks or repair an allocation failure.

A short exact substring need not produce an alignment. Minimizers must survive seeding and chaining before a hit is reported. For the fixture below, 30- and 40-base prefixes produce no hits under the default settings, whereas a 50-base prefix does. This is an example, not

a universal length threshold. The walkthrough uses 256- and 512-base probes. The seeding and chaining requirements are described in Sections 4.13 and 5.13.

The displayed plans are complete except for the explicitly marked vector aggregate excerpt. Times, cache counters, worker counts and CPU instruction names will vary between runs. They are illustrations of the fields, not performance benchmarks.

Listing 1.1: A continuous SQL walkthrough of BioseqDB.

SQL or execution plan	Explanation

`\set ON_ERROR_STOP on`
`\pset pager off`
`CREATE EXTENSION bioseq;`
 	
`CREATE TABLE samples (`
`  id integer PRIMARY KEY,`
`  upload_date date NOT NULL,`
`  patient_name text NOT NULL,`
`  age integer NOT NULL,`
`  seq bioseq.seq('dna4', 64) NOT NULL`
`) USING bioseq;`
	1. Create a sequence table. The four ordinary columns hold patient metadata. The sequence column uses the DNA4 alphabet, which accepts A, C, G and T. The number 64 is the packed storage-unit width, not a sequence-length limit.
	
	`USING bioseq` selects BioSeqDB storage and search planning. Figure 9.2 shows how its ordinary columns are stored. Section 7.2 explains sequence packing. The first two lines tell `psql` to stop on errors and avoid paging.

`WITH dna AS (`
`  SELECT string_agg(translate(md5(i::text),`
`  '0123456789abcdef',`
`  'ACGTACGTACGTACGT'), '' ORDER BY i) AS s`
`  FROM generate_series(1, 64) AS g(i)`
`)`
`INSERT INTO samples`
`SELECT 1, DATE '2026-09-01', 'Anna', 34,`
`  (s || s)::bioseq.seq('dna4', 64) FROM dna`
`UNION ALL`
`SELECT 2, DATE '2026-09-02', 'Ben', 52,`
`  substring(s FROM 257 FOR 512)`
`  ::bioseq.seq('dna4', 64) FROM dna`
`UNION ALL`
`SELECT 3, DATE '2026-09-03', 'Cara', 29,`
`  s::bioseq.seq('dna4', 64) FROM dna;`
	2. Insert three related sequences. The common expression makes a reproducible 2,048-base string by translating hexadecimal digits to DNA letters. This is synthetic data, not an MD5-based biological model.
	
	Anna receives two copies of that string, Ben a 512-base piece, and Cara one copy. Their lengths are 4,096, 512 and 2,048 bases. Ben's sequence occurs twice in Anna's sequence, which will make multiple alignment rows easy to see. PostgreSQL's text `substring` starts at 1, so position 257 here corresponds to BioSeqDB position 256.

`SELECT id, upload_date, patient_name, age` `FROM samples;`	3. Read ordinary columns. This returns all three patients. No sequence symbols are needed, because the projection contains only metadata. Without `ORDER BY`, SQL does not promise a row order.

`SELECT id, upper(patient_name) AS patient,`
`  age, bioseq.len(seq) AS bases,`
`  seq[:16]::text AS prefix`
`FROM samples`
`WHERE age BETWEEN 30 AND 60`
`  AND upload_date >= DATE '2026-09-01'`
`  AND (patient_name LIKE 'A%'`
`  OR patient_name LIKE 'B%')`
`ORDER BY age DESC, id;`
	4. Combine metadata conditions and a sequence preview. This returns Ben and Anna, in that order. The age interval, date and name conditions must all hold.
	
	`len` reports sequence symbols. The slice `[:16]` returns the first sixteen symbols and the text cast makes them readable. BioSeqDB slices use zero-based, half-open bounds, as shown in Figure 7.2.

SQL or execution plan	Explanation

`EXPLAIN (ANALYZE, COSTS OFF)`
`SELECT id, upload_date, patient_name, age`
`FROM samples;`
	5. Explain the unfiltered read. `ANALYZE` executes statement 3 and measures it. `COSTS OFF` hides planner estimates to keep this first plan small. It does not disable execution measurements.

Custom Scan (**BioseqScan**) on samples (actual time =0.010..0.015 rows=3 loops=1) Bioseq Projected Columns: id, upload_date, patient_name, age −> Cache Cache Hits: 0 Cache Misses: 4 Cache Evictions: 0 Cache Writes: 4 Cache Maximum Size: 60 Cache Ending Size: 60 Total Cache Entries: 4 Planning Time: 0.028 ms Execution Time: 0.031 ms	`BioseqScan` reads the table. `Projected Columns` lists exactly the four requested columns. `actual time` gives milliseconds to the first and last output row, averaged per loop. Here there are three rows and one loop. Planning and execution times describe the whole statement. The cache stores decompressed column chunks, as described in Table 9.1. Hits and misses count buffer lookups. Writes count buffers added or replaced, evictions count removed entries, and entries count current buffers. Maximum and ending sizes are peak and current payload bytes, not the configured cache capacity. These counters belong to the backend's cache state, not to a cache of SQL results. There is no chunk-skipping field here because there is no filter. The filtered search below shows that field.

`SELECT seq::text AS probe`
`FROM samples WHERE id = 2 \gset`
 	
`SELECT id, patient_name,`
`  bioseq.target_start(seq) AS target_start,`
`  bioseq.target_end(seq) AS target_end,`
`  bioseq.score(seq) AS score,`
`  bioseq.cigar(seq) AS cigar`
`FROM samples`
`WHERE age >= 30`
`  AND mmsearch(seq,`
`  :'probe'::bioseq.seq('dna4', 64));`
	6. Search with a simple metadata filter. `\gset` saves Ben's DNA in the `psql` variable `probe`. The spelling `:'probe'` safely inserts it as an SQL string literal. The following constant-query searches reuse this 512-base query.
	
	The stored target is the first `mmsearch` argument. This returns two matches for Anna and one for Ben. Each has score 1,024 and CIGAR `512M`.
	
	Requesting `cigar` enables base alignment. In this alpha version, `score` alone does not enable it and can return zero. Section 6.1 explains that stage. For the misleading allocation error, see Warning 1.

SQL or execution plan	Explanation
`SELECT bioseq.update_index(` `  'samples', 'minimizers_per_fragment');`	7. Build the MPF index explicitly. This processes three sequence values and returns 3. It extracts minimizers using the current algorithm and k, w, groups their positions, and publishes the relation data and its reader files. See Algorithm 19. The default grouped format requires the two asynchronous-persistence settings to be off. They already are off, so this example needs no preparatory `SET` commands. Update policies appear in Table 10.2.

`SELECT id, bioseq.cigar(seq) AS cigar`
`FROM samples`
`WHERE age >= 30`
`  AND mmsearch(seq,`
`    :'probe'::bioseq.seq('dna4', 64));`
	8. Repeat the search after construction. The same three matches remain. A successful index build does not imply that every query reads its file. This ordinary constant-query scan recomputes target minimizers in the current default configuration.
	
	The two-table join later uses the required persisted-reader path. Figure 10.10 shows the distinction.

`SELECT id,`
`  bioseq.target_start(seq) AS target_start,`
`  bioseq.target_end(seq) AS target_end,`
`  bioseq.score(seq) AS score,`
`  bioseq.cigar(seq) AS cigar`
`FROM samples`
`WHERE age >= 30 AND mmsearch(seq,`
`  :'probe'::bioseq.seq('dna4', 64),`
`  aligner_options(`
`    kmer_size => 15, window_size => 10,`
`    minimizer_algorithm => 4,`
`    chaining_algorithm => 1,`
`    base_alignment_algorithm => 1));`
	Choose algorithms for one call. These are the same defaults written explicitly to demonstrate `aligner_options`, so the result still has three matches. The codes choose bounded ntHash SIMD minimizers (4), SIMD minimap2 chaining (1), and BlockAligner (1). Each minimizer window contains ten consecutive 15-mers.
	
	Omitted options inherit session settings. Appendix 2 lists the numeric codes beside their named GUC values. Section 11.4.1 explains precedence.

`EXPLAIN (ANALYZE, COSTS OFF)`
`SELECT id,`
`  bioseq.target_start(seq) AS target_start,`
`  bioseq.target_end(seq) AS target_end,`
`  bioseq.score(seq) AS score,`
`  bioseq.cigar(seq) AS cigar`
`FROM samples`
`WHERE age >= 30 AND mmsearch(seq,`
`  :'probe'::bioseq.seq('dna4', 64),`
`  aligner_options(`
`    kmer_size => 15, window_size => 10,`
`    minimizer_algorithm => 4,`
`    chaining_algorithm => 1,`
`    base_alignment_algorithm => 1));`
	9. Explain exactly that search. The following consecutive blocks contain its complete text plan. Keeping the CIGAR projection matters, because removing it would also remove the request for base alignment.

SQL or execution plan	Explanation
Result (actual time=1.281..1.295 rows=3 loops=1) −> Custom Scan (BioseqScan) on samples (actual time =1.281..1.294 rows=3 loops=1) Bioseq Projected Columns: id, age, seq Bioseq Chunk Group Filters: (age >= 30) Bioseq Chunk Groups Removed by Filter: 0 Bioseq Vectorized Filter: (age >= 30) Bioseq Index Read Requirement: allow Bioseq Recomputed: minimizers 100%	`Result` projects the output from `BioseqScan`. Three alignment rows come from two qualifying patients. The scan reads the identifier, age and sequence. A chunk-group filter compares the age condition with stored minima and maxima. Zero groups were skipped here. The vectorized filter tests individual ages in the remaining batches. Listing 2 explains this difference. `allow` permits ordinary index reuse or recomputation. The planned share is 100% recomputed. The actual counters below confirm it. See Table 10.4.
Bioseq Scheduler: supervisor=local mode= task_parallel workers=6/6 batches=8/8 awaiting=0 queries=8/0 qps=0.000 async_persist_waiting=0 index_memory_reserved=0 One−to−many alignment: Task−parallel [mode=task_parallel, workers=6, index_memory_reserved=0 bytes]: Planned Architecture: NEON_BF16 CPU−Supported Targets: NEON_BF16, NEON, NEON_WITHOUT_AES, EMU128, SCALAR Highway Compiled Targets: NEON_BF16, NEON, NEON_WITHOUT_AES, EMU128 Runnable Targets: NEON_BF16, NEON, NEON_WITHOUT_AES, EMU128 ∗ Executed Architecture: NEON_BF16 | QuerySketchStage: Bounded u32 ntHash SIMD Minimizers Highway Function: minimizer::bounded_nthash [planned architecture: NEON_BF16]	The scheduler line is a snapshot of the local task scheduler. It reports workers, processed/scheduled batches, waiting work and processed/waiting queries. The local supervisor leaves the query-rate field at zero. The last fields count pending persistence jobs and reserved index bytes. These are not timings for individual alignment stages. Section 11.5.2 explains its queues. Highway selects a CPU instruction set. Runnable targets are the intersection of compiled and CPU-supported targets. Planned and executed architectures agree here. Your CPU may show different names. `QuerySketchStage` extracts the query minimizers, and `Highway Function` names its selected kernel. See Section 11.4.2.
| index_stage: pipeline configuration [cep.window_len: 0] [cep.kmer_len: 0] [cep.sequence_len_log2: 63] [cep.specialised: fallback] | seed_hit_stage: Seed hits + canonical CPU ordering max_occ: 500 query_len: 0 [reserved: 64]	These are internal pipeline descriptions. The zero `cep.window_len`, `cep.kmer_len` and `query_len` values are placeholders in that description. They do not mean that this query has length zero or uses $k = w = 0$. `sequence_len_log2` is a compiled size category. `specialised: fallback` means a general parameter implementation, not a failed GPU request. `max_occ` limits accepted occurrences of one minimizer hash. `reserved` is a planned anchor-buffer capacity in elements, not a byte count or the number of matches.

SQL or execution plan	Explanation
| chain_stage: Minimap2 SIMD min_chain_score: 40 min_seed_count: 3 max_chain_skip: 25 max_chain_iter: 5000 bandwidth: 500 max_gap: 5000 simd_chaining_policy: bounded_batch_approximate [vectorised: true] [requires_zero_skip_penalty: false] [specialised: fallback] Highway **Function**: chaining::chain_anchors_simd [planned architecture: NEON_BF16] | region_stage: Region generation query_len: 0	Chaining joins seed anchors into candidate regions. `min_chain_score` and `min_seed_count` reject weak chains. `max_chain_skip` and `max_chain_iter` limit predecessor exploration. Bandwidth and maximum gap constrain how far anchors may separate. This is the bounded SIMD approximation described in Section 5.13. Its flags describe the vectorized implementation, whether a zero skip penalty is required, and the general specialization. The next stage makes regions from those chains.
| primary_secondary_stage: Classify primary/secondary mask_level: 0.500000 mask_len: 2147483647 sub_diff: 8 | secondary_filter_stage: **Filter** secondary regions pri_ratio: 0.800000 min_diff: 30 best_n: 5	Regions are classified as primary or secondary by their overlap. `mask_level` is the overlap threshold and `mask_len` limits uncovered query length. `sub_diff` is the close-score margin used when counting competitors. Secondary filtering uses a score ratio, an absolute score margin (`min_diff`), and `best_n`. The last value limits retained secondary regions, not SQL result rows or primary matches. See Section 6.3.
| align_stage: Block Aligner match_score: 2 mismatch_pen: 4 gap_open1: 4 gap_ext1: 2 block_min_size: 32 block_max_size: 256 [vectorised: block−affine] [single_affine_only: true] [specialised: fallback] Highway **Function**: alignment::block_aligner [planned architecture: NEON_BF16]	BlockAligner computes the base-level alignment and CIGAR. Matching bases add 2 to the score, substitutions cost 4, and the displayed gap-opening and extension penalties define its single affine-gap scheme. Its adaptive block size ranges from 32 to 256. The vectorized and single-affine flags identify this implementation. The Highway line identifies its CPU kernel. See Section 6.1.1.
| primary_secondary_stage: Classify primary/secondary mask_level: 0.500000 mask_len: 2147483647 sub_diff: 8 | secondary_filter_stage: **Filter** secondary regions pri_ratio: 0.800000 min_diff: 30 best_n: 5 ∗ Submit results [targets=2 persisted_index=0 recomputed=2]	Primary/secondary classification and filtering run again after base alignment has refined the regions. Their parameters have the same meanings as above. `Submit results` reports two processed targets. Both used recomputed minimizers and neither used a persisted reader. This confirms why the preceding index build did not change this scan's chosen read path.

SQL or execution plan	Explanation
* Execution statistics [requested_backend= simd_cpu actual_backend=mixed_invalid requested_index_source=persisted actual_index_source= recomputed attempted=2 fully_attested=0 unattested=2 environment_mixed=false persisted_identity_mixed=false identity_conflict=false saturated=false] * First unattested [target_row=1 query_ordinal=0] * **Index** outcomes [targets=2 persisted=0 missing_persisted=2 recomputed=2]	These diagnostics try to certify which implementations completed the work. `attempted`, `fully_attested` and `unattested` count attempted and certified target/query jobs. The first uncertified target and query ordinal are printed below them. `mixed_invalid` with two unattested targets means the current instrumentation did not certify a complete backend combination. It is not evidence of GPU execution or a scalar fallback. The flags report mixed environments, mixed persisted identities, conflicting identity evidence and saturated counters. All are false here. Requested and actual index sources differ. `missing_persisted` means no suitable persisted reader was used for these targets, even though physical MPF data exists. The permissive policy allows recomputation.
* GPU dispatch [seed=0 chain=0 chain_backtrack =0 coordinate_vote=0 extend=0 backend_declines=0 backend_fallbacks=0] GPU Chain Dispatches: 0 GPU Chain Backtrack Dispatches: 0 GPU Coordinate Vote Dispatches: 0 * Host stats [query_sketch_ns=0 target_index_build_ns=0 query_hash_readbacks=0 query_symbol_packing=0 target_symbol_packing=0 alignment_classification=0 cigar_construction=0]	GPU counters cover seed lookup, chaining, backtracking, coordinate voting and extension. Declines and fallbacks describe execution-route decisions. All are zero in this CPU example. Three counters are also printed under their longer names. Host statistics include query-sketch and target-build nanoseconds, hash readbacks, symbol-packing calls, alignment classification and CIGAR construction. Zeros in this incompletely attested execution do not mean that these operations took no time or that no CIGAR was produced.
-> Cache Cache Hits: 0 Cache Misses: 5 Cache Evictions: 0 Cache Writes: 5 Cache Maximum Size: 16608 Cache Ending Size: 16608 Total Cache Entries: 5 Planning Time: 0.241 ms Execution Time: 1.432 ms	The cache fields have the same meanings as in the first plan. This search needs sequence data as well as metadata, so the buffers and their sizes differ. Cache entries count column chunks, not matches. The final two lines give statement planning and execution time in milliseconds. They include work outside the alignment stages.

SQL or execution plan	Explanation

`SELECT id,`
`  bioseq.target_start(seq) AS target_start,`
`  bioseq.target_end(seq) AS target_end,`
`  bioseq.score(seq) AS score,`
`  bioseq.cigar(seq) AS cigar`
`FROM samples`
`WHERE mmsearch(seq, seq)`
`ORDER BY id;`
	10. Search each sequence within itself. Each of the three rows produces a hit. With CIGAR requested, these exact self-alignments cover their complete sequences and score twice their length.
	
	This is a same-row search. It performs alignment rather than replacing `mmsearch(seq,seq)` with a constant Boolean value.

`SELECT t.id AS target_id, q.id AS query_id,`
`  bioseq.target_start(t.seq) AS target_start,`
`  bioseq.target_end(t.seq) AS target_end,`
`  bioseq.query_start(t.seq) AS query_start,`
`  bioseq.query_end(t.seq) AS query_end,`
`  bioseq.score(t.seq) AS score,`
`  bioseq.cigar(t.seq) AS cigar`
`FROM samples AS t JOIN samples AS q`
`  ON mmsearch(t.seq, q.seq)`
`ORDER BY target_id, query_id, target_start,`
`  query_start;`
	Compare rows using a self-join. The two aliases independently choose target and query rows. The result has 13 alignments, including the three equal-ID pairs. Section 8.4 explains these roles.
	
	For target 1 and query 2, score 1,024 occurs at both $[256, 768)$ and $[2304, 2816)$. These are different occurrences, even though both have `512M`. Keep positions as well as scores. All helper arguments name the target `t.seq`, including the query coordinate helpers. Figure 8.1 explains why a stored row can produce several output rows.

`COPY (`
`  SELECT line FROM samples`
`  CROSS JOIN LATERAL (VALUES`
`  (0, '>sample_' || id || ' synthetic DNA'),`
`  (1, seq::text)`
`  ) AS fasta(part, line)`
`  ORDER BY id, part`
`) TO '/tmp/bioseq-walkthrough.fasta';`
 	
`CREATE TABLE imported (`
`  header text,`
`  seq bioseq.seq('dna4', 64) NOT NULL`
`) USING bioseq;`
	11. Load a FASTA file into another table. The first statement writes our three records as alternating header and sequence lines. This makes the exercise independent of external downloads. The server must be able to write and then read the named path.
	
	`load` returns 3. With one text column, one sequence column and no explicit mappings, its default `auto` mode keeps each full header without the leading `>`. It does not create an identity column or invent numeric IDs.

SQL or execution plan	Explanation
`SELECT bioseq.load('imported',` `  '/tmp/bioseq-walkthrough.fasta');`	Header parsing options. This call loads the file once. Running it again would append duplicate records. `header_format` accepts `auto`, `plain`, `uni_prot`, `uni_ref`, `uni_parc`, `uni_mes`, `spd`, `ncbi`, `ncbi_gi`, `ncbi_pdb`, `refseq`, `genbank`, `gisaid` and `imgt`. Auto can detect a dialect when field parsing is needed. Plain keeps an unsplit identifier. The fourth argument maps parsed fields to table columns using `bioseq.column_mapping`, with destination first and source second. For example, `('accession','accession')` copies that parsed field. Section 11.6.3 explains the mappings and loader limits.

`SELECT header, bioseq.len(seq) AS bases,`
`  seq[:16]::text AS prefix`
`FROM imported ORDER BY header;`
 	
`SELECT seq[128:384]::text AS short_probe`
`FROM imported`
`WHERE header LIKE 'sample_2 %' \gset`
 	
`SELECT header,`
`  bioseq.target_start(seq) AS target_start,`
`  bioseq.target_end(seq) AS target_end,`
`  bioseq.cigar(seq) AS cigar`
`FROM imported`
`WHERE mmsearch(seq,`
`  :'short_probe'::bioseq.seq('dna4', 64))`
`ORDER BY header, target_start;`
	12. Inspect prefixes, then search a loaded slice. The first query returns three headers, lengths and sixteen-base previews. The next saves a 256-base slice of the loaded Ben record as `short_probe`.
	
	Searching it returns four alignments: two in Anna's sequence and one in each of the other records. The search query is still a typed DNA4 value. Taking its slice before the search avoids displaying a long literal.

`SELECT t.id AS target_id, q.header,`
`  bioseq.score(t.seq) AS score,`
`  bioseq.dp_score(t.seq) AS dp_score,`
`  bioseq.dp_max_score(t.seq) AS dp_max_score,`
`  bioseq.chain_score(t.seq) AS chain_score,`
`  bioseq.secondary_chain_score(t.seq)`
`  AS secondary_chain_score,`
`  bioseq.query_len(t.seq) AS query_len,`
`  bioseq.target_len(t.seq) AS target_len,`
`  bioseq.query_start(t.seq) AS query_start,`
`  bioseq.query_end(t.seq) AS query_end,`
`  bioseq.target_start(t.seq) AS target_start,`
`  bioseq.target_end(t.seq) AS target_end,`
`  bioseq.num_matches(t.seq) AS num_matches,`
`  bioseq.block_length(t.seq) AS block_length,`
	13. Return every alignment field. This statement continues in the next block. Scores and coordinates describe the current match. `score` and `dp_score` are aliases. On this CPU path, `dp_max_score` currently repeats that score. `chain_score` describes the seed chain and the secondary score describes a competitor.
	
	Lengths count logical symbols. Both query and target coordinates use zero-based, half-open intervals. `block_length` is the alignment block length.
	
	`num_matches` is a reported aligned-residue count, not a result-row count. The current CPU code counts CIGAR `M` and `=` runs. Since `M` can include substitutions, this field is not a reliable count of identical bases for an imperfect alignment.

SQL or execution plan	Explanation

`  bioseq.edit(t.seq) AS edits,`
`  bioseq.mapq(t.seq) AS mapq,`
`  bioseq.seed_count(t.seq) AS seed_count,`
`  bioseq.is_reverse(t.seq) AS is_reverse,`
`  bioseq.is_primary(t.seq) AS is_primary,`
`  bioseq.has_cigar(t.seq) AS has_cigar,`
`  bioseq.cigar(t.seq) AS cigar,`
`  bioseq.divergence(t.seq) AS divergence`
	`edit` counts alignment edits. `mapq` estimates mapping quality, while `divergence` is estimated from seeds. Neither is a percentage identity. `seed_count` counts chain anchors.
	
	The Boolean fields report reverse orientation, primary classification and whether CIGAR is present. CIGAR describes runs of alignment operations. `M` consumes both sequences and can include a substitution. `I` consumes the query, and `D` consumes the target. Section 8.3 explains how these helpers receive the current match's values.

`FROM samples AS t JOIN imported AS q`
`  ON mmsearch(t.seq, q.seq,`
`    aligner_options(index_read_requirement => 2))`
`ORDER BY score DESC, target_id, q.header,`
`  target_start, query_start;`
	This join requires the compatible persisted target index built in step 7. Code 2 is a per-call read policy, not a GUC. It lets this supported `BioseqBatchJoin` use the grouped MPF reader files. A missing or incompatible reader is an error, as explained in Section 10.7.2.
	
	The result has 13 alignments, ordered by descending score and then by identifiers and positions. The complete statement includes all 21 helpers.

`SELECT count(*) AS patients,`
`  min(age) AS youngest, max(age) AS oldest,`
`  sum(age) AS total_age, avg(age) AS mean_age,`
`  min(upload_date) AS first_upload,`
`  max(upload_date) AS last_upload`
`FROM samples`
`WHERE age >= 30`
`  AND upload_date >= DATE '2026-09-01';`
	14. Aggregate metadata in batches. This returns two patients, minimum age 34, maximum age 52, sum 86 and average 43. The dates span 1–2 September.
	
	The eligible aggregate functions operate directly on ordinary integer and date columns. This query does not need sequence symbols or alignments.

`EXPLAIN (ANALYZE, COSTS OFF)`
`SELECT count(*) AS patients,`
`  min(age) AS youngest, max(age) AS oldest,`
`  sum(age) AS total_age, avg(age) AS mean_age,`
`  min(upload_date) AS first_upload,`
`  max(upload_date) AS last_upload`
`FROM samples`
`WHERE age >= 30`
`  AND upload_date >= DATE '2026-09-01';`
	Explain the same aggregate. The excerpt below keeps only the node and filter fields relevant to vectorization. The dots stand for the omitted cache and timing lines, not additional SQL to execute.

SQL or execution plan	Explanation
Custom Scan (VectorAggNode) (actual time=0.067..0.067 rows=1 loops=1) −> Custom Scan (BioseqScan) on samples (actual time =0.058..0.063 rows=1 loops=1) Bioseq Projected Columns: upload_date, age Bioseq Chunk Group Filters: ((age >= 30) AND (upload_date >= '2026−09−01'::date)) Bioseq Chunk Groups Removed by Filter: 0 Bioseq Vectorized Filter: ((age >= 30) AND (upload_date >= '2026−09−01'::date)) ...	`VectorAggNode` is the vector aggregate. Its child supplies column batches and applies the vectorized age/date filter. Here the child reports one batch slot, while the aggregate returns one summary row. Neither `rows=1` means that only one patient was included. This node currently has no separate per-lane or SIMD-speedup counter. The analyzed node and its batch-reading child show the chosen execution path. Grouping or aggregating alignment results can select an ordinary aggregate instead. Section 8.3.5 distinguishes batch filtering from other execution stages.

`UPDATE samples SET age = age + 1`
`WHERE id = 1`
`RETURNING id, patient_name, age;`
 	
`INSERT INTO samples`
`VALUES (4, DATE '2026-09-04', 'Daria', 41,`
`  :'probe'::bioseq.seq('dna4', 64))`
`RETURNING id, patient_name, bioseq.len(seq);`
 	
`SELECT bioseq.update_index(`
`  'samples', 'minimizers_per_fragment');`
`SELECT bioseq.update_index(`
`  'samples', 'symbol_counts');`
	15. Update, insert and rebuild. Anna becomes 35 and Daria receives Ben's sequence. Both writes return their affected row. There are now four stored sequences.
	
	With insert maintenance off, these writes do not build new minimizer generations. Even a metadata update creates a replacement row version. The two explicit calls prepare MPF and symbol-count indexes for all four visible values. See Figure 9.9 and Section 10.6.3.

`SELECT index_name, update_policy,`
`  storage_size, time_since_update,`
`  bioseq.index_table_row_count(`
`  'samples', index_name) AS physical_rows`
`FROM bioseq.list_metadata()`
`WHERE table_name = 'samples'::regclass`
`ORDER BY index_name;`
	Inspect the three registered index families. MPF and symbol counts have stored data. MPS remains unbuilt because this exercise never requested it.
	
	`storage_size` measures the index relations and excludes separate MPF reader files. Physical rows count internal index records, not patients. The update age is the time since the recorded build. `UPDATE_ON_INSERT` is the family's policy, which still needs the separate maintenance switch. See Table 10.3.

`SELECT id, bioseq.score(seq) AS score,`
`  bioseq.cigar(seq) AS cigar`
`FROM samples`
`WHERE mmsearch(seq,`
`  :'probe'::bioseq.seq('dna4', 64),`
`  aligner_options(`
`  minimizer_algorithm => 0,`
`  chaining_algorithm => 0,`
`  base_alignment_algorithm => 0,`
`  index_read_requirement => 1))`
`ORDER BY id;`
	Try scalar minimizers, scalar chaining and KSW2. The three algorithm codes are all 0. The query returns five matches, each scoring 1,024 with `512M`.
	
	Read requirement 1 forbids persisted target-index reads for this call. That matters because the stored index was built with a different minimizer algorithm. It remains stored for later compatible queries. This local override does not change session defaults. See Section 10.4.1.

SQL or execution plan	Explanation

`SELECT id,`
`  bioseq.symbol_count(seq, 'A') AS adenines,`
`  (seq[:8] OPERATOR(bioseq.||) seq[-8:])`
`  ::text AS joined_ends,`
`  seq[0]::text AS first_letter,`
`  bioseq.reverse(seq[:8])::text AS reversed,`
`  bioseq.reverse_complement(seq[:8])::text`
`  AS reverse_complement,`
`  bioseq.reverse(bioseq.reverse_complement(`
`  seq[:8]))::text AS complemented,`
`  bioseq.edit(seq) AS edits,`
`  bioseq.cigar(seq) AS cigar`
`FROM samples`
`WHERE age >= 30`
`  AND mmsearch(seq,`
`  :'probe'::bioseq.seq('dna4', 64))`
`  AND bioseq.edit(seq) < 5`
`ORDER BY id;`
	Combine sequence operations and an edit filter. This returns four alignment rows for Anna, Ben and Daria. It counts A bases, joins two eight-base slices, reads one letter, and reverses or reverse-complements a prefix. Reversing the reverse complement gives the ordinary complement. There is no separate SQL complement helper.
	
	The exact predicate `bioseq.edit(seq) < 5` permits an early edit prefilter and still checks the retained alignment's edit count. Use the target sequence column here, not its ID. Prefilter dispatch depends on the supported candidate shape, as explained in Section 6.2.
	
	The four matches have zero edits. The sequence expressions remain typed until their final text casts, following Section 7.3.

Appendix 2

Configuration reference

This appendix lists all runtime settings registered by BioseqDB (GUCs) - their types, defaults, possible values and descriptions. Each setting can be viewed and set with the `SHOW` and `SET` commands.[1] Some settings are hidden so they're not visible with `SHOW ALL`.

Listing 2.1: Inspecting and settingBioseqDB config values

```
SHOW bioseq.aligner.minimizer.algorithm;

SELECT name, setting, boot_val, reset_val,
       vartype, context, min_val, max_val, enumvals
FROM pg_settings
WHERE name = 'bioseq.aligner.minimizer.algorithm';

SET bioseq.aligner.minimizer.algorithm = 'scalar';
```

2.1. Physical storage

Table 2.1: Physical storage settings.

GUC name	Type	GUC values	Description
`bioseq.storage.compression.algorithm`	enum	`zstd` (default)	Choose the compression algorithm used for storing data columns Use Zstandard when compiled in. This is the preferred build-time default. Changing the default does not rewrite any existing indices!
		`none`	Store column chunks without compression.
		`pglz`	Use Postgres built-in coded. This becomes the default when neither of previous two is available.
		`lz4`	Use LZ4 when compiled in. This becomes the default when Zstandard is unavailable.

Continued on the next page

[1] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: pg_settings; PostgreSQL Global Development Group, PostgreSQL 17: Configuration Inspection Functions.

GUC name	Type	GUC values	Description
`bioseq.storage.compression.level`	int	`3` (default) 1…19	Set the zstd compression level if zstd is used.
`bioseq.storage.stripe.row_limit`	int	`150000` (default) 1000…100000000 rows	Sets the maximum rows in a new table's column stripe, when writing rows. Existing table options and stored stripes are unchanged.
`bioseq.storage.chunk_group.row_limit`	int	`10000` (default) 1000…100000000 rows	Sets the maximum rows in a new table's chunk group, when writing rows. Chunk groups divide a stripe into smaller column-reading units.
`bioseq.storage.dml.enabled` hidden	bool	`on` (default)	Special setting that controls whether reads apply deleted-row masks. Default, shouldn't be changed. Normally deleted rows that are marked as such, should not be returned in queries.
		`off`	Bypass any deletion filtering. Useful for debugging, the queries will return all rows including deleted ones. Note that if the table is rewritten using VACCUM FULL, then deleted rows will physically dissapear!
`bioseq.column_cache.enabled`	bool	`on` (default)	Controls the backend's decompressed column-chunk cache. After decompression save the buffer, so it's avaialable for reuse Decompression cache only applies to one query. After completion of the query, the cache is cleared.
		`off`	Read and decompress but do not save buffers in cache.
`bioseq.column_cache.capacity_mb`	int	`200` (default) 20…20000 MiB	Limits the decompressed column-chunk cache BioseqDB will still allocate buffers for decompression, so this setting does not control the total memory used during decompression
`bioseq.load.write_overlap.enabled`	bool	`off` (default)	Experimental setting to use parallelized writing and decoding of data chunks through queue when loading FASTA/FASTQ files. Default reader perform decoding and writing (placing sequences in buffer) sequentially.
		`on`	Enable parallelized writing and decoding of data chunks through queue.
`bioseq.storage.sequence_shard.max_size_gib` superuser	int	`1024` (default) 0…2147483647 GiB	Sets the sequence-shard size limit. If the shard table exceeds that size, then new table will be created. Setting it to 0 means that each sequence is stored in a separate table (useful for debugging, but extremally ineffective in practice)

2.2. Planning and execution

Table 2.2: Planning and execution settings.

GUC name	Type	GUC values	Description
`bioseq.execution.parallel.enabled` hidden	bool	`on` (default)	Enable Postgres parallel queries Allow possible parallel scan paths (actual worker availability still depends on PostgreSQL planner, so this setting does not force the planner to use parallel scans). Additionally, any query using mmsearch won't be parallelized, due to current code architecture. The parallel scans however are possible for metadata queries. Alignment won't use parallel scans even with this setting enabled.
		`off`	Disable all parallel scans on BioseqDB tables.
`bioseq.execution.parallel.minimum_processes`	int	`8` (default) 1…32	Requests the PostgreSQL worker count for possbible Bioseq parallel scan paths. This is soft minimum and the actual worker count is determined by the PostgreSQL planner.
`bioseq.vectorization.enabled` hidden	bool	`on` (default)	Can disable vectorized SQL operations for aggregates like SUM(), COUNT(), etc. Use supported vectorized SQL operations when the plan allows them. Vectorized operations only apply to non-sequence aggregates, not alignment pipeline.
		`off`	Use normal SQL processing for those operations.
`bioseq.planner.index_scan.enabled` hidden	bool	`on` (default)	Can allow usage of non-BioseqDB indices with BioseqDB tables. Allow using non-BioseqDB indices during queries, for example it becomes possible to use B-tree index on metadata column. This settings does not control any BioseqDB indices like MPF/MPS etc.
		`off`	Do not allow using any non-BioseqDB indices on BioseqDB tables. This can heavily degrade performance as Postgres will select plan that ignores existing B-trees, hash indexes etc.
`bioseq.planner.custom_scan.enabled` hidden	bool	`on` (default)	Entirely disable all BioseqDB custom scans. Allow the planner to install Bioseq custom scans. Useful to force Postgres to select non-BioseqDB plans. Do not use this unless you know what you're doing.
		`off`	Disable custom scans. This means that it won't be possible to use 'mmsearch' function anymore as it requires custom scan path. All other metadata queries still will be possible.

Continued on the next page

GUC name	Type	GUC values	Description
bioseq.planner.custom_scan.max_paths hidden	int	64 (default) 1…1024	Limit paths explored by GQO for BioseqDB custom scans. Increasing the number may yield better plans, but increase planning time.
bioseq.planner.qual_pushdown.enabled hidden	bool	on (default)	Controls behaviour of predicate pushdown. Normally filters can be pushed down into the scan, which improved performance. The predicates that Postgres reports as highly-correlated will be used to possibly reduce number of loaded chunks (min/max prunning mechanism) Pushdown uses correlation statistics for column reported by Postgres to decide if this predicate can be useful to avoid loading chunks
		off	Leave predicates outside the scan, which may severely degrade performance.
bioseq.planner.qual_pushdown.correlation_threshold hidden	real	0.4 (default) 0…1	Sets the minimum absolute column correlation value (reported by Postgres table statistics) used by chunk prunning logic described for bioseq.planner.qual_pushdown.enabled. Values closer to one, mean stronger correlation.
bioseq.planner.sequence_sort_scan.enabled hidden	bool	off (default)	Enable or disable custom sort for BioseqDB sequences. Sort works only if there's single sorting key that is sequence column itself. Use default Postgres sort.
		on	Consider the custom batch-sort where possible.

2.3. Task scheduling

Table 2.3: Task scheduling settings.

GUC name	Type	GUC values	Description
bioseq.scheduler.backend.max_concurrency reload	int	0 (default) 0…1024	Limits maximum number of workers in TBB worker pool (this is separate to Postgres max workers setting)
bioseq.scheduler.pipeline.mode	enum	task_parallel (default)	Changes how the pipeline execution is performed Each query dispatches separate worker (exxtension of parallel_dispatch) and additionally each stage of pipeline can submit independent tasks to TBB worker pool In the benchmark I use task_parallel, because it enables the highest degree of parallelism
		single_thread	Each query dispatches separate worker, but one pipeline itself isn't parallelized.
		parallel_dispatch	

Continued on the next page

GUC name	Type	GUC values	Description
`bioseq.scheduler.postgres_parallel.enabled`	bool	`off` (default)	Reduce tasks slots to 1 Disable any parallelism completely (note that if bioseq.scheduler.pipeline.mode is set to task_parallel, then pipeline will be executed via task queues, but only one worker will be active at the time. This setting is the highest level limiter on parallelism of workers) This setting effectively forces bioseq.scheduler.backend.max_concurrency to be 1
		`on`	Allow >1 parallel tasks limit

2.4. Index construction and reading

Table 2.4: Index construction and reading settings.

GUC name	Type	GUC values	Description
`bioseq.index.insert_maintenance.enabled`	bool	`off` (default)	Enables automatic index incremental update when row is inserted into the table. Save new source rows without automatic index builds. You can always still manually update index via update_index function.
		`on`	Currently grouped MPF index (the default one) does not support this function and reports an error.
`bioseq.index.sequence_read.enabled`	bool	`on` (default)	Controls whether the pipeline executor can read indexes. If the option is disabled, then pipeline will always recompute all seeds from scratch. Normally allow reading BioseqDB indices. There's also per-search option passed via aligner_options: index_read_requirement codes 1 and 2 override this setting.
		`off`	Force pipeline to always recompute all seeds from scratch.

Continued on the next page

GUC name	Type	GUC values	Description
`bioseq.index.async_persistence.enabled`	bool	`off` (default)	Very experimental setting that allows main process to asynchronously update missing indices for sequences: If query does not have seeds in index for certain sequence and needs to compute them, after the query the index will be updated. Do not stage asynchronous persistence work. Grouped MPF index (default) does not implement this feature and using it with this option enabled will produce an error. Additionally you need to enable bioseq.index.recompute_async_persistence.enabled as well.
		`on`	Allow the limited ordinary-MPS submission path when its other requirements are met.
`bioseq.index.recompute_async_persistence.enabled`	bool	`off` (default)	Separate setting, but effectively to enable async persistence you need to enable bioseq.index.async_persistence.enabled together with this one. Same as bioseq.index.async_persistence.enabled Set bioseq.index.async_persistence.enabled to the same value as this one.
		`on`	Same as bioseq.index.async_persistence.enabled
`bioseq.index.memory_cap_mb`	int	`0` (default) `0`…`2147483647` MiB	Specialized limit on the memory allocated by index builders/readers in MB. Using 0 means that they share the same memory budget as other query buffers.
`bioseq.index.sequence_minimizer_sort.algorithm` hidden	enum	`std_sort` (default)	Change sorting algorithm used for non-grouped MPF index builds Use the standard-library sort SIMD versions weren't tested by me in terms of correctness and they were slower than the default
		`simd_u128_vqsort`	Use Highway VQSort on packed 128-bit keys (implementation from C++ Highway repo).
		`simd_u128_vec_stream_1`	Use the custom SIMD sorter with one vector stream.
		`simd_u128_vec_stream_2`	Use the custom SIMD sorter with two vector streams.
		`simd_u128_vec_stream_4`	Use the custom SIMD sorter with four vector streams.
`bioseq.index_build.run_buffer_bytes` hidden, superuser	int	`0` (default) `0`…`2147483647` bytes	For MPS overrides the external-sort run buffer size. Zero computes a size from the common BioseqDB memory budget.

Continued on the next page

GUC name	Type	GUC values	Description
bioseq.aligner.fragment_minimizer.lookup_route	enum	direct_compact_hash_lookup (default)	Changes how grouped MPF works when reading is performed By default read hash directly from the hash bucket. The directory row should be faster in theory, but it's disabled, because I'm not sure if it's correct
		control_block_scan	Search the directory row for the hash bucket first
bioseq.aligner.grouped_soa_generation.enabled	bool	on (default)	Selects if we want to use grouped MPF version of index or non-grouped one (for writing only) Prefer grouped version of MPF index over non-grouped (for writing only) Changing this does not rewrite existing indexes, only applies to new ones and does not affect reading!
		off	Use only non-grouped MPF when possible (for writing only)

2.5. Minimizers and seed filtering

Table 2.5: Minimizers and seed filtering settings.

GUC name	Type	GUC values	Description
bioseq.aligner.minimizer.kmer_size	int	15 (default) 1…64 symbols	Sets the number of sequence symbols in each k-mer Per search: kmer_size.
bioseq.aligner.minimizer.window_size	int	10 (default) 1…1024 k-mers	Sets the number of consecutive k-mers in one minimizer window Per search: window_size.
bioseq.aligner.minimizer.algorithm	enum	bounded_nthash_simd code 4 (default)	Chooses how minimizers are calculated Use the bounded SIMD BioseqDB algorithm Changing the algorithm can invalidate compatibility with saved indexes! Per search: minimizer_algorithm.
		scalar code 0	Minimap2 scalar algorithm
		simd_minimap2_reference code 1	Use the naive version of SIMD implementation of minimap2's
		two_stack_nthash code 2	Use rolling ntHash and two-stack window-minimum inspired by simd-minimizers paper.
		vulkan_minimap_integer code 3	Use BioseqDB GPU algorithm

Continued on the next page

GUC name	Type	GUC values	Description
bioseq.aligner.seeding. maximum_occurrence	int	500 (default) 1…16777216 positions	Excludes minimizer keys with more target occurrences than this limit. Per search: max_occ.
bioseq.aligner.seeding. intermediate_occurrence_ threshold	int	500 (default) 1…16777216 positions	Currently unused Per search: mid_occ.

2.6. Chaining

Table 2.6: Chaining settings.

GUC name	Type	GUC values	Description
bioseq.aligner.chaining. maximum_gap	int	5000 (default) 1…1073741824 positions	Sets the requested maximum anchor gap used by chaining. Scalar (minimap2)chaining raises this limit to at least the bandwidth value. Per search: max_gap.
bioseq.aligner.chaining. maximum_reference_gap	int	-1 (default) −1…1073741824 positions	Currently unused Per search: max_gap_ref.
bioseq.aligner.chaining. bandwidth	int	500 (default) 1…1048576 positions	Despite the name it's also used by aligner stage, limits maximum distane to diagonal. Per search: bandwidth.
bioseq.aligner.chaining. long_join_bandwidth	int	0 (default) 0…1048576 positions	Currently unused. Per search: bandwidth_long.
bioseq.aligner.chaining. maximum_skipped_anchors	int	25 (default) 0…1048576	Limit of consecutive updates for anchors that do not improve score (only for supported algorithms like scalar) Per search: max_chain_skip.
bioseq.aligner.chaining. maximum_predecessors	int	5000 (default) 1…4194304	Limits how many predecessors chainer can consider, but scalar chaining (depending on the actual algorithm) can examine one additional candidate. RMQ does not use this field. Per search: max_chain_iter.
bioseq.aligner.chaining. minimum_seed_count	int	3 (default) 0…1048576	Sets the minimum seed count for an accepted chain. Per search: min_seed_count.
bioseq.aligner.chaining. minimum_score	int	40 (default) 0…1048576	Sets the minimum accepted chain score (the score is computed by the selected chaining algorithm) Per search: min_chain_score.

Continued on the next page

GUC name	Type	GUC values	Description
bioseq.aligner.chaining. gap_penalty_scale	real	0.8 (default) 0…100	Scales the gap penalty used by chaining (larger values penalize that contribution more strongly) Per search: chain_gap_scale.
bioseq.aligner.chaining. skip_penalty_scale	real	0.0 (default) 0…100	Scales the chaining skip-distance penalty. Per search: chain_skip_scale.
bioseq.aligner.chaining. algorithm	enum	simd_minimap2 code 1 (default)	Selects algorithm used for chaining BioseqDB implementation of accelerated minimap2-inspired chaining Selected algorithm might still be rejected if the hardware does not support it Per search: chaining_algorithm.
		scalar_minimap2 code 0	Minimap2 scalar chaining
		range_minimum_query code 2	BioseqDB RMQ on AVL trees similar to what minimap2 uses
		vulkan_tiled_ dynamic_programming code 3	Use tiled GPU chaining

2.7. Primary and secondary alignments

Table 2.7: Primary and secondary alignments settings.

GUC name	Type	GUC values	Description
bioseq.aligner.secondary. overlap_threshold	real	0.5 (default) 0…1	Sets the primary/secondary classification thresholds Per search: mask_level.
bioseq.aligner.secondary. uncovered_length_limit	int	2147483647 (default) 0…2147483647 symbols	Limits the uncovered query length used when classifying an alignment as secondary to an existing primary. Per search: mask_len.
bioseq.aligner.secondary. score_ratio_threshold	real	0.8 (default) 0…1	Sets the secondary-to-primary score-ratio threshold. Zero bypasses this filtering step, including its secondary-count limit. Per search: pri_ratio.
bioseq.aligner.secondary. maximum_count	int	5 (default) 0…1024	Limits retained secondary alignments across the complete list passed to the filtering stage Per search: best_n.

2.8. Base alignment

Table 2.8: Base alignment settings.

GUC name	Type	GUC values	Description
bioseq.aligner.scoring.match_score	int	2 (default) 1…127	Sets the score added for an aligned matching symbol. Per search: match_score.
bioseq.aligner.scoring.mismatch_penalty	int	4 (default) 1…127	Sets the score penalty for an aligned mismatch. Per search: mismatch_pen.
bioseq.aligner.scoring.short_gap_open_penalty	int	4 (default) 0…127	Sets the first gap-opening penalty. KSW2 and BlockAligner use this pair. BlockAligner requires positive opening and extension penalties whose sum is at most 128. Per search: gap_open1.
bioseq.aligner.scoring.short_gap_extension_penalty	int	2 (default) 0…127	Sets the first gap-extension penalty. BlockAligner requires it to be positive and its sum with the first opening penalty to be at most 128. Per search: gap_ext1.
bioseq.aligner.scoring.long_gap_open_penalty	int	24 (default) 0…127	Sets KSW2's second gap-opening penalty. Native BlockAligner uses only the first gap-penalty pair. Per search: gap_open2.
bioseq.aligner.scoring.long_gap_extension_penalty	int	1 (default) 0…127	Sets KSW2's second gap-extension penalty. Native BlockAligner does not use this second pair. Per search: gap_ext2.
bioseq.aligner.scoring.ambiguous_base_penalty	int	1 (default) 0…127	Sets the alignment penalty for ambiguous symbols on paths that support them. Per search: sc_ambi.
bioseq.aligner.scoring.z_drop_threshold	int	400 (default) −1…1048576	Sets KSW2's Z-drop threshold or BlockAligner's X-drop threshold for end extensions. BlockAligner extensions require a nonnegative value. Per search: zdrop.
bioseq.aligner.scoring.end_bonus	int	-1 (default) −1…1048576	Sets KSW2's sequence-end bonus. Negative one disables the bonus. Native BlockAligner does not use it. Per search: end_bonus.
bioseq.aligner.block_alignment.minimum_block_size	int	32 (default) 8…4096	Sets BlockAligner's minimum adaptive block size. (block sizes must be powers of two and the minimum cannot exceed the maximum) Per search: block_min_size.
bioseq.aligner.block_alignment.maximum_block_size	int	256 (default) 8…16384	Sets BlockAligner's maximum adaptive block size. (block sizes must be powers of two and the minimum cannot exceed the maximum) Per search: block_max_size.
bioseq.aligner.base_alignment.algorithm	enum	block_aligner code 1 (default) ksw2 code 0	Selects the base alignment algorithm Adaptive block aligner Per search: base_alignment_algorithm. Use KSW2 aligner

2.9. Vulkan execution

Table 2.9: Vulkan execution settings.

GUC name	Type	GUC values	Description
`bioseq.aligner.vulkan.enabled`	bool	`on` (default)	Enables GPU execution Allow supported GPU work GPU algorithms currently are not optimised and severly increase the query time. Enabling execution doesn't mean the GPU will be used (for example software-emulated GPUs are disabled by default). The correct GPU algorithm also need to be selected by setting respective GUCs for aligner/chaining. Per search: `vulkan_enabled`.
		`off`	Setting aligner/chaining algorithms (or other) to Vulkan will immediately fail.
`bioseq.aligner.vulkan.seed_lookup.enabled`	bool	`off` (default)	Enabled GPU-acclelerated minimizers (reading) Disable GPU for reading minimizer indexes Per search: `vulkan_seed_hits_enabled`.
		`on`	Enable GPU minimizer lookup (but other GPU settings apply)
`bioseq.aligner.vulkan.index_cache.enabled`	bool	`on` (default)	Cache minimizer index data to be reused across GPU calls (applies only to single pipeline execution and transfers between GPU/CPU memory) Save buffer with minimizers Per search: `vulkan_index_cache_enabled`.
		`off`	GPU doesn't reuse minimizers during computation
`bioseq.aligner.vulkan.fused_frontend.enabled`	bool	`off` (default)	Enable experimental GPU fused kernel (this doesn't work correctly) Leave this frontend unselected.
		`on`	Enable special optimisation for GPU kernels
`bioseq.aligner.vulkan.index_build.enabled`	bool	`off` (default)	Enabled GPU-acclelerated minimizers (writing) Disable GPU for wrikting minimizer indexes
		`on`	Enable GPU minimizer index building (but other GPU settings apply)
`bioseq.aligner.vulkan.alignment_extension.enabled`	bool	`off` (default)	Enabled CIGAR computation on GPU Disbable CIGAR computation on GPU
		`on`	Allow computing CIGAR on GPU (but other GPU settings apply)
`bioseq.aligner.vulkan.ordered_sketch_tile_bp`	int	`2048` (default) 1…16777216 symbols	Currently unused

Continued on the next page

GUC name	Type	GUC values	Description
`bioseq.aligner.vulkan.radix_sort_digit_bits`	int	`8` (default) 1…16 bits	Currently unused

2.10. Diagnostics

Table 2.10: Diagnostics settings.

GUC name	Type	GUC values	Description
`bioseq.diagnostics.logging.enabled` superuser	bool	`off` (default)	Enable or dissable internal logging. BioseqDB internally supports logging to file, but current builds are all configured to use only postgres logging mechanisms. Disable internal logger.
		`on`	Enable internal logger.
`bioseq.planner.diagnostics.log_level`	enum	`debug3` (default)	Chooses the PostgreSQL severity for planner diagnostic messages (logging levels are Postgres specific). Emit them at DEBUG3 severity. PostgreSQL's message thresholds determine where the messages appear.
		`debug5`	Emit them at DEBUG5 severity.
		`debug4`	Emit them at DEBUG4 severity.
		`debug2`	Emit them at DEBUG2 severity.
		`debug1`	Emit them at DEBUG1 severity.
		`log`	Emit them at LOG severity.
		`info`	Emit them at INFO severity.
		`notice`	Emit them at NOTICE severity.
		`warning`	Emit them at WARNING severity.
`bioseq.aligner.profiling.enabled`	bool	`off` (default)	Enables internal profiling of the pipeline execution profiling measures statistics like number of minimizers/chains/timings per pipeline stage etc.
		`on`	Profiling can increase execution time

2.11. Optional query-result cache

Table 2.11: Optional query-result cache settings.

GUC name	Type	GUC values	Description
`bioseq_cache.query_cache.mode`	enum	`disabled` (default)	Enables experimental query result cache (provided by BioseqDB bioseq_cache subextension) Do not use the query result cache (disabled due to my low confidence if it works correctly) Any mode other than 'disabled' required Postgres shared libraries reload.
		`exact_query`	Reuse completed identical requests and coordinate concurrent identical computations.
		`reusable_fragments`	Permit completed-fragment reuse for providers that support it. The Bioseq alignment adapter currently supports exact reuse only (this mode is not implemented)
`bioseq_cache.query_cache.completed_result_ttl_ms`	int	`3600000` (default) 0…2147483647 ms	Maximum age of the records in the bioseq_cache tables that can be reused

2.12. Controls present only in test builds

Table 2.12: Settings available only in test builds.

GUC name	Type	GUC values	Description
`bioseq.testing.chunked_scanner.enabled` hidden, superuser	bool	`off` (default)	Forces BioseqDB to load entire sequence into memory and create 'fake' lazy-loading interface that accesses it. Used by me to test C++ interface for lazy-loading. Use the normal scanner.
		`on`	Force load entire sequence into memory and create lazy-loader interface for that RAM sequence.
`bioseq.testing.in_memory_scanner.enabled` hidden, superuser	bool	`off` (default)	Forces BioseqDB to load entire sequence into memory and passes it directly to the pipeline. (Note that this is the same for user as bioseq.testing.chunked_scanner.enabled with the only difference being internally used C++ API). Used by me to test C++ interface for lazy-loading. Use the normal scanner.
		`on`	Force load entire sequence into memory and pass it directly to the pipeline.

Continued on the next page

GUC name	Type	GUC values	Description
`bioseq.testing.persisted_index.failure_mode` hidden, superuser	enum	`none` (default) `unsupported_vector_shape`	Cause 'fake' index reader failure to be reported during queries. Do nothing. Make the index reader report an unsupported vector shape (due to wrong w/k etc. parameters used during index build).

2.13. Per-search options

Additionally instead of operating on global settings, user can also provide single object as third parameter to the `mmsearch` function call.

Parameter encoding

Currently the enum options are encoded by integers - they correspond to the global GUCs, but you can use only numbers. This is just unimplemented feature - the types of the options should be changed to strings and encoded to internal enum int values. It was requiring less effort to expose internal representation, so it was left as is, see listing below 2.2.

Encoding of parameters as integer values:

Listing 2.2: Passing options by 3rd argument of mmsearch

```
SELECT (public.aligner_options(
  minimizer_algorithm => 0,
  index_read_requirement => 1)).*;
```

Every options is available below, for meaning and usage check the GUC tables:

Table 2.13: All nullable alignment options (for `aligner_options` call) in the order as specified in the SQL API

Options field	SQL type	Inherited GUC or independent policy
`kmer_size`	int	`bioseq.aligner.minimizer.kmer_size`
`window_size`	int	`bioseq.aligner.minimizer.window_size`
`max_gap`	int	`bioseq.aligner.chaining.maximum_gap`
`max_gap_ref`	int	`bioseq.aligner.chaining.maximum_reference_gap`
`bandwidth`	int	`bioseq.aligner.chaining.bandwidth`
`bandwidth_long`	int	`bioseq.aligner.chaining.long_join_bandwidth`
`max_chain_skip`	int	`bioseq.aligner.chaining.maximum_skipped_anchors`
`max_chain_iter`	int	`bioseq.aligner.chaining.maximum_predecessors`
`min_seed_count`	int	`bioseq.aligner.chaining.minimum_seed_count`
`min_chain_score`	int	`bioseq.aligner.chaining.minimum_score`
`chain_gap_scale`	double	`bioseq.aligner.chaining.gap_penalty_scale`
`chain_skip_scale`	double	`bioseq.aligner.chaining.skip_penalty_scale`

Continued on the next page

Options field	SQL type	Inherited GUC or independent policy
mask_level	double	bioseq.aligner.secondary.overlap_threshold
mask_len	int	bioseq.aligner.secondary.uncovered_length_limit
pri_ratio	double	bioseq.aligner.secondary.score_ratio_threshold
best_n	int	bioseq.aligner.secondary.maximum_count
match_score	int	bioseq.aligner.scoring.match_score
mismatch_pen	int	bioseq.aligner.scoring.mismatch_penalty
gap_open1	int	bioseq.aligner.scoring.short_gap_open_penalty
gap_ext1	int	bioseq.aligner.scoring.short_gap_extension_penalty
gap_open2	int	bioseq.aligner.scoring.long_gap_open_penalty
gap_ext2	int	bioseq.aligner.scoring.long_gap_extension_penalty
sc_ambi	int	bioseq.aligner.scoring.ambiguous_base_penalty
zdrop	int	bioseq.aligner.scoring.z_drop_threshold
end_bonus	int	bioseq.aligner.scoring.end_bonus
max_occ	int	bioseq.aligner.seeding.maximum_occurrence
mid_occ	int	bioseq.aligner.seeding.intermediate_occurrence_threshold
block_min_size	int	bioseq.aligner.block_alignment.minimum_block_size
block_max_size	int	bioseq.aligner.block_alignment.maximum_block_size
chaining_algorithm	int	bioseq.aligner.chaining.algorithm Integer codes should be checked in the GUC table.
base_alignment_algorithm	int	bioseq.aligner.base_alignment.algorithm Integer codes should be checked in the GUC table.
minimizer_algorithm	int	bioseq.aligner.minimizer.algorithm Integer codes should be checked in the GUC table.
vulkan_enabled	bool	bioseq.aligner.vulkan.enabled
vulkan_seed_hits_enabled	bool	bioseq.aligner.vulkan.seed_lookup.enabled
vulkan_index_cache_enabled	bool	bioseq.aligner.vulkan.index_cache.enabled
index_read_requirement	int	No GUC. See Table 2.14.

2.13.1. The index-read requirement

This is special field that has no GUC equivalent, that is only available via `aligner_options`. This is special flag very useful during testing and controls how index reader behaves.

Table 2.14: Special option controling index reader

Options field	Type	Value	Description
`index_read_requirement`	int	0: **`allow`** (default)	If no compatible index is available, seeds can be recomputed from scratch. This is the most useful in practical sense as it allows the largest flexibility for user.
		1: `require_absent`	Never read index even if it exists, always recompute seeds from scratch. Additionally setting this option will produce errors if deffered index update is ON. Practically used in testing scenarios to see what happens if index does not exist.
		2: `require_compatible_persisted`	Hard requirement that index must allways exist. If the index does not exist or is incompatible (different k/w parameters etc.), the query will produce error before execution.